

Course Notes

10.34: Numerical Methods in Chemical Engineering

Matthew Cox, Sungho Shin, Connor W. Coley



Department of Chemical Engineering
Massachusetts Institute of Technology

Based on lecture material developed at MIT over the years by James Swan, William H. Green, Joey Gu, Richard Braatz, Kenneth Beers, and others

Contents

1	Introduction to Numerical Methods	13
1.1	Numerical Error	13
1.2	Computational Complexity	17
1.3	Exercises	19
2	Linear Algebra	23
2.1	Scalars, Vectors, and Matrices	23
2.1.1	Scalars	23
2.1.2	Vectors	24
2.1.2.1	Vector Operations	25
2.1.3	Matrices	26
2.1.3.1	Matrix Operations	27
2.1.3.2	Special Matrices	30
2.1.3.3	Computational Complexity of Matrix and Vector Operations	31
2.2	Vector and Matrix Norms	34
2.2.1	Norms of Vectors	34
2.2.2	Norms of Matrices	35
2.2.2.1	Induced Matrix Norms	35
2.2.2.2	Common Matrix Norms	35
2.2.2.3	Relationships Between Matrix Norms	36
2.3	Introduction to Systems of Linear Equations	36
2.4	Invertibility and Determinants	38
2.4.1	Inverse of a Matrix	38
2.4.2	Determinant	39
2.5	Existence and Uniqueness of Solutions	40
2.5.1	Vector Spaces	40
2.5.2	Orthogonality	41
2.5.3	The four fundamental subspaces	41
2.5.4	Consistency via the left null space (Fredholm alternative)	42
2.5.5	Rank and nullity	43
2.5.6	Square case and invertibility	43
2.6	Numerical Solution of Linear Systems: Direct Methods	43
2.6.1	Back Substitution	44
2.6.2	Gaussian Elimination	44
2.6.3	Condition Number	46
2.7	Eigenvalues and Eigenvectors	48
2.7.1	Finding Eigenvalues and Eigenvectors	49
2.7.2	Multiplicity and Diagonalizability	50
2.7.3	Eigendecomposition	50
2.7.4	Singular Values	51
2.8	Numerical Solution of Linear Systems: Iterative Methods	51

2.8.1	Jacobi Iteration	52
2.8.2	Convergence of Jacobi Iteration	53
2.8.3	Gauss-Seidel Iteration	54
2.8.4	Convergence of Gauss-Seidel Iteration	55
2.8.5	Condition Number and Convergence Rate	56
2.9	Exercises	58
3	Systems of Nonlinear Equations	65
3.1	Introduction to Nonlinear Systems	65
3.2	The Jacobian	66
3.3	Local Uniqueness	67
3.4	Connection to Linear Systems	69
3.5	Linearization via Taylor Expansions	70
3.5.1	Local Uniqueness via Taylor Expansions	70
3.6	Iterative Methods for Solving Nonlinear Equations	71
3.6.1	Stopping Criteria	72
3.6.1.1	Failure Modes of Stopping Criteria	72
3.6.2	Convergence Analysis	73
3.6.3	The Newton-Raphson Method	74
3.6.4	Convergence of Newton's Method	77
3.6.4.1	Local Convergence	79
3.6.4.2	Failure Modes of Newton's Method	79
3.6.4.3	Basins of Attraction	80
3.6.5	Damped Newton's Method	81
3.6.6	Calculating the Jacobian	83
3.6.6.1	1D Finite Differences	84
3.6.6.2	ND Finite Differences	85
3.6.6.3	Automatic Differentiation	86
3.6.7	Quasi-Newton Methods	87
3.6.7.1	Broyden's Method	88
3.7	Continuation and Homotopy Methods	88
3.7.1	Continuation	88
3.7.2	Homotopy Methods	89
3.8	Bifurcation Analysis	91
3.9	Summary	94
3.10	Exercises	94
4	Optimization	95
4.1	Preliminaries: Gradients, Hessians, and Taylor Expansions	95
4.2	Foundations of Optimization	96
4.2.1	The Optimization Problem	96
4.2.2	Classification of Optimization Problems	97
4.2.3	Local vs. Global Solutions	98
4.2.4	Convexity	99
4.3	Optimality Conditions for Unconstrained Problems	100
4.3.1	Proofs for Optimality Conditions	101
4.3.1.1	First-Order Necessity*	102
4.3.1.2	Second-Order Necessity*	102
4.3.1.3	Second-Order Sufficiency*	102
4.3.1.4	Intuition	102
4.4	Algorithms for Unconstrained Optimization	103
4.4.1	Steepest Descent	103
4.4.1.1	Backtracking Line Search	103
4.4.1.2	Convergence	104

4.4.1.3	Stochastic Gradient Descent	105
4.4.2	Newton and Modified-Newton Methods	105
4.4.2.1	Hessian Modification	106
4.4.2.2	Global and Local Convergence	106
4.4.2.3	Comparison with Steepest Descent	107
4.4.3	Quasi-Newton Methods	107
4.4.3.1	Broyden-Fletcher-Goldfarb-Shanno (BFGS) Update	108
4.4.3.2	Convergence	108
4.4.4	Step Selection Paradigms*	109
4.5	Constrained Optimization	111
4.5.1	Equality-Constrained Optimization	111
4.5.1.1	Lagrange Multipliers	112
4.5.2	Penalty Methods: A Questionable Alternative	113
4.5.3	Inequality-Constrained Problems	114
4.5.4	Interior-Point Methods	115
4.6	Mixed-Integer Optimization*	117
4.6.1	Branch and Bound*	118
4.7	Gradient-Free Optimization*	120
4.7.1	The Nelder-Mead Method*	120
4.8	Optimization: A Summary	121
4.9	Exercises	122
5	Probability	131
5.1	Events, Sample Spaces, and Axioms	131
5.2	Conditional Probability and Independence	132
5.2.1	Law of Total Probability	134
5.2.2	Inclusion-Exclusion and Union Bounds*	134
5.2.3	Simpson's Paradox	136
5.2.4	Bayes' Theorem	137
5.3	Random Variables	138
5.4	Describing Random Variables: Probability Distributions	139
5.4.1	The Cumulative Distribution Function	139
5.4.2	The Probability Mass Function for Discrete RVs	140
5.4.3	The Probability Density Function for Continuous RVs	140
5.5	Sampling and Expectations	141
5.6	Some Notable Probability Distributions	143
5.7	Recognizing Distributions by Shape	146
5.8	Empirical Distributions	146
5.9	The Central Limit Theorem	147
5.10	Estimators and Bias	149
5.11	Multivariate Random Variables	151
5.11.1	Random Vectors	151
5.12	Covariance and Correlation	153
5.12.1	Conditional Probability	155
5.12.2	Marginalization	157
5.13	A Unified View of Probability Distributions	158
5.13.1	Convolution	159
5.14	Random Number Generation	160
5.14.1	Pseudorandom Numbers and Seeding	160
5.14.2	Sampling from Distributions	161
5.14.3	Distances Between Distributions	162
5.14.3.0.1	Kolmogorov-Smirnov Statistic	164
5.15	Exercises	164

6	Parameter Estimation	167
6.1	Linear Model Fitting	167
6.1.1	Constructing the Design Matrix	168
6.1.2	Solving for the Best Parameters	169
6.1.3	Regularization	171
6.2	Frequentist vs. Bayesian Schools of Thought	173
6.3	Maximum Likelihood Estimation	174
6.4	Maximum a Posteriori Estimation	175
6.5	The Chi-Square Statistic	177
6.5.1	The Chi-Square Distribution and Hypothesis Testing	178
6.6	Regions of Indifference	180
6.7	The Bayesian View of Uncertainty	186
6.8	Conjugate Priors*	187
6.9	Prediction Uncertainty	189
6.10	Frequentist vs. Bayesian: A Summary	191
6.11	Optimal Experimental Design	192
6.11.1	Space-Filling Designs	193
6.11.2	Model-Based Experimental Design for Linear Models	195
6.11.3	Optimality Criteria for Experimental Design	197
6.12	Bayesian Optimal Experimental Design*	199
6.12.1	Model Comparison and the Bayes Factor*	200
6.13	Sequential Experimental Design*	201
6.14	Exercises	201
7	Monte Carlo Methods	203
7.1	The Curse of Dimensionality	203
7.2	Monte Carlo Integration	205
7.3	Importance Sampling	208
7.4	Rejection Sampling*	212
7.5	Markov Chain Monte Carlo	214
7.5.1	Markov Chains	214
7.5.2	The Metropolis-Hastings Algorithm	216
7.5.3	Convergence and Ergodicity	218
7.5.4	Practical MCMC	219
7.5.5	Applications	221
7.6	Kinetic Monte Carlo	223
7.7	Exercises	225
8	Numerical Solution of Ordinary Differential Equations	229
8.1	Ordinary Differential Equations: Initial Value Problems (ODE-IVPs)	229
8.1.1	Rewriting Higher-Order ODEs	230
8.1.2	Local Truncation Error	230
8.2	Explicit Methods for Solving ODE-IVPs	231
8.2.1	The Explicit (Forward) Euler Method	231
8.2.2	The Midpoint Method	235
8.2.3	Comparing Explicit Methods: Euler vs. Midpoint	237
8.2.4	Runge-Kutta Methods	237
8.3	Implicit Methods for Solving ODE-IVPs	239
8.3.1	The Implicit (Backward) Euler Method	239
8.3.2	Stability of Implicit Euler	241
8.3.3	The Crank-Nicolson Method	242
8.3.4	Comparison of Implicit Behavior	244
8.3.5	Revisiting the Prototype Problem	245
8.4	Stiff Differential Equations	246

8.5	Adaptive Time Stepping*	248
8.6	Existence and Uniqueness of Solutions*	249
8.6.1	Uniqueness*	249
8.7	Exercises	250
9	Boundary Value Problems	253
9.1	Introduction to Boundary Value Problems	254
9.1.1	Classification of Boundary Conditions	254
9.1.2	Linear BVPs	254
9.2	Shooting Methods	256
9.2.1	The Shooting Method	256
9.2.2	Sensitivity Analysis	256
9.2.3	The Multiple Shooting Method	257
9.3	Discretization Methods	259
9.3.1	The Finite Difference Method	259
9.3.2	Basis Function Expansions	262
9.3.3	The Finite Element Method	264
9.3.4	The Finite Volume Method	267
9.4	Summary of BVP Methods	269
9.5	Exercises	270
10	Numerical Solution of Partial Differential Equations	273
10.1	Classification of PDEs	274
10.1.1	Elliptic PDEs	275
10.1.2	Parabolic PDEs	275
10.1.3	Hyperbolic PDEs	276
10.2	Spatial Discretization (Elliptic Examples)	278
10.2.1	The Finite Difference Method	278
10.2.1.1	Discretization and the 5-Point Stencil	279
10.2.1.2	Handling Neumann Boundary Conditions	280
10.2.1.3	The System of Linear Equations	280
10.2.1.4	Solving the Sparse System	281
10.2.2	The Finite Volume Method	282
10.2.2.1	Deriving the Conservation Law	283
10.2.2.2	Boundary Conditions in FVM	283
10.3	Method of Lines for Time-Dependent PDEs (Parabolic/Hyperbolic)	285
10.3.1	The Method of Lines	285
10.3.2	Implementation and Boundary Handling	286
10.4	Finite Differences for Linear Advection: Upwinding	286
10.5	Stability of Time-Stepping Schemes: CFL and von Neumann Analysis	287
10.5.1	The CFL Condition	287
10.5.2	Von Neumann Stability Analysis	288
10.5.2.1	Hyperbolic Case: Advection	288
10.5.2.2	Parabolic Case: Diffusion	290
10.5.2.3	Implicit Methods for Parabolic Equations	291
10.6	Method of Characteristics for First-Order Hyperbolic PDEs	292
10.6.1	Initial and Boundary Value Problems	292
10.6.2	Second-Order Hyperbolic Equations	294
10.7	Summary of PDE Methods	294
10.8	Exercises	295
11	Differential-Algebraic Equations	297
11.1	Standard Forms of DAEs	298
11.1.0.1	Connection to ODEs	299

11.1.0.2 Example DAE Systems	299
11.2 The Differential Index (or Differentiation Index)	301
11.2.1 Index 1 vs. Index 2: The Tank Filling Problem	301
11.2.1.1 Case A: Specified Outlet Flow (Index 1)	302
11.2.1.2 Case B: Specified Internal Volume (Index 2)	302
11.2.2 Another Example: Calculation of Higher Indices	303
11.3 Distinguishing Index 1 from Higher Index Systems	304
11.4 Reducing the Index of a DAE	305
11.4.1 Method of Auxiliary Variables	305
11.5 Numerical Simulation of DAEs	307
11.5.1 Connection to Stiffness in ODEs	307
11.5.2 Failure of Forward Euler	307
11.5.3 Backward Euler and Simulation Errors	308
11.6 Initialization of Differential-Algebraic Equations	309
11.6.1 Consistent Initialization and Implicit Constraints	309
11.7 Simulation of Mixed Continuous-Discrete Systems	311
11.8 Exercises	312
A Solutions to Exercises	315

Preface

This course reader was originally developed based on the Fall 2024 offering of 10.34 (Numerical Methods Applied to Chemical Engineering) at MIT taught by Profs. Sungho Shin and Connor Coley; it was designed as a transcription of the course slides/lectures into document form. In fact, this document began as Matthew Cox’s personal notes for the course.

This book is meant to be self-contained, but at times it may be helpful to refer to external resources for more in-depth explanations.

Warning: This document is still a work in progress, and is not yet completely polished. It is recommended to always refer to the course slides/lectures for the most up-to-date information. If you spot any errors, feel free to email Matthew at mcox340@mit.edu (or directly submit an issue/PR on the corresponding GitHub repository if you know how to use GitHub).

Exercises are provided at the end of each chapter, with solutions provided at the end of the reader. Exercises that are marked with an asterisk, “(*)”, may take more effort or require derivations to answer.

Notation

The following notational conventions are used throughout these notes.

Notation	Meaning
a, x, λ	scalars (italic letters)
$\mathbf{v}, \mathbf{x}$	vectors (bold lowercase letters); v_i is the i -th entry of $\mathbf{v}$
$\mathbf{A}, \mathbf{B}$	matrices (bold uppercase letters); a_{ij} is the entry in row i , column j
x, y	random variables (sans-serif letters); realizations are italic (x, y)
$\mathbf{x}$	random vectors (bold sans-serif letters)
$\mathbb{R}, \mathbb{C}$	the sets of real and complex numbers
$\hat{i}$	the imaginary unit ($\hat{i} = \sqrt{-1}$); the plain letter i is reserved for indices
$\mathbb{R}^n, \mathbb{R}^{m \times n}$	the sets of n -dimensional real vectors and $m \times n$ real matrices
$\mathcal{N}, \mathcal{U}$	probability distributions (calligraphic letters)
$\mathbb{E}[\cdot], \Pr(\cdot)$	expectation and probability
$\mathbf{A}^\top$	transpose of $\mathbf{A}$
$\bar{z}$	complex conjugate of z
$O(\cdot)$	asymptotic computational complexity (big-O)
<code>typewriter</code>	MATLAB code

When the same letter appears in several forms, the forms are related: for example, x may denote a scalar, $\mathbf{x}$ a vector with entries x_i , and x a random variable whose realization is x .

Chapter 1

Introduction to Numerical Methods

When we solve a problem numerically, we care about two things: how *accurate* the computed answer is, and how much it *costs* to compute it. These two concerns are characterized by *numerical error* and *computational complexity*, the two major topics of this chapter. Whenever we discuss an algorithm in this course, we will talk about these two concepts: we introduce them here, and they will reappear in every subsequent topic—linear algebra, nonlinear equations, optimization, statistics, and differential equations.

1.1 Numerical Error

For most real-world problems, an analytical solution does not exist, so we must resort to numerical methods, which solve problems approximately rather than analytically/symbolically. As we will see later, numerical methods always encounter **numerical error**: the computed result deviates from the exact one. Understanding the size of this error is essential for judging how much a computed result can be trusted.

Absolute and Relative Error There are two versions of numerical error: the absolute error and the relative error, defined as

$$\begin{aligned}\epsilon_{\text{abs}} &= |x_{\text{exact}} - x_{\text{approx}}| \quad (\text{absolute error}) \\ \epsilon_{\text{rel}} &= \frac{|x_{\text{exact}} - x_{\text{approx}}|}{|x_{\text{exact}}|} \quad (\text{relative error}),\end{aligned}$$

where x_{exact} is the exact value and x_{approx} is its computed approximation. Absolute error is the difference between the exact and approximate values, while relative error is the absolute error divided by the exact value's magnitude. Relative error is often more useful as it is scale-independent, whereas absolute error can be more relevant when the exact value is near zero. In practice, both are considered to gain a comprehensive understanding of numerical error, particularly in convergence criteria for iterative algorithms.

Finite-Precision Arithmetic Why do numerical errors occur in the first place? At the basic level, all real numbers are represented as *floating-point numbers*. Modern computers operate using binary memory, which can represent ones and zeros. Due to the finite nature of computer memory, we can only represent real numbers *approximately*. This finite, binary representation of real numbers is referred to as floating-point representation. In the floating point representation, a number is represented by three components: a sign bit s , an exponent e , and a significand m :

$$\text{value} = (-1)^s \times m \times 2^e.$$

Since these values must be stored in computer memory, we represent the sign, exponent, and significand in binary format. For example, the number π can be represented as:

$$\pi = 3.141592\dots = 1.100100100001111101101010\dots \times 2^1.$$

Note that the same number could also be written with a different exponent (e.g., $0.11001001 \dots \times 2^2$). We usually *normalize* the representation by choosing the exponent such that the leading significand bit is 1 (equivalently, $1 \leq m < 2$), which makes the representation unique. If you compare this to scientific notation:

$$\text{value} = \pm d \times 10^n \quad (\text{scientific notation})$$

you will notice many similarities; however, in floating-point format, everything is expressed in ones and zeros.

In practice, only a finite amount of memory is available for storing the sign, the exponent, and the significand. Since the binary expansions of most real numbers are non-terminating, as in the π example above, the significand must be truncated to fit in the available memory. There are several standard memory configurations for storing floating-point numbers, determined by the number of bits allocated for the sign, exponent, and significand. The most common formats are single-precision (float32, also called FP32) and double-precision (float64, also called FP64), which represent numbers with 32 and 64 bits, respectively (Figure 1.1.1). Other formats exist as well: half-precision (float16) uses 16 bits, quadruple-precision (float128) uses 128 bits, and specialized variants such as bfloat16 trade significand bits for a wider exponent range; the shorter formats are increasingly common in ML-focused hardware. Integers, on the other hand, are typically represented separately as their own data type (commonly 32-bit or, increasingly, 64-bit), using only a sign bit and the binary digits.

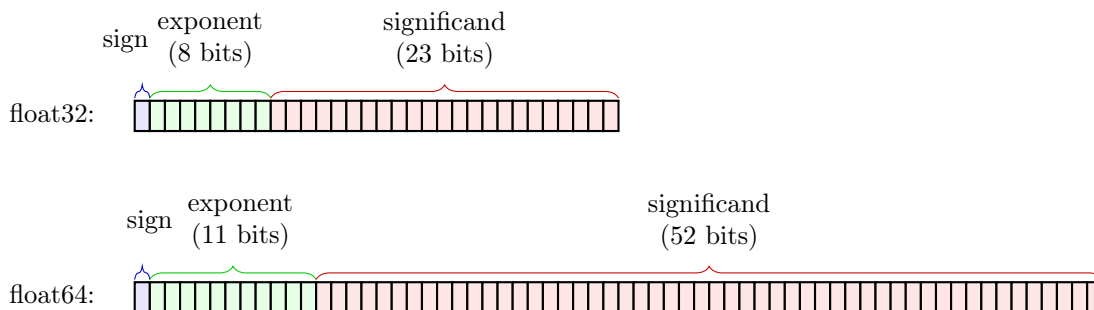


Figure 1.1.1: Floating-point representation of numbers.

The number of bits determines the precision of the number. For example, float32 has 23 explicitly stored bits of precision, which is roughly equivalent to 7 decimal digits. Meanwhile, float64 has 52 explicitly stored bits of precision, which is roughly equivalent to 16 decimal digits.

For example, π is stored as

$$\pi \approx 1.10010010000111111011011 \times 2^1 \quad (\text{float32})$$

$$\pi \approx 1.1001001000011111101101010100010001000010110100011000 \times 2^1 \quad (\text{float64})$$

where the significand carries 23 (float32) or 52 (float64) explicitly stored bits, introducing **representation error**.

Precision and Hardware There exists a practical trade-off between accuracy and speed/memory usage. Higher precision demands increased memory for storage and more time for arithmetic operations. The optimal point of this trade-off is context and hardware-dependent. For many engineering and scientific applications, float64 is the preferred choice. In these applications, we often seek the solution of a physical governing equation, and the relative error introduced by single precision is often not acceptable. Classically, these computation tasks were performed on CPUs. On most CPUs, double precision (float64) is the default and well optimized (e.g., MATLAB uses float64 by default). However, in machine learning workflows, float32 is often the default, as it often provides sufficient precision and more benefits come from reduced memory usage and increased speed; AI models often use even shorter quantization, such as float16 or 8-bit integer formats. Many GPUs deliver highest throughput with float32 or even float16 (sometimes used in mix).

Round-off and Machine Epsilon One specific type of numerical error arising from finite precision is round-off error. Every time an arithmetic operation is performed, the result must be rounded to the nearest representable number. Machine epsilon (ϵ_{mach}) is an upper bound on the relative error due to rounding:

$$\epsilon_{\text{mach}} = \begin{cases} 1.192 \cdot 10^{-7} & \text{for float32} \\ 2.2204 \cdot 10^{-16} & \text{for float64} \end{cases}$$

Simply put, ϵ_{mach} is the difference between 1 and the next largest floating point number for a given representation. In practice, if your algorithm does not converge beyond a relative error of about 10^{-16} in double precision, this does not indicate an error or a bug in the algorithm; it is the highest accuracy that can be achieved with the double-precision representation of numbers. Also note that the bound ϵ_{mach} applies to each elementary arithmetic operation (addition, subtraction, multiplication, and so on). When a series of operations is performed, the errors are transferred through the computation and can be magnified, so this level of accuracy cannot always be achieved by a complex algorithm. For example, as we will see later, when solving a linear system of equations, the relative error can be magnified by a factor of the condition number of the matrix.

Example: Predict the output of the following three MATLAB commands: `eps(1)`, `1 + eps(1) == 1`, and `1 + eps(1)/2 == 1`.

Solution:

```
>> eps(1)
ans =
    2.2204e-16

>> 1 + eps(1) == 1
ans =
    logical
     0

>> 1 + eps(1)/2 == 1
ans =
    logical
     1
```

The first command, `eps(1)`, returns the machine epsilon for double precision, which is approximately 2.2204×10^{-16} . This value is the difference between 1 and the next representable floating-point number in double precision. This also represents the worst-case relative error for basic arithmetic operations in double precision.

Next, the second command, `1 + eps(1) == 1`, checks whether adding the machine epsilon to 1 results in a value that is still considered equal to 1. The output is logical 0 (false), indicating that $1 + \epsilon_{\text{mach}}$ is distinguishable from 1.

Finally, the third command, `1 + eps(1)/2 == 1`, checks whether adding half of the machine epsilon to 1 results in a value that is still considered equal to 1. The output is logical 1 (true), indicating that $1 + \epsilon_{\text{mach}}/2$ is not distinguishable from 1 in double precision. The difference, $\epsilon_{\text{mach}}/2$, is lower than the threshold value and is lost during the rounding process. This demonstrates that the machine epsilon defines the threshold for distinguishing between numbers in floating-point arithmetic.

Ordering Calculations Because floating-point numbers are only approximations, the way we arrange computations can create or avoid error. The most common manifestation of this is known as “catastrophic cancellation”: subtracting nearly equal quantities wipes out correct digits and magnifies rounding noise. For example, the algebraically equivalent expressions $1 - 3/(4/3 - 1)$ and $1 - (3/3 \cdot 4 - 3)$ can result in 2.22×10^{-16} versus 0 in double precision; this is because the first forces a small difference subtraction while the second

avoids it. Similar issues show up when computing $1 - \cos x$ for small x (use the identity $1 - \cos x = 2 \sin^2(x/2)$, which follows from the double-angle formula, to compute this instead), summing terms with very different magnitudes (sum smaller terms first or use compensated summation), or applying the quadratic formula when $b^2 \gg 4ac$ (use $2c/(-b \mp \sqrt{b^2 - 4ac})$ for the small root). The practical rule you should know for 10.34 is simple: choose algebraic forms that avoid subtracting nearly equal numbers, keep intermediate values on similar scales (rescale when helpful), and use numerically stable library routines (i.e., no need to reinvent the wheel if MATLAB has something you need). Otherwise, tiny rounding errors can accumulate over many operations into visible inaccuracies.

Overflow and Underflow Overflow/underflow describes the situation where numerical calculation exceeds the limits of floating-point formats. It occurs when we try to store numbers that are too large (overflow) or too small (underflow) for our chosen representation. The finite exponent limits the range of representable magnitudes. For float32, the smallest and largest positive normalized numbers are $2^{-126} \approx 1.18 \times 10^{-38}$ and $(2 - 2^{-23}) \times 2^{127} \approx 3.40 \times 10^{38}$; for float64, they are $2^{-1022} \approx 2.23 \times 10^{-308}$ and $(2 - 2^{-52}) \times 2^{1023} \approx 1.80 \times 10^{308}$. Typically, this issue can be avoided by using proper units for the variables. If you know that you may encounter a very large number, rescale it or change the unit so that the numerical value falls into the range representable with double-precision numbers.

Warning: Intermediate Steps Can Overflow! An expression can overflow during an intermediate step even if the final answer is perfectly representable.

For example, calculating the hypotenuse $\sqrt{a^2 + b^2}$ with $a = 10^{200}$ and $b = 10^{200}$ will fail because a^2 overflows. A robust function like MATLAB's `hypot(A, B)` avoids this by rescaling first.

Truncation Error and Convergence Even with infinite precision, numerical errors arise due to the approximate nature of many algorithms. Series and polynomial approximations are used widely in numerical computing: elementary functions such as sine and cosine are calculated using polynomial approximations, and constants such as π are approximated using series representations. Since only a finite number of terms can be evaluated, these approximations naturally introduce **truncation error**, which depends on the number of terms or iterations used. For example, the sine function is commonly computed from a truncated Taylor series. After the argument is reduced to a small range (e.g., $|x| \leq \pi/4$), the seventh-order polynomial

$$\sin x \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!}$$

is evaluated, typically using Horner's method. The first neglected term is of order $x^9/9!$, which for $|x| \leq \pi/4$ is at most about 3×10^{-7} , on the order of the single-precision machine epsilon ($\approx 1.2 \times 10^{-7}$). Truncating at the seventh order is therefore sufficient for single precision; carrying more terms would add computation without improving the achievable accuracy. There are often multiple ways to approximate the same quantity, but some converge faster than others, and we typically want to use the fastest-converging one. To illustrate this, let's compare two different series for approximating the same mathematical constant: π . The Leibniz series is a simple infinite series that approximates π as

$$\pi = 4 \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$$

This series converges very slowly. To achieve just two decimal places of accuracy, we need to sum over several hundred terms. The slow convergence means more iterations are required for high precision, increasing computational effort and the potential for cumulative numerical errors. In contrast, we can consider one of Ramanujan's series for π , which converges extremely rapidly:

$$\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} \sum_{n=0}^{\infty} \frac{(4n)!(1103 + 26390n)}{(n!)^4 396^{4n}}$$

This series is remarkable because each term adds approximately eight additional correct decimal places to the approximation of π . Therefore, summing just a few terms already yields an accurate approximation.

The convergence of these two series to the true value of π is shown in Figure 1.1.2. The Ramanujan series converges to within machine epsilon after just 3 terms!

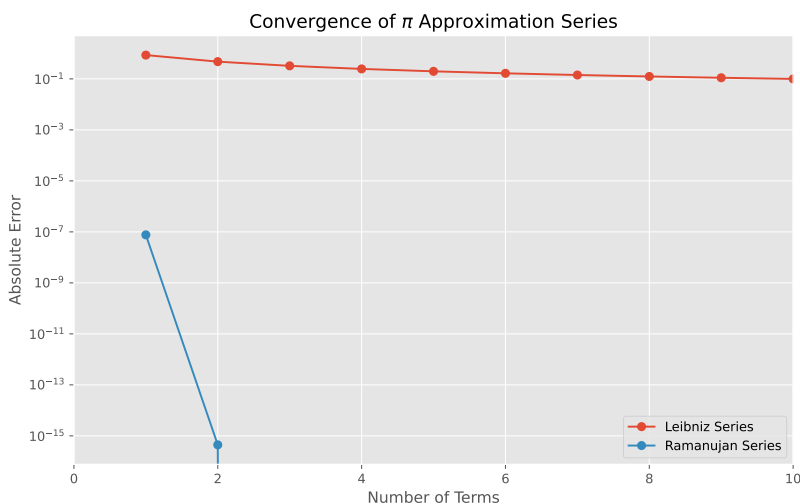


Figure 1.1.2: Convergence of two series approximating π . Machine epsilon is approximately $2.22 \cdot 10^{-16}$ here, so values within this range are essentially as converged as numerically possible.

1.2 Computational Complexity

As suggested earlier when discussing numerical error, we need to understand not just how to solve a problem, but also how efficiently it can be solved as the problem size increases. This efficiency is quantified using the concept of **computational complexity**, which comes in two kinds: time complexity and space complexity. **Time complexity** measures the number of elementary operations (e.g., $+$, $-$, $*$, $/$) an algorithm requires relative to the size of its input, typically denoted by N . **Space complexity** measures the amount of memory the algorithm requires. Within this course, we mostly discuss time complexity. In numerical computing, the elementary arithmetic operations on floating-point numbers are called **floating-point operations (FLOPs)**, and the computational cost of an algorithm is often reported as a FLOP count. Not only algorithms but also hardware capabilities are often specified this way: hardware specifications commonly report the number of floating-point operations the device can perform per second (FLOPS).

To express computational complexity, we generally use **big-O notation**. This mathematical notation describes an upper bound on the growth rate of a function so that we can focus on the most significant factors that affect an algorithm's performance as the input size becomes large. Mathematically, we say that a function $f(N)$ is $O(g(N))$ if there exist positive constants c and N_0 , both independent of N , such that

$$|f(N)| \leq c \cdot |g(N)| \quad \text{for all } N \geq N_0.$$

This definition means that for sufficiently large values of N , the function $f(N)$ does not grow faster than a constant multiple of $g(N)$. Again, we stress that big-O notation captures the dominant term of $f(N)$ as N approaches infinity, ignoring lower-order terms and constant factors that become insignificant at large scales.

Example: Determine the time complexity of the following MATLAB function, which finds the maximum of an array x of length N .

```
function m = array_max(x)
    N = length(x);
    m = x(1);
    for i = 2:N
```

```

        if x(i) > m
            m = x(i);
        end
    end
end

```

Solution: The function scans the array once, performing one comparison per element, for a total of $N - 1$ comparisons. The number of operations grows linearly with the input size, so the time complexity is $O(N)$.

Example: Two algorithms are designed to solve the same problem with input size N . Algorithm A has a time complexity of $f_A(N) = 5N^2 + 10000$. Algorithm B has a time complexity of $f_B(N) = 2N^3 + N$. Express both in big-O notation, and compare how their operation counts and runtimes grow with N .

Solution: In big-O notation,

$$f_A(N) = O(N^2)$$

$$f_B(N) = O(N^3)$$

To see the practical impact, let's compare the number of operations required as N increases:

Problem Size (N)	Operations in Algorithm A	Operations in Algorithm B
10	$1.05 \cdot 10^4$	$2.01 \cdot 10^3$
100	$6 \cdot 10^4$	$2.0001 \cdot 10^6$
1000	$5.01 \cdot 10^6$	$2.000001 \cdot 10^9$
10000	$5.0001 \cdot 10^8$	$2.00000001 \cdot 10^{12}$
100000	$5.000001 \cdot 10^{10}$	$2.0000000001 \cdot 10^{15}$

As the table shows, Algorithm B requires significantly more operations than Algorithm A as N grows (but notice this is NOT true at small N !), making it less practical for large problem sizes. This also follows directly from the big-O notation, which tells us that $f_B(N)$ grows faster than $f_A(N)$.

To be even more concrete, let's assume that each operation takes 10^{-8} seconds, which is reasonable for a modern CPU. Then, we can construct the table below:

Problem Size (N)	Algorithm A Runtime	Algorithm B Runtime
10	0.105 ms	0.0201 ms
100	0.6 ms	20.001 ms
1000	50.1 ms	20 s
10000	5.00 s	5.56 hours
100000	8.33 min	231 days

The above example shows how algorithms with lower computational complexity dramatically reduce computation time, especially for large N . The difference between $O(N^3)$ and $O(N^2)$ algorithms (and more generally, any two algorithms with different associated time complexities) can be the difference between a project being feasible or not.

Space complexity is also expressed using big-O notation, and although we will not discuss it further in this course, memory usage can be just as important as time complexity, especially when working with large datasets or on memory-constrained devices. Unless we explicitly mention space complexity, all discussions regarding complexity will pertain to time complexity.

Parallelism (Optional) The time complexity discussed above implicitly assumes that the operations are executed one after another, in series. Modern hardware, however, often has the capability to perform many operations in parallel. For example, consider computing $a\mathbf{x} + \mathbf{y}$ for a scalar a and vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^N$: this requires $O(N)$ operations, but each of the N components can be computed independently of the others, so

with adequate parallel resources (N processing units) the computation can be completed in $O(1)$ time. Such perfect parallelization is not always possible, even with a sufficient amount of parallel computing resources. If we want to take the maximum of an array, the comparisons cannot all be performed independently, so the operations must be executed partially in series; parallel computing resources can still reduce the time complexity (to $O(\log N)$ with a tree-structured reduction), but not to $O(1)$. Figure 1.2.1 illustrates the contrast between the two cases.

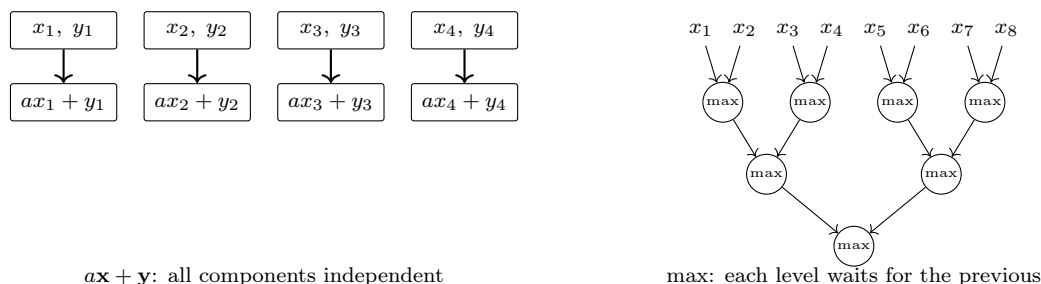


Figure 1.2.1: Parallel evaluation of $ax + y$ versus a tree-structured reduction for $\max$. Each component of $ax + y$ is computed independently, so with N processing units the computation completes in $O(1)$ time. In the reduction, each level depends on the results of the previous one, so even with sufficient parallel resources the time complexity is $O(\log_2 N)$.

1.3 Exercises

Exercise 1.1

True or False? Briefly explain your answer.

- Machine epsilon is the smallest positive number that can be represented in double precision.
- Every real number between the smallest and largest representable values can be represented exactly in double precision.
- If an iterative algorithm running in double precision stops improving at a relative error of about 10^{-16} , this indicates a bug in the implementation.
- An algorithm with $O(N^2)$ time complexity is always faster than an algorithm with $O(N^2 \log N)$ time complexity.
- Running the same function in float32 instead of float64 (without parallelism) speeds it up by a factor that grows with the input size N .
- In double precision, $1 + 1e-20 == 1$ evaluates to true; this means that $1e-20$ was rounded to zero when it was stored.

Exercise 1.2

Modern AI models store enormous numbers of parameters, and the choice of number representation directly determines their memory footprint. As a concrete example, consider the Llama-3-70B large language model, which has 70 billion parameters, each stored as a real number. Lowering the precision typically degrades the performance of the model, but the model often remains functional. One might therefore be interested in finding the optimal trade-off between the performance of the model and the memory usage and associated hardware cost.

- How much memory is required to store the parameters in float64, float32, and float16 format?
- A high-end GPU such as the NVIDIA A100 or H100 provides 80 GB of memory. For each format in (a), can the model parameters fit on a single such GPU?

Exercise 1.3

We implement a MATLAB function `my_sum` that computes the sum of the elements of an array:

```
function s = my_sum(x)
    s = 0;
    for i = 1:length(x)
        s = s + x(i);
    end
end
```

To test the correctness of the implementation, we sum ten copies of 0.1 and compare against the known reference value 1:

```
x = 0.1 * ones(10, 1);
s = my_sum(x);
s_ref = 1;                % known reference value

if s == s_ref
    disp('PASS');
else
    disp('FAIL');
end
```

Even though the implementation is correct, this test prints **FAIL**: floating-point arithmetic rounds after every operation, so the computed sum differs slightly from the exact value (recall that 0.1 is not exactly representable in binary).

- How would you modify the testing script so that it accepts a correct implementation?
- Now suppose we instead sum the array $\mathbf{x} = (1:100)' * 1e8 + 0.1$, whose exact sum is 505,000,000,010. Would your modified test from (a) still be reasonable? What would be a more reasonable criterion for this case?

Exercise 1.4

For each pair of functions, determine which one grows faster as $N \rightarrow \infty$.

- $\log N$ versus $N^{0.01}$
- $(\log N)^{100}$ versus N
- N^{100} versus 1.01^N
- $\log N$ versus $\log(N^2)$
- $\log_2 N$ versus $\log_{10} N$

Exercise 1.5

Express the time complexity of each expression using big-O notation.

- $f(N) = 3N^2 + 100N + 7$
- $f(N) = 0.01N^3 + 10^6N^2$
- $f(N) = (2N + 5)(N^2 + 3N)$
- $f(N) = 42$
- $f(N) = N \log N + 100N$

$$(f) f(N) = 2^N + N^{100}$$

$$(g) f(N) = \frac{N^3 + 2N}{N + 1}$$

$$(h) f(N) = \frac{N^2 + N}{N + 1}$$

Exercise 1.6

An algorithm is often built from several parts. Determine the overall time complexity in each case.

- (a) The algorithm first runs a preprocessing step that costs $O(N^2)$, and then a main computation that costs $O(N^3)$.
- (b) The algorithm performs N iterations, and each iteration costs $O(N^2)$.
- (c) In general, under what circumstances do the complexities of two parts *add*, and under what circumstances do they *multiply*?

Exercise 1.7

A matrix is **tridiagonal** if all of its nonzero entries lie on the main diagonal and the two diagonals directly above and below it. The Thomas algorithm allows for efficient solving of linear systems involving tridiagonal matrices,

$$\begin{bmatrix} a_{11} & a_{12} & & & 0 \\ a_{21} & a_{22} & a_{23} & & \\ & a_{32} & a_{33} & \ddots & \\ & & \ddots & \ddots & a_{n-1,n} \\ 0 & & & a_{n,n-1} & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix}.$$

The forward sweep computes updated coefficients (denoted with primes):

$$a'_{i,i+1} = \begin{cases} \frac{a_{i,i+1}}{a_{ii}}, & i = 1, \\ \frac{a_{i,i+1}}{a_{ii} - a_{i,i-1} a'_{i-1,i}}, & i = 2, 3, \dots, n-1, \end{cases} \quad b'_i = \begin{cases} \frac{b_i}{a_{ii}}, & i = 1, \\ \frac{b_i - a_{i,i-1} b'_{i-1}}{a_{ii} - a_{i,i-1} a'_{i-1,i}}, & i = 2, 3, \dots, n. \end{cases}$$

The solution is then obtained by back substitution:

$$x_n = b'_n, \quad x_i = b'_i - a'_{i,i+1} x_{i+1}, \quad i = n-1, n-2, \dots, 1.$$

- (a) How many entries of an $n \times n$ tridiagonal matrix can be nonzero? How does the required storage scale with n if only the nonzero entries are stored?
- (b) What is the time complexity of multiplying a tridiagonal matrix with a vector, if the multiplications with zero entries are skipped?
- (c) How does the Thomas algorithm scale with the size of the matrix?
- (d) Can the matrix-vector multiplication from (b) be parallelized? Can the Thomas algorithm?

Chapter 2

Linear Algebra

Linear algebra is on its own a way to formulate scientific and engineering problems: many physical relationships—mass and energy balances, circuit laws, discretized transport equations—are linear, and expressing them as systems of linear equations makes a vast toolbox of theory and algorithms available. More importantly for this course, linear algebra serves as the key building block of the more complicated problems we shall discuss subsequently: solving nonlinear equations, optimization, statistics, and differential equations all reduce, at their computational core, to a sequence of linear-algebraic subproblems. In this chapter, we review the key concepts of linear algebra (with which some readers may already be familiar), and then move on to a more in-depth discussion of how to solve systems of linear equations.

2.1 Scalars, Vectors, and Matrices

2.1.1 Scalars

Scalars are single numbers, which can be used to represent quantities, like mass, temperature, or time. In linear algebra, we typically work with scalars from a **field**, which is a set equipped with two operations (addition and multiplication). Commonly used fields—the sets of scalars—include real numbers $\mathbb{R}$ and complex numbers $\mathbb{C}$.

Real Numbers

The set of real numbers, commonly denoted by $\mathbb{R}$, includes all rational and irrational numbers. That is, it includes integers like $-1, 0, 1$, fractions like $\frac{1}{2}$, and irrational numbers like π and $\sqrt{2}$.

Complex Numbers

Complex numbers are the numbers that involve imaginary numbers. Imaginary numbers can be constructed by using $\hat{i} = \sqrt{-1}$ as a building block.¹ Whether this number actually exists or not is not important, but what's important is we can *conceptually define this number*, and with this definition, we can derive so many *useful properties*.

Complex numbers are obtained by combining real numbers and imaginary numbers. Conversely, any element in the set of complex numbers $\mathbb{C}$ can be expressed as the combination of real numbers and imaginary numbers, as follows:

$$z = a + b\hat{i}, \quad a, b \in \mathbb{R}$$

We also often denote the real and imaginary parts of the complex number z by

$$\text{Re}(z) = a, \quad \text{Im}(z) = b.$$

¹Throughout these notes, the imaginary unit is written as $\hat{i}$, matching the course slides; this keeps the plain letter i free for use as an index.

The **complex conjugate** of z is denoted by $\bar{z}$ and is defined as:

$$\bar{z} := a - b\hat{i}$$

For real values, we have absolute values that represent the magnitude of the real number. For complex numbers, we can also define a similar notion of magnitude by using the complex conjugate defined above. The **magnitude** (or modulus) of z is

$$|z| := \sqrt{z\bar{z}}$$

The magnitude of a complex number $z = a + b\hat{i}$ can be conveniently computed as follows:

$$|z| = \sqrt{a^2 + b^2}.$$

Example: Compute the conjugate and the magnitude of $z = 3 + 4\hat{i}$.

Solution: The conjugate is $\bar{z} = 3 - 4\hat{i}$, and the magnitude is $|z| = \sqrt{3^2 + 4^2} = 5$.

We can visualize complex numbers as points in the complex plane, with the real part on the horizontal axis and the imaginary part on the vertical axis (Figure 2.1.1).

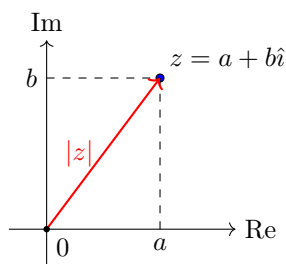


Figure 2.1.1: The complex plane.

In the complex plane, a complex number can also be specified by its magnitude $r = |z|$ and the angle θ it makes with the positive real axis. The two descriptions are connected through **Euler's formula**,

$$e^{i\theta} = \cos \theta + \hat{i} \sin \theta,$$

so any complex number can be written in the polar form $z = r e^{i\theta}$. This representation is particularly convenient for multiplication: multiplying two complex numbers multiplies their magnitudes and adds their angles.

Example: Convert $z = 1 + \hat{i}$ to polar form, and $z = 2e^{i\pi/6}$ to rectangular form.

Solution: For $z = 1 + \hat{i}$, compute $r = \sqrt{1^2 + 1^2} = \sqrt{2}$ and $\theta = \arctan(1/1) = \pi/4$, giving $z = \sqrt{2} e^{i\pi/4}$. For $z = 2e^{i\pi/6}$, compute $a = 2 \cos(\pi/6) = \sqrt{3}$ and $b = 2 \sin(\pi/6) = 1$, giving $z = \sqrt{3} + \hat{i}$.

2.1.2 Vectors

A **vector** is a tuple—an ordered list—of scalars; it has magnitude and direction. In n -dimensional space, a vector $\mathbf{v}$ over the field $\mathbb{R}$ or $\mathbb{C}$ is written as

$$\mathbf{v} = (v_1, v_2, \dots, v_n) = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}, \quad v_i \in \mathbb{R} \text{ or } \mathbb{C}$$

It is common to write vectors either as tuples (arranged horizontally with comma separators) or as column vectors. We will use both notations interchangeably. Unless specified otherwise, all vectors are assumed to be column vectors.

The set of n -dimensional vectors over the field $\mathbb{R}$ is denoted by $\mathbb{R}^n$, and the set of n -dimensional vectors over the field $\mathbb{C}$ is denoted by $\mathbb{C}^n$. The set of all such vectors forms a **vector space** $\mathbb{R}^n$ (or $\mathbb{C}^n$). Later in Section 2.5.1, we will see that $\mathbb{R}^n$ and $\mathbb{C}^n$ form vector spaces, and several useful properties can be derived from the nature of vector spaces.

2.1.2.1 Vector Operations

Vector Addition Two basic arithmetic operations can be defined over the vector space: *addition* and *scalar multiplication*, discussed in this paragraph and the next. Vector addition is defined for a pair of vectors with the same dimension. Given two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ (or $\mathbb{C}^n$), the vector addition $\mathbf{u} + \mathbf{v}$ is defined as follows:

$$\mathbf{u} + \mathbf{v} = \begin{bmatrix} u_1 + v_1 \\ u_2 + v_2 \\ \vdots \\ u_n + v_n \end{bmatrix}$$

That is, the addition is performed entry-wise.

Warning: Vector addition is only defined for vectors of the same dimension. Vector addition between vectors of different dimensions is not defined. If you attempt to add vectors of different dimensions on a computer (e.g., in MATLAB), you will encounter an error:

```
>> [1; 2] + [3; 4; 5]
Arrays have incompatible sizes for this operation.
```

Vector addition satisfies the following properties, for any vectors $\mathbf{u}, \mathbf{v}, \mathbf{w}$ in the same vector space:

- commutativity: $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$;
- associativity: $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$.

Geometrically, vector addition can be interpreted as a “tip-to-tail” construction (see Figure 2.1.2).

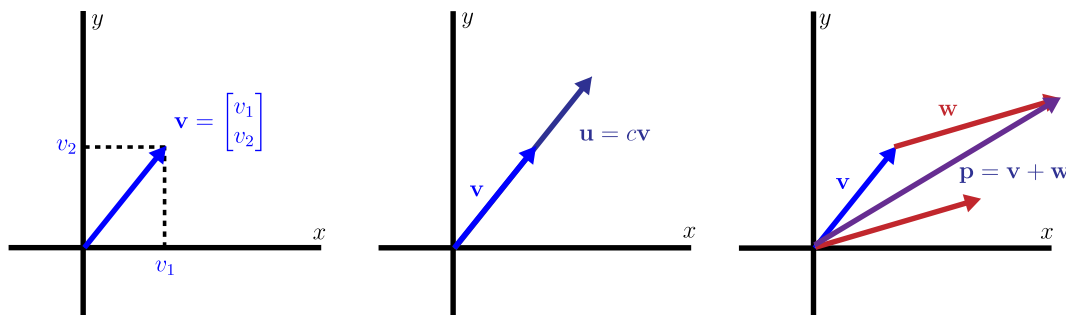
Scalar Multiplication Scalar multiplication is defined for a vector and a scalar. In particular, for a scalar $c \in \mathbb{R}$ (or $\mathbb{C}$) and a vector $\mathbf{v} \in \mathbb{R}^n$ (or $\mathbb{C}^n$), the scalar multiplication is defined as follows:

$$c\mathbf{v} = \begin{bmatrix} cv_1 \\ cv_2 \\ \vdots \\ cv_n \end{bmatrix}$$

This operation is distributive over vector addition and scalar addition. That is,

$$\begin{aligned} c(\mathbf{u} + \mathbf{v}) &= c\mathbf{u} + c\mathbf{v} \\ (c + d)\mathbf{v} &= c\mathbf{v} + d\mathbf{v}, \end{aligned}$$

for any vectors $\mathbf{u}, \mathbf{v}$ in the same vector space and any scalars c, d in the same field. Geometrically, scalar multiplication stretches or shrinks the vector by a factor of c (see Figure 2.1.2).


 Figure 2.1.2: Vector addition and scalar multiplication in $\mathbb{R}^2$.

You can also combine these two operations.

$$a\mathbf{x} + b\mathbf{y} = \begin{bmatrix} ax_1 + by_1 \\ ax_2 + by_2 \\ \vdots \\ ax_n + by_n \end{bmatrix} \quad (2.1.1)$$

where a and b are scalar values, and $\mathbf{x}$ and $\mathbf{y}$ are vectors in $\mathbb{R}^n$. We don't need to separately define subtraction, since $\mathbf{x} - \mathbf{y}$ is equivalent to eq. (2.1.1) with $a = 1$ and $b = -1$.

Closedness We can observe that the result of scalar multiplication, vector addition, and even their combination is *closed* within the vector space where $\mathbf{x}$ and $\mathbf{y}$ live. That is,

$$a\mathbf{x} + b\mathbf{y} \in \mathbb{R}^n \quad \text{for all } \mathbf{x}, \mathbf{y} \in \mathbb{R}^n \text{ and } a, b \in \mathbb{R}.$$

The same holds true if you replace $\mathbb{R}$ with $\mathbb{C}$. This observation is one of the key properties of vector spaces, and we will see more about vector spaces in Section 2.5.1.

Null vector, or zero vector, is a special vector all of whose entries are zero:

$$\mathbf{0} = (0, 0, \dots, 0).$$

Note that the symbol $\mathbf{0}$ itself does not indicate how many entries the vector has; unless the dimension is clear from the context, we need to specify it explicitly (e.g., $\mathbf{0} \in \mathbb{R}^n$).

2.1.3 Matrices

A **matrix** is a rectangular array of scalars arranged in rows and columns. An $m \times n$ matrix $\mathbf{A}$ has m rows and n columns:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}.$$

The notation $\mathbf{A} \in \mathbb{R}^{m \times n}$ indicates the dimensions of $\mathbf{A}$ (and the fact that the entries are drawn from $\mathbb{R}$). In the equation above, we used the lowercase a_{ij} to denote the entry in the i -th row and j -th column of $\mathbf{A}$. Additionally, we can think of vectors as matrices with a single column—that is, as $n \times 1$ matrices. This perspective allows us to treat vectors and matrices in a unified manner. Conversely, a matrix can be decomposed into its columns or its rows:

$$\mathbf{A} = [\mathbf{a}_1^C \quad \mathbf{a}_2^C \quad \dots \quad \mathbf{a}_n^C] = \begin{bmatrix} \mathbf{a}_1^R \\ \mathbf{a}_2^R \\ \vdots \\ \mathbf{a}_m^R \end{bmatrix},$$

where the j -th column $\mathbf{a}_j^C$ (a column vector with m entries) and the i -th row $\mathbf{a}_i^R$ (a row vector with n entries) of $\mathbf{A}$ are defined as

$$\mathbf{a}_j^C = \begin{bmatrix} a_{1j} \\ a_{2j} \\ \vdots \\ a_{mj} \end{bmatrix}, \quad \mathbf{a}_i^R = [a_{i1} \quad a_{i2} \quad \cdots \quad a_{in}].$$

In MATLAB, these can be extracted with the indexing syntax $\mathbf{A}(:, j)$ and $\mathbf{A}(i, :)$.

2.1.3.1 Matrix Operations

Matrix Addition and Scalar Multiplication Like vectors, matrices can be added and scaled entry-wise. Given two matrices $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$ with the same dimensions, the matrix addition $\mathbf{A} + \mathbf{B}$ is defined as follows:

$$\mathbf{A} + \mathbf{B} = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \cdots & a_{1n} + b_{1n} \\ a_{21} + b_{21} & a_{22} + b_{22} & \cdots & a_{2n} + b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} + b_{m1} & a_{m2} + b_{m2} & \cdots & a_{mn} + b_{mn} \end{bmatrix}.$$

Similarly, for a scalar $c \in \mathbb{R}$ (or $\mathbb{C}$), the scalar multiplication $c\mathbf{A}$ is defined as follows:

$$c\mathbf{A} = \begin{bmatrix} c a_{11} & c a_{12} & \cdots & c a_{1n} \\ c a_{21} & c a_{22} & \cdots & c a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c a_{m1} & c a_{m2} & \cdots & c a_{mn} \end{bmatrix}.$$

Entry-wise, $(\mathbf{A} + \mathbf{B})_{ij} = a_{ij} + b_{ij}$ and $(c\mathbf{A})_{ij} = c \cdot a_{ij}$.

Dot Product The **dot product** (inner product) of vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ is

$$\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^n u_i v_i$$

It measures the projection of one vector onto another, which we can geometrically interpret as the product of the magnitudes of the vectors and the cosine of the angle between them (Figure 2.1.3).

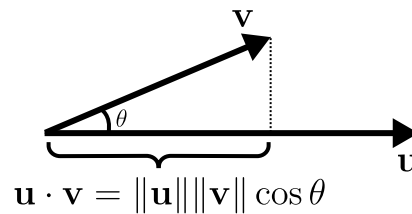


Figure 2.1.3: Geometric interpretation of the dot product.

The dot product satisfies the following properties:

- $\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$ (commutativity)
- $\mathbf{u} \cdot (\mathbf{v} + \mathbf{w}) = \mathbf{u} \cdot \mathbf{v} + \mathbf{u} \cdot \mathbf{w}$ (distributivity)
- $(\alpha \mathbf{u}) \cdot \mathbf{v} = \alpha(\mathbf{u} \cdot \mathbf{v})$ (homogeneity)
- $\mathbf{u} \cdot \mathbf{u} \geq 0$ and $\mathbf{u} \cdot \mathbf{u} = 0$ if and only if $\mathbf{u} = \mathbf{0}$ (positive definiteness)

Matrix-Vector Products The product of an $m \times n$ matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and an n -dimensional vector $\mathbf{x}$ is an m -dimensional vector $\mathbf{b} \in \mathbb{R}^m$:

$$b_i = \sum_{j=1}^n a_{ij}x_j,$$

where b_i denotes the i -th entry of $\mathbf{b}$. Writing out everything explicitly, we have

$$\mathbf{b} = \mathbf{Ax} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \end{bmatrix}.$$

Using the column and row vector notation introduced above, this product can be interpreted in two ways. In the *row form*, each entry of $\mathbf{b}$ is the dot product of a row of $\mathbf{A}$ with $\mathbf{x}$:

$$\mathbf{b} = \mathbf{Ax} = \begin{bmatrix} \mathbf{a}_1^R \mathbf{x} \\ \mathbf{a}_2^R \mathbf{x} \\ \vdots \\ \mathbf{a}_m^R \mathbf{x} \end{bmatrix}.$$

In the *column form*, $\mathbf{b}$ is a linear combination of the columns of $\mathbf{A}$ weighted by the entries of $\mathbf{x}$:

$$\mathbf{b} = \mathbf{Ax} = x_1\mathbf{a}_1^C + x_2\mathbf{a}_2^C + \cdots + x_n\mathbf{a}_n^C.$$

The number of columns in $\mathbf{A}$ must equal the dimension of $\mathbf{x}$ for the product to be defined. Since vectors can be thought of as matrices with one column, matrix-vector multiplication is just a special case of matrix multiplication.

Example: Compute $\mathbf{Ax}$ for $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $\mathbf{x} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$.

Solution:

$$\mathbf{Ax} = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 6 \\ 3 \cdot 5 + 4 \cdot 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

Matrix Multiplication The product of an $m \times n$ matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and an $n \times p$ matrix $\mathbf{B} \in \mathbb{R}^{n \times p}$ is an $m \times p$ matrix $\mathbf{C} \in \mathbb{R}^{m \times p}$:

$$c_{ij} = \sum_{k=1}^n a_{ik}b_{kj}.$$

Writing out everything explicitly, we have

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + \cdots + a_{1n}b_{n1} & \cdots & a_{11}b_{1p} + a_{12}b_{2p} + \cdots + a_{1n}b_{np} \\ \vdots & \ddots & \vdots \\ a_{m1}b_{11} + a_{m2}b_{21} + \cdots + a_{mn}b_{n1} & \cdots & a_{m1}b_{1p} + a_{m2}b_{2p} + \cdots + a_{mn}b_{np} \end{bmatrix}$$

The same interpretations apply here: the entry c_{ij} is the product of the i -th row of $\mathbf{A}$ and the j -th column of $\mathbf{B}$, and each column of $\mathbf{C}$ is a matrix-vector product,

$$c_{ij} = \mathbf{a}_i^R \mathbf{b}_j^C, \quad \mathbf{c}_j^C = \mathbf{A} \mathbf{b}_j^C.$$

Decomposing $\mathbf{B}$ into its columns makes the second interpretation visual: matrix multiplication is a collection of matrix-vector products, one per column of $\mathbf{B}$,

$$\mathbf{AB} = \mathbf{A} [\mathbf{b}_1^C \quad \mathbf{b}_2^C \quad \cdots \quad \mathbf{b}_p^C] = [\mathbf{Ab}_1^C \quad \mathbf{Ab}_2^C \quad \cdots \quad \mathbf{Ab}_p^C],$$

so matrix multiplication is a generalization of the matrix-vector product. Matrix multiplication is associative and distributive but generally not commutative ($\mathbf{AB} \neq \mathbf{BA}$ in general). Additionally, the number of columns in $\mathbf{A}$ must equal the number of rows in $\mathbf{B}$ for the product to be defined.

Example: Multiply $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $\mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$.

Solution:

$$\mathbf{AB} = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$

The product of two matrices, $\mathbf{AB}$, represents the composition of two linear transformations. If matrix $\mathbf{B}$ transforms a vector $\mathbf{x}$ into a new vector $\mathbf{y} = \mathbf{Bx}$, and matrix $\mathbf{A}$ then transforms $\mathbf{y}$ into $\mathbf{z} = \mathbf{Ay}$, the product matrix $\mathbf{C} = \mathbf{AB}$ is the single matrix that represents the entire transformation from $\mathbf{x}$ to $\mathbf{z}$ directly ($\mathbf{z} = \mathbf{Cx}$). This is why the order of multiplication matters; applying transformation $\mathbf{A}$ then $\mathbf{B}$ is generally not the same as applying $\mathbf{B}$ then $\mathbf{A}$.

Warning: As with vector addition, all of the matrix multiplication operations discussed here require compatible dimensions: $\mathbf{AB}$ is defined only when the number of columns of $\mathbf{A}$ equals the number of rows of $\mathbf{B}$. For example, $\mathbf{A} \in \mathbb{R}^{2 \times 3}$ and $\mathbf{B} \in \mathbb{R}^{3 \times 4}$ can be multiplied as $\mathbf{AB} \in \mathbb{R}^{2 \times 4}$ (the inner dimensions, 3 and 3, match), but the product $\mathbf{BA}$ is simply not defined mathematically (the inner dimensions are now 4 and 2). If you attempt such a product on a computer (e.g., in MATLAB), you will encounter an error:

```
>> ones(3, 4) * ones(2, 3)
Error using *
Incorrect dimensions for matrix multiplication.
```

Outer Product The **outer product** of vectors $\mathbf{u} \in \mathbb{R}^m$ and $\mathbf{v} \in \mathbb{R}^n$ is an $m \times n$ matrix:

$$\mathbf{u} \otimes \mathbf{v} = \mathbf{uv}^\top = \begin{bmatrix} u_1 v_1 & u_1 v_2 & \dots & u_1 v_n \\ u_2 v_1 & u_2 v_2 & \dots & u_2 v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_m v_1 & u_m v_2 & \dots & u_m v_n \end{bmatrix}$$

The outer product of vectors satisfies the following properties:

- $(\mathbf{u} \otimes \mathbf{v})^\top = (\mathbf{v} \otimes \mathbf{u})$
- $(\mathbf{v} + \mathbf{w}) \otimes \mathbf{u} = \mathbf{v} \otimes \mathbf{u} + \mathbf{w} \otimes \mathbf{u}$
- $\mathbf{u} \otimes (\mathbf{v} + \mathbf{w}) = \mathbf{u} \otimes \mathbf{v} + \mathbf{u} \otimes \mathbf{w}$
- $c(\mathbf{v} \otimes \mathbf{u}) = (c\mathbf{v}) \otimes \mathbf{u} = \mathbf{v} \otimes (c\mathbf{u})$

There is no need to memorize these properties (or those of the dot product above): they are immediate consequences of the definition of matrix multiplication. To emphasize, the outer product constructs a matrix from the product of two vectors, while the inner product (dot product) constructs a scalar from the product of two vectors.

Transpose of a Matrix The **transpose** of $\mathbf{A}$, denoted $\mathbf{A}^\top$, is obtained by swapping rows with columns:

$$(\mathbf{A}^\top)_{ij} = a_{ji},$$

where $(\mathbf{A}^\top)_{ij}$ denotes the (i, j) -th entry of $\mathbf{A}^\top$. Writing out everything explicitly, we have

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \implies \mathbf{A}^\top = \begin{bmatrix} a_{11} & a_{21} & \dots & a_{m1} \\ a_{12} & a_{22} & \dots & a_{m2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1n} & a_{2n} & \dots & a_{mn} \end{bmatrix}.$$

The transpose satisfies the following properties:

- $(\mathbf{A}^\top)^\top = \mathbf{A}$ (transposing twice recovers the original matrix);
- $(\mathbf{A} + \mathbf{B})^\top = \mathbf{A}^\top + \mathbf{B}^\top$ (the transpose of a sum is the sum of the transposes);
- $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$ whenever the product is defined (the transpose of a product reverses the order).

Example: Find $\mathbf{A}^\top$ for $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$.

Solution: $\mathbf{A}^\top = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$.

Trace of a Matrix For a square matrix $\mathbf{A}$, the **trace** is the sum of its diagonal elements:

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^n a_{ii}$$

The trace is linear: $\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B})$. The trace also satisfies the following properties:

- $\text{tr}(\mathbf{A}^\top) = \text{tr}(\mathbf{A})$
- $\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B})$
- $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$

As a generalization of the last property, the trace is invariant under **cyclic permutations**: for any square matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ such that the product $\mathbf{ABC}$ is defined, $\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{CAB}) = \text{tr}(\mathbf{BCA})$. This does not extend to arbitrary permutations.

Example: For $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, $\mathbf{C} = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}$, compute the trace of each cyclic permutation $\mathbf{ABC}$, $\mathbf{CAB}$, $\mathbf{BCA}$ and of the non-cyclic permutation $\mathbf{ACB}$.

Solution: The cyclic permutations give $\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{CAB}) = \text{tr}(\mathbf{BCA}) = 8$, while the non-cyclic permutation gives $\text{tr}(\mathbf{ACB}) = 12$.

Example: Compute $\text{tr}(\mathbf{A})$ for $\mathbf{A} = \begin{bmatrix} 2 & -1 \\ 4 & 3 \end{bmatrix}$.

Solution: $\text{tr}(\mathbf{A}) = 2 + 3 = 5$.

2.1.3.2 Special Matrices

- **Zero Matrix:** All entries are zero; acts as additive identity ($\mathbf{A} + \mathbf{0} = \mathbf{A}$).

$$\mathbf{0} = \begin{bmatrix} 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}$$

As with the zero vector, the dimensions of $\mathbf{0}$ should be inferred from the context; otherwise, they need to be specified (e.g., $\mathbf{0} \in \mathbb{R}^{m \times n}$).

- **Identity Matrix $\mathbf{I}$:** Square matrix with ones on the diagonal and zeros elsewhere; acts as multiplicative identity ($\mathbf{A}\mathbf{I} = \mathbf{I}\mathbf{A} = \mathbf{A}$).

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

The same goes for the identity matrix: the dimension of $\mathbf{I}$ should be inferred from the context; otherwise, it needs to be specified (e.g., $\mathbf{I} \in \mathbb{R}^{n \times n}$).

- **Symmetric Matrix:** $\mathbf{A} = \mathbf{A}^\top$.
- **Positive Semi-Definite Matrix:** Symmetric and satisfies $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ for all $\mathbf{x} \neq \mathbf{0}$. This is denoted $\mathbf{A} \succeq 0$. Also common is the notion of a positive definite matrix, which requires strict positivity ($\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$, denoted $\mathbf{A} \succ 0$).
- **Diagonal Matrix:** All off-diagonal entries are zero.
- **Triangular Matrix:** A matrix is **lower triangular** if all entries above the main diagonal are zero, and **upper triangular** if all entries below the main diagonal are zero.

2.1.3.3 Computational Complexity of Matrix and Vector Operations

We must understand the computational costs of basic matrix and vector operations in order to develop efficient numerical algorithms. Let's explore three fundamental operations—dot product, matrix-vector multiplication, and matrix-matrix multiplication—and analyze their computational complexities by counting the number of required multiplications and additions.

Dot Product For two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, the dot product $\mathbf{y}^\top \mathbf{x}$ is calculated by multiplying corresponding elements and summing the results:

$$\mathbf{y}^\top \mathbf{x} = \sum_{i=1}^n y_i x_i$$

This operation involves n multiplications (one for each pair $y_i x_i$) and $n - 1$ additions (to sum the n products). The computational complexity is therefore $O(n + n - 1) = O(n)$.

Matrix-Vector Multiplication Given a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and a vector $\mathbf{x} \in \mathbb{R}^n$, the product $\mathbf{A}\mathbf{x}$ results in a vector $\mathbf{b} \in \mathbb{R}^m$:

$$b_i = \sum_{j=1}^n a_{ij} x_j, \quad \text{for } i = 1, 2, \dots, m$$

In the row form, computing $\mathbf{b}$ amounts to m dot products, one per entry of $\mathbf{b}$: each costs $O(n)$, so the total is $O(mn)$. (Counting element-wise gives the same result: each entry requires n multiplications and $n - 1$ additions.)

Matrix-Matrix Multiplication When multiplying two matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$, the resulting matrix $\mathbf{C} = \mathbf{A}\mathbf{B}$ has dimensions $m \times p$:

$$c_{ik} = \sum_{j=1}^n a_{ij} b_{jk}, \quad \text{for } i = 1, 2, \dots, m \text{ and } k = 1, 2, \dots, p$$

By the column decomposition, computing $\mathbf{C}$ amounts to p matrix-vector products, one per column of $\mathbf{B}$: each costs $O(mn)$, so the total is $O(mnp)$. (Counting element-wise gives the same result: each of the mp entries requires n multiplications and $n - 1$ additions.)

Example: Count the number of scalar operations needed to compute the product of two 3×3 matrices:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix}, \quad \mathbf{C} = \mathbf{AB}$$

Solution: The resulting matrix $\mathbf{C} \in \mathbb{R}^{3 \times 3}$ has elements:

$$\begin{aligned} c_{11} &= a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} \\ c_{12} &= a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32} \\ c_{13} &= a_{11}b_{13} + a_{12}b_{23} + a_{13}b_{33} \\ &\vdots \end{aligned}$$

We see that each element of $\mathbf{C}$ requires 3 multiplications and 2 additions. And since $\mathbf{C}$ has 9 elements, the total number of operations is $9 \cdot (3 + 2) = 45$.

But wait, isn't the complexity of matrix-matrix multiplication $O(mnp)$? And in this case $m = n = p = 3$, so shouldn't we have $3^3 = 27$ operations? No! That is not how big-O notation works. Big-O notation is NOT a precise count of operations but rather an upper bound on the growth rate of the number of operations as the size of the matrices grows.

State-of-the-Art Matrix Multiplication (Optional)

In reality, the computational complexity of matrix-matrix multiplication is not exactly cubic. A lot of research has been done to optimize matrix-matrix multiplication due to its extreme importance in the field of optimization and machine learning. For square $n \times n$ matrices, the fastest known algorithms run in $O(n^\omega)$ time, where, as of 2026, the record is $\omega < 2.371177$ [1], which improved on the previous record of $\omega < 2.371339$ [2]. Very roughly, these bounds come from the Coppersmith-Winograd line of ideas: view matrix multiply as a tensor and look for low-rank decompositions. One recent twist is to allow more asymmetry between the three dimensions (don't force two to behave identically); similar tricks help with rectangular cases, too. Another, used in the 2026 record, is to attack the resulting enormous optimization problems with modern nonlinear solvers and even AI-assisted search (DeepMind's AlphaEvolve). But these bounds are galactic algorithms, meaning the constants (for big O) are huge, so real code sticks to near-cubic methods with blocking, cache-friendly layouts, vectorization, and parallelism. Strassen's method is the oldest of these ideas: partition the matrices into 2×2 blocks; the naive block multiplication needs 8 block products, but a clever rearrangement computes the result with only 7 (at the cost of extra additions), and applying the trick recursively gives $O(n^{\log_2 7}) \approx O(n^{2.8074})$. It sometimes shows up in practice for very large n , but most production matrix multiplication is effectively $O(n^3)$, so you should stick with $O(n^3)$ as a rule of thumb.

Optimizing Computations in Complex Expressions

When working with expressions involving multiple matrix and vector operations, the order in which you perform these operations can greatly affect the computational cost. By strategically choosing the sequence of calculations, you can minimize the number of required operations and optimize performance.

Matrix multiplication is associative but generally not commutative; matrix addition is both associative and commutative. The dot product is commutative, and scalars commute with everything. Some structured matrices (e.g., diagonal) commute with each other, but in general do *not* assume $\mathbf{AB} = \mathbf{BA}$. These properties allow some flexibility in how calculations are grouped and ordered.

Matrix-Vector Multiplication in Computation Chains As noted previously, the computational complexity of matrix-vector multiplication is "quadratic" while the complexity of matrix-matrix multiplication is "cubic." Therefore, when given a chain of matrix-matrix multiplications that has at least one matrix-vector multiplication, it is often more efficient to perform the operations in a way that minimizes the number of

matrix-matrix multiplications and maximizes the number of matrix-vector multiplications.

It is easiest to see this by example. Consider the expression:

$$\mathbf{y} = \mathbf{ABCx}$$

where $\mathbf{A} \in \mathbb{R}^{3 \times 3}$, $\mathbf{B} \in \mathbb{R}^{3 \times 3}$, $\mathbf{C} \in \mathbb{R}^{3 \times 3}$, and $\mathbf{x} \in \mathbb{R}^3$. To compute $\mathbf{y}$, we can perform the operations in a few different ways by using the associativity property of matrix multiplication. The first way we consider is to rewrite the expression as

$$\mathbf{y} = \mathbf{AB}(\mathbf{Cx}) = \mathbf{A}(\mathbf{Bd}) = \mathbf{Ae}$$

where we compute $\mathbf{d} = \mathbf{Cx}$ first, then $\mathbf{e} = \mathbf{Bd}$, and finally $\mathbf{y} = \mathbf{Ae}$. Importantly, this sequence of operations requires only matrix-vector multiplications (three in total) and no matrix-matrix multiplications. We can compare this to a second way of computing $\mathbf{y}$:

$$\mathbf{y} = (\mathbf{AB})\mathbf{Cx} = (\mathbf{DC})\mathbf{x} = \mathbf{Fx}$$

This requires two matrix-matrix multiplications and one matrix-vector multiplication. Therefore, the first way is more efficient than the second way (concretely, 45 operations vs. 105 operations—check this yourself!).

Warning: The advice to always perform matrix-vector products first is an excellent rule of thumb, but it is not a universal law. The true underlying principle is to **avoid creating large intermediate products**, whether they are matrices or vectors. In cases involving non-square matrices, this can lead to surprising results. Consider the expression $\mathbf{y} = \mathbf{ABx}$ where the dimensions are not square:

- $\mathbf{A} \in \mathbb{R}^{1 \times 1000}$
- $\mathbf{B} \in \mathbb{R}^{1000 \times 100}$
- $\mathbf{x} \in \mathbb{R}^{100}$

Let's compare the two ways of computing the result based on the number of multiplications:

1. **Matrix-Vector First:** $\mathbf{y} = \mathbf{A}(\mathbf{Bx})$ We first compute $\mathbf{d} = \mathbf{Bx}$. This requires $1000 \times 100 = 100,000$ multiplications and creates a large intermediate vector $\mathbf{d} \in \mathbb{R}^{1000}$. Then, computing $\mathbf{y} = \mathbf{Ad}$ requires $1 \times 1000 = 1,000$ multiplications. Total Cost: $100,000 + 1,000 = 101,000$ multiplications.
2. **Matrix-Matrix First:** $\mathbf{y} = (\mathbf{AB})\mathbf{x}$ We first compute $\mathbf{C} = \mathbf{AB}$. This requires $1 \times 1000 \times 100 = 100,000$ multiplications and creates a small intermediate row vector $\mathbf{C} \in \mathbb{R}^{1 \times 100}$. Then, computing $\mathbf{y} = \mathbf{Cx}$ requires $1 \times 100 = 100$ multiplications. Total Cost: $100,000 + 100 = 100,100$ multiplications.

In this case, performing the matrix-matrix multiplication first is slightly more efficient in terms of number of floating point operations required (the amount of additions is the same in either case: 99,999). It avoids creating the huge intermediate vector ($\mathbf{d} \in \mathbb{R}^{1000}$) and instead creates a much smaller intermediate vector ($\mathbf{C} \in \mathbb{R}^{1 \times 100}$). So we can indeed see that the optimal order depends entirely on the dimensions of all matrices in the chain. Actually, finding this optimal order is a classic computer science problem known as the matrix chain ordering problem (you can read its [Wikipedia page](#) if you're curious!).

Diagonal Matrices Suppose $\mathbf{D} \in \mathbb{R}^{n \times n}$ is a diagonal matrix and $\mathbf{x} \in \mathbb{R}^n$, and you need to compute $\mathbf{y} = \mathbf{Dx}$. Using standard matrix multiplication, this operation would be $O(n^2)$. But since only the diagonal elements D_{ii} are non-zero, we only need to perform n multiplications (one multiplication per diagonal element) and zero additions, meaning the computational complexity can be reduced to $O(n)$.

Sparse Matrices Sparse matrices (which are *very* common in scientific applications!) are matrices for which most elements are zero. They are stored in a way that only the non-zero elements are explicitly represented (in MATLAB, this is never automatically done for you—you must use the `sparse` function yourself to convert from dense to sparse format). Commonly, we must multiply a sparse matrix with a dense vector, which means that the result is the dot product of each sparse row of the matrix with the dense vector. This requires significantly fewer operations than a full matrix-vector multiplication (since we do not need to actually multiply anything with zero).

Symmetric Matrices Symmetric matrices are matrices that are equal to their own transpose: $\mathbf{A} = \mathbf{A}^\top$. For a symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, a natural question to ask is: can we halve the work in $\mathbf{y} = \mathbf{A}\mathbf{x}$ by only touching one triangle? You *can* store just one triangle (saving about half the memory), but you *cannot* cut the arithmetic in half. To see why, write out the updates if you traverse only the upper triangle, and you'll find that the number of operations needed still comes out to $O(n^2)$. Where symmetry really shines is in factorizations/solves.

2.2 Vector and Matrix Norms

2.2.1 Norms of Vectors

A **norm** is a function that assigns a non-negative length or size to vectors. The most commonly used family of norms is the **p -norm**, defined for $p \geq 1$ as

$$\|\mathbf{v}\|_p = \left(\sum_{i=1}^n |v_i|^p \right)^{1/p}.$$

Several special cases are used so frequently that they have their own names:

- **1-Norm** ($p = 1$; also called the Manhattan norm):

$$\|\mathbf{v}\|_1 = \sum_{i=1}^n |v_i|$$

- **2-Norm** ($p = 2$; Euclidean norm):

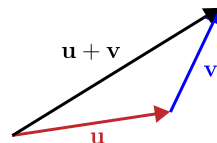
$$\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^n v_i^2} = \sqrt{\mathbf{v}^\top \mathbf{v}}$$

- **Infinity Norm** (the limit $p \rightarrow \infty$; also called the maximum norm):

$$\|\mathbf{v}\|_\infty = \max_{1 \leq i \leq n} |v_i|$$

These norms satisfy the following properties:

- Positive definiteness: $\|\mathbf{v}\| \geq 0$, and $\|\mathbf{v}\| = 0$ if and only if $\mathbf{v} = \mathbf{0}$.
- Absolute homogeneity: $\|\alpha \mathbf{v}\| = |\alpha| \|\mathbf{v}\|$ for scalar α .
- Triangle inequality: $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$.



Example: Compute the 1-, 2-, and ∞ -norms of $\mathbf{v} = \begin{bmatrix} 3 \\ -4 \end{bmatrix}$.

Solution:

$$\begin{aligned} \|\mathbf{v}\|_1 &= |3| + |-4| = 7, \\ \|\mathbf{v}\|_2 &= \sqrt{3^2 + (-4)^2} = 5, \\ \|\mathbf{v}\|_\infty &= \max\{|3|, |-4|\} = 4. \end{aligned}$$

2.2.2 Norms of Matrices

Matrix norms extend the concept of vector norms to matrices, providing a measure of the “size” or “magnitude” of a matrix. Just as vector norms quantify the length of vectors, matrix norms help assess various properties of matrices, such as their behavior under linear transformations, stability in numerical computations, and convergence in iterative methods.

2.2.2.1 Induced Matrix Norms

One fundamental way to define a matrix norm is through an **induced (operator) norm**, which is derived from a vector norm $\|\cdot\|$ on $\mathbb{R}^n$ or $\mathbb{C}^n$, the induced matrix norm for a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is defined as

$$\|\mathbf{A}\| = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{A}\mathbf{x}\|}{\|\mathbf{x}\|}$$

The induced matrix norm measures the maximum stretching factor of the matrix $\mathbf{A}$ when it acts as a linear transformation on vectors. Specifically, it identifies the largest possible ratio of the norm of $\mathbf{A}\mathbf{x}$ to the norm of $\mathbf{x}$ across all non-zero vectors $\mathbf{x}$. This tells us how much $\mathbf{A}$ can amplify a vector.

Induced norms satisfy the following properties:

- $\|\mathbf{A}\| \geq 0$, $\|\mathbf{A}\| = 0$ only if $\mathbf{A} = \mathbf{0}$
- $\|c\mathbf{A}\| = |c|\|\mathbf{A}\|$
- $\|\mathbf{A}\mathbf{x}\| \leq \|\mathbf{A}\|\|\mathbf{x}\|$ (submultiplicative property)
- $\|\mathbf{A}\mathbf{B}\| \leq \|\mathbf{A}\|\|\mathbf{B}\|$ (submultiplicative property)
- $\|\mathbf{A} + \mathbf{B}\| \leq \|\mathbf{A}\| + \|\mathbf{B}\|$

The submultiplicative property is a very important property of induced/operator norms! This property allows us to bound the norm of a product of matrices by the product of their norms and will be very useful in the analysis of iterative methods.

2.2.2.2 Common Matrix Norms

Several matrix norms are commonly used. Below, we take a look at a few of these norms and their interpretations. The first three are induced norms derived from vector norms (and in particular p -norms), while the last norm is a matrix-specific norm.

- **1-Norm (Maximum Column Sum):**

$$\|\mathbf{A}\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|$$

The 1-norm of a matrix is the largest sum of absolute values of the elements in any single column. It represents the maximum effect that $\mathbf{A}$ can have on the 1-norm of a vector, emphasizing the most “influential” column in terms of contributing to the overall transformation.

- **2-Norm (Spectral Norm):**

$$\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A})$$

where $\sigma_{\max}(\mathbf{A})$ denotes the largest singular value of $\mathbf{A}$ (singular values are introduced in Section 2.7.4). The 2-norm measures the maximum stretching factor of $\mathbf{A}$ in any direction, corresponding to the largest singular value. It is closely related to the concept of eigenvalues and provides a precise measure of the matrix’s action on vectors in terms of scaling.

- **Infinity Norm (Maximum Row Sum):**

$$\|\mathbf{A}\|_{\infty} = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$$

The infinity norm is the largest sum of absolute values of the elements in any single row. It reflects the maximum effect that $\mathbf{A}$ can have on the infinity norm of a vector, highlighting the most “influential” row in terms of transformation impact.

- **Frobenius Norm:**

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2}$$

Again, this is NOT an induced norm. The Frobenius norm is very similar to the Euclidean norm (which is defined for vectors). It measures the “overall size” of the matrix by taking the square root of the sum of the squares of all its entries. This norm is particularly useful in applications where the collective magnitude of all matrix elements is of interest, such as in least squares problems and matrix approximations.

2.2.2.3 Relationships Between Matrix Norms

For any matrix $\mathbf{A}$ of size $m \times n$ and rank r , the following inequalities hold:

$$\begin{aligned}\|\mathbf{A}\|_2 &\leq \|\mathbf{A}\|_F \leq \sqrt{r} \|\mathbf{A}\|_2, \\ \|\mathbf{A}\|_2 &\leq \sqrt{\|\mathbf{A}\|_1 \|\mathbf{A}\|_\infty}.\end{aligned}$$

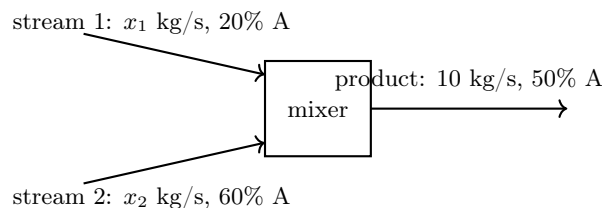
2.3 Introduction to Systems of Linear Equations

Linear systems arise in many scientific and engineering applications (even in seemingly nonlinear cases, we can Taylor expand to a locally linear system), so we should develop tools to numerically solve them. The general form of a linear system is

$$\mathbf{A}\mathbf{x} = \mathbf{b} \quad (2.3.1)$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a known coefficient matrix, $\mathbf{x} \in \mathbb{R}^n$ is the vector of unknowns, and $\mathbf{b} \in \mathbb{R}^m$ is a known vector. The goal is to find $\mathbf{x}$ that satisfies this equation. We know from linear algebra that this linear system has a unique solution for any possible $\mathbf{b}$ if and only if $\mathbf{A}$ is square ($m = n$) and has full rank (i.e., $\text{rank}(\mathbf{A}) = n$). Otherwise, the system may have no solution or infinitely many solutions. If $m \neq n$, we typically seek a least-squares ($m > n$) or minimum-norm ($m < n$) solution.

To build intuition, consider a simple mass balance. Two streams are mixed to produce 10 kg/s of product. Stream 1 carries a mass fraction 0.2 of species A, and stream 2 carries 0.6; the product must carry 0.5.



Denoting the unknown flow rates of the two streams by x_1 and x_2 , the total and species mass balances read

$$\begin{aligned}x_1 + x_2 &= 10 \\ 0.2x_1 + 0.6x_2 &= 5,\end{aligned}$$

or, in the matrix form of eq. (2.3.1),

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0.2 & 0.6 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 10 \\ 5 \end{bmatrix}.$$

This system can be viewed in two ways. In the **row view**, each row of $\mathbf{A}$ is one balance equation,

$$\mathbf{a}_1^R \mathbf{x} = [1 \quad 1] \mathbf{x} = 10, \quad \mathbf{a}_2^R \mathbf{x} = [0.2 \quad 0.6] \mathbf{x} = 5,$$

each defining a line in the (x_1, x_2) plane, and the solution is the point where the two lines intersect (Figure 2.3.1, left). In the **column view**, the same system reads

$$x_1 \mathbf{a}_1^C + x_2 \mathbf{a}_2^C = \mathbf{b}, \quad \mathbf{a}_1^C = \begin{bmatrix} 1 \\ 0.2 \end{bmatrix}, \quad \mathbf{a}_2^C = \begin{bmatrix} 1 \\ 0.6 \end{bmatrix},$$

where each column collects the contributions of one unit of a stream (one unit of flow together with its species content); solving the system means finding the combination of the columns that reproduces the target $\mathbf{b}$ (Figure 2.3.1, right). Here, the solution is $x_1 = 2.5$ and $x_2 = 7.5$ kg/s.

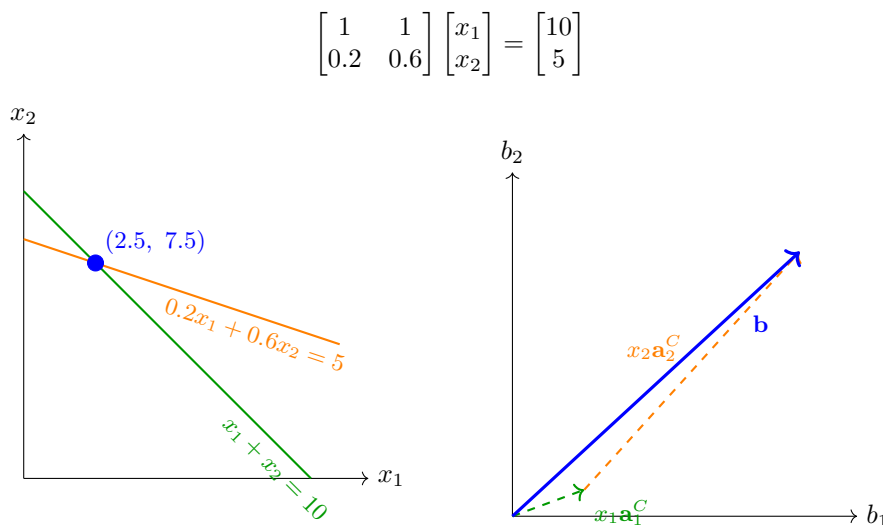


Figure 2.3.1: The same linear system, viewed from two perspectives. Row view (left): each balance equation is a line in the (x_1, x_2) plane, and the solution is their intersection. Column view (right): in the (b_1, b_2) plane of attainable products, the solution synthesizes the target $\mathbf{b}$ from the two column “ingredients” laid tip to tail, $\mathbf{b} = x_1 \mathbf{a}_1^C + x_2 \mathbf{a}_2^C$.

A solution does not always exist, and when it exists, it need not be unique. In the example, suppose the two streams carry the *same* mass fraction, say 0.5, so that the two columns become parallel (equivalently, one row becomes a multiple of the other). A product with fraction 0.5 can then be blended in infinitely many ways: the two lines in the row view coincide, and in the column view, $\mathbf{b}$ lies on the line spanned by the two parallel columns. Any other target fraction, however, is impossible: the two lines are parallel and never intersect, and $\mathbf{b}$ lies off the line spanned by the columns (Figure 2.3.2).

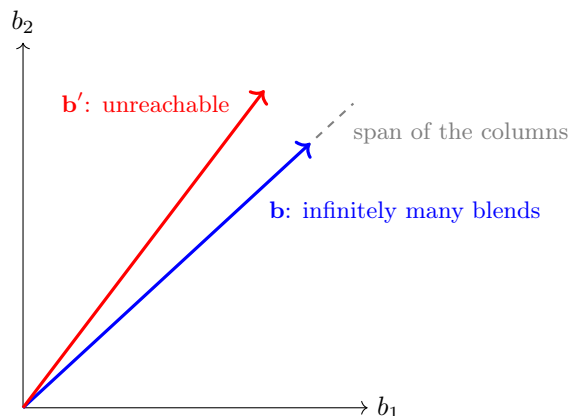


Figure 2.3.2: The degenerate case in the column view: when both streams carry the same mass fraction, the two columns point along the same line. A target on that line ($\mathbf{b}$) can be synthesized in infinitely many ways; a target off the line ($\mathbf{b}'$) cannot be reached.

We conclude by setting the baseline for the rest of this chapter: a system of linear equations may have zero, one, or infinitely many solutions. We are now interested in characterizing under what circumstances each case holds and, when a unique solution exists, how to find it numerically.

2.4 Invertibility and Determinants

2.4.1 Inverse of a Matrix

Conceptually, the inverse of a matrix is an operator that reverses the linear transformation performed by the original matrix $\mathbf{A}$. If $\mathbf{A}$ transforms a vector $\mathbf{x}$ into $\mathbf{y}$, then its inverse transforms $\mathbf{y}$ back to the original vector $\mathbf{x}$.

Formally, a square matrix $\mathbf{A}$ is **invertible** (or **non-singular**) if there exists a matrix $\mathbf{B}$ such that

$$\mathbf{AB} = \mathbf{BA} = \mathbf{I},$$

where $\mathbf{I}$ is the identity matrix. Such a matrix $\mathbf{B}$, when it exists, is unique; it is called the *inverse* of $\mathbf{A}$, and we denote it by $\mathbf{A}^{-1}$. This property is analogous to a number and its reciprocal (e.g., $5 \cdot 5^{-1} = 1$).

This “undoing” capability is fundamental for solving systems of linear equations. A system represented as $\mathbf{Ax} = \mathbf{b}$ can be solved for the unknown vector $\mathbf{x}$ by applying the inverse: $\mathbf{A}^{-1}\mathbf{Ax} = \mathbf{A}^{-1}\mathbf{b}$, which simplifies to $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$.

The inverse satisfies the following properties, whenever the inverses involved exist:

- $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ (as with the transpose, the order is reversed);
- $(\mathbf{A}^\top)^{-1} = (\mathbf{A}^{-1})^\top$.

For a 2×2 matrix, the inverse can be calculated directly:

$$\text{If } \mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \text{ then } \mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}, \text{ provided } ad - bc \neq 0$$

While the 2×2 formula is simple, computing the inverse for a general $n \times n$ matrix is a numerically intensive task. In practice, one typically avoids computing the inverse explicitly. For problems like solving $\mathbf{Ax} = \mathbf{b}$, it is both more efficient and more numerically stable to solve the system directly using matrix factorization or other methods instead, which we will discuss in more detail later.

2.4.2 Determinant

For a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the **determinant** $\det(\mathbf{A})$ is a scalar value that can tell us a lot about the matrix's properties. Geometrically, it represents the signed volume-scaling factor of the linear transformation described by $\mathbf{A}$. If $|\det(\mathbf{A})| = 2$, the transformation doubles the volume of a region, while if $|\det(\mathbf{A})| = 0.5$, it halves the volume. The sign of the determinant tells us whether the transformation preserves or reverses orientation. A positive determinant means the orientation is preserved, while a negative determinant means it's flipped (think of it as turning a shape "inside-out" like a reflection). If $\det(\mathbf{A}) = 0$, the transformation collapses the space into a lower dimension (e.g., squashing a 2D square into a line segment). When this happens, information is lost, and you can't reverse the transformation.

Theorem 1 (Determinant and invertibility). *A square matrix $\mathbf{A}$ is invertible if and only if $\det(\mathbf{A}) \neq 0$.*

Formally, the determinant can be defined recursively. For $n = 1$, $\det([a]) = a$. For $n \geq 2$, let the *minor* M_{ij} denote the determinant of the $(n - 1) \times (n - 1)$ submatrix obtained by deleting row i and column j of $\mathbf{A}$; then, for *any* choice of row i ,

$$\det(\mathbf{A}) = \sum_{j=1}^n (-1)^{i+j} a_{ij} M_{ij},$$

which is called the *Laplace expansion* along row i . Although this formula appears to depend on the choice of the row i , every choice gives the same value, so the definition is unambiguous; the same value is even obtained by expanding along any *column*. We will see this freedom put to use in the exercises, where expanding along the row or column with the most zeros simplifies the computation.

The determinant has several important properties:

- $\det(\mathbf{A}^\top) = \det(\mathbf{A})$;
- $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$ (the multiplicative property);
- for an invertible matrix, $\det(\mathbf{A}^{-1}) = 1/\det(\mathbf{A})$;
- if $\mathbf{A}$ is diagonal or triangular, $\det(\mathbf{A}) = a_{11}a_{22} \cdots a_{nn}$, the product of the diagonal entries (repeatedly expand along the first row or column);
- the determinant is linear in each row (and each column) separately; in particular, multiplying a single row by c multiplies the determinant by c ;
- swapping two rows (or two columns) flips the sign of the determinant;
- adding a multiple of one row to another row leaves the determinant unchanged (which is why Gaussian elimination can be used to compute determinants).

For a 2×2 matrix, the definition reduces to

$$\det\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = ad - bc$$

For a 3×3 matrix, one common method is cofactor expansion along the first row:

$$\begin{aligned} \det\left(\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}\right) &= a \det\left(\begin{bmatrix} e & f \\ h & i \end{bmatrix}\right) - b \det\left(\begin{bmatrix} d & f \\ g & i \end{bmatrix}\right) + c \det\left(\begin{bmatrix} d & e \\ g & h \end{bmatrix}\right) \\ &= a(ei - fh) - b(di - fg) + c(dh - eg) \end{aligned}$$

In practice, Laplace expansion is only used for small matrices; large determinants are almost always computed using Gaussian elimination or LU decomposition instead.

2.5 Existence and Uniqueness of Solutions

2.5.1 Vector Spaces

A **vector space** is a set of vectors equipped with two operations, vector addition and scalar multiplication, that is *closed* under both: adding two vectors in the set, or multiplying one by a scalar, yields another vector in the set. The space $\mathbb{R}^n$, with the operations defined earlier in this chapter, is the primary example. A subset of a vector space that is itself closed under the same two operations is called a **subspace**. For example, a line or a plane through the origin is a subspace of $\mathbb{R}^3$, while a line that misses the origin is not: closedness requires the set to contain $0 \cdot \mathbf{v} = \mathbf{0}$.

Closedness under Linear Combinations The two closedness requirements can be stated as one: for any vectors $\mathbf{u}, \mathbf{v}$ in the space and any scalars c, d , the combination $c\mathbf{u} + d\mathbf{v}$ is again in the space. (Taking $c = d = 1$ recovers closedness under vector addition, and taking $d = 0$ recovers closedness under scalar multiplication.) The familiar arithmetic properties of the two operations—commutativity, associativity, and distributivity, as listed earlier in this chapter—continue to hold. Closedness under linear combinations is the essential feature of a vector space, and the concepts that follow—span, linear independence, basis, and dimension—are all built on it.

Linear Combinations and Span A **linear combination** of vectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ is an expression of the form:

$$\mathbf{w} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \dots + c_k\mathbf{v}_k$$

where $c_1, c_2, \dots, c_k$ are scalars. The set of all possible linear combinations of $\mathbf{v}_1, \dots, \mathbf{v}_k$ is called their **span**:

$$\text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\} = \left\{ \sum_{i=1}^k c_i \mathbf{v}_i \mid c_i \in \mathbb{R} \text{ or } \mathbb{C} \right\}$$

Example: What is the span of the vectors $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ in $\mathbb{R}^2$?

Solution: Their span is the entire $\mathbb{R}^2$ plane since any vector $\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$ can be written as $w_1\mathbf{v}_1 + w_2\mathbf{v}_2$

Linear Independence and Basis Vectors $\mathbf{v}_1, \dots, \mathbf{v}_k$ are **linearly independent** if the only solution to

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \dots + c_k\mathbf{v}_k = \mathbf{0}$$

is $c_1 = c_2 = \dots = c_k = 0$.

A set of vectors that is linearly independent and spans the vector space is called a **basis**. The number of vectors in a basis equals the **dimension** of the space.

The Unit Vectors The **unit vectors** in $\mathbb{R}^n$ are:

$$\mathbf{e}_i = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \quad \text{where } (\mathbf{e}_i)_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

The unit vectors form a basis for $\mathbb{R}^n$, meaning any vector $\mathbf{v}$ can be written as a linear combination of the unit vectors.

Example: Write down the unit vectors of $\mathbb{R}^3$.

Solution: The unit vectors are $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$, $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$, and $\mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$.

2.5.2 Orthogonality

Vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ are **orthogonal** if $\mathbf{u}^\top \mathbf{v} = 0$. A set of vectors is **orthonormal** if vectors are mutually orthogonal and each has unit norm. A square matrix $\mathbf{Q}$ is **orthogonal** if $\mathbf{Q}^\top \mathbf{Q} = \mathbf{Q} \mathbf{Q}^\top = \mathbf{I}$, which implies $\mathbf{Q}^{-1} = \mathbf{Q}^\top$ and that $\|\mathbf{Q}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$ for all $\mathbf{x}$.

2.5.3 The four fundamental subspaces

We next formalize when a linear system has a solution and when that solution is unique. When needed, we denote by $\mathbf{A}_i^c$ the i -th column and by $\mathbf{A}_i^r$ the i -th row.

For a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ there are four canonical subspaces:

1. **Column space (range):** $\mathcal{R}(\mathbf{A}) = \text{span}\{\mathbf{a}_1^c, \dots, \mathbf{a}_M^c\} \subseteq \mathbb{R}^m$
2. **Null space (kernel):** $\mathcal{N}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{0}\}$;
3. **Row space:** $\mathcal{R}(\mathbf{A}^\top) = \text{span}\{\mathbf{a}_1^r, \dots, \mathbf{a}_N^r\} \subseteq \mathbb{R}^n$
4. **Left null space:** $\mathcal{N}(\mathbf{A}^\top) = \{\mathbf{y} \in \mathbb{R}^m : \mathbf{A}^\top \mathbf{y} = \mathbf{0}\}$

These subspaces organize the two core questions for a system $\mathbf{A}\mathbf{x} = \mathbf{b}$: *existence* (is $\mathbf{b}$ achievable?) and *uniqueness* (if achievable, is the achiever $\mathbf{x}$ unique?).

Think of $\mathbf{A} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ as a machine that takes an input vector $\mathbf{x}$ and produces an output $\mathbf{b} = \mathbf{A}\mathbf{x}$. The set of all outputs the machine can possibly produce is the *column space* $\mathcal{R}(\mathbf{A})$. Directions in input space that the machine completely ignores (they map to $\mathbf{0}$) form the *null space* $\mathcal{N}(\mathbf{A})$. Thus:

$$\text{existence is about } \mathbf{b} \in \mathcal{R}(\mathbf{A}), \quad \text{uniqueness is about } \mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$$

The four subspaces come in orthogonal pairs:

$$\mathcal{R}(\mathbf{A}) = \mathcal{N}(\mathbf{A}^\top)^\perp \quad \text{and} \quad \mathcal{R}(\mathbf{A}^\top) = \mathcal{N}(\mathbf{A})^\perp$$

Consequently,

$$\mathbb{R}^m = \mathcal{R}(\mathbf{A}) \oplus \mathcal{N}(\mathbf{A}^\top), \quad \mathbb{R}^n = \mathcal{R}(\mathbf{A}^\top) \oplus \mathcal{N}(\mathbf{A})$$

In words, this means that for an $m \times n$ matrix $\mathbf{A}$, every vector in $\mathbb{R}^m$ can be written uniquely as a sum of something in the column space $\mathcal{R}(\mathbf{A})$ and something in the left null space $\mathcal{N}(\mathbf{A}^\top)$. Likewise, every vector in $\mathbb{R}^n$ splits uniquely into a part in the row space $\mathcal{R}(\mathbf{A}^\top)$ and a part in the null space $\mathcal{N}(\mathbf{A})$. As a dimension check this yields

$$\dim \mathcal{R}(\mathbf{A}) + \dim \mathcal{N}(\mathbf{A}^\top) = m \quad \dim \mathcal{R}(\mathbf{A}^\top) + \dim \mathcal{N}(\mathbf{A}) = n$$

It is also a basic fact that row rank equals column rank:

$$\text{rank}(\mathbf{A}) = \dim \mathcal{R}(\mathbf{A}) = \dim \mathcal{R}(\mathbf{A}^\top) = \text{rank}(\mathbf{A}^\top)$$

Theorem 2 (Existence via the column space). *A solution to $\mathbf{A}\mathbf{x} = \mathbf{b}$ exists for a given $\mathbf{b}$ if and only if $\mathbf{b} \in \mathcal{R}(\mathbf{A})$.*

Proof. Using the column view,

$$\mathbf{Ax} = \sum_{i=1}^n x_i \mathbf{a}_i^C$$

If $\mathbf{b} \in \mathcal{R}(\mathbf{A})$, then by definition there exist coefficients x_i so that the sum equals $\mathbf{b}$, hence a solution exists. Conversely, if $\mathbf{Ax} = \mathbf{b}$ for some $\mathbf{x}$, then $\mathbf{b}$ is a linear combination of the columns of $\mathbf{A}$, so $\mathbf{b} \in \mathcal{R}(\mathbf{A})$. $\square$

We can also look at things geometrically. The column space is a subspace (a line/plane/higher-dimensional flat) inside $\mathbb{R}^m$. Asking for a solution is asking whether $\mathbf{b}$ lies *on* that flat. If not, there is no exact solution; if yes, there is at least one.

Example (existence): Let

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{b}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}.$$

For which of $\mathbf{b}_1$ and $\mathbf{b}_2$ does the system $\mathbf{Ax} = \mathbf{b}$ have a solution?

Solution: The column space is $\mathcal{R}(\mathbf{A}) = \text{span}\{(1, 0, 0)^\top\}$, a 1-dimensional line in $\mathbb{R}^3$. Thus $\mathbf{b}_1$ lies on that line, so solutions exist (e.g., $x_1 = 1$ and x_2, x_3 arbitrary). The vector $\mathbf{b}_2$ lies off the line, so no solution exists.

2.5.4 Consistency via the left null space (Fredholm alternative)

The system $\mathbf{Ax} = \mathbf{b}$ is solvable if and only if $\mathbf{b} \perp \mathcal{N}(\mathbf{A}^\top)$; equivalently, $\mathbf{b} \in \mathcal{R}(\mathbf{A})$. In practice this gives a test: if $\{\mathbf{y}_1, \dots, \mathbf{y}_k\}$ is a basis of $\mathcal{N}(\mathbf{A}^\top)$, then $\mathbf{Ax} = \mathbf{b}$ is consistent precisely when $\mathbf{y}_j^\top \mathbf{b} = 0$ for all j .

Theorem 3 (Uniqueness via the null space). *If a solution to $\mathbf{Ax} = \mathbf{b}$ exists, it is unique if and only if $\dim \mathcal{N}(\mathbf{A}) = 0$ (equivalently, $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$).*

Proof. ($\Leftarrow$) If $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$ and $\mathbf{Ax} = \mathbf{Ay} = \mathbf{b}$, then $\mathbf{A}(\mathbf{x} - \mathbf{y}) = \mathbf{0}$, hence $\mathbf{x} - \mathbf{y} = \mathbf{0}$ and $\mathbf{x} = \mathbf{y}$. ($\Rightarrow$) Suppose a solution exists and is unique. If there were a nonzero $\mathbf{x}^H \in \mathcal{N}(\mathbf{A})$, then for any particular solution $\mathbf{x}^P$, the vector $\mathbf{x}^P + \mathbf{x}^H$ would be a distinct solution, a contradiction. $\square$

Example (uniqueness with $n < m$): Consider

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 2}.$$

Which right-hand sides $\mathbf{b}$ make the system $\mathbf{Ax} = \mathbf{b}$ consistent, and is the solution then unique?

Solution: The columns are $\mathbf{c}_1 = (1, 1, 1, 0)^\top$ and $\mathbf{c}_2 = (0, 0, 0, 1)^\top$, which are linearly independent. Hence $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$ and any consistent system has a unique solution. Concretely, a right-hand side $\mathbf{b} = (b_1, b_2, b_3, b_4)^\top$ is in $\mathcal{R}(\mathbf{A})$ iff $b_1 = b_2 = b_3$; when this holds, the unique solution is $x_1 = b_1$ and $x_2 = b_4$.

2.5.5 Rank and nullity

The rank and nullity of $\mathbf{A} \in \mathbb{R}^{m \times n}$ are

$$\text{rank}(\mathbf{A}) = \dim \mathcal{R}(\mathbf{A}), \quad \text{nullity}(\mathbf{A}) = \dim \mathcal{N}(\mathbf{A}).$$

Always, $\text{rank}(\mathbf{A}) \leq \min\{n, m\}$. We say that $\mathbf{A}$ has full column rank if $\text{rank}(\mathbf{A}) = n$ and full row rank if $\text{rank}(\mathbf{A}) = m$.

Rank–nullity theorem: For every $\mathbf{A} \in \mathbb{R}^{m \times n}$,

$$\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = n$$

An immediate consequence of the rank-nullity theorem is that if $n > m$ (more variables than equations), then $\text{rank}(\mathbf{A}) \leq m$ so $\text{nullity}(\mathbf{A}) \geq n - m > 0$. Thus, whenever a solution exists, it cannot be unique. Conversely, if $m > n$ (more equations than variables), uniqueness (when consistent) is possible exactly when $\mathbf{A}$ has full column rank n , but solvability for *every* $\mathbf{b} \in \mathbb{R}^m$ is impossible since that would require $\text{rank } m > n$.

2.5.6 Square case and invertibility

When $\mathbf{A} \in \mathbb{R}^{n \times n}$, the following are equivalent for a square system:

- for every $\mathbf{b} \in \mathbb{R}^n$ there exists a *unique* solution of $\mathbf{A}\mathbf{x} = \mathbf{b}$
- $\mathbf{A}$ is invertible
- $\det(\mathbf{A}) \neq 0$
- $\text{rank}(\mathbf{A}) = n$
- $\dim \mathcal{N}(\mathbf{A}) = 0$

Each statement enforces both existence (full row rank) and uniqueness (full column rank), which coincide in the square case.

Now, we return to the problem of finding the solution of [Eq. 2.3.1](#). At first glance, it might seem straightforward to solve for $\mathbf{x}$ by simply computing the inverse of $\mathbf{A}$, since theoretically, if $\mathbf{A}$ is invertible, the solution is given by

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$$

However, in practice, computing $\mathbf{A}^{-1}$ directly is generally avoided for several important reasons: (1) computational cost, (2) numerical stability, and (3) memory requirements. For these reasons, numerical methods focus on solving linear systems without explicitly computing the inverse of $\mathbf{A}$. There are two primary classes of numerical methods for solving linear systems: **direct methods** and **iterative methods**. Each has its own advantages and is suitable for different types of problems.

2.6 Numerical Solution of Linear Systems: Direct Methods

Direct methods are named for their ability to (given no round-off errors) determine the exact solution to a linear system within a finite number of steps. In practical applications, however, finite precision limits these methods to producing only approximate solutions.

Direct methods take advantage of the fact that we can easily solve the system [Eq. 2.3.1](#) when $\mathbf{A}$ is upper triangular by using back substitution. Direct methods typically involve some variant of Gaussian elimination, which is a systematic procedure for transforming the coefficient matrix $\mathbf{A}$ into an upper triangular form in order to be able to solve the system by back substitution.

2.6.1 Back Substitution

Back substitution is a method for solving linear systems of equations that are in upper triangular form. The idea is to start with the last equation and solve for the last unknown, then use that result to solve for the second-to-last unknown, and so on (working backwards, hence the name). In mathematical terms, this is expressed as

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=i+1}^n a_{ij} x_j \right) \quad \text{for } i = n, n-1, \dots, 1$$

where a_{ij} are the elements of the upper triangular matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, b_i are the elements of the vector $\mathbf{b}$, and x_i are the unknowns we are solving for. Notice that the formula for x_k for some $k < n$ relies on the values of $x_{k+1}, x_{k+2}, \dots, x_n$, which is why we must work backwards.

Example: Solve the following upper triangular linear system:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 13 \\ 6 \end{bmatrix}$$

Solution: Let's work backwards naively to gain some intuition. The last equation is $6x_3 = 6$, so $x_3 = 6/6 = 1$. The second-to-last equation is $4x_2 + 5(1) = 13$, so $x_2 = (13 - 5)/4 = 2$. Finally, the first equation is $x_1 + 2(2) + 3(1) = 1$, so $x_1 = 1 - 4 - 3 = -6$. Therefore, the solution to the system is $x_1 = -6$, $x_2 = 2$, and $x_3 = 1$.

We can also just directly apply the formula for back substitution (note that the formula is the exact same as the naive/intuitive approach given above, just written in a way that generalizes to larger systems):

$$\begin{aligned} x_3 &= \frac{1}{6} (6 - 0) = 1 \\ x_2 &= \frac{1}{4} (13 - 5(1)) = 2 \\ x_1 &= \frac{1}{1} (1 - 2(2) - 3(1)) = -6 \end{aligned}$$

2.6.2 Gaussian Elimination

We've seen how easy it is to solve a linear system when the coefficient matrix is upper triangular, so how can we get $\mathbf{A}$ into this form? One way is via Gaussian elimination. This involves a series of elementary row operations (interchanging rows, multiplying a row by a scalar, and adding a multiple of one row to another) to eliminate the elements below the diagonal. The key linear algebraic fact that makes this possible is that elementary row operations do not change the solution to the linear system.

In general, the Gaussian elimination process begins by selecting the first pivot element a_{11} in the first row. If a_{11} is zero, a row swap is performed with a lower row that has a non-zero entry in the first column to avoid division by zero. Once the pivot is established, elementary row operations are applied to eliminate all entries below a_{11} in the first column. Specifically, for each row $k > 1$, the operation $\text{Row}_k \leftarrow \text{Row}_k - \lambda_{k1} \cdot \text{Row}_1$ is performed, where the multiplier $\lambda_{k1} = a_{k1}/a_{11}$.

After eliminating the first column below the pivot, the process moves to the next pivot a'_{22} in the second row and second column. Similar row operations are applied to eliminate all entries below a'_{22} in the second column, using multipliers $\lambda_{k2} = a'_{k2}/a'_{22}$ for $k > 2$. This procedure is repeated for each subsequent pivot until

the augmented matrix is in upper triangular form:

$$\left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ 0 & a'_{22} & \cdots & a'_{2n} & b'_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & a'_{nn} & b'_n \end{array} \right]$$

Once the matrix is upper triangular, we can use back substitution to solve for the unknowns.

Example: Use Gaussian elimination to bring the following system to upper triangular form:

$$\begin{bmatrix} 0 & 2 & 1 \\ 1 & 3 & 2 \\ 2 & 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} \implies \left[\begin{array}{ccc|c} 0 & 2 & 1 & 4 \\ 1 & 3 & 2 & 5 \\ 2 & 1 & 3 & 6 \end{array} \right]$$

Solution: The first pivot element is $a_{11} = 0$. Since $a_{11} = 0$, we perform a row swap with a lower row that has a non-zero entry in the first column. Here, $a_{21} = 1 \neq 0$, so we swap Row 1 (R_1) with Row 2 (R_2):

$$R_1 \leftrightarrow R_2$$

The augmented matrix becomes

$$\left[\begin{array}{ccc|c} 1 & 3 & 2 & 5 \\ 0 & 2 & 1 & 4 \\ 2 & 1 & 3 & 6 \end{array} \right]$$

The first pivot is now $a_{11} = 1$. We eliminate the entries below a_{11} in the first column using the following row operations:

$$\lambda_{31} = \frac{a_{31}}{a_{11}} = \frac{2}{1} = 2$$

$$R_3 \leftarrow R_3 - \lambda_{31} \cdot R_1 = R_3 - 2R_1$$

Performing the operation,

$$R_3 = [2 \quad 1 \quad 3 \mid 6] - 2 \cdot [1 \quad 3 \quad 2 \mid 5] = [0 \quad -5 \quad -1 \mid -4]$$

The augmented matrix is now

$$\left[\begin{array}{ccc|c} 1 & 3 & 2 & 5 \\ 0 & 2 & 1 & 4 \\ 0 & -5 & -1 & -4 \end{array} \right]$$

The second pivot element is $a'_{22} = 2$. Since $a'_{22} \neq 0$, no row swap is needed. We proceed to eliminate the entry below a'_{22} in the second column:

$$\lambda_{32} = \frac{a'_{32}}{a'_{22}} = \frac{-5}{2} R_3 \leftarrow R_3 - \lambda_{32} \cdot R_2 = R_3 - \left(\frac{-5}{2} \right) R_2 = R_3 + \frac{5}{2} R_2$$

Performing the operation,

$$R_3 = [0 \quad -5 \quad -1 \mid -4] + \frac{5}{2} \cdot [0 \quad 2 \quad 1 \mid 4] = [0 \quad 0 \quad 1.5 \mid 6]$$

The augmented matrix is now

$$\left[\begin{array}{ccc|c} 1 & 3 & 2 & 5 \\ 0 & 2 & 1 & 4 \\ 0 & 0 & 1.5 & 6 \end{array} \right]$$

which is in upper triangular form. We can now use back substitution to solve for the unknowns.

Hopefully the above example elucidates the fact that Gaussian elimination is a very intuitive process, even if the formalism can seem a bit confusing at first. It is a very powerful tool for solving linear systems, and it is the foundation of many direct methods for solving linear systems.

Additionally, to increase numerical stability, **partial pivoting** is often used in practice. This involves selecting the pivot element as the largest element in the column (in magnitude) to avoid division by very small numbers, which can lead to numerical instability.

Gaussian Elimination Applications Gaussian elimination can be used for more than just solving linear systems. For example, it can be used to compute the inverse of a matrix by augmenting the matrix with the identity matrix and performing Gaussian elimination to transform the matrix into the identity matrix:

$$[\mathbf{A} \mid \mathbf{I}] \xrightarrow{\text{Gauss. elim.}} [\mathbf{I} \mid \mathbf{A}^{-1}]$$

Another application is determining the rank of a matrix. After reducing $\mathbf{A}$ to its row-echelon form using Gaussian elimination, the rank is equal to the number of non-zero rows.

Moreover, for a square matrix $\mathbf{A}$, Gaussian elimination can transform $\mathbf{A}$ into an upper triangular matrix $\mathbf{U}$. The determinant of $\mathbf{A}$ is then the product of the diagonal entries of $\mathbf{U}$ adjusted by the number of row swaps performed:

$$\det(\mathbf{A}) = (-1)^{\text{swaps}} \prod_{i=1}^n u_{ii}$$

Computational Complexity of Gaussian Elimination and Back Substitution The computational complexity of Gaussian elimination for an $n \times n$ matrix is $O(n^3)$. We can understand this as follows: We have to perform one division, $n - i + 1$ multiplications, and $n - i + 1$ subtractions to form the j -th row at the i -th stage. There are $n - i$ rows in total. Therefore, at the i -th stage, we must perform $n - i$ divisions, $(n - i)(n - i + 1)$ multiplications, and $(n - i)(n - i + 1)$ subtractions. Summing these operations over all stages, we have to perform in total

$$\sum_{i=1}^{n-1} (n - i) = \frac{n(n - 1)}{2}$$

divisions and

$$\sum_{i=1}^{n-1} (n - i)(n - i + 1) = \frac{n(n^2 - 1)}{3}$$

multiplications and subtractions (each). This results in an overall computational complexity of $O(n^3)$ operations.

For back substitution, for each of the n variables, we perform one division and $n - i$ multiplications and subtractions to solve for x_i . Consequently, we perform n divisions and

$$\sum_{i=1}^n (n - i) = \frac{n(n - 1)}{2}$$

multiplications and subtractions (each). This results in an overall computational complexity of $O(n^2)$ operations.

Therefore, the overall computational complexity of solving a linear system with n equations and n unknowns using Gaussian elimination and back substitution is $O(n^3)$. This is a very important result to keep in mind when working with large linear systems, as the computational cost grows cubically with the size of the system.

2.6.3 Condition Number

Before looking at specific direct and iterative methods, we introduce the notion of the **condition number** of a matrix $\mathbf{A}$, denoted $\kappa(\mathbf{A})$, which helps us understand the sensitivity of a linear system to perturbations and

numerical errors. We hope that for our linear system $\mathbf{Ax} = \mathbf{b}$, small perturbations in $\mathbf{b}$ will lead to small changes in $\mathbf{x}$. Such a system is said to be well-conditioned. However, if small perturbations in $\mathbf{b}$ lead to large changes in $\mathbf{x}$, the system is said to be ill-conditioned. The meaning of “small” and “large” changes depending on the context (e.g., it could be relative or absolute). The condition number quantifies this sensitivity.

To define the condition number, we consider a linear system $\mathbf{Ax} = \mathbf{b}$ with an exact solution $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$. If we add a small perturbation (error) $\delta\mathbf{b}$ to $\mathbf{b}$, then the solution vector $\mathbf{x}$ will also change by some amount $\delta\mathbf{x}$ (which we think of as the error in $\mathbf{x}$ since the original perturbation is assumed to represent an error) so that the linear system remains satisfied:

$$\mathbf{A}(\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b} \implies \delta\mathbf{x} = \mathbf{A}^{-1}\delta\mathbf{b}$$

Taking the p -norm of both sides, we have

$$\|\delta\mathbf{x}\|_p = \|\mathbf{A}^{-1}\delta\mathbf{b}\|_p \leq \|\mathbf{A}^{-1}\|_p \|\delta\mathbf{b}\|_p$$

where the inequality follows from the submultiplicative property of matrix norms. We also have $\|\mathbf{b}\|_p = \|\mathbf{Ax}\|_p \leq \|\mathbf{A}\|_p \|\mathbf{x}\|_p$, hence $\|\mathbf{x}\|_p \geq \|\mathbf{b}\|_p / \|\mathbf{A}\|_p$. Dividing the two inequalities, we arrive at an expression for the relative error in the solution:

$$\frac{\|\delta\mathbf{x}\|_p}{\|\mathbf{x}\|_p} \leq \|\mathbf{A}\|_p \|\mathbf{A}^{-1}\|_p \frac{\|\delta\mathbf{b}\|_p}{\|\mathbf{b}\|_p}$$

This motivates the definition of the condition number of $\mathbf{A}$ as

$$\kappa_p(\mathbf{A}) = \|\mathbf{A}\|_p \|\mathbf{A}^{-1}\|_p$$

where we add a subscript p on κ to denote the norm used and to emphasize that its value depends on the choice of norm (but the qualitative behavior is the same across norms). If $\mathbf{A}$ is singular, $\kappa_p(\mathbf{A}) = \infty$. We note that clearly $\kappa_p(\mathbf{A}) \geq 1$ since $\|\mathbf{A}\|_p \|\mathbf{A}^{-1}\|_p \geq \|\mathbf{AA}^{-1}\|_p = \|\mathbf{I}\|_p = 1$. Moreover, a matrix with a large condition number (an “ill-conditioned” matrix) indicates that the linear system is more sensitive to perturbations and numerical errors, while a matrix with a small condition number (a “well-conditioned” matrix) indicates greater numerical robustness. As a practical rule of thumb for backward-stable solvers (e.g., Gaussian elimination with partial pivoting), the number of correct digits $\approx$ (working-precision digits) $-\log_{10} \kappa(\mathbf{A})$, and the relative forward error is typically $O(\kappa(\mathbf{A}) \varepsilon_{\text{mach}})$. A small residual $\|\mathbf{r}\| = \|\mathbf{b} - \mathbf{Ax}\|$ does not guarantee a small error when $\kappa(\mathbf{A})$ is large; roughly,

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \lesssim \kappa(\mathbf{A}) \frac{\|\mathbf{r}\|}{\|\mathbf{b}\|}.$$

Let’s take a look at the condition number for the 2-norm specifically. Here, the condition number is

$$\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2 \|\mathbf{A}^{-1}\|_2 = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})}$$

where $\sigma_{\max}(\mathbf{A})$ and $\sigma_{\min}(\mathbf{A})$ are the largest and smallest singular values of $\mathbf{A}$ (see Section 2.7.4), respectively. For $\mathbf{A} \succ 0$,² this is equivalent to

$$\kappa_2(\mathbf{A}) = \frac{\lambda_{\max}(\mathbf{A})}{\lambda_{\min}(\mathbf{A})}$$

where $\lambda_{\max}(\mathbf{A})$ and $\lambda_{\min}(\mathbf{A})$ are the largest and smallest eigenvalues of $\mathbf{A}$, respectively.

One way to mitigate the effects of ill-conditioning is *preconditioning*, which transforms the system into an equivalent one with better spectral properties; we discuss it together with the iterative methods at the end of this chapter.

Even simpler than preconditioning is **scaling** (often called equilibration), which rescales rows and/or columns to reduce element-wise dynamic range and improve pivoting. A common heuristic is to equalize row/column ∞ -norms. This helps numerics and pivoting but does not magically fix ill-conditioning rooted in the model; think nondimensionalization and consistent units (ChemE context: Pa vs bar, mol vs kmol).

²If $\mathbf{A}$ is normal ($\mathbf{AA}^* = \mathbf{A}^*\mathbf{A}$) over $\mathbb{C}$, then $\sigma_i(\mathbf{A}) = |\lambda_i(\mathbf{A})|$ and $\kappa_2(\mathbf{A}) = \max_i |\lambda_i(\mathbf{A})| / \min_i |\lambda_i(\mathbf{A})|$.

Example: Compute the 2-norm condition number of the matrix

$$\mathbf{A} = \begin{bmatrix} 10^{14} & 0 \\ 0 & 10^{-9} \end{bmatrix}$$

and investigate how row scaling affects it.

Solution: The condition number with respect to the 2-norm is

$$\kappa_2(\mathbf{A}) = \frac{\lambda_{\max}(\mathbf{A})}{\lambda_{\min}(\mathbf{A})} = \frac{10^{14}}{10^{-9}} = 10^{23}$$

which means that $\mathbf{A}$ is severely ill-conditioned. Such a high condition number implies that even tiny perturbations in $\mathbf{b}$ can lead to enormous errors in the computed solution $\mathbf{x}$.

To mitigate this issue, we can apply scaling by rescaling the rows of $\mathbf{A}$ to reduce the condition number. Specifically, we divide each row by the maximum absolute value in that row. For matrix $\mathbf{A}$, the scaling factors for the rows are 10^{14} and 10^{-9} , respectively. We define a diagonal scaling matrix $\mathbf{D}$ as

$$\mathbf{D} = \begin{bmatrix} 10^{-14} & 0 \\ 0 & 10^9 \end{bmatrix}$$

Applying the scaling, we obtain the scaled matrix $\mathbf{A}_{\text{scaled}}$:

$$\mathbf{A}_{\text{scaled}} = \mathbf{D}\mathbf{A} = \begin{bmatrix} 10^{-14} & 0 \\ 0 & 10^9 \end{bmatrix} \begin{bmatrix} 10^{14} & 0 \\ 0 & 10^{-9} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The left-scaling forms the equivalent system $\mathbf{D}\mathbf{A}\mathbf{x} = \mathbf{D}\mathbf{b}$ (same solution $\mathbf{x}$). Here, because $\mathbf{A}$ is diagonal, $\mathbf{D}\mathbf{A} = \mathbf{I}$. In general, equilibration reduces dynamic range and can reduce κ , but need not drive it to 1.

The condition number of the scaled matrix is now

$$\kappa_2(\mathbf{A}_{\text{scaled}}) = \frac{\lambda_{\max}(\mathbf{A}_{\text{scaled}})}{\lambda_{\min}(\mathbf{A}_{\text{scaled}})} = \frac{1}{1} = 1$$

which means that $\mathbf{A}_{\text{scaled}}$ is perfectly conditioned. This dramatic reduction shows how scaling can improve numerical behavior in favorable cases (in this case, $\mathbf{A}$ is diagonal, so $\mathbf{D} = \mathbf{A}^{-1}$ is an exact preconditioner, hence the perfect condition number). In general, scaling is often helpful for stability and pivoting, but it does not eliminate all ill-conditioning.

2.7 Eigenvalues and Eigenvectors

A linear transformation, represented by a matrix $\mathbf{A}$, acts on vectors, typically changing both their directions and magnitudes. However, for many square matrices, there exist special non-zero vectors called **eigenvectors** whose direction is unchanged by the transformation. When multiplied by the matrix, these vectors are simply scaled: stretched, shrunk, and/or flipped.

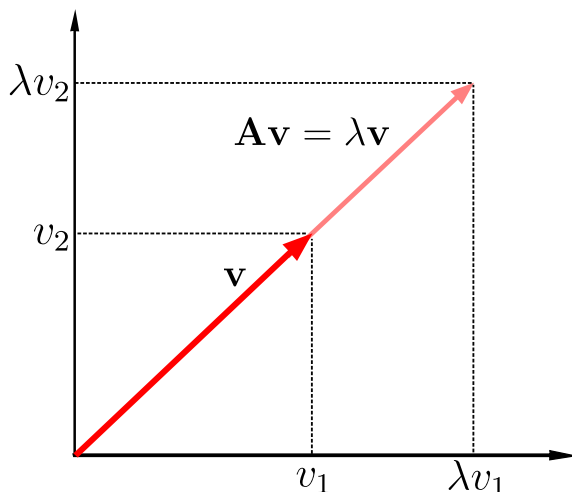


Figure 2.7.1: Visualization of the eigenvalue equation $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$, where $\mathbf{v} = (v_1, v_2)$.

Formally, for a matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, a non-zero vector $\mathbf{v}$ is an eigenvector if it satisfies the equation:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

The scalar λ is the corresponding **eigenvalue**, which represents the scaling factor. An $n \times n$ matrix has at most n distinct eigenvalues.

Warning: An eigenvector is not unique. If $\mathbf{v}$ is an eigenvector, then so is any scalar multiple $c\mathbf{v}$ for $c \neq 0$. They both correspond to the same eigenvalue and define the same invariant direction.

The concept is very important for understanding iterative methods. For example, look at the Jacobi iteration error propagation (which we will encounter shortly), $e_{t+1} = \mathbf{S}e_t$, which implies $e_t = \mathbf{S}^t e_0$. Eigenvalues and eigenvectors tell us what happens to a vector when it's multiplied by the same matrix repeatedly.

2.7.1 Finding Eigenvalues and Eigenvectors

To find eigenvalues, we rearrange the defining equation:

$$\mathbf{A}\mathbf{v} - \lambda\mathbf{v} = 0 \implies (\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = 0$$

For this equation to have a non-trivial solution for $\mathbf{v}$ (i.e., $\mathbf{v} \neq 0$), the matrix $(\mathbf{A} - \lambda\mathbf{I})$ must be singular. This means its determinant must be zero.

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0$$

This equation defines the **characteristic polynomial** of $\mathbf{A}$, an n -degree polynomial in λ . Its n roots (which may be real or complex and are counted with algebraic multiplicity) are the eigenvalues of $\mathbf{A}$.

Solving the characteristic polynomial is practical for small, textbook examples. However, this method is generally ill-conditioned and not recommended for numerical work with larger matrices; robust iterative algorithms (e.g., QR/Schur) are used in practice.

Once an eigenvalue λ_i is found, the corresponding eigenvector(s) $\mathbf{v}_i$ are found by solving the linear system $(\mathbf{A} - \lambda_i\mathbf{I})\mathbf{v}_i = 0$, which is equivalent to finding the null space of $(\mathbf{A} - \lambda_i\mathbf{I})$.

Example (finding eigenvalues and eigenvectors): Find the eigenvalues and eigenvectors of $\mathbf{A} = \begin{bmatrix} -2 & 1 \\ 1 & -2 \end{bmatrix}$.

Solution:

1. Find Eigenvalues: Solve $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$:

$$\det \begin{bmatrix} -2-\lambda & 1 \\ 1 & -2-\lambda \end{bmatrix} = (-2-\lambda)^2 - 1 = \lambda^2 + 4\lambda + 3 = 0$$

Factoring gives $(\lambda + 3)(\lambda + 1) = 0$. The eigenvalues are $\lambda_1 = -3$ and $\lambda_2 = -1$.

2. Find Eigenvectors: For $\lambda_1 = -3$, we solve $(\mathbf{A} - (-3)\mathbf{I})\mathbf{v}_1 = \mathbf{0}$:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies v_1 + v_2 = 0$$

An eigenvector is $\mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

For $\lambda_2 = -1$, we solve $(\mathbf{A} - (-1)\mathbf{I})\mathbf{v}_2 = \mathbf{0}$:

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies -v_1 + v_2 = 0$$

An eigenvector is $\mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

2.7.2 Multiplicity and Diagonalizability

The number of times an eigenvalue appears as a root of the characteristic polynomial is its **algebraic multiplicity**. The number of linearly independent eigenvectors for that eigenvalue is its **geometric multiplicity**. For any eigenvalue, $1 \leq \text{geometric multiplicity} \leq \text{algebraic multiplicity}$. A matrix is called **defective** if for some eigenvalue its geometric multiplicity is less than its algebraic multiplicity; equivalently, it does not have n linearly independent eigenvectors. A matrix with a “complete set” of eigenvectors (geometric = algebraic multiplicity for all eigenvalues) is **diagonalizable**.

Example: The matrices

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

both have the single eigenvalue $\lambda = 2$ with algebraic multiplicity 2. Find the geometric multiplicity of λ for each.

Solution: For $\mathbf{A}$, the eigenvector equation $(\mathbf{A} - 2\mathbf{I})\mathbf{v} = \mathbf{0}$ reads $v_2 = 0$, so the eigenvectors span only one direction: the geometric multiplicity is 1, and $\mathbf{A}$ is defective. For $\mathbf{B}$, every nonzero vector is an eigenvector: the geometric multiplicity is 2, and $\mathbf{B}$ is diagonalizable (it is already diagonal).

2.7.3 Eigendecomposition

If an $n \times n$ matrix $\mathbf{A}$ is diagonalizable, it has n linearly independent eigenvectors $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$. We can bundle these into a matrix $\mathbf{W} = [\mathbf{v}_1 | \dots | \mathbf{v}_n]$ and the corresponding eigenvalues into a diagonal matrix $\mathbf{\Lambda}$. The eigenvalue equation for all eigenvectors can be written compactly as:

$$\mathbf{A}\mathbf{W} = \mathbf{W}\mathbf{\Lambda}$$

This leads to the **eigendecomposition** of $\mathbf{A}$:

$$\mathbf{A} = \mathbf{W}\mathbf{\Lambda}\mathbf{W}^{-1} \quad \text{and} \quad \mathbf{\Lambda} = \mathbf{W}^{-1}\mathbf{A}\mathbf{W}$$

This decomposition reveals the fundamental behavior of $\mathbf{A}$ as a simple scaling along the eigenvector axes.

Properties and Special Cases

For a matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ with eigenvalues $\lambda_1, \dots, \lambda_n$:

- The determinant is the product of its eigenvalues: $\det(\mathbf{A}) = \prod_{i=1}^n \lambda_i$.
- The trace is the sum of its eigenvalues: $\text{tr}(\mathbf{A}) = \sum_{i=1}^n \lambda_i$.
- The eigenvalues of a triangular matrix are its diagonal entries.

If $\mathbf{A}$ is a real symmetric matrix ($\mathbf{A} = \mathbf{A}^T$):

- It is always diagonalizable and has a complete set of orthonormal eigenvectors.
- All its eigenvalues are real.
- Its eigendecomposition is $\mathbf{A} = \mathbf{W}\mathbf{\Lambda}\mathbf{W}^T$, where $\mathbf{W}$ is an orthogonal matrix ($\mathbf{W}^{-1} = \mathbf{W}^T$).
- If $\mathbf{A}$ is positive definite, all its eigenvalues are positive ($\lambda_i > 0$).

2.7.4 Singular Values

Eigenvalues are defined only for square matrices. A closely related concept that applies to any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the **singular values**, defined as

$$\sigma_i(\mathbf{A}) = \sqrt{\lambda_i(\mathbf{A}^T \mathbf{A})}, \quad i = 1, 2, \dots, n,$$

where $\lambda_i(\mathbf{A}^T \mathbf{A})$ denotes the i -th eigenvalue of $\mathbf{A}^T \mathbf{A}$. Since $\mathbf{A}^T \mathbf{A}$ is symmetric and positive semi-definite, its eigenvalues are real and non-negative, so the singular values are well defined. They are conventionally ordered as $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$, and we write $\sigma_{\max}(\mathbf{A}) = \sigma_1$ and $\sigma_{\min}(\mathbf{A}) = \sigma_n$. Geometrically, the singular values measure how much the multiplication with $\mathbf{A}$ can stretch a vector: $\sigma_{\max}(\mathbf{A})$ and $\sigma_{\min}(\mathbf{A})$ are the largest and smallest factors by which the length of a vector can be scaled. This is why the singular values appear in the matrix 2-norm, $\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A})$, and in the 2-norm condition number, $\kappa_2(\mathbf{A}) = \sigma_{\max}(\mathbf{A})/\sigma_{\min}(\mathbf{A})$.

More generally, every matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ of rank r admits an economy-sized **singular value decomposition** (SVD),

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T, \quad \mathbf{U} \in \mathbb{R}^{m \times r}, \quad \mathbf{\Sigma} \in \mathbb{R}^{r \times r}, \quad \mathbf{V} \in \mathbb{R}^{n \times r},$$

where the columns of $\mathbf{U}$ and $\mathbf{V}$ are orthonormal ($\mathbf{U}^T \mathbf{U} = \mathbf{V}^T \mathbf{V} = \mathbf{I}$) and are called the left and right *singular vectors*, and $\mathbf{\Sigma}$ carries the singular values $\sigma_1 \geq \dots \geq \sigma_r > 0$ on its diagonal. The decomposition can be computed through eigendecompositions: the columns of $\mathbf{V}$ are the eigenvectors of $\mathbf{A}^T \mathbf{A}$, the columns of $\mathbf{U}$ are the eigenvectors of $\mathbf{A}\mathbf{A}^T$, and $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^T \mathbf{A})}$ (in MATLAB, `[U, S, V] = svd(A, 'econ')`). Keeping only the $t < r$ largest singular values gives the *truncated SVD*, $\mathbf{A} \approx \mathbf{U}_t \mathbf{\Sigma}_t \mathbf{V}_t^T$, where $\mathbf{U}_t$ and $\mathbf{V}_t$ collect the first t columns of $\mathbf{U}$ and $\mathbf{V}$; this is the best rank- t approximation of $\mathbf{A}$, and the storage drops from mn numbers to $(m+n+1)t$. The SVD is used across numerical practice: principal component analysis, low-rank approximation and data compression, and least-squares problems via the pseudoinverse.

2.8 Numerical Solution of Linear Systems: Iterative Methods

Iterative methods attempt to solve linear systems ($\mathbf{A}\mathbf{x} = \mathbf{b}$) by generating a sequence of improving approximate solutions. They do not require Gaussian elimination and generally scale better for large systems. However, even with infinite precision, iterative methods can only obtain an approximate solution in a finite number

of steps³. In this chapter, we cover Jacobi and Gauss-Seidel iteration, two of the most common iterative methods. The conjugate gradient method, and the broader class of *Krylov subspace methods* to which it belongs, will be covered in the part of the course on partial differential equations.⁴

2.8.1 Jacobi Iteration

Jacobi iteration is an iterative method for solving linear systems. Let's build some intuition for this method before giving the matrix form of the update formula. Suppose we have the linear system

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n \end{aligned}$$

and some initial approximation/guess for the solution, $x_1^{(0)}, x_2^{(0)}, \dots, x_n^{(0)}$. How can we improve this guess? Well, rewriting the i -th equation in terms of the other variables, we have

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij}x_j \right)$$

If this equation is satisfied, then the i -th equation of the linear system is also satisfied. So, to improve our guess for x_i , we can use the current values of the other variables to estimate the value of x_i . After doing this for each variable (which we can do in parallel—we only update the values of the variables *after* we have computed the new values for all of them), we have a new approximation for the solution. We can then repeat this process until the solution converges (assuming that it does—more on this later). This is the main idea behind Jacobi iteration.

Warning: One might be tempted to think that after one iteration, we have the exact solution for x_i (after all, we solve each i -th equation exactly, right?). You should make sure you completely understand why this is not the case before moving on. The Jacobi method is an iterative method, meaning that we must repeat this process multiple times to get a better approximation for the solution.

Matrix Formulation Now, let's put this in matrix form, which is much more convenient for computer implementation. We decompose the coefficient matrix $\mathbf{A}$ into its diagonal component $\mathbf{D}$ and the remainder $\mathbf{R}$, such that $\mathbf{A} = \mathbf{D} + \mathbf{R}$. Here, $\mathbf{D}$ consists of the diagonal elements of $\mathbf{A}$, and $\mathbf{R}$ contains the off-diagonal elements. Thus, the linear system can be rewritten as

$$(\mathbf{D} + \mathbf{R})\mathbf{x} = \mathbf{b} \implies \mathbf{D}\mathbf{x} = -\mathbf{R}\mathbf{x} + \mathbf{b}$$

This defines the iterative map

$$\mathbf{D}\mathbf{x}^{(k+1)} = -\mathbf{R}\mathbf{x}^{(k)} + \mathbf{b}$$

Assuming that $\mathbf{D}$ is nonsingular, this can be equivalently written as

$$\mathbf{x}^{(k+1)} = -\mathbf{D}^{-1}(\mathbf{R}\mathbf{x}^{(k)} - \mathbf{b})$$

This formulation allows us to compute the next approximation $\mathbf{x}^{(k+1)}$ based on the current approximation $\mathbf{x}^{(k)}$. Note how this is the *exact* same formula as the one we derived earlier for the i -th variable in the linear system, just written in matrix form so that we can update all variables simultaneously!

³Actually, some iterative methods, like CG on SPD, are finite-termination in exact arithmetic, but we aren't going to cover that in this course.

⁴A good resource for this is Prof. Jonathan Shewchuk's guide *An Introduction to the Conjugate Gradient Method Without the Agonizing Pain*.³

As a quick note on notation, many sources commonly split the matrix $\mathbf{A}$ into the sum of a lower triangular matrix $\mathbf{L}$, a diagonal matrix $\mathbf{D}$, and an upper triangular matrix $\mathbf{U}$ (i.e., $\mathbf{A} = \mathbf{L} + \mathbf{D} + \mathbf{U}$). The Jacobi method is then written as

$$\mathbf{x}^{(k+1)} = -\mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)} + \mathbf{D}^{-1}\mathbf{b}$$

This is mathematically equivalent to the formulation we have presented here; don't be confused if you see this alternate notation.

Computational Complexity Each iteration of the Jacobi method involves diagonal matrix inversion, matrix-vector multiplications, and vector updates. Specifically, computing $\mathbf{R}\mathbf{x}^{(k)}$ requires $O(n^2)$ operations for an $n \times n$ matrix $\mathbf{A}$, and the subsequent vector operations also scale quadratically. Inverting a diagonal matrix is trivial and requires only n divisions. Therefore, the computational complexity of each Jacobi iteration is $O(n^2)$.

2.8.2 Convergence of Jacobi Iteration

We have seen that we can generate a sequence of improving approximations $\{\mathbf{x}^{(k)}\}$ to the solution of a linear system using the Jacobi method. But how do we know that this sequence converges at all, and if it converges, that it converges to the true solution $\mathbf{x}^*$? Convergence is not generally guaranteed for iterative methods, let alone convergence to the true solution.

Convergence to the Correct Solution We claim that if the Jacobi sequence $\{\mathbf{x}^{(k)}\}$ converges, it indeed converges to the correct solution. To see this, we take the limit on both sides of the iterative equation:

$$\begin{aligned} \lim_{k \rightarrow \infty} \mathbf{x}^{(k+1)} &= \lim_{k \rightarrow \infty} -\mathbf{D}^{-1}(\mathbf{R}\mathbf{x}^{(k)} - \mathbf{b}) \\ \mathbf{x}^* &= -\mathbf{D}^{-1}(\mathbf{R}\mathbf{x}^* - \mathbf{b}) \end{aligned}$$

Rearranging terms gives

$$\mathbf{D}\mathbf{x}^* = -\mathbf{R}\mathbf{x}^* + \mathbf{b} \implies \mathbf{A}\mathbf{x}^* = \mathbf{b}$$

Thus, the limit $\mathbf{x}^*$ satisfies the original system, so convergence implies correctness.

Conditions for Convergence So if we converge, we converge to the correct solution. But how do we know if we converge at all? Let's derive the conditions for convergence. We consider the exact solution satisfying

$$\mathbf{A}\mathbf{x}^* = \mathbf{b} \iff \mathbf{x}^* = -\mathbf{D}^{-1}(\mathbf{R}\mathbf{x}^* - \mathbf{b})$$

which means that the error at the $(k+1)$ -th iteration is given by (note: the symbol $\stackrel{\text{def}}{=}$ means "defined as")

$$\begin{aligned} \mathbf{e}^{(k+1)} &\stackrel{\text{def}}{=} \mathbf{x}^{(k+1)} - \mathbf{x}^* = -\mathbf{D}^{-1}\mathbf{R}(\mathbf{x}^{(k)} - \mathbf{x}^*) \\ &= -\mathbf{D}^{-1}\mathbf{R}\mathbf{e}^{(k)} \end{aligned}$$

Notice how this equation is set up; the error at the $(k+1)$ -th iteration is a scaled version of the error at the preceding iteration. So if our initial guess is $\mathbf{x}^{(0)}$, we can keep applying this formula up to the i -th iteration to get

$$\mathbf{e}^{(k)} = (-\mathbf{D}^{-1}\mathbf{R})^k \mathbf{e}^{(0)}$$

This is a very powerful result. To see why, let's use the eigendecomposition⁵ of $-\mathbf{D}^{-1}\mathbf{R}$ to write

$$\begin{aligned} \mathbf{e}^{(k)} &= (-\mathbf{D}^{-1}\mathbf{R})^k \mathbf{e}^{(0)} \\ &= (\mathbf{W}\mathbf{\Lambda}\mathbf{W}^{-1})^k \mathbf{e}^{(0)} && \text{(eigendecomposition)} \\ &= (\mathbf{W}\mathbf{\Lambda}\mathbf{W}^{-1})(\mathbf{W}\mathbf{\Lambda}\mathbf{W}^{-1}) \cdots (\mathbf{W}\mathbf{\Lambda}\mathbf{W}^{-1}) \mathbf{e}^{(0)} && \text{(expand power)} \\ &= \mathbf{W}\mathbf{\Lambda}(\mathbf{W}^{-1}\mathbf{W}) \cdots \mathbf{\Lambda}(\mathbf{W}^{-1}\mathbf{W})\mathbf{\Lambda}\mathbf{W}^{-1}\mathbf{e}^{(0)} && \text{(exploit associativity)} \\ &= \mathbf{W}\mathbf{\Lambda}(\mathbf{I}) \cdots \mathbf{\Lambda}(\mathbf{I})\mathbf{\Lambda}\mathbf{W}^{-1}\mathbf{e}^{(0)} && \text{(definition of inverse)} \\ &= \mathbf{W}\mathbf{\Lambda}^k\mathbf{W}^{-1}\mathbf{e}^{(0)} \end{aligned}$$

⁵If the matrix is not diagonalizable, analogous bounds via Jordan form or induced norms still give that $\rho < 1$ is necessary and sufficient for linear convergence.

Thus, if any of the eigenvalues of $-\mathbf{D}^{-1}\mathbf{R}$ have magnitude greater than 1, the error will grow with i , and the Jacobi method will not converge. On the other hand, if all the eigenvalues have magnitude less than 1, the error will decay with i , and the Jacobi method will converge. And if any eigenvalues have magnitude equal to 1, the error will either remain constant or oscillate. In linear algebraic terms, this is captured by the notion of the **spectral radius** ρ of $-\mathbf{D}^{-1}\mathbf{R}$, which is the maximum magnitude of the eigenvalues of $-\mathbf{D}^{-1}\mathbf{R}$. Indeed, since we also know that convergence implies correctness, we have that

$$\rho(-\mathbf{D}^{-1}\mathbf{R}) < 1 \implies \|\mathbf{x}^{(k)} - \mathbf{x}^*\|_p \rightarrow 0 \quad \text{as } k \rightarrow \infty$$

or, in words, the Jacobi method converges to the correct solution if the spectral radius of $-\mathbf{D}^{-1}\mathbf{R}$ is less than 1.

The most natural question to ask is when the spectral radius of $-\mathbf{D}^{-1}\mathbf{R}$ is less than 1. The answer is that this is true if $\mathbf{A}$ (recall that $\mathbf{A} = \mathbf{D} + \mathbf{R}$) is diagonally dominant. A matrix is diagonally dominant if for every row i , the diagonal element a_{ii} is greater in magnitude than the sum of the magnitudes of the off-diagonal elements in that row. In mathematical terms, this is written as

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

Rate of Convergence The speed at which the Jacobi iteration converges to the solution $\mathbf{x}^*$ is characterized by the spectral radius $\alpha = \rho(-\mathbf{D}^{-1}\mathbf{R})$. The error after i iterations can be approximated as

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_p \approx O(\alpha^k) \|\mathbf{x}^{(0)} - \mathbf{x}^*\|_p$$

Here, α is referred to as the **rate of convergence**. To achieve a desired accuracy ϵ , the number of iterations required satisfies

$$\frac{\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_p}{\|\mathbf{x}^{(0)} - \mathbf{x}^*\|_p} \leq \epsilon \implies i \geq \frac{\log(\epsilon)}{\log(\alpha)}$$

A smaller α indicates faster convergence. However, the Jacobi method can be relatively slow compared to other iterative methods like Gauss-Seidel or successive over-relaxation (SOR, not covered in this course), especially when α is close to 1.

2.8.3 Gauss-Seidel Iteration

Gauss-Seidel iteration is another iterative method for solving linear systems, similar to the Jacobi method but often exhibiting faster convergence. Like Jacobi iteration, it builds a sequence of approximations to the solution, but it uses the most recently updated values within each iteration to enhance convergence speed.

Consider the same linear system as before:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n \end{aligned}$$

and an initial guess for the solution, $x_1^{(0)}, x_2^{(0)}, \dots, x_n^{(0)}$. To improve this guess, Gauss-Seidel iteration updates each variable sequentially, immediately using the latest available values. Specifically, the update rule for the i -th variable is

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)} \right)$$

for $i = 1, 2, \dots, n$, where k denotes the current iteration. This means that when computing $x_i^{(k+1)}$, the most recent values $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ are already updated, whereas $x_{i+1}^{(k)}, \dots, x_n^{(k)}$ remain from the previous iteration. You should convince yourself that this update rule makes sense and compare it to the Jacobi update rule before proceeding.

Warning: Unlike the Jacobi method, Gauss-Seidel updates variables sequentially and relies on the latest available estimates within the same iteration. This sequential dependency can lead to faster convergence but may complicate parallel implementations.

Matrix Formulation To express Gauss-Seidel iteration in matrix form, we decompose the coefficient matrix $\mathbf{A}$ into its lower triangular component $\mathbf{L}$, diagonal component $\mathbf{D}$, and upper triangular component $\mathbf{U}$, such that

$$\mathbf{A} = \mathbf{L} + \mathbf{D} + \mathbf{U}$$

where $\mathbf{L}$ contains the strictly lower triangular elements of $\mathbf{A}$, $\mathbf{D}$ is the diagonal matrix of $\mathbf{A}$, and $\mathbf{U}$ contains the strictly upper triangular elements of $\mathbf{A}$. The linear system can then be rewritten as

$$(\mathbf{D} + \mathbf{L})\mathbf{x} = -\mathbf{U}\mathbf{x} + \mathbf{b}$$

This defines the iterative map for Gauss-Seidel:

$$(\mathbf{D} + \mathbf{L})\mathbf{x}^{(k+1)} = -\mathbf{U}\mathbf{x}^{(k)} + \mathbf{b}$$

Assuming $\mathbf{D} + \mathbf{L}$ is nonsingular, we can solve for $\mathbf{x}^{(k+1)}$ to get

$$\mathbf{x}^{(k+1)} = -(\mathbf{D} + \mathbf{L})^{-1}\mathbf{U}\mathbf{x}^{(k)} + (\mathbf{D} + \mathbf{L})^{-1}\mathbf{b}$$

This formulation allows for sequential updates of the solution vector, utilizing the latest available estimates within each iteration. This is exactly the same as the update rule we derived earlier, just written in matrix form for convenience.

Computational Complexity Each Gauss-Seidel iteration involves solving a lower triangular system due to the $\mathbf{L} + \mathbf{D}$ term. Luckily, we can do this with forward substitution with $O(n^2)$ operations, so the computational complexity remains similar to Jacobi. That is, it is $O(n^2)$ operations per iteration for an $n \times n$ matrix $\mathbf{A}$.

2.8.4 Convergence of Gauss-Seidel Iteration

As with the Jacobi method, the convergence of Gauss-Seidel iteration is not universally guaranteed. However, under certain conditions, convergence can be assured, and when it does converge, it does so to the correct solution.

Convergence to the Correct Solution Assuming the Gauss-Seidel sequence $\{\mathbf{x}^{(k)}\}$ converges to a limit $\mathbf{x}^*$, we can take the limit on both sides of the iterative equation:

$$\begin{aligned} \lim_{k \rightarrow \infty} (\mathbf{D} + \mathbf{L})\mathbf{x}^{(k+1)} &= \lim_{k \rightarrow \infty} (-\mathbf{U}\mathbf{x}^{(k)} + \mathbf{b}) \\ (\mathbf{D} + \mathbf{L})\mathbf{x}^* &= -\mathbf{U}\mathbf{x}^* + \mathbf{b} \\ \mathbf{A}\mathbf{x}^* &= \mathbf{b} \end{aligned}$$

Thus, the limit $\mathbf{x}^*$ satisfies the original linear system, meaning that convergence implies correctness.

Conditions for Convergence To determine when the Gauss-Seidel method converges, we analyze the error propagation similarly to Jacobi iteration. Let $\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^*$ denote the error at the k -th iteration. The iterative relation for the error becomes

$$\begin{aligned} (\mathbf{D} + \mathbf{L})\mathbf{e}^{(k+1)} &= -\mathbf{U}\mathbf{e}^{(k)} \\ \mathbf{e}^{(k+1)} &= -(\mathbf{D} + \mathbf{L})^{-1}\mathbf{U}\mathbf{e}^{(k)} \end{aligned}$$

Letting $\mathbf{G} = -(\mathbf{D} + \mathbf{L})^{-1}\mathbf{U}$, we have

$$\mathbf{e}^{(k+1)} = \mathbf{G}\mathbf{e}^{(k)}$$

By iterating, the error after k iterations is

$$\mathbf{e}^{(k)} = \mathbf{G}^k \mathbf{e}^{(0)}$$

Using the eigendecomposition of $\mathbf{G}$, similar to the Jacobi method, we find that the spectral radius $\rho(\mathbf{G})$ governs convergence:

$$\rho(\mathbf{G}) < 1 \implies \|\mathbf{x}^{(k)} - \mathbf{x}^*\|_p \rightarrow 0 \quad \text{as } k \rightarrow \infty$$

When is $\rho(\mathbf{G}) < 1$? If the matrix $\mathbf{A}$ is strictly diagonally dominant, i.e.,

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \forall i$$

then the spectral radius $\rho(\mathbf{G}) < 1$, ensuring convergence of the Gauss-Seidel method.

Diagonal dominance is only a *sufficient* condition for convergence, not a necessary one. For example,

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$

is not diagonally dominant (in the first row, $1 < 2$), yet Jacobi iteration converges: the iteration matrix is $\begin{bmatrix} 0 & -2 \\ 0 & 0 \end{bmatrix}$, whose spectral radius is 0. Additionally, if $\mathbf{A}$ is symmetric positive definite, convergence is also guaranteed.

Rate of Convergence The rate at which Gauss-Seidel iteration converges is determined by the spectral radius $\alpha = \rho(\mathbf{G})$. The error after k iterations can be approximated by

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_p \approx O(\alpha^k) \|\mathbf{x}^{(0)} - \mathbf{x}^*\|_p$$

Since $\alpha < 1$ for convergent systems, a smaller α results in faster convergence. Often, it is empirically true that Gauss-Seidel converges faster than Jacobi because it effectively incorporates more up-to-date information within each iteration. However, like Jacobi, the convergence rate depends on the properties of $\mathbf{A}$, and for systems with a spectral radius close to 1, convergence can still be slow.

Comparison with Jacobi Iteration While both methods are simple and easy to implement, Gauss-Seidel typically outperforms Jacobi in terms of convergence speed due to its sequential update strategy. However, this comes at the cost of reduced parallelizability, as each update depends on the results of the previous ones within the same iteration. In contrast, Jacobi's parallel updates make it more suitable for parallel computing environments despite its slower convergence.

In practice, Gauss-Seidel is preferred over Jacobi when faster convergence is desired and when the system structure permits sequential updates without significant computational overhead. For large-scale systems where parallel computation is essential, Jacobi may still be advantageous despite its slower convergence rate.

2.8.5 Condition Number and Convergence Rate

To understand the impact of the condition number on the convergence rate of iterative methods, we consider the Richardson method, an iterative method defined by the update rule

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \omega(\mathbf{b} - \mathbf{A}\mathbf{x}^{(k)})$$

where ω is a relaxation parameter. The choice of ω strongly affects the convergence behavior of the method. Assuming that $\mathbf{A}$ is symmetric positive definite, the optimal convergence rate for the Richardson method is achieved with the choice

$$\omega_{\text{opt}} = \frac{2}{\lambda_{\min}(\mathbf{A}) + \lambda_{\max}(\mathbf{A})}$$

With this optimal ω , the spectral radius of the iteration matrix $\mathbf{I} - \omega_{\text{opt}}\mathbf{A}$ becomes

$$\rho(\mathbf{I} - \omega_{\text{opt}}\mathbf{A}) = \frac{\kappa(\mathbf{A}) - 1}{\kappa(\mathbf{A}) + 1}$$

This expression shows that the spectral radius—and consequently the convergence rate—is directly related to the condition number $\kappa(\mathbf{A})$. Specifically, a smaller $\kappa(\mathbf{A})$ leads to a smaller spectral radius, resulting in faster convergence of the iterative method. Conversely, a large condition number implies that the spectral radius is closer to one, which can significantly slow down convergence (Figure 2.8.1).

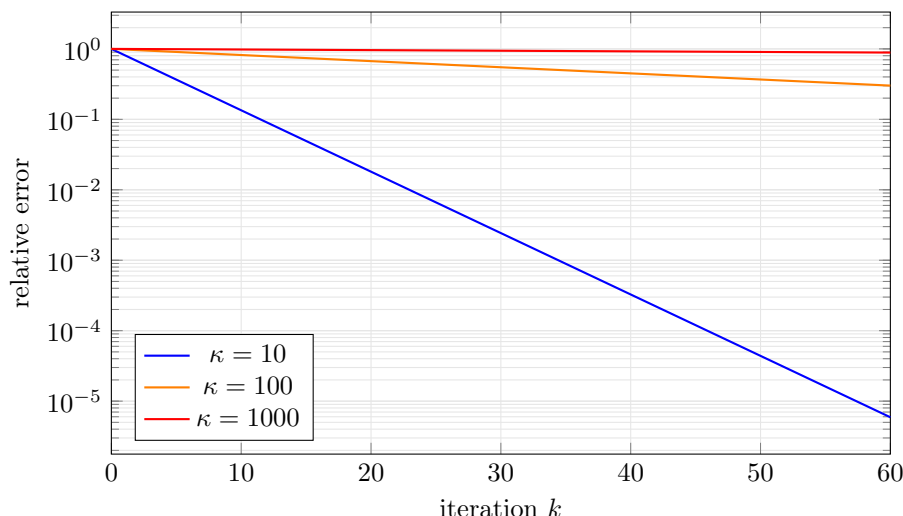


Figure 2.8.1: Convergence of the Richardson method with the optimal ω on diagonal test matrices with eigenvalues spread evenly in $[1, \kappa]$, for three condition numbers. The initial error is taken along the slowest eigendirection (the worst case), so the error contracts by exactly $\rho = (\kappa - 1)/(\kappa + 1)$ per iteration: the lines are straight on the logarithmic axis and become flatter as κ grows.

This relationship (not the literal equation, but the idea we just talked about) between the condition number and the convergence rate is not unique to the Richardson method but holds generally for a wide class of iterative methods, including Jacobi and Gauss-Seidel iterations. Therefore, we should make sure that the coefficient matrix $\mathbf{A}$ is well-conditioned in order to achieve efficiency and reliability of these iterative solvers. If $\kappa(\mathbf{A})$ is large, it may be beneficial to precondition the system to reduce the condition number and improve the convergence rate of the iterative method.

Preconditioning When solving linear systems numerically, we often encounter ill-conditioned matrices. One way to mitigate the effects of ill-conditioning is through **preconditioning**. Preconditioning transforms $\mathbf{Ax} = \mathbf{b}$ into a system with more favorable spectral properties. Common forms are left $(\mathbf{P}^{-1}\mathbf{A})\mathbf{x} = \mathbf{P}^{-1}\mathbf{b}$, right $\mathbf{A}(\mathbf{P}^{-1}\mathbf{y}) = \mathbf{b}$ with $\mathbf{x} = \mathbf{P}^{-1}\mathbf{y}$, and symmetric $\mathbf{P}^{-1/2}\mathbf{A}\mathbf{P}^{-1/2}\mathbf{z} = \mathbf{P}^{-1/2}\mathbf{b}$ with $\mathbf{x} = \mathbf{P}^{-1/2}\mathbf{z}$. Examples include Jacobi (diagonal), incomplete LU/Cholesky (ILU/IC), and algebraic multigrid (AMG). The goal is to cluster eigenvalues and reduce the effective condition number to accelerate and stabilize iterative methods.

Warning: Preconditioning/scaling improves the matrix the algorithm sees (e.g., reduces $\kappa(\mathbf{P}^{-1}\mathbf{A})$), which speeds and stabilizes iterations, but it *doesn't change the intrinsic sensitivity* of the original mapping $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ to perturbations in $\mathbf{b}$. So it helps the solver's numerics, not the problem's conditioning.

The Jacobi, Gauss-Seidel, and Richardson iterations presented here are the simplest members of a much larger family. In modern practice, **Krylov subspace methods**, such as the conjugate gradient (CG) method for symmetric positive definite systems and GMRES for general systems, are the workhorses of large-scale iterative linear algebra; they build increasingly accurate solutions from repeated matrix-vector products and combine naturally with preconditioning. We will cover them in the part of the course on the numerical solution of partial differential equations, where the linear systems are large, sparse, and structured—iterative methods at their best.

Warning: While a lower condition number generally promotes faster convergence, it is important to recognize that not all matrices can be easily preconditioned to achieve a desirable condition number. Additionally, some iterative methods may exhibit different sensitivities to the condition number, so the choice of method should consider both the properties of $\mathbf{A}$ and the specific requirements of the problem at hand.

2.9 Exercises

Exercise 2.1

True or False? Briefly explain your answer; if the statement is false, you may simply provide a counterexample.

- The product of two complex numbers is never a real number unless both of them are real.
- If both $\mathbf{AB}$ and $\mathbf{BA}$ are defined, then $\mathbf{A}$ and $\mathbf{B}$ must be square matrices.
- If $\mathbf{A}$ and $\mathbf{B}$ are diagonal matrices of the same dimensions, then $\mathbf{AB} = \mathbf{BA}$.
- For $\mathbf{u} \in \mathbb{R}^m$ and $\mathbf{v} \in \mathbb{R}^n$ with $m \neq n$, the outer product $\mathbf{uv}^\top$ is not defined.
- There exist matrices $\mathbf{A}$ and $\mathbf{B}$ for which $(\mathbf{AB})^\top = \mathbf{A}^\top \mathbf{B}^\top$.
- If $\text{tr}(\mathbf{AB}) = 0$, then $\text{tr}(\mathbf{A}) = 0$ or $\text{tr}(\mathbf{B}) = 0$.
- If $\|\mathbf{v}\|_1 \leq 1$, then $\|\mathbf{v}\|_2 \leq 1$; but the converse fails for some $\mathbf{v} \in \mathbb{R}^n$ with $n \geq 2$.
- Suppose $\|\mathbf{A}\|_\infty = 0.5$. Then $\|\mathbf{Ax}\|_\infty$ can be made greater than $\|\mathbf{x}\|_\infty$ by choosing a sufficiently large $\mathbf{x}$.
- A linear system with more unknowns than equations has either no solution or infinitely many solutions—exactly one solution is impossible.
- $\det(2\mathbf{A}) = 2 \det(\mathbf{A})$ for every $\mathbf{A} \in \mathbb{R}^{n \times n}$.
- If $\det(\mathbf{A}) = 0$, then $\mathbf{Ax} = \mathbf{b}$ always has no solution.
- The set of unit vectors $\{\mathbf{v} \in \mathbb{R}^n : \|\mathbf{v}\|_2 = 1\}$ is a subspace of $\mathbb{R}^n$.
- For $\mathbf{b} \neq \mathbf{0}$, the set of solutions of $\mathbf{Ax} = \mathbf{b}$ is a subspace of $\mathbb{R}^n$.
- A subspace of a subspace of $\mathbb{R}^n$ is itself a subspace of $\mathbb{R}^n$.
- Three vectors in $\mathbb{R}^2$ always span $\mathbb{R}^2$.
- Three vectors in $\mathbb{R}^2$ can be linearly independent.
- Any set of mutually orthogonal vectors is linearly independent.
- If the columns of a square matrix are orthonormal, then its rows are orthonormal as well.
- For any (possibly rectangular) matrix $\mathbf{A}$: if $\mathbf{Ax} = \mathbf{0}$ has only the trivial solution $\mathbf{x} = \mathbf{0}$, then $\mathbf{Ax} = \mathbf{b}$ has a solution for every $\mathbf{b}$.
- Let $\mathbf{A}$ be square. If $\mathbf{Ax} = \mathbf{b}$ has a solution for a particular $\mathbf{b}$, then $\mathbf{A}^\top \mathbf{x} = \mathbf{b}$ also has a solution.
- Let $\mathbf{A}$ be square. If $\mathbf{Ax} = \mathbf{b}$ has a solution for *every* $\mathbf{b}$, then $\mathbf{A}^\top \mathbf{x} = \mathbf{c}$ has a solution for every $\mathbf{c}$.
- $\text{rank}(\mathbf{A}^\top \mathbf{A}) = \text{rank}(\mathbf{A})$ for every $\mathbf{A} \in \mathbb{R}^{m \times n}$.
- Gaussian elimination costs $O(n^3)$, so solving $\mathbf{AX} = \mathbf{B}$ with k right-hand sides costs $O(n^3k)$.
- Let $\hat{\mathbf{x}}$ be an approximate solution of $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ is invertible, $\mathbf{b} \neq \mathbf{0}$, and $\mathbf{x}^* = \mathbf{A}^{-1}\mathbf{b}$. If the relative residual satisfies $\|\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}\|/\|\mathbf{b}\| \leq 10^{-10}$, then the relative error satisfies $\|\hat{\mathbf{x}} - \mathbf{x}^*\|/\|\mathbf{x}^*\| \leq 10^{-10}$.
- Eigenvectors corresponding to distinct eigenvalues are linearly independent; consequently, a matrix with n distinct eigenvalues is diagonalizable.

- (z) For every square matrix, the singular values are equal to the absolute values of the eigenvalues.
- (aa) If $\mathbf{A}$ is strictly diagonally dominant, Jacobi iteration converges regardless of the initial guess.

Exercise 2.2

Prove the following properties stated in this chapter.

- (a) $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$, whenever the product is defined.
- (b) $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$, for $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times m}$.
- (c) The triangle inequality for the 1-norm: $\|\mathbf{u} + \mathbf{v}\|_1 \leq \|\mathbf{u}\|_1 + \|\mathbf{v}\|_1$.

Exercise 2.3

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be sparse, and let $\mathbf{x} \in \mathbb{R}^n$. In each case, express the computational complexity in terms of the matrix dimensions and k .

- (a) Suppose each *row* of $\mathbf{A}$ has at most k nonzero entries. What is the computational complexity of the product $\mathbf{Ax}$ if the multiplications with zero entries are skipped?
- (b) Now suppose instead that each *column* of $\mathbf{A}$ has at most k nonzero entries. What is the complexity of $\mathbf{Ax}$?

Exercise 2.4

Numerical linear algebra software is built on a standardized set of building blocks known as the *basic linear algebra subprograms* (BLAS), organized into three levels: Level 1 for vector operations, Level 2 for matrix-vector operations, and Level 3 for matrix-matrix operations. Analyze the computational complexity of the following BLAS operations. In the descriptions below, $\leftarrow$ denotes *assignment*: the expression on the right is computed and its result overwrites the variable on the left.

- (a) **ger** (Level 2, the *rank-1 update*): $\mathbf{A} \leftarrow \mathbf{A} + \alpha \mathbf{xy}^\top$, for $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{y} \in \mathbb{R}^n$, and a scalar α .
- (b) **syrk** (Level 3, the *symmetric rank-k update*): $\mathbf{C} \leftarrow \mathbf{C} + \mathbf{AA}^\top$, for $\mathbf{A} \in \mathbb{R}^{m \times n}$ and symmetric $\mathbf{C} \in \mathbb{R}^{m \times m}$.
- (c) **trsm** (Level 3, the *triangular solve with multiple right-hand sides*): solve $\mathbf{LX} = \mathbf{B}$ for $\mathbf{X}$, where $\mathbf{L} \in \mathbb{R}^{n \times n}$ is lower triangular and $\mathbf{B} \in \mathbb{R}^{n \times p}$.

Exercise 2.5

Write MATLAB code that generates (a) a random $n \times n$ symmetric matrix, (b) a random $n \times n$ symmetric positive definite matrix, and (c) a random $n \times n$ symmetric positive semi-definite matrix that is *not* positive definite, (d) a random $n \times n$ matrix with rank at most k , for a given $k < n$, and (e) a random *symmetric* $n \times n$ matrix with rank at most k . In each case, the code must return a different (random) matrix every time it is run. [Hint for (b)–(e): consider products of random factors, as in the previous exercise.]

Exercise 2.6

Compute the 1-, 2-, and ∞ -norms of $\mathbf{v} = [1, 1, -1, 2, -3]$.

Exercise 2.7

Compute $\det(\mathbf{A})$ where

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 2 & 0 \\ 2 & 7 & 0 & 0 \\ 4 & 0 & 0 & 9 \\ 0 & 1 & 2 & 1 \end{bmatrix}.$$

Exercise 2.8

What is the dimension of the column space of

$$\mathbf{A} = \begin{bmatrix} 1 & 3 & 1 & 0 \\ 2 & 6 & 3 & 1 \\ 1 & 3 & 2 & 1 \end{bmatrix}?$$

Exercise 2.9

Answer using the rank-nullity theorem and the existence and uniqueness results of this chapter.

- (a) Let $\mathbf{A} \in \mathbb{R}^{4 \times 6}$ with $\text{rank}(\mathbf{A}) = 4$. What is the nullity of $\mathbf{A}$?
- (b) For the same $\mathbf{A}$, does $\mathbf{Ax} = \mathbf{b}$ have a solution for every $\mathbf{b} \in \mathbb{R}^4$? Can a solution ever be unique?
- (c) Now let $\mathbf{B} \in \mathbb{R}^{5 \times 3}$ with $\text{rank}(\mathbf{B}) = 3$. Does $\mathbf{Bx} = \mathbf{c}$ have a solution for every $\mathbf{c} \in \mathbb{R}^5$? If a solution exists, is it unique?

Exercise 2.10

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ have full column rank, $\text{rank}(\mathbf{A}) = n \leq m$.

- (a) What do we know about the existence and uniqueness of solutions of $\mathbf{Ax} = \mathbf{b}$?
- (b) Find all solutions of $\mathbf{Ax} = \mathbf{0}$.
- (c) Suppose $\mathbf{b} = \alpha \mathbf{a}_1^C$ for some scalar α . Find all solutions of $\mathbf{Ax} = \mathbf{b}$.

Exercise 2.11

Vector spaces need not consist of column vectors. A polynomial of degree at most k , $p(x) = c_0 + c_1x + \cdots + c_kx^k$, can be represented by its coefficient vector $\mathbf{c} = (c_0, c_1, \dots, c_k) \in \mathbb{R}^{k+1}$.

- (a) Derive the matrix $\mathbf{D} \in \mathbb{R}^{(k+1) \times (k+1)}$ that represents first-order differentiation: if $\mathbf{c}$ represents p , then $\mathbf{Dc}$ represents p' . (Write it out for $k = 3$.)
- (b) What is the meaning of $\mathbf{D}^j$? What is $\mathbf{D}^{k+1}$?
- (c) Is there a matrix representing integration, i.e., an inverse of $\mathbf{D}$?
- (d) Are the even polynomials (only even powers) and the odd polynomials (only odd powers) subspaces of this vector space?
- (e) Is the set of polynomials that have $x = 3$ as a root a subspace of this vector space?

Exercise 2.12

The *Gram-Schmidt process* turns linearly independent vectors $\mathbf{a}_1, \dots, \mathbf{a}_k \in \mathbb{R}^n$ into orthonormal vectors $\mathbf{q}_1, \dots, \mathbf{q}_k$ spanning the same space: each $\mathbf{a}_j$ is stripped of its components along the previously constructed directions and then normalized,

$$\mathbf{v}_j = \mathbf{a}_j - \sum_{i=1}^{j-1} (\mathbf{q}_i^\top \mathbf{a}_j) \mathbf{q}_i, \quad \mathbf{q}_j = \mathbf{v}_j / \|\mathbf{v}_j\|_2.$$

- (a) Complete the following MATLAB function by filling in the blank marked ???.

```
function Q = gram_schmidt(A)
% Orthonormalize the columns of A.
[n, k] = size(A);
Q = zeros(n, k);
```

```

for j = 1:k
    v = A(:, j);
    for i = 1:j-1
        v = ??? ;
    end
    Q(:, j) = v / norm(v);
end
end

```

(b) Analyze the computational complexity of the process in terms of n and k .

Exercise 2.13

Solve the following system via Gaussian Elimination:

$$\begin{bmatrix} 3 & 1 & 0 \\ 1 & 1 & -1 \\ 2 & 0 & 1 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

Exercise 2.14

Solve the linear system $\mathbf{Ax} = \mathbf{b}$ with the *arrow-structured* matrix

$$\mathbf{A} = \begin{bmatrix} 4 & 1 & 1 & 1 & 1 \\ 1 & 2 & 0 & 0 & 0 \\ 1 & 0 & 2 & 0 & 0 \\ 1 & 0 & 0 & 2 & 0 \\ 1 & 0 & 0 & 0 & 2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 8 \\ 3 \\ 3 \\ 3 \\ 3 \end{bmatrix}.$$

[Hint: applying Gaussian elimination in the given order destroys the sparsity—eliminating the first column fills in the whole matrix. Consider reordering the equations and unknowns so that the dense row and column come *last*, and then eliminate.]

Exercise 2.15

Recall the Thomas algorithm from Exercise 1.7. A tridiagonal matrix is sparse: of its n^2 entries, only $3n - 2$ are nonzero. What is the computational complexity of solving a tridiagonal linear system (a) with the Thomas algorithm, which exploits the sparsity, and (b) with standard Gaussian elimination, which treats the matrix as dense?

Exercise 2.16

In the primal-dual interior-point method for linear programming (discussed in the optimization chapter), each iteration requires solving the so-called KKT system

$$\begin{bmatrix} \mathbf{D} & \mathbf{A}^\top \\ \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{x} \\ \Delta \mathbf{y} \end{bmatrix} = \begin{bmatrix} \mathbf{r}_1 \\ \mathbf{r}_2 \end{bmatrix},$$

where $\mathbf{D} \in \mathbb{R}^{n \times n}$ is diagonal with positive entries, $\mathbf{A} \in \mathbb{R}^{m \times n}$, and $m < n$.

- What are the dimensions of the KKT system, and what is the complexity of solving it directly with dense Gaussian elimination?
- Show that eliminating $\Delta \mathbf{x}$ reduces the system to the *normal equations* $(\mathbf{AD}^{-1}\mathbf{A}^\top) \Delta \mathbf{y} = \mathbf{AD}^{-1}\mathbf{r}_1 - \mathbf{r}_2$.
- What is the complexity of the normal-equation route (forming the $m \times m$ matrix, solving, and recovering $\Delta \mathbf{x}$)? Compare with (a) when $m \ll n$.

Exercise 2.17

The *Cholesky factorization* decomposes a symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ as $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$, where $\mathbf{L}$ is lower triangular. The algorithm computes the entries of $\mathbf{L}$ column by column: for $j = 1, 2, \dots, n$,

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}, \quad l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right), \quad i = j+1, \dots, n.$$

- What is the failure mode of this algorithm? Provide an example of a 2×2 symmetric matrix $\mathbf{A}$ for which the algorithm fails.
- Suppose the algorithm succeeds, with $l_{jj} > 0$ for all j . What does this tell us about the invertibility of $\mathbf{A}$?
- What does it tell us about the eigenvalues of $\mathbf{A}$?
- How can this algorithm be used to check whether a given symmetric matrix is positive definite?
- Suppose the algorithm fails for $\mathbf{A}$, and we want to find $\delta > 0$ such that $\mathbf{A} + \delta \mathbf{I}$ is positive definite. Design an algorithm that finds such a δ using the Cholesky factorization.

Exercise 2.18

For $\mathbf{A} = \begin{bmatrix} 3 & 0 & 2 \\ 0 & 1 & 0 \\ -1 & 0 & 3 \end{bmatrix}$:

- Find the eigenvalues of $\mathbf{A}$.
- Find an eigenvector associated with each eigenvalue.
- Are these eigenvectors unique?

Exercise 2.19

For

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix},$$

- Determine $\text{rank}(\mathbf{A} - \lambda \mathbf{I})$ for every $\lambda \in \mathbb{R}$.
- More generally, suppose λ is an eigenvalue of $\mathbf{B} \in \mathbb{R}^{n \times n}$ with geometric multiplicity g . What is $\text{rank}(\mathbf{B} - \lambda \mathbf{I})$? Does the algebraic multiplicity enter?

Exercise 2.20

(*) Write a matrix $\mathbf{A}$ that has eigenvalues 0 and 2 with corresponding eigenvectors $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Is such a matrix unique?

Exercise 2.21

Not every matrix is diagonalizable, but every matrix admits a *Jordan decomposition*,

$$\mathbf{A} = \mathbf{P}\mathbf{J}\mathbf{P}^{-1},$$

where $\mathbf{P}$ is invertible and $\mathbf{J}$ is a block-diagonal matrix built from *Jordan blocks*, which carry an eigenvalue λ on the diagonal and ones directly above it. Consider the 2×2 Jordan block

$$\mathbf{J} = \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}.$$

- (a) Compute $\mathbf{J}^2$ and $\mathbf{J}^3$, and find the general pattern for $\mathbf{J}^k$.
- (b) Suppose $|\lambda| < 1$ strictly. Does $\mathbf{J}^k$ converge to $\mathbf{0}$ as $k \rightarrow \infty$?
- (c) Now suppose $|\lambda| = 1$. Does $\mathbf{J}^k$ remain bounded? Compare with a *diagonalizable* matrix whose eigenvalues all satisfy $|\lambda_i| \leq 1$, and draw a conclusion about the convergence condition $\rho < 1$ for iterative methods.

Exercise 2.22

For $\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 0 & 3 \\ 1 & 0 \end{bmatrix}$:

- (a) Compute the singular values of $\mathbf{A}$.
- (b) Find $\|\mathbf{A}\|_2$. [Do not compute it from the definition; use the singular values from (a).]
- (c) Find $\text{tr}(\mathbf{A}\mathbf{A}^\top)$. [Again, do not compute the product explicitly; use the singular values from (a).]

Exercise 2.23

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ have the singular value decomposition $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, and construct

$$\mathbf{B} = \mathbf{U}(\mathbf{\Sigma} + \mathbf{I})\mathbf{V}^\top,$$

which shifts every singular value up by 1. What is $\|\mathbf{B}\|_2$?

Exercise 2.24

Give an example of a 2×2 symmetric matrix $\mathbf{A}$ for which both Jacobi iteration and Gauss-Seidel iteration fail to converge for a generic initial guess, and explain why they fail. [Hint: recall that convergence is governed by the spectral radius of the iteration matrix.]

Exercise 2.25

Consider solving the tridiagonal system of Exercise 1.7 with an iterative method instead of the Thomas algorithm: starting from a guess $\mathbf{x}^{(0)}$, sweep

$$x_i^{(k+1)} = \frac{b_i - a_{i,i-1} x_{i-1}^{(k)} - a_{i,i+1} x_{i+1}^{(k)}}{a_{ii}}, \quad i = 1, \dots, n,$$

with $x_0 = x_{n+1} = 0$ understood.

- (a) Which of the iterative methods of this chapter is this?
- (b) What is the computational cost of one sweep?
- (c) Derive a sufficient condition on the coefficients for convergence.
- (d) The Thomas algorithm solves the system exactly in $O(n)$ operations. When could the iteration nevertheless be preferable?

Exercise 2.26

A polynomial $p(x)$ that approximates $1/x$ on an interval containing all eigenvalues of a symmetric matrix $\mathbf{A}$ can be used as a *polynomial preconditioner*: the matrix $p(\mathbf{A})$, obtained by substituting $\mathbf{A}$ into the polynomial, then approximates $\mathbf{A}^{-1}$. In this exercise, we work with the explicit degree-2 polynomial

$$p(x) = 1 + (1 - x) + (1 - x)^2,$$

so that $p(\mathbf{A}) = 3\mathbf{I} - 3\mathbf{A} + \mathbf{A}^2$. Expanding, $p(x) = 3 - 3x + x^2$, and a short calculation shows that

$$1 - xp(x) = (1 - x)^3,$$

so if $|1 - x| \leq \rho$ for some $\rho < 1$, then $|p(x) - \frac{1}{x}| = \frac{|1-x|^3}{x} \leq \frac{\rho^3}{x}$: the polynomial approximates $1/x$ with an error proportional to the *cube* of the distance from x to 1. A preconditioner is only ever applied to vectors, $\mathbf{z} = p(\mathbf{A})\mathbf{v}$; the matrix $p(\mathbf{A})$ itself is never formed.

- (a) Show that a similar approximation error holds for matrices: for a diagonalizable $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$, the eigenvalues of the error matrix $p(\mathbf{A}) - \mathbf{A}^{-1}$ are $-(1 - \lambda_i)^3/\lambda_i$; hence $p(\mathbf{A}) \approx \mathbf{A}^{-1}$ whenever every eigenvalue of $\mathbf{A}$ satisfies $|1 - \lambda_i| \leq \rho$ for some $\rho < 1$.
- (b) The following function computes $\mathbf{z} = p(\mathbf{A})\mathbf{v}$. Since it only involves products with vectors, its cost was expected to scale *quadratically* with n for $\mathbf{A} \in \mathbb{R}^{n \times n}$ — but a timing experiment shows the cost growing *cubically*. Find what is wrong, explain why the cost is cubic, and correct the code so that it scales quadratically.

```
function z = poly_precond(A, v)
% Apply z = p(A) * v for p(x) = 3 - 3x + x^2.
z = 3 * v - 3 * A * v + A * A * v;
end
```

- (c) Using the function from (b), implement the preconditioned iteration

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + p(\mathbf{A})(\mathbf{b} - \mathbf{A}\mathbf{x}^{(k)})$$

for solving $\mathbf{Ax} = \mathbf{b}$, starting from $\mathbf{x}^{(0)} = \mathbf{0}$ and stopping when the relative residual falls below a tolerance.

Chapter 3

Systems of Nonlinear Equations

Many problems in chemical engineering require solving systems of nonlinear equations. Nonlinear systems can exhibit interesting and sometimes surprising behavior. As we will see, a system of nonlinear equations may have no solutions, a single unique solution, a finite number of solutions, or an infinite number of solutions; we will not have the same guarantees as we saw for systems of linear equations. This section will explore methods for finding these solutions and understanding when they exist.

3.1 Introduction to Nonlinear Systems

We are interested in solving systems of the form:

$$\mathbf{f}(\mathbf{x}) = \mathbf{0} \quad (3.1.1)$$

where $\mathbf{x} \in \mathbb{R}^N$ is our vector of unknowns and $\mathbf{f} : \mathbb{R}^N \rightarrow \mathbb{R}^N$ is a vector-valued function. The solutions $\mathbf{x}^*$ that satisfy $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$ are called the *roots* of $\mathbf{f}$. Linear equations can be written in this framework as $\mathbf{f}(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}$.

Common examples of nonlinear systems of equations in chemical engineering include: (1) equations of state, (2) steady-state energy balances, and (3) steady-state mass balances with nonlinear reaction rates.

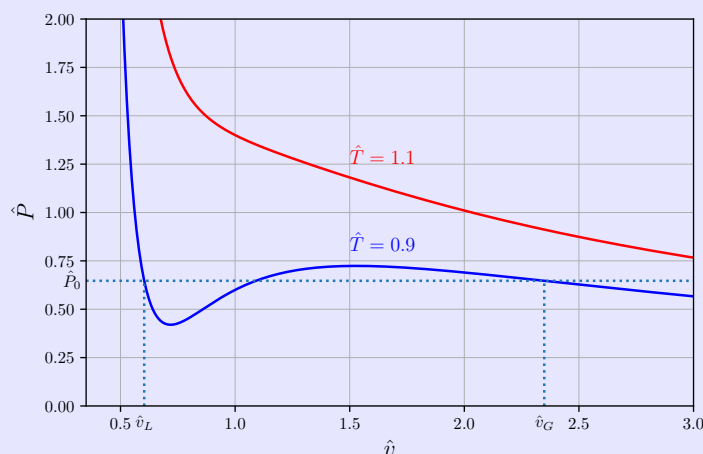
Example: van der Waals Equation of State

Consider finding the molar volume $\hat{v}$ for a given reduced pressure $\hat{P}$ and reduced temperature $\hat{T}$ using the van der Waals equation:

$$\left(\hat{P} + \frac{3}{\hat{v}^2}\right) \left(\hat{v} - \frac{1}{3}\right) = \frac{8}{3}\hat{T}$$

This can be rewritten as finding roots of

$$f(\hat{v}; \hat{P}, \hat{T}) = \left(\hat{P} + \frac{3}{\hat{v}^2}\right) \left(\hat{v} - \frac{1}{3}\right) - \frac{8}{3}\hat{T} = 0$$



Depending on the values of $\hat{P}$ and $\hat{T}$, this equation can have 1 or 3 real solutions for the molar volume, corresponding to different phases of matter. In the plot above, we see that for a given reduced temperature $\hat{T} = 0.9$ (blue line) and reduced pressure $\hat{P} = \hat{P}_0$, there are three real solutions for the molar volume (only two of which are physically stable—the middle one leads to phase separation, as you will learn in 10.40). Meanwhile, for the higher reduced temperature $\hat{T} = 1.1$ (red line), there is only one real solution to $\hat{v}$ that will satisfy the original system of equations for any given value of $\hat{P}$.

For linear systems $\mathbf{Ax} = \mathbf{b}$, we have a clean criterion: the rank of $\mathbf{A}$ tells us exactly when solutions exist and how many there are. Nonlinear systems are different. There is no universal test that applies across all problems. Instead, the number of solutions depends on the specific structure of the system, and this global picture can shift as parameters vary (as illustrated, for example, by the van der Waals equation example above). Only in special cases do we have similar guarantees; for example, an N^{th} order polynomial has at most N distinct roots. Without additional structure or assumptions, we cannot know in advance if a nonlinear system $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ has no solutions, exactly one solution, or multiple solutions. The numerical methods/techniques we choose to solve systems of nonlinear equations must be robust to this uncertainty.

3.2 The Jacobian

Throughout this section of the course and beyond, when working with vector-valued functions, we will make extensive use of the **Jacobian matrix**. This is one way to refer to the first derivative of a vector-valued function.

Definition: Jacobian Matrix. For a function $\mathbf{f} : \mathbb{R}^M \rightarrow \mathbb{R}^N$, the Jacobian matrix $\mathbf{J}(\mathbf{x})$ is

$$\mathbf{J}(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_M} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_M} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_N}{\partial x_1} & \frac{\partial f_N}{\partial x_2} & \cdots & \frac{\partial f_N}{\partial x_M} \end{bmatrix} = \nabla_{\mathbf{x}} \mathbf{f}(\mathbf{x}) = \nabla \mathbf{f}(\mathbf{x}) \in \mathbb{R}^{N \times M} \quad (3.2.1)$$

Note that in this section of the course, we typically deal with the case that $N = M$.

We'll prefer to use the notation $\mathbf{J}$ for the Jacobian matrix just to avoid confusion with the gradient of a scalar function. There are actually very deep connections between the Jacobian, the gradient, the Hessian, etc. If you're interested, you should take [this IAP course on matrix calculus \(18.063\)](#). For the purposes of 10.34, we do not need to worry about these deeper connections—we will talk about some of the connections once we

introduce concepts in optimization later. However, it is important to be very familiar with the mechanics of computing the Jacobian and its structure/properties.

The Jacobian describes how the function $\mathbf{f}$ changes with respect to all its variables. More specifically, each column in the Jacobian describes the partial derivative of the vector-valued function with respect to a single input variable. It plays the same role for vector functions that the derivative plays for scalar functions. As such, we can think about it as the linear map that best approximates how $\mathbf{f}$ changes at a point $\mathbf{x}_0$: for any small increment $\Delta\mathbf{x}$, assuming that $\mathbf{f}$ is twice differentiable, we have

$$\mathbf{f}(\mathbf{x}_0 + \Delta\mathbf{x}) = \mathbf{f}(\mathbf{x}_0) + \mathbf{J}(\mathbf{x}_0)\Delta\mathbf{x} + O(\|\Delta\mathbf{x}\|_2^2) \quad (3.2.2)$$

Eq. 3.2.2 represents a Taylor expansion of the function $\mathbf{f}$ around $\mathbf{x}_0$, which we will elaborate on later in [section 3.5](#). The higher-order terms involving the second, third, etc. derivatives of the function are omitted and replaced with the expression $O(\|\Delta\mathbf{x}\|_2^2)$ that defines how the truncation error scales with the magnitude of our increment $\Delta\mathbf{x}$. We expect the truncated linear approximation to be increasingly accurate as $\|\Delta\mathbf{x}\|_2 \rightarrow 0$.

3.3 Local Uniqueness

Before we look into numerical methods to help us find solutions to nonlinear equations, we can first consider analyzing a hypothetical solution $\mathbf{x}^*$ assuming that $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$, i.e., that it is indeed a solution to our original system of equations. The concept of local uniqueness is an important aspect of a solution.

Definition: Local Uniqueness. A solution $\mathbf{x}^*$ is **locally unique** if there exists a ball of finite radius $r > 0$ such that $\mathbf{x}^*$ is the only solution within that ball:

$$\|\mathbf{x} - \mathbf{x}^*\| < r \quad \text{and} \quad \mathbf{f}(\mathbf{x}) = \mathbf{0} \implies \mathbf{x} = \mathbf{x}^* \quad (3.3.1)$$

It turns out that evaluating the Jacobian at a solution $\mathbf{x}^*$ allows us to easily determine local uniqueness.

Inverse Function Theorem Let $\mathbf{f} : \mathbb{R}^N \rightarrow \mathbb{R}^N$ be differentiable on an open neighborhood of $\mathbf{x}^*$, and suppose that its Jacobian $\mathbf{J}(\mathbf{x})$ varies continuously with $\mathbf{x}$ near $\mathbf{x}^*$ (i.e., $\mathbf{f}$ has a continuous derivative in a neighborhood of $\mathbf{x}^*$). If $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$ and $\det \mathbf{J}(\mathbf{x}^*) \neq 0$, then $\mathbf{x}^*$ is the unique solution of $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ in the neighborhood of $\mathbf{x}^*$ (i.e., the solution is locally unique).

This theorem generalizes the inverse function theorem from single-variable calculus to higher dimensions. There are two main ideas to highlight. First, the conditions it gives are sufficient to guarantee local uniqueness of a solution:

1. the function must vanish at the root, and
2. the Jacobian at that root must be nonsingular.

Second, and equally important, we must be careful about what happens when the Jacobian is singular. A singular Jacobian does *not* automatically mean the solution is not locally unique. In this case, uniqueness may or may not hold; it depends on the specific function and the particular root. We will meet this exact ambiguity again in a more dynamic setting: the bifurcation-analysis section ([section 3.8](#)) uses precisely the condition $\det \mathbf{J} = 0$ as its defining criterion for locating turning points, where the number of solutions to a parameterized system changes.

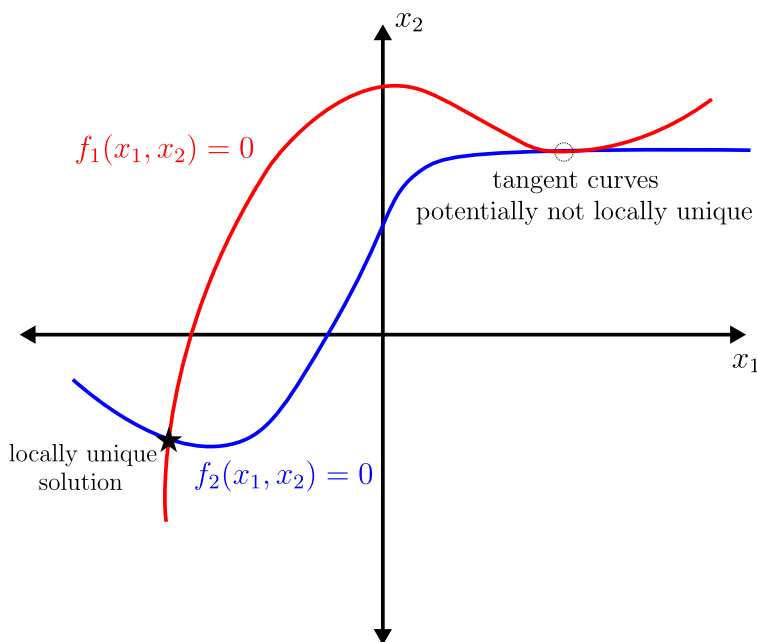


Figure 3.3.1: Application of the inverse function theorem in 2D. The red and blue lines indicate the values of x_1 and x_2 for which two components of a vector-valued function, f_1 and f_2 respectively, are zero. Intersections of these curves define the roots of the function. Where the curves are tangent to each other indicates that their gradients with respect to x_1 and x_2 will be parallel, and hence the rows of the Jacobian will be linearly dependent, rendering it singular. Star: A solution where the curves intersect transversely; the Jacobian is clearly nonsingular, and therefore the solution is guaranteed to be locally unique. Dashed circle: A solution where the curves are tangent; the Jacobian is singular, and therefore the solution is *potentially* non-unique.

A good way to see this is to compare with the single-variable setting. There, the analogous¹ condition is that the derivative at the root should be nonzero. If the derivative vanishes, the situation is ambiguous: for instance, $f(x) = x^3$ has a unique root at $x = 0$ even though $f'(0) = 0$, whereas $f(x) = 0$ has infinitely many roots (all x) and uniqueness clearly fails in a local sense at $x = 0$. The multivariable case mirrors this: a nonsingular Jacobian guarantees local uniqueness, but a singular one leaves the question open.

Warning: The converse of the inverse function theorem is not always true. In other words, we may ask, “If $\mathbf{x}^*$ is a locally unique root of $\mathbf{f}(\mathbf{x}) = 0$, must $\det \mathbf{J}(\mathbf{x}^*) \neq 0$?” The answer is no. Local uniqueness of the solution is strictly weaker than non-singularity of the Jacobian. A simple one-dimensional counterexample is

$$f(x) = x^3, \quad f(0) = 0, \quad f'(0) = 0$$

Here, $x = 0$ is *the only* solution of $f(x) = 0$ in any neighborhood of 0, yet $f'(0) = 0$. So you can have a unique root even though the Jacobian vanishes at that root.

Example: Jacobian and Uniqueness for an Exponential System

Consider the map $\mathbf{f} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ given by

$$\mathbf{f}(x, y) = \begin{bmatrix} xe^y - 1 \\ xe^{-y} - 1 \end{bmatrix}$$

¹Okay, this is not really “analogous” so much as it is the exact same condition. If this imprecision in language bothers you, you should definitely take 18.063 over IAP!

Its Jacobian matrix is

$$\mathbf{J}(x, y) = \begin{bmatrix} \frac{\partial}{\partial x}(xe^y - 1) & \frac{\partial}{\partial y}(xe^y - 1) \\ \frac{\partial}{\partial x}(xe^{-y} - 1) & \frac{\partial}{\partial y}(xe^{-y} - 1) \end{bmatrix} = \begin{bmatrix} e^y & xe^y \\ e^{-y} & -xe^{-y} \end{bmatrix}$$

To find a root $\mathbf{f}(x, y) = \mathbf{0}$, we solve

$$xe^y = 1, \quad xe^{-y} = 1$$

Multiplying the two equations gives $x^2 = 1$, so $x = \pm 1$. If $x = 1$, then $e^y = 1$ so $y = 0$. If $x = -1$, there is no real y with $e^y = -1$. Thus, the unique real solution is

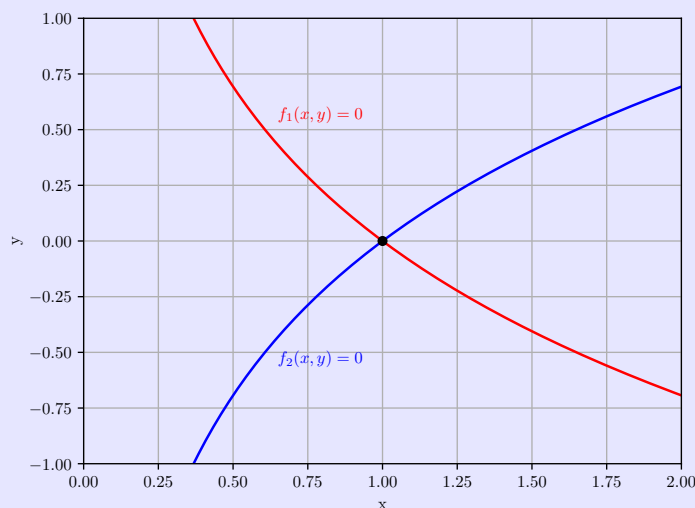
$$(x, y) = (1, 0)$$

Uniqueness check. We now verify local uniqueness at the solution by checking the determinant of the Jacobian there:

$$\det \mathbf{J}(1, 0) = (e^0) \cdot (-1 \cdot e^0) - (1 \cdot e^0) \cdot (e^0) = -1 - 1 = -2 \neq 0$$

Since $\det \mathbf{J}(1, 0) \neq 0$, the inverse function theorem guarantees the root $(1, 0)$ is locally unique.

We can also visually verify this result from the plot of the contours of $f_1(x, y) = 0$ and $f_2(x, y) = 0$ below. It is obvious that the only point where the contours intersect is at $(1, 0)$ and that this point is locally unique.



3.4 Connection to Linear Systems

There's a deep connection between nonlinear systems and the linear systems we studied earlier. Consider a linear system $\mathbf{f}(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b}$. The Jacobian is simply

$$\mathbf{J}(\mathbf{x}) = \mathbf{A} \tag{3.4.1}$$

(To see this, it's easiest to use the notion of the Jacobian given by [Eq. 3.2.2](#). We can read off the derivative of $\mathbf{f}$ by looking at how $\mathbf{f}$ changes under a small increment, and in this case it's simply $\mathbf{A}$. The [Matrix Cookbook](#) is a terrific resource for more complex identities than this.)

The condition for local uniqueness, $\det \mathbf{J}(\mathbf{x}) \neq 0$, becomes $\det \mathbf{A} \neq 0$, which is exactly the condition for $\mathbf{A}$ to be invertible! For systems of linear equations, we know that if $\mathbf{A}\mathbf{x} = \mathbf{b}$ has a solution $\mathbf{x}^*$, that solution is guaranteed to be (locally) unique if $\mathbf{A}$ is nonsingular. So, our theory of nonlinear equations naturally extends

our understanding of linear systems. In fact, for the linear case this upgrades to *global* uniqueness: because the Jacobian of a linear map is the map itself, $\mathbf{J}(\mathbf{x}) = \mathbf{A}$ for *every* $\mathbf{x}$ (not just near $\mathbf{x}^*$), so the “ball of radius r ” in the local-uniqueness definition can be taken arbitrarily large.

3.5 Linearization via Taylor Expansions

A Taylor expansion of a sufficiently smooth (this is usually a safe assumption in 10.34) vector-valued function $\mathbf{f}: \mathbb{R}^N \rightarrow \mathbb{R}^N$ about the point $\mathbf{x}$ shows that, for any small increment $\Delta\mathbf{x} \in \mathbb{R}^N$,

$$\mathbf{f}(\mathbf{x} + \Delta\mathbf{x}) = \mathbf{f}(\mathbf{x}) + \mathbf{J}(\mathbf{x})\Delta\mathbf{x} + O(\|\Delta\mathbf{x}\|_2^2) \quad (3.5.1)$$

where the Jacobian $\mathbf{J}(\mathbf{x})$ is exactly the matrix of first-order partial derivatives defined earlier. In one dimension, this reduces to the familiar result that

$$f(x + \Delta x) = f(x) + f'(x)\Delta x + O(\Delta x^2) \quad (3.5.2)$$

again making it clear that $\mathbf{J}(\mathbf{x})$ plays the role of the derivative in higher dimensions.

3.5.1 Local Uniqueness via Taylor Expansions

To see how this yields local uniqueness of a root, suppose $\mathbf{x}^*$ satisfies $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. A Taylor expansion about $\mathbf{x}^*$ gives

$$\mathbf{f}(\mathbf{x}) = \underbrace{\mathbf{f}(\mathbf{x}^*)}_{\mathbf{0}} + \mathbf{J}(\mathbf{x}^*)(\mathbf{x} - \mathbf{x}^*) + O(\|\mathbf{x} - \mathbf{x}^*\|^2) = \mathbf{J}(\mathbf{x}^*)\Delta\mathbf{x} + \underbrace{O(\|\Delta\mathbf{x}\|^2)}_{\mathbf{R}(\Delta\mathbf{x})} \quad (3.5.3)$$

where $\Delta\mathbf{x} = \mathbf{x} - \mathbf{x}^*$. Here, the remainder term $\mathbf{R}(\Delta\mathbf{x}) \triangleq O(\|\Delta\mathbf{x}\|^2)$ denotes an error that vanishes faster than $\|\Delta\mathbf{x}\|$ as $\mathbf{x} \rightarrow \mathbf{x}^*$, so for sufficiently small $\Delta\mathbf{x}$ the linear part dominates.

If $\det \mathbf{J}(\mathbf{x}^*) \neq 0$, the matrix $\mathbf{J}(\mathbf{x}^*)$ is invertible and its null-space consists only of the zero vector. We can prove the inverse function theorem using a proof by contradiction. Suppose that there were a distinct root $\mathbf{x} = \mathbf{x}^* + \Delta\mathbf{x}$ with $0 < \|\Delta\mathbf{x}\| \ll 1$. If this is a root, then $\mathbf{f}(\mathbf{x}^* + \Delta\mathbf{x}) = \mathbf{0}$ by definition. Therefore,

$$\mathbf{0} = \mathbf{J}(\mathbf{x}^*)\Delta\mathbf{x} + \mathbf{R}(\Delta\mathbf{x}) \implies \mathbf{J}(\mathbf{x}^*)\Delta\mathbf{x} = -\mathbf{R}(\Delta\mathbf{x}) \quad (3.5.4)$$

Since $\mathbf{J}(\mathbf{x}^*)$ is invertible, we can solve for $\Delta\mathbf{x}$:

$$\Delta\mathbf{x} = -\mathbf{J}(\mathbf{x}^*)^{-1}\mathbf{R}(\Delta\mathbf{x}) \quad (3.5.5)$$

Now, taking norms on both sides, we have

$$\|\Delta\mathbf{x}\| = \|\mathbf{J}(\mathbf{x}^*)^{-1}\mathbf{R}(\Delta\mathbf{x})\| \leq \|\mathbf{J}(\mathbf{x}^*)^{-1}\| \|\mathbf{R}(\Delta\mathbf{x})\| \quad (3.5.6)$$

By the definition of $\mathbf{R}$, there exists $C > 0$ with $\|\mathbf{R}(\Delta\mathbf{x})\| \leq C\|\Delta\mathbf{x}\|^2$ for small $\|\Delta\mathbf{x}\|$. Hence

$$\|\Delta\mathbf{x}\| \leq \|\mathbf{J}(\mathbf{x}^*)^{-1}\| C \|\Delta\mathbf{x}\|^2 \quad (3.5.7)$$

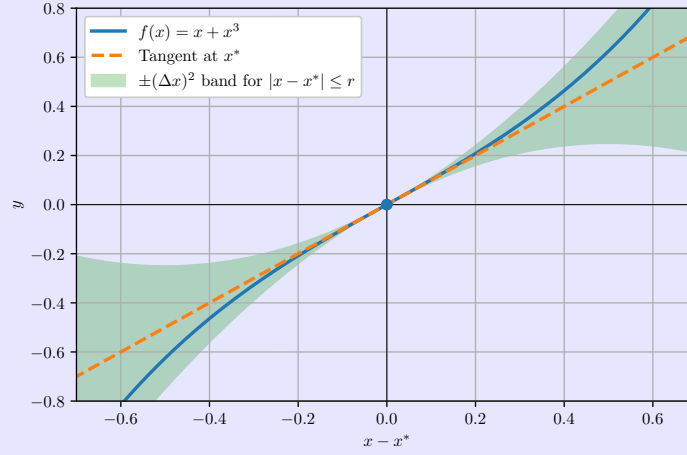
If $0 < \|\Delta\mathbf{x}\| < (\|\mathbf{J}(\mathbf{x}^*)^{-1}\| C)^{-1}$, then dividing by $\|\Delta\mathbf{x}\|$ yields $1 \leq \|\mathbf{J}(\mathbf{x}^*)^{-1}\| C \|\Delta\mathbf{x}\| < 1$, a contradiction. Therefore, $\Delta\mathbf{x} = \mathbf{0}$ is the only possibility in a sufficiently small neighborhood, which completes the argument that no other root can lie nearby.

Example: Visualizing the Quadratic Remainder Consider the scalar (1D) function

$$f(x) = x + x^3$$

which has a simple root at $x^* = 0$. We plot $f(x)$ together with its tangent line (first-order Taylor approximation) $\ell(x) = f'(x^*)(x - x^*)$, and we shade the region where the Taylor remainder satisfies

$|R(\Delta x)| \leq C \cdot (\Delta x)^2$. Concretely, we choose a small radius r and display the band $\pm(\Delta x)^2$ for $|x - x^*| \leq r$. Since the tangent line crosses the axis only once and the nonlinear curve stays inside this narrow tube, no second root can lie in that neighborhood, visually confirming the local uniqueness result.



3.6 Iterative Methods for Solving Nonlinear Equations

We are concerned with finding solutions $\mathbf{x}^* \in \mathbb{R}^N$ to a system of N nonlinear equations $\mathbf{f}(\mathbf{x}) = \mathbf{0}$, where $\mathbf{f} : \mathbb{R}^N \rightarrow \mathbb{R}^N$:

$$\mathbf{f}(\mathbf{x}^*) = \begin{bmatrix} f_1(x_1^*, x_2^*, \dots, x_N^*) \\ f_2(x_1^*, x_2^*, \dots, x_N^*) \\ \vdots \\ f_N(x_1^*, x_2^*, \dots, x_N^*) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \mathbf{0} \quad (3.6.1)$$

Unlike linear systems, most nonlinear systems do not admit a closed-form solution. Therefore, we almost always resort to iterative methods. An iterative method generates a sequence of approximate solutions $\{\mathbf{x}_k\}$ starting from an initial guess $\mathbf{x}_0$ with the aim that this sequence converges to a true solution $\mathbf{x}^*$, i.e., $\lim_{k \rightarrow \infty} \mathbf{x}_k = \mathbf{x}^*$.

Many iterative methods make use of **fixed-point iteration**. The idea is to transform the original root-finding problem $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ into an equivalent fixed-point problem of the form

$$\mathbf{x} = \mathbf{g}(\mathbf{x}) \quad (3.6.2)$$

$\mathbf{g}$ serves as a map from any provided value of $\mathbf{x}$ to an updated value. Given an initial value $\mathbf{x}_0$, we feed it into the function to get $\mathbf{x}_1 = \mathbf{g}(\mathbf{x}_0)$, then again to get $\mathbf{x}_2 = \mathbf{g}(\mathbf{x}_1)$, and so on.

Depending on the definition of $\mathbf{g}$, there may be special values of $\mathbf{x}$ that remain unchanged by $\mathbf{g}$ and, therefore, correspond to solutions to the fixed-point problem Eq. 3.6.2. More formally, a solution $\mathbf{x}^*$ to this equation is called a fixed point of the function $\mathbf{g}$ if applying $\mathbf{g}$ to $\mathbf{x}^*$ yields $\mathbf{x}^*$ itself: $\mathbf{x}^* = \mathbf{g}(\mathbf{x}^*)$.

The key idea is to construct $\mathbf{g}(\mathbf{x})$ such that any fixed point of $\mathbf{g}$ is also a root of $\mathbf{f}$. When we can ensure this is the case, identifying solutions to $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ becomes equivalent to finding these fixed points. There are many ways to define such a $\mathbf{g}(\mathbf{x})$. A common family of transformations is

$$\mathbf{g}(\mathbf{x}) = \mathbf{x} + \mathbf{A}(\mathbf{x})\mathbf{f}(\mathbf{x}) \quad (3.6.3)$$

where $\mathbf{A}(\mathbf{x})$ is a suitably chosen matrix (which could be constant, or depend on $\mathbf{x}$ as suggested by the notation). If $\mathbf{x}^* = \mathbf{g}(\mathbf{x}^*)$, then $\mathbf{x}^* = \mathbf{x}^* + \mathbf{A}(\mathbf{x}^*)\mathbf{f}(\mathbf{x}^*)$, which implies $\mathbf{A}(\mathbf{x}^*)\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. If $\mathbf{A}(\mathbf{x}^*)$ is non-singular, then this ensures $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. Once the problem is in fixed-point form, the iteration proceeds as

$$\mathbf{x}_{k+1} = \mathbf{g}(\mathbf{x}_k), \quad k = 0, 1, 2, \dots \quad (3.6.4)$$

If this sequence converges, i.e., $\mathbf{x}_k \rightarrow \mathbf{x}^*$ as $k \rightarrow \infty$, and if $\mathbf{g}$ is continuous in a neighborhood of $\mathbf{x}^*$, then taking the limit on both sides gives

$$\lim_{k \rightarrow \infty} \mathbf{x}_{k+1} = \lim_{k \rightarrow \infty} \mathbf{g}(\mathbf{x}_k) \implies \mathbf{x}^* = \mathbf{g}(\mathbf{x}^*) \quad (3.6.5)$$

Thus, the limit, if it exists, is indeed a fixed point of $\mathbf{g}$ and, by construction, a solution to $\mathbf{f}(\mathbf{x}) = \mathbf{0}$.

3.6.1 Stopping Criteria

In practice, iterative methods cannot run indefinitely. We need criteria to decide when an iterate $\mathbf{x}_k$ is “close enough” to the true solution $\mathbf{x}^*$ to stop the process. Given finite machine precision, it is not reasonable to expect that we can (and often not possible to) perfectly achieve $\mathbf{f} = \mathbf{0}$. Since $\mathbf{x}^*$ is generally unknown – it’s what we are trying to find, after all – these criteria are based on observable quantities. Two common stopping criteria are as follows:

1. **Function Norm (Residual) Test²:** This criterion checks if the function value $\mathbf{f}(\mathbf{x}_{k+1})$ is close to zero.

$$\|\mathbf{f}(\mathbf{x}_{k+1})\|_p \leq \varepsilon_f \quad (3.6.6)$$

Here, $\|\cdot\|_p$ denotes a chosen vector p -norm (e.g., $p = 1, 2, \infty$), and $\varepsilon_f > 0$ is a prescribed absolute tolerance for the residual. A small residual means that $\mathbf{x}_{k+1}$ nearly satisfies the system of equations.

2. **Step Size (Iterate Difference) Test:** This criterion checks if the change between successive iterates is small.

$$\|\mathbf{x}_{k+1} - \mathbf{x}_k\|_p \leq \varepsilon_R \|\mathbf{x}_{k+1}\|_p + \varepsilon_A \quad (3.6.7)$$

Here, $\varepsilon_R \geq 0$ is a relative tolerance and $\varepsilon_A \geq 0$ is an absolute tolerance. The relative tolerance $\varepsilon_R \|\mathbf{x}_{k+1}\|_p$ is useful when the magnitude of $\mathbf{x}_{k+1}$ is large, while the absolute tolerance ε_A is important when $\mathbf{x}_{k+1}$ is close to zero. A small difference between successive values of $\mathbf{x}$ means that $\mathbf{x}_{k+1}$ nearly satisfies the definition of a fixed point in the iterative map.

Warning: Scale Your Residuals. A practical complication in ChemE applications is that a single system of equations often mixes physically dissimilar equations—for instance, a mass balance (with residuals of order 1) and an energy balance (with residuals of order 10^4 if temperature is in Kelvin and enthalpy in Joules)—in the same vector $\mathbf{f}(\mathbf{x})$. An unscaled norm $\|\mathbf{f}(\mathbf{x})\|_p$ is then dominated entirely by whichever equation happens to have the largest characteristic magnitude, so a single tolerance ε_f can be effectively meaningless for the other equations. Practical codes address this by nondimensionalizing or scaling each residual component (e.g., dividing by a characteristic magnitude for that equation, or by its value $|f_i(\mathbf{x}_0)|$ at the initial guess) before applying a single tolerance.

Often, a combination of these tests (or others, like a maximum iteration count or detection of NaNs/infs) is used. (When implementing an iterative algorithm from scratch, it is always a good idea to impose a maximum iteration count and raise an error or warning if it is exceeded.)

3.6.1.1 Failure Modes of Stopping Criteria

While standard, and often the best possible choices given information available to the algorithm, these stopping criteria are far from foolproof and can sometimes indicate convergence prematurely or incorrectly, or fail to indicate convergence when it occurs slowly.

Residual Test Failure: A small residual $\|\mathbf{f}(\mathbf{x}_k)\|$ does not always imply that $\mathbf{x}_k$ is close to $\mathbf{x}^*$. If the function $\mathbf{f}$ is very “flat” near the root (i.e., its Jacobian is ill-conditioned or nearly singular), then many points $\mathbf{x}$ far from $\mathbf{x}^*$ can have small function values. Geometrically, $\mathbf{x}_k$ might be on a nearly level set of $\mathbf{f}$ close to zero, but this level set could be wide.

²We call this a residual test because in this case, the residual is by definition $\mathbf{R}_{k+1} = \mathbf{f}(\mathbf{x}_{k+1}) - \mathbf{0} = \mathbf{f}(\mathbf{x}_{k+1})$

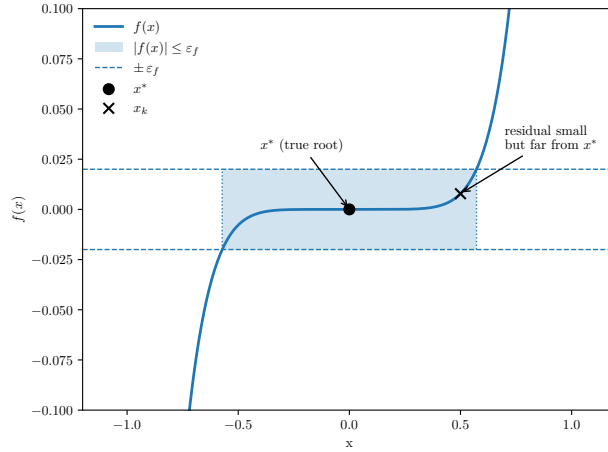


Figure 3.6.1: Failure of the residual test: A small residual $\|f(\mathbf{x}_k)\| \leq \varepsilon_f$ may occur even if $\mathbf{x}_k$ is far from the true root $\mathbf{x}^*$, especially if $\mathbf{f}$ is flat near $\mathbf{x}^*$.

Step-Size Test Failure: A small step size $\|\mathbf{x}_{k+1} - \mathbf{x}_k\|$ only means that the iteration has stalled or is converging very slowly. The iterates might be making negligible progress while still being far from any actual root, or they could be slowly converging in a narrow valley region of the function landscape.

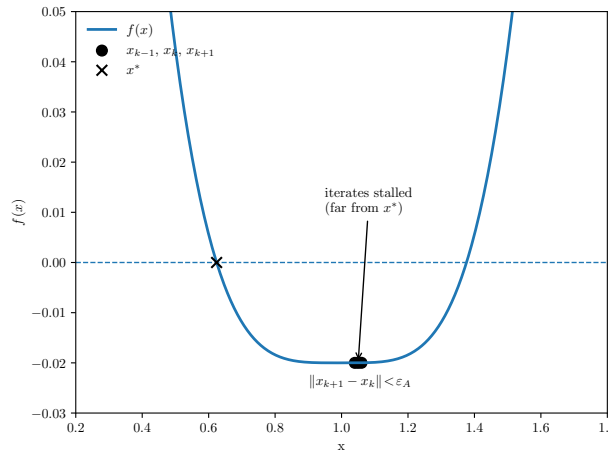


Figure 3.6.2: Failure of the step-size test: Successive iterates $\mathbf{x}_k$ and $\mathbf{x}_{k+1}$ may be very close, indicating stalled progress, even if they are far from the true root $\mathbf{x}^*$.

It is generally advisable to use a combination of these criteria and to have some understanding of the problem's characteristics such as the expected magnitudes of $\|f(\mathbf{x})\|$, $\|\mathbf{x}\|$, and the gradient.

3.6.2 Convergence Analysis

The speed at which an iterative method $\mathbf{x}_{k+1} = \mathbf{g}(\mathbf{x}_k)$ converges to a root $\mathbf{x}^*$ is an important characteristic of that method. Let $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}^*$ be the error at iteration k . The iteration is said to converge with order $q \geq 1$ if there exists a constant C (called the asymptotic error constant) with $0 < C < \infty$ such that

$$\lim_{k \rightarrow \infty} \frac{\|\mathbf{e}_{k+1}\|_p}{\|\mathbf{e}_k\|_p^q} = C \quad (3.6.8)$$

We can take a look at some common convergence orders. **Linear convergence** occurs if $q = 1$ and $0 < C < 1$. The error is reduced by a roughly constant factor at each step when close to the root. The number of correct significant digits increases linearly with k ; more precisely, digits gained per step $\approx -\log_{10} C$. For example, if $C \approx 0.1$, about one new correct digit is gained per iteration. Classical stationary methods like Jacobi and Gauss-Seidel for linear systems exhibit linear convergence under standard assumptions (e.g., when the iteration matrix has spectral radius < 1).

Superlinear convergence occurs if $q > 1$. Equivalently, we can characterize it by

$$\lim_{k \rightarrow \infty} \frac{\|\mathbf{e}_{k+1}\|_p}{\|\mathbf{e}_k\|_p} = 0 \quad (3.6.9)$$

which corresponds informally to the $q = 1$ case with $C = 0$ (noting this lies outside the $0 < C < \infty$ definition above). This is faster than linear.

Quadratic convergence is a specific case within superlinear convergence when $q = 2$. The number of correct significant digits roughly doubles at each iteration, leading to very rapid convergence once the iterates are close to the root.

There are two important notes to add to the discussion of convergence analysis. First, in finite dimensions, the choice of p -norm does not affect the order q (all norms are equivalent), though the constant C may change. Second, the phrase “close to the root” was used very intentionally when describing convergence behavior above. Convergence analysis is an examination of the algorithm’s ability to get arbitrarily close to a root $\mathbf{x}^*$ that we assume to exist. The typical strategy for deriving convergence rates that we will see in [subsection 3.6.4](#) relies on the ability to treat higher-order terms of a Taylor expansion as dominated by lower-order terms.

3.6.3 The Newton-Raphson Method

The Newton-Raphson method, often just called Newton’s method, is one of the most powerful and widely used iterative algorithms for finding roots of nonlinear equations. Its strength lies in its typically fast convergence rate when the initial guess is sufficiently close to the root.

Newton’s method can be viewed as a specific instance of fixed-point iteration. As we will see, the iteration function $\mathbf{g}(\mathbf{x})$ is chosen based on a linear approximation of $\mathbf{f}(\mathbf{x})$ at the current iterate $\mathbf{x}_k$. Specifically, if we consider the general fixed-point construction $\mathbf{g}(\mathbf{x}) = \mathbf{x} + \mathbf{A}(\mathbf{x})\mathbf{f}(\mathbf{x})$, Newton’s method makes the choice $\mathbf{A}(\mathbf{x}_k) = -[\mathbf{J}(\mathbf{x}_k)]^{-1}$, where $\mathbf{J}(\mathbf{x}_k)$ is the Jacobian matrix of $\mathbf{f}$ evaluated at $\mathbf{x}_k$. This leads to the iteration:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\mathbf{J}(\mathbf{x}_k)]^{-1}\mathbf{f}(\mathbf{x}_k) \quad (3.6.10)$$

We can understand where this comes from quite easily by considering a first-order Taylor expansion of $\mathbf{f}(\mathbf{x})$ around the current iterate $\mathbf{x}_k$. For an increment $\Delta\mathbf{x} = \mathbf{x} - \mathbf{x}_k$, we have the expansion

$$\mathbf{f}(\mathbf{x}_k + \Delta\mathbf{x}) \approx \mathbf{f}(\mathbf{x}_k) + \mathbf{J}(\mathbf{x}_k)\Delta\mathbf{x} \quad (3.6.11)$$

where $\mathbf{J}(\mathbf{x}_k)$ is the Jacobian matrix (with elements $J_{ij}(\mathbf{x}_k) = \frac{\partial f_i}{\partial x_j}(\mathbf{x}_k)$ as always). Newton’s method proposes that the next iterate $\mathbf{x}_{k+1}$ should satisfy $\mathbf{f}(\mathbf{x}_{k+1}) = \mathbf{0}$ under this assumption of linearity. Let $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{d}_k$, where $\mathbf{d}_k$ is the step/correction/displacement. Substituting this into the linearized equation [Eq. 3.6.11](#), we get

$$\mathbf{0} \approx \mathbf{f}(\mathbf{x}_k) + \mathbf{J}(\mathbf{x}_k)\mathbf{d}_k \quad (3.6.12)$$

This can be rearranged into a system of linear equations for the unknown step $\mathbf{d}_k$:

$$\mathbf{J}(\mathbf{x}_k)\mathbf{d}_k = -\mathbf{f}(\mathbf{x}_k) \quad (3.6.13)$$

Assuming the Jacobian $\mathbf{J}(\mathbf{x}_k)$ is non-singular, we can write an analytical expression for $\mathbf{d}_k$ using the matrix inverse:

$$\mathbf{d}_k = -[\mathbf{J}(\mathbf{x}_k)]^{-1}\mathbf{f}(\mathbf{x}_k) \quad (3.6.14)$$

The next iterate is then found by updating the current one:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{d}_k = \mathbf{x}_k - [\mathbf{J}(\mathbf{x}_k)]^{-1}\mathbf{f}(\mathbf{x}_k) \quad (3.6.15)$$

The update formula [Eq. 3.6.15](#) fully defines the iterative map used by Newton-Raphson.

Warning: It might look like each iteration of Newton's method requires us to invert the Jacobian matrix! However, we very rarely compute the inverse $[\mathbf{J}(\mathbf{x}_k)]^{-1}$ explicitly. Instead, the linear system for $\mathbf{d}_k$ is solved numerically (using, for example, the techniques discussed in [section 2.6](#) and [section 2.8](#)) for better efficiency. Advanced solvers may also take advantage of sparsity in the Jacobian, if known, to reduce computational cost.

In summary, each step of Newton's method involves:

1. Evaluating the function $\mathbf{f}(\mathbf{x}_k)$ and the Jacobian matrix $\mathbf{J}(\mathbf{x}_k)$ at the current iterate $\mathbf{x}_k$.
2. Solving the linear system $\mathbf{J}(\mathbf{x}_k)\mathbf{d}_k = -\mathbf{f}(\mathbf{x}_k)$ for the step $\mathbf{d}_k$.
3. Updating the iterate: $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{d}_k$.

To apply this method in practice requires defining an initial guess $\mathbf{x}_0$ as well as convergence criteria.

Scalar Case (1D Newton's Method): For a single scalar function $f(x)$ (where $f : \mathbb{R} \rightarrow \mathbb{R}$), the derivation simplifies considerably. The Jacobian matrix $\mathbf{J}(\mathbf{x}_k)$ from the N-dimensional case becomes the scalar first derivative $f'(x_k)$. The Taylor expansion is

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k) \quad (3.6.16)$$

We want to find x_{k+1} such that this linear approximation is zero:

$$0 = f(x_k) + f'(x_k)(x_{k+1} - x_k) \quad (3.6.17)$$

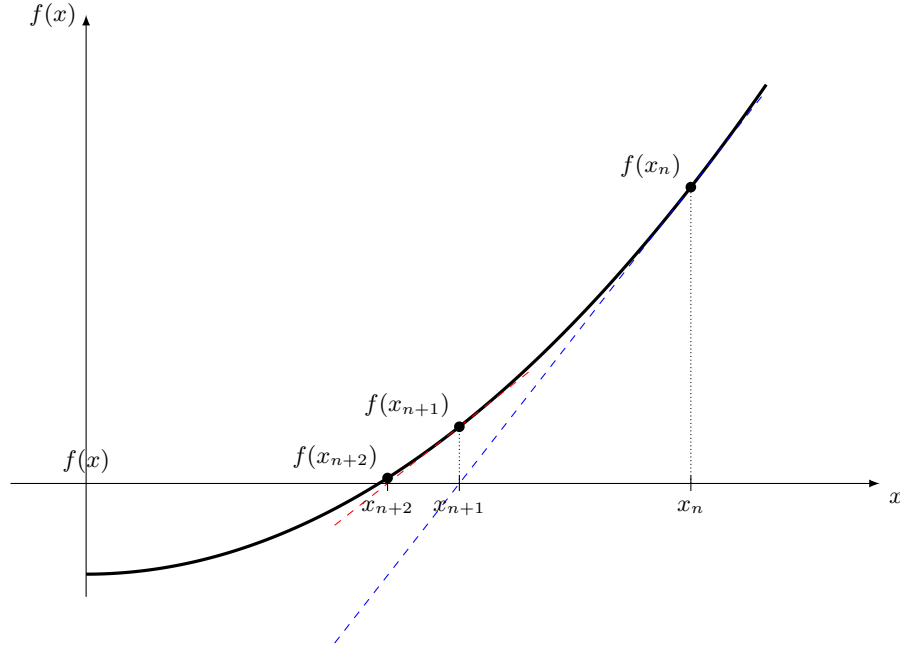
Solving for x_{k+1} , assuming $f'(x_k) \neq 0$,

$$f'(x_k)(x_{k+1} - x_k) = -f(x_k) \quad (3.6.18)$$

$$x_{k+1} - x_k = -\frac{f(x_k)}{f'(x_k)} \quad (3.6.19)$$

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad (3.6.20)$$

This should be quite familiar from previous calculus courses. Geometrically, x_{k+1} is the point where the tangent line to the graph of $f(x)$ at x_k intersects the x-axis ([Figure 3.6.3](#)).


 Figure 3.6.3: Visual depiction of Newton's method for a scalar function $f(x)$.

It is worth taking a step back and reminding oneself why a numerical method needs to make this kind of assumption of linearity. When we look at the 1D example in [Figure 3.6.3](#), it is easy to think that there must be a better strategy. Recall that we have made no assumptions about the structure of $\mathbf{f}(\mathbf{x})$ and that the algorithm only has the ability to call this function for different values of $\mathbf{x}$ in search of roots. At each iteration, Newton's method relies on a “local” analysis of the function value and its Jacobian to decide where it should search next. But why not also consider the second derivative? Or third derivative? Computing or estimating these can come at great computational cost, and we know that as we get closer to a root and take smaller and smaller steps, the linear term will dominate; however, algorithms using derivatives past the first derivative do exist (e.g., **Halley's method**, which uses f'' to achieve cubic convergence; in practice, though, computing or estimating a second derivative is itself costly, and the extra order of convergence rarely pays for itself outside special cases).

The Newton-Raphson algorithm for systems is summarized below using both the discussed stopping criteria simultaneously.

Algorithm 1 Newton's Method for Systems (robust variant)

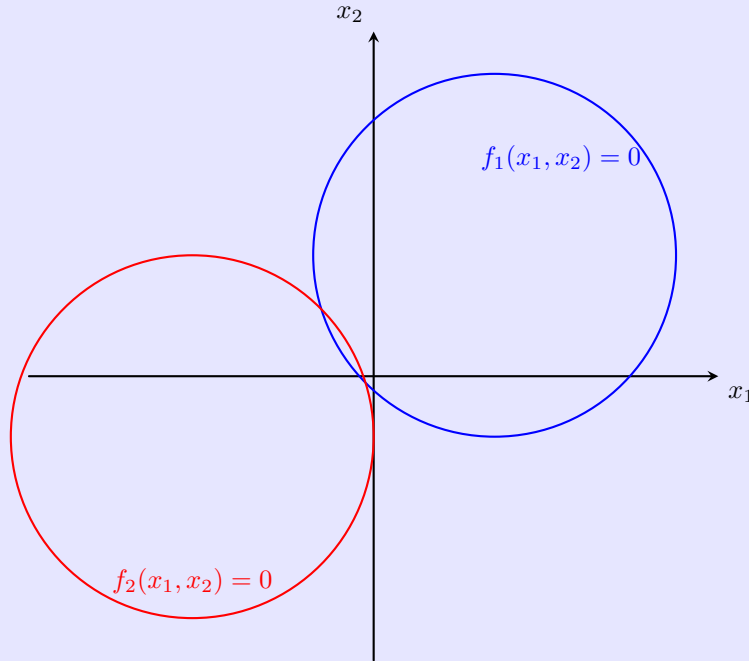
- 1: Choose $\mathbf{x}_0$, tolerances ε_f , ε_R , ε_A , and $K_{\max}$.
 - 2: **for** $k = 0, 1, 2, \dots, K_{\max} - 1$ **do**
 - 3: $\mathbf{f}_k \leftarrow \mathbf{f}(\mathbf{x}_k)$
 - 4: **if** $\|\mathbf{f}_k\|_p \leq \varepsilon_f$ **then**
 - 5: **return** $\mathbf{x}_k$ ▷ converged by residual
 - 6: $\mathbf{J}_k \leftarrow \mathbf{J}(\mathbf{x}_k)$
 - 7: Solve $\mathbf{J}_k \mathbf{d}_k = -\mathbf{f}_k$ for $\mathbf{d}_k$ (numerically)
 - 8: $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \mathbf{d}_k$
 - 9: **if** $\|\mathbf{d}_k\|_p \leq \varepsilon_R \|\mathbf{x}_{k+1}\|_p + \varepsilon_A$ **then**
 - 10: **return** $\mathbf{x}_{k+1}$ ▷ converged by step size
 - 11: **return** $\mathbf{x}_{k+1}$ ▷ or best of $\mathbf{x}_k, \mathbf{x}_{k+1}$ if available
-

Example: Intersection of Two Circles. Consider the problem of finding the intersection points of two circles:

$$f_1(x_1, x_2) = (x_1 - 2)^2 + (x_2 - 2)^2 - 9 = 0$$

$$f_2(x_1, x_2) = (x_1 + 3)^2 + (x_2 + 1)^2 - 9 = 0$$

Here, $\mathbf{x} = (x_1, x_2)^T$. The system is $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}))^T = \mathbf{0}$.



The Jacobian matrix $\mathbf{J}(\mathbf{x})$ is given by:

$$\mathbf{J}(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2(x_1 - 2) & 2(x_2 - 2) \\ 2(x_1 + 3) & 2(x_2 + 1) \end{bmatrix}$$

Starting with an initial guess, say $\mathbf{x}_0 = (-1, 3)^T$, we can apply Newton's method. The table below shows the first few iterations:

k	$\mathbf{x}^{(k)T}$	$\mathbf{f}(\mathbf{x}^{(k)})^T$	$\ \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\ _2$	$\ \mathbf{f}(\mathbf{x}^{(k)})\ _2$
0	(-1.000, 3.000)	(1.000, 11.000)	—	11.045
1	(-1.250, 1.750)	(1.625, 1.625)	1.275	2.298
2	(-0.963, 1.272)	(0.311, 0.311)	0.557	0.439
3	(-0.874, 1.124)	(0.030, 0.030)	0.173	0.042
4	(-0.864, 1.107)	(0.0004, 0.0004)	0.020	0.0006

The iteration rapidly converges to one of the intersection points, approximately $(-0.864, 1.106)$. The choice of initial guess determines which of the two intersection points is found.

3.6.4 Convergence of Newton's Method

One of the primary reasons for Newton's method's popularity is its typically **quadratic convergence rate**, provided certain conditions are met.

1D Proof of Quadratic Convergence: Let's analyze the convergence for the scalar case

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad (3.6.21)$$

assuming $f'(x^*) \neq 0$. The error at iteration $k+1$ is $e_{k+1} = x_{k+1} - x^*$. This means that

$$e_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} - x^* = (x_k - x^*) - \frac{f(x_k)}{f'(x_k)} = e_k - \frac{f(x_k)}{f'(x_k)} = \frac{e_k f'(x_k) - f(x_k)}{f'(x_k)} \quad (3.6.22)$$

Now, we consider the Taylor expansion of $f(x^*)$ around x_k :

$$f(x^*) = 0 = f(x_k) + f'(x_k)(x^* - x_k) + \frac{1}{2}f''(\xi_k)(x^* - x_k)^2 \quad (3.6.23)$$

for some ξ_k between x_k and x^* (this is coming from the Lagrange-remainder form of Taylor's theorem). Rearranging this and noting $x^* - x_k = -e_k$, we get

$$-f(x_k) = f'(x_k)(-e_k) + \frac{1}{2}f''(\xi_k)(-e_k)^2 \implies e_k f'(x_k) - f(x_k) = \frac{1}{2}f''(\xi_k)e_k^2 \quad (3.6.24)$$

Substituting this back into the expression for e_{k+1} , we get

$$e_{k+1} = \frac{\frac{1}{2}f''(\xi_k)e_k^2}{f'(x_k)} = \frac{f''(\xi_k)}{2f'(x_k)}e_k^2 \implies |e_{k+1}| = \left| \frac{f''(\xi_k)}{2f'(x_k)} \right| |e_k|^2 \quad (3.6.25)$$

As $x_k \rightarrow x^*$, $\xi_k \rightarrow x^*$. Thus, if $f'(x^*) \neq 0$,

$$\lim_{k \rightarrow \infty} \frac{|e_{k+1}|}{|e_k|^2} = \left| \frac{f''(x^*)}{2f'(x^*)} \right| \quad (3.6.26)$$

So we have quadratic convergence for the 1D case provided that f is twice continuously differentiable in a neighborhood of x^* , $f'(x^*) \neq 0$, and x_0 is sufficiently close to x^* .

Singular case (1D): When $f'(x^*) = 0$, the convergence order typically degrades to being linear. Proving this in the most general case is challenging, as there are different reasons why this occurs. Instead, consider the specific case where $f(x)$ has a root of multiplicity $m \geq 2$ at x^* . This means that $f(x)$ can be factored as

$$f(x) = (x - x^*)^m g(x) \quad (3.6.27)$$

where $g(x)$ is a function such that $g(x^*) \neq 0$. The derivative is

$$f'(x) = m(x - x^*)^{m-1}g(x) + (x - x^*)^m g'(x) \quad (3.6.28)$$

At $x = x^*$, we have $f'(x^*) = 0$ since the first term vanishes and the second term also vanishes because of the $(x - x^*)^m$ factor. Now, applying Newton's method:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{(x_k - x^*)^m g(x_k)}{m(x_k - x^*)^{m-1}g(x_k) + (x_k - x^*)^m g'(x_k)} \quad (3.6.29)$$

Assuming x_k is close to x^* , we can approximate $g(x_k) \approx g(x^*)$ and $g'(x_k) \approx g'(x^*)$. This gives

$$x_{k+1} \approx x_k - \frac{(x_k - x^*)^m g(x^*)}{m(x_k - x^*)^{m-1}g(x^*) + (x_k - x^*)^m g'(x^*)} \quad (3.6.30)$$

Factoring out $(x_k - x^*)^{m-1}$ from the denominator:

$$x_{k+1} \approx x_k - \frac{(x_k - x^*)^m g(x^*)}{(x_k - x^*)^{m-1} (mg(x^*) + (x_k - x^*)g'(x^*))} \quad (3.6.31)$$

Simplifying, we get

$$x_{k+1} \approx x_k - \frac{(x_k - x^*)g(x^*)}{mg(x^*) + (x_k - x^*)g'(x^*)} \quad (3.6.32)$$

As $x_k \rightarrow x^*$, the term $(x_k - x^*)g'(x^*)$ becomes negligible compared to $mg(x^*)$. Thus, we have

$$x_{k+1} \approx x_k - \frac{(x_k - x^*)g(x^*)}{mg(x^*)} = x_k - \frac{x_k - x^*}{m} = x_k \left(1 - \frac{1}{m}\right) + \frac{x^*}{m} \quad (3.6.33)$$

This shows that the error $e_k = x_k - x^*$ evolves as

$$e_{k+1} \approx \left(1 - \frac{1}{m}\right) e_k \quad (3.6.34)$$

This indicates linear convergence with a convergence factor of $1 - \frac{1}{m}$. Therefore, when $f'(x^*) = 0$ and the root has multiplicity $m \geq 2$, Newton's method converges linearly rather than quadratically. This ties directly back to the warning box earlier in the chapter: $f(x) = x^3$ has $x = 0$ as a root of multiplicity $m = 3$, which is simultaneously why $x = 0$ remains the unique root despite $f'(0) = 0$ and why Newton's method, started near $x = 0$, would only converge to it linearly (with factor $1 - \frac{1}{3} = \frac{2}{3}$) rather than quadratically.

If the multiplicity m is known (or can be estimated numerically, e.g. from the observed linear rate $1 - 1/m$), this degradation is straightforward to fix: the modified iteration

$$x_{k+1} = x_k - m \frac{f(x_k)}{f'(x_k)} \quad (3.6.35)$$

restores quadratic convergence, since rescaling by m exactly cancels the $1/m$ factor that otherwise survives in the error recursion above.

General Case (Multivariate): For systems of equations, Newton's method converges quadratically to $\mathbf{x}^*$ if:

1. $\mathbf{x}_0$ is sufficiently close to $\mathbf{x}^*$.
2. The Jacobian $\mathbf{J}(\mathbf{x}^*)$ is non-singular.
3. The components of $\mathbf{f}$ are twice continuously differentiable in a neighborhood of $\mathbf{x}^*$.

If $\mathbf{J}(\mathbf{x}^*)$ is singular, convergence typically degrades to at most linear or the method can fail. We omit the proof of these facts here; you don't need to know how to prove this for 10.34.

3.6.4.1 Local Convergence

The guarantee of quadratic convergence for Newton's method is *local*, meaning it only holds if the initial guess $\mathbf{x}_0$ is "sufficiently close" to the actual root $\mathbf{x}^*$. The set of starting points from which Newton's method converges to a particular root is called the *basin of attraction* (discussed more in Section 3.6.4.3) for that root. Good initial guesses are therefore essential for the success and efficiency of Newton's method. If the initial guess is poor, the sequence of iterates can behave erratically, diverge, or converge to an unintended root.

3.6.4.2 Failure Modes of Newton's Method

Newton's method can fail or perform poorly under certain conditions:

- **Singular or Ill-conditioned Jacobian:** If $f'(x_k) \approx 0$ (in 1D) or $\mathbf{J}(\mathbf{x}_k)$ is singular or nearly singular, the step $\mathbf{d}_k$ can become excessively large, potentially causing the next iterate to overshoot the root wildly, or the linear system for $\mathbf{d}_k$ may not have a unique solution. This often occurs near local minima/maxima of $f(x)$ or in regions where components of $\mathbf{f}$ are insensitive to changes in some variables. This is the same failure mode as depicted in Figure 3.6.4(a). In practice, damping/line-search or a trust-region variant (or using a pseudo-inverse when $\mathbf{J}(\mathbf{x}_k)$ is rank-deficient) mitigates these failures.

- **Cycling:** Iterates may enter a cycle, repeating a sequence of values without converging. For example, let $f(x) = x^3 - 2x + 2$, so $f'(x) = 3x^2 - 2$ and

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{x_k^3 - 2x_k + 2}{3x_k^2 - 2}$$

Starting at $x_0 = 0$,

$$x_1 = 1, \quad x_2 = 0 = x_0$$

so the iterates enter the cycle of $0 \leftrightarrow 1$ and never converge to a root.

- **Divergence to Infinity:** Iterates may move progressively further from any root. For example, take $f(x) = \arctan x$ (root at $x^* = 0$). Then $f'(x) = 1/(1+x^2)$ and

$$x_{k+1} = x_k - \frac{\arctan(x_k)}{1/(1+x_k^2)} = x_k - \arctan(x_k)(1+x_k^2)$$

With $x_0 = 2$,

$$x_1 \approx -3.5357, \quad x_2 \approx 13.9510, \quad x_3 \approx -279.3441, \quad x_4 \approx 1.22017 \times 10^5$$

so $|x_k| \rightarrow \infty$. For large $|x|$, $\arctan x \approx \pm \frac{\pi}{2}$, giving $x_{k+1} \approx x_k \mp \frac{\pi}{2}x_k^2$, which drives the blow-up. (This is the same failure mode as depicted in Figure 3.6.4(b).)

We visualize some of these failure modes in Figure 3.6.4.

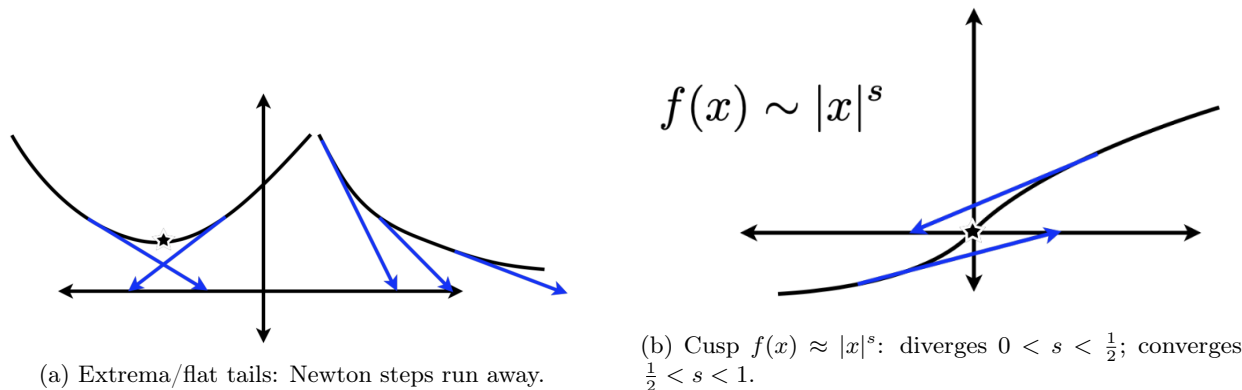


Figure 3.6.4: Newton's method failure modes (one Newton step shown in blue).

3.6.4.3 Basins of Attraction

The *basin of attraction* of a root $\mathbf{x}^*$ is the set of initial guesses $\mathbf{x}_0$ for which the iterates converge to $\mathbf{x}^*$. When multiple roots exist, these basins can have complicated shapes; points near the boundaries of different basins often exhibit slow or erratic convergence because the local linearization (through $\mathbf{J}$) is nearly singular or unstable there.

Example: Two Circles: Consider the two-circle system

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} (x_1 + 3)^2 + (x_2 + 1)^2 - 9 \\ (x_1 - 2)^2 + (x_2 - 2)^2 - 9 \end{bmatrix} = \mathbf{0},$$

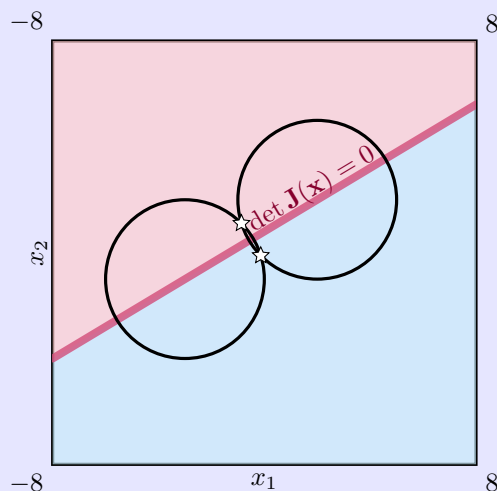
which has two intersection points (two roots). The Jacobian is

$$\mathbf{J}(\mathbf{x}) = \begin{bmatrix} 2(x_1 + 3) & 2(x_2 + 1) \\ 2(x_1 - 2) & 2(x_2 - 2) \end{bmatrix}, \quad \det \mathbf{J}(\mathbf{x}) = 4[(x_1 + 3)(x_2 - 2) - (x_2 + 1)(x_1 - 2)]$$

Expanding gives

$$\det \mathbf{J}(\mathbf{x}) = -12x_1 + 20x_2 - 16 \iff 3x_1 - 5x_2 + 4 = 0$$

This straight line is the *singular set*: on it, the linear system for $\mathbf{d}_k$ is singular, and near it the system is ill-conditioned. Consequently, initial guesses close to this line tend to require many Newton steps (or may fail unless a modification of Newton's method is used), whereas guesses well away from it converge rapidly to one of the two roots. The basin boundary between the two solutions usually tracks regions where $\det \mathbf{J}$ changes sign or becomes small.



For some problems, the basin boundaries are not smooth curves but intricate fractals. A well-known example is solving $f(z) = z^3 - 1 = 0$ in the complex plane. Newton's map

$$z_{k+1} = z_k - \frac{f(z_k)}{f'(z_k)} = z_k - \frac{z_k^3 - 1}{3z_k^2} = \frac{2z_k^3 + 1}{3z_k^2} \quad (3.6.36)$$

has three attracting fixed points (the cube roots of unity). Each fixed point has its own basin of attraction, but the boundaries between these three basins are fractal and highly sensitive: tiny changes in the initial guess can send the iterates to a different root.

Practical note. Near singular/ill-conditioned regions, methods such as damping/line search, trust-region steps, or continuation strategies can dramatically improve robustness, and plotting $\det \mathbf{J}$ (or its condition number) alongside iteration counts is a helpful diagnostic when exploring basins of attraction. These are discussed later in this chapter.

3.6.5 Damped Newton's Method

Newton's method can be sensitive to the choice of initial guess, and in some cases, the iterates can behave erratically or diverge, especially when far from a root (as discussed in [subsubsection 3.6.4.2](#)). To enhance robustness, a common modification is to introduce damping (or a line search) to control the step size. This approach is known as the **damped Newton method** or **Newton-Raphson with damping**.

Recall that Newton's method is based on a linear (first-order) approximation that is only locally accurate in a small neighborhood. We make two important observations:

- The *direction* of the Newton step, $\mathbf{d}_k = -\mathbf{J}(\mathbf{x}_k)^{-1}\mathbf{f}(\mathbf{x}_k)$, is typically good: if we took a sufficiently small step in this direction, the residual norm $\|\mathbf{f}(\mathbf{x}_k + \alpha\mathbf{d}_k)\|_p$ would decrease.
- The *magnitude* of the step can be problematic: the size $\|\mathbf{d}_k\|_p = \|\mathbf{J}(\mathbf{x}_k)^{-1}\mathbf{f}(\mathbf{x}_k)\|_p$ may be so large that the next iterate actually increases the residual (such that $\|\mathbf{f}(\mathbf{x}_{k+1})\|_p > \|\mathbf{f}(\mathbf{x}_k)\|_p$), which pushes us away from the solution instead of toward it.

Because our goal is to drive $\|\mathbf{f}(\mathbf{x})\|_p \rightarrow 0$, we can correct this undesirable behavior by *reducing the step size*. This leads to the damped Newton method:

Definition: Damped Newton Method. The damped Newton method is given by the update rule

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \mathbf{J}(\mathbf{x}_k)^{-1} \mathbf{f}(\mathbf{x}_k), \quad 0 < \alpha \leq 1 \quad (3.6.37)$$

Here, α is the damping parameter that controls the step size by reducing it.

1D intuition and the “right” amount of damping. In one dimension, the damped Newton method is

$$x_{k+1} = x_k - \alpha \frac{f(x_k)}{f'(x_k)} \quad (3.6.38)$$

Ideally, we would pick the α that minimizes the size of the residual after the step,

$$\alpha^* = \arg \min_{\alpha \in (0,1]} \left| f\left(x_k - \alpha \frac{f(x_k)}{f'(x_k)}\right) \right| \quad (3.6.39)$$

With such a choice, we guarantee $|f(x_{k+1})| < |f(x_k)|$ and avoid overshooting.

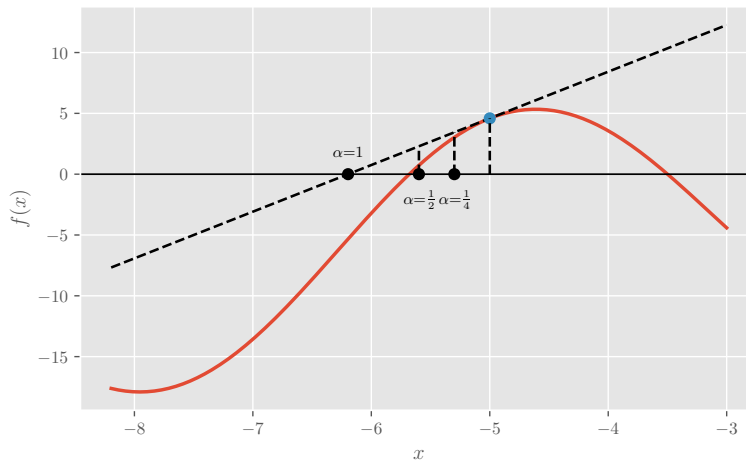


Figure 3.6.5: Damping in 1D prevents overshoot by shortening the Newton step along the same direction. We can see that here, $\alpha = 1$ overshoots, while $\alpha = 1/2$ and $\alpha = 1/4$ do not.

In N dimensions, this generalizes to

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \mathbf{J}(\mathbf{x}_k)^{-1} \mathbf{f}(\mathbf{x}_k), \quad \alpha^* = \arg \min_{0 < \alpha \leq 1} \left\| \mathbf{f}(\mathbf{x}_k - \alpha \mathbf{J}(\mathbf{x}_k)^{-1} \mathbf{f}(\mathbf{x}_k)) \right\|_p \quad (3.6.40)$$

Finding the optimal α^* can be just as hard (or harder) than solving the original system of nonlinear equations. As a more practical solution, we typically use an inexpensive backtracking **line search** that merely ensures the residual decreases. An example algorithm that implements this is shown below.

Algorithm 2 Backtracking line search (damped Newton).

1. Compute the Newton direction: $\mathbf{d}_k = -\mathbf{J}(\mathbf{x}_k)^{-1} \mathbf{f}(\mathbf{x}_k)$.
 2. Set $\alpha \leftarrow 1$ (attempt the full Newton step).
 3. While $\|\mathbf{f}(\mathbf{x}_k + \alpha \mathbf{d}_k)\|_p > \|\mathbf{f}(\mathbf{x}_k)\|_p$:
 - Set $\alpha \leftarrow \alpha/2$ (backtrack)
 4. Accept the step: $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \mathbf{d}_k$.
-

This strategy ensures that every iteration decreases $\|\mathbf{f}\|_p$ and avoids catastrophic overshooting. We visualize an example of the backtracking line search in Figure 3.6.6.

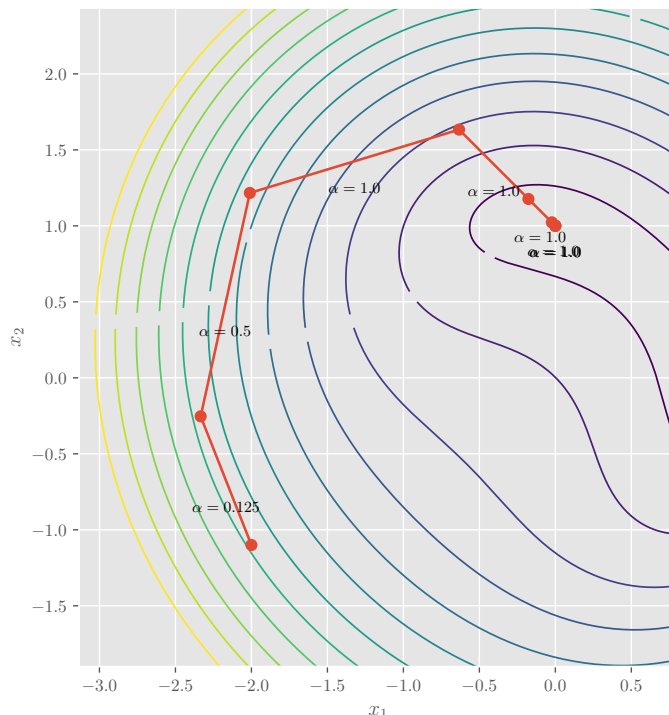


Figure 3.6.6: Backtracking line search for damped Newton on $\mathbf{f}(x_1, x_2) = [x_1 + x_2 - 1, x_1^2 + x_2^2 - 1]$. Contours show level sets of $\|\mathbf{f}(\mathbf{x})\|_2$, and dots trace the iterates from $\mathbf{x}_0 = (-2, -1.1)$. The full step ($\alpha = 1$) is initially rejected; halving yields the first accepted step at $\alpha = \frac{1}{8}$. Subsequent iterations accept $\alpha = \frac{1}{2}$ and then $\alpha = 1$ as the iterates approach a root, with the residual decreasing monotonically as guaranteed by the backtracking rule.

Convergence properties of damped Newton. Once the iterates enter a small neighborhood of a root, the full Newton step ($\alpha = 1$) is eventually accepted and quadratic convergence is recovered, provided that the Jacobian is nonsingular at the root. With backtracking, the method is globally convergent to a stationary point of $\frac{1}{2}\|\mathbf{f}\|_2^2$ (if we use the 2-norm). Thus, it may converge to a root or to a nonroot stationary point (local minimum or saddle point) of the residual norm.

Looking ahead, other tweaks (trust regions, inexact Newton, etc.) can further improve reliability and are closely connected to optimization methods. We will revisit them in that context.

3.6.6 Calculating the Jacobian

A major challenge in the practical execution of Newton's method comes from its reliance on the Jacobian matrix at each iteration and solving the associated linear system. We have already discussed various methods for the latter. For the former, there are several options to consider.

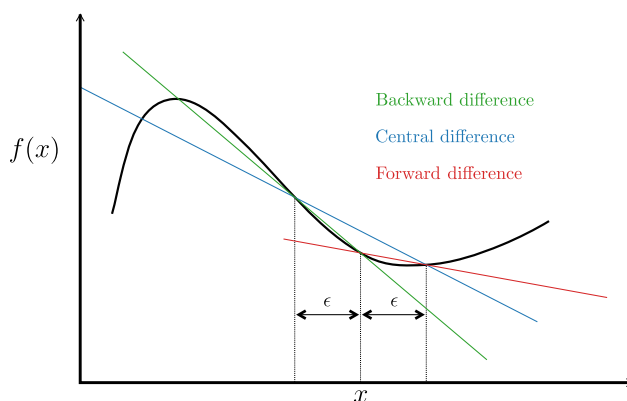
Ideally, an analytical form of the Jacobian would be available alongside the function $\mathbf{f}$ itself. For complex systems, deriving an analytical expression for $\mathbf{J}$ can be difficult and error-prone. This includes functions that despite having a continuous first derivative, can contain control flow statements (e.g., if statements, for loops, while loops) which mathematically result in highly complex branching conditionals. But rather than an analytical expression for the Jacobian, we can estimate it at a given value $\mathbf{x}_k$ using finite differences, or, when we have access to the function's source code, compute it automatically and exactly using automatic differentiation, discussed at the end of this section.

3.6.6.1 1D Finite Differences

When an analytical derivative is unavailable, one can approximate it with finite differences. With step size $\epsilon > 0$, the first derivative satisfies

$$\begin{aligned} f'(x) &\approx \frac{f(x+\epsilon) - f(x)}{\epsilon} && \text{(forward difference, truncation error } O(\epsilon)), \\ f'(x) &\approx \frac{f(x) - f(x-\epsilon)}{\epsilon} && \text{(backward difference, truncation error } O(\epsilon)), \\ f'(x) &\approx \frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon} && \text{(centered difference, truncation error } O(\epsilon^2)). \end{aligned}$$

The first two formulas should be intuitive based on the limit definition of the derivative. All three can be obtained, including their truncation error, as algebraic manipulations of truncated Taylor series. The centered difference formula is often preferred for accuracy, though it generally requires two additional function evaluations at $x \pm \epsilon$ even if $f(x)$ is already known. The tradeoff between computational cost and accuracy is one we continually face in this course. These difference schemes are visualized below.



The choice of ϵ is very important because two competing error sources balance against each other. If ϵ is too large, the truncation error from neglecting higher-order terms dominates; for forward/backward differences this scales like $O(\epsilon)$, while for the centered formula it scales like $O(\epsilon^2)$. If ϵ is too small, subtractive cancellation makes round-off error dominate when forming differences of nearly equal quantities; this round-off contribution is on the order of $O(\epsilon_M/\epsilon)$ for these quotients, where ϵ_M is the machine epsilon. Balancing these terms yields practical rules of thumb for the step size. For forward or backward differences one typically takes

$$\epsilon \approx \sqrt{\epsilon_M} \max(1, |x|)$$

while for the centered difference a common choice is

$$\epsilon \approx \epsilon_M^{1/3} \max(1, |x|)$$

Example: Finite differences and the U-shaped error trade-off. Consider estimating $f'(1)$ for $f(x) = e^x$ with the forward-difference quotient

$$D_\epsilon f(1) = \frac{f(1+\epsilon) - f(1)}{\epsilon}, \quad \epsilon > 0$$

Two effects control the accuracy as ϵ varies. The first is truncation error from the Taylor remainder (again, to be clear, we're using the Lagrange form of the remainder here):

$$f(1+\epsilon) = f(1) + \epsilon f'(1) + \frac{1}{2}\epsilon^2 f''(\xi_\epsilon), \quad \text{so} \quad D_\epsilon f(1) - f'(1) = \frac{1}{2}\epsilon f''(\xi_\epsilon),$$

which grows linearly like $O(\epsilon)$. The second is round-off error from subtracting nearly equal numbers in

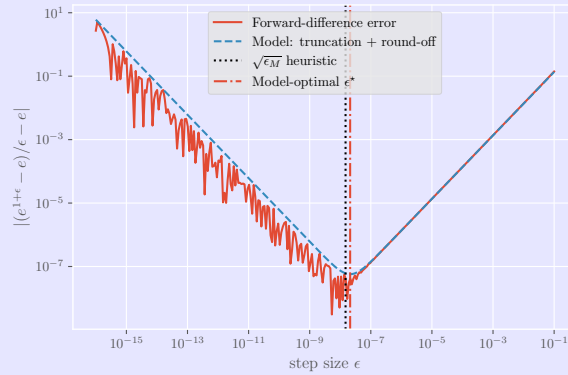
floating-point arithmetic, which behaves like $O(\epsilon_M/\epsilon)$. For $f(x) = e^x$ at $x = 1$ we therefore expect the total error to behave like

$$\underbrace{C_1 \epsilon}_{\text{truncation}} + \underbrace{C_2 \frac{\epsilon_M}{\epsilon}}_{\text{round-off}} \quad \text{with} \quad C_1 \approx \frac{1}{2}e, \quad C_2 \approx e,$$

which produces a characteristic U-shaped curve when plotted against ϵ on logarithmic axes. The minimum of this model occurs near

$$\epsilon^* \approx \sqrt{\frac{C_2}{C_1} \epsilon_M} \approx \sqrt{2\epsilon_M} \sim 10^{-8}$$

in close agreement with the practical rule of thumb $\epsilon \approx \sqrt{\epsilon_M} \max(1, |x|)$ for forward or backward differences. The figure below confirms this behavior numerically: as ϵ decreases from coarse values, the error falls (truncation dominates), then rises again for very small ϵ (round-off dominates).



3.6.6.2 ND Finite Differences

For $\mathbf{f} : \mathbb{R}^N \rightarrow \mathbb{R}^N$ with components f_i , the Jacobian entries $J_{ij}(\mathbf{x}) = \partial f_i / \partial x_j$ can be approximated by finite differences in a manner analogous to the 1D case. A column-wise forward difference forms the j th column of $\mathbf{J}(\mathbf{x})$ by

$$\mathbf{J}(\mathbf{x})\mathbf{e}_j \approx \frac{\mathbf{f}(\mathbf{x} + \epsilon_j \mathbf{e}_j) - \mathbf{f}(\mathbf{x})}{\epsilon_j}, \quad j = 1, \dots, N, \quad (3.6.41)$$

where $\mathbf{e}_j$ is the j th standard basis vector and ϵ_j is a componentwise step. That is, we only perturb the j^{th} index of $\mathbf{x}$ each time in order to isolate the impact of x_j on the function outputs. A scale-aware and numerically stable choice, as above, is

$$\epsilon_j \approx \sqrt{\epsilon_M} \max(1, |x_j|), \quad (3.6.42)$$

Constructing the full $N \times N$ Jacobian this way requires one evaluation of $\mathbf{f}(\mathbf{x})$ and then one additional evaluation of $\mathbf{f}$ per column for a total of N extra function evaluations. Using centered differences improves accuracy but roughly doubles this columnwise cost to $2N$. When $\mathbf{J}$ is sparse, advanced algorithms can use techniques to group independent columns and cut the number of function evaluations substantially.

When we use a finite-difference Jacobian inside Newton's method, we expect worse convergence than the quadratic rate that Newton's method achieves with analytical precision. Whether we observe linear or superlinear rates close to quadratic depends on the error incurred by the finite difference approximation.

Example (MATLAB): Below is an example MATLAB implementation of using column-wise forward differences to approximate the Jacobian:

```

function J = my_jacobian(x, my_func)
% Column-wise forward-difference Jacobian with scale-aware steps.
% x      : n-by-1 vector
% my_func : function handle returning n-by-1 vector f(x)

    n = length(x);
    J = zeros(n, n);

    Fx = my_func(x);

    epsM = eps;           % machine epsilon, defining for clarity
    h_base = sqrt(epsM);  % heuristic step magnitude

    for j = 1:n           % loop over columns
        x_pert = x;
        hj = h_base * max(1, abs(x(j)));
        x_pert(j) = x_pert(j) + hj;

        F_pert = my_func(x_pert);
        J(:, j) = (F_pert - Fx) / hj;
    end
end

```

3.6.6.3 Automatic Differentiation

Symbolic differentiation is a third option for obtaining $\mathbf{J}$: apply the rules of calculus (the chain rule, and the known derivatives of elementary functions like $\sin$, $\exp$, powers, etc.) directly to the algebraic expression for $\mathbf{f}$, exactly as you would by hand. Software like MATLAB's Symbolic Math Toolbox, Mathematica, or SymPy automates this. The result is an exact, closed-form expression for each derivative, which can then be evaluated to machine precision at any point without truncation error, unlike finite differences.

Symbolic differentiation runs into two practical problems, though. First, repeated application of the chain rule can cause *expression swell*: the symbolic derivative of even a moderately complicated function can be dramatically larger, and more expensive to evaluate, than the function itself. Second, and more fundamentally, many of the functions we actually need to differentiate aren't given as a single algebraic expression at all. We might be evaluating complex computer programs that contain control flow (`if` statements, `for/while` loops) that symbolic differentiation has no clean way to handle, since the sequence of operations actually executed can itself depend on the input.

Automatic differentiation (AD) sidesteps both problems. Instead of manipulating a symbolic expression, AD works directly with the sequence of elementary operations (addition, multiplication, $\sin$, $\exp$, ...) that the computer actually executes to evaluate $\mathbf{f}$ at a specific input. Because every elementary operation has a simple, known derivative, the chain rule can be applied mechanically along the trace to obtain an exact derivative *value* at that input, again to machine precision, without ever constructing (or growing) a symbolic expression. Since AD only ever operates on the trace of what was actually executed, it also handles control flow without requiring any additional considerations. An `if` statement or a loop that runs a different number of times for a different input is simply differentiated along whichever computational trace that particular input happened to produce.

There are two basic ways to propagate derivative information through the trace: *forward mode* and *reverse mode*.

Example: A computational graph. Consider $z = f(x_1, x_2) = x_1^2 + 3x_1x_2 + 1$. We can decompose this into a sequence of elementary operations, introducing an intermediate variable for each one:

$$v_1 = 3x_1, \quad v_2 = x_1^2, \quad v_3 = v_1x_2, \quad v_4 = v_2 + v_3, \quad z = v_4 + 1.$$

Evaluating this sequence at $(x_1, x_2) = (1, 2)$ gives $v_1 = 3$, $v_2 = 1$, $v_3 = 6$, $v_4 = 7$, $z = 8$.

Forward mode propagates a *tangent* $\dot{v} = \partial v / \partial x_1$ alongside every intermediate variable, seeding $\dot{x}_1 = 1$, $\dot{x}_2 = 0$, and applying the chain rule to each elementary operation in the same order they were originally evaluated:

$$\dot{v}_1 = 3\dot{x}_1, \quad \dot{v}_2 = 2x_1\dot{x}_1, \quad \dot{v}_3 = \dot{v}_1x_2 + v_1\dot{x}_2, \quad \dot{v}_4 = \dot{v}_2 + \dot{v}_3, \quad \dot{z} = \dot{v}_4.$$

At $(x_1, x_2) = (1, 2)$ this gives $\dot{v}_1 = 3$, $\dot{v}_2 = 2$, $\dot{v}_3 = 6$, $\dot{v}_4 = 8$, $\dot{z} = 8$ —so $\partial z / \partial x_1 = 8$, matching the analytical result $2x_1 + 3x_2 = 8$. One forward pass, costing only a small constant multiple of evaluating f itself, gives one directional derivative—here, the partial derivative with respect to x_1 .

Reverse mode instead propagates an *adjoint* $\bar{v} = \partial z / \partial v$ backward through the graph, seeded at the output with $\bar{z} = 1$, accumulating at each node the derivative of the final output with respect to that node:

$$\bar{v}_4 = \bar{z}, \quad \bar{v}_3 = \bar{v}_4, \quad \bar{v}_2 = \bar{v}_4, \quad \bar{v}_1 = \bar{v}_3x_2, \quad \bar{x}_1 = 3\bar{v}_1 + 2x_1\bar{v}_2, \quad \bar{x}_2 = \bar{v}_3v_1.$$

At $(x_1, x_2) = (1, 2)$ this gives $\bar{v}_1 = 2$, $\bar{x}_1 = 8$, $\bar{x}_2 = 3$ —so *one* backward pass recovers *both* $\partial z / \partial x_1 = 8$ and $\partial z / \partial x_2 = 3$ simultaneously, again at a cost of only a small constant multiple of evaluating f once.

This example illustrates the asymmetry of the two directions. Forward mode costs one pass per *input*; reverse mode costs one pass per *output*. For the Jacobian of a general $\mathbf{f} : \mathbb{R}^N \rightarrow \mathbb{R}^M$, forward mode therefore requires $O(N)$ passes and reverse mode requires $O(M)$ passes. Forward mode will be more efficient when there are few inputs and many outputs, and reverse mode will be more efficient when there are many inputs and few outputs. For Newton's method on a square $N \times N$ system, $M = N$, so neither mode has an asymptotic advantage over the other (or, for that matter, over a finite-difference Jacobian, in terms of the number of passes required). The difference becomes meaningful in the common case of a single scalar output ($M = 1$) with many inputs; this is the situation in gradient-based optimization, where f is a scalar objective and N , the number of parameters, can be in the millions or billions, especially in modern deep learning models. Reverse mode gets the full gradient ∇f in one backward pass, independent of N . In the machine learning community, reverse-mode AD used to train large neural networks is given the special name of *backpropagation*.

3.6.7 Quasi-Newton Methods

Quasi-Newton methods are a class of numerical algorithms that mimic Newton's method but use estimates of $\mathbf{J}$ or its inverse. These estimates are qualitatively different from finite difference approximations to the Jacobian. They are most often constructed and updated on-the-fly by low-rank updates using the values of $\mathbf{f}(\mathbf{x})$ obtained as iterations proceed. While the computational cost per iteration is substantially reduced compared to finite difference approximations in terms of the number of function calls, the price typically paid is a reduction in the convergence rate from quadratic (for exact Newton under standard assumptions) to superlinear. Under poor conditions some schemes may be only linear, but under smoothness they often achieve superlinear behavior.

3.6.7.1 Broyden's Method

Broyden's method maintains an estimate $\mathbf{B}_k \approx \mathbf{J}(\mathbf{x}_k)$ and enforces the latest secant equation with a rank-1 correction:³

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{(\Delta \mathbf{f}_k - \mathbf{B}_k \mathbf{s}_k) \mathbf{s}_k^T}{\mathbf{s}_k^T \mathbf{s}_k}, \quad \mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k, \quad \Delta \mathbf{f}_k = \mathbf{f}(\mathbf{x}_{k+1}) - \mathbf{f}(\mathbf{x}_k) \quad (3.6.43)$$

At iteration k , we solve the linear system $\mathbf{B}_k \mathbf{d}_k = -\mathbf{f}(\mathbf{x}_k)$ and update $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{d}_k$, exactly like Newton's method. An initial $\mathbf{B}_0$ can be obtained from a finite-difference approximation to the Jacobian, after which the rank-one updates avoid repeated full differencing. Under standard smoothness and regularity assumptions, these methods are typically superlinear, and in practice they can greatly reduce per-iteration cost while retaining robust local convergence behavior. The real computational benefit of Broyden's method is its rank-1 structure. Because each update to $\mathbf{B}_k$ is an outer product, the Sherman-Morrison formula (recall the discussion of rank-1 updates in the linear algebra chapter) lets us update $\mathbf{B}_k^{-1}$ directly in $O(N^2)$ operations, instead of paying the $O(N^3)$ cost of factoring a fresh Jacobian from scratch at every iteration. The downside is that this same rank-1 update tends to fill in $\mathbf{B}_k$ densely even when the true Jacobian is sparse. We will revisit quasi-Newton methods in more detail in the optimization chapter.

3.7 Continuation and Homotopy Methods

Finding a good initial guess for Newton's method can be difficult, especially when a problem has multiple solutions. Where do starting points come from? And how can we be confident we've found *all* the solutions?⁴

A simple strategy for looking for multiple solutions is to try many random initial guesses and see what happens. This can work well for low-dimensional problems with a few solutions, but it becomes inefficient in high dimensions or when solutions are rare. More systematic approaches are thankfully available.

This section introduces a powerful family of techniques for systematically exploring a problem's solution landscape. The core idea is simple: instead of solving one hard problem, we solve a sequence of related, easier ones. This general strategy is called continuation. A homotopy provides a general recipe for creating that sequence of problems. Together, they allow us to trace entire families of solutions as parameters change, generate rational initial guesses for root-finders, and discover when new solutions appear or disappear at special points called bifurcations.

3.7.1 Continuation

Continuation turns a hard equation into a sequence of easy ones. Instead of directly solving $f(x) = 0$, we introduce a new parameter $\lambda \in \mathbb{R}$ and consider a family $F(x, \lambda) = 0$ so that:

1. the target problem appears at some $\lambda = \lambda_*$, and
2. there is at least one value $\lambda = \lambda_0$ for which a solution is known or trivial.

We then “walk” from λ_0 to λ_* in small steps, using the previously computed solution as the initial guess at the next step. The intuition here is simple: the solution we just found at the previous λ value is an excellent initial guess for the next λ . You can plug that guess into any root finder you already know. If the solver struggles at some step, just reduce the λ step size and try again.

Common uses include sweeping Reynolds or Péclet numbers in transport problems, temperature/pressure in phase equilibria, or rate constants in reaction networks. In fact, the multiphysics simulator COMSOL, which you will encounter later in the course, has a built-in continuation engine that you can use to explore solutions as different physical quantities vary.

³Note that we call this a rank-1 correction because we are simply adding to $\mathbf{B}_k$ an outer product between two vectors. Any matrix constructed as an outer product between two vectors is rank one.

⁴“All solutions” is generally unattainable without structure. For arbitrary nonlinear systems, it is impossible to decide or enumerate all solutions from finite samples alone.

Example: Continuation for $f(x) = x^3 - 2x + 1$.

Define the family

$$f_\lambda(x) = x^3 - 2\lambda x + 1, \quad \lambda \in [0, 1]$$

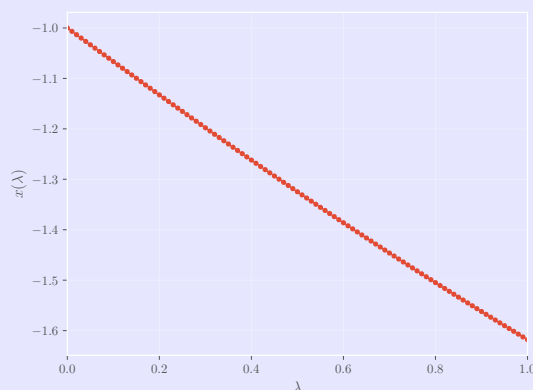
so that $f_0(x) = x^3 + 1$ and $f_1(x) = f(x)$.

At $\lambda = 0$ there is an obvious root $x = -1$. We now sweep λ from 0 to 1, carrying the last root forward as the next initial guess. In MATLAB-like pseudocode:

```
lambdas = 0:0.01:1;
xguess = -1;                                % solves f_0(x) = x^3 + 1 = 0

for i = 1:length(lambdas)
    f = @(x) x.^3 - 2*lambdas(i).*x + 1;
    x(i) = fzero(@(x) f(x), xguess);        % any 1D solver works
    xguess = x(i);                          % continuation: carry forward
end
```

This produces a discrete approximation of the solution path $x(\lambda)$ and, in particular, a root for $\lambda = 1$. If a step fails, reduce the increment in λ near that region and try again. Below, we visualize the solution path.



Optional extension (finding other roots). The polynomial at $\lambda = 1$ has three real roots. To trace other branches, rerun the same loop but start from a different initial value. One practical trick is to start at a larger λ (say $\lambda = 10$) where the balances $x \approx 1/(2\lambda)$ or $x \approx \pm\sqrt{2\lambda}$ give three distinct initial guesses, then sweep λ back down to 1.

3.7.2 Homotopy Methods

The basic premise of continuation should feel intuitive: take a harder problem and break it into a series of easier ones. After all, that premise also describes our approach to solving problems iteratively. But in practice, how do we choose the family of problems to solve $F(x, \lambda)$? A **homotopy** provides a recipe for this. In this context, a homotopy is a continuous deformation of one function into another. Given the original problem we want to solve, $\mathbf{f}(\mathbf{x}) = \mathbf{0}$, we construct a simple system $\mathbf{g}(\mathbf{x}) = \mathbf{0}$ that we know how to solve. We can then define the homotopy

$$\mathbf{h}(\mathbf{x}, \lambda) = \lambda \mathbf{f}(\mathbf{x}) + (1 - \lambda) \mathbf{g}(\mathbf{x}). \quad (3.7.1)$$

which smoothly interpolates between the two systems if we start at $\lambda = 0$ and traverse to $\lambda = 1$. At $\lambda = 0$, we have the easy start system $\mathbf{g}(\mathbf{x}) = \mathbf{0}$, and at $\lambda = 1$, we recover our target system $\mathbf{f}(\mathbf{x}) = \mathbf{0}$. In continuation and bifurcation analysis, we treat λ as a free real parameter; allowing $\lambda \notin [0, 1]$ is often necessary to systematically

track all solution branches, as we will see later.

The next question is how to choose $\mathbf{g}(\mathbf{x})$. The only real requirement is that $\mathbf{g}(\mathbf{x})$ has known roots—or at least lets us make sensible initial guesses to find those roots—so the choice is very flexible. The simplest ones include the following:

- Fixed-point: $\mathbf{g}(\mathbf{x}) = \mathbf{x} - \mathbf{x}_0$, where $\mathbf{x}_0$ is a known starting point.
- Linear: $\mathbf{g}(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}$, where you can easily solve the linear system for the starting roots.

Example: Fixed-point homotopy We consider the target system

$$\mathbf{f}(x, y) = \begin{bmatrix} x^2 - 1 \\ y^2 - 1 \end{bmatrix} = \mathbf{0}$$

Obviously, we can analytically solve this simple system to find the roots, but we will use this as an example to illustrate the homotopy method. For the sake of illustration, let's choose an arbitrary known point $(x_0, y_0) = (0.2, -0.5)$ and set

$$\mathbf{g}(x, y) = \begin{bmatrix} x - x_0 \\ y - y_0 \end{bmatrix}$$

which allows us to define the homotopy

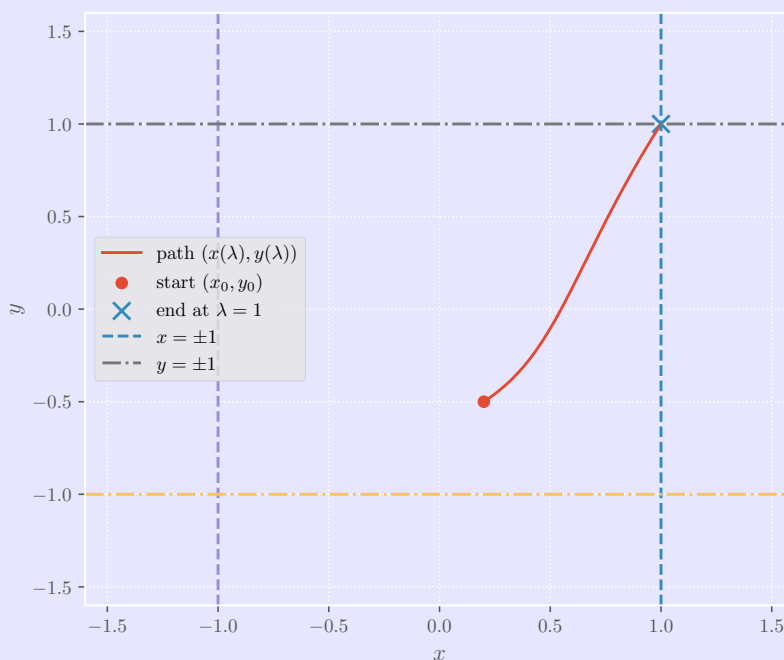
$$\mathbf{h}(x, y, \lambda) = \lambda \begin{bmatrix} x^2 - 1 \\ y^2 - 1 \end{bmatrix} + (1 - \lambda) \begin{bmatrix} x - x_0 \\ y - y_0 \end{bmatrix} = \mathbf{0}, \quad \lambda \in [0, 1]$$

At $\lambda = 0$, there is a single unique solution $(x, y) = (x_0, y_0)$ by construction. At each λ as we slowly increase from 0 to 1:

$$\lambda(x^2 - 1) + (1 - \lambda)(x - x_0) = 0 \implies \lambda x^2 + (1 - \lambda)x - [\lambda + (1 - \lambda)x_0] = 0.$$

$$\lambda(y^2 - 1) + (1 - \lambda)(y - y_0) = 0 \implies \lambda y^2 + (1 - \lambda)y - [\lambda + (1 - \lambda)y_0] = 0.$$

Following the root that satisfies $\mathbf{h}(x, y, \lambda) = \mathbf{0}$ starting from $(x, y) = (x_0, y_0)$ at $\lambda = 0$ as λ increases to 1 traces a continuous path from (x_0, y_0) to a solution of $\mathbf{f} = \mathbf{0}$. The figure shows the path in the (x, y) -plane together with the lines $x = \pm 1$ and $y = \pm 1$.



For chemical engineering problems describing physical systems, we often have some simplified model that we know how to solve, and we can use that to construct the homotopy. We will see this soon in the context of the van der Waals equation of state.

3.8 Bifurcation Analysis

As the homotopy parameter λ varies, the number of solutions of

$$\mathbf{h}(\mathbf{x}, \lambda) = \mathbf{0}$$

can change. Sometimes this change happens discontinuously as we change λ : a solution branch folds back in $(\mathbf{x}, \lambda)$ -space so that, when viewed as a function of λ , a solution suddenly appears or disappears. Such events are called **turning points**. At a turning point, the Jacobian with respect to $\mathbf{x}$ becomes singular, and the usual implicit function theorem parameterization by λ breaks down.

In other situations, additional solutions appear continuously: a solution branch smoothly sprouts extra branches (for example, a symmetric branch splitting into three). These are called **bifurcations**. They are likewise characterized by a singular Jacobian: the rank of $\mathbf{J}_h = \partial \mathbf{h} / \partial \mathbf{x}$ drops and new directions of solutions open up.

We can localize *where* such cases occur by identifying the points $(\mathbf{x}^*, \lambda^*)$ where $\mathbf{h}(\mathbf{x}^*, \lambda^*) = \mathbf{0}$ and the Jacobian is singular. So we can make the following definition:

Definition: Bifurcation Point. A point $(\mathbf{x}^*, \lambda^*)$ is a bifurcation point of a homotopy $\mathbf{h}(\mathbf{x}, \lambda) = \mathbf{0}$ if

1. $\mathbf{h}(\mathbf{x}^*, \lambda^*) = \mathbf{0}$ (it lies on the solution set), and
2. $\det \mathbf{J}_h(\mathbf{x}^*, \lambda^*) = 0$ (the Jacobian $\mathbf{J}_h = \partial \mathbf{h} / \partial \mathbf{x}$ is singular)

Near such points, solution branches either turn around in λ (turning point) or split/merge continuously (bifurcation). Practically, we can detect them by monitoring the smallest singular value of $\mathbf{J}_h$ along a continuation to determine when we might be close to a bifurcation point. We can then pinpoint them by solving an augmented system of equations that enforces $\mathbf{h} = 0$ together with singularity of $\mathbf{J}_h$:

$$\begin{bmatrix} \mathbf{h}(\mathbf{x}^*, \lambda^*) \\ \det \mathbf{J}_h(\mathbf{x}^*, \lambda^*) \end{bmatrix} = \mathbf{0} \quad (3.8.1)$$

This is itself a nonlinear system that can be solved using Newton's method!

Example: Ignition-Extinction in a Nonisothermal CSTR

The definition above tells us *where* to look for a qualitative change in the solution set ($\det \mathbf{J}_h = 0$), but not which kind we've found. Distinguishing a simple turning point from a true branch point requires one more piece of information: how the parameter enters the problem at that singular point. Let ϕ span the null space of $\mathbf{J}_h$ at a singular point $(\mathbf{x}^*, \lambda^*)$, and let ψ span the null space of $\mathbf{J}_h^\top$ (the "left" null vector). Generically, if $\psi^\top \frac{\partial \mathbf{h}}{\partial \lambda} \neq 0$ at that point—i.e., the parameter derivative is not orthogonal to the left null vector—the singularity is a simple turning point: the branch folds back in λ but does not split. If instead $\psi^\top \frac{\partial \mathbf{h}}{\partial \lambda} = 0$, the fold condition fails, and (generically) the point is a true branch point where distinct solution branches cross. We won't derive this rigorously in 10.34, but it's worth knowing that "singular Jacobian" by itself doesn't fully classify what's happening.

A classic chemical engineering illustration of a turning point is the multiplicity of steady states in a well-mixed, nonisothermal CSTR running an exothermic reaction. At steady state, the heat generated by reaction (roughly S-shaped in reactor temperature T , since it's governed by an Arrhenius rate that saturates once the reactant is depleted) must balance the heat removed by flow and cooling

(roughly linear in T). For an intermediate feed temperature (or, in dimensionless terms, an intermediate Damköhler number), the generation and removal curves can cross at three temperatures instead of one: a low-temperature “extinguished” steady state, an unstable intermediate one, and a high-temperature “ignited” one. Treating a parameter such as the feed temperature $\lambda = T_f$ as a continuation parameter and tracing the steady-state temperature $T(\lambda)$ produces exactly the kind of S-shaped curve we saw for the van der Waals equation above. Sweeping λ up from a low value, the reactor state jumps discontinuously from the extinguished branch to the ignited one once λ passes the *ignition point*; sweeping back down, it does not retrace the same path and instead jumps down at a lower *extinction point*, producing hysteresis. Both the ignition and extinction points are turning points. At each one, $\det \mathbf{J}_h = 0$ for $\mathbf{J}_h = \partial h / \partial T$ of the steady-state energy balance $h(T, \lambda) = 0$, and the (never physically observed) middle branch is the unstable solution connecting them.

Example: van der Waals equation via homotopy Suppose we want to calculate the molar volumes of all possible states given the van der Waals equation of state. As in the beginning of the chapter, we can pose this as finding the roots of a function $f(\hat{v})$ at a given reduced pressure $\hat{P}$ and reduced temperature $\hat{T}$. The van der Waals equation is given in reduced form by

$$f(\hat{v}) = \left(\hat{P} + \frac{3}{\hat{v}^2} \right) \left(\hat{v} - \frac{1}{3} \right) - \frac{8}{3} \hat{T}$$

A physically-grounded homotopy could start from a simple model with a known solution (ideal gas), then continuously deform to the more realistic model (van der Waals) and track the root(s). For $\hat{P} = 0.1$, $\hat{T} = 0.5$, the ideal gas law has a known solution for these parameters:

$$g(\hat{v}) = \hat{P}\hat{v} - \frac{8}{3}\hat{T} = 0 \implies \hat{v}_0 = \frac{8\hat{T}}{3\hat{P}} = \frac{4}{0.3} \approx 13.3$$

Now, we can define the homotopy between these two systems as

$$h(\hat{v}, \lambda) = \lambda f(\hat{v}) + (1 - \lambda)g(\hat{v}), \quad \lambda \in [0, 1]$$

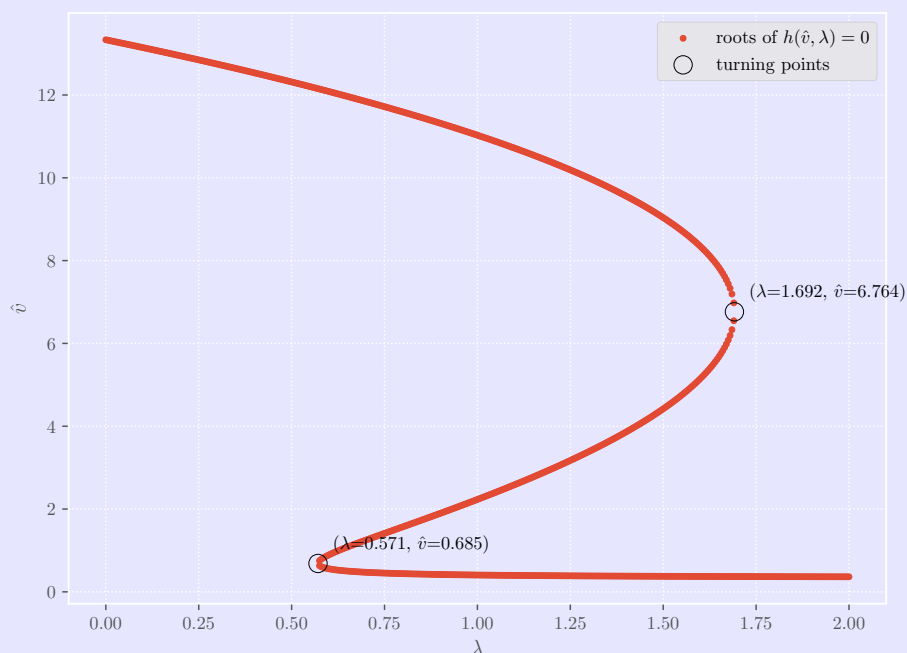
At $\lambda = 0$, the root is the ideal-gas volume $\hat{v}_0$. As λ increases, we track the solution(s) of $h(\hat{v}, \lambda) = 0$. A turning point for the homotopy is a pair $(\hat{v}^*, \lambda^*)$ with

$$h(\hat{v}^*, \lambda^*) = 0, \quad \frac{\partial h}{\partial \hat{v}}(\hat{v}^*, \lambda^*) = 0 \iff \det \mathbf{J}_h(\hat{v}^*, \lambda^*) = 0 \text{ (scalar case)}$$

Substituting the derivative into the augmented system and then solving numerically produces two turning points and, correspondingly, the familiar S-shaped set of roots in $(\lambda, \hat{v})$:

$$(\lambda_1^*, \hat{v}_1^*) \approx (0.571, 0.685), \quad (\lambda_2^*, \hat{v}_2^*) \approx (1.692, 6.764).$$

For $\lambda < \lambda_1^*$ there is a single positive root. As λ passes through λ_1^* , two additional roots appear and persist continuously up to λ_2^* , after which only one root remains. These points are highlighted in the figure below.

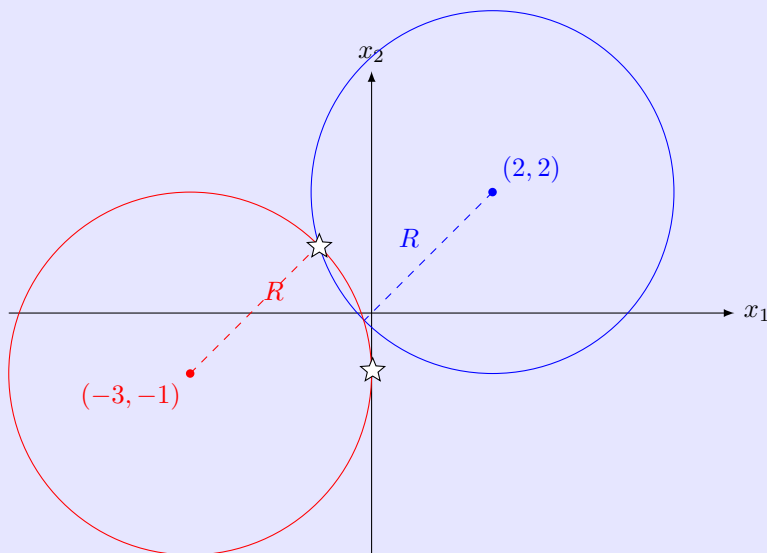


Note that $\lambda_2^* > 1$. This is possible (and common) with linear homotopies. Turning points can lie outside the interval $[0, 1]$ even if the problem we actually want to solve corresponds to $\lambda = 1$. If $\lambda = 0$ is the ideal gas law and $\lambda = 1$ is the van der Waals equation of state, then what might $\lambda = 2$ mean, physically? Fortunately, that is not something we need to worry about in the context of 10.34; we are simply using the homotopy as a mathematical tool.

Example: Circle intersection via homotopy Consider the problem of finding the intersection points of two circles in 2D. We can formulate this as solving the system of equations:

$$\mathbf{f}(\mathbf{x}) = \begin{pmatrix} (x_1 + 3)^2 + (x_2 + 1)^2 - R^2 \\ (x_1 - 2)^2 + (x_2 - 2)^2 - R^2 \end{pmatrix}$$

for two circles centered at $(-3, -1)$ and $(2, 2)$. As R varies, the number of intersection points changes. For small R , there are no intersections; at a critical radius, the circles touch at one point (a bifurcation); and for larger R , there are two intersection points.



The radius at which the circles *just* start to touch is a bifurcation point from 0 to 2 solutions. We can find this point by solving the augmented system of three equations for (x_1^*, x_2^*, R^*) :

$$\begin{pmatrix} \mathbf{f}(\mathbf{x}) \\ \det \mathbf{J}(\mathbf{x}) \end{pmatrix} = \begin{pmatrix} (x_1 + 3)^2 + (x_2 + 1)^2 - R^2 \\ (x_1 - 2)^2 + (x_2 - 2)^2 - R^2 \\ \det \begin{pmatrix} 2(x_1 + 3) & 2(x_2 + 1) \\ 2(x_1 - 2) & 2(x_2 - 2) \end{pmatrix} \end{pmatrix} = \mathbf{0}$$

3.9 Summary

This chapter has introduced several important concepts and techniques for solving systems of nonlinear equations (NLEs). We have emphasized Newton's method, a powerful iterative algorithm that uses the Jacobian matrix to find roots of nonlinear functions. We explored the convergence properties of Newton's method and highlighted practical considerations such as the choice of initial guess, convergence, obtaining the Jacobian, and the several variations and extensions of numerical methods that can help address these challenges.

3.10 Exercises

Exercise 3.1

Compute the Jacobian of $\mathbf{f}(\mathbf{x}) = \begin{pmatrix} \sin x_1 + x_2 e^{x_3} \\ x_1 x_2 x_3 \\ \cos x_3 + x_1 e^{-x_2} \end{pmatrix}$.

Exercise 3.2

(*) Linearize $\mathbf{f}(\mathbf{x}) = (\mathbf{x}^T \mathbf{A} \mathbf{x}) \mathbf{x}$, about $\mathbf{x} = \mathbf{x}_0$, for $\mathbf{x} \in \mathbb{R}^N$, $\mathbf{A} \in \mathbb{R}^{N \times N}$.

Chapter 4

Optimization

Optimization pervades chemical engineering. Engineers use it to estimate reaction rate constants from data, to design efficient processes, to determine optimal control actions, to calculate thermodynamic equilibrium by minimizing Gibbs free energy, to train machine learning models by adjusting their parameters for greater accuracy, and much more.

In any optimization problem, we define the variables that can be changed, specify the objective we want to achieve, and then search for the conditions that yield the best performance, either a maximum or a minimum. This chapter introduces the foundations of optimization and the numerical methods used to solve these problems.

4.1 Preliminaries: Gradients, Hessians, and Taylor Expansions

Gradients and Hessians Let's refresh our memory on some important concepts from calculus that will be used throughout this chapter. For a differentiable $f : \mathbb{R}^N \rightarrow \mathbb{R}$ and $\mathbf{x} = (x_1, \dots, x_N)^\top$, the gradient evaluated at $\mathbf{x}$ is

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \frac{\partial f}{\partial x_2}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_N}(\mathbf{x}) \end{bmatrix} \quad (4.1.1)$$

The Hessian evaluated at $\mathbf{x}$ is

$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1}(\mathbf{x}) & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_N}(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_N \partial x_1}(\mathbf{x}) & \cdots & \frac{\partial^2 f}{\partial x_N \partial x_N}(\mathbf{x}) \end{bmatrix} \quad (4.1.2)$$

For smooth functions, $\frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x}) = \frac{\partial^2}{\partial x_j \partial x_i} f(\mathbf{x})$, and thus, $\nabla^2 f(\mathbf{x})$ is symmetric. The Hessian can be viewed as a Jacobian of the gradient, i.e., $\nabla^2 f(\mathbf{x}) = \nabla(\nabla f(\mathbf{x}))$.

Taylor Expansions For univariate $f : \mathbb{R} \rightarrow \mathbb{R}$, if f has a continuous second derivative in a neighborhood of x^* , a first-order Taylor expansion at x^* is

$$f(x) = f(x^*) + f'(x^*)(x - x^*) + O(|x - x^*|^2) \quad (4.1.3)$$

and a second-order Taylor expansion (assuming a third derivative continuous in a neighborhood of x^*) at x^* is

$$f(x) = f(x^*) + f'(x^*)(x - x^*) + \frac{1}{2}f''(x^*)(x - x^*)^2 + O(|x - x^*|^3) \quad (4.1.4)$$

For multivariate $f : \mathbb{R}^N \rightarrow \mathbb{R}$, if f has a continuous second derivative in a neighborhood of $\mathbf{x}^*$, we similarly can write a first-order Taylor expansion at $\mathbf{x}^*$ as

$$f(\mathbf{x}) = f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*)^\top (\mathbf{x} - \mathbf{x}^*) + O(\|\mathbf{x} - \mathbf{x}^*\|^2) \quad (4.1.5)$$

and a second-order Taylor expansion (assuming a third derivative continuous in a neighborhood of $\mathbf{x}^*$) at $\mathbf{x}^*$ as

$$f(\mathbf{x}) = f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*)^\top (\mathbf{x} - \mathbf{x}^*) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^*)^\top \nabla^2 f(\mathbf{x}^*)(\mathbf{x} - \mathbf{x}^*) + O(\|\mathbf{x} - \mathbf{x}^*\|^3) \quad (4.1.6)$$

These expansions will be used repeatedly throughout this chapter.

4.2 Foundations of Optimization

4.2.1 The Optimization Problem

An optimization problem can be expressed in a standard form. For a minimization problem, this is

$$\min_{\mathbf{x} \in \mathcal{D}} f(\mathbf{x}) \quad (4.2.1)$$

Here, $f(\mathbf{x})$ is the **objective function** (also known as a cost, loss, or energy function) that we want to minimize. The vector $\mathbf{x}$ contains the **decision variables**. The set $\mathcal{D}$ is the **feasible set** or feasible region, which defines the space of permissible values for $\mathbf{x}$. We call a problem unconstrained when there are no explicit constraints on $\mathbf{x}$ beyond the natural domain of f (often $\mathbb{R}^N$, but not always, like in the case of $f(x) = \log(x)$, where the domain is $(0, \infty)$). If $\mathcal{D}$ is a proper subset of the domain of f (e.g., defined by equalities or inequalities, like $1 \leq x \leq 2$), the problem is **constrained**. It is worth noting that maximizing an objective function is equivalent to minimizing its negative; that is, $\max_{\mathbf{x} \in \mathcal{D}} f(\mathbf{x}) = -\min_{\mathbf{x} \in \mathcal{D}} (-f(\mathbf{x}))$. Similarly, the maximizer and minimizer coincide once we negate the objective: $\arg \max_{\mathbf{x} \in \mathcal{D}} f(\mathbf{x}) = \arg \min_{\mathbf{x} \in \mathcal{D}} (-f(\mathbf{x}))$. Thus, in this chapter, we will focus on minimization problems without loss of generality.

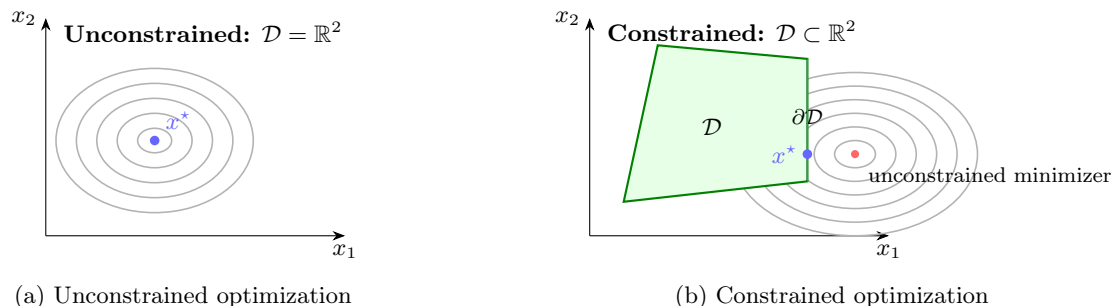


Figure 4.2.1: Objective level sets (thin gray) for $f(x_1, x_2)$. Left: the global minimizer x^* is the center of the contours when $\mathcal{D} = \mathbb{R}^2$. Right: when $\mathcal{D}$ restricts the search, the unconstrained minimizer (red) is infeasible and the constrained optimum x^* lies on $\partial\mathcal{D}$. Of course, we could also have a case where the unconstrained minimizer is feasible even in the constrained region so that the minimizers are the same in both the unconstrained and constrained cases, but that is quite rare.

For example, one can describe the feasible region as $\mathcal{D} = \{\mathbf{x} \in \mathbb{R}^N : c(\mathbf{x}) \geq 0\}$. A key difference from solving systems of nonlinear equations is that we retain *degrees of freedom* to improve the objective value. In other words, once the feasibility conditions are met, there are still directions in which the decision variables can move while maintaining feasibility. These free directions are what allow us to keep adjusting $\mathbf{x}$ to reduce

(or increase) the objective, whereas in equation solving the goal is only to satisfy all conditions, leaving no flexibility once a solution is found.

Example: Minimizing a quadratic function. Consider minimizing $f(x) = x^2 + 2x + 1$. Unconstrained, completing the square shows that the minimizer is $x = -1$. If we constrain x to lie in the interval $[0, 1]$, the unconstrained solution is infeasible and the unique minimizer moves to the boundary at $x = 0$.

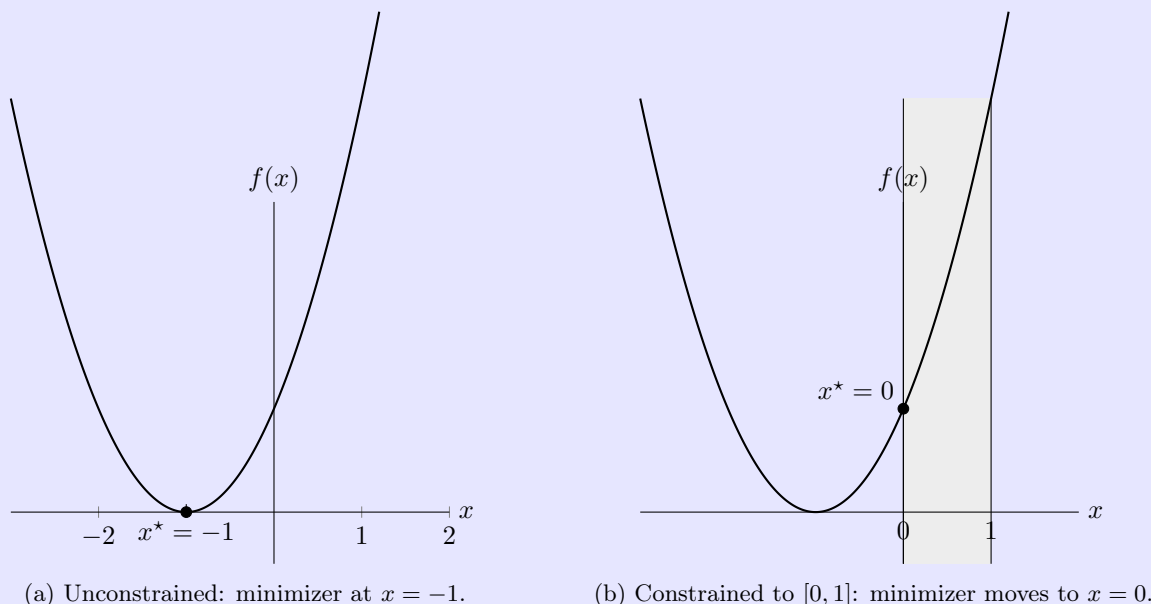


Figure 4.2.2: Minimizing $f(x) = x^2 + 2x + 1 = (x + 1)^2$ with and without constraints.

4.2.2 Classification of Optimization Problems

Numerical algorithms for optimization are designed to exploit problem structure. It is helpful to classify a problem along three axes: the nature of the decision variables, the presence of constraints, and the properties of the objective and constraints. Variables may be continuous, taking values in $\mathbb{R}^N$; discrete, restricted to $\mathbb{Z}^N$ (integers); or mixed, combining continuous components with integer ones. The feasible set $\mathcal{D}$ may span all of $\mathbb{R}^N$ (unconstrained) or be a proper subset defined by equalities and inequalities (constrained). These distinctions give rise to standard classes. Linear programming features linear objectives and linear constraints. Nonlinear programming allows nonlinearities with continuous variables. Mixed-integer linear programming adds integrality to linear models. Mixed-integer nonlinear programming combines integrality with nonlinearity. Recognizing where your problem lies in this landscape guides the algorithmic choices that follow.

Example: Computational Chemistry Planning. We plan a binding free-energy campaign using two calculation types:

- **Resources available:** 200 GPU-hours, 800 CPU-hours, 2 TB storage.
- **MD production (per ns):** uses 2 GPU-hours and 5 GB; yields 0.8 information points.
- **QM/MM snapshot (per calc):** uses 20 CPU-hours and 10 GB; yields 5 points.

Let $x_1 \in \mathbb{R}_{\geq 0}$ denote MD nanoseconds and $x_2 \in \mathbb{Z}_{\geq 0}$ the number of QM/MM snapshots. The objective is to maximize total information gain subject to resource limits:

$$\begin{aligned} \max_{x_1, x_2} \quad & 0.8x_1 + 5x_2 \\ \text{s.t.} \quad & 2x_1 \leq 200 && (\text{GPU-hours}) \\ & 20x_2 \leq 800 && (\text{CPU-hours}) \\ & 5x_1 + 10x_2 \leq 2000 && (\text{Storage, GB}) \\ & x_1 \geq 0, \quad x_2 \in \mathbb{Z}_{\geq 0} \end{aligned}$$

This is a mixed-integer linear program because the objective and constraints are linear, with x_2 constrained to be integer. *Continuous relaxation*: replacing $x_2 \in \mathbb{Z}_{\geq 0}$ by $x_2 \geq 0$ gives a linear program that can provide an upper bound and sometimes coincides with the MILP optimum. We will see this relaxation idea again in Section 4.6.1 when we discuss branch-and-bound methods.

4.2.3 Local vs. Global Solutions

The goal of an optimization algorithm is to find a solution, but the notion of solution varies. A point x^* is a **feasible solution** if $x^* \in \mathcal{D}$. A point x^* is a **local solution** if it is feasible and there exists $\epsilon > 0$ such that $f(x^*) \leq f(x)$ for all $x \in \mathcal{D} \cap B_\epsilon(x^*)$, where $B_\epsilon(x^*) := \{x : \|x - x^*\|_2 < \epsilon\}$ denotes the ball of radius ϵ around x^* (Figure 4.2.3). It is a **global solution** if the inequality holds for every feasible $x \in \mathcal{D}$. Finding global solutions can be difficult in practice, especially for nonconvex problems that feature many local minima (right side of Figure 4.2.4). For convex problems, as we discuss below, any local solution is also global. The solvers we use within this lecture, such as `fminunc` and `fmincon` in MATLAB, are local optimizers. To solve the problems to global optimality, it is necessary to use a specialized type of solver (so-called global optimizers), such as BARON.

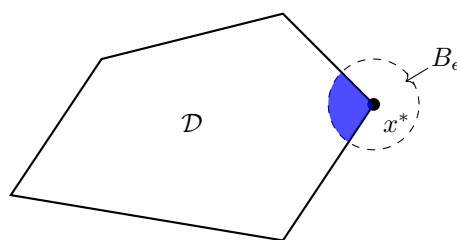


Figure 4.2.3: A local solution x^* lies in the feasible set $\mathcal{D}$. Within the neighborhood $B_\epsilon(x^*)$, $f(x^*) \leq f(x)$ for all feasible x .

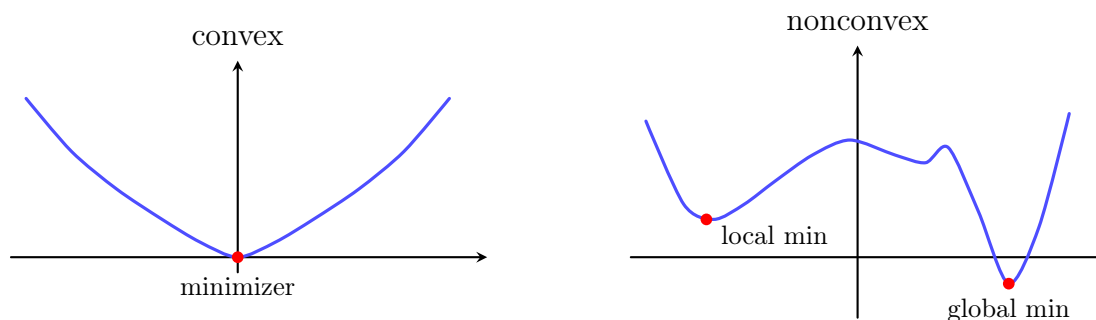


Figure 4.2.4: A convex function (left) and a nonconvex function (right). For convex functions, any local minimum is global; when the function is strictly convex, the minimizer is unique.

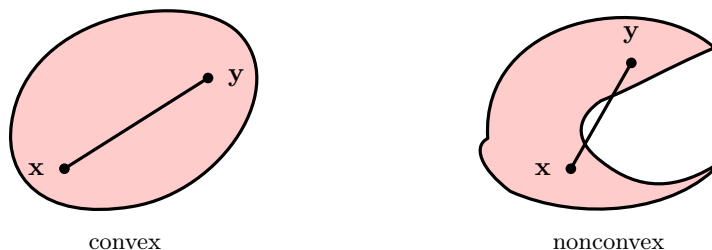
In a manner similar to nonlinear equations, existence and uniqueness are not guaranteed. For example, $\min_{x \in \mathbb{R}}(-x)$ has no minimizer (the objective is unbounded below), whereas $\min_{x \in \mathbb{R}}(0 \cdot x)$ has infinitely many minimizers. This kind of non-uniqueness shows up in practice whenever the data cannot distinguish between parameter combinations: fitting an Arrhenius expression $k = A \exp(-E_a/RT)$ to rate data over a narrow temperature range often produces a whole ridge of (A, E_a) pairs with nearly identical objective values.

4.2.4 Convexity

Before proceeding with our discussion of optimization, we need to define the concept of **convexity**. Convexity plays an important role in optimization because of the following key property: If the objective function and constraints are convex, it is guaranteed that any local minimum is also a global minimum. Thus, if we know that a problem is convex, we can use local optimization methods with confidence that they will find a globally optimal solution.

There are two related concepts: convex sets and convex functions.

Convex Sets: A set $\mathcal{D} \subseteq \mathbb{R}^N$ is convex if for all $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{D}$ and $t \in [0, 1]$, the point $t\mathbf{x}_1 + (1-t)\mathbf{x}_2$ also lies in $\mathcal{D}$. Geometrically, this means that the straight line segment connecting any two points in $\mathcal{D}$ lies entirely within $\mathcal{D}$.



Convex Functions: A function $f : \mathbb{R}^N \rightarrow \mathbb{R}$ is convex if for any two points $\mathbf{x}_1, \mathbf{x}_2$ in its domain and any $\lambda \in [0, 1]$:

$$f(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2) \quad (4.2.2)$$

Similarly, we call a function *strictly* convex if it is convex and the inequality Eq. 4.2.2 is strict for all $\mathbf{x}_1 \neq \mathbf{x}_2$ and $\lambda \in (0, 1)$.

The definitions above indicate that the function value at any point on the line segment between $\mathbf{x}_1$ and $\mathbf{x}_2$ is less than or equal to the linear interpolation of the function values at the endpoints. Geometrically, this means that a function is convex if, when you draw a line segment between any two points on the function's graph, the line segment lies above or on the function. For a univariate function, this means the function “curves upward” like a bowl. For a multivariate smooth function, this means that the function lies above the tangent plane at any point.

Second-Order Characterization: For twice-differentiable functions, convexity can be checked via curvature: in one dimension, $f''(x) \geq 0$ everywhere; in multiple dimensions, the Hessian satisfies $\nabla^2 f(\mathbf{x}) \succeq 0$ for all $\mathbf{x}$.

Examples of Convex and nonconvex Sets and Functions

Sets. You should use the line-segment test here for quick intuition. Below are some examples:

- $\{\mathbf{x} \in \mathbb{R}^N : \mathbf{Ax} = \mathbf{b}\}$ is convex (an affine subspace).
- $\{\mathbf{x} \in \mathbb{R}^N : -\mathbf{b} \leq \mathbf{Ax} \leq \mathbf{b}\}$ is convex (a polyhedron).
- $\{(x, y) \in \mathbb{R}^2 : y = e^x\}$ is *not* convex as a set: the midpoint of two points on the curve generally does not lie on the curve. (But note that by contrast, the *function* $f(x) = e^x$ is convex.)

Functions. Here are some examples of convex and nonconvex functions:

- $f(x) = 1$ is convex. Its second derivative is 0.
- $f(x) = -\log x$ on $(0, \infty)$ is convex since $f''(x) = 1/x^2 > 0$ for all $x > 0$.
- $f(\mathbf{x}) = \|\mathbf{x}\|_2^2$ is convex; its Hessian is $\nabla^2 f(\mathbf{x}) = 2\mathbf{I} \succeq \mathbf{0}$.
- $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x}$ is convex if and only if $\frac{1}{2}(\mathbf{A} + \mathbf{A}^\top) \succeq \mathbf{0}$. If $\mathbf{A}$ is symmetric, this reduces to $\mathbf{A} \succeq \mathbf{0}$.
- Sums preserve convexity: if f and g are convex on the same convex domain, then $f + g$ is convex.

Why Convexity Matters: Convexity confers powerful guarantees. Any local minimum is global, and many gradient-based algorithms have global convergence guarantees under standard conditions (e.g., continuous gradients and appropriate step sizes). Even when the underlying problem is nonconvex, convex relaxations and approximations are valuable for initialization, bounding, and analysis. Without convexity, finding a global solution is often NP-hard. In this case, specialized global optimization algorithms are then required.

Warning: Many chemical engineering optimization problems are nonconvex due to nonlinear thermodynamics, reaction kinetics, and phase behavior. While this makes global optimization challenging, understanding convex relaxations and convex approximations can still be valuable for developing good starting points and bounds for global optimization solvers.

4.3 Optimality Conditions for Unconstrained Problems

To find a local minimum, we need a way to identify one mathematically. We consider the unconstrained optimization problem for a function $f: \mathbb{R}^N \rightarrow \mathbb{R}$:

$$\min_{\mathbf{x} \in \mathbb{R}^N} f(\mathbf{x}) \quad (4.3.1)$$

If the objective function $f(\mathbf{x})$ is twice continuously differentiable, we have a set of optimality conditions based on the gradient and Hessian of the function. A point $\mathbf{x}^*$ that satisfies $\nabla f(\mathbf{x}^*) = \mathbf{0}$ is known as a **stationary point**.

First-Order Necessary Condition If $\mathbf{x}^*$ is a local minimum, then its gradient must be the zero vector:

$$\nabla f(\mathbf{x}^*) = \mathbf{0} \quad (4.3.2)$$

Second-Order Necessary Condition If $\mathbf{x}^*$ is a local minimum, then its Hessian matrix must be positive semi-definite:

$$\nabla^2 f(\mathbf{x}^*) \succeq \mathbf{0} \quad (4.3.3)$$

Second-Order Sufficient Condition If a point $\mathbf{x}^*$ satisfies both $\nabla f(\mathbf{x}^*) = \mathbf{0}$ and its Hessian matrix is positive definite, then $\mathbf{x}^*$ is a strict local minimum:

$$\nabla f(\mathbf{x}^*) = \mathbf{0} \quad \text{and} \quad \nabla^2 f(\mathbf{x}^*) \succ \mathbf{0} \implies \mathbf{x}^* \text{ is a strict local minimum} \quad (4.3.4)$$

These conditions are the foundation for the iterative algorithms we will discuss. They provide a target ($\nabla f(\mathbf{x}^*) = \mathbf{0}$) and a way to verify if we have found a minimum.

Together, these results delineate the landscape of stationary points. A stationary point $\mathbf{x}^*$ satisfies $\nabla f(\mathbf{x}^*) = \mathbf{0}$. Among stationary points, those with $\nabla^2 f(\mathbf{x}^*) \succeq \mathbf{0}$ satisfy the second-order necessity condition and are candidates for minima; if, in addition, $\nabla^2 f(\mathbf{x}^*) \succ \mathbf{0}$, the second-order sufficiency condition guarantees a strict local minimum.

Examples: Classifying Critical Points. Consider three simple functions on $\mathbb{R}^2$:

(a) Quadratic bowl: $f(\mathbf{x}) = x_1^2 + x_2^2$. The gradient is $\nabla f(\mathbf{x}) = (2x_1, 2x_2)^\top$, so the unique stationary point is $\mathbf{x}^* = (0, 0)^\top$. The Hessian is

$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \succ 0$$

which means that $\mathbf{x}^*$ is a strict (indeed global) minimizer.

(b) Saddle: $f(\mathbf{x}) = x_1^2 - x_2^2$. Again $\nabla f(\mathbf{x}) = (2x_1, -2x_2)^\top$ vanishes only at $\mathbf{x}^* = (0, 0)^\top$, but

$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix}$$

is indefinite. Hence, $\mathbf{x}^*$ is a saddle point, and f is unbounded below (and above).

(c) Flat minimum: $f(\mathbf{x}) = x_1^4 + x_2^4$. At $\mathbf{x}^* = (0, 0)^\top$, the Hessian $\nabla^2 f(\mathbf{x}^*) = \mathbf{0}$ is positive semidefinite, so the second-order sufficient condition does not apply; curvature alone does not guarantee optimality. (In fact, $\mathbf{x}^*$ is the strict global minimizer, but this requires a separate argument.)

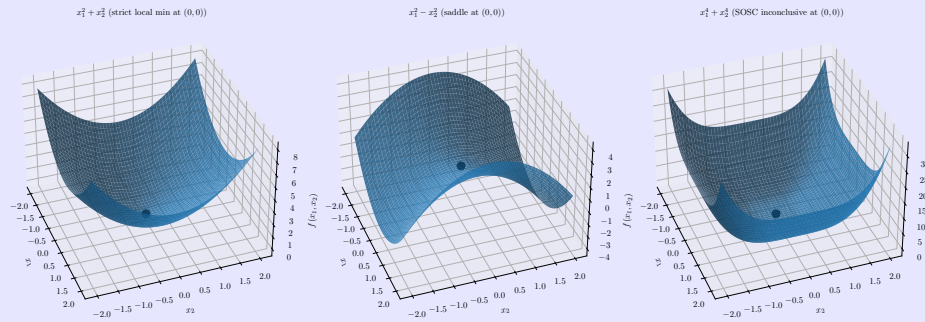


Figure 4.3.1: The three functions from the example with the critical points labeled (black).

Warning: A stationary point (where $\nabla f(\mathbf{x}^*) = 0$) is not necessarily a minimum. It could be a maximum or a saddle point. The second-order conditions using the Hessian are needed to distinguish between these cases. Optimization algorithms may converge to any stationary point, so verifying the nature of the solution is essential.

4.3.1 Proofs for Optimality Conditions

We now provide proofs for the optimality conditions. Don't worry too much about the proofs if you find them difficult to parse—they are really just here for completeness, hence the asterisks on the section titles—but definitely make sure you understand the intuition behind them (Section 4.3.1.4).

Throughout, we assume that $f : \mathbb{R}^N \rightarrow \mathbb{R}$ is twice continuously differentiable in a neighborhood of $\mathbf{x}^*$. We repeatedly use the second-order Taylor expansion with remainder:

$$f(\mathbf{x}^* + \epsilon \mathbf{d}) = f(\mathbf{x}^*) + \epsilon \nabla f(\mathbf{x}^*)^\top \mathbf{d} + \frac{1}{2} \epsilon^2 \mathbf{d}^\top \nabla^2 f(\mathbf{x}^*) \mathbf{d} + r(\epsilon, \mathbf{d}) \quad (4.3.5)$$

where the condition on the remainder $r(\epsilon, \mathbf{d})$ is that for every $\eta > 0$ there exists $\bar{\epsilon} > 0$ such that $|r(\epsilon, \mathbf{d})| \leq \eta \epsilon^2$ whenever $0 < \epsilon \leq \bar{\epsilon}$ and $\|\mathbf{d}\|_2 = 1$.

4.3.1.1 First-Order Necessity*

Claim. If $\mathbf{x}^*$ is a local minimizer of f , then $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

Proof. Assume by contradiction that $\nabla f(\mathbf{x}^*) \neq \mathbf{0}$. Let $\mathbf{d} := -\nabla f(\mathbf{x}^*)/\|\nabla f(\mathbf{x}^*)\|_2$ (the negative gradient direction). Then $\nabla f(\mathbf{x}^*)^\top \mathbf{d} = -\|\nabla f(\mathbf{x}^*)\|_2 < 0$. Using Eq. 4.3.5 and the bound on the remainder,

$$f(\mathbf{x}^* + \epsilon \mathbf{d}) = f(\mathbf{x}^*) + \epsilon \nabla f(\mathbf{x}^*)^\top \mathbf{d} + \frac{1}{2} \epsilon^2 \mathbf{d}^\top \nabla^2 f(\mathbf{x}^*) \mathbf{d} + r(\epsilon, \mathbf{d}) \quad (4.3.6)$$

For sufficiently small $\epsilon > 0$, the negative linear term $\epsilon \nabla f(\mathbf{x}^*)^\top \mathbf{d}$ dominates the $O(\epsilon^2)$ terms, hence $f(\mathbf{x}^* + \epsilon \mathbf{d}) < f(\mathbf{x}^*)$, contradicting local minimality. Therefore, $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

4.3.1.2 Second-Order Necessity*

Claim. If $\mathbf{x}^*$ is a local minimizer of f , then $\nabla^2 f(\mathbf{x}^*) \succeq \mathbf{0}$.

Proof. By the first order conditions, $\nabla f(\mathbf{x}^*) = \mathbf{0}$. Thus, the linear term in Eq. 4.3.5 vanishes and we have, for any unit direction $\mathbf{d}$,

$$f(\mathbf{x}^* + \epsilon \mathbf{d}) = f(\mathbf{x}^*) + \frac{1}{2} \epsilon^2 \mathbf{d}^\top \nabla^2 f(\mathbf{x}^*) \mathbf{d} + r(\epsilon, \mathbf{d}) \quad (4.3.7)$$

Suppose, toward a contradiction, that $\nabla^2 f(\mathbf{x}^*) \not\succeq \mathbf{0}$. Then there exists $\mathbf{d}$ with $\|\mathbf{d}\|_2 = 1$ such that $\mathbf{d}^\top \nabla^2 f(\mathbf{x}^*) \mathbf{d} < 0$ (e.g., an eigenvector corresponding to a negative eigenvalue). Choosing $\epsilon > 0$ small enough so that

$$|r(\epsilon, \mathbf{d})| \leq \frac{1}{4} \epsilon^2 |\mathbf{d}^\top \nabla^2 f(\mathbf{x}^*) \mathbf{d}| \quad (4.3.8)$$

we obtain

$$f(\mathbf{x}^* + \epsilon \mathbf{d}) \leq f(\mathbf{x}^*) + \frac{1}{2} \epsilon^2 \mathbf{d}^\top \nabla^2 f(\mathbf{x}^*) \mathbf{d} + \frac{1}{4} \epsilon^2 |\mathbf{d}^\top \nabla^2 f(\mathbf{x}^*) \mathbf{d}| < f(\mathbf{x}^*) \quad (4.3.9)$$

contradicting local minimality. Hence, $\nabla^2 f(\mathbf{x}^*) \succeq \mathbf{0}$.

4.3.1.3 Second-Order Sufficiency*

Claim. If $\nabla f(\mathbf{x}^*) = \mathbf{0}$ and $\nabla^2 f(\mathbf{x}^*) \succ \mathbf{0}$, then $\mathbf{x}^*$ is a *strict* local minimizer.

Proof. Let $\mathbf{H}^* := \nabla^2 f(\mathbf{x}^*) \succ \mathbf{0}$ with smallest eigenvalue $\lambda_{\min}(\mathbf{H}^*) > 0$, so

$$\mathbf{d}^\top \mathbf{H}^* \mathbf{d} \geq \lambda_{\min}(\mathbf{H}^*) \|\mathbf{d}\|_2^2 \quad \text{for all } \mathbf{d} \in \mathbb{R}^N \quad (4.3.10)$$

Using Eq. 4.3.5 with $\nabla f(\mathbf{x}^*) = \mathbf{0}$, for any unit direction $\mathbf{d}$ and $\epsilon > 0$,

$$f(\mathbf{x}^* + \epsilon \mathbf{d}) = f(\mathbf{x}^*) + \frac{1}{2} \epsilon^2 \mathbf{d}^\top \mathbf{H}^* \mathbf{d} + r(\epsilon, \mathbf{d}). \quad (4.3.11)$$

We fix $\eta \in (0, \frac{1}{2} \lambda_{\min}(\mathbf{H}^*))$. By the stated remainder bound, there exists $\bar{\epsilon} > 0$ such that $|r(\epsilon, \mathbf{d})| \leq \eta \epsilon^2$ for all unit $\mathbf{d}$ whenever $0 < \epsilon \leq \bar{\epsilon}$. Hence, for all unit $\mathbf{d}$ and $0 < \epsilon \leq \bar{\epsilon}$,

$$f(\mathbf{x}^* + \epsilon \mathbf{d}) \geq f(\mathbf{x}^*) + \left(\frac{1}{2} \lambda_{\min}(\mathbf{H}^*) - \eta \right) \epsilon^2 > f(\mathbf{x}^*) \quad (4.3.12)$$

To handle an arbitrary nonzero increment Δ with $\|\Delta\|_2 \leq \bar{\epsilon}$, we can write $\Delta = \epsilon \mathbf{d}$ with $\epsilon = \|\Delta\|_2$ and $\mathbf{d} = \Delta/\|\Delta\|_2$ (so $\|\mathbf{d}\|_2 = 1$); the previous inequality applies and gives $f(\mathbf{x}^* + \Delta) > f(\mathbf{x}^*)$. Thus, $\mathbf{x}^*$ is a strict local minimizer.

4.3.1.4 Intuition

The proofs above formalize the intuition behind the optimality conditions:

- If $\nabla f(\mathbf{x}^*) \neq \mathbf{0}$, a small step in the negative gradient direction strictly decreases the objective (first-order term dominates), contradicting local minimality.

- If $\nabla f(\mathbf{x}^*) = \mathbf{0}$ but $\nabla^2 f(\mathbf{x}^*) \not\succeq \mathbf{0}$, there exists a direction with negative curvature. Along that direction, the second-order term is negative and again decreases the objective.
- If $\nabla f(\mathbf{x}^*) = \mathbf{0}$ and $\nabla^2 f(\mathbf{x}^*) \succ \mathbf{0}$, then the quadratic term is positive in every direction and dominates the remainder term, so nearby points have strictly larger objective value.

4.4 Algorithms for Unconstrained Optimization

Most numerical methods for unconstrained optimization are iterative. They start from an initial guess, x_0 , and generate a sequence of points $\{x_k\}$ that converge to a local minimum. Indeed, we saw this before when we discussed fixed-point iterations for solving systems of linear equations (Jacobi and Gauss-Seidel iterations) and root-finding (Newton's method). The general form of such an iteration is

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k \quad (4.4.1)$$

where $\mathbf{d}_k$ is the **search direction** and $\alpha_k > 0$ is the **step size** (or step length). The primary difference between the methods discussed below lies in how they choose the search direction $\mathbf{d}_k$.

4.4.1 Steepest Descent

The **steepest descent** method, also called gradient descent and a first-order method, uses the most intuitive search direction: the one in which the objective function $f(\mathbf{x})$ decreases most rapidly. We consider a first-order Taylor expansion of $f(\mathbf{x})$ around $\mathbf{x}_k$:

$$f(\mathbf{x}_k + \alpha_k \mathbf{d}_k) \approx f(\mathbf{x}_k) + \alpha_k \nabla f(\mathbf{x}_k)^\top \mathbf{d}_k \quad (4.4.2)$$

For a fixed $\alpha_k > 0$, the linear term determines whether we go up or down. We decrease f as fast as possible by pointing $\mathbf{d}_k$ directly opposite the gradient because, among all unit directions, this minimizes $\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k$. This gives the steepest-descent direction

$$\mathbf{d}_k = -\nabla f(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k) \quad (4.4.3)$$

The step size α_k plays a very important role here. If the step is tiny, we faithfully follow the gradient but creep along level sets, so progress is slow, and the path looks like short arrows tangent to the valley walls (perpendicular to the level sets) as we inch toward the center. If the step is too large, we overshoot the valley, bounce across contours, and the iterates can zig-zag or even blow up. We show this in [Figure 4.4.1](#).

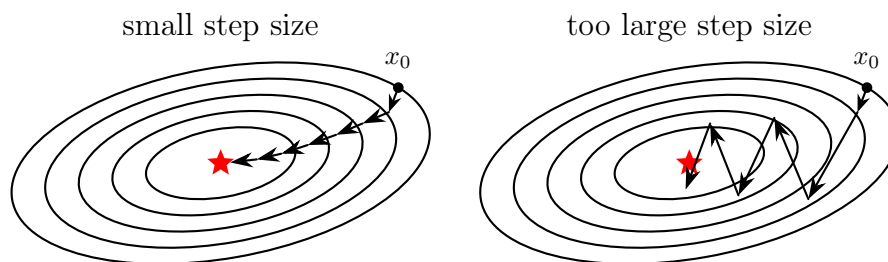


Figure 4.4.1: Steepest descent method. Left: small step size; right: too large step size.

4.4.1.1 Backtracking Line Search

How, then, should we pick α_k in practice? One way is via a backtracking line search. Start with a trial step (often $\alpha_k = 1$ or a value carried over from the previous iteration) and check whether the new point actually reduces the objective by a reasonable amount. A convenient “sufficient decrease” test is the Armijo condition with constant $1/2$:

$$f(\mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)) \leq f(\mathbf{x}_k) - \frac{1}{2} \alpha_k \|\nabla f(\mathbf{x}_k)\|_2^2 \quad (4.4.4)$$

If this inequality holds, we accept the step and move on. If it fails, we shrink the step (e.g., halve it) and try again. This simple loop tends to accept larger steps when the local model is reliable and automatically backs off when the function is curved or the first-order approximation is poor.

It is sometimes helpful to start backtracking with an educated guess for the step length. A second-order Taylor model around $\mathbf{x}_k$ gives

$$f(\mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)) \approx f(\mathbf{x}_k) - \alpha_k \nabla f(\mathbf{x}_k)^\top \nabla f(\mathbf{x}_k) + \frac{1}{2} \alpha_k^2 \nabla f(\mathbf{x}_k)^\top \nabla^2 f(\mathbf{x}_k) \nabla f(\mathbf{x}_k) \quad (4.4.5)$$

Minimizing this quadratic in α_k yields

$$\alpha_k^* = \frac{\nabla f(\mathbf{x}_k)^\top \nabla f(\mathbf{x}_k)}{\nabla f(\mathbf{x}_k)^\top \nabla^2 f(\mathbf{x}_k) \nabla f(\mathbf{x}_k)} \quad (4.4.6)$$

Using α_k^* as the initial guess often reduces the number of backtracking trials, although it requires access to second derivatives (or at least Hessian-vector products), which may be expensive or unavailable.

Example: Backtracking line search. The use of backtracking line search becomes especially clear when we consider a function such as

$$f(\mathbf{x}) = x_1^2 + 10x_2^2$$

whose level sets are elongated ellipses. Pure steepest descent alternates between moving mostly in the x_1 direction and then mostly in x_2 , producing a characteristic zig-zag down the valley. Small steps make the zig-zag gentle but slow; large steps leap across the valley and risk divergence; backtracking threads the needle by taking steps that are as large as the local geometry allows (see Figure 4.4.2).

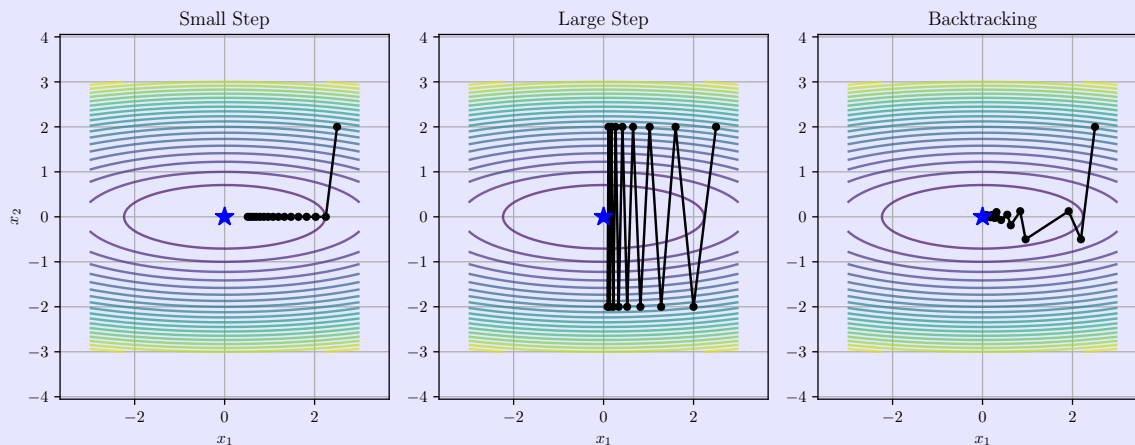


Figure 4.4.2: Gradient descent trajectories on $f(x_1, x_2) = x_1^2 + 10x_2^2$: small step size (left), too large step size (middle), and backtracking (right). The blue star is the global minimum.

4.4.1.2 Convergence

From a theoretical standpoint, under mild smoothness assumptions and with a proper line search, the gradient norms go to zero:

$$\lim_{k \rightarrow \infty} \|\nabla f(\mathbf{x}_k)\|_p = 0 \quad (4.4.7)$$

so every limit point is stationary. This property is also known as global convergence. When the iterates enter a neighborhood of a nondegenerate minimizer, the method converges linearly:

$$\frac{f(\mathbf{x}_{k+1}) - f(\mathbf{x}^*)}{f(\mathbf{x}_k) - f(\mathbf{x}^*)} \leq r \quad \text{for some } r \in (0, 1) \quad (4.4.8)$$

The constant r is governed by the condition number of the Hessian near $\mathbf{x}^*$. Poor conditioning (long, narrow valleys) slows the rate and accentuates the zig-zag, which is why steepest descent can require many iterations on ill-conditioned problems.

4.4.1.3 Stochastic Gradient Descent

Modern large-scale problems introduce another complication: evaluating the full gradient can be the dominant cost. If the objective is a sum of many data-dependent losses, $\min_{\mathbf{x}} \sum_{k=1}^N f_k(\mathbf{x})$, computing $\nabla \sum_k f_k$ each iteration may be prohibitive. Stochastic gradient descent replaces the full sum with a randomly selected data term or a small mini-batch, producing the update $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f_{i_k}(\mathbf{x}_k)$. The price is additional noise, and the benefit is dramatically cheaper steps and faster progress per unit time on huge datasets. Learning-rate schedules and variance-reduction tricks are then used to recover stable convergence.

All of this explains why steepest descent is so common and powerful while still having some (sometimes major) drawbacks. It is conceptually simple and easy to implement—just compute a gradient, pick a step length, and move downhill—and it requires only first-order information. Yet, the convergence is merely linear and can be painfully slow on ill-conditioned landscapes, and even computing the gradient may be costly at scale. These drawbacks motivate practical enhancements (line search, preconditioning, momentum, quasi-Newton methods) and sampling strategies (SGD and its variants) that preserve the core idea (follow the direction of fastest local decrease) while coping better with curvature and data size.

4.4.2 Newton and Modified-Newton Methods

In steepest descent, we search for a minimum by moving in the direction of greatest decrease. Newton's method takes a different perspective: instead of searching, it directly tries to solve the optimality condition

$$\nabla f(\mathbf{x}) = \mathbf{0} \quad (4.4.9)$$

which characterizes stationary points of f . Notice the similarity with a general nonlinear system of equations. Earlier, we solved problems of the form

$$\mathbf{g}(\mathbf{x}) = \mathbf{0}, \quad \mathbf{g} : \mathbb{R}^N \rightarrow \mathbb{R}^N \quad (4.4.10)$$

using Newton's method. Optimization fits this framework if we set

$$\mathbf{g}(\mathbf{x}) := \nabla f(\mathbf{x}) \quad (4.4.11)$$

Thus, finding a stationary point of f is equivalent to solving a system of nonlinear equations (solving for the roots of $\mathbf{g}(\mathbf{x})$). The Jacobian of $\mathbf{g}(\mathbf{x})$ is

$$\mathbf{J}_g(\mathbf{x}) = \nabla(\nabla f(\mathbf{x})) = \nabla^2 f(\mathbf{x}) \quad (4.4.12)$$

which is precisely the Hessian of f . Therefore, Newton's method for optimization naturally uses the Hessian. A Newton update step takes the form

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k, \quad \mathbf{d}_k = -(\nabla^2 f(\mathbf{x}_k))^{-1} \nabla f(\mathbf{x}_k) \quad (4.4.13)$$

where $\alpha_k > 0$ is the step length (often chosen by line search). So we can see that Newton's method for optimization is nothing more than Newton's method for nonlinear equations, specialized to the gradient system $\nabla f(\mathbf{x}) = \mathbf{0}$, with the Hessian playing the role of the Jacobian.

To understand why this works, recall that near a point $\mathbf{x}_k$, the function f can be locally approximated by its second-order Taylor expansion:

$$f(\mathbf{x}) \approx f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top (\mathbf{x} - \mathbf{x}_k) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_k)^\top \nabla^2 f(\mathbf{x}_k) (\mathbf{x} - \mathbf{x}_k) \quad (4.4.14)$$

This quadratic model has a stationary point where its gradient vanishes. Solving for that point gives exactly the Newton step direction $\mathbf{d}_k$. Notice that we are using curvature information to find the step direction in contrast to only the first-order information being used in steepest descent.

Choosing the step length. Close to a minimizer and under mild smoothness, taking a full step $\alpha_k = 1$ is appropriate and yields rapid local convergence. Farther away, a line search is employed for best performance in exactly the same way as in steepest descent.

4.4.2.1 Hessian Modification

Solving the stationarity equation $\nabla f(\mathbf{x}) = \mathbf{0}$ does not by itself distinguish minima from saddles or maxima. Consequently, the plain Newton step $\mathbf{d}_k$ in (4.4.13) can fail to decrease f when the Hessian $\nabla^2 f(\mathbf{x}_k)$ is indefinite or singular.

A simple way to see what we need is to examine the predicted change in f from a Newton step. Using the quadratic model of f at $\mathbf{x}_k$ and substituting $\mathbf{d}_k = -(\nabla^2 f(\mathbf{x}_k))^{-1} \nabla f(\mathbf{x}_k)$, we obtain the model-predicted decrease

$$f(\mathbf{x}_{k+1}) - f(\mathbf{x}_k) \approx -\nabla f(\mathbf{x}_k)^\top (\nabla^2 f(\mathbf{x}_k))^{-1} \nabla f(\mathbf{x}_k) \quad (4.4.15)$$

For the right-hand side to be negative for every nonzero gradient, we must have

$$(\nabla^2 f(\mathbf{x}_k))^{-1} \succ 0 \iff \nabla^2 f(\mathbf{x}_k) \succ 0 \quad (4.4.16)$$

i.e., the Hessian must be positive definite (PD). If the Hessian is not PD, Newton's direction is not guaranteed to be a descent direction.

So what does this tell us? We want a PD Hessian. Luckily, we can modify the Hessian to make it PD whenever necessary. The simplest and very effective modification is a diagonal shift:

$$\mathbf{d}_k(\delta_k) = -(\nabla^2 f(\mathbf{x}_k) + \delta_k \mathbf{I})^{-1} \nabla f(\mathbf{x}_k), \quad \delta_k \geq 0 \quad (4.4.17)$$

followed by a (backtracking) line search to pick $\alpha_k > 0$, and then

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k(\delta_k). \quad (4.4.18)$$

As soon as $\nabla^2 f(\mathbf{x}_k) + \delta_k \mathbf{I} \succ 0$, the direction is guaranteed to be a descent direction because

$$\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k(\delta_k) = -\nabla f(\mathbf{x}_k)^\top (\nabla^2 f(\mathbf{x}_k) + \delta_k \mathbf{I})^{-1} \nabla f(\mathbf{x}_k) < 0 \quad \text{whenever} \quad \nabla f(\mathbf{x}_k) \neq \mathbf{0} \quad (4.4.19)$$

The line search then ensures an actual decrease in the objective. When choosing δ_k , we want to keep Newton's fast local behavior, so we aim for a small positive shift. Note that this shift perturbs every eigendirection of the Hessian by the same amount δ_k , including directions that were already well-conditioned and positive, which can needlessly slow convergence along those directions. Production solvers therefore tend to use fixes like modified Cholesky factorization, which adds the minimum perturbation needed to fix only the non-positive-definite directions while leaving the rest of the Hessian untouched.

4.4.2.2 Global and Local Convergence

With standard line-search safeguards (e.g., sufficient decrease), Newton and modified-Newton iterations produce a sequence for which the gradient norms tend to zero, i.e., global convergence to a stationary point. Once the iterates enter a neighborhood of a nondegenerate minimizer $\mathbf{x}^*$ with positive-definite Hessian, the pure Newton method (i.e., $\alpha_k = 1$, no modification to the Hessian) is *quadratically convergent*:

$$\lim_{k \rightarrow \infty} \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|^2} = C, \quad C > 0 \quad (4.4.20)$$

This is faster than the linear convergence characteristic of steepest descent and explains Newton's dramatic acceleration in practice as the solution is approached.

R-convergence. Let $e_k := \|\mathbf{x}_k - \mathbf{x}^*\|$. The limit Eq. 4.4.20 (known as Q-convergence) is a stepwise statement about the actual errors: for any $\varepsilon \in (0, 1)$ there exists K such that

$$e_{k+1} \leq (C + \varepsilon) e_k^2 \quad (k \geq K) \quad (4.4.21)$$

R-convergence instead bounds these (possibly wiggly) errors by a smooth envelope. We say $\{\mathbf{x}_k\}$ R-converges to $\mathbf{x}^*$ with order $p \geq 1$ if there exist $K \in \mathbb{N}$, $C' > 0$, and a sequence $\{v_k\}$ with $v_k \rightarrow 0$ such that for all $k \geq K$,

$$e_k \leq v_k, \quad v_{k+1} \leq C' v_k^p \quad (4.4.22)$$

Special cases are $p = 1$ (R-linear), $p > 1$ (R-superlinear), and $p = 2$ (R-quadratic).

We can use Eq. 4.4.20 to show that Newton has local R-quadratic convergence, i.e., capturing quadratic overall decay even if individual steps fluctuate (e.g., due to line search or noise). We fix $\varepsilon \in (0, 1)$ and K as above. We set $v_K := e_K$ and define $v_{k+1} := (C + \varepsilon)v_k^2$. Then $e_k \leq v_k$ for all $k \geq K$ and $v_{k+1} \leq (C + \varepsilon)v_k^2$, so Newton has local R-quadratic convergence.

R-convergence is weaker than the stepwise Q-quadratic statement in Eq. 4.4.20: Q-quadratic $\implies$ R-quadratic, but not conversely. We care about R-convergence because line searches, Hessian modifications, or noise can make the actual errors oscillate. R-convergence still guarantees an overall quadratic decay by bounding the errors with a smooth envelope even when the per-iteration ratio does not stabilize.

4.4.2.3 Comparison with Steepest Descent

Steepest descent takes steps along $-\nabla f(\mathbf{x}_k)$ and relies on a one-dimensional line search; Newton takes the N -dimensional quadratic-model minimizer $\mathbf{d}_k = -\nabla^2 f(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k)$. Consequently, Newton typically requires far fewer iterations and exhibits a much steeper drop in the objective near the optimum, at the price of assembling curvature information and solving a linear system each iteration.

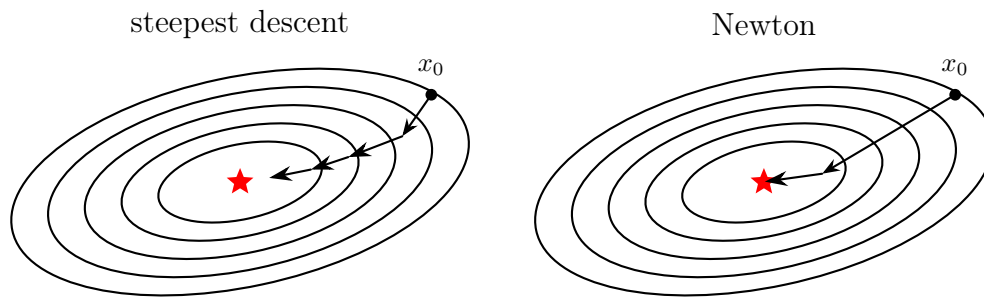


Figure 4.4.3: Steepest descent “searches” along the gradient direction, while Newton “solves” a local quadratic model and often reaches the minimizer in far fewer steps on well-conditioned problems.

4.4.3 Quasi-Newton Methods

Newton’s method converges quickly (assuming the Hessian is positive definite and we are “close enough” to the solution initially), but each step depends on the exact Hessian $\nabla^2 f(\mathbf{x}_k)$ and on solving a linear system with that matrix. For problems of dimension N , forming and storing the Hessian already costs on the order of $O(N^2)$, and solving a dense linear system costs about $O(N^3)$. On large problems, these costs make a Newton step impractical. This motivates **quasi-Newton methods**. These methods reuse information gathered along the iterations to build a good approximation to curvature—either to the Hessian itself or to its inverse—while using only gradient evaluations.¹

To connect this idea with Newton’s method and make it more concrete, recall the Newton direction $\mathbf{d}_k$ obtained from

$$\nabla^2 f(\mathbf{x}_k) \mathbf{d}_k = -\nabla f(\mathbf{x}_k) \quad (4.4.23)$$

followed by a step $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$ with a line search choosing $\alpha_k > 0$. In a quasi-Newton method, we replace the exact Hessian by an *approximate* Hessian $\mathbf{H}_k$ and compute the direction from

$$\mathbf{H}_k \mathbf{d}_k = -\nabla f(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k \quad (4.4.24)$$

¹It’s worth keeping in mind that we (almost always) do not explicitly invert a matrix in practice. When we “use the inverse,” we mean that we apply an approximation that plays the role of $[\nabla^2 f(\mathbf{x}_k)]^{-1}$.

Equivalently, one may make an approximation to the inverse Hessian and apply it to the gradient (using then $\mathbf{d}_k = -\mathbf{H}_k^{-1}\nabla f(\mathbf{x}_k)$); both views are common. The key question is how to update $\mathbf{H}_k$ as we learn more about the function. Taylor's theorem suggests that over a single step the gradient changes approximately linearly with displacement:

$$\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) \approx \nabla^2 f(\mathbf{x}_{k+1})(\mathbf{x}_{k+1} - \mathbf{x}_k) \quad (4.4.25)$$

Quasi-Newton updates enforce this relationship exactly for the model we are building, leading to the **secant equation**

$$\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) = \mathbf{H}_{k+1}(\mathbf{x}_{k+1} - \mathbf{x}_k) \quad (4.4.26)$$

4.4.3.1 Broyden-Fletcher-Goldfarb-Shanno (BFGS) Update

We introduce the notation

$$\Delta \mathbf{x}_k := \mathbf{x}_{k+1} - \mathbf{x}_k, \quad \Delta \mathbf{g}_k := \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) \quad (4.4.27)$$

so the constraint Eq. 4.4.26 becomes

$$\mathbf{H}_{k+1} \Delta \mathbf{x}_k = \Delta \mathbf{g}_k \quad (4.4.28)$$

Among all matrices that satisfy this linear relation, we want an update that (1) keeps $\mathbf{H}_{k+1}$ close to $\mathbf{H}_k$, and (2) preserves positive definiteness so that the search direction remains a descent direction. Imposing these requirements leads to the **BFGS** formula in Hessian form:

$$\mathbf{H}_{k+1} = \mathbf{H}_k + \frac{\Delta \mathbf{g}_k \Delta \mathbf{g}_k^\top}{\Delta \mathbf{g}_k^\top \Delta \mathbf{x}_k} - \frac{\mathbf{H}_k \Delta \mathbf{x}_k \Delta \mathbf{x}_k^\top \mathbf{H}_k}{\Delta \mathbf{x}_k^\top \mathbf{H}_k \Delta \mathbf{x}_k} \quad (4.4.29)$$

By construction, the secant equation Eq. 4.4.28 holds exactly. Moreover, if $\mathbf{H}_k \succ 0$ and the line search enforces the curvature condition $\Delta \mathbf{g}_k^\top \Delta \mathbf{x}_k > 0$ (e.g., Wolfe conditions), then $\mathbf{H}_{k+1} \succ 0$ as well. Thus, no separate Hessian modification is required.

It is often convenient to apply the update directly to the inverse. Exploiting that the BFGS step is a rank-2 modification and using the Sherman-Morrison-Woodbury identity yields the inverse form

$$\mathbf{H}_{k+1}^{-1} = \mathbf{H}_k^{-1} + \frac{(\Delta \mathbf{x}_k^\top \Delta \mathbf{g}_k) + \Delta \mathbf{g}_k^\top \mathbf{H}_k^{-1} \Delta \mathbf{g}_k}{(\Delta \mathbf{g}_k^\top \Delta \mathbf{x}_k)^2} \Delta \mathbf{x}_k \Delta \mathbf{x}_k^\top - \frac{\mathbf{H}_k^{-1} \Delta \mathbf{g}_k \Delta \mathbf{x}_k^\top + \Delta \mathbf{x}_k \Delta \mathbf{g}_k^\top \mathbf{H}_k^{-1}}{\Delta \mathbf{g}_k^\top \Delta \mathbf{x}_k} \quad (4.4.30)$$

which allows updating and applying $\mathbf{H}_{k+1}^{-1}$ without solving a new linear system, thereby avoiding the $O(N^3)$ cost of a dense factorization at each iteration.

With $\mathbf{H}_k \approx \nabla^2 f(\mathbf{x}_k)$ (or $\mathbf{H}_k^{-1} \approx [\nabla^2 f(\mathbf{x}_k)]^{-1}$) available, the search direction is

$$\mathbf{d}_k = -\mathbf{H}_k^{-1} \nabla f(\mathbf{x}_k) \quad (4.4.31)$$

and a line search selects $\alpha_k > 0$. The iterate is then updated by $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$. There is no universally optimal choice of the initial matrix $\mathbf{H}_0$. We often start from a positive multiple of the identity, $\mathbf{H}_0 = \gamma \mathbf{I}$ with $\gamma > 0$.

4.4.3.2 Convergence

Under standard smoothness assumptions, BFGS with a proper line search enjoys the same global convergence guarantees as gradient descent (convergence to a stationary point). Locally, once $\mathbf{x}_k$ is sufficiently close to a nondegenerate minimizer $\mathbf{x}^*$, the convergence is *superlinear*:

$$\lim_{k \rightarrow \infty} \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|_p}{\|\mathbf{x}_k - \mathbf{x}^*\|_p} = 0. \quad (4.4.32)$$

This means BFGS is typically faster than steepest descent and somewhat slower than exact Newton steps, which means it often gives an effective balance between per-iteration cost and speed of convergence. Because of this, it is the default algorithm used by `fminunc` in MATLAB.

Example: Adversarial Surfaces to Compare GD, BFGS, and Newton Methods. We visualize how different update rules behave on three 2D objectives specifically chosen to challenge the assumptions of each method. We take inspiration from the Wikipedia page on [test functions for optimization](#).

All optimizers use a backtracking line search. We compare four algorithms: standard gradient descent (GD), BFGS (in inverse form with a curvature check), a robust modified Newton that shifts the Hessian to be positive definite using a small positive constant, and a standard, unmodified Newton’s method. In the plots, a star marks the start point and an ‘x’ marks the endpoint of each trajectory.

Case 1: A Sharp Crease. Here, we use a smoothed absolute value function

$$f(x, y) = \sqrt{x^2 + \epsilon} + \frac{1}{2}y^2$$

which has a sharp “V”-shaped valley. This geometry directly violates the core assumption of Newton’s method: that the function is well-approximated by a quadratic.

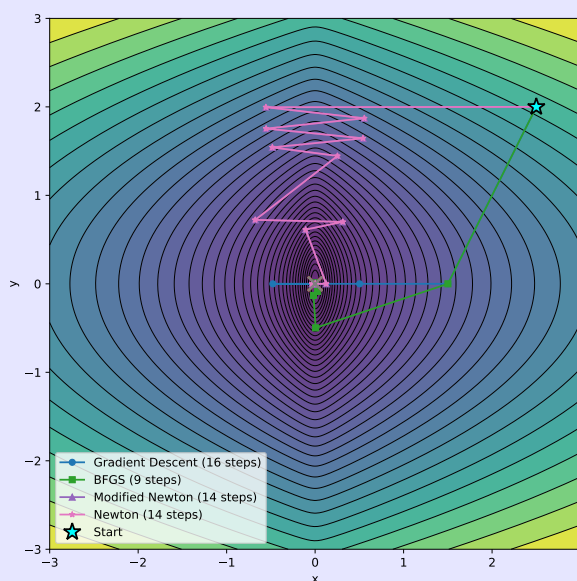


Figure 4.4.4: Sharp Crease: Newton’s method fails spectacularly due to a poor quadratic fit, while the more adaptive BFGS excels.

This scenario demonstrates a common failure mode for Newton’s method. At the start point, the function is almost linear along the x -axis, so the Hessian is nearly zero. The resulting quadratic model is a terrible fit, causing Newton’s method to overshoot the minimum and then oscillate back and forth. The line search prevents divergence, but the path is highly inefficient. On the other hand, BFGS is the clear winner. By building its curvature model gradually over several steps, it creates a more stable, averaged approximation that allows it to navigate the sharp turn smoothly and efficiently.

4.4.4 Step Selection Paradigms*

There are two standard ways to decide how far to move at each iterate and in which direction. They differ in the order of these choices.

Line search. We have always used this paradigm so far. First determine the step direction (e.g., steepest descent $-\nabla f(\mathbf{x}_k)$, Newton $-(\nabla^2 f(\mathbf{x}_k))^{-1}\nabla f(\mathbf{x}_k)$, or a quasi-Newton direction), and then determine the step

length $\alpha_k > 0$ by searching along the ray

$$\mathbf{x}_k + \alpha \mathbf{d}_k, \quad \alpha > 0 \quad (4.4.33)$$

This is exactly the paradigm used above: pick $\mathbf{d}_k$, then use a line search (e.g., backtracking with a sufficient-decrease test) to select α_k and set $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$. Intuitively, we look in one direction and choose how far to go along that line so that the actual decrease in f is acceptable.

Trust region. Trust-region methods reverse the order: they determine the step length first by fixing a region around the current point in which a local model of f is trusted, and only then determine the step direction by minimizing that model inside the region. Concretely, at iterate $\mathbf{x}_k$:

1. Initialize the trust-region size $\Delta > 0$.
2. Build the quadratic model

$$m_k(\mathbf{p}_k) := f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k + \frac{1}{2} \mathbf{p}_k^\top \nabla^2 f(\mathbf{x}_k) \mathbf{p}_k \quad (4.4.34)$$

and find the step by solving the constrained subproblem

$$\min_{\mathbf{p}_k} m_k(\mathbf{p}_k) \quad \text{s.t.} \quad \|\mathbf{p}_k\|_2 \leq \Delta \quad (4.4.35)$$

Set $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{p}_k$ if the step is accepted. In practice, this subproblem is rarely solved exactly. Standard approaches include the **dogleg method**, which builds an approximate solution by combining the steepest-descent direction with the full (Newton) step, both truncated to the trust-region boundary if necessary.

3. Compare the actual and predicted reductions to judge whether the model was accurate on the tried step size:

$$\text{actual reduction} = f(\mathbf{x}_k) - f(\mathbf{x}_k + \mathbf{p}_k), \quad \text{predicted reduction} = m_k(\mathbf{0}) - m_k(\mathbf{p}_k) \quad (4.4.36)$$

If the agreement is good (the model predicted the decrease well), expand the trust region (increase Δ) to allow larger future steps; if the agreement is poor, reduce it (decrease Δ) and resolve the subproblem.

Both approaches (line search and trust region) fit our generic update $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$ but choose $(\alpha_k, \mathbf{d}_k)$ in opposite orders. This difference has practical consequences. First, when $\nabla^2 f(\mathbf{x}_k) \succ 0$ and the unconstrained minimizer of m_k (the Newton step) satisfies $\|\mathbf{p}_k\|_2 \leq \Delta$, the trust-region step coincides with Newton's step, recovering Newton's rapid local behavior without needing a separate line search. Secondly, if $\nabla^2 f(\mathbf{x}_k)$ is indefinite or poorly conditioned, the trust-region subproblem remains well-posed because the constraint $\|\mathbf{p}_k\|_2 \leq \Delta$ caps how far the (possibly unreliable) quadratic model may be used. In contrast, a pure Newton direction might not even be a descent direction unless the Hessian is modified. Finally, acceptance and region updates play the same role that a sufficient-decrease line search plays for line-search methods: they ensure that when the local model is trustworthy we take large, efficient steps (by expanding Δ), and when it is not we automatically back off (by shrinking Δ).

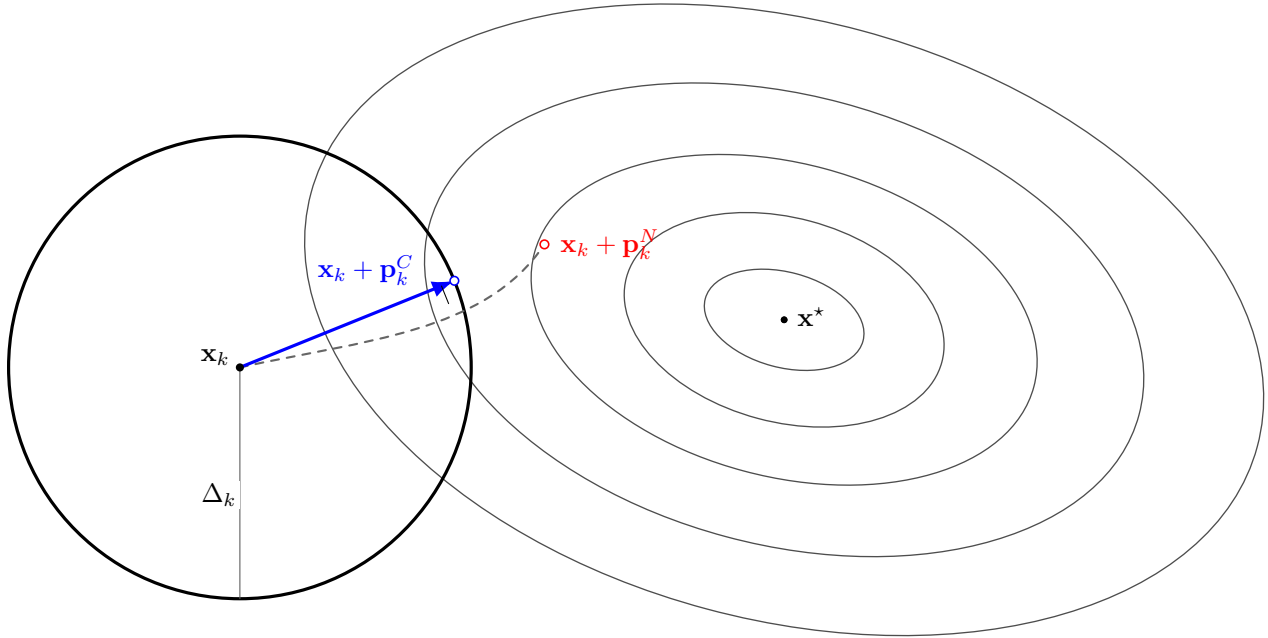


Figure 4.4.5: Trust-region step at iterate $\mathbf{x}_k$. Concentric ellipses are level sets of f around the minimizer $\mathbf{x}^*$. The thick circle is the trust region $\{\mathbf{x}_k + \mathbf{p} : \|\mathbf{p}\|_2 \leq \Delta_k\}$ of radius Δ_k . The dashed curve suggests the path toward the unconstrained minimizer of the quadratic model m_k , which would give the Newton/model step $\mathbf{x}_k + \mathbf{p}_k^N$ (shown outside the region). The solid arrow shows the accepted step on this iteration, the point $\mathbf{x}_k + \mathbf{p}_k^C$, i.e., the minimizer of m_k along the steepest-descent direction truncated to the boundary. If $\|\mathbf{p}_k^N\| \leq \Delta_k$, the trust-region step coincides with $\mathbf{p}_k^N$. After evaluating $f(\mathbf{x}_k + \mathbf{p}_k)$, the radius Δ_k is increased or decreased based on the ratio of actual to predicted reduction.

4.5 Constrained Optimization

So far, we've focused on unconstrained problems where the decision variables $\mathbf{x}$ could live anywhere in $\mathbb{R}^N$ (or whatever space the function f has its natural domain on). Now, we turn to the more common and practical scenario of **constrained optimization**, where we must find the minimum of an objective function while also satisfying a set of rules. These rules define the feasible set $\mathcal{D}$.

The standard form for a general constrained optimization problem is

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{c}(\mathbf{x}) = \mathbf{0} \\ & \mathbf{h}(\mathbf{x}) \geq \mathbf{0} \end{aligned} \tag{4.5.1}$$

Here, $\mathbf{c}(\mathbf{x}) = \mathbf{0}$ represents a set of **equality constraints**, and $\mathbf{h}(\mathbf{x}) \geq \mathbf{0}$ represents a set of **inequality constraints**. We'll assume the functions f , $\mathbf{c}$, and $\mathbf{h}$ are all sufficiently smooth.

This problem becomes a convex optimization problem, which is much easier to solve, if the objective function $f(\cdot)$ is convex, the equality constraints $\mathbf{c}(\cdot)$ are linear (affine), and the inequality constraint functions $\mathbf{h}(\cdot)$ are concave. We start by focusing only on problems with equality constraints.

4.5.1 Equality-Constrained Optimization

Let's consider an equality-constrained optimization problem of the form:

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{c}(\mathbf{x}) = \mathbf{0} \end{aligned} \tag{4.5.2}$$

where $\mathbf{c} : \mathbb{R}^N \rightarrow \mathbb{R}^M$ defines M constraints.

What are the necessary conditions for a point $\mathbf{x}^*$ to be an optimal solution? It can't be as simple as $\nabla f(\mathbf{x}^*) = \mathbf{0}$ because that point might not be feasible (i.e., it might not satisfy $\mathbf{c}(\mathbf{x}^*) = \mathbf{0}$). The main insight comes from visualizing the problem. At a constrained minimum $\mathbf{x}^*$, you can't make any further progress downhill (decreasing f) without violating the constraint. This means that at the solution, the level set of the objective function f must be tangent to the constraint surface defined by $\mathbf{c}(\mathbf{x}) = \mathbf{0}$.

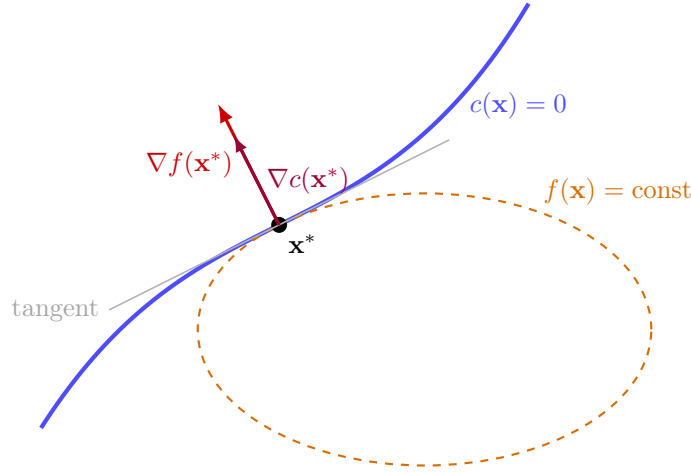


Figure 4.5.1: Equality-constrained optimum. At the solution $\mathbf{x}^*$, the feasible curve $\mathbf{c}(\mathbf{x}) = 0$ is tangent to a level set $f(\mathbf{x}) = \text{const}$. Their normals (gradients) are parallel.

Analytical derivation. We can also derive this result analytically. Let $\mathbf{x}^*$ be the solution and let $\mathbf{d} \in \mathbb{R}^N$ be a (small) search direction. To remain feasible to first order we require

$$\mathbf{c}(\mathbf{x}^* + \mathbf{d}) \approx \mathbf{c}(\mathbf{x}^*) + \nabla \mathbf{c}(\mathbf{x}^*)\mathbf{d} = \mathbf{0} \implies \nabla \mathbf{c}(\mathbf{x}^*)\mathbf{d} = \mathbf{0} \quad (4.5.3)$$

so any feasible direction $\mathbf{d}$ lies in the null space of $\nabla \mathbf{c}(\mathbf{x}^*)$. Meanwhile,

$$f(\mathbf{x}^* + \mathbf{d}) \approx f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*)^\top \mathbf{d} \quad (4.5.4)$$

For $\mathbf{x}^*$ to be a constrained minimum, the directional derivative along every feasible $\mathbf{d}$ (and its negative) must vanish, hence

$$\nabla f(\mathbf{x}^*)^\top \mathbf{d} = 0 \quad \text{for all } \mathbf{d} \text{ with } \nabla \mathbf{c}(\mathbf{x}^*)\mathbf{d} = \mathbf{0} \quad (4.5.5)$$

Equivalently, $\nabla f(\mathbf{x}^*)$ lies in the row space of $\nabla \mathbf{c}(\mathbf{x}^*)$; therefore there exists $\boldsymbol{\lambda} \in \mathbb{R}^M$ such that

$$\nabla f(\mathbf{x}^*) - \nabla \mathbf{c}(\mathbf{x}^*)^\top \boldsymbol{\lambda} = \mathbf{0} \quad (4.5.6)$$

4.5.1.1 Lagrange Multipliers

When two surfaces are tangent, their normal vectors must be parallel. The normal vector to a level set of a function is simply its gradient. Therefore, at the optimal point $\mathbf{x}^*$, the gradient of the objective function, $\nabla f(\mathbf{x}^*)$, must be parallel to the gradient of the constraint function, $\nabla \mathbf{c}(\mathbf{x}^*)$.

Mathematically, two vectors are parallel if one is a scalar multiple of the other. For a single constraint, this gives us the famous condition:

$$\nabla f(\mathbf{x}^*) = \lambda \nabla c(\mathbf{x}^*) \quad (4.5.7)$$

for some scalar λ , which we call the **Lagrange multiplier**. This generalizes to M constraints as

$$\begin{aligned} \nabla f(\mathbf{x}^*) &= \sum_{i=1}^M \lambda_i \nabla c_i(\mathbf{x}^*) \\ &= \nabla \mathbf{c}(\mathbf{x}^*)^\top \boldsymbol{\lambda} \end{aligned} \quad (4.5.8)$$

for some scalar Lagrange multipliers λ_i . This idea is elegantly captured by defining a new function, the **Lagrangian**:

$$L(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) - \boldsymbol{\lambda}^\top \mathbf{c}(\mathbf{x}) \quad (4.5.9)$$

The genius of this formulation is that the necessary conditions for a constrained optimum can be found by finding a stationary point of the Lagrangian. Let's see how. We set the gradient of L with respect to both its primal variables ($\mathbf{x}$) and its dual² variables ($\boldsymbol{\lambda}$) to zero:

$$\nabla L(\mathbf{x}, \boldsymbol{\lambda}) = \begin{bmatrix} \nabla_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda}) \\ \nabla_{\boldsymbol{\lambda}} L(\mathbf{x}, \boldsymbol{\lambda}) \end{bmatrix} = \mathbf{0} \implies \begin{cases} \nabla_{\mathbf{x}} L = \nabla f(\mathbf{x}) - \nabla \mathbf{c}(\mathbf{x})^\top \boldsymbol{\lambda} = \mathbf{0} \\ \nabla_{\boldsymbol{\lambda}} L = -\mathbf{c}(\mathbf{x}) = \mathbf{0} \end{cases} \quad (4.5.10)$$

Look at what we've recovered!

1. The top equation is exactly the parallelism condition for the gradients. (Note that for multiple constraints, $\nabla \mathbf{c}(\mathbf{x})$ is the Jacobian matrix, and $\boldsymbol{\lambda}$ is a vector of multipliers).
2. The bottom equation is simply our original feasibility constraint, $\mathbf{c}(\mathbf{x}) = \mathbf{0}$.

So, the task of solving a constrained optimization problem has been transformed into solving a system of nonlinear equations for the unknowns $\mathbf{x}$ and $\boldsymbol{\lambda}$. We know how to solve a system of nonlinear equations already! We can apply Newton's method to find the roots of the Lagrangian's gradient. Let our system of equations be $\mathbf{g}(\mathbf{x}, \boldsymbol{\lambda}) = \nabla L(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{0}$. The Jacobian of this system is

$$\mathbf{J}_g = \begin{bmatrix} \nabla_{\mathbf{xx}}^2 L(\mathbf{x}, \boldsymbol{\lambda}) & -\nabla \mathbf{c}(\mathbf{x})^\top \\ -\nabla \mathbf{c}(\mathbf{x}) & \mathbf{0} \end{bmatrix} \quad (4.5.11)$$

The top-left block, $\nabla_{\mathbf{xx}}^2 L$, is the Hessian of the Lagrangian with respect to $\mathbf{x}$. The off-diagonal blocks are the Jacobian of the constraints (and its transpose), and the bottom-right block is zero because the constraints $\mathbf{c}(\mathbf{x})$ do not depend on $\boldsymbol{\lambda}$. This structured matrix is often called the KKT (Karush-Kuhn-Tucker) matrix.

Applying the standard Newton update gives the update rule for an iterate $(\mathbf{x}_k, \boldsymbol{\lambda}_k)^\top$:

$$\begin{bmatrix} \nabla_{\mathbf{xx}}^2 L(\mathbf{x}_k, \boldsymbol{\lambda}_k) & -\nabla \mathbf{c}(\mathbf{x}_k)^\top \\ -\nabla \mathbf{c}(\mathbf{x}_k) & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{d}_k^{\mathbf{x}} \\ \mathbf{d}_k^{\boldsymbol{\lambda}} \end{bmatrix} = - \begin{bmatrix} \nabla f(\mathbf{x}_k) - \nabla \mathbf{c}(\mathbf{x}_k)^\top \boldsymbol{\lambda}_k \\ -\mathbf{c}(\mathbf{x}_k) \end{bmatrix} \quad (4.5.12)$$

We solve this linear system for the step directions $(\mathbf{d}_k^{\mathbf{x}}, \mathbf{d}_k^{\boldsymbol{\lambda}})$ and then update our variables:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{d}_k^{\mathbf{x}} \quad \boldsymbol{\lambda}_{k+1} = \boldsymbol{\lambda}_k + \mathbf{d}_k^{\boldsymbol{\lambda}} \quad (4.5.13)$$

Just like earlier, this method has local quadratic convergence, making it very efficient near a solution. Of course, strategies like line searches or trust regions are often added to ensure robust global performance.

4.5.2 Penalty Methods: A Questionable Alternative

An entirely different approach is to approximate the constrained problem with an unconstrained one. This is the idea behind penalty methods. We combine the objective and constraints into a single function, where violations of the constraints incur a "penalty" cost. The general form of the penalized objective is:

$$\min_{\mathbf{x}} (f(\mathbf{x}) + \mu (\|\mathbf{c}(\mathbf{x})\|_p^p + \|\mathbf{h}(\mathbf{x})_-\|_p^p)) \quad (4.5.14)$$

where $\mu > 0$ is a penalty parameter and $\mathbf{h}(\mathbf{x})_-$ represents the parts of $\mathbf{h}$ that violate the constraint (i.e., are negative). The norm, typically with $p = 1$ or $p = 2$, measures the magnitude of the constraint violation. By driving μ to be very large, we force the solution of this unconstrained problem to satisfy the constraints.

There's a difference in theoretical guarantees between the two common choices for p :

²Don't worry about the primal/dual formalism for 10.34, but if you're curious, you should check out [the Wikipedia page on duality](#).

- $p = 1$ (L1 norm): The solution to the penalty formulation exactly recovers the original solution for a sufficiently large, but finite, value of μ . This is often called an “exact penalty method.”
- $p = 2$ (L2 norm): The solution to the penalty formulation only approaches the original solution in the limit as $\mu \rightarrow \infty$.

So why is the penalty method approach questionable, as suggested in the title of this section? The problem lies in the numerics. Using very large values of μ creates an objective function with extremely steep valleys/walls around the feasible region. This makes the Hessian of the penalized function highly ill-conditioned. Numerical methods struggle to solve linear systems involving ill-conditioned matrices, leading to instability and inaccurate steps. This practical difficulty is why methods based on the Lagrangian are often preferred.

4.5.3 Inequality-Constrained Problems

We’ll now consider optimization problems with only inequality constraints, which have the following form:

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{h}(\mathbf{x}) \geq \mathbf{0} \end{aligned} \tag{4.5.15}$$

The difficulty with inequalities is that we don’t know *a priori* which constraints will be important at the solution. At a local solution $\mathbf{x}^*$, each constraint $h_i(\mathbf{x})$ can be in one of two states: active or inactive. If $h_i(\mathbf{x}^*) = 0$, the constraint is active. The solution lies directly on the boundary of this constraint. If $h_i(\mathbf{x}^*) > 0$, the constraint is inactive. The solution is in the interior of this constraint’s feasible region. [Figure 4.5.2](#) illustrates this concept.

If we knew the active set (the set of all active constraints) beforehand, we could simply treat those constraints as equalities ($h_i(\mathbf{x}) = 0$) and ignore the inactive ones. But we don’t know the active set until we’ve found the solution! This creates a chicken-and-egg problem that has led to two major paradigms for solving these problems.

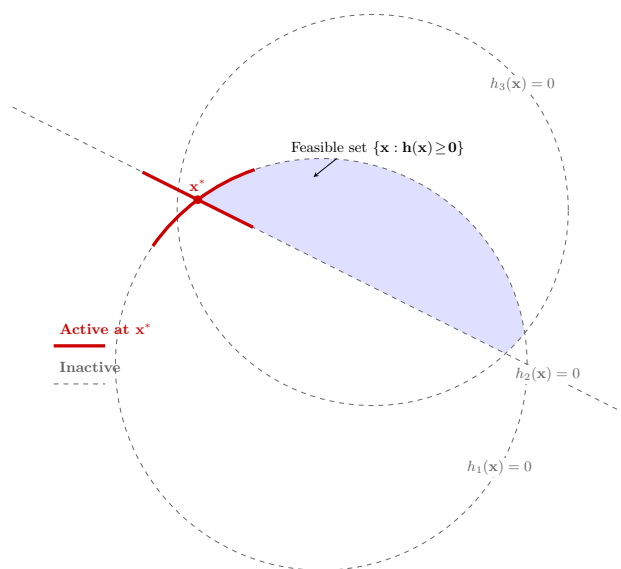


Figure 4.5.2: Active (binding) vs. inactive constraints at a solution $\mathbf{x}^*$ for inequality-constrained problems with $\mathbf{h}(\mathbf{x}) \geq \mathbf{0}$.

There are two major approaches to solving inequality-constrained problems: active-set methods and interior-point methods. Popular since the 1970s, active-set methods essentially try to “guess” the active set. They solve an equality-constrained subproblem using a working set of active constraints, then use the solution to

update the working set (e.g., by adding a violated constraint or removing one that's not needed). While effective for smaller problems, their major drawback is combinatorial complexity. With M constraints, there are 2^M possible active sets to check in the worst case, which is infeasible for large problems. Meanwhile, interior-point methods, which gained popularity in the 1990s, take a completely different approach. Instead of figuring out which constraints are on the boundary, they cleverly force the iterates to always stay inside the feasible region. They are very well-suited for large-scale problems. We will focus on the more modern and scalable interior-point approach.

4.5.4 Interior-Point Methods

The magic behind interior-point methods is to transform the constrained problem into an unconstrained one using a **barrier function**. The most common choice is the logarithmic barrier. We create a new objective function:

$$\phi(\mathbf{x}, \mu) = f(\mathbf{x}) - \mu \sum_{i=1}^M \log(h_i(\mathbf{x})) \quad (4.5.16)$$

Here, $\mu > 0$ is a **barrier parameter** and M is the number of constraints.

How does this work? The logarithm has a natural barrier: $\log(z) \rightarrow -\infty$ as $z \rightarrow 0^+$. This means that as an iterate $\mathbf{x}$ gets close to a boundary where some $h_i(\mathbf{x}) \rightarrow 0$, the term $-\mu \log(h_i(\mathbf{x}))$ explodes to $+\infty$. This creates a massive penalty (indeed, an infinitely high wall) that prevents the optimizer from ever leaving the interior of the feasible region.

The overall algorithm is beautifully simple:

1. Choose an initial $\mu > 0$ and a starting point $\mathbf{x}_0$ that is strictly feasible (i.e., $h_i(\mathbf{x}_0) > 0$ for all i).
2. Solve the unconstrained problem $\min_{\mathbf{x}} \phi(\mathbf{x}, \mu)$ with, e.g., Newton's method.
3. Decrease μ (e.g., $\mu \leftarrow 0.1\mu$).
4. Repeat from step 2, using the previous solution as the new starting point, until μ is very close to zero.

As $\mu \rightarrow 0^+$, the influence of the barrier term diminishes, and the solution to the unconstrained problem converges to the solution of the original constrained problem. This approach avoids the combinatorial nightmare of active-set methods and can achieve a fast (superlinear) convergence rate. The one catch is that it technically only finds an *approximate* solution, but in practice, the accuracy is more than sufficient for most engineering applications.

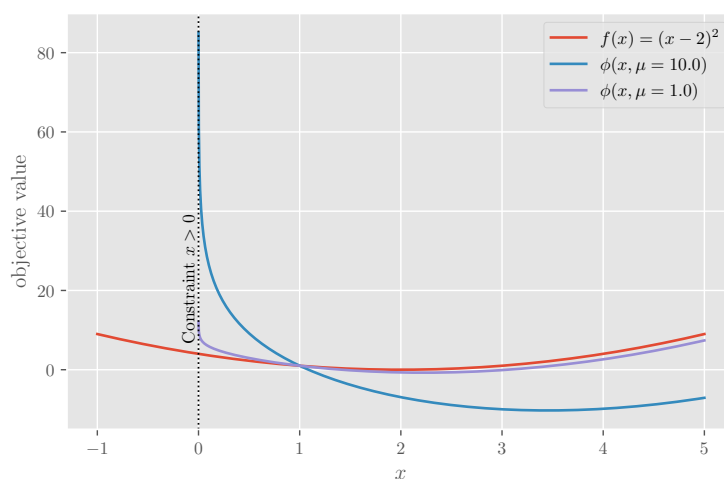


Figure 4.5.3: Logarithmic barrier for the constraint $x > 0$: plots of $f(x) = (x-2)^2$ and $\phi(x, \mu) = f(x) - \mu \log x$ ($\mu \in \{10, 1.0\}$). The barrier diverges as $x \rightarrow 0^+$, so smaller μ more closely tracks f while keeping iterates in the interior.

Example: Consider the problem:

$$\begin{aligned} \min_{x_1, x_2} \quad & f(x_1, x_2) = x_1^2 + 10x_2^2 \\ \text{subject to} \quad & h(x_1, x_2) = 1 - (x_1 - 2)^2 - (x_2 - 2)^2 \geq 0 \end{aligned} \quad (4.5.17)$$

The objective function's minimum is at $(0, 0)$, but the constraint is a circular region centered at $(2, 2)$ with a radius of 1, which doesn't contain the unconstrained minimum. The solution must therefore lie on the boundary of the circle.

The barrier problem is

$$\min_{x_1, x_2} \quad \phi(\mathbf{x}, \mu) = (x_1^2 + 10x_2^2) - \mu \log(1 - (x_1 - 2)^2 - (x_2 - 2)^2) \quad (4.5.18)$$

We can solve this using Newton's method. The core of the MATLAB code would look like this:

```
% Define objective and constraint functions, plus their gradients and Hessians
f = @(x) x(1)^2 + 10*x(2)^2;
grad_f = @(x) [2*x(1); 20*x(2)];
H_f = @(x) [2, 0; 0, 20];

h = @(x) 1 - (x(1)-2)^2 - (x(2)-2)^2;
grad_h = @(x) [-2*(x(1)-2); -2*(x(2)-2)];
H_h = @(x) [-2, 0; 0, -2];

% Define the barrier objective and its derivatives
phi = @(x, mu) f(x) - mu * log(h(x));
grad_phi = @(x, mu) grad_f(x) - (mu/h(x)) * grad_h(x);
H_phi = @(x, mu) H_f(x) - (mu/h(x))*H_h(x) + (mu/h(x)^2)*grad_h(x)*grad_h(x)';

% Initial guess (must be strictly feasible)
x = [2; 2];

% Loop over decreasing mu values
for mu = [10:-0.001:0.001]
    % Newton's method to minimize phi for the current mu
    while norm(grad_phi(x, mu)) > 1e-8
        x = x - H_phi(x, mu) \ grad_phi(x, mu);
    end
end
```

As we decrease μ , the solution to the barrier problem traces a path from the “analytic center” of the feasible region toward the true constrained optimum on the boundary. [Figure 4.5.4](#) shows the solution path for $\mu \in \{10, 0.001\}$. The final solution found by the algorithm is the point on the circle closest (in a weighted sense) to the unconstrained minimum at $(0, 0)$ and corresponds to $\mu = 0.001$.

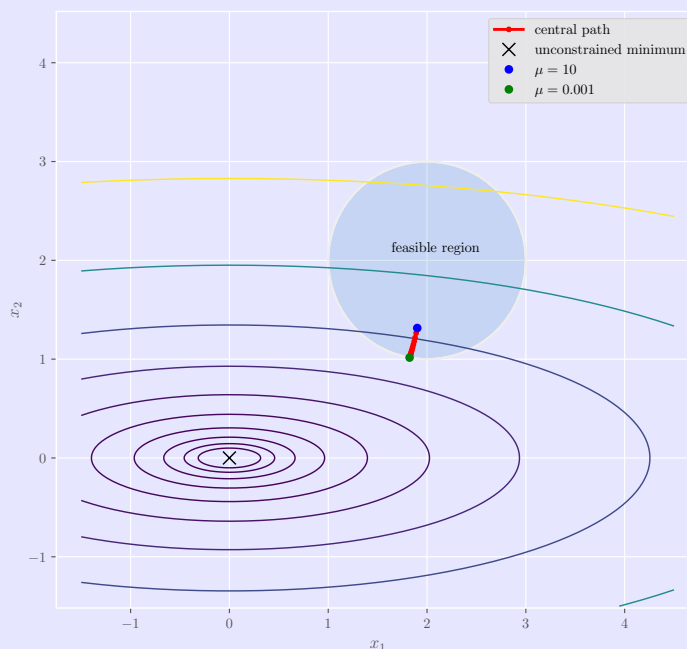


Figure 4.5.4: Solution path for the barrier problem Eq. 4.5.18: $\mu \in \{10, 0.001\}$.

4.6 Mixed-Integer Optimization*

We now move into a class of optimization problems where the feasible set $\mathcal{D}$ is no longer a continuous region within $\mathbb{R}^N$. Many engineering decisions are not continuous but discrete. For example, we might need to decide which molecular species to include in a reaction, whether a reactor is on or off, the exact number of trays in a distillation column, or any decision that can be framed as a “yes” or “no” choice. When the decision variables $\mathbf{x}$ are restricted to discrete values, we have an integer problem. If the problem involves a mix of both discrete and continuous variables, it is called a **mixed-integer problem**.

One might initially think that because the feasible set of integers $\mathbb{Z}^N$ is smaller than the set of real numbers $\mathbb{R}^N$, these problems should be easier. Unfortunately, the opposite is true. Mixed-integer problems are notoriously difficult and are generally classified as NP-hard. This comes from a combinatorial explosion. Consider designing a 10-amino-acid peptide where there are 20 possible amino acids for each position. This creates 20^{10} different possible peptide sequences, a number far too large to check one by one.

The difficulty is that we have lost the smooth landscape that guided our continuous methods: neighboring integer points can have very different objective values, so gradients are not informative. A common approach is to temporarily drop the integrality requirements and solve the resulting continuous problem; this is called a *relaxation*. The relaxation is easier to solve, and its objective value is optimistic for the objective (“too good to be true”) serving as a lower bound in minimization and an upper bound in maximization. However, taking the relaxed solution and simply rounding the integer variables is rarely reliable, as rounded points may violate constraints or be far from optimal. Instead, relaxations are used as building blocks inside exact algorithms such as branch-and-bound (and variants like branch-and-cut) to compute bounds and systematically prove optimality, and inside heuristics to generate good candidates. In 10.34, we will focus on branch-and-bound and use relaxations primarily for bounding rather than as standalone solutions.

4.6.1 Branch and Bound*

Since brute-force checking every possibility is out of the question, we need a more intelligent strategy. While stochastic methods like simulated annealing exist, the most widely used exact framework for these problems is **branch and bound**. The goal of branch and bound is to perform an exhaustive search implicitly, meaning it explores the entire space of possibilities while evaluating as few of them as possible. Its power comes from the ability to find and compare lower and upper bounds on the optimal solution.

First, we introduce the concept of the relaxed problem. For an integer problem where variables must be, for example, either 0 or 1 ($\mathbf{x} \in \{0, 1\}^N$), we can define a relaxed version where the variables can take any continuous value between 0 and 1 ($\mathbf{x} \in [0, 1]^N$):

$$\begin{array}{ll} \min_{\mathbf{x} \in \{0, 1\}^N} f(\mathbf{x}) & \min_{\mathbf{x} \in [0, 1]^N} f(\mathbf{x}) \\ \text{(original problem)} & \text{(relaxed problem)} \end{array} \quad (4.6.1)$$

Because the feasible set of the relaxed problem is larger than the integer one, its optimal objective value provides a powerful piece of information: it is a lower bound on the best possible value for the original integer problem. This is because for minimization, enlarging the feasible set cannot increase the optimal value (it can only decrease or leave it unchanged).

The branch and bound algorithm uses this idea to build a search tree. The first step is to **branch**. We take the original problem and split it into two or more subproblems. For a binary variable x_1 , we would create one subproblem where x_1 is fixed to 0 and another where x_1 is fixed to 1. This divides the search space.

The second step is to **bound**. For each of these new nodes in our search tree, we calculate a lower and an upper bound on the best possible solution within that branch. (For minimization problems, the relaxation's objective value is a lower bound and any feasible integer solution gives an upper bound; for maximization, the directions are reversed.) The lower bound is found by solving the continuous relaxation of the subproblem. For the node where $x_1 = 0$, we would solve the problem with x_1 fixed at 0 and all other variables relaxed to be in $[0, 1]$. The upper bound is established by finding any valid integer solution in that branch. The best feasible integer solution found so far across the entire tree serves as the global upper bound for the whole problem.

The third and most important step is to **fathom**, or prune, the tree. This is where the efficiency comes from. If we find that the lower bound of a particular node is higher than our current best global upper bound, we can completely discard that node and its entire subtree. There is no need to explore it further because we are guaranteed that no integer solution in that entire branch can ever be better than the integer solution we have already found. This allows us to cut away vast portions of the combinatorial search space without ever looking at them.

The algorithm proceeds by repeatedly branching on variables, calculating bounds for the new subproblems, and fathoming branches until the entire space has been implicitly covered. The best integer solution found during this process is the global optimum.

Example: Branch-and-Bound (Minimization). Imagine a simple graph: a triangle with three vertices (1, 2, and 3) and the three edges connecting them ((1,2), (2,3), and (1,3)). Our goal is to solve the minimum vertex cover problem on this graph. This means we want to find the smallest set of vertices such that every single edge is touched by at least one selected vertex.

We can frame this as a 0-1 integer program. Let's define a binary variable $x_i \in \{0, 1\}$ for each vertex i . We'll set $x_i = 1$ if we decide to include vertex i in our cover, and $x_i = 0$ otherwise. Formally, we write the problem as

$$\begin{array}{ll} \min & z = x_1 + x_2 + x_3 \\ \text{s.t.} & x_1 + x_2 \geq 1, \quad x_2 + x_3 \geq 1, \quad x_1 + x_3 \geq 1, \\ & x_i \in \{0, 1\} \quad (i = 1, 2, 3) \end{array} \quad (4.6.2)$$

Now, we begin by relaxing the integrality constraints. Replace $x_i \in \{0, 1\}$ by $0 \leq x_i \leq 1$. Summing the

three covering constraints gives

$$(x_1+x_2) + (x_2+x_3) + (x_1+x_3) \geq 3 \implies 2(x_1+x_2+x_3) \geq 3 \quad (4.6.3)$$

so any feasible continuous point satisfies $x_1+x_2+x_3 \geq \frac{3}{2}$. By symmetry, the continuous optimum is

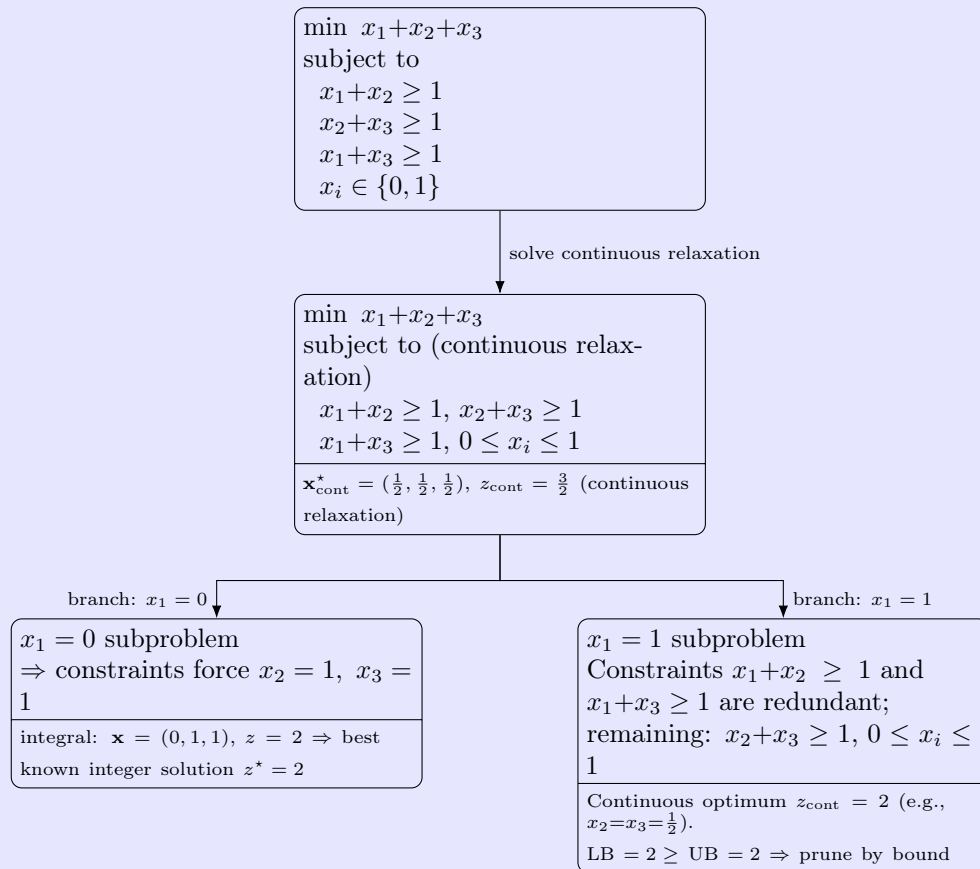
$$\mathbf{x}_{\text{cont}}^* = \left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}\right) \quad \text{with} \quad z_{\text{cont}} = \frac{3}{2} \quad (4.6.4)$$

Because this is a minimization problem, every continuous optimal value is an LB on the true integer optimum. Next, we branch on the fractional variable x_1 . We create two subproblems: one where $x_1 = 0$ and one where $x_1 = 1$.

For the left child ($x_1 = 0$), the constraints $x_1+x_2 \geq 1$ and $x_1+x_3 \geq 1$ become $x_2 \geq 1$ and $x_3 \geq 1$. With $0 \leq x_i \leq 1$, this forces $x_2 = x_3 = 1$, so the node is already integral: $\mathbf{x} = (0, 1, 1)$ with $z = 2$. This becomes the best known integer solution (incumbent), so the current global upper bound is 2, and thus $z^* \leq 2$.

For the right child ($x_1 = 1$), the constraints $x_1+x_2 \geq 1$ and $x_1+x_3 \geq 1$ are redundant; the remaining condition is $x_2+x_3 \geq 1$ with $0 \leq x_i \leq 1$. The continuous relaxation has optimal value $z_{\text{cont}} = 2$ (achieved, for example, by $x_2 = x_3 = \frac{1}{2}$). This node also contains integer solutions $(1, 1, 0)$ and $(1, 0, 1)$ with the same objective 2. Since the node LB equals the current global UB (2), we prune by bound.

Now, we have a best-known integer solution $\mathbf{x} = (0, 1, 1)$ with $z = 2$. Every open subtree is either infeasible, integral, or has LB ≥ 2 . Thus, no better solution exists and $z^* = 2$ is globally optimal.



It is worth noting that modern MILP/MINLP solvers further strengthen the relaxation with *branch-and-cut*: at each node they derive and add valid inequalities (“cuts”), such as cover, clique, flow/disjunctive, or

lift-and-project cuts, to eliminate fractional solutions and improve the bound before deciding to branch. Branch-and-cut is especially effective for 0-1 models where strong, problem-specific cuts are available (e.g., network design, facility location, set covering, routing). For completeness, another common variant is *branch-and-price*, which couples branch-and-bound with column generation and is used when the natural formulation has exponentially many variables (crew scheduling, vehicle routing, etc.).

We conclude this section by observing that the entire strategy of branch-and-bound hinges on our ability to efficiently solve the continuous, relaxed problems that provide lower bounds. Branch-and-bound is most effective when these relaxations are convex because we can compute tight global lower bounds quickly. For nonconvex MINLPs, global solvers typically use *spatial* branch-and-bound together with convex underestimators/relaxations to obtain valid bounds; however, constructing and tightening these relaxations can itself be NP-hard and often requires substantial additional branching and cutting, which is why such problems scale much more poorly. Suffice it to say that modern solvers are quite sophisticated and can handle a wide range of problems. The field of optimization is vast and complex, and we are only scratching the surface here.

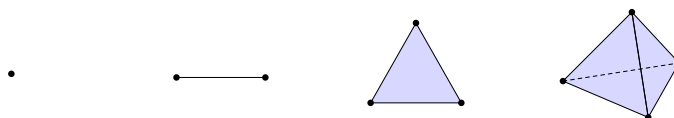
4.7 Gradient-Free Optimization*

Throughout our discussion, we have relied on a crucial piece of information: the gradient $\nabla f(\mathbf{x})$, and often the Hessian $\nabla^2 f(\mathbf{x})$. These derivatives guided our search for a minimum. But what if we do not have access to them? This situation is common when the objective function $f(\mathbf{x})$ is the result of a complex simulation or a physical experiment, where the underlying mathematical model is unknown or too complex to differentiate. This gives rise to a field also known as **derivative-free** or **black-box optimization**. The methods used are sometimes called **zero-th order methods** because they only use function values (zero-th order information), not derivatives.

One option is to estimate the gradient numerically using finite differences, but this can be very computationally expensive and may introduce significant numerical error. A better approach is often to use an algorithm specifically designed for a derivative-free setting.

4.7.1 The Nelder-Mead Method*

A classic and widely used gradient-free algorithm is the **Nelder-Mead method**. Instead of following a gradient, this method explores the search space using a geometric object called a simplex. A simplex is the generalization of a triangle to arbitrary dimensions: in one dimension it is a line segment, in two dimensions it is a triangle, and in three dimensions it is a tetrahedron. In an N -dimensional space, a simplex is formed by $N + 1$ vertices.



The algorithm begins with an initial simplex in the search space. At each iteration, it evaluates the objective function at all $N + 1$ vertices and identifies the worst vertex (highest objective value), the best vertex (lowest value), and the second-worst vertex. The goal is to replace the worst point with a new, better point located by moving “downhill”. This is achieved through a series of adaptive moves. The primary move is **reflection**, where the worst point is flipped through the centroid of the remaining points. If this new point is better, the algorithm might get optimistic and try an **expansion** to stretch the simplex even further in this promising direction. If the reflected point is still poor, the algorithm performs a **contraction**, pulling the point back towards the other vertices to search more locally. If even contraction fails, the simplex performs a **shrink** move, pulling all points closer to the single best vertex.

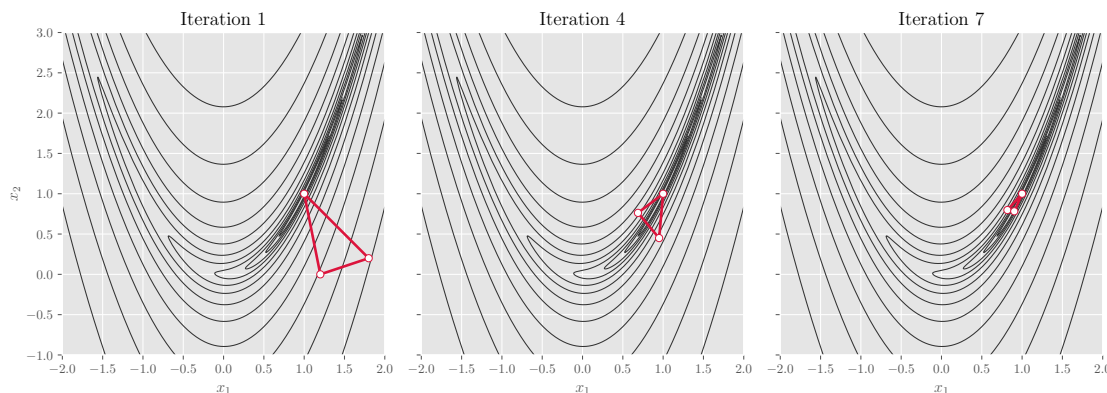


Figure 4.7.1: Three snapshots (iterations 1, 4, and 7) of the Nelder-Mead method on the 2D Rosenbrock function. Contours show level sets of $f(x_1, x_2)$; the red triangle is the simplex. At each step the worst vertex is replaced via reflection/expansion/contraction, causing the simplex to translate and shrink along the curved valley toward the minimizer at $(1, 1)$.

Through this sequence of reflections, expansions, and contractions, the simplex tumbles and morphs its way across the landscape of the objective function, gradually crawling downhill toward a minimum. The ability of the simplex to grow and shrink gives the method a built-in notion similar to a trust region, allowing it to adapt its search scale. The Nelder-Mead method often performs remarkably well for problems with low dimensionality, typically in the range of 2 to 8 dimensions. However, it does not scale well to high-dimensional problems, as the simplex becomes an increasingly inefficient tool for exploring a space whose volume grows exponentially with dimension—a classic example of the curse of dimensionality.

Convergence and practicalities. The classical Nelder-Mead method has no general convergence guarantee in dimension ≥ 2 for smooth objectives. Smooth counterexamples exist where it converges to a nonstationary point. Positive results hold only in special cases, like strictly convex 1D problems with bounded level sets and certain restricted 2D variants, and do not cover typical nonconvex, higher-dimensional settings. In practice, performance relies on a few basics: rescale variables; use both a simplex-size and a function-decrease tolerance for stopping; watch for degeneracy or stalling and consider restarts or re-expansions; be cautious with noise or discontinuities; handle constraints via penalties, projections, or box-aware variants; and note that effectiveness degrades as dimension grows. Beyond roughly $N \gtrsim 10$, trust-region or model-based derivative-free solvers (and quasi-Newton methods when gradients or reliable finite differences are available) typically scale better.

4.8 Optimization: A Summary

We have now surveyed some of the most fundamental concepts and algorithms in numerical optimization. The general problem is always of the form $\min_{\mathbf{x} \in \mathcal{D}} f(\mathbf{x})$, but the choice of method depends heavily on the nature of the feasible set $\mathcal{D}$ and the information available about the objective function f .

When the problem is unconstrained, meaning $\mathcal{D} = \mathbb{R}^N$ (or, more generally, $\mathcal{D}$ coincides with the natural domain of f), we can use gradient information to find a solution. Steepest descent offers the simplest approach, moving in the direction of the negative gradient at each step. While guaranteed to converge under mild conditions, this method can be quite slow, particularly near the solution. Newton’s method addresses this limitation by incorporating second-order information through the Hessian matrix, leading to much faster convergence when close to the optimum. However, computing and inverting the Hessian at each iteration can be computationally expensive. Quasi-Newton methods like BFGS provide an attractive middle ground, building up an approximation to the Hessian using only gradient information from previous iterations.

The presence of constraints changes the optimization landscape. For equality constraints of the form $\mathbf{c}(\mathbf{x}) = \mathbf{0}$, the method of Lagrange multipliers transforms the constrained problem into finding critical points of the

Lagrangian:

$$L(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) - \boldsymbol{\lambda}^\top \mathbf{c}(\mathbf{x}) \quad (4.8.1)$$

This reduces the problem to solving the system $\nabla_{\mathbf{x}} L = \mathbf{0}$ and $\mathbf{c}(\mathbf{x}) = \mathbf{0}$, which can be tackled using Newton's method for systems of equations.

Inequality constraints require different techniques since the active set (the constraints that are binding at the solution) is unknown *a priori*. Interior-point methods handle this by introducing a barrier function that prevents iterates from leaving the feasible region. The constrained problem is then solved as a sequence of unconstrained barrier problems with the barrier parameter gradually reduced to recover the original problem.

When decision variables must take discrete values, we enter the world of integer programming. These problems are fundamentally harder due to the combinatorial nature of the search space. Branch-and-bound provides a systematic enumeration strategy, using bounds on subproblems to eliminate large portions of the search tree without explicitly evaluating every possibility.

Finally, when gradient information is unavailable, whether due to noisy function evaluations, discontinuities, or computational constraints, gradient-free methods become necessary. The Nelder-Mead simplex algorithm is one example of such a method, using only function values at a collection of points to guide the search toward better solutions.

4.9 Exercises

In the solutions below, a paragraph marked *Side note* gives additional context. It is not part of what a complete answer requires.

Exercise 4.1

True or False? Briefly explain your answers. For a statement that is false, it is enough to give a counterexample.

- (a) The Hessian $\nabla^2 f(\mathbf{x})$ of a twice continuously differentiable function $f : \mathbb{R}^N \rightarrow \mathbb{R}$ is always positive semi-definite.
- (b) The maximization problem

$$\max_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad c(\mathbf{x}) \leq 0$$

can be equivalently solved as the minimization problem

$$\min_{\mathbf{x}} -f(\mathbf{x}) \quad \text{subject to} \quad -c(\mathbf{x}) \leq 0.$$

- (c) Let $\mathbf{F} : \mathbb{R}^N \rightarrow \mathbb{R}^M$ be a smooth function. The two problems

$$\min_{\mathbf{x}} \|\mathbf{F}(\mathbf{x})\|_2 \quad \text{and} \quad \min_{\mathbf{x}} \|\mathbf{F}(\mathbf{x})\|_2^2$$

have the same set of solutions.

- (d) A global minimum always has a better (strictly lower) objective value than a local minimum.
- (e) Any optimization problem has at least one local optimum.
- (f) A local maximum cannot be a local minimum.
- (g) The union $A \cup B$ of two convex sets A and B is a convex set.
- (h) The intersection $A \cap B$ of two convex sets A and B is a convex set.
- (i) The sum of two convex functions is a convex function.
- (j) The product of two convex functions is a convex function.

- (k) Consider the equality-constrained problem

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad c(\mathbf{x}) = 0,$$

where $f : \mathbb{R}^N \rightarrow \mathbb{R}$ is a convex function and $c : \mathbb{R}^N \rightarrow \mathbb{R}$ is a convex function. Then this is a convex optimization problem.

- (l) When minimizing a smooth function using gradient descent with a fixed, pre-specified step size, the objective function value improves (decreases) at every iteration.
- (m) Consider gradient descent at an iterate $\mathbf{x}_k$ with $\nabla f(\mathbf{x}_k) \neq \mathbf{0}$ and search direction $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$, and suppose the step size is chosen by exactly minimizing the objective function along the search direction:

$$\alpha^* = \arg \min_{\alpha \geq 0} f(\mathbf{x}_k + \alpha \mathbf{d}_k).$$

Then, at the optimal step size α^* , the function f is at a local minimum, i.e., $\mathbf{x}_k + \alpha^* \mathbf{d}_k$ is a local minimum of f .

- (n) We can use Newton's method to find the minimum of $f([x_1, x_2]) = \max(x_1, 0) + \max(x_2, -2)$.
- (o) Consider minimizing a twice continuously differentiable function $f : \mathbb{R}^N \rightarrow \mathbb{R}$, and suppose the current iterate $\mathbf{x}_k$ satisfies $\nabla f(\mathbf{x}_k) \neq \mathbf{0}$. We take the step

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k, \quad \mathbf{d}_k = -(\nabla^2 f(\mathbf{x}_k))^{-1} \nabla f(\mathbf{x}_k),$$

where $\nabla^2 f(\mathbf{x}_k)$ is invertible. Then, for a sufficiently small step size $\alpha_k > 0$, the objective function is guaranteed to improve, i.e., $f(\mathbf{x}_{k+1}) < f(\mathbf{x}_k)$.

- (p) In the same setting as the previous part, suppose instead we take the quasi-Newton (BFGS) step

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k, \quad \mathbf{d}_k = -\mathbf{H}_k^{-1} \nabla f(\mathbf{x}_k),$$

where $\mathbf{H}_k \succ \mathbf{0}$ is the BFGS approximation of the Hessian, and α_k is chosen by a backtracking line search with a sufficient-decrease (Armijo) condition. Then the objective function is guaranteed to improve, i.e., $f(\mathbf{x}_{k+1}) < f(\mathbf{x}_k)$.

- (q) We don't need to use Hessian modification if we use the BFGS method to estimate the Hessian.
- (r) Consider minimizing a finite-sum objective

$$f(\mathbf{x}) = \sum_{j=1}^M f_j(\mathbf{x}),$$

where each f_j is smooth, using stochastic gradient descent: at each iteration, sample an index j_k uniformly at random and update

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f_{j_k}(\mathbf{x}_k).$$

Then the iterates converge linearly to a local minimum of f .

- (s) In the same finite-sum setting as the previous part, consider applying a Newton step to a single randomly sampled objective term:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - (\nabla^2 f_{j_k}(\mathbf{x}_k))^{-1} \nabla f_{j_k}(\mathbf{x}_k).$$

Then the iterates converge quadratically to a local minimum of f .

Exercise 4.2

Calculate the gradient and Hessian of

$$f(\mathbf{x}) = e^{-x_1^2} \sin x_2 + \log x_3, \quad \mathbf{x} \in \mathbb{R}^3.$$

Exercise 4.3

Let $f : \mathbb{R}^N \rightarrow \mathbb{R}$ be twice continuously differentiable, and suppose we store its Hessian $\nabla^2 f(\mathbf{x})$ as a dense $N \times N$ array.

- Using a property of the Hessian, propose a way to reduce the number of entries that have to be stored. How many entries does your scheme need?
- Does this improve the storage complexity in big-O notation?

Exercise 4.4

Write the second-order Taylor expansion of

$$f(\mathbf{x}) = x_1^2 x_2 + e^{x_2}, \quad \mathbf{x} \in \mathbb{R}^2,$$

about the point $\mathbf{x}^* = (1, 0)^\top$.

Exercise 4.5

Consider the function

$$f(\mathbf{x}) = \sum_{i=1}^{N-1} (x_{i+1} - x_i)^2 + \sum_{i=1}^N e^{x_i}, \quad \mathbf{x} \in \mathbb{R}^N.$$

- Evaluate the Hessian $\nabla^2 f(\mathbf{x})$.
- How many nonzero entries does the Hessian have? What is the storage complexity of storing the Hessian in a sparse format (where only the nonzero entries are stored), and how does it compare to storing it as a dense matrix?

Exercise 4.6

Species A costs \$5 per mole. Species B costs \$3 per mole. We want to minimize the cost of material in a batch reactor.

- Write the unconstrained minimization problem. Does it have a solution? If so, provide one; if not, explain why. [Hint: “unconstrained” means we do not even consider the nonnegativity of the amounts.]
- Add the constraint that the total moles must equal 10 and we need at least 5 moles of B. What is the solution? Which constraints are active at the optimum? Sketch the objective gradient and the constraints to explain where the solution is located.
- Instead of the constraint $n_B \geq 5$, suppose we require $n_A \geq 5$. What is the new solution? Which constraints are active at the optimum?

Exercise 4.7

A splitter separates an inlet flow of 3 L/s into two outlet streams such that one split contains twice as much material as the other.

- Formulate this as a norm minimization problem by defining an error function that captures the constraints.
- Is the solution of this norm minimization problem the same as the solution of the corresponding linear-algebra problem? Explain why.

Exercise 4.8

Classify each of the following optimization problems as a linear program (LP), a nonlinear program (NLP), a mixed-integer linear program (MILP), or a mixed-integer nonlinear program (MINLP). Briefly justify your answers.

(a) $\min_{x_1, x_2} 3x_1 + 2x_2$ subject to $x_1 + x_2 \geq 4$, $x_1 - x_2 \leq 2$, $x_1, x_2 \geq 0$

(b) $\min_{x_1, x_2} (x_1 - 1)^2 + x_2^2$ subject to $x_1^2 + x_2^2 \leq 4$

(c) $\min_{x_1, x_2, y} 5x_1 + 4x_2 + 10y$ subject to $x_1 + x_2 \geq 3$, $x_1 \leq 6y$, $x_1, x_2 \geq 0$, $y \in \{0, 1\}$

(d) $\min_{x, y} (x - 2)^2 + 3y$ subject to $x \leq 5y$, $x \geq 0$, $y \in \{0, 1\}$

Exercise 4.9

For each of the following sets, determine whether it is a convex set; for each of the following functions, determine whether it is a convex function. Briefly justify your answers.

(a) $\{\mathbf{x} \in \mathbb{R}^2 : x_1^2 + x_2^2 = 1\}$

(b) $\{\mathbf{x} \in \mathbb{R}^2 : x_2 \geq x_1^2\}$

(c) $\{x \in \mathbb{R} : x^2 \geq 1\}$

(d) $f(x) = x^3$, for $x \in \mathbb{R}$

(e) $f(\mathbf{x}) = x_1x_2$, for $\mathbf{x} \in \mathbb{R}^2$

(f) $f(\mathbf{x}) = \max\{g(\mathbf{x}), h(\mathbf{x})\}$, where g and h are convex functions on $\mathbb{R}^N$

Exercise 4.10

For each of the following functions $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ and points $\bar{\mathbf{x}}$, determine whether $\bar{\mathbf{x}}$ is a local minimum, a local maximum, a saddle point, or none of these. Justify your answers using the gradient and the Hessian.

(a) $f(\mathbf{x}) = x_1^2 - 2x_1 + 4x_2^2$, $\bar{\mathbf{x}} = (1, 0)^\top$

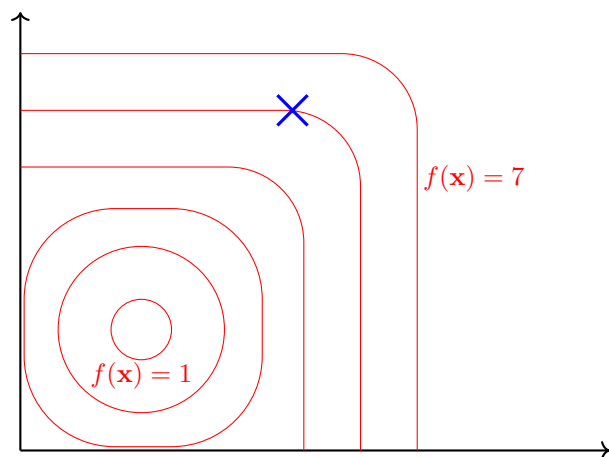
(b) $f(\mathbf{x}) = -x_1^2 + 2x_1x_2 - 2x_2^2$, $\bar{\mathbf{x}} = (0, 0)^\top$

(c) $f(\mathbf{x}) = x_1^2 + 4x_1x_2 + x_2^2$, $\bar{\mathbf{x}} = (0, 0)^\top$

(d) $f(\mathbf{x}) = x_1^2 + x_2^2 - 2x_2$, $\bar{\mathbf{x}} = (0, 0)^\top$

Exercise 4.11

Estimate the path taken by the steepest-descent method with large and small step sizes.



Exercise 4.12

(*) Consider optimization algorithms with step updates of the form $\mathbf{x}_{i+1} = \mathbf{x}_i + \alpha d$ where d is a descent direction. To choose the step size α , we model the objective along the step by its second-order Taylor expansion:

$$m(\alpha) = f(\mathbf{x}_i) + \alpha \nabla f(\mathbf{x}_i)^\top d + \frac{1}{2} \alpha^2 d^\top \nabla^2 f(\mathbf{x}_i) d.$$

Suppose we design the algorithm treating this model as exact; that is, we choose the optimal step size α_{opt} that minimizes $m(\alpha)$.

- Derive α_{opt} for steepest descent, where $d = -\nabla f(\mathbf{x}_i)$.
- Derive α_{opt} for Newton's method, where $d = -(\nabla^2 f(\mathbf{x}_i))^{-1} \nabla f(\mathbf{x}_i)$.

Exercise 4.13

Consider the convex quadratic optimization problem

$$\min_{\mathbf{x} \in \mathbb{R}^N} f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x},$$

where $\mathbf{A} \in \mathbb{R}^{N \times N}$ is symmetric positive definite, with eigenvalues $0 < \lambda_{\min} \leq \dots \leq \lambda_{\max}$, and $\mathbf{b} \in \mathbb{R}^N$. We minimize f by gradient descent with a fixed step size $\omega > 0$.

- Write the gradient descent iteration, and show that it is exactly the Richardson iteration (Section 2.8.5) applied to a linear system. What is that linear system?
- Let $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}^*$ be the error at iteration k , where $\mathbf{x}^*$ is the minimizer. Derive the recursion satisfied by $\mathbf{e}_k$, and state the condition under which the iteration converges for every starting point. For which step sizes $\omega > 0$ does gradient descent converge?
- Express the spectral radius of the iteration matrix in terms of ω , $\lambda_{\min}$, and $\lambda_{\max}$, and find the step size ω_{opt} that minimizes it.
- Express the resulting convergence rate in terms of the condition number $\kappa(\mathbf{A})$. What happens as $\kappa(\mathbf{A}) \rightarrow \infty$, and roughly how many iterations are needed to reduce the error by a factor of ϵ ?

Exercise 4.14

Recall from Exercise 2.17 that the Cholesky factorization of a symmetric matrix succeeds if and only if that matrix is positive definite, so simply attempting the factorization is a cheap test for positive definiteness. In MATLAB, the two-output form `[L, p] = chol(A, 'lower')` performs exactly this test without raising an error: it returns $\mathbf{p} = 0$ when $\mathbf{A}$ is positive definite, and $\mathbf{p} > 0$ otherwise.

Below is a MATLAB implementation of a modified Newton method for $\min_{\mathbf{x}} f(\mathbf{x})$, with two blanks marked ???.

```
function [x, status] = newton(f, grad_f, hess_f, x0, tol)
% Modified Newton method for min f(x).
    x      = x0;
    delta0 = 1e-3;
    n      = numel(x);

    for k = 1:100
        g = grad_f(x);
        H = hess_f(x);

        delta = 0;
        while true
            [L, p] = chol(H + delta*eye(n), 'lower');
            if ???
                break
            end
            delta = max(2*delta, delta0);
        end

        if norm(g) <= tol
            if ???
                status = 'local minimum';
            else
                status = 'first-order point';
            end
            return
        end

        d      = L' \ (L \ (-g));
        alpha = backtracking(f, x, g, d);
        x      = x + alpha*d;
    end

    status = 'iteration limit reached';
end

function alpha = backtracking(f, x, g, d)
    alpha = 1;
    c      = 1e-4;
    fx      = f(x);
    while f(x + alpha*d) > fx + c*alpha*(g'*d)
        alpha = alpha/2;
        if alpha < 1e-12
            break
        end
    end
end
```

- Why must the matrix $H + \text{delta} \cdot \text{eye}(n)$ used in the linear solve be positive definite?
- A student proposes deleting the `while` loop and simply setting `delta = 1e-3` at every iteration. What is wrong with this?

- (c) Fill in the first blank, which decides when the shift is large enough.
- (d) Fill in the second blank, which decides which of the two statuses is reported. Explain why the two reports differ: what can be concluded about $\mathbf{x}$ in each case?

Exercise 4.15

A student writes the following MATLAB function for the unconstrained problem $\min_{\mathbf{x}} f(\mathbf{x})$ and calls it Newton's method:

```
function x = frozen_newton(grad_f, hess_f, x0, tol)
% Is this Newton's method?
    x = x0;
    H = hess_f(x0);

    for k = 1:100
        g = grad_f(x);
        if norm(g) <= tol
            return
        end
        x = x - H \ g;
    end
end
```

Run on a problem where Newton's method is known to converge quadratically, and started close to the solution, the algorithm is observed to converge, but only *linearly*: the error is seen to contract by a roughly constant factor at every iteration, instead of the number of correct digits doubling.

Explain why the convergence is not quadratic.

Exercise 4.16

Consider minimizing the function

$$f(\mathbf{x}) = \sum_{j=1}^M (\mathbf{a}_j^\top \mathbf{x} - b_j)^2, \quad \mathbf{x} \in \mathbb{R}^N,$$

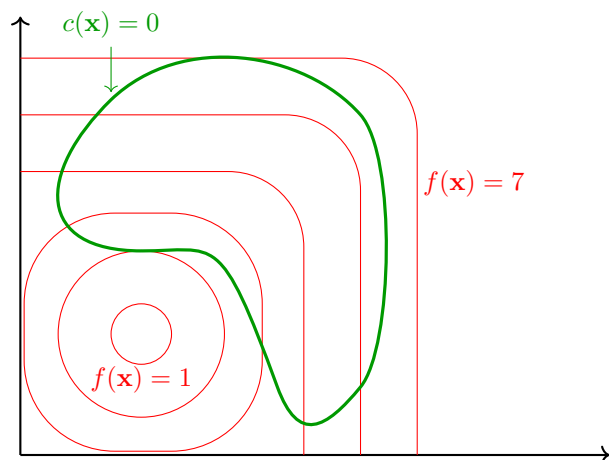
where each $\mathbf{a}_j \in \mathbb{R}^N$ is a dense vector and $b_j \in \mathbb{R}$, and the number of terms M need not equal N .

- (a) What is the computational complexity of performing one gradient descent step $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)$ (with a given step size α_k)?
- (b) What is the computational complexity of performing one Newton step, i.e., solving $\nabla^2 f(\mathbf{x}_k) \mathbf{d}_k = -\nabla f(\mathbf{x}_k)$ and setting $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{d}_k$?

Exercise 4.17

Identify the solution to the constrained optimization problem

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad c(\mathbf{x}) = 0.$$



Exercise 4.18

Consider $f(\mathbf{x}) : \mathbb{R}^3 \rightarrow \mathbb{R}$ and $\mathbf{c}(\mathbf{x}) : \mathbb{R}^3 \rightarrow \mathbb{R}^2$ given by

$$f(\mathbf{x}) = x_1^2 + 2x_2^2 + 3x_3^2, \quad \mathbf{c}(\mathbf{x}) = \begin{bmatrix} x_1 + x_2 + x_3 - 3 \\ x_1 - x_2 \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}.$$

For each of the following candidate points, determine whether it satisfies the first-order necessary conditions for a constrained minimum.

- (a) $\mathbf{x} = [1, 1, 1]^\top$
- (b) $\mathbf{x} = [1, 0, 1/6]^\top$
- (c) $\mathbf{x} = [1.2, 1.2, 0.6]^\top$

Exercise 4.19

Consider the equality-constrained optimization problem

$$\begin{aligned} \min_{x_1, x_2} \quad & x_1 + x_2 \\ \text{subject to} \quad & x_1^2 + x_2^2 = 2. \end{aligned}$$

- (a) Find all points $(\mathbf{x}, \lambda)$ that satisfy the first-order necessary conditions of optimality.
- (b) For each point found in part (a), determine whether $\mathbf{x}$ is a local solution (a local minimum) of the problem.

Exercise 4.20

Consider the equality-constrained optimization problem

$$\begin{aligned} \min_{x_1, x_2} \quad & x_1^2 + x_2^2 \\ \text{subject to} \quad & x_1 + x_2 = 2. \end{aligned}$$

You are given the candidate pair

$$\bar{\mathbf{x}} = (1, 1)^\top, \quad \bar{\lambda} = 1.$$

Verify that this pair does not satisfy the first-order necessary conditions of optimality. Does it follow that $\bar{\mathbf{x}}$ is not a local solution?

Exercise 4.21

Consider the equality-constrained quadratic optimization problem

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^N} \quad & \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{q}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A} \mathbf{x} = \mathbf{b}, \end{aligned}$$

where $\mathbf{Q} \in \mathbb{R}^{N \times N}$ is symmetric, $\mathbf{q} \in \mathbb{R}^N$, $\mathbf{A} \in \mathbb{R}^{M \times N}$, and $\mathbf{b} \in \mathbb{R}^M$. Write the linear system of equations that corresponds to the first-order necessary conditions of optimality.

Exercise 4.22

The interior-point method applied to

$$\begin{aligned} \min_{x_1, x_2} \quad & (x_1 - 2)^2 + (x_2 - 1)^2 \\ \text{subject to} \quad & x_1 + x_2 \leq 2 \end{aligned}$$

solves a sequence of barrier subproblems

$$\min_{\mathbf{x}} \phi(\mathbf{x}, \mu) = (x_1 - 2)^2 + (x_2 - 1)^2 - \mu \log(h(\mathbf{x})), \quad h(\mathbf{x}) = 2 - x_1 - x_2.$$

The following MATLAB function is meant to minimize one such subproblem by Newton's method.

```
function x = barrier_newton(x0, mu, tol)
% Newton's method for the barrier subproblem
% phi(x,mu) = (x1-2)^2 + (x2-1)^2 - mu*log(2 - x1 - x2)
x = x0;

for k = 1:100
    h = 2 - x(1) - x(2);
    g = [2*(x(1)-2); 2*(x(2)-1)] + (mu/h)*[1; 1];
    H = [2 0; 0 2];

    if norm(g) <= tol
        return
    end
    x = x - H \ g;
end
end
```

Called with $\mathbf{x}_0 = [1; 0]$, $\mu = 1$ and $\text{tol} = 1\text{e-}10$, it fails: the iterates leave the feasible region, where $h(\mathbf{x}) \leq 0$ and the barrier is not defined. The function contains one bug. Find it and give the corrected line.

Exercise 4.23

We want to solve the inequality-constrained optimization problem

$$\begin{aligned} \min_{x_1, x_2} \quad & (x_1 - 3)^2 + x_2^2 \\ \text{subject to} \quad & x_1 \leq 1 \end{aligned}$$

using the interior-point method: with $h(\mathbf{x}) = 1 - x_1$, we minimize the barrier subproblem

$$\min_{\mathbf{x}} \phi(\mathbf{x}, \mu) = (x_1 - 3)^2 + x_2^2 - \mu \log(h(\mathbf{x}))$$

with Newton's method for a decreasing sequence of barrier parameters μ , starting from the initial point $\mathbf{x}_0 = (2, 0)^\top$. Identify the potential issue with this setup.

Chapter 5

Probability

Many processes are not deterministic. Factors such as molecular fluctuations, measurement inaccuracies, and turbulent flow introduce inherent randomness to physical observables. At the microscale, events like chemical reactions are the result of stochastic collisions between molecules. Even at the macroscale, uncertainties in parameter values (e.g., reaction rates, heat transfer coefficients) and external disturbances (e.g., feed composition variations, environmental changes) lead to stochastic system behavior. Uncertainty in values propagates through models, leading to uncertainty in predictions.

To model, simulate, and design systems under these conditions, we need a basic understanding of probability theory. This chapter provides a foundation in the principles of probability, random variables, and distributions that we will need for the numerical methods discussed later on in this book. While not exhaustive, this chapter covers the essential concepts and tools required to be mathematically precise about uncertainty and randomness in engineering contexts. We'll start simple and almost overly formal, and build our way up in complexity so that we're able to talk about probabilities in a quantitative, mathematical sense as needed for model fitting, parameter estimation, and simulation.

5.1 Events, Sample Spaces, and Axioms

An **experiment** is any process with a well-defined set of outcomes. The set of all possible outcomes is the **sample space**, denoted by $\mathcal{E}$. For the simple experiment of rolling a standard six-sided die, the sample space is a discrete set of six **outcomes**: $\mathcal{E} = \{1, 2, 3, 4, 5, 6\}$. For a more complex experiment, like measuring a reactor's temperature in Kelvin, the sample space is continuous: $\mathcal{E} = (0, \infty)$ K. A single performance of an experiment is called a **trial**. If we run one trial, we will observe one outcome.

We are usually interested in more than just single outcomes. An **event** is any subset of the sample space ($A \subseteq \mathcal{E}$). For our die roll, the event “the outcome is even” corresponds to the subset $A = \{2, 4, 6\}$. For the temperature measurement, the event “the reactor is between 350 K and 360 K” is $A = [350, 360]$ K. This framework allows us to define the phenomena whose likelihood we want to quantify.

We often describe complex situations by combining events using the language of set theory. The **intersection** of two events, $A \cap B$, represents the event where *both* A and B occur (“A and B”). The **union**, $A \cup B$, represents the outcome where *at least one* of the events occurs (“A or B”). For example, if we define events $A = (300, \infty)$ K and $B = [0, 400)$ K, then their intersection is the event that the temperature is between 300 and 400 K, and their union is the event that the temperature is greater than 0 K. In mathematical notation, these are $A \cap B = (300, 400)$ K and $A \cup B = [0, \infty)$ K, respectively. Both of these operators are commutative, i.e., $A \cap B = B \cap A$ and $A \cup B = B \cup A$.

The **probability** of an event A , denoted $\Pr(A)$ or $P(A)$ or $\mathbb{P}(A)$, is a number in $[0, 1]$ that quantifies its likelihood. A useful way to estimate $\Pr(A)$ is via long-run frequency: if we repeat an experiment N times under identical conditions and A occurs N_A times, then N_A/N is an estimate of $\Pr(A)$. In idealized settings,

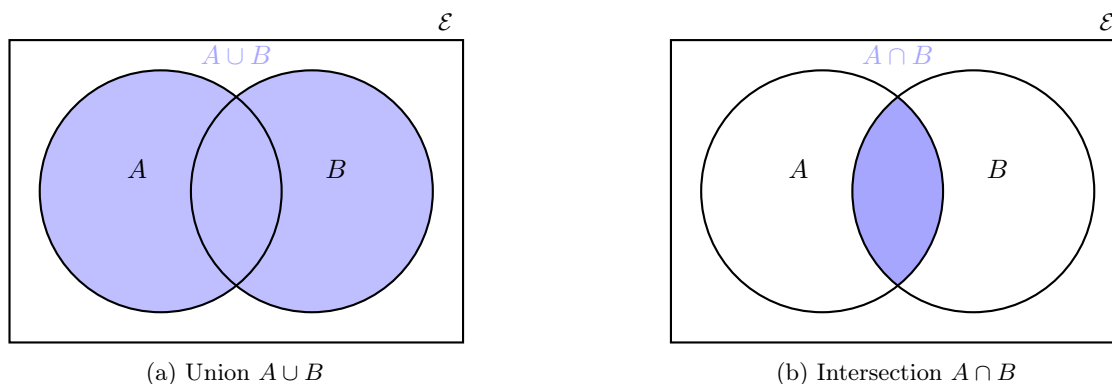


Figure 5.1.1: Venn diagrams for events A and B in sample space $\mathcal{E}$: union and intersection.

according to the strong law of large numbers, this converges to the true probability as $N \rightarrow \infty$:

$$\Pr(A) = \lim_{N \rightarrow \infty} \frac{N_A}{N} \quad (5.1.1)$$

Probabilities must obey a few simple rules. These are Kolmogorov's axioms and are listed below.

The Axioms of Probability: For any events A_i within a sample space $\mathcal{E}$:

1. **Non-negativity:** The probability of any event is non-negative.

$$\Pr(A_i) \geq 0 \quad (5.1.2)$$

2. **Unity:** The probability of the entire sample space is 1. Some outcome must occur.

$$\Pr(\mathcal{E}) = 1 \quad (5.1.3)$$

3. **Additivity:** If events $\{A_i\}_{i=1}^{\infty}$ are pairwise disjoint,

$$\Pr\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} \Pr(A_i) \quad (5.1.4)$$

For example, if two events are mutually exclusive (i.e., they cannot both occur, $A \cap B = \emptyset$), the probability that either one occurs is the sum of their individual probabilities: $\Pr(A \cup B) = \Pr(A) + \Pr(B)$. To help visualize this, one can think of this two-event, mutually exclusive case as a figure similar to Figure 5.1.1a but with the two circles non-overlapping. To get the total area of the two circles, the areas can just be added. The non-mutually exclusive case is covered by the law of total probability later on.

Note that from these axioms, we immediately have that (1) the probability of the complement of an event A^c (the event that A does not occur) is $\Pr(A^c) = 1 - \Pr(A)$, and (2) $\Pr(\emptyset) = 0$.

5.2 Conditional Probability and Independence

The axioms of probability describe static situations. However, we often need to update our assessment of an event's likelihood in light of new information. The framework for this is **conditional probability**.

The conditional probability of event A occurring given that event B has occurred is denoted $\Pr(A | B)$. When talking about this value, we would say that it is the “probability of A given B ”. Intuitively, knowing that B

has happened restricts our sample space from $\mathcal{E}$ to just the outcomes in B —it gives us information. The probability of A is then the fraction of this new, smaller sample space that is also occupied by A (Figure 5.2.1). This intuition is formalized by the definition:

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)}, \quad \text{provided } \Pr(B) > 0 \quad (5.2.1)$$

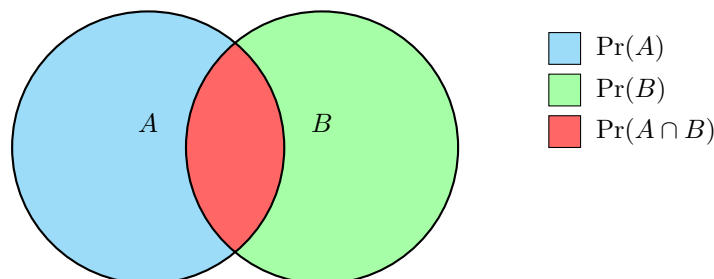


Figure 5.2.1: When we condition on B , the sample space is restricted to just the B circle, so the probability of A given B is $\Pr(A \mid B) = \Pr(A \cap B) / \Pr(B)$

The formula in Eq. 5.2.1 is one of the most important in all of probability. By rearranging it, we get the multiplication rule:

$$\Pr(A \cap B) = \Pr(A \mid B) \Pr(B) \quad (5.2.2)$$

Example (Rolling a Die): Let's roll a standard six-sided die, so $\mathcal{E} = \{1, 2, 3, 4, 5, 6\}$. Let A be the event of rolling a 3, so $A = \{3\}$, and B be the event of rolling an odd number, so $B = \{1, 3, 5\}$. The unconditional probability is clearly $\Pr(A) = 1/6$. Now, suppose we are told the roll was odd (event B). What is the new probability of A ? The intersection is $A \cap B = \{3\}$, with $\Pr(A \cap B) = 1/6$. The probability of our condition is $\Pr(B) = 3/6 = 1/2$. Using Eq. 5.2.1,

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)} = \frac{1/6}{3/6} = \frac{1}{3} \quad (5.2.3)$$

This matches our intuition: once we know the number is odd, there are only three possibilities (1, 3, or 5), each equally likely, so the chance of it being a 3 is $1/3$. Our belief was updated by the new information.

When the occurrence of one event gives us no information about another, we say the events are **independent**. We define independence via the product rule:

$$A \text{ and } B \text{ are independent} \iff \Pr(A \cap B) = \Pr(A) \Pr(B) \quad (5.2.4)$$

When $\Pr(B) > 0$, this is equivalent to the conditional statement $\Pr(A \mid B) = \Pr(A)$; conversely, $\Pr(B \mid A) = \Pr(B)$ together with $\Pr(B) > 0$ implies Eq. 5.2.4. This property extends to any number of *mutually independent* events. Specifically, events $A_1, \dots, A_N$ are mutually independent if for every nonempty subset $\mathcal{I} \subseteq \{1, \dots, N\}$,

$$\Pr\left(\bigcap_{i \in \mathcal{I}} A_i\right) = \prod_{i \in \mathcal{I}} \Pr(A_i) \quad (5.2.5)$$

For example, $\Pr(A \cap B \cap C) = \Pr(A) \Pr(B) \Pr(C)$ holds when A, B, C are mutually independent.

Warning: Pairwise independence (every pair independent) does not, in general, imply mutual independence! Here's an example: Toss two fair coins. Let $A = \{\text{first is H}\}$, $B = \{\text{second is H}\}$, and $C = \{\text{coins show the same face}\}$. Then $\Pr(A) = \Pr(B) = \Pr(C) = 1/2$ and each pair is independent

(e.g., $\Pr(A \cap B) = 1/4 = \Pr(A) \Pr(B)$, $\Pr(A \cap C) = 1/4 = \Pr(A) \Pr(C)$, $\Pr(B \cap C) = 1/4 = \Pr(B) \Pr(C)$), but $\Pr(A \cap B \cap C) = 1/4 \neq \Pr(A) \Pr(B) \Pr(C) = 1/8$, so A, B, C are pairwise independent yet not mutually independent.

Example (Independent Safety Valves): A process has three independent safety valves. Each valve has a 5% chance of being defective. What is the probability that at least one valve is defective?

Let D_i be the event that valve i is defective. We are given $\Pr(D_i) = 0.05$. The event that a valve works correctly, W_i , is the complement, so $\Pr(W_i) = 1 - \Pr(D_i) = 0.95$.

Calculating the probability of “at least one defective” directly is complicated (one fails, or two fail, or all three fail). It is far easier to calculate the probability of the complement event: “all three valves work correctly.” Since the valves are independent, we can use [Eq. 5.2.4](#):

$$\Pr(W_1 \cap W_2 \cap W_3) = \Pr(W_1) \Pr(W_2) \Pr(W_3) = (0.95)^3 \approx 0.8574 \quad (5.2.6)$$

The event “at least one valve is defective” is the complement of “all three valves work correctly.” Therefore, its probability is

$$\Pr(\text{at least one defective}) = 1 - \Pr(\text{all work}) = 1 - 0.8574 = 0.1426 \quad (5.2.7)$$

So, there is a 14.26% chance that the system has at least one point of failure.

5.2.1 Law of Total Probability

It is often the case that we might not know $\Pr(B)$ directly, but we can compute the conditional probabilities $\Pr(B | A_j)$ and $\Pr(A_j)$ for each j . In this case, we can use the law of total probability to compute $\Pr(B)$. Let $\{A_j\}_{j=1}^m$ be a pairwise disjoint ($A_j \cap A_k = \emptyset$ for $j \neq k$) and collectively exhaustive ($\bigcup_{j=1}^m A_j = \mathcal{E}$) partition of the sample space $\mathcal{E}$ with $\Pr(A_j) > 0$. In simple terms, one and only one A_i occurs. For any event B ,

$$\Pr(B) = \sum_{j=1}^m \Pr(B | A_j) \Pr(A_j) \quad (5.2.8)$$

5.2.2 Inclusion-Exclusion and Union Bounds*

Often, we want the probability that at least one of several events occurs. The Venn diagrams in [Figures 5.1.1a](#) and [5.1.1b](#) already hint at the key idea: if we simply add $\Pr(A)$ and $\Pr(B)$, we overcount the overlap $A \cap B$, so we must subtract it once. This leads to the **inclusion-exclusion principle**. The case of two and three events ([Eq. 5.2.9](#) and [Eq. 5.2.10](#)) are the most common and are potentially familiar to you from previous math classes. The general formula is given by [Eq. 5.2.11](#). All the general formula is doing is applying the addition rule repeatedly and then accounting for the overcounting appropriately. In particular, the alternating signs in [Eq. 5.2.11](#) correct for overcounting: singles are counted once, pairwise overlaps were counted twice so we subtract them, triple overlaps were then subtracted too many times so we add them back, and so on.

Inclusion-Exclusion (IE)

$$\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B) \quad (5.2.9)$$

$$\begin{aligned} \Pr(A \cup B \cup C) &= \Pr(A) + \Pr(B) + \Pr(C) \\ &\quad - [\Pr(A \cap B) + \Pr(A \cap C) + \Pr(B \cap C)] \\ &\quad + \Pr(A \cap B \cap C) \end{aligned} \quad (5.2.10)$$

$$\begin{aligned} \Pr\left(\bigcup_{i=1}^N A_i\right) &= \sum_i \Pr(A_i) - \sum_{i < j} \Pr(A_i \cap A_j) + \sum_{i < j < k} \Pr(A_i \cap A_j \cap A_k) \mp \cdots \\ &\quad + (-1)^{r+1} \sum_{i_1 < \cdots < i_r} \Pr(A_{i_1} \cap \cdots \cap A_{i_r}) \mp \cdots \\ &\quad + (-1)^{N+1} \Pr(A_1 \cap \cdots \cap A_N) \end{aligned} \quad (5.2.11)$$

We note that IE holds without any independence assumptions. Independence is only used, when available, to evaluate the intersections, e.g., $\Pr(A \cap B) = \Pr(A) \Pr(B)$ if A and B are independent. When the events are pairwise disjoint, all intersections vanish and [Eq. 5.2.9](#) collapses to the additivity axiom, [Eq. 5.1.4](#).

Example: At Least One Defective, Revisited. Let D_1, D_2, D_3 be the events that each of three safety valves is defective. With independence and $\Pr(D_i) = 0.05$,

$$\sum_i \Pr(D_i) = 3(0.05) = 0.15 \quad (5.2.12)$$

$$\sum_{i < j} \Pr(D_i \cap D_j) = 3(0.05)^2 = 0.0075 \quad (5.2.13)$$

$$\Pr(D_1 \cap D_2 \cap D_3) = (0.05)^3 = 0.000125 \quad (5.2.14)$$

By [Eq. 5.2.10](#),

$$\Pr(D_1 \cup D_2 \cup D_3) = 0.15 - 0.0075 + 0.000125 = 0.142625 \quad (5.2.15)$$

which matches the complement calculation $1 - (0.95)^3$ from earlier.

Union bounds and Bonferroni inequalities* Exact IE can be tedious for large N . Truncating [Eq. 5.2.11](#) gives us useful bounds, known as Bonferroni inequalities. Keeping only the first (positive) term gives the union bound (a.k.a. Boole's inequality):

$$\Pr\left(\bigcup_{i=1}^N A_i\right) \leq \sum_{i=1}^N \Pr(A_i) \quad (5.2.16)$$

Including the pairwise term gives a lower bound:

$$\Pr\left(\bigcup_{i=1}^N A_i\right) \geq \sum_i \Pr(A_i) - \sum_{i < j} \Pr(A_i \cap A_j) \quad (5.2.17)$$

More generally, truncating after an odd number of terms gives an upper bound, while truncating after an even number gives a lower bound. We emphasize that these bounds follow directly from [Eq. 5.2.11](#).

Example: Quick Estimate with a Union Bound. If sensors $S_1, \dots, S_5$ have false-alarm probabilities at most 1% each, then

$$\Pr(\text{at least one false alarm}) \leq \sum_{i=1}^5 0.01 = 0.05 \quad (5.2.18)$$

This bound is immediate and does not require independence.

The takeaway here is to use [Eq. 5.2.9](#)-[Eq. 5.2.11](#) for exact computations when intersections are accessible (by symmetry, structure, or independence). When they are not, [Eq. 5.2.16](#) and [Eq. 5.2.17](#) provide fast, informative bounds. The pictures in [Figures 5.1.1a](#) and [5.1.1b](#) are a helpful guide to remember the alternating-sign corrections.

5.2.3 Simpson's Paradox

Conditional probability can lead to counterintuitive results when we fail to account for all relevant information. **Simpson's paradox** occurs when a trend appears in different groups of data but disappears or reverses when the groups are combined. When a confounding variable influences both the condition and the outcome, ignoring this variable can lead to misleading conclusions. Simpson's paradox is not truly a paradox. It's easiest to see this with an example.

Example (Simpson's Paradox): Let x denote weekly exercise hours, y resting blood pressure, and $S \in \{\text{Younger (Group A), Older (Group B)}\}$ indicate age group. In the scatter of [Figure 5.2.2](#), each age group shows the expected *negative* association: within a fixed age group, more exercise is linked to lower blood pressure. However, when the groups are pooled and the age indicator S is omitted,

$$y = \alpha + \beta x + \varepsilon, \quad \beta > 0 \quad (5.2.19)$$

so exercise appears (misleadingly) to increase blood pressure. The reversal occurs because age is a confounder: older adults (Group B) both exercise more on average (x larger) and have higher baseline blood pressure ($\alpha_B > \alpha_A$). We can see then how important it is to condition on the relevant variables.

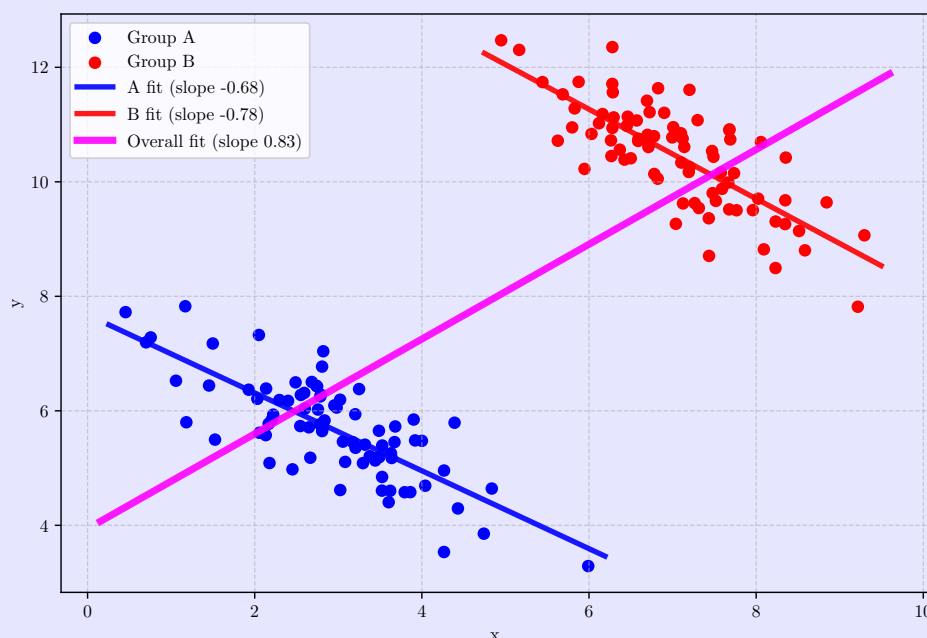


Figure 5.2.2: Simpson's paradox in a continuous setting. Within each age group (A: younger, B: older) the fitted slope is negative; pooled across groups, the fitted slope is positive.

The main message to get across here is that when analyzing data, we must carefully consider what variables to condition on. If important conditioning variables are omitted from the analysis, the results can be completely misleading. Always consider what additional information might be relevant to properly interpret probabilistic relationships. Simpson's paradox frequently arises in real-world data analysis, such as patient outcomes in healthcare stratified by demographics. There is a related converse failure mode when conditioning on the *wrong* variable—specifically a “collider” (a variable causally downstream of both the thing you're studying and an outcome)—can itself induce a spurious statistical association that wasn't present unconditionally (sometimes called Berkson's paradox, or collider bias).

5.2.4 Bayes' Theorem

The law of total probability allows us to synthesize the probability of an effect (D) from various potential causes (A, B, C). But what if we observe the effect and want to infer the cause? For instance, if we find a defective part, what is the probability that it came from Machine A? We are looking to invert the conditional probability from $\Pr(\text{Effect} \mid \text{Cause})$ to $\Pr(\text{Cause} \mid \text{Effect})$. This is the purpose of **Bayes' theorem**. The frequency-based definition of probability given earlier in this chapter and the “degree of belief” language we're about to use are, strictly speaking, two different interpretations of probability—frequentist and Bayesian, respectively. We will use both informally throughout this course.

Bayes' theorem is a direct consequence of the definition of conditional probability. From the multiplication rule, we have two ways of expressing the intersection $A_i \cap B$:

$$\begin{aligned}\Pr(A_i \cap B) &= \Pr(B \mid A_i) \Pr(A_i) \\ \Pr(A_i \cap B) &= \Pr(A_i \mid B) \Pr(B)\end{aligned}$$

Setting these equal and rearranging to solve for $\Pr(A_i \mid B)$ gives the simple form of Bayes' theorem (assuming $\Pr(B) > 0$):

$$\Pr(A_i \mid B) = \frac{\Pr(B \mid A_i) \Pr(A_i)}{\Pr(B)} \quad (5.2.20)$$

A lot of the time, we don't know $\Pr(B)$ directly, so we expand it via the law of total probability. This gives us the full form of Bayes' theorem:

Bayes' Theorem. Let $\{A_j\}$ be a partition of the sample space (mutually exclusive and collectively exhaustive) with $\Pr(A_j) > 0$, and let $\Pr(B) > 0$. Then

$$\Pr(A_i | B) = \frac{\Pr(B | A_i) \Pr(A_i)}{\sum_j \Pr(B | A_j) \Pr(A_j)} \quad (5.2.21)$$

This formula provides a systematic way to update our beliefs in light of new evidence. The terms have special names that reflect this process:

- $\Pr(A_i)$ is the **prior probability**: our initial belief about the cause before seeing any evidence.
- $\Pr(B | A_i)$ is the **likelihood**: the probability of observing the evidence B if the cause A_i were true.
- $\Pr(A_i | B)$ is the **posterior probability**: our updated belief about the cause after observing the evidence.
- $\Pr(B)$ is the **marginal likelihood** of the evidence (also just called the evidence), which serves as a normalization constant. For high-dimensional continuous parameter spaces this normalizing integral is generally intractable in closed form, which is the computational bottleneck that motivates sampling-based methods (e.g., Markov chain Monte Carlo) covered later in the course.

We will revisit these ideas in more depth in the context of Bayesian inference later on. For now, we illustrate Bayes' theorem with a couple of examples.

Example (Diagnostic Testing): A diagnostic test for pipeline corrosion has a sensitivity of 85% (it correctly identifies corrosion 85% of the time) and a specificity of 99.9% (it correctly reports no corrosion 99.9% of the time). Industry data suggests that this corrosion is present in 1% of pipes of a certain age. If a pipe tests positive, what is the probability it actually has corrosion?

Let C be the event the pipe has corrosion, and C^c be the event it does not. Let T^+ be a positive test result. We are given the prior that $\Pr(C) = 0.01$, so $\Pr(C^c) = 0.99$. We also know that the sensitivity is $\Pr(T^+ | C) = 0.85$, and the specificity is $\Pr(T^- | C^c) = 0.999$, which implies a false positive rate of $\Pr(T^+ | C^c) = 1 - 0.999 = 0.001$.

We want to find the posterior, $\Pr(C | T^+)$. Using Bayes' Theorem (Eq. 5.2.21):

$$\Pr(C | T^+) = \frac{\Pr(T^+ | C) \Pr(C)}{\Pr(T^+ | C) \Pr(C) + \Pr(T^+ | C^c) \Pr(C^c)} \quad (5.2.22)$$

Plugging in the numbers:

$$\Pr(C | T^+) = \frac{0.85 \times 0.01}{0.85 \times 0.01 + 0.001 \times 0.99} = \frac{0.0085}{0.0085 + 0.00099} = \frac{0.0085}{0.00949} \approx 0.896 \quad (5.2.23)$$

A positive result suggests that the probability the pipe has corrosion is $\approx 89.6\%$. Even though the false positive rate is only 0.1%, the large proportion of non-corroded pipes (i.e., a low baseline rate of 1% corroded before we do any testing) means that false positives still make up a significant fraction ($\approx 10.4\%$) of positive test results.

5.3 Random Variables

So far, we have worked with events defined on a sample space $\mathcal{E}$. The outcomes in $\mathcal{E}$ can be numerical (like the result of a die roll) or categorical (like “heads” or “defective”). To unify the analysis of these experiments,

we introduce the concept of a **random variable (RV)**, which provides a consistent way to assign a numerical value to every possible outcome. Formally, a random variable is a function that maps an outcome from the sample space $\mathcal{E}$ to a real number.

We make a brief note on notation: To distinguish between the RV itself (which is like a die we have not yet rolled, a coin we have not yet flipped, or a non-deterministic function we have not yet evaluated) and the specific values it can take, we will adopt the convention that an RV will be denoted with a sans-serif font, like x or t , while a specific, single value that the random variable takes after a trial (known as a *realization*) will be denoted with a standard italic font, like x or t . So, if ξ is the outcome of a particular trial, $x = x(\xi)$ is the numerical value assigned to that outcome by the RV x .

Random variables allow us to define events using numerical statements and with mathematical precision. For example, instead of describing a complex set of outcomes, we can make a simple statement like “the temperature is greater than 400 K.” This translates an event in the numerical space back to a corresponding event in the original sample space:

$$\Pr(x \geq a) := \Pr(\{\xi \in \mathcal{E} \mid x(\xi) \geq a\}) \quad (5.3.1)$$

Random variables are usually classified into two types based on the values they can take. **Discrete random variables** can take on a finite or countably infinite number of distinct values. For example, we can use a discrete RV to describe the result of a coin flip: x such that $x(\text{heads}) = 1$ and $x(\text{tails}) = 0$. Here, the set of possible values is $\{0, 1\}$. Meanwhile, **continuous random variables** can take on any value within a given continuous range. The number of possible values is uncountably infinite. For example, consider the measured temperature of a reactor, T , which can be any value in $[0, \infty)$. In such cases, single points have vanishingly small probabilities: the probability that the random variable T takes on a value that is *exactly* equal to t , $\Pr(T = t)$, is infinitesimally small. Understanding the type of a random variable is the first step toward describing its behavior using a probability distribution, which we will cover next.

5.4 Describing Random Variables: Probability Distributions

Random variables give us a tool to map experimental outcomes to numbers. The next step is to describe the probabilistic behavior of that variable. We need to answer the question: “What are the probabilities that the random variable x will take on certain values?” This is the role of a **probability distribution**. There are two main functions we use to fully characterize a random variable’s distribution. The first is the **cumulative distribution function (CDF)**, which applies to all types of random variables that take on numerical values. The second function depends on whether the random variable is discrete or continuous: for discrete random variables, we use the **probability mass function (PMF)**, which also applies to random variables that take on non-numerical values; for continuous random variables, we use the **probability density function (PDF)**.

Definition: The **support** of a random variable x is the set of values where the distribution puts positive probability (discrete) or positive density (continuous). Essentially, it describes the range of values that the random variable can take on with non-zero probability. We write $\mathcal{S}_x$ for the support of x .

5.4.1 The Cumulative Distribution Function

The most fundamental description of any random variable’s distribution is the **cumulative distribution function (CDF)**. The CDF, typically denoted by $F_x(x)$, is defined as the probability that the random variable x takes on a value less than or equal to a specific value x .

$$F_x(x) := \Pr(x \leq x) \quad (5.4.1)$$

The CDF accumulates probability as x increases. By definition, it is a non-decreasing function that starts at 0 for $x \rightarrow -\infty$ and ends at 1 for $x \rightarrow +\infty$; it should be obvious why these limits are necessary given the

axioms of probability. A consequence of Eq. 5.4.1 is that for any $a < b$,

$$\Pr(a < x \leq b) = F_x(b) - F_x(a) \quad (5.4.2)$$

The use of $<$ and $\leq$ is very deliberate here: the CDF is right-continuous, but not necessarily left-continuous. Only if the CDF is (fully) continuous at a and b are we allowed to switch both bounds to $\leq$. This distinction matters for discrete random variables, where there can be jumps in the CDF at points in the support, but does not generally matter for continuous random variables.

5.4.2 The Probability Mass Function for Discrete RVs

For a discrete random variable, it's most intuitive to describe the “lump” of probability at each specific value it can take. This is the job of the **probability mass function (PMF)**, denoted by $p_x(x)$.

$$p_x(x) = \Pr(x = x) \quad (5.4.3)$$

PMFs directly give the probability of each possible value. For example, if x is the result of rolling a fair six-sided die, then its PMF is $p_x(k) = 1/6$ for $k = 1, 2, 3, 4, 5, 6$ and $p_x(k) = 0$ otherwise.

Any valid PMF must satisfy two conditions:

1. Non-negativity: $p_x(x) \geq 0$ for all x .
2. Normalization: $\sum_{x \in \mathcal{S}_x} p_x(x) = 1$.

The PMF links to the CDF via

$$F_x(x) = \sum_{a \leq x} p_x(a) \quad (5.4.4)$$

5.4.3 The Probability Density Function for Continuous RVs

For a continuous random variable, the situation is more subtle. Since there are infinitely many possible values in any range, the probability of the RV taking on any single, exact value is zero. Instead, we talk about the probability of the variable falling within a range $[a, b]$. We describe this using a **probability density function (PDF)**. We'll again use $p_x(x)$ to denote a PDF; the context should make clear whether it's a PMF or PDF. The PDF is a function we can integrate to find the probability of the random variable taking on a value in a given range:

$$\Pr(a \leq x \leq b) = \int_a^b p_x(x) dx \quad (5.4.5)$$

Like the PMF, the PDF must satisfy two conditions:

1. Non-negativity: $p_x(x) \geq 0$ for all x .
2. Normalization: $\int_{-\infty}^{\infty} p_x(x) dx = 1$.

We take the support to be $\mathcal{S}_x = \{x : p_x(x) > 0\}$, and probabilities of intervals outside $\mathcal{S}_x$ are zero. Assuming the PDF exists and is continuous (which is not always true), the PDF links to the CDF via

$$F_x(x) = \int_{-\infty}^x p_x(t) dt \iff p_x(x) = \frac{d}{dx} F_x(x) \quad (5.4.6)$$

Warning: PDF Danger Zone. Here are two points to watch out for when working with PDFs:

1. A PDF value is *not* a probability. Unlike a PMF, the value of a PDF, $p_x(x)$, is a probability *density*. As a result, it can be greater than 1. The probability is the area under the curve. For a very small interval, the probability is approximately the area of a rectangle: $\Pr(x \leq x \leq x + dx) \approx p_x(x)dx$.
2. The probability at a point is zero (if in doubt, check out Eq. 5.4.5 and realize that you would be

integrating over a zero-width interval). For any continuous RV, $\Pr(x = c) = 0$. This means that $\Pr(a \leq x \leq b)$ is the same as $\Pr(a < x < b)$.

5.5 Sampling and Expectations

The link between a theoretical probability distribution and real-world data is **sampling**. When we perform an experiment and get a realization x of a random variable x , we are “drawing a sample” from its distribution. With independent and identically distributed sampling (“i.i.d.”, a concept that comes up many times in probability theory), relative frequencies (discrete) converge to the true probabilities, and histograms/density estimators (continuous) approximate the PDF as the sample size grows. This connects our formal definitions back to the intuitive idea of probability as long-run frequency, [Eq. 5.1.1](#).

While the PDF or CDF of a RV gives us a full description of a random variable’s behavior, there are simpler summary statistics we often want to compute. We will cover some of these below and see how they can be estimated from sample data.

The Expected Value (Mean) The most important summary statistic is the **expected value** of a RV (also known as the expectation or the mean). It represents the long-run average value of the random variable over many trials and is the center of mass of the distribution. The expectation operator is denoted by $\mathbb{E}[\cdot]$ or angle brackets $\langle \cdot \rangle$. For a discrete RV,

$$\mu_x = \mathbb{E}[x] = \sum_x x p_x(x) \quad (\text{discrete}) \quad (5.5.1)$$

And for a continuous RV,

$$\mu_x = \mathbb{E}[x] = \int_{-\infty}^{\infty} x p_x(x) dx \quad (\text{continuous}) \quad (5.5.2)$$

These integrals/sums assume the expectation exists (is finite).

When we are dealing with more complex expressions involving expectation operators, it’s important to be clear about what random variable(s) the expectation is taken over and what their distributions are. We can write the above expressions equivalently as $\mathbb{E}_{x \sim p_x}[x]$ to explicitly indicate that the expectation is with respect to the distribution of x , $p_x(x)$. This notation becomes especially useful when dealing with multiple random variables or conditional expectations, where we need to specify which variable’s distribution we are averaging over. As a short-hand for conditional expectation values, $\mathbb{E}[x | y]$ represents the mean value of x according to the conditional distribution $p_{x|y}$, which we might also write as $\mathbb{E}_{x|y}[x]$. This notational clarity will be especially important when we discuss Monte Carlo methods later on.

Expectation of a Function of an RV We can also calculate the expected value of a function of a random variable, $g(x)$. Functions of random variables are themselves random variables, since their values depend on the outcome of the original random variable. You can think of $g(x)$ as applying a transformation to the original random variable x . When we sample x , we can compute $g(x)$ for that sample, and over many samples, we can analyze the distribution of $g(x)$.

For a discrete RV,

$$\mathbb{E}[g(x)] = \sum_x g(x) p_x(x) \quad (\text{discrete}) \quad (5.5.3)$$

And for a continuous RV,

$$\mathbb{E}[g(x)] = \int_{-\infty}^{\infty} g(x) p_x(x) dx \quad (\text{continuous}) \quad (5.5.4)$$

These formulas are often referred to as the law of the unconscious statistician (commonly known as LOTUS).

A very important property of the expectation operator is that it is a linear operator, meaning $\mathbb{E}[ax + b] = a\mathbb{E}[x] + b$. This property allows us to convert the expectation of a sum to the sum of individual expectations. However, for a general nonlinear function g , it is *not* true that $\mathbb{E}[g(x)] = g(\mathbb{E}[x])$.

Example ($\mathbb{E}[g(x)] \neq g(\mathbb{E}[x])$): Consider rolling a fair six-sided die. The RV x can take values $\{1, 2, 3, 4, 5, 6\}$, each with $p_x(x) = 1/6$. The expected value is $\mathbb{E}[x] = (1 + 2 + 3 + 4 + 5 + 6)/6 = 3.5$. Now, let's consider the function $g(x) = x^2$. What is $\mathbb{E}[x^2]$?

$$\mathbb{E}[x^2] = \sum_{x=1}^6 x^2 p_x(x) = \frac{1}{6}(1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 6^2) = \frac{91}{6} \approx 15.17$$

However, $g(\mathbb{E}[x]) = (\mathbb{E}[x])^2 = (3.5)^2 = 12.25$. As we can see, $\mathbb{E}[x^2] \neq (\mathbb{E}[x])^2$.

Variance and Standard Deviation The **variance** measures the spread or dispersion of a distribution around its mean. It is defined as the expected squared deviation from the mean.

$$\sigma_x^2 = \text{Var}(x) := \mathbb{E}[(x - \mu_x)^2] \quad (5.5.5)$$

A more convenient formula for computation can be derived using the linearity of expectation:

$$\begin{aligned} \sigma_x^2 &= \mathbb{E}[(x - \mathbb{E}[x])^2] \\ &= \mathbb{E}[x^2 - 2x\mathbb{E}[x] + (\mathbb{E}[x])^2] \\ &= \mathbb{E}[x^2] - \mathbb{E}[2x\mathbb{E}[x]] + \mathbb{E}[(\mathbb{E}[x])^2] \quad (\text{by linearity}) \\ &= \mathbb{E}[x^2] - 2\mathbb{E}[x]\mathbb{E}[x] + (\mathbb{E}[x])^2 \quad (\text{since } \mathbb{E}[x] \text{ is a constant}) \\ &= \mathbb{E}[x^2] - (\mathbb{E}[x])^2 \end{aligned} \quad (5.5.6)$$

The **standard deviation**, $\sigma_x = \sqrt{\text{Var}(x)}$, is the square root of the variance and has the same units as the random variable itself. Other properties like skewness (asymmetry) and kurtosis (tailedness) are defined by higher-order centered moments.

$$\text{Skewness} = \frac{\mathbb{E}[(x - \mu_x)^3]}{\sigma_x^3} \quad (5.5.7)$$

$$\text{Kurtosis} = \frac{\mathbb{E}[(x - \mu_x)^4]}{\sigma_x^4} \quad (5.5.8)$$

Means and variances of linear combinations of RVs For random variables x and y , and constants a and b , the linearity of the expectation operator gives

$$\mathbb{E}[ax + by] = a\mathbb{E}[x] + b\mathbb{E}[y] \quad (5.5.9)$$

If x and y are independent, then

$$\text{Var}(ax + by) = a^2 \text{Var}(x) + b^2 \text{Var}(y) \quad (5.5.10)$$

If they are not independent, then

$$\text{Var}(ax + by) = a^2 \text{Var}(x) + b^2 \text{Var}(y) + 2ab \text{Cov}(x, y) \quad (5.5.11)$$

where the covariance is introduced later in this chapter. These properties are completely general and do not depend on the type of distribution of the random variables.

Law of Iterated Expectations For random variables x and s ,

$$\mathbb{E}_{x \sim p_x}[x] = \mathbb{E}_{s \sim p_s}[\mathbb{E}_{x \sim p_{x|s}}[x | s]] \quad (5.5.12)$$

In words, this means that we can first find the conditional mean within each group s , then average across groups with the distribution of s . In discrete form,

$$\mathbb{E}[x] = \sum_j \mathbb{E}[x | s = s_j] \Pr(s = s_j) \quad (5.5.13)$$

where the sum runs over the support of s . In continuous form with $\mathcal{S}_s$ being the support of s ,

$$\mathbb{E}[x] = \int_{\mathcal{S}_s} \mathbb{E}[x | s = s] p_s(s) ds \quad (5.5.14)$$

The intuition behind LIE is that the conditional expectation $\mathbb{E}[x | s]$ is the best mean-squared-error predictor of x when s is known exactly. Before seeing s , the random variable $\mathbb{E}[x | s]$ reflects our best guess for each possible value of s . Averaging these guesses with the natural probabilities of s returns the overall best uninformed guess, $\mathbb{E}[x]$.

Example: Hits by Scaffold. In a small-molecule generator, each proposal is first sampled by scaffold cluster $s \in \{A, B, C\}$ and then instantiated with substituents. Let $h \in \{0, 1\}$ indicate whether the molecule hits (e.g., passes a docking/physics threshold). By construction, the scaffold determines the conditional hit rate $p_{\text{hit}}(s) = \mathbb{E}[h | s = s]$. Suppose the generator samples scaffolds with

$$\Pr(s = A) = 0.5, \quad \Pr(s = B) = 0.3, \quad \Pr(s = C) = 0.2 \quad (5.5.15)$$

and the conditional hit rates (estimated from prior campaigns or a calibrated surrogate) are

$$p_{\text{hit}}(A) = 0.008, \quad p_{\text{hit}}(B) = 0.021, \quad p_{\text{hit}}(C) = 0.003 \quad (5.5.16)$$

The law of iterated expectations gives the overall hit probability for a random proposal,

$$\mathbb{E}[h] = \mathbb{E}[\mathbb{E}[h | s]] = \sum_{s \in \{A, B, C\}} p_{\text{hit}}(s) \Pr(s = s) = (0.008)(0.5) + (0.021)(0.3) + (0.003)(0.2) = 0.0109 \quad (5.5.17)$$

Thus the expected hit rate is 1.09%. By linearity, for a batch of N proposals, the expected number of hits is

$$\mathbb{E}[\text{hits in batch}] = N \mathbb{E}[h] = 0.0109 N \quad (5.5.18)$$

e.g., with $N = 2000$ molecules, $\mathbb{E}[\text{hits}] = 21.8 \approx 22$.

Law of Total Variance For random variables x and s ,

$$\text{Var}(x) = \mathbb{E}[\text{Var}(x | s)] + \text{Var}(\mathbb{E}[x | s]) \quad (5.5.19)$$

We can interpret each term as follows:

- $\mathbb{E}[\text{Var}(x | s)]$: average *within-group* variability (irreducible noise once s is known).
- $\text{Var}(\mathbb{E}[x | s])$: *between-group* variability (how group means differ across s).

We will not need to invoke these laws in 10.34, but they are very useful results in probability theory and statistics.

5.6 Some Notable Probability Distributions

While there are countless probability distributions, a small number of them appear so frequently in science and engineering that they deserve special attention. They serve as fundamental building blocks for modeling a vast range of stochastic phenomena.

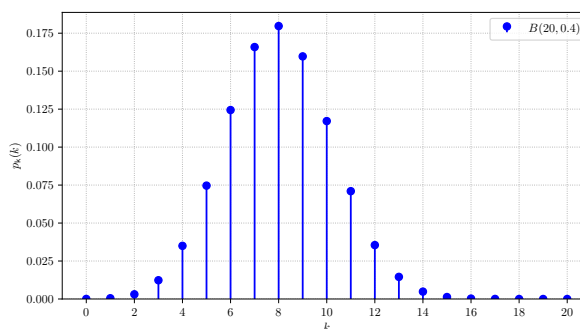
The Binomial Distribution The binomial distribution is a discrete distribution that answers the question: “If I run n independent trials, each with a success probability of p , what is the probability of getting exactly k successes?” It’s used for modeling binary outcomes (yes/no, success/failure, on/off) in a fixed number of trials. The classic example is flipping a (potentially biased) coin n times and counting the number of heads.

Let k be the random variable for the number of successes. The PMF is given by

$$p_k(k) = \binom{n}{k} p^k (1-p)^{n-k} \quad \text{for } k \in \{0, 1, \dots, n\} \quad (5.6.1)$$

This formula combines the probability of observing one specific sequence of k successes and $n - k$ failures, $p^k (1-p)^{n-k}$, multiplied by the number of ways that sequence can occur given by the binomial coefficient $\binom{n}{k} = n!/(k!(n-k)!)$.

- **Notation:** $k \sim B(n, p)$
- **Mean:** $\mathbb{E}[k] = np$
- **Variance:** $\text{Var}(k) = np(1-p)$
- **Support:** $k \in \{0, 1, \dots, n\} = \mathbb{Z}_{\geq 0}$



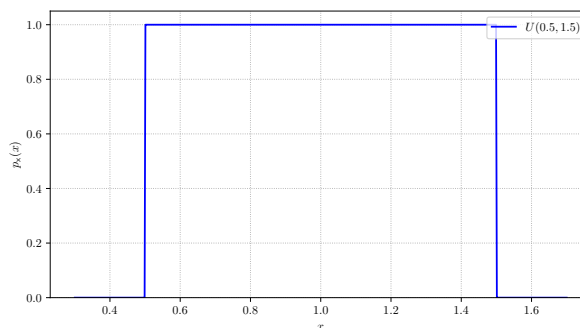
The Uniform Distribution The uniform distribution is the simplest continuous distribution. It describes a situation of complete uncertainty over a fixed interval $[a, b]$, where the probability of the outcome falling in any sub-interval is proportional only to the length of that sub-interval. Every value within the range is equally likely.

The PDF is a simple constant function:

$$p_x(x) = \begin{cases} \frac{1}{b-a} & \text{for } a \leq x \leq b \\ 0 & \text{otherwise} \end{cases} \quad (5.6.2)$$

The height of the PDF, $1/(b-a)$, is set to ensure that the total area under the curve is equal to 1.

- **Notation:** $x \sim \mathcal{U}(a, b)$
- **Mean:** $\mathbb{E}[x] = \frac{a+b}{2}$
- **Variance:** $\text{Var}(x) = \frac{(b-a)^2}{12}$
- **Support:** $x \in [a, b]$



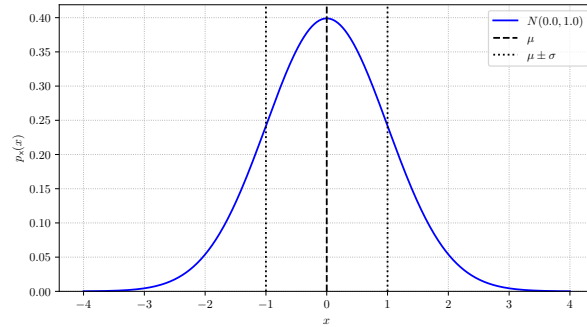
The Normal (Gaussian) Distribution The normal distribution, also known as the Gaussian distribution, is arguably the most important continuous distribution in all of probability and statistics. Its importance comes from the central limit theorem, which states that the sum of a large number of independent and identically distributed (i.i.d.) random variables will be approximately normally distributed, regardless of their individual distributions (we’ll discuss this in more detail later). This is why it appears everywhere. It is the classic “bell curve.”

The PDF is parameterized by the mean μ and the variance σ^2 :

$$p_{\mathbf{x}}(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (5.6.3)$$

The mean μ sets the center of the peak, and the standard deviation σ controls the spread of the curve. The special case where $\mu = 0$ and $\sigma^2 = 1$ is called the **standard normal distribution**. Any normal random variable can be converted to a standard normal by the transformation $z = (x - \mu)/\sigma$.

- **Notation:** $x \sim \mathcal{N}(\mu, \sigma^2)$
- **Mean:** $\mathbb{E}[x] = \mu$
- **Variance:** $\text{Var}(x) = \sigma^2$
- **Support:** $x \in \mathbb{R}$



The Gaussian distribution gives rise to additional convenient properties related to linear combinations of Gaussian random variables. If x and y are jointly normally distributed, then any linear combination $ax + by$ is also normally distributed. Combined with the above formulas for the mean and variance of linear combinations of RVs, this allows us to fully characterize the distribution of $ax + by$ as $\mathcal{N}(a\mathbb{E}[x] + b\mathbb{E}[y], a^2 \text{Var}(x) + b^2 \text{Var}(y) + 2ab \text{Cov}(x, y))$. If x and y are independent, the covariance term is zero.

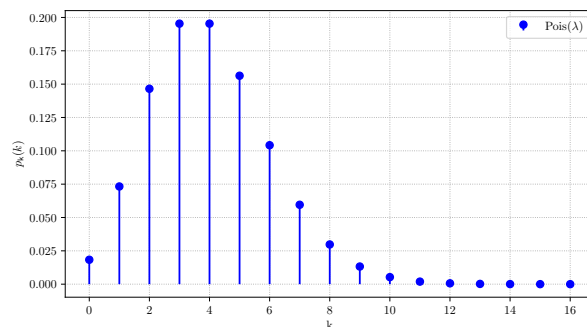
The Poisson Distribution The Poisson distribution is a discrete distribution used to model the number of times an event occurs within a fixed interval of time or space. It is the quintessential model for counting rare, independent events happening at a constant average rate, such as the number of radioactive decays in a second or the number of defects per square meter of a material.

The Poisson distribution is defined by a single parameter, λ , which is the average number of events in the given interval. The PMF for the number of observed events, k , is:

$$p_k(k) = \frac{\lambda^k e^{-\lambda}}{k!} \quad \text{for } k \in \{0, 1, 2, \dots\} \quad (5.6.4)$$

An important feature of the Poisson distribution is that its mean and variance are equal.

- **Notation:** $k \sim \text{Pois}(\lambda)$
- **Mean:** $\mathbb{E}[k] = \lambda$
- **Variance:** $\text{Var}(k) = \lambda$
- **Support:** $k \in \{0, 1, 2, \dots\} = \mathbb{Z}_{\geq 0}$



The Poisson distribution can also be viewed as an approximation of the Binomial distribution $B(n, p)$ when the number of trials n is very large and the success probability p is very small, such that $\lambda = np$.

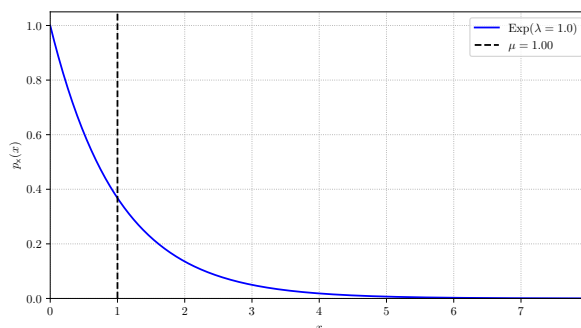
The Exponential Distribution The exponential distribution models the waiting time between events in a Poisson process with constant rate λ . It is the continuous analog of the geometric distribution and (uniquely among continuous distributions on $[0, \infty)$) has the memoryless property that the probability of an

event occurring in the next t units of time is independent of the time already elapsed. We will invoke the exponential distribution when discussing kinetic Monte Carlo methods.

The PDF is

$$p_x(x) = \begin{cases} \lambda e^{-\lambda x} & \text{for } x \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad (5.6.5)$$

- **Notation:** $x \sim \text{Exp}(\lambda)$
- **Mean:** $\mathbb{E}[x] = \frac{1}{\lambda}$
- **Variance:** $\text{Var}(x) = \frac{1}{\lambda^2}$
- **Support:** $x \in [0, \infty) = \mathbb{R}_{\geq 0}$



5.7 Recognizing Distributions by Shape

The requirement that a probability density function be normalized means that when we are describing the behavior of a random variable x with PDF $p_x(x)$, we are really describing the *shape* of the distribution—how that function depends on x . The area under the curve must equal 1, so if we know the shape, we know everything there is to know about the distribution. The normalization is necessary for $p_x(x)$ to be a valid PDF, but it is not what distinguishes one distribution from another.

As a result, distributions can be recognized by their dependence on the argument without worrying about the normalization constant. If we know that a PDF $p_x(x)$ is proportional to $\exp(-x^2/2)$, which might be written as $p_x(x) \propto \exp(-x^2/2)$, we can immediately identify it as a normal distribution with mean 0 and variance 1 without needing to compute the normalization constant $1/\sqrt{2\pi}$. Similarly, if we see a PMF proportional to $\lambda^k/k!$, we can identify it as a Poisson distribution with rate parameter λ despite the lack of the $e^{-\lambda}$ term.

Being able to recognize distributions by their shape is a useful skill in probability and statistics that will be useful when we start discussing Bayesian inference and Monte Carlo methods later in the course.

5.8 Empirical Distributions

The probability distributions we have discussed (normal, Poisson, etc.) are theoretical models. They represent a complete and perfect description of a random variable, assuming we know its underlying parameters (like μ , σ^2 , or λ). In most real-world scenarios, however, we do not have access to this true, latent distribution. Instead, what we might have is a finite collection of data points, or samples, drawn from it.

The distribution of our collected data is called the **empirical distribution**. It is our best approximation of the true, underlying distribution. For a discrete random variable, we can define the empirical probability mass function, $\hat{p}_x(x)$ ¹, as the relative frequency of each outcome in our data:

$$\hat{p}_x(x) = \frac{\text{number of times } x \text{ was observed}}{\text{total number of samples}} = \frac{N_x}{N} \quad (5.8.1)$$

The **law of large numbers** guarantees that as our sample size N grows, our empirical distribution must converge to the true underlying distribution, i.e., $\lim_{N \rightarrow \infty} \hat{p}_x(x) = p_x(x)$. The plot in [Figure 5.8.1](#) illustrates

¹The hat notation is used universally in statistics to denote an estimator, a value calculated from data that is our best guess for an unknown true value. We'll revisit estimators momentarily.

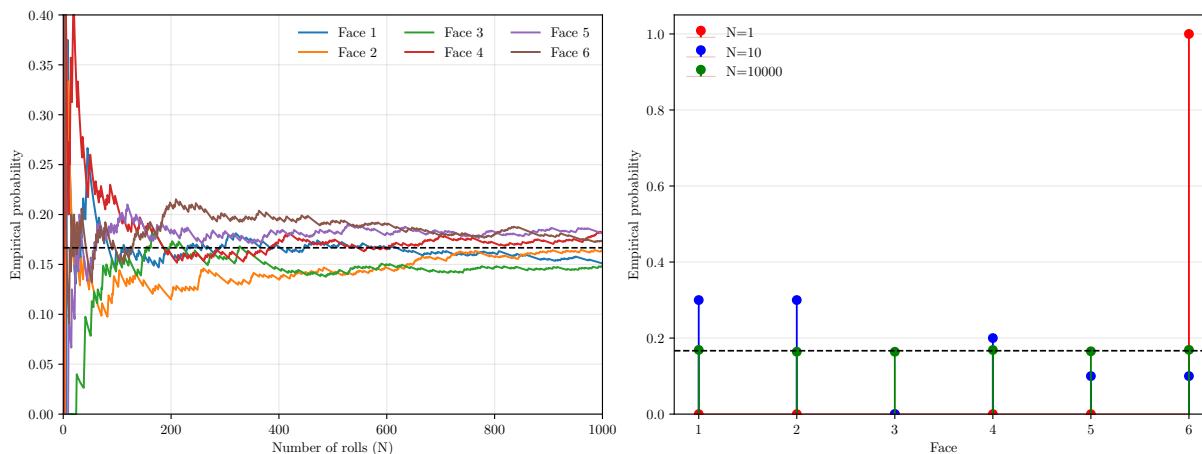


Figure 5.8.1: Convergence of empirical die-face probabilities (left) and overlaid distribution snapshots at $N = 1, 10, 10000$ (right).

this. The left panel traces the running empirical probability of each die face as the number of rolls N increases, with the dashed line at $1/6$ marking the theoretical value for this fair die. At small N , the empirical probabilities of different numbers fluctuate widely but the sampling variability steadily diminishes as N increases. The right panel overlays stem plots of the full discrete distribution at selected sample sizes ($N = 1, 10, 10000$). We can see how a single observation yields an extreme, noisy snapshot, a handful of rolls produces visible imbalance, and a large sample approaches the uniform distribution across faces. The same is true for any underlying and corresponding empirical distribution. If we draw only five samples from a normal distribution, the resulting histogram might look nothing like a bell curve, and any parameters we estimate from it (like the mean) could be very inaccurate. But if we keep adding more samples, we are guaranteed to get a better and better approximation of the true distribution.

5.9 The Central Limit Theorem

When using empirical samples to estimate an underlying probability distribution, one of the properties we might be interested in is the population mean. If we sample values $\{x_1, x_2, \dots, x_N\}$, a reasonable estimate for the mean of the underlying distribution is the sample mean, $\frac{1}{N} \sum_{i=1}^N x_i$. A natural question arises: how accurate is this estimate? How close is it likely to be to the true mean of the underlying distribution? The answer is provided by the **central limit theorem (CLT)**.

First, we have to recognize that the sample mean is a random variable itself. If we have a set of N independent and identically distributed (i.i.d.) random variables $\{x_1, \dots, x_N\}$, their sample mean, S_N , is also a random variable:

$$S_N = \frac{1}{N} \sum_{i=1}^N x_i \quad (5.9.1)$$

If we were to run an experiment once, we would calculate one sample mean, S_N . If we repeat it with another N samples, we might get a different sample mean. The CLT describes the distributional shape of this sample mean. For large N , the distribution of S_N converges to the normal distribution regardless of the parent distribution's shape (under mild conditions). The formal statement follows.

Central Limit Theorem. Let $\{x_i\}_{i=1}^N$ be i.i.d. with mean μ and finite, nonzero variance σ^2 . Then, as $N \rightarrow \infty$,

$$\sqrt{N}(S_N - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2), \quad (5.9.2)$$

where $S_N = \frac{1}{N} \sum_{i=1}^N x_i$ and $\xrightarrow{d}$ denotes convergence in distribution.

Equivalently, for large N , we have the useful normal approximation

$$S_N \approx \mathcal{N}\left(\mu, \frac{\sigma^2}{N}\right) \quad (\text{large } N) \quad (5.9.3)$$

and, in fact, when the observations are i.i.d. with variance σ^2 , the variance identity

$$\text{Var}(S_N) = \frac{\sigma^2}{N} \quad (5.9.4)$$

holds exactly (the normality is the approximation).

There are several important aspects of the CLT to highlight:

1. The distribution of sample means is centered at the true mean μ . Therefore, $\mathbb{E}[S_N] = \mu$. On average, the sample mean is correct.
2. The variance of the sample means is σ^2/N . This means that as our sample size N increases, the sample means cluster more and more tightly around the true mean. Our uncertainty in the estimate decreases proportionally to $1/\sqrt{N}$.
3. The CLT does not require the underlying distribution to be normal. Even if the original data are drawn from a highly skewed or heavy-tailed distribution, the distribution of the sample mean will still approach a normal distribution as N increases.

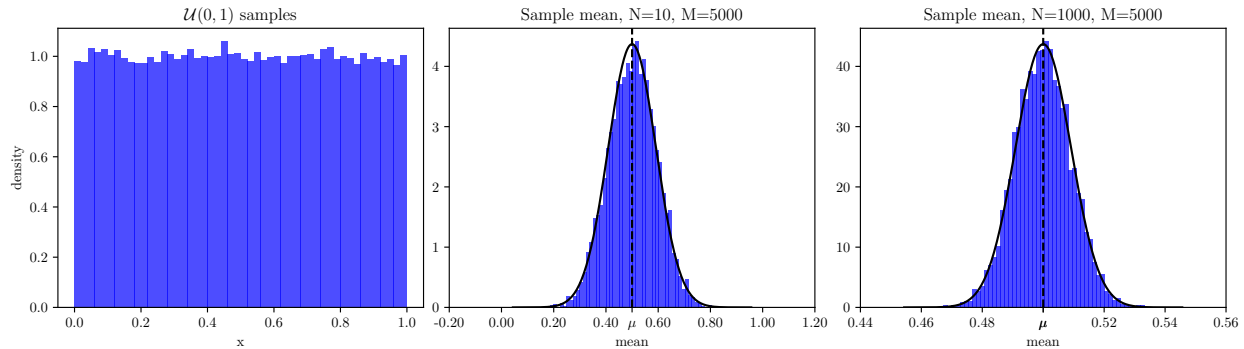


Figure 5.9.1: Central Limit Theorem demonstration. Left: histogram of raw $\mathcal{U}(0,1)$ draws. Middle/Right: histograms of the sample mean S_N from $M = 5000$ repeated experiments with $N = 10$ and $N = 1000$ per experiment. The solid curve is the normal approximation $\mathcal{N}(\mu, \sigma^2/N)$ with $\mu = 0.5$ and $\sigma^2 = 1/12$; the dashed line marks μ . As N increases, the distribution of S_N becomes more Gaussian and concentrates around μ with standard deviation $\sigma/\sqrt{N}$. Here, M is the number of independent repetitions used to build each histogram.

Warning: IID required. The CLT assumes the observations $\{x_i\}$ are independent and identically distributed with finite variance. If they are dependent or drawn from changing distributions, S_N may not be approximately $\mathcal{N}(\mu, \sigma^2/N)$ and its variance can be badly misestimated.

As an example of when CLT would not hold, suppose data arrive in identical pairs: $x_{2k-1} = x_{2k}$, each pair drawn i.i.d. from a distribution with variance σ^2 . Then with the sample mean $S_N = \frac{1}{N} \sum_{i=1}^N x_i$, we get

$$\text{Var}(S_N) = \frac{2\sigma^2}{N}, \quad (5.9.5)$$

the same as having only $N/2$ independent points. Treating them as i.i.d. would give confidence intervals that are too narrow by a factor of $\sqrt{2}$.

5.10 Estimators and Bias

The central limit theorem describes the behavior of a specific calculation, the sample mean. This is an example of an **estimator**: a rule for calculating a value from data to approximate an unknown property of the underlying distribution. We denote estimators using a hat by convention. For example, the sample mean $\hat{\mu}_x \equiv S_N$, which we used to estimate the true mean μ_x . But how do we know if an estimator is a good one? One way is to look at its **bias**. The bias of an estimator ($\hat{\theta}$) is the difference between its expected value and the true value of the parameter it is trying to estimate (θ):

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta \quad (5.10.1)$$

An estimator is unbiased if and only if its bias is zero, meaning $\mathbb{E}[\hat{\theta}] = \theta$.

The Sample Mean is Unbiased Assume we observe N i.i.d. random variables $\{x_1, \dots, x_N\}$ drawn from the same distribution p_x with mean $\mu = \mathbb{E}[x_i]$ and variance $\sigma^2 = \text{Var}(x_i)$:

$$x_1, \dots, x_N \stackrel{\text{i.i.d.}}{\sim} p_x \quad (5.10.2)$$

We can calculate the expectation of this estimator this easily using the linearity of expectation:

$$\mathbb{E}[\hat{\mu}_x] = \mathbb{E}\left[\frac{1}{N} \sum_{i=1}^N x_i\right] = \frac{1}{N} \sum_{i=1}^N \mathbb{E}[x_i] = \frac{1}{N} \sum_{i=1}^N \mu = \frac{N\mu}{N} = \mu \quad (5.10.3)$$

Since $\mathbb{E}[\hat{\mu}_x] = \mu$, the bias is zero.

Estimating Variance and Bessel's Correction Now, let's consider trying to estimate the true variance, σ^2 of the same distribution p_x from which $x_1, \dots, x_N$ are drawn as in [Eq. 5.10.2](#). A natural first guess might be to take the average of the squared differences from our estimated mean:

$$\hat{\sigma}_{\text{naive}}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu}_x)^2 \quad (5.10.4)$$

It turns out that this estimator is biased and will underestimate the true variance, σ^2 . To see this, we first rewrite deviations from the sample mean in terms of deviations from the population mean:

$$x_i - \hat{\mu}_x = (x_i - \mu) - (\hat{\mu}_x - \mu) \quad (5.10.5)$$

Next, we expand the square and take expectations (the expectation is over repeated random samples from p_x):

$$\begin{aligned} \mathbb{E}[\hat{\sigma}_{\text{naive}}^2] &= \mathbb{E}\left[\frac{1}{N} \sum_{i=1}^N ((x_i - \mu) - (\hat{\mu}_x - \mu))^2\right] \\ &= \mathbb{E}\left[\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2\right] - 2\mathbb{E}\left[\frac{1}{N} \sum_{i=1}^N (x_i - \mu)(\hat{\mu}_x - \mu)\right] + \mathbb{E}\left[\frac{1}{N} \sum_{i=1}^N (\hat{\mu}_x - \mu)^2\right] \end{aligned} \quad (5.10.6)$$

The first term evaluates to

$$\frac{1}{N} \sum_{i=1}^N \mathbb{E}[(x_i - \mu)^2] = \frac{1}{N} \sum_{i=1}^N \sigma^2 = \sigma^2 \quad (5.10.7)$$

But $\frac{1}{N} \sum_{i=1}^N (x_i - \mu) = \hat{\mu}_x - \mu$, so the second term becomes

$$-2 \mathbb{E}[(\hat{\mu}_x - \mu)^2] = -2 \text{Var}(\hat{\mu}_x) \quad (5.10.8)$$

For the third term, the summand does not depend on i , so

$$\mathbb{E}\left[\frac{1}{N} \sum_{i=1}^N (\hat{\mu}_x - \mu)^2\right] = \mathbb{E}[(\hat{\mu}_x - \mu)^2] = \text{Var}(\hat{\mu}_x) \quad (5.10.9)$$

Putting the pieces together, we get

$$\mathbb{E}[\hat{\sigma}_{\text{naive}}^2] = \sigma^2 - 2 \text{Var}(\hat{\mu}_x) + \text{Var}(\hat{\mu}_x) = \sigma^2 - \text{Var}(\hat{\mu}_x) \quad (5.10.10)$$

For i.i.d. data, $\text{Var}(\hat{\mu}_x) = \sigma^2/N$ according to the Central Limit Theorem, since $\hat{\mu}_x \equiv S_N$. Therefore,

$$\mathbb{E}[\hat{\sigma}_{\text{naive}}^2] = \left(1 - \frac{1}{N}\right) \sigma^2 = \frac{N-1}{N} \sigma^2 \quad (5.10.11)$$

which shows the bias: the naive estimator systematically underestimates the population variance σ^2 .

It's clear how to remove this bias: just multiply the naive estimator by $N/(N-1)$. We thus define the *unbiased* sample variance

$$\hat{\sigma}_{\text{unbiased}}^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \hat{\mu}_x)^2 \quad (5.10.12)$$

Then, we have

$$\mathbb{E}[\hat{\sigma}_{\text{unbiased}}^2] = \frac{N}{N-1} \mathbb{E}[\hat{\sigma}_{\text{naive}}^2] = \frac{N}{N-1} \left(\frac{N-1}{N} \sigma^2\right) = \sigma^2 \quad (5.10.13)$$

so $\hat{\sigma}_{\text{unbiased}}^2$ is an unbiased estimator of the population variance σ^2 . The intuition here is that the sample mean $\hat{\mu}_x$ is computed from the same sample and “uses up” one degree of freedom: once $N-1$ deviations $x_i - \hat{\mu}_x$ are fixed, the last one must sum to zero. Dividing by $N-1$ instead of N compensates for this constraint and exactly cancels the downward bias. This is known as Bessel's correction.

Warning: The Sample Standard Deviation is *Not* Unbiased. We just showed that $\hat{\sigma}_{\text{unbiased}}^2$ is an unbiased estimator of σ^2 . Does that mean $\sqrt{\hat{\sigma}_{\text{unbiased}}^2}$, the sample standard deviation, is an unbiased estimator of σ ? No.

Recall that $\mathbb{E}[g(x)] \neq g(\mathbb{E}[x])$ in general for nonlinear g —the expectation operator only “passes through” *linear* functions, and the square root is nonlinear (concave). By Jensen's inequality,

$$\mathbb{E}\left[\sqrt{\hat{\sigma}_{\text{unbiased}}^2}\right] \leq \sqrt{\mathbb{E}[\hat{\sigma}_{\text{unbiased}}^2]} = \sigma, \quad (5.10.14)$$

with equality only in degenerate cases (e.g., $\hat{\sigma}_{\text{unbiased}}^2$ constant). So the sample standard deviation is (slightly) biased low. More generally, unbiasedness of an estimator $\hat{\theta}$ for a parameter θ does *not* imply that $g(\hat{\theta})$ is unbiased for $g(\theta)$ when g is nonlinear.

Unbiased Variance Estimator if True Mean is Known What if we know the true mean, μ ? Do we still need Bessel's correction? Intuition tells us the answer is no, and the unbiased estimator of the true variance σ^2 should be

$$\tilde{\sigma}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2 \quad (5.10.15)$$

Let's check that this is indeed unbiased.

$$\mathbb{E}[\tilde{\sigma}^2] = \frac{1}{N} \sum_{i=1}^N \mathbb{E}[(x_i - \mu)^2] = \frac{1}{N} \sum_{i=1}^N \text{Var}(x_i) = \sigma^2 \quad (5.10.16)$$

Thus, the $N - 1$ (Bessel) correction is only needed when μ is unknown and replaced by the sample mean $\hat{\mu}_x$ computed from the same data.

Unbiased $\neq$ Best Mean-Squared Error Unbiasedness concerns the mean of an estimator, not its expected absolute or squared error. The “best” way to estimate a parameter is subjective, but we might consider the mean-squared error (MSE) as a reasonable criterion: if we use our estimator with a finite number of samples, what is its expected squared deviation from the true parameter? The mean-squared error (MSE) decomposes as

$$\text{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2] = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2 \quad (5.10.17)$$

A slightly biased estimator can have *smaller* MSE if it meaningfully reduces variance. For example, let $\hat{\theta}$ be unbiased with $\text{Var}(\hat{\theta}) = \alpha$ (so $\text{MSE}(\hat{\theta}) = \alpha$), and consider another estimator $\tilde{\theta}$ that just shrinks $\hat{\theta}$:

$$\tilde{\theta} = (1 - \lambda)\hat{\theta}, \quad 0 < \lambda < 1 \quad (5.10.18)$$

Then we have

$$\text{Bias}(\tilde{\theta}) = \mathbb{E}[(1 - \lambda)\hat{\theta}] - \theta = (1 - \lambda)\mathbb{E}[\hat{\theta}] - \theta = (1 - \lambda)\theta - \theta = -\lambda\theta \quad (5.10.19)$$

and

$$\text{MSE}(\tilde{\theta}) = (1 - \lambda)^2\alpha + \lambda^2\theta^2 \quad (5.10.20)$$

For sufficiently small $\lambda > 0$, we have $\text{MSE}(\tilde{\theta}) < \text{MSE}(\hat{\theta}) = \alpha$, so clearly unbiased $\neq$ minimum MSE.

5.11 Multivariate Random Variables

Our discussion so far has focused on single, or univariate, random variables. However, in many engineering and scientific problems, we are interested in multiple random quantities at once. For example, an experiment might measure the temperature and pressure in a reactor, or the velocity of a particle in three dimensions. To handle these cases, we generalize our discussion from “random variables” to “random vectors”.

5.11.1 Random Vectors

A **random vector** is simply a vector whose elements are random variables, and is equivalently described as a **multivariate random variable**. A random vector $\mathbf{x}$ of dimension N can be written as

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \quad (5.11.1)$$

For example, the velocity of a particle in 3D space is a random vector $\mathbf{v} = [v_x, v_y, v_z]^\top$. The behavior of a random vector is described by a **joint probability distribution**. The concepts of CDF and PDF extend directly from the scalar case: the **joint PDF**, $p_{\mathbf{x}}(\mathbf{x})$, is a function over $\mathbb{R}^N$ that we integrate over a region to find the probability that the vector will fall in that region. Similarly, the **joint CDF**, $F_{\mathbf{x}}(\mathbf{x})$, is the probability that the vector will fall in a region below $\mathbf{x}$:

$$F_{\mathbf{x}}(\mathbf{x}) = \mathbb{P}(x_1 \leq x_1 \cap x_2 \leq x_2 \cap \cdots \cap x_N \leq x_N) \quad (5.11.2)$$

If $\mathbf{x}$ has a joint PDF $p_{\mathbf{x}}(\mathbf{x})$, then for any $\mathbf{x} = (x_1, \dots, x_N)$,

$$F_{\mathbf{x}}(\mathbf{x}) = \int_{-\infty}^{x_1} \cdots \int_{-\infty}^{x_N} p_{\mathbf{x}}(u_1, \dots, u_N) du_1 \cdots du_N \quad (5.11.3)$$

with $p_{\mathbf{x}}(\mathbf{x}) \geq 0$ and $\int_{\mathbb{R}^N} p_{\mathbf{x}}(\mathbf{x}) d\mathbf{x} = 1$ For any region $\Omega \subseteq \mathbb{R}^N$,

$$\mathbb{P}(\mathbf{x} \in \Omega) = \int_{\Omega} p_{\mathbf{x}}(\mathbf{x}) d\mathbf{x} \quad (5.11.4)$$

The components $x_1, \dots, x_N$ are **independent** if and only if their joint PDF is the product of their individual (marginal) PDFs:

$$p_{\mathbf{x}}(\mathbf{x}) = \prod_{i=1}^N p_{x_i}(x_i) \iff x_1, \dots, x_N \text{ are independent} \quad (5.11.5)$$

The mean of a random vector is a vector containing the means of each of its components:

$$\boldsymbol{\mu}_{\mathbf{x}} = \mathbb{E}[\mathbf{x}] = \langle \mathbf{x} \rangle = \begin{bmatrix} \mathbb{E}[x_1] \\ \mathbb{E}[x_2] \\ \vdots \\ \mathbb{E}[x_N] \end{bmatrix} \quad (5.11.6)$$

And more generally, the expectation of any function $f : \mathbb{R}^N \rightarrow \mathbb{R}$ of a random vector $\mathbf{x}$ is given by

$$\mathbb{E}[f(\mathbf{x})] = \int_{\mathbb{R}^N} f(\mathbf{x}) p_{\mathbf{x}}(\mathbf{x}) d\mathbf{x} \quad (5.11.7)$$

Example (Bivariate PDF as a Product of Marginals): We take two *independent* scalar random variables:

$$x \sim \mathcal{N}(0, 1), \quad y \sim \mathcal{N}(0, 4) \quad (5.11.8)$$

with independence meaning $p_{x,y}(x, y) = p_x(x) p_y(y)$. The marginal PDFs are

$$p_x(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right), \quad p_y(y) = \frac{1}{\sqrt{2\pi} \cdot 2} \exp\left(-\frac{y^2}{8}\right) \quad (5.11.9)$$

So the joint PDF is the simple product

$$p_{x,y}(x, y) = p_x(x) p_y(y) = \frac{1}{2\pi \cdot 2} \exp\left(-\frac{x^2}{2} - \frac{y^2}{8}\right) \quad (5.11.10)$$

The density is shown in [Figure 5.11.1](#).

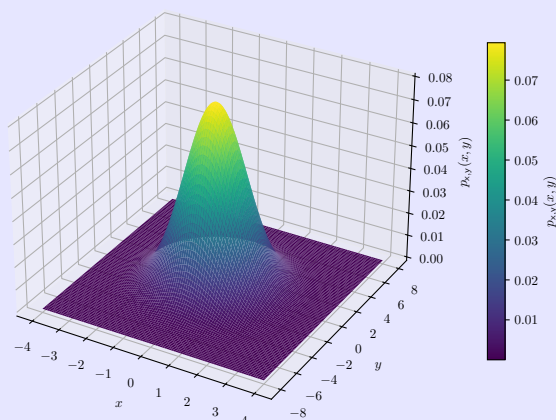


Figure 5.11.1: Bivariate Gaussian density with independent components.

5.12 Covariance and Correlation

While the mean vector describes the central tendency of each component, it tells us nothing about the relationships *between* the components. Do they tend to increase together? Or does one tend to decrease when the other increases? This is measured by the **covariance**.

The covariance between two scalar random variables x_i and x_j is the expected product of their deviations from their respective means μ_i and μ_j :

$$\text{Cov}(x_i, x_j) = \mathbb{E}[(x_i - \mu_i)(x_j - \mu_j)] = \mathbb{E}[x_i x_j] - \mu_i \mu_j \quad (5.12.1)$$

The sign of the covariance tells us the nature of the linear relationship:

- $\text{Cov}(x_i, x_j) > 0$: x_i and x_j tend to be on the same side of their means. They have a positive linear relationship.
- $\text{Cov}(x_i, x_j) < 0$: x_i and x_j tend to be on opposite sides of their means. They have a negative linear relationship.
- $\text{Cov}(x_i, x_j) = 0$: There is no linear relationship between them. They are said to be uncorrelated.

For a random vector $\mathbf{x}$, we summarize all of these pairwise relationships in the **covariance matrix** $\mathbf{C}$, also commonly denoted by $\mathbf{\Sigma}$. The element C_{ij} is the covariance between x_i and x_j .

$$\mathbf{C} = \begin{bmatrix} \text{Var}(x_1) & \text{Cov}(x_1, x_2) & \cdots & \text{Cov}(x_1, x_N) \\ \text{Cov}(x_2, x_1) & \text{Var}(x_2) & \cdots & \text{Cov}(x_2, x_N) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(x_N, x_1) & \text{Cov}(x_N, x_2) & \cdots & \text{Var}(x_N) \end{bmatrix} \equiv \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T] \quad (5.12.2)$$

The diagonal entries of the covariance matrix are the variances of each component, $C_{ii} = \text{Cov}(x_i, x_i) = \text{Var}(x_i)$, and the off-diagonal entries describe how the components co-vary. If the off-diagonal entries are all zero, the covariance matrix is diagonal, and the components are said to be pairwise uncorrelated.

Note also that the covariance matrix is always symmetric ($C_{ij} = C_{ji}$) and positive semi-definite. The fact that a covariance matrix must be positive semi-definite won't be used directly in the remainder of our discussion of probability, but connects back to our earlier discussions of linear algebra, so we will write it out here as a

simple proof. Consider any non-zero vector $\mathbf{z} \in \mathbb{R}^N$. We can express the quadratic form $\mathbf{z}^\top \mathbf{C} \mathbf{z}$ as follows:

$$\mathbf{z}^\top \mathbf{C} \mathbf{z} = \mathbf{z}^\top \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top] \mathbf{z} \quad (5.12.3)$$

$$= \mathbb{E}[\mathbf{z}^\top (\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{z}] \quad (5.12.4)$$

$$= \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{z}]^\top (\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{z}] \quad (5.12.5)$$

$$= \mathbb{E}[\mathbf{v}^\top \mathbf{v}] \quad (5.12.6)$$

where we define the random variable $\mathbf{v} = (\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{z}$. Since the inner product of a vector with itself is strictly non-negative, this expectation value is also non-negative. Therefore, we have $\mathbf{z}^\top \mathbf{C} \mathbf{z} \geq 0$ for all non-zero vectors $\mathbf{z}$, which confirms that the covariance matrix $\mathbf{C}$ is positive semi-definite.

Example Covariance and Scatter Patterns for Multivariate Gaussian Random Variables:

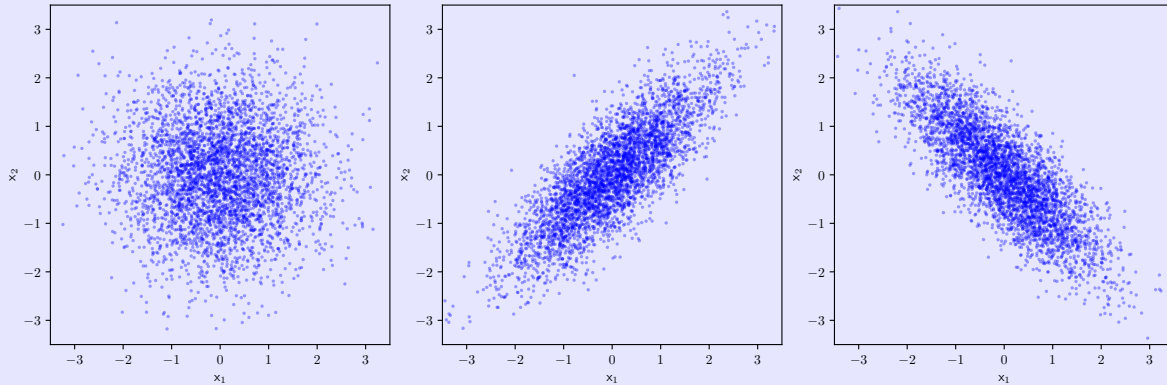
Consider three zero-mean bivariate normal random vectors $\mathbf{x}^{(k)} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}^{(k)})$ with covariance matrices

$$\boldsymbol{\Sigma}^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \boldsymbol{\Sigma}^{(2)} = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}, \quad \boldsymbol{\Sigma}^{(3)} = \begin{bmatrix} 1 & -0.8 \\ -0.8 & 1 \end{bmatrix}.$$

A major point here is that for the cases with off-diagonal entries, we do *not* have the product of the marginal PDFs as the joint PDF. Instead, the joint PDF has the following quadratic-exponential form, which is the general expression for a k -dimensional Gaussian random variable with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$:

$$p_{\mathbf{x}}(\mathbf{x}) = \frac{1}{(2\pi)^{k/2} \sqrt{\det \boldsymbol{\Sigma}}} \exp\left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})\right) \quad (5.12.7)$$

Note a gap between this formula and what we proved above: the earlier proof only established that $\boldsymbol{\Sigma}$ is positive *semi*-definite, whereas this PDF requires $\boldsymbol{\Sigma}$ to be invertible (positive *definite*), since it appears in the denominator and is inverted in the exponent. We draw i.i.d. samples from each $p_{\mathbf{x}}(\mathbf{x})$ and plot them:



Interpretation. $\boldsymbol{\Sigma}^{(1)}$ is diagonal (uncorrelated); the cloud is circular/axis-aligned. $\boldsymbol{\Sigma}^{(2)}$ has positive off-diagonals; the cloud tilts upward (positive linear relationship). $\boldsymbol{\Sigma}^{(3)}$ has negative off-diagonals; the cloud tilts downward (negative linear relationship).

Independence vs. Uncorrelatedness The relationship between independence and correlation is a common point of confusion. The rule is simple: independence implies uncorrelatedness, but uncorrelatedness does not imply independence. Pondering upon this a bit, it is clear why. If two random variables are independent, their covariance is zero by definition, so they must be uncorrelated also by definition. However, if all we know is that the covariance is zero, this only means that there is no *linear* relationship between the two. A strong nonlinear relationship can still exist, as shown in [Figure 5.12.1](#).²

²We note that for the special case of jointly Gaussian variables, zero covariance *does* imply independence.

Example (Uncorrelated but Dependent). Let $x_1 \sim \mathcal{N}(0, 1)$ and let ε be independent noise with $\mathbb{E}[\varepsilon] = 0$. Define

$$x_2 = x_1^2 + \varepsilon \quad (5.12.8)$$

Then $\mathbb{E}[x_1] = 0$ and $\mathbb{E}[x_1^3] = 0$ (symmetry), so

$$\text{Cov}(x_1, x_2) = \text{Cov}(x_1, x_1^2) + \text{Cov}(x_1, \varepsilon) = \mathbb{E}[x_1^3] - \mathbb{E}[x_1]\mathbb{E}[x_1^2] + 0 = 0 \quad (5.12.9)$$

Thus the variables are *uncorrelated*. However, x_2 is (almost) a deterministic function of x_1 , so they are clearly *dependent*. The scatter below shows the strong nonlinear relation without any linear trend:

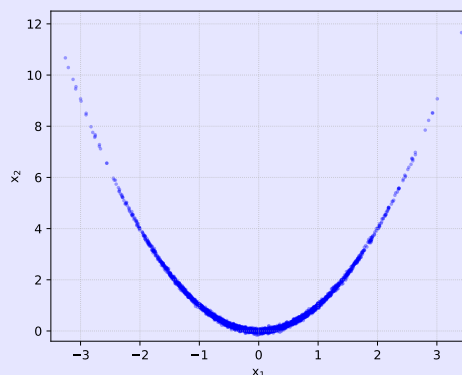
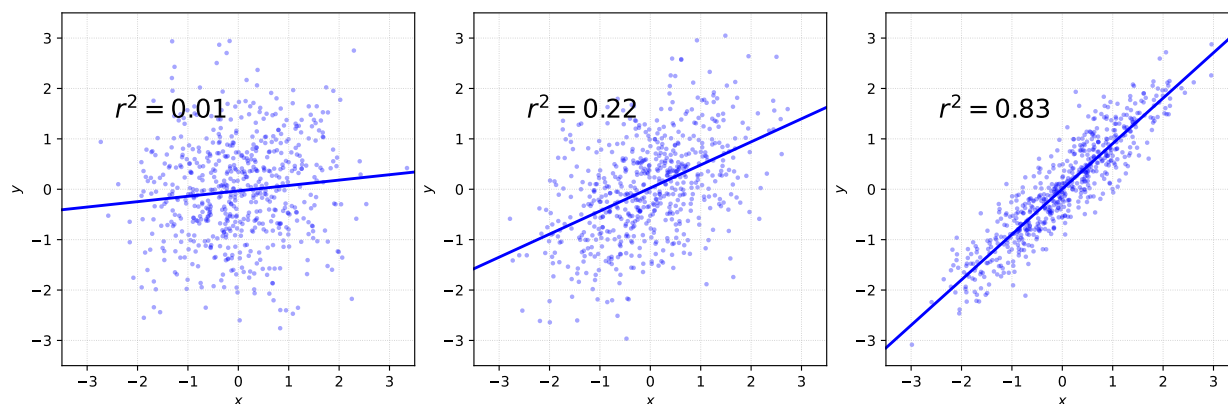


Figure 5.12.1: Uncorrelated but dependent random variables.

Finally, since the magnitude of covariance depends on the units of the variables, it is often normalized into a dimensionless quantity called the Pearson correlation coefficient, r . The square of this value, the **coefficient of determination** r^2 , measures the proportion of the variance in one variable that is predictable from the other variable:

$$r^2(x, y) = \left(\frac{\text{Cov}(x, y)}{\sigma_x \sigma_y} \right)^2 = \frac{(\mathbb{E}[xy] - \mathbb{E}[x]\mathbb{E}[y])^2}{(\mathbb{E}[x^2] - \mathbb{E}[x]^2)(\mathbb{E}[y^2] - \mathbb{E}[y]^2)} \quad (5.12.10)$$

To visualize this, we simulate three bivariate normals, plot the scatter, and overlay the least-squares line (which we will define more formally in the next chapter).



5.12.1 Conditional Probability

We have seen how a joint PDF $p_{\mathbf{x}}(\mathbf{x})$ describes the behavior of a random vector. But how does our knowledge about one component of the vector change once we observe the value of another? This is the domain of **conditional probability distributions**. The mathematical relationships here will be directly analogous

to what we saw for discrete random variables at the beginning of the chapter. The conditional PDF of x_2 given that we have observed $x_1 = x_1$ is defined as the joint PDF divided by the marginal PDF of the known variable:

$$p_{x_2|x_1}(x_2 | x_1) = \frac{p_{x_1, x_2}(x_1, x_2)}{p_{x_1}(x_1)} \quad \text{provided } p_{x_1}(x_1) > 0 \quad (5.12.11)$$

Intuitively, you can think of the joint PDF in the 2D case as a mountain range over the (x_1, x_2) plane. Observing that $x_1 = x_1$ is like taking a vertical slice of that mountain at a specific x_1 value. The curve you see on that slice is the shape of the conditional distribution. We divide by $p_{x_1}(x_1)$ to re-normalize the area under this curve to 1, making it a valid PDF.

By rearranging this definition, we get the **chain rule of probability** for random variables, which allows us to build a joint distribution from conditional and marginal parts:

$$p_{x_1, x_2}(x_1, x_2) = p_{x_2|x_1}(x_2 | x_1) p_{x_1}(x_1) \quad (5.12.12)$$

Example: Consider a 2D random vector where $x_1 \sim \mathcal{N}(0, 0.25)$ and the conditional distribution of x_2 is given by $x_2 | x_1 \sim \mathcal{N}(-2x_1, 1)$. How does the distribution of x_2 change for different observations of x_1 ?

- If we observe $x_1 = 0$, the distribution of x_2 is $\mathcal{N}(-2(0), 1) = \mathcal{N}(0, 1)$.
- If we observe $x_1 = 1$, the distribution of x_2 becomes $\mathcal{N}(-2(1), 1) = \mathcal{N}(-2, 1)$.

Our knowledge of x_1 directly influences our expectation of x_2 . This is visualized below, where the conditional distribution is a vertical slice of the joint distribution.

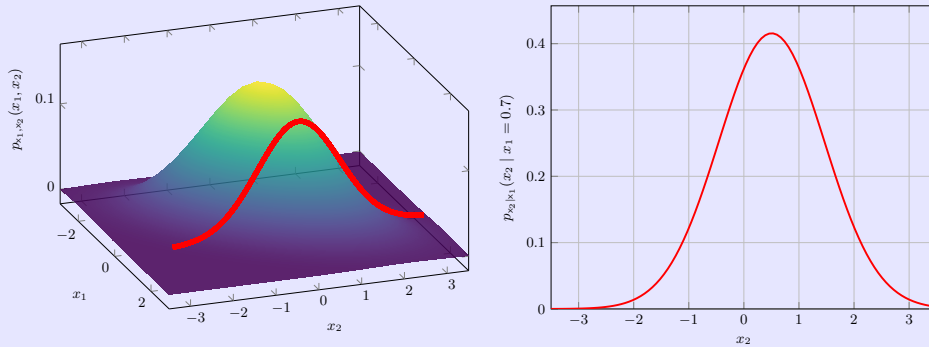


Figure 5.12.2: A visualization of conditional probability. The colored surface illustrates the joint PDF of two random variables. Fixing a value of x_1 (e.g., $x_1 = 0$ or $x_1 = 1$) is like taking a single slice. The distribution of the points within that slice is the conditional distribution of x_2 given x_1 . The visualization above shows the PDF of x_2 given $x_1 = 0.7$.

Conditional Independence A related concept is conditional independence. Two random variables x_1 and x_2 are conditionally independent given a third variable x_3 if, once we know x_3 , knowing x_1 gives us no additional information about x_2 . Mathematically:

$$p(x_1, x_2 | x_3) = p(x_1 | x_3) p(x_2 | x_3) \quad (5.12.13)$$

For example, two noisy temperature sensors placed next to each other are not independent; if one reads high, the other likely will too. But they are *conditionally independent given the true temperature*. Once we know the true temperature, the noise in one sensor reading is independent of the noise in the other. This assumption that measurement noise from different sensors is conditionally independent given the true underlying state is the key modeling assumption behind multi-sensor data fusion techniques such as the Kalman filter, widely used in process control.

5.12.2 Marginalization

We might have a high-dimensional joint distribution that describes a large set of random variables but be particularly interested in the distribution of a single component. How do we get the PDF of x_1 alone from a joint PDF? The process is called **marginalization**. To find the marginal distribution of one variable, we “integrate out” the uncertainty in all other variables. This is the continuous version of the law of total probability.

$$p_{x_1}(x_1) = \int_{-\infty}^{\infty} p_{x_1, x_2}(x_1, x_2) dx_2 \quad (5.12.14)$$

We can substitute the chain rule into this expression to see the connection to conditional probabilities; this is simply expanding the integrand of [Eq. 5.12.14](#):

$$p_{x_1}(x_1) = \int_{-\infty}^{\infty} p_{x_1|x_2}(x_1 | x_2) p_{x_2}(x_2) dx_2 \quad (5.12.15)$$

Visually, marginalization is like taking the 3D mountain of the joint PDF and projecting or collapsing its mass onto one of the axes. The resulting shadow on that axis is the marginal PDF. An example of marginalizing a 2D empirical distribution—essentially, projecting a 2D scatterplot onto two 1D histograms—is shown in [Figure 5.12.3](#).

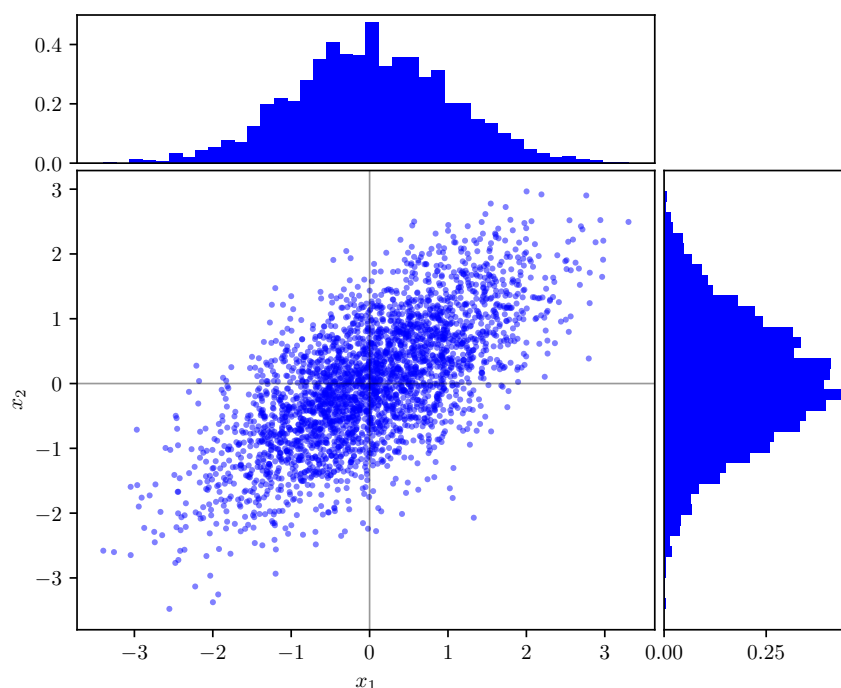


Figure 5.12.3: A scatter plot with marginal distributions shown as histograms. The central scatter plot shows samples from the joint distribution $p_{x_1, x_2}(x_1, x_2)$. The histograms on the margins (bottom and left) show the marginal distributions $p_{x_1}(x_1)$ and $p_{x_2}(x_2)$, obtained by projecting the joint data onto each axis.

Example (Marginalizing a Joint PDF). Suppose we build a 2D random vector by using the following:

$$x_1 \sim \mathcal{N}(0, 0.25), \quad x_2 | x_1 = x_1 \sim \mathcal{N}(-2x_1, 1) \quad (5.12.16)$$

What is the marginal PDF of x_2 , $p_{x_2}(x_2)$?

First, note that in general, the joint PDF can be written using the chain rule as

$$p_{x_1, x_2}(x_1, x_2) = p_{x_2|x_1}(x_2 | x_1) p_{x_1}(x_1) \quad (5.12.17)$$

The standard way of finding the marginal distribution is to integrate out the unneeded variable. We can integrate over x_1 to find $p_{x_2}(x_2)$:

$$p_{x_2}(x_2) = \int_{-\infty}^{\infty} p_{x_1, x_2}(x_1, x_2) dx_1 \quad (5.12.18)$$

$$= \int_{-\infty}^{\infty} p_{x_2|x_1}(x_2 | x_1) p_{x_1}(x_1) dx_1 \quad (5.12.19)$$

$$= \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(x_2 + 2x_1)^2}{2}\right) \frac{1}{\sqrt{2\pi \cdot 0.25}} \exp\left(-\frac{x_1^2}{2 \cdot 0.25}\right) dx_1 \quad (5.12.20)$$

The result of this integral can be found by completing the square or using a standard identity of Gaussian integrals to yield:

$$p_{x_2}(x_2) = \frac{1}{\sqrt{4\pi}} \exp\left(-\frac{x_2^2}{4}\right) = \mathcal{N}(0, 2) \quad (5.12.21)$$

We can also use a shortcut by taking advantage of the fact that these are Gaussian random variables. We can equivalently write $x_2 = -2x_1 + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$ independent of x_1 . Because linear combinations of Gaussian random variables are also Gaussian, we can compute the mean and variance of x_2 directly:

$$x_2 \sim \mathcal{N}(-2 \cdot 0, (-2)^2 \cdot 0.25 + 1) = \mathcal{N}(0, 2) \quad (5.12.22)$$

Note that this only works because we are dealing with Gaussian random variables; in general, we must perform the integral.

5.13 A Unified View of Probability Distributions

This chapter has introduced several probability distributions as separate entities, but it is important to recognize that they exist within a deeply interconnected web. Indeed, many distributions can be seen as special cases, generalizations, or limiting transformations of others. A classic example of this is the relationship between the Bernoulli, binomial, and normal distributions.

Bernoulli to Binomial: A single coin flip, with a binary outcome (e.g., success or failure), is a Bernoulli trial. The probability of success, p , is the only parameter. If you perform n independent and identical Bernoulli trials and count the total number of successes, this count follows a binomial distribution. Thus, a binomial random variable is simply the sum of n independent Bernoulli random variables that share a single parameter p .

Binomial to Normal: The central limit theorem states that the sum of a large number of independent and identically distributed random variables will be approximately normally distributed. As a result, when the number of trials n in the binomial distribution becomes large, the shape of the discrete binomial distribution becomes so close to the continuous normal distribution that the normal can be used as an excellent approximation for the binomial.

These connections are illustrative, not exhaustive. An even simpler example is connecting a normal distribution to a standard normal distribution via the Z-score transformation; e.g., if $x \sim \mathcal{N}(\mu, \sigma^2)$, then the standardized variable $z = \frac{x - \mu}{\sigma} \sim \mathcal{N}(0, 1)$. Many further bridges exist: the rare-event limit links binomial to Poisson; Poisson processes connect counts to exponential and gamma waiting times; the normal, χ^2 , t , and F families arise from sums and ratios; conjugate mixtures (e.g., beta-binomial, Dirichlet-multinomial) tie

priors to likelihoods; monotonic transformations give rise to new laws (e.g., lognormal from a normal); and multivariate generalizations introduce structures such as the Wishart distribution (a generalization of the gamma distribution). A fun resource for a comprehensive overview of the relationships between univariate distributions is [this beautiful interactive chart](#) produced by Prof. Larry Leemis at William & Mary.

5.13.1 Convolution

The relationship we just discussed, where a binomial random variable is the sum of independent Bernoulli random variables, is a specific case of a more general concept called convolution. Convolution is the mathematical operation that gives the probability distribution of a sum of two independent random variables.

Suppose we have two independent random variables, x and y . We are interested in the distribution of their sum, $s = x + y$. To find the probability of a specific total outcome s , we must consider every possible pair of outcomes, one from x and one from y , that sum to s . Because x and y are independent, the probability of each such pair is the product of their individual probabilities, i.e., $p_{x,y}(x, y) = p_x(x) p_y(y)$. We then add up the probabilities of all these possible pairs to get the total probability for s . This process of pairing up, multiplying, and summing is precisely what convolution represents.

The Discrete Case Let x and y be independent, integer-valued random variables with PMFs p_x and p_y , respectively. The PMF of their sum, $s = x + y$, is given by the discrete convolution of p_x and p_y , denoted $(p_x * p_y)(s)$:

$$p_s(s) = (p_x * p_y)(s) = \sum_x p_x(x) p_y(s - x) \quad (5.13.1)$$

The sum is over all possible values x in the support of x . For each x , the corresponding value from y that would result in the sum s is $s - x$.

Example: Deriving the Discrete Convolution Formula

Our goal is to find the probability that the sum $s = x + y$ equals some specific value s . The event “the sum is s ” can happen in many mutually exclusive ways. For every possible value x that the random variable x can take, the event occurs if y takes the value $s - x$. We can write the total event as a union of these disjoint sub-events:

$$\{s = s\} = \bigcup_x \{x = x \text{ and } y = s - x\} \quad (5.13.2)$$

For example, if we’re rolling two dice and want a total of 4, the event is $\{\text{die 1} = 1, \text{die 2} = 3\} \cup \{\text{die 1} = 2, \text{die 2} = 2\} \cup \{\text{die 1} = 3, \text{die 2} = 1\}$. Because these ways are mutually exclusive (x cannot be both 1 and 2 at the same time), the probability of the total event is the sum of the probabilities of the sub-events:

$$\Pr(s = s) = \sum_x \Pr(x = x \text{ and } y = s - x) \quad (5.13.3)$$

Now, we use our key assumption that x and y are independent. The probability of two independent events both occurring is the product of their individual probabilities:

$$\Pr(x = x \text{ and } y = s - x) = \Pr(x = x) \cdot \Pr(y = s - x) \quad (5.13.4)$$

Substituting this product back into the sum gives us the discrete convolution formula:

$$\begin{aligned} p_s(s) = \Pr(s = s) &= \sum_x \Pr(x = x) \cdot \Pr(y = s - x) \\ &= \sum_x p_x(x) \cdot p_y(s - x) \end{aligned} \quad (5.13.5)$$

Therefore, convolution is simply the mathematical embodiment of considering all possible ways an outcome can be formed ($s = x + (s - x)$), calculating the probability of each way by multiplying (due to independence), and summing them all up.

Example: Convolution of Two Geometric Random Variables

Let x and y be independent geometric random variables with parameter θ . The PMF of a geometric random variable is given by:

$$p_x(k) = \theta(1 - \theta)^k, \quad k = 0, 1, 2, \dots \quad (5.13.6)$$

We want to find the PMF of their sum $s = x + y$. Using the discrete convolution formula, we have:

$$p_s(s) = \sum_{k=0}^s p_x(k) \cdot p_y(s - k) \quad (5.13.7)$$

$$= \sum_{k=0}^s \theta(1 - \theta)^k \cdot \theta(1 - \theta)^{s-k} \quad (5.13.8)$$

$$= \theta^2(1 - \theta)^s \sum_{k=0}^s 1 \quad (5.13.9)$$

$$= \theta^2(1 - \theta)^s (s + 1) \quad (5.13.10)$$

Thus, the PMF of the sum $s = x + y$ is:

$$p_s(s) = \theta^2(s + 1)(1 - \theta)^s, \quad s = 0, 1, 2, \dots \quad (5.13.11)$$

This distribution turns out to have a special name: it is a negative binomial distribution with parameters $r = 2$ and $p = \theta$.

The Continuous Case For independent continuous random variables x and y with PDFs p_x and p_y , the PDF of their sum $s = x + y$ is given by the continuous convolution, which replaces the sum with an integral:

$$p_s(s) = (p_x * p_y)(s) = \int_{-\infty}^{\infty} p_x(x) \cdot p_y(s - x) dx \quad (5.13.12)$$

The derivation of the continuous convolution formula is similar to the discrete case shown in the previous example but with a little bit different mechanics to handle continuity appropriately.

5.14 Random Number Generation

So far, this chapter has laid the theoretical groundwork of probability and introduced us to some useful mathematical formalisms and notation. Now, we turn to the practical question of how these concepts are used in a computational setting. All of the scatter plots, histograms, and convergence demonstrations in this chapter have relied on our ability to generate data that behaves according to a specific probability distribution. We will discuss more about methods for sampling from arbitrary distributions later, but for now, we will focus on one of the most fundamental concepts in this area that other methods will rely on: random number generation.

5.14.1 Pseudorandom Numbers and Seeding

A computer is a deterministic machine, so it is incapable of generating truly random numbers. Instead, it uses algorithms called pseudorandom number generators (PRNGs) to create long sequences of numbers that are—for all practical purposes—indistinguishable from random. These algorithms start with an initial value called a **seed**. Given the same seed, a PRNG will always produce the exact same sequence of numbers.

A very simple (and not very good!) example of a PRNG is the middle squares method. This method starts with a seed number, squares it, pads it to 8 digits with leading zeros, and then extracts the middle digits to form the next number in the sequence. This process is repeated to generate more numbers. For example,

generating four-digit integers seeded with “1034”, we might have:

$$\begin{aligned} 1034 &\rightarrow 1034^2 = 01,069,156 \rightarrow \text{next number: } 0691 \\ 0691 &\rightarrow 0691^2 = 00,477,481 \rightarrow \text{next number: } 4774 \\ 4774 &\rightarrow \dots \end{aligned}$$

While this method can produce a sequence that appears random, it has several weaknesses, such as short periods and poor distribution properties.

This determinism is a feature, not a bug. By setting the seed manually at the beginning of a simulation, we can make sure that our results are perfectly reproducible. This is helpful for debugging code and for allowing other scientists to verify our work. Most programming environments provide a simple command for this (e.g., `rng(seed)` in MATLAB or `np.random.seed(seed)` in Python’s NumPy library).

Minecraft’s Worlds The video game Minecraft is a large-scale example of using PRNGs and seeding as a core feature. The game procedurally generates enormous, unique worlds from a single starting value (the seed) so that players can share a short string and explore the same large-scale terrain and biomes. (This reproducibility holds for the same edition, version, and settings.) Under the hood, many parts of world generation use a seeded PRNG. In Java Edition, this involves `java.util.Random`, whose state follows a linear congruential generator, which is a specific type of PRNG. The deterministic outputs feed Perlin-style layered gradient noise to build smooth fields and heightmaps that shape mountains, valleys, and cave systems. Moreover, the game instantiates multiple generators with seeds derived from the world seed for specific tasks such as caves, ores, trees, keeping everything reproducible from that initial seed. The result is a (very) complex, seemingly random world that is actually perfectly reproducible from the initial seed.

Note that generating good random numbers is computationally expensive. When writing computer programs whose main cost involves calls to a random number generator, which occurs in Monte Carlo methods described later, one goal is to design the algorithm to avoid performing any unnecessary calls to the random number generator.

5.14.2 Sampling from Distributions

At the heart of most PRNGs is a generator that produces samples from the standard uniform distribution, $\mathcal{U}(0, 1)$. We can then use these uniform samples to generate samples from any other distribution using various transformation methods.

The simplest transformation is for the uniform distribution itself. If we want samples from a uniform distribution over an arbitrary interval $[a, b]$, we can take a sample $u \sim \mathcal{U}(0, 1)$ and transform it as follows:

$$x = a + (b - a) \cdot u \implies x \sim \mathcal{U}(a, b) \quad (5.14.1)$$

A general and powerful technique is **inverse transform sampling**. We will only cover the discrete case here and mention the continuous case later: For a discrete distribution with probabilities $\{P_i\}$ for outcomes $\{x_i\}$, we can partition the interval $[0, 1]$ into segments of length P_i . We then draw a sample $u \sim \mathcal{U}(0, 1)$ and see which segment it falls into. This is illustrated in [Figure 5.14.1](#).



Figure 5.14.1: Sampling from a discrete distribution using a uniform random number. The interval $[0, 1]$ is split into segments of lengths P_i ; the outcome is determined by where $u \sim \mathcal{U}(0, 1)$ lands.

Modern programming environments have built-in functions to sample directly from dozens of common distributions (e.g., `normrnd`, `poissrnd`, etc.), which should almost always be used in practice.

5.14.3 Distances Between Distributions

In many applications, especially in statistics and machine learning, we need to quantify how different two probability distributions, $p_x(x)$ and $q_x(x)$, are.³ This turns a modeling question into a geometric one: how far is our model's predictive law q_x from the data-generating law p_x ? A first note is that some notions are true metrics (in a formal mathematical sense: non-negative, symmetric, and obeying the triangle inequality) while others are asymmetric divergences that act more like directional penalties. Choosing between different distance metrics can be subjective or application-dependent.

Kullback-Leibler Divergence The Kullback-Leibler (KL) divergence is the canonical asymmetric example and is the most popular choice in machine learning to quantify differences in distributions. In the continuous case, it is defined as

$$D_{\text{KL}}(p_x \| q_x) = \mathbb{E}_{x \sim p_x(x)} \left[\log \frac{p_x(x)}{q_x(x)} \right] = \int p_x(x) \log \left(\frac{p_x(x)}{q_x(x)} \right) dx \quad (\text{continuous}) \quad (5.14.2)$$

$$= \sum_x p_x(x) \log \left(\frac{p_x(x)}{q_x(x)} \right) \quad (\text{discrete}) \quad (5.14.3)$$

The KL divergence measures the expected extra log-loss incurred when coding samples from p_x with a code optimized for q_x . Because the expectation is taken under p_x , regions where p_x assigns non-zero probability density but q_x is small are penalized heavily; if $q_x(x) = 0$ for any values of x where $p_x(x) > 0$, the divergence is infinite. The reverse divergence, $D_{\text{KL}}(q_x \| p_x)$, flips this emphasis and tends to be “mode-seeking,” preferring to place q_x 's mass on a subset of p_x 's high-density regions rather than cover all of p_x . The convention $0 \log(0/q) = 0$ is used to handle zero probabilities gracefully.

Example: Bernoulli and Gaussian KL Divergence

Bernoulli case: For Bernoulli distributions with success probabilities θ_p and θ_q ,

$$D_{\text{KL}}(\text{Bernoulli}(\theta_p) \| \text{Bernoulli}(\theta_q)) = \theta_p \log \left(\frac{\theta_p}{\theta_q} \right) + (1 - \theta_p) \log \left(\frac{1 - \theta_p}{1 - \theta_q} \right) \quad (5.14.4)$$

Note that swapping p_x and q_x gives a generally different number like we expect since KL divergence is asymmetric.

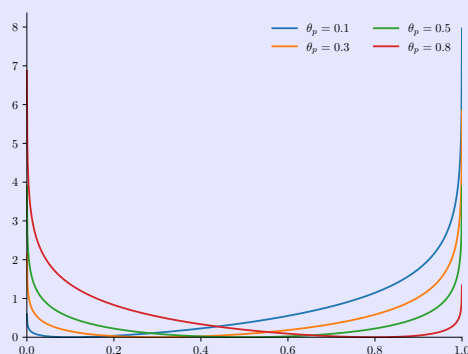


Figure 5.14.2: For several fixed θ_p , the curve shows $D_{\text{KL}}(\text{Bernoulli}(\theta_p) \| \text{Bernoulli}(\theta_q))$ as θ_q varies. The divergence is minimized at $\theta_q = \theta_p$ and rises sharply as $\theta_q \rightarrow 0$ (if $\theta_p \neq 0$) or $\theta_q \rightarrow 1$ (if $\theta_p \neq 1$) since there is an enormous penalty for assigning near-zero probability to outcomes that p deems likely. Different θ_p produce different profiles since KL is directional and depends on which distribution is treated as p .

³For example, generative modeling objectives (MLE, VAEs, GANs, WGANs, diffusion) are built around specific divergences/distances (KL, JS, Wasserstein, Fisher).

Gaussian case: In Gaussian families, we also have a closed form for the KL divergence. If $p_x = \mathcal{N}(\mu_p, \sigma_p^2)$ and $q_x = \mathcal{N}(\mu_q, \sigma_q^2)$,

$$D_{\text{KL}}(p_x \| q_x) = \frac{1}{2} \left(\frac{\sigma_p^2}{\sigma_q^2} + \frac{(\mu_q - \mu_p)^2}{\sigma_q^2} - 1 + \log \left(\frac{\sigma_q^2}{\sigma_p^2} \right) \right) \quad (5.14.5)$$

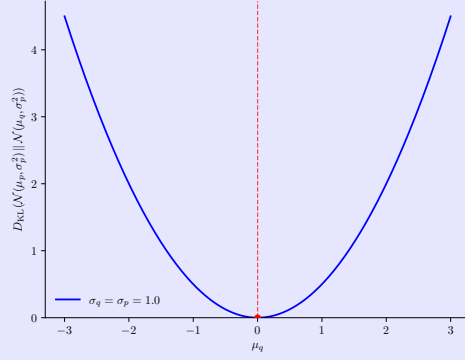


Figure 5.14.3: With equal variances $\sigma_p^2 = \sigma_q^2 = \sigma^2$, the KL reduces to $\frac{1}{2}(\mu_q - \mu_p)^2 / \sigma^2$. The plot is a convex parabola in μ_q with a unique minimum of 0 at $\mu_q = \mu_p$; the cost grows quadratically with mean error and is inversely scaled by the variance.

Jensen-Shannon Divergence* For a symmetric, bounded alternative closely related to KL, we can use the Jensen-Shannon divergence:

$$\text{JSD}(p_x, q_x) = \frac{1}{2} D_{\text{KL}}(p_x \| m_x) + \frac{1}{2} D_{\text{KL}}(q_x \| m_x), \quad m_x = \frac{1}{2}(p_x + q_x) \quad (5.14.6)$$

It behaves like a metric (its square root is a true metric) and remains finite even when supports do not. That is, we avoid the issue of infinite divergence when comparing distributions with disjoint supports. This is used in text, image, and generative-model comparisons.

Wasserstein Distance* Optimal-transport distances, or Wasserstein metrics, take a geometric view of discrepancy: they measure the minimal “work” needed to move one distribution into another by transporting probability mass through space. Concretely, if moving unit mass from x to y costs $c(x, y) = |x - y|^p$, the p -Wasserstein distance is

$$W_p(p_x, q_x) = \left(\inf_{\gamma \in \Gamma(p_x, q_x)} \int |x - y|^p d\gamma(x, y) \right)^{1/p} \quad (5.14.7)$$

where $\Gamma(p_x, q_x)$ is the set of couplings with marginals p_x and q_x . In one dimension, this becomes especially tangible because the 1-Wasserstein distance equals the L^1 distance between the cumulative distribution functions:

$$W_1(p_x, q_x) = \int_{-\infty}^{\infty} |F_p(t) - F_q(t)| dt \quad (5.14.8)$$

(Very importantly, this allows for a simple sample estimator by sorting observations and comparing empirical quantiles.) For Gaussian laws, there are also closed forms for the squared 2-Wasserstein distance:

$$W_2^2(\mathcal{N}(\mu_p, \sigma_p^2), \mathcal{N}(\mu_q, \sigma_q^2)) = (\mu_p - \mu_q)^2 + (\sigma_p - \sigma_q)^2 \quad (5.14.9)$$

In higher dimensions, the formula involves matrix square roots:

$$W_2^2(\mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1), \mathcal{N}(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)) = \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2 + \text{Tr} \left(\boldsymbol{\Sigma}_1 + \boldsymbol{\Sigma}_2 - 2 \left(\boldsymbol{\Sigma}_1^{1/2} \boldsymbol{\Sigma}_2 \boldsymbol{\Sigma}_1^{1/2} \right)^{1/2} \right) \quad (5.14.10)$$

This version has become a staple in comparing embeddings and generative models. Indeed, its optimal-transport foundation makes it especially useful in normalizing flows and diffusion models, where it gives robust, physically interpretable discrepancies between distributions (even with support mismatch!) and connects learning to Fokker-Planck/Schrödinger bridge dynamics. These types of models are extremely popular in current research (image/video/audio/small molecule generation, etc.).

5.14.3.0.1 Kolmogorov-Smirnov Statistic The Kolmogorov-Smirnov (KS) statistic is a nonparametric test that compares the cumulative distribution functions (CDFs) of two samples. Given a sample $\mathbf{x}_{1:n}$ from $p_{\mathbf{x}}$, with empirical CDF F_n , and a fully specified theoretical CDF F_{∞} for $q_{\mathbf{x}}$, the one-sample statistic is

$$D_n = \sup_x |F_n(x) - F_{\infty}(x)| \quad (5.14.11)$$

That is, it's the maximum vertical gap between the empirical and theoretical CDFs. Large values indicate a poor fit. Because KS looks only at CDF gaps, it is most sensitive near the median and can miss tail differences. If tail sensitivity is important, one might instead consider something like the Cramér-von Mises or Anderson-Darling statistics, which reweight discrepancies, though these require more setup than KS's simple sup-norm. The KS statistic is widely used in goodness-of-fit testing and model validation.

5.15 Exercises

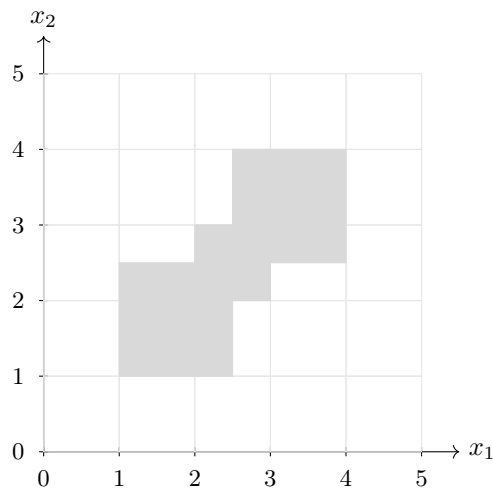
Exercise 5.1

Suppose that we are going (rare) birdwatching and the number of birds we see per hour follows a Poisson distribution with $\lambda = 3$. What is the probability we see at least one bird in an hour?

Reminder: $P_k(k) = \frac{\lambda^k e^{-\lambda}}{k!}$

Exercise 5.2

Consider the joint distribution $P_{\mathbf{x}_1, \mathbf{x}_2}(x_1, x_2)$:



$$P_{\mathbf{x}_1, \mathbf{x}_2}(x_1, x_2) = \begin{cases} 1/5 & (x_1, x_2) \in \Omega \\ 0 & \text{otherwise} \end{cases}$$

What is $p(x_1 | x_2 > 3)$?

Exercise 5.3

If my prior is

$$P_{\mathbf{p}}(p) = \begin{cases} 2 & 0 < p < 0.5 \\ 0 & \text{otherwise} \end{cases}$$

(i.e., I believe tails is more likely), what will the posterior distribution converge to if I flip N heads in a row with $N \rightarrow \infty$?

Exercise 5.4

Let $\mathbf{x} \sim \mathcal{U}(2, 5)$. The PDF of $\mathbf{x}$ takes the form

$$p_{\mathbf{x}}(x) = \begin{cases} c, & x \in [2, 5], \\ 0, & \text{otherwise,} \end{cases}$$

for some constant c .

- Find c that makes $p_{\mathbf{x}}$ a valid PDF.
- Find the cumulative distribution function (CDF). Make sure to define it over the entire real line $\mathbb{R}$.
- Calculate the expected value of $\mathbf{x}$.
- Let $y = \mathbf{x}^2$. The PDF of y , obtained via a change of variables, is

$$p_y(y) = \begin{cases} \frac{1}{6\sqrt{y}}, & y \in [4, 25], \\ 0, & \text{otherwise.} \end{cases}$$

Calculate $\mathbb{E}[y]$ in two ways: (i) directly from $p_y(y)$, and (ii) using $p_{\mathbf{x}}(x)$.

Exercise 5.5

Let the random vector $\mathbf{x}$ be defined by

$$\mathbf{x} = \begin{bmatrix} \xi_1 \\ \xi_1 + \xi_2 \end{bmatrix}, \quad \xi_1, \xi_2 \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1).$$

- Find the covariance matrix of $\mathbf{x}$.
- Using the result of (a), write the joint PDF $p_{\mathbf{x}}(\mathbf{x})$.
- Find the marginal distribution $p_{x_2}(x_2)$ by integrating x_1 out of the joint PDF.
- Derive the conditional distribution $p_{x_2|x_1}(x_2 | x_1)$.
- Using the result of (d) together with $p_{x_1}(x_1)$, rederive $p_{x_2}(x_2)$. Check that it matches the result of (c).

Exercise 5.6

The PDF of the Erlang distribution with shape parameter k and rate λ is

$$p_{\mathbf{x}}(x; k, \lambda) = \frac{\lambda^k x^{k-1} e^{-\lambda x}}{(k-1)!}, \quad x \geq 0.$$

Using this fact, calculate

$$\int_0^{\infty} \frac{32 x^4 e^{-2x}}{4!} dx.$$

[Hint: this does not require integration by parts.]

Exercise 5.7

A call center receives an average of $\lambda = 5$ calls per minute. Assuming the time between successive calls follows an exponential distribution, the probability that the k -th call arrives by time x is given by the Erlang CDF

$$F_x(x; k, \lambda) = 1 - e^{-\lambda x} \sum_{n=0}^{k-1} \frac{(\lambda x)^n}{n!}.$$

Compute $F_x(x = 5 \text{ min}, k = 10 \text{ calls}, \lambda = 5 \text{ calls/min})$, i.e. the probability that at least 10 calls arrive within 5 minutes. Implement the calculation in MATLAB.

Exercise 5.8

Starting on day $t = 0$, one cake is baked each day. Each day your friend consumes $\frac{1}{2}$ of the cake baked that day, $\frac{1}{3}$ of the cake baked the previous day, and $\frac{1}{6}$ of the cake baked two days ago. The consumption-delay distribution is described by the PMF

$$p_t(0) = \frac{1}{2}, \quad p_t(1) = \frac{1}{3}, \quad p_t(2) = \frac{1}{6}.$$

Using the discrete convolution $p_s(s) = \sum_{x=0}^s p_x(x) p_y(s-x)$, find how much cake is eaten on day $t = 0$, day $t = 1$, and every day from $t = 2$ onward. What general relationship does this illustrate between a step function, a PMF, and the CMF?

Exercise 5.9

Let $u(t)$ denote the unit step function,

$$u(t) = \begin{cases} 1, & t \geq 0, \\ 0, & t < 0, \end{cases}$$

and let $p_t(t)$ be a continuous PDF. Show that the convolution

$$(u * p_t)(t) = \int_{-\infty}^{\infty} u(\tau) p_t(t - \tau) d\tau$$

equals the CDF $F_t(t)$.

Exercise 5.10

Calculate the empirical covariance matrix for this set of measurements.

Measurement	x_1	x_2
1	1	10
2	2	9
3	3	11

Chapter 6

Parameter Estimation

The previous chapter provided us with the language of probability, a formal framework for describing uncertainty. Now, we will put that language to work. We frequently rely on mathematical models to describe physical phenomena, predict system behavior, and optimize processes. However, a model is only as good as its parameters, which are often unknown and must be determined from experimental measurements. The process of relating models to data involves a cycle of designing experiments, collecting data, estimating model parameters, and quantifying the uncertainty associated with those estimates. This chapter explores the fundamental concepts and numerical methods that we use throughout this process.

We will begin by establishing the formal problem of parameter estimation. We will then look at two major philosophical frameworks for statistical inference: the frequentist and Bayesian approaches. For each, we will discuss methods for obtaining point estimates of parameters and for characterizing the uncertainty of those estimates. Finally, we will explore the principles of optimal experimental design, which provides a systematic way to choose experiments that are maximally informative to our models and their parameters. Each of these topics could be (and is) the subject of entire textbooks and courses, and we will only be able to scratch the surface here. However, the goal is to provide a solid foundation that will allow you to understand and apply these concepts in your own work.

6.1 Linear Model Fitting

The simplest and most ubiquitous type of model is a linear one. When we construct a linear model, we are asserting that a single scalar output, y , can be described as a weighted sum of n input variables, or features, contained in a vector $\mathbf{x}$. We assume that this linear relationship is the true underlying process, but our measurements are corrupted by some random noise, ϵ . Mathematically, we write this as

$$y = \sum_{i=1}^n x_i \theta_i + \epsilon = \mathbf{x}^\top \boldsymbol{\theta} + \epsilon \quad (6.1.1)$$

Here, $\boldsymbol{\theta}$ is a vector of unknown regression parameters we want to find. Our model's prediction, denoted $\tilde{y}$, is the deterministic part of this relationship given a particular input vector, $\mathbf{x}$:

$$\tilde{y}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{x}^\top \boldsymbol{\theta} \quad (6.1.2)$$

The noise term ϵ is a random variable. It is very often assumed to be drawn from a zero-mean normal distribution, $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$, meaning our measurements are unbiased but subject to Gaussian fluctuations; however, other noise models are possible depending on the experimental context and this assumption should not be made blindly.

Linearity in the Parameters A linear model is one that is linear in its parameters θ , not necessarily in its inputs $\mathbf{x}$. For example, we can model the temperature T as a quadratic function of radius r , and it is still a linear model:

$$T(r; \theta) = \theta_0 + \theta_1 r + \theta_2 r^2 \quad (6.1.3)$$

This is a linear model because we can define an input vector $\mathbf{x} = [1, r, r^2]^\top$ and the model becomes $T(r; \theta) = \mathbf{x}^\top \theta$. In contrast, a model like $\tilde{y}(x; \theta_1) = \theta_1 x_1 + \theta_1^2 x_2$ would be considered nonlinear, as it is nonlinear in the parameter θ_1 .

Note that while we wrote our most general linear model, Eq. 6.1.1 with sans-serif fonts to denote random variables, we will often drop this distinction in this chapter and use regular math fonts. We will commonly write our model as we would write a function instead, like in Eq. 6.1.2. The context will usually make it clear whether we are referring to a random variable or a specific realization (measurement). Similarly, sometimes we may drop the “; θ ” in the function arguments when it is clear we are referring to a model with fixed parameters.

6.1.1 Constructing the Design Matrix

Suppose we have performed N experiments, giving us a set of N data points. For each experiment j , we have a known input vector $\mathbf{x}_j$ and a measured output y_j . We can stack these observations into a single matrix equation:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} = \begin{bmatrix} - & \mathbf{x}_1^\top & - \\ - & \mathbf{x}_2^\top & - \\ & \vdots & \\ - & \mathbf{x}_N^\top & - \end{bmatrix} \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_N \end{bmatrix} \quad (6.1.4)$$

This is more compactly written as

$$\mathbf{y} = \mathbf{X}\theta + \epsilon \quad (6.1.5)$$

Here, $\mathbf{y}$ is the vector of all measured values, ϵ is the vector of all noise terms, and $\mathbf{X}$ is the $N \times n$ matrix known as the **design matrix**, whose rows are our input vectors. The entire problem of fitting now comes down to this: given the observed data $(\mathbf{X}, \mathbf{y})$, how do we find the optimal parameter vector, θ^* ?

Example: Constructing the Design Matrix: Consider a simple linear regression problem where we want to model the relationship between two input features, x_1 and x_2 , and a single output variable, y . We have collected data from 4 experiments, resulting in the following observations:

Experiment	Input Features (x_1, x_2)	Measured Output y
1	(1.0, 2.0)	5.0
2	(2.0, 1.0)	6.0
3	(3.0, 3.0)	10.0
4	(4.0, 2.0)	9.0

To construct the design matrix $\mathbf{X}$, we arrange the input features of each experiment into rows. Each row corresponds to an experiment, and each column corresponds to a feature. Thus, our design matrix $\mathbf{X}$ and output vector $\mathbf{y}$ are:

$$\mathbf{X} = \begin{bmatrix} 1.0 & 2.0 \\ 2.0 & 1.0 \\ 3.0 & 3.0 \\ 4.0 & 2.0 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 5.0 \\ 6.0 \\ 10.0 \\ 9.0 \end{bmatrix} \quad (6.1.6)$$

With this setup, our parameter vector θ will have two components in the linear relationship we are modeling: $y = \theta_1 x_1 + \theta_2 x_2 + \epsilon$.

When performing a linear regression, we might also choose to include an intercept term (bias) in our model. To do this, we add a column of ones to the design matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & 1.0 & 2.0 \\ 1 & 2.0 & 1.0 \\ 1 & 3.0 & 3.0 \\ 1 & 4.0 & 2.0 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 5.0 \\ 6.0 \\ 10.0 \\ 9.0 \end{bmatrix} \quad (6.1.7)$$

This extra column acts like a constant $x_0 = 1$ and allows the model to learn an intercept term in addition to the coefficients for x_1 and x_2 . Thus, our parameter vector $\boldsymbol{\theta}$ will now have three components in the linear relationship we are modeling: $y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \epsilon$. Note that θ_1 and θ_2 still pair with x_1 and x_2 as before, with the intercept singled out as θ_0 ; it is important to be consistent about this kind of indexing convention, since different sources (and even different examples in these notes) may index coefficients starting from either 0 or 1.

6.1.2 Solving for the Best Parameters

With the design matrix constructed, we can now focus on finding the optimal parameters, $\boldsymbol{\theta}^*$. But what does “optimal” even mean in this context? There are several methods to achieve this, each with its own assumptions and implications. We will discuss two common approaches: Ordinary Least Squares (OLS) and Weighted Least Squares (WLS), and how they can be extended with regularization techniques to improve model robustness.

Ordinary Least Squares (OLS) The most common definition of optimality is to minimize the discrepancy between our model’s predictions and the observed data. Specifically, we seek to minimize the sum of squared errors (SSE), also known as the residual sum of squares.

$$\boldsymbol{\theta}_{\text{OLS}}^* = \arg \min_{\boldsymbol{\theta}} \sum_{j=1}^N (y_j - \tilde{y}(\mathbf{x}_j; \boldsymbol{\theta}))^2 \quad (6.1.8)$$

This objective function can also be written in vector form as the squared Euclidean norm of the residual vector:

$$\boldsymbol{\theta}_{\text{OLS}}^* = \arg \min_{\boldsymbol{\theta}} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) \quad (6.1.9)$$

To find the minimum, we can first expand this expression as

$$\text{SSE} = (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) \quad (6.1.10)$$

$$= \mathbf{y}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\boldsymbol{\theta} - \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y} + \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} \quad (6.1.11)$$

$$= \mathbf{y}^\top \mathbf{y} - 2\boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y} + \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} \quad (6.1.12)$$

Taking the gradient with respect to $\boldsymbol{\theta}$ gives

$$\frac{\partial \text{SSE}}{\partial \boldsymbol{\theta}} = -2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} \quad (6.1.13)$$

Setting this to zero (the necessary first-order optimality condition for an optimization problem) gives us:

$$\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y} \quad (6.1.14)$$

The solution for the optimal parameters, $\boldsymbol{\theta}_{\text{OLS}}^*$, is found by solving this system of linear equations and is given by

$$\boldsymbol{\theta}_{\text{OLS}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (6.1.15)$$

This is the famous OLS solution for linear regression. The term $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ is often called the pseudoinverse of $\mathbf{X}$ (when $\mathbf{X}$ has full column rank). We note that if $\mathbf{X}$ is square and invertible, then $\boldsymbol{\theta}_{\text{OLS}}^* = \mathbf{X}^{-1} \mathbf{y}$, as we would expect.

In practice, one should rarely form and invert $\mathbf{X}^\top \mathbf{X}$ explicitly: doing so squares the condition number of the problem, $\kappa(\mathbf{X}^\top \mathbf{X}) = \kappa(\mathbf{X})^2$, which can turn a mildly ill-conditioned design matrix into a numerically disastrous linear system. Even if solving that linear system through strategies that avoid explicit matrix inversion, this can create rounding errors and numerical imprecision. For all but small, well-conditioned problems, it is numerically preferable to solve for $\boldsymbol{\theta}_{\text{OLS}}^*$ via a “QR” or SVD decomposition of $\mathbf{X}$ directly, which is what standard least-squares routines (e.g., MATLAB’s backslash operator or `lstsq` in NumPy/SciPy) do internally.

Warning...? Since we have defined $\boldsymbol{\theta}_{\text{OLS}}^*$ using only the first-order optimality criterion, is there a risk that this solution is not the global minimum, but rather a local minimum or even a maximum?

Fortunately, the objective function we are minimizing, the sum of squared errors, is a convex function in $\boldsymbol{\theta}$. Taking the derivative of Eq. 6.1.13 once more, the Hessian is $\mathbf{X}^\top \mathbf{X}$, which is a positive semi-definite matrix. We know this because for any vector $\mathbf{v} \neq \mathbf{0}$, $\mathbf{v}^\top \mathbf{X}^\top \mathbf{X} \mathbf{v} = (\mathbf{X} \mathbf{v})^\top (\mathbf{X} \mathbf{v}) \equiv \|\mathbf{X} \mathbf{v}\|_2^2 \geq 0$. Thus, the function is convex, and any stationary point is guaranteed to be a global minimum.

Note, however, that “convex” is not the same as “strictly convex”: if $\mathbf{X}$ is rank-deficient—for example, if there are more parameters than independent data directions—then $\mathbf{X}^\top \mathbf{X}$ is singular and the SSE has a whole set of global minimizers rather than a unique $\boldsymbol{\theta}_{\text{OLS}}^*$. The first-order condition $\mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y}$ no longer defines a unique value of $\boldsymbol{\theta}$. This is one scenario that motivates the regularization techniques we introduce below.

Weighted Least Squares (WLS) The OLS solution treats all data points as equally important. Implicitly, the assumption is that all are equally trustworthy because they have identical measurement noise—this property is called **homoscedasticity**. In many experiments, however, we will have higher confidence in some measurements than others. This situation, where the noise variance is different for different observations, is called **heteroscedasticity**. In this case, it makes sense to give more weight to the low-noise data points when minimizing the error. If we are more confident in some data than others, we prioritize fitting the high-confidence data.

We can achieve this **weighted least squares** regression by minimizing a weighted sum of squared errors, where the weighting is done by the inverse of the noise covariance matrix, $\mathbf{V}_\epsilon = \text{Cov}(\epsilon)$. If our measurement errors are uncorrelated, this matrix is diagonal with entries equal to the variances of each measurement error, $\sigma_{\epsilon_j}^2$. The objective function in this special case is

$$\boldsymbol{\theta}_{\text{WLS}}^* = \arg \min_{\boldsymbol{\theta}} \sum_{j=1}^N \frac{(y_j - \tilde{y}(\mathbf{x}_j; \boldsymbol{\theta}))^2}{\sigma_{\epsilon_j}^2} \quad (6.1.16)$$

In general, however, the errors may be correlated, leading to off-diagonal terms in $\mathbf{V}_\epsilon$. The objective function in this more general case cannot be expressed as a simple sum over individual experiments; instead, we write it in matrix form as

$$\boldsymbol{\theta}_{\text{WLS}}^* = \arg \min_{\boldsymbol{\theta}} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) \quad (6.1.17)$$

The solution to this weighted least squares problem can be found through the same process of differentiation and algebraic manipulation we saw for OLS. The result is

$$\boldsymbol{\theta}_{\text{WLS}}^* = (\mathbf{X}^\top \mathbf{V}_\epsilon^{-1} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{V}_\epsilon^{-1} \mathbf{y} \quad (6.1.18)$$

This ensures that experiments with low variance (high confidence) have a greater influence on the final parameter estimates. Of course, this method requires that we know or can estimate the noise covariance matrix $\mathbf{V}_\epsilon$. If repeated trials are available, an empirical estimator for the measurement-error covariance is

$$[\mathbf{V}_\epsilon]_{ij} \approx \frac{1}{N} \sum_{n=1}^N (y_{i,n} - \langle y_i \rangle) (y_{j,n} - \langle y_j \rangle)$$

where $y_{i,n}$ is the n th measurement of experiment i , and $\langle y_i \rangle$ is the mean of all measurements for experiment i .¹

Example: OLS vs. WLS in the Presence of Heteroscedastic Errors

Consider a spectroscopic calibration experiment where we measure analyte concentration using absorbance. At low concentrations, measurements are precise due to optimal detector response. However, at high concentrations, the detector approaches saturation, leading to increasingly noisy measurements with systematic positive bias (high readings). Figure 6.1.1 simulates this scenario with 100 measurements where measurement variance increases quadratically with concentration. Additionally, 20 high-concentration measurements (> 85 units) exhibit both high variance and systematic positive bias, mimicking detector saturation effects.

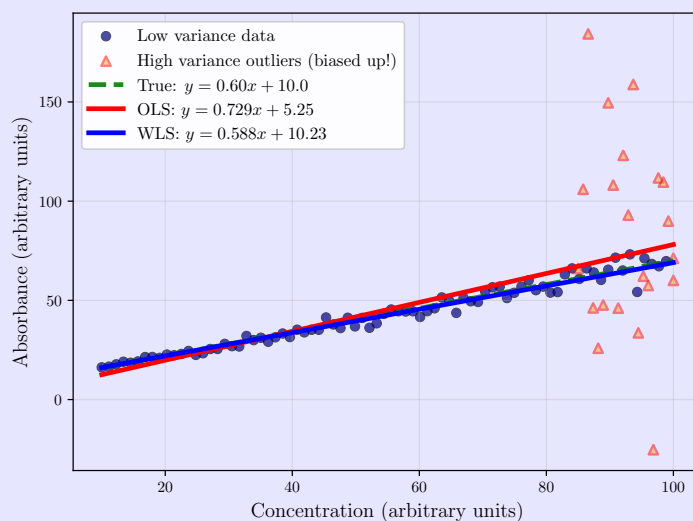


Figure 6.1.1: OLS vs. WLS in the presence of heteroscedastic errors.

OLS treats all measurements equally, regardless of their precision. The biased high-concentration outliers heavily influence the fit, pulling the slope upward. The resulting OLS estimate has a slope of 0.729, representing a 21.5% error from the true value of 0.600.

WLS assigns weights inversely proportional to measurement variance ($w_i = 1/\sigma_i^2$), effectively down-weighting unreliable high-concentration data while trusting precise low-concentration measurements. The WLS fit yields a slope of 0.588, only 2.0% from the true value. This is roughly 10.8 times more accurate than OLS! So we can see that when measurement precision varies in a known manner, WLS provides substantially better parameter estimates by appropriately weighting each data point according to its reliability. Common sources of heteroscedastic errors include detector saturation, signal-dependent noise, and concentration-dependent matrix effects.

6.1.3 Regularization

The ordinary and weighted least squares solutions provide a direct way to find parameters that minimize the fit error. However, these methods can encounter problems, particularly when: (1) the number of features n is large, approaching or exceeding the number of data points N ; (2) the input features are highly correlated (multicollinearity), making the $\mathbf{X}^\top \mathbf{X}$ matrix ill-conditioned or singular, leading to unstable solutions; or

¹Note that this requires estimating $N(N+1)/2$ independent entries of an $N \times N$ covariance matrix. In practice this demands far more replicate experiments than are typically available, so the raw empirical estimate is often itself ill-conditioned or singular unless N is small or a structured covariance model (e.g., diagonal, or a parametric form) is imposed instead.

(3) we want to encourage a simpler model, perhaps one where many parameters are exactly zero. In these situations, the OLS/WLS solution may fit the training data well but perform poorly on new, unseen data (i.e., it overfits). To combat this, we introduce **regularization terms** into our objective function. These terms add a penalty for complexity, which drives the parameter estimates towards more stable and generalizable solutions.

The general idea is to modify the optimization problem to minimize not just the sum of squared errors, but also a function of the parameter magnitudes:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) + \text{regularization penalty} \quad (6.1.19)$$

The choice of the regularization penalty defines different types of regularized regression.

L2 Regularization L2 regularization, also known as ridge regression, adds a penalty proportional to the sum of the squared magnitudes of the parameters. This encourages parameters to be small (i.e., it more evenly distributes the weight across all parameters) but rarely drives them exactly to zero.

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) + \beta \|\boldsymbol{\theta}\|_2^2 \quad (6.1.20)$$

where $\|\boldsymbol{\theta}\|_2^2 = \sum_{i=1}^n \theta_i^2$ is the squared Euclidean (L2) norm of the parameter vector, and $\beta > 0$ is a tuning parameter that controls the strength of the regularization. A larger β implies a stronger penalty on parameter magnitudes. The solution for ridge regression also has a closed form:

$$\boldsymbol{\theta}^* = (\mathbf{X}^\top \mathbf{V}_\epsilon^{-1} \mathbf{X} + \beta \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{V}_\epsilon^{-1} \mathbf{y} \quad (6.1.21)$$

where $\mathbf{I}$ is the identity matrix. The addition of $\beta \mathbf{I}$ to $\mathbf{X}^\top \mathbf{V}_\epsilon^{-1} \mathbf{X}$ makes the matrix more stable and invertible (hopefully you remember that we've seen this type of trick to address ill-conditioning before in the context of iterative methods' update rules, e.g., the Hessian modification for Newton's method!).

L1 Regularization L1 regularization, or lasso regression (least absolute shrinkage and selection operator), adds a penalty proportional to the 1-norm: the sum of the absolute magnitudes of the parameters. A unique property of L1 regularization is that it tends to drive some parameters exactly to zero. This means it is effectively performing feature selection and thus giving a sparse model. The minimization problem is

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) + \alpha \|\boldsymbol{\theta}\|_1 \quad (6.1.22)$$

where $\|\boldsymbol{\theta}\|_1 = \sum_{i=1}^n |\theta_i|$ is the L1 norm of the parameter vector, and $\alpha > 0$ controls the regularization strength. Unlike ridge regression, lasso generally does not have a closed-form solution and typically requires iterative optimization algorithms to solve. Lasso regression is popular in many scientific and engineering applications where interpretability and simplicity of the model are desired.

Remark: Bias-Variance Tradeoff and Choosing the Regularization Strength

Regularization works by deliberately introducing a controlled amount of bias into the parameter estimates in exchange for a (hopefully larger) reduction in their variance; this can lower the expected error on new, unseen data even though it increases the error on the training data itself. This bias-variance tradeoff is exactly what the tuning parameters β (ridge) and α (lasso) control. If the values are too small, regularization has no effect; if they are too large, the model is oversimplified. In practice, these tuning parameters are themselves typically chosen using cross-validation, generalized cross-validation, information criteria (AIC/BIC), or heuristics related to how the error changes as a function of hyperparameter value; we will not derive or discuss these methods here, but you should know they exist.

Elastic Net Regularization The elastic net combines both L1 and L2 regularization penalties. It encourages both small parameters (like ridge) and sparsity (like lasso).

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) + \alpha \|\boldsymbol{\theta}\|_1 + \beta \|\boldsymbol{\theta}\|_2^2 \quad (6.1.23)$$

Example: Regularization in Action (Polynomial Regression). To illustrate the effects of different regularization methods, we fit a degree-15 polynomial to 30 noisy observations from a sine function. Figure 6.1.2 shows the predictions (top row) and coefficient magnitudes (bottom row) for each method.

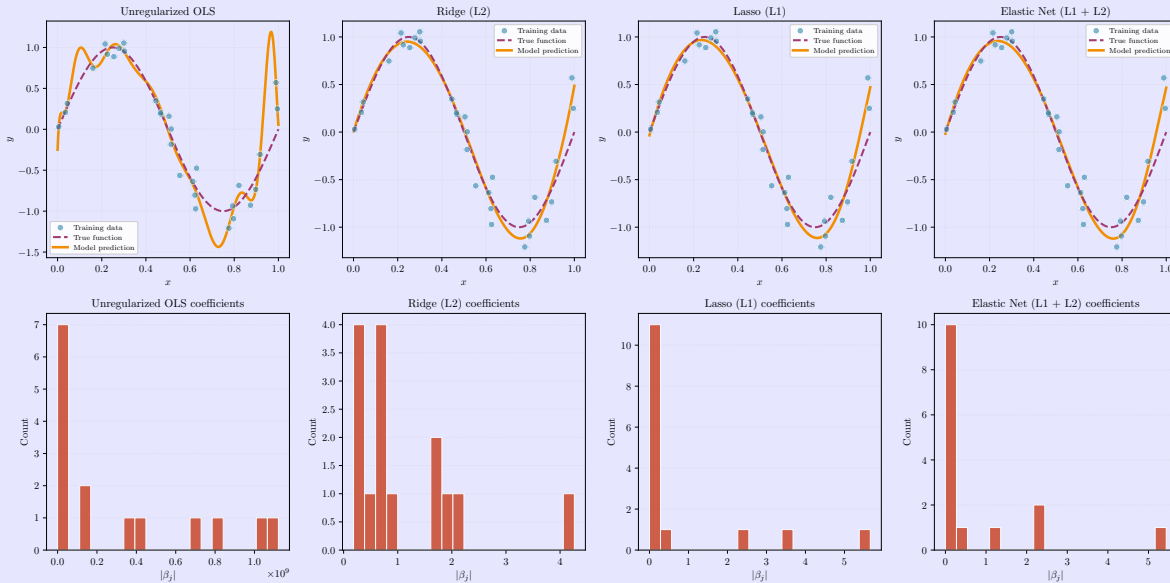


Figure 6.1.2: Regularization in action (polynomial regression).

Unregularized OLS: Without regularization, the model severely overfits the training data. Despite fitting the training points well, it exhibits wild oscillations between observations and fails to capture the true underlying function. We can see from the coefficient histogram that there are extreme parameter values (some exceeding 10^9 in magnitude). This is typical when fitting high-degree polynomials without regularization. The ill-conditioned $\mathbf{X}^\top \mathbf{X}$ matrix leads to these pathological solutions.

Ridge (L2): L2 regularization dramatically improves the fit. The prediction closely follows the true function with smooth interpolation between training points. The coefficient histogram shows that all parameters are shrunk to modest magnitudes, with most coefficients distributed around similar scales.

Lasso (L1): L1 regularization also produces a smooth fit. Notably, the coefficient histogram reveals the sparsity-inducing property of lasso: the large bar at zero means that many coefficients are driven to exactly zero. Only a handful of the 15 polynomial terms are deemed necessary, so we get a simpler model.

Elastic Net (L1 + L2): Combining both penalties, elastic net produces results similar to lasso with some coefficient sparsity, though perhaps not as aggressively as pure lasso.

6.2 Frequentist vs. Bayesian Schools of Thought

Above, we derived the OLS and WLS solutions by seeking the parameters that minimized the sum of squared errors. This is an intuitive and reasonable goal. However, this approach can be more rigorously justified by

reframing the entire problem of parameter estimation from a probabilistic viewpoint. This is what we'll do next.

When we think about the uncertainty in model fitting, there are two dominant philosophical approaches for treating the model parameters θ . The frequentist view assumes that there is one true, fixed, but unknown parameter vector, θ^* . The data we observe, $\mathcal{D}$, is considered a random sample from a process governed by these true parameters. If we were to repeat our entire experiment, we would get different data, but the true θ^* would remain the same. The goal of a frequentist is to devise procedures (estimators) that have good long-run properties, like being unbiased. On the other hand, the Bayesian view treats the parameters themselves as uncertain quantities. It assumes that θ is a random vector, and we can use probability distributions to describe our degree of belief about its values.

The Bayesian approach uses Bayes' theorem to formalize the process of learning. Given our observed data $\mathcal{D}$, we update our initial beliefs about the parameters (the *prior*) to arrive at an updated belief (the *posterior*).

$$\underbrace{p(\theta|\mathcal{D})}_{\text{posterior}} = \frac{\overbrace{p(\mathcal{D}|\theta)}^{\text{likelihood}} \overbrace{p(\theta)}^{\text{prior}}}{\underbrace{p(\mathcal{D})}_{\text{evidence}}} \quad (6.2.1)$$

We've seen this before when we introduced Bayes' theorem in the context of probability, but we'll review what each piece means here again. The prior, $p(\theta)$, reflects our beliefs about the parameters *before* we see any data. The likelihood, $p(\mathcal{D}|\theta)$, is the probability of observing our specific dataset $\mathcal{D}$ if the parameters were in fact θ . The posterior, $p(\theta|\mathcal{D})$, is our updated belief about the parameters *after* observing the data. It combines our prior knowledge with the information from the data. We will now see that the concept of the likelihood is the basis of the frequentist approach to parameter estimation.

6.3 Maximum Likelihood Estimation

The most common frequentist method for parameter estimation is maximum likelihood estimation (MLE). MLE takes the simple stance that we should choose the parameters θ^* that make our observed data $\mathcal{D}$ the most probable. In other words, we seek the parameters that maximize the likelihood function.

$$\theta_{\text{MLE}}^* = \arg \max_{\theta} p(\mathcal{D}|\theta) \quad (6.3.1)$$

Let's apply this to the linear model we've been using: $\mathbf{y} = \mathbf{X}\theta + \epsilon$, where we assume the noise is drawn from a multivariate Gaussian distribution with zero mean and covariance $\mathbf{V}_\epsilon$, i.e., $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{V}_\epsilon)$. Under this assumption, the likelihood of observing our data vector $\mathbf{y}$ for a given set of parameters θ is given by the multivariate normal PDF:

$$p(\mathcal{D}|\theta) = p(\mathbf{y}|\mathbf{X}, \theta) = \frac{1}{(2\pi)^{N/2} |\mathbf{V}_\epsilon|^{1/2}} \exp \left[-\frac{1}{2} (\mathbf{y} - \mathbf{X}\theta)^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\theta) \right] \quad (6.3.2)$$

Our goal is to find the θ that maximizes this expression. Because the logarithm is a monotonically increasing function, maximizing a function is equivalent to maximizing its logarithm, so we can work with the log-likelihood instead. This is a lot more convenient since it turns multiplications into additions and exponentials into linear terms.

$$\begin{aligned} \log p(\mathcal{D}|\theta) &= \log \left(\frac{1}{(2\pi)^{N/2} |\mathbf{V}_\epsilon|^{1/2}} \exp \left[-\frac{1}{2} (\mathbf{y} - \mathbf{X}\theta)^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\theta) \right] \right) \\ &= -\frac{N}{2} \log(2\pi) - \frac{1}{2} \log |\mathbf{V}_\epsilon| - \frac{1}{2} (\mathbf{y} - \mathbf{X}\theta)^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\theta) \end{aligned}$$

To find the optimal θ , we can discard any terms that do not depend on it, giving us

$$\theta_{\text{MLE}}^* = \arg \max_{\theta} \left[-\frac{1}{2} (\mathbf{y} - \mathbf{X}\theta)^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\theta) \right] \quad (6.3.3)$$

Maximizing a negative quantity is the same as minimizing its positive counterpart, and we can throw away the prefactor of $1/2$ since it doesn't change the location of the minimum. Therefore, the MLE solution is

$$\boldsymbol{\theta}_{\text{MLE}}^* = \arg \min_{\boldsymbol{\theta}} [(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})] \quad (6.3.4)$$

This is precisely the objective function for WLS! So the intuitive method of minimizing the sum of squared errors is equivalent to the more formal method of MLE provided that we assume the experimental noise is independent and Gaussian. This gives a nice probabilistic justification for the widespread use of least-squares methods.

Two Caveats on This Derivation

First, this derivation implicitly assumes that the input vector $\mathbf{x}_j$ is known exactly and only the output y_j is corrupted by noise. This is not always true in practice. Flow rates, temperatures, and concentrations used as regressors typically carry their own measurement error too. When the error in the inputs is significant relative to the noise in the outputs, ordinary/weighted least squares is not necessarily the right approach. **Errors-in-variables regression** (also called **total least squares**) are terms you can look up if you are interested in learning more.

Second, it is both true and convenient that least squares is equivalent to MLE under Gaussian noise. This is not the only justification for using MLE, however. The Gauss–Markov theorem shows that OLS is the **Best Linear Unbiased Estimator (BLUE)** under much weaker assumptions of zero-mean, constant-variance (homoscedasticity), uncorrelated errors, with no assumption of normality required.

6.4 Maximum a Posteriori Estimation

The MLE approach assumes there is one true parameter set that we are trying to find. By contrast, the Bayesian approach allows for uncertainty in the parameters themselves by treating them as random variables. This lets us incorporate our own prior knowledge or beliefs into the estimation process. While a full Bayesian analysis involves finding the entire posterior distribution $p(\boldsymbol{\theta}|\mathcal{D})$, we often want a single point estimate for the parameters. The Bayesian equivalent of the MLE is the **maximum a posteriori (MAP)** estimate, which seeks the most probable parameter vector *given the data*. In other words, we find the peak, or mode, of the posterior distribution.

$$\boldsymbol{\theta}_{\text{MAP}}^* = \arg \max_{\boldsymbol{\theta}} p(\boldsymbol{\theta}|\mathcal{D}) \quad (6.4.1)$$

Using Bayes' theorem, we can rewrite this as

$$\boldsymbol{\theta}_{\text{MAP}}^* = \arg \max_{\boldsymbol{\theta}} \frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})} \quad (6.4.2)$$

Since the evidence term $p(\mathcal{D})$ in the denominator does not depend on our parameters $\boldsymbol{\theta}$, it doesn't affect the location of the maximum. We can therefore simplify the optimization problem to maximizing the numerator:

$$\boldsymbol{\theta}_{\text{MAP}}^* = \arg \max_{\boldsymbol{\theta}} p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}) \quad (6.4.3)$$

The difference between MLE and MAP is just one term, the prior distribution $p(\boldsymbol{\theta})$. MLE maximizes only the likelihood, whereas MAP maximizes the likelihood multiplied by the prior.

Warning: MAP is Not Reparameterization-Invariant

One important caveat: the MAP estimate is **not** invariant to reparameterizations of $\boldsymbol{\theta}$. If the variable is transformed nonlinearly, the mode of the transformed density is generally not located at the transformed location of the original mode. This matters in kinetics, where rate constants k are often given a prior on $\log k$ rather than k itself (since k can span many orders of magnitude): the MAP estimate of k and

$\exp(\text{MAP estimate of } \log k)$ are generally *not* the same point, even if we intuitively treat k and $\log k$ as "the same" parameter expressed in two different coordinates.

The Role of the Prior The prior distribution $p(\theta)$ is our way of encoding beliefs about the parameters *before* observing any data. This knowledge might come from physical laws, previous experiments, or expert opinion. The resulting posterior distribution is a principled compromise between our prior beliefs and the evidence from the new data (Figure 6.4.1).

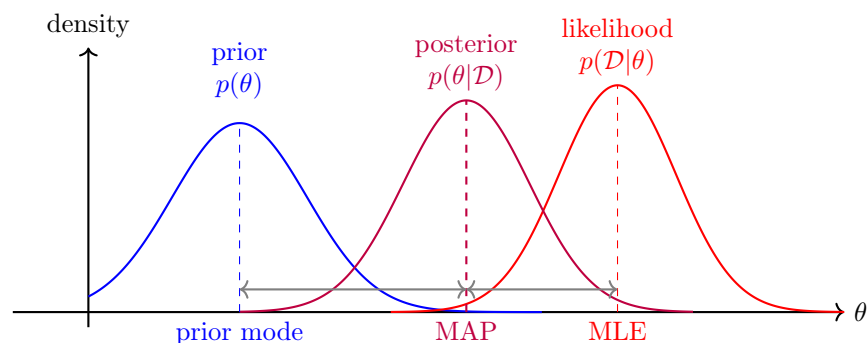


Figure 6.4.1: The relationship between prior, likelihood, and posterior. The posterior distribution's peak (the MAP estimate) is a compromise, pulled away from the likelihood's peak by the influence of the prior.

The strength of the prior matters. A broad, uninformative prior (e.g., a uniform distribution) suggests we have little initial knowledge, and the posterior will be dominated by the data (the likelihood). A narrow, strong prior suggests we are very confident in our initial belief, and it will take a large amount of contradictory data to change our minds.

Example: How Prior Beliefs Shape What We Learn from Data

Imagine you find a coin and flip it $N = 20$ times, observing $k = 12$ heads (60% heads). Is this coin fair, or is it biased? Your answer depends not just on what you observed, but also on what you believed *before* seeing the data. The data alone (12 heads in 20 flips) suggests the coin favors heads, with peak likelihood around $\theta \approx 0.6$. But should we trust this conclusion fully? Let's consider three different starting points. We'll omit the explicit calculations of the posteriors here for brevity and simply state the results in each case.^a

First, suppose you know nothing about this coin. You adopt a uniform prior $\text{Beta}(1, 1)$, which assigns equal probability to any value of θ from 0 to 1. You're saying: "I have no reason to favor any hypothesis over another." In this case, the posterior $\text{Beta}(13, 9)$ closely follows the data, concentrating around $\theta \approx 0.6$. With weak prior beliefs, the data dominates your conclusion. This is shown in Figure 6.4.2 (A).

Now, suppose you have strong reason to believe the coin is fair; perhaps it's a fresh coin from the national mint, where quality control is rigorous. You encode this belief with $\text{Beta}(20, 20)$, a prior that strongly expects $\theta \approx 0.5$. Even after seeing 12 heads, your posterior $\text{Beta}(32, 28)$ remains centered near fairness (approximately 0.53), only slightly nudged by the data. Strong, well-justified priors act as regularizers, preventing you from overreacting to moderate evidence. This skepticism is valuable when you have domain knowledge suggesting the data might be noisy or sample sizes small. This is shown in Figure 6.4.2 (B).

Finally, suppose you suspect this coin is heavily biased toward heads; perhaps someone told you it was a trick coin. You start with $\text{Beta}(8, 2)$, centered around $\theta \approx 0.8$. Now, your posterior $\text{Beta}(20, 10)$ is a compromise: the data pulled you down from 0.8 toward 0.6, but you haven't fully accepted the data's verdict. You end up around $\theta \approx 0.67$. Misspecified priors can bias your conclusions, anchoring you away

from what the data actually shows. This is both a feature (if your prior knowledge is genuine) and a bug (if your prior is wrong). This is shown in Figure 6.4.2 (C).

Bayesian inference is a negotiation between prior beliefs and observed data. Weak priors let the data speak loudly, while strong priors demand more evidence before changing your mind. The art lies in choosing priors that honestly reflect your knowledge (or lack thereof) before seeing the data.

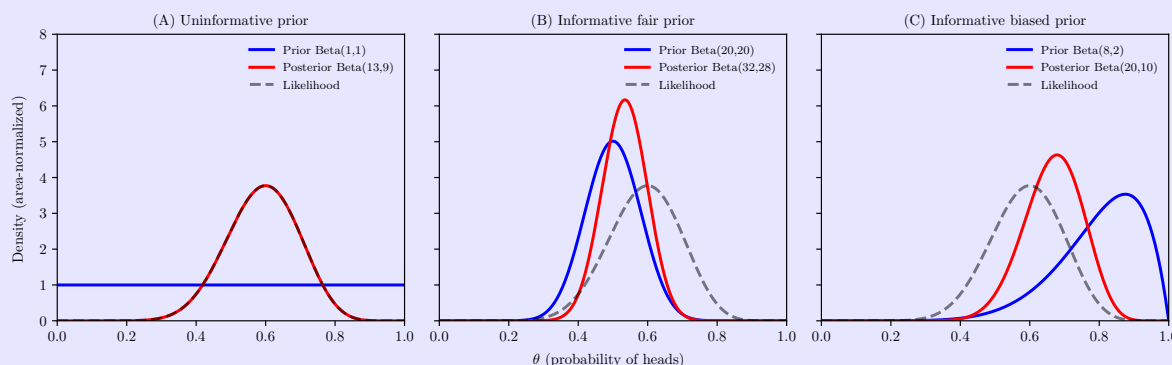


Figure 6.4.2: The effect of the prior on the posterior for a coin-toss experiment (12 heads, 8 tails out of 20 flips). Each panel shows how different prior beliefs (blue) combine with the same likelihood (dashed) to produce different posteriors (red).

^aCheck out the optional section on conjugate priors for some extra info on the particular class of distribution chosen for our modeling here!

Priors as Regularization The connection between priors and the regularization techniques we saw earlier is one of the most elegant results in statistical modeling. Let’s reconsider our linear model fitting, but this time from a MAP perspective. The log of the posterior is

$$\log p(\boldsymbol{\theta}|\mathcal{D}) = \log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) + \text{constant} \quad (6.4.4)$$

Maximizing the posterior is equivalent to maximizing the log-likelihood plus the log-prior. Now, let’s assume a zero-mean Gaussian prior on our parameters, $\boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}/\beta)$. This prior encodes a belief that parameter values are more likely to be small than large. The log of this prior is

$$\log p(\boldsymbol{\theta}) = -\frac{\beta}{2} \boldsymbol{\theta}^\top \boldsymbol{\theta} + \text{constant} \quad (6.4.5)$$

Substituting this and the log-likelihood into the MAP objective gives

$$\boldsymbol{\theta}_{\text{MAP}}^* = \arg \max_{\boldsymbol{\theta}} \left(-\frac{1}{2} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) - \frac{\beta}{2} \boldsymbol{\theta}^\top \boldsymbol{\theta} \right) \quad (6.4.6)$$

Maximizing this is equivalent to minimizing its negative, so

$$\boldsymbol{\theta}_{\text{MAP}}^* = \arg \min_{\boldsymbol{\theta}} ((\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) + \beta \boldsymbol{\theta}^\top \boldsymbol{\theta}) \quad (6.4.7)$$

This is exactly the objective function for L2 regularization (ridge regression)! So we have a Bayesian interpretation for regularization: it is equivalent to MAP estimation with a prior belief that the parameters should be small and centered around zero. A similar result holds for L1 regularization using a Laplace prior.

6.5 The Chi-Square Statistic

We have established that for a linear model with Gaussian noise, the MLE is equivalent to the WLS solution. This procedure gives us the single “best” set of parameters, $\boldsymbol{\theta}^*$, that minimizes the discrepancy between our

model's predictions and the observed data. However, finding the best-fit parameters does not tell us if the fit is actually any good. The model might be fundamentally wrong (imagine fitting a line to a parabolic dataset), yet our optimization will still dutifully find the parameter values that make this bad model look as good as possible. We need a way to step back and ask a more fundamental question: Is the data consistent with the model itself, even with its optimal parameters? We have already encountered the necessary tool to answer this. When we assumed Gaussian noise to derive the MLE solution, we sought to minimize the quantity in the exponent of the likelihood function. This quantity is so fundamental to goodness-of-fit testing that it is given its own name: the **chi-square** (χ^2) statistic.

$$\chi^2 = (\mathbf{y} - \tilde{\mathbf{y}}(\mathbf{X}; \boldsymbol{\theta}))^\top \mathbf{V}_\epsilon^{-1} (\mathbf{y} - \tilde{\mathbf{y}}(\mathbf{X}; \boldsymbol{\theta})) \quad (6.5.1)$$

Here, $\tilde{\mathbf{y}}(\mathbf{X}; \boldsymbol{\theta})$ represents the model's predictions using a specific set of parameters $\boldsymbol{\theta}$. Finding the maximum likelihood estimate, $\boldsymbol{\theta}_{\text{MLE}}^*$, is therefore entirely equivalent to finding the parameters that *minimize* the chi-square value: $\boldsymbol{\theta}_{\text{MLE}}^* = \arg \min_{\boldsymbol{\theta}} \chi^2$.

The χ^2 value has an intuitive meaning. If the noise is uncorrelated so that the covariance matrix $\mathbf{V}_\epsilon$ is diagonal with entries $[\mathbf{V}_\epsilon]_{ii} = \sigma_i^2$, the expression simplifies to a sum:

$$\chi^2 = \sum_{i=1}^N \frac{(y_i - \tilde{y}(\mathbf{x}_i; \boldsymbol{\theta}))^2}{\sigma_i^2} \quad (6.5.2)$$

This is the sum of squared residuals where each residual is normalized by the variance of its corresponding measurement. It measures the discrepancy between the data and the model in units of the expected statistical error. If the model is a good fit, we would expect the typical deviation $(y_i - \tilde{y}_i)$ to be on the order of the standard deviation σ_i . Thus, each term in the sum should be roughly equal to one, and we would expect the total χ^2 value to be approximately equal to the number of degrees of freedom, $\nu = N - n$, where n is the number of fitted parameters.

Known vs. Estimated Error Variances

In this treatment, we assume the error covariance matrix $\mathbf{V}_\epsilon$ (or equivalently, the individual error variances σ_i^2) is known a priori from prior calibration, repeated measurements, or theoretical considerations. Under this assumption, the MLE/WLS estimate $\boldsymbol{\theta}_{\text{MLE}}^*$ minimizes χ^2 , and for linear models, $\chi^2(\boldsymbol{\theta}_{\text{MLE}}^*)$ follows a χ_{N-n}^2 distribution exactly. If instead the errors have an unknown common scale σ^2 (e.g., $\sigma_i^2 = \sigma^2 w_i$ with known weights w_i), then σ^2 must also be estimated from the data. In this case, the MLE simultaneously estimates both $\boldsymbol{\theta}$ and σ^2 . The distribution of the rescaled residuals changes: $\chi^2/\hat{\sigma}^2$ follows different distributions (related to Student's t and F distributions), and the expectation of χ_ν^2 is slightly below 1. The methods we develop here still apply, but interpretation of goodness-of-fit (discussed later on) requires care. Throughout this text, we assume $\mathbf{V}_\epsilon$ is known unless stated otherwise.

6.5.1 The Chi-Square Distribution and Hypothesis Testing

Much of the utility of the χ^2 statistic comes from recognizing that it can be treated as a random variable. If our model is indeed correct, and we were to repeat the experiment, we would obtain a new data vector $\mathbf{y}$ due to random noise, which in turn would yield a new, slightly different χ^2 value. The distribution of these possible χ^2 values, under the assumption that the model is correct, is known as the **chi-square distribution**. Its PDF (visualized in the leftmost panel of [Figure 6.5.1](#)) is given by

$$p_{\chi^2}(\chi^2; \nu) = \frac{1}{2^{\nu/2} \Gamma(\nu/2)} e^{-\chi^2/2} (\chi^2)^{(\nu/2)-1} \quad (6.5.3)$$

The chi-square distribution arises naturally in this context. Under our assumptions of Gaussian noise, each normalized residual $(y_i - \tilde{y}_i)/\sigma_i$ is a standard normal random variable. The χ^2 statistic is a sum of the squares of such variables (accounting for correlations through $\mathbf{V}_\epsilon^{-1}$). When these residuals are independent and we evaluate at the fitted parameters, this sum follows a chi-square distribution with ν degrees of freedom.

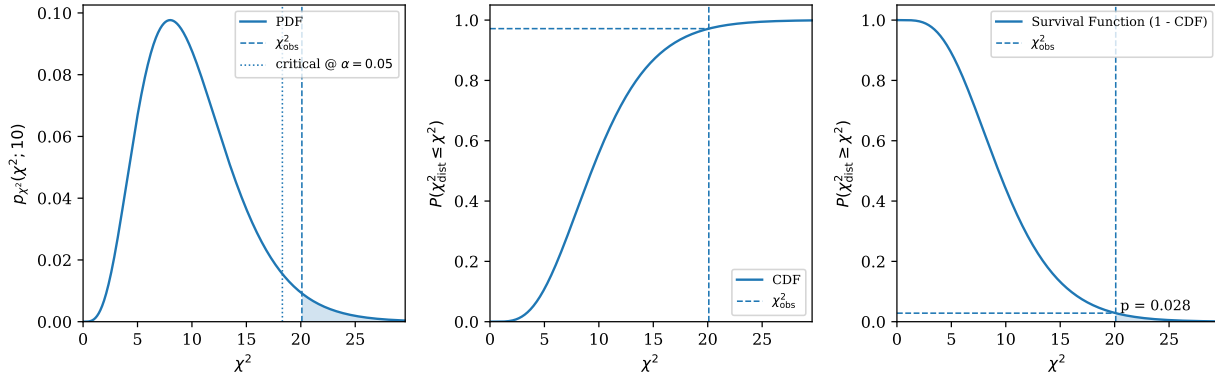


Figure 6.5.1: The chi-square distribution for $\nu = 10$ degrees of freedom. The PDF (left) shows the probability density for any given χ^2 value. The CDF (center) shows the cumulative probability $\Pr(\chi^2_{\text{dist}} \leq \chi^2)$. The survival function (right), or $1 - \text{CDF}$, directly gives the **upper-tail p-value** corresponding to an observed χ^2 value. The shaded region illustrates $p = \Pr(\chi^2_{10} \geq \chi^2_{\text{obs}})$ for a hypothetical observed value.

Warning: The χ^2 distribution is only valid under specific conditions: (1) the model structure is correct, (2) the measurement errors are truly Gaussian distributed, (3) the error estimates (σ_i or $\mathbf{V}_\epsilon$) are accurate, and (4) the errors are independent (or their correlations are correctly specified in $\mathbf{V}_\epsilon$). Violations of these assumptions can lead to unreliable p-values and incorrect conclusions about model fit.

Additionally, for linear models with known $\mathbf{V}_\epsilon$, the distribution $\chi^2(\boldsymbol{\theta}^*_{\text{MLE}}) \sim \chi^2_{N-n}$ is exact. For nonlinear models, this result is *asymptotic* (valid for large N). With small sample sizes or highly nonlinear models, the actual sampling distribution may deviate from χ^2_{N-n} , leading to unreliable p-values.

As mentioned, this distribution is defined by a single parameter, ν , called the degrees of freedom. The degrees of freedom represent the number of independent pieces of information available for assessing the model's fit. When evaluating χ^2 at the MLE parameters $\boldsymbol{\theta}^*$, it is calculated as the number of data points (N) minus the number of parameters (n) that were estimated from the data:

$$\nu = N - n \quad (6.5.4)$$

Intuitively, each time we use the data to estimate a parameter (like a slope or an intercept), we spend one degree of freedom. That parameter is now tuned to fit the data, so the data is no longer completely free to vary. The remaining degrees of freedom, ν , reflect how much the data is still free to challenge the model's overall structure.

This framework allows us to perform a formal hypothesis test. Our null hypothesis, H_0 , is that our model is a correct description of the data-generating process. We then ask, "Assuming our model is correct, what is the probability that random chance would produce a χ^2 value at least as large as the one we actually observed?" This probability is the **p-value**. A very small p-value implies that our observed data is highly unlikely under the model (i.e., the model is probably a poor fit).

To find the p-value, we compute the upper-tail probability: the probability that a χ^2 random variable with ν degrees of freedom would equal or exceed our observed value. This is $p = \Pr(\chi^2_\nu \geq \chi^2_{\text{obs}}) = 1 - \text{CDF}(\chi^2_{\text{obs}})$, which is shown as the survival function in the rightmost panel of Figure 6.5.1. Before running the experiment, we decide on a significance level, α , typically 0.05. If our calculated p-value is less than α , we reject the null hypothesis and conclude that the data provides statistically significant evidence against the model. For example, with $\nu = 10$ degrees of freedom, the critical value (upper $(1 - \alpha)$ quantile) corresponding to $\alpha = 0.05$ is $\chi^2_{\text{inv}}(0.95, 10) = 18.307$. This means there is only a 5% chance of observing $\chi^2 \geq 18.307$ if the model is correct. If our experiment yielded $\chi^2_{\text{obs}} = 20.1$, our p-value would be less than 0.05, and we would reject the model.

Warning: We should be careful to correctly interpret the p-value. If we calculate $p = 0.03$, it means there is a 3% chance of observing data at least this discrepant *if the model were true*. It does **not** mean there is a 3% chance the model is correct or a 97% chance the model is false. The p-value is a statement about the probability of the data conditional on the model, not the other way around.

The Reduced Chi-Square A useful rule of thumb in practice is the **reduced chi-square**, defined as $\chi^2_\nu = \chi^2/\nu$. For a linear model with known error covariance $\mathbf{V}_\epsilon$, we have $\mathbb{E}[\chi^2(\boldsymbol{\theta}^*_{\text{MLE}})] = \nu$, so we expect $\chi^2_\nu \approx 1$ for a good fit. A large value ($\chi^2_\nu \gg 1$) suggests a poor model fit: the discrepancy between the data and the model is significantly larger than what the measurement noise would justify. Conversely, a value much less than one ($\chi^2_\nu \ll 1$) is also problematic and can indicate several issues: the model may be overfitting the data (too many free parameters relative to the information content), the measurement errors (σ_i) have been systematically overestimated, or there may be correlations in the noise that have not been properly accounted for.

Notation: Chi-Square Quantiles Throughout this text, we use $\chi^2_{\text{inv}}(1 - \alpha, \nu)$ to denote the upper $(1 - \alpha)$ quantile of the chi-square distribution with ν degrees of freedom. This is the critical value c such that $\Pr(\chi^2_\nu \leq c) = 1 - \alpha$, or equivalently, $\Pr(\chi^2_\nu \geq c) = \alpha$. For example, $\chi^2_{\text{inv}}(0.95, 1) = 3.84$ means 95% of χ^2_1 values fall below 3.84, and $\chi^2_{\text{inv}}(0.95, 10) = 18.31$ means 95% of χ^2_{10} values fall below 18.31. (Be aware that some texts or papers use notation like $\chi^2_{\alpha, \nu}$ or $\chi^2_\nu(\alpha)$ for the same concept.)

6.6 Regions of Indifference

With the above chi-square goodness-of-fit test, we have laid the groundwork for defining a **region of indifference**, which is the set of all parameter vectors $\boldsymbol{\theta}$ that agree “well enough” with our data. The question of what constitutes “well enough” can be answered in two different ways, each with its own statistical interpretation and practical implications. Both approaches use the chi-square distribution but differ in their philosophical stance on what we’re testing.

Two Approaches to Confidence Regions

The Absolute Fit Approach: We might ask, “Which parameter values would produce a model that we would not reject as inconsistent with the data?” This leads us to define the confidence region as all parameter vectors $\boldsymbol{\theta}$ where the chi-square goodness-of-fit test does not reject the model. Formally, given a significance level α , this is

$$\{\boldsymbol{\theta} : \chi^2(\boldsymbol{\theta}) \leq \chi^2_{\text{inv}}(1 - \alpha, N - n)\} \quad (6.6.1)$$

where $N - n$ are the degrees of freedom from the goodness-of-fit test (N observations, n parameters). This approach is conservative and guarantees that any parameter vector in this region corresponds to an acceptable fit to the data. However, the size of this confidence region depends on how well the best-fit model performs: better-fitting models give larger confidence regions.

The Likelihood Ratio Approach: Alternatively, we might ask, “How much can the parameters deviate from their maximum likelihood estimates before the degradation in fit becomes statistically significant?” This perspective focuses on the *change* in fit quality rather than the absolute fit. Under this approach, the confidence region consists of parameter values where

$$\Delta\chi^2 = \chi^2(\boldsymbol{\theta}) - \chi^2(\boldsymbol{\theta}^*_{\text{MLE}}) \leq \chi^2_{\text{inv}}(1 - \alpha, n) \quad (6.6.2)$$

where now the degrees of freedom equal the number of parameters n . This is the standard method in likelihood-based inference^a. The confidence region size is independent of overall fit quality and depends only on the local curvature of the χ^2 surface near the minimum; this is a very nice property to have!

Both approaches are mathematically valid and define ellipsoidal confidence regions (in the linear approximation), but they answer different questions and are appropriate for different purposes. The

absolute fit approach is useful when we want to ensure any parameter set in our confidence region produces a model that passes a goodness-of-fit test. The likelihood ratio approach is the standard choice for quantifying parameter uncertainty in modern statistical practice, as it provides a direct connection to hypothesis testing via the likelihood ratio test and produces confidence intervals with the familiar interpretation independent of overall model fit quality. In what follows, we develop the theory using the absolute fit approach. The likelihood ratio approach follows identically by simply changing the threshold from $\chi_{\text{inv}}^2(1 - \alpha, N - n)$ to $\chi_{\text{inv}}^2(1 - \alpha, n)$ and measuring from $\chi^2(\boldsymbol{\theta}_{\text{MLE}}^*)$ rather than from zero.

^aThis approach comes from Wilks' theorem on the asymptotic distribution of the likelihood ratio statistic; check out the Wikipedia page on Wilks' theorem for more details.

For convenience, we define χ_{max}^2 as the threshold value in each approach: for the absolute fit approach, $\chi_{\text{max}}^2 = \chi_{\text{inv}}^2(1 - \alpha, N - n)$ and for the likelihood ratio approach, $\chi_{\text{max}}^2 = \chi^2(\boldsymbol{\theta}_{\text{MLE}}^*) + \chi_{\text{inv}}^2(1 - \alpha, n)$. As mentioned, both approaches lead to ellipsoidal confidence regions with identical local geometry and differ only in the threshold value used to define the boundary.

Important Prerequisite: Model Goodness-of-Fit

A prerequisite for constructing meaningful confidence regions is that the model must first pass a goodness-of-fit test. If $\chi^2(\boldsymbol{\theta}_{\text{MLE}}^*)$ itself is unacceptably large (p-value $< \alpha$ using $\nu = N - n$ degrees of freedom), then the model is fundamentally inconsistent with the data. In this case, constructing confidence regions for the model's parameters is not meaningful, and we should reject the model entirely, not attempt to quantify uncertainty in its parameters.

An important edge case arises when using the absolute fit approach: if $\chi^2(\boldsymbol{\theta}_{\text{MLE}}^*)$ exceeds $\chi_{\text{inv}}^2(1 - \alpha, N - n)$, the confidence region would be empty. This is a strong signal that even the most optimal parameterization of our model is statistically inconsistent with the data. Under the absolute fit criterion, there is also a subtle issue that as $\chi^2(\boldsymbol{\theta}_{\text{MLE}}^*)$ approaches but does not exceed $\chi_{\text{inv}}^2(1 - \alpha, N - n)$, the confidence region shrinks toward a single point. This means that marginally acceptable models (those that barely pass the goodness-of-fit test) have extremely small confidence regions, even though they satisfy the statistical criterion for model acceptance. This counterintuitive behavior is avoided in the likelihood ratio approach.

Assuming the model passes this basic sanity check of goodness-of-fit, we now want to characterize the *shape* and *size* of this confidence region near the best-fit point, $\boldsymbol{\theta}_{\text{MLE}}^*$. Regardless of which approach we choose (absolute fit or likelihood ratio), the local geometry of the confidence region is determined by the curvature of the χ^2 surface. To understand how χ^2 varies as we move a small distance away from the minimum, we use a Taylor series expansion:

$$\chi^2(\boldsymbol{\theta}) \approx \chi^2(\boldsymbol{\theta}_{\text{MLE}}^*) + (\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{MLE}}^*)^\top \left. \frac{\partial \chi^2}{\partial \boldsymbol{\theta}} \right|_{\boldsymbol{\theta}_{\text{MLE}}^*} + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{MLE}}^*)^\top \left. \frac{\partial^2 \chi^2}{\partial^2 \boldsymbol{\theta}} \right|_{\boldsymbol{\theta}_{\text{MLE}}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{MLE}}^*) + \dots \quad (6.6.3)$$

Since $\boldsymbol{\theta}_{\text{MLE}}^*$ is the minimum of the χ^2 surface, the first derivative (the gradient) is zero. Letting $\Delta \boldsymbol{\theta} = \boldsymbol{\theta} - \boldsymbol{\theta}_{\text{MLE}}^*$, the change in χ^2 is dominated by the second-order term involving the Hessian matrix (the matrix of second partial derivatives). For a linear model, this expansion is exact. For nonlinear models, this is an excellent approximation near the minimum, where higher-order terms are negligible. The Hessian of the χ^2 statistic can be calculated. For the general case with non-independent observations, where $\chi^2(\boldsymbol{\theta}) = \sum_{i=1}^N \sum_{j=1}^N (y_i - \tilde{y}(\mathbf{x}_i; \boldsymbol{\theta})) [\mathbf{V}_\epsilon^{-1}]_{ij} (y_j - \tilde{y}(\mathbf{x}_j; \boldsymbol{\theta}))$, the Hessian is approximately:

$$\frac{\partial^2 \chi^2}{\partial \theta_k \partial \theta_l} \approx 2 \sum_{i=1}^N \sum_{j=1}^N \left. \frac{\partial \tilde{y}}{\partial \theta_k} \right|_{\mathbf{x}=\mathbf{x}_i} [\mathbf{V}_\epsilon^{-1}]_{ij} \left. \frac{\partial \tilde{y}}{\partial \theta_l} \right|_{\mathbf{x}=\mathbf{x}_j} \quad (6.6.4)$$

The Gauss-Newton Approximation to the Hessian

The expression in Eq. 6.6.4 for the Hessian is an approximation known as the Gauss-Newton matrix. The complete second derivative of χ^2 includes an additional term:

$$\frac{\partial^2 \chi^2}{\partial \theta_k \partial \theta_l} = 2\mathbf{J}^\top \mathbf{V}_\epsilon^{-1} \mathbf{J} - 2 \sum_{i=1}^N \sum_{j=1}^N (y_i - \tilde{y}_i) [\mathbf{V}_\epsilon^{-1}]_{ij} \frac{\partial^2 \tilde{y}_j}{\partial \theta_k \partial \theta_l} \quad (6.6.5)$$

The second term involves both second derivatives of the model (which vanish for linear models) and the residuals $(y_i - \tilde{y}_i)$. This term is almost universally neglected in practice for several reasons. First, its expectation is zero: if the model is correct, the residuals are noise with zero mean, so this term averages to zero over many realizations of the data. Second, it is dominated by the first term near the MLE when residuals are small compared to the signal, a condition we expect if the fit is good. Third, it fluctuates wildly with random measurement noise, unlike the first term which depends only on the model structure. Finally, computing second derivatives of complex models is computationally prohibitive and numerically unstable.

The Gauss-Newton approximation is exact at the MLE for linear models (where $\partial^2 \tilde{y} / \partial \theta_k \partial \theta_l = 0$) and provides an excellent approximation for nonlinear models when residuals are small relative to the signal. This approximation is the foundation of virtually all practical parameter uncertainty quantification.

Eq. 6.6.4 introduces the Jacobian matrix, $\mathbf{J}$, which describes the model's sensitivity. Each element J_{jl} is the sensitivity of the j^{th} model prediction to a change in the l^{th} parameter: $J_{jl} = \partial \tilde{y}(\mathbf{x}_j; \boldsymbol{\theta}) / \partial \theta_l$. For the simple linear model $\tilde{y}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{x}^\top \boldsymbol{\theta}$, the Jacobian is simply the design matrix, $\mathbf{J} = \mathbf{X}$, and is independent of the data. Using this matrix notation, the Hessian can be written compactly as $\approx 2\mathbf{J}^\top \mathbf{V}_\epsilon^{-1} \mathbf{J}$. Substituting this back into our Taylor expansion, the region of indifference is described by the inequality:

$$\chi^2(\boldsymbol{\theta}_{MLE}^*) + \Delta \boldsymbol{\theta}^\top (\mathbf{J}^\top \mathbf{V}_\epsilon^{-1} \mathbf{J}) \Delta \boldsymbol{\theta} \leq \chi_{\max}^2 \quad (6.6.6)$$

This can be rearranged to define the confidence region for the parameter deviation $\Delta \boldsymbol{\theta}$:

$$\{\boldsymbol{\theta}_{MLE}^* + \Delta \boldsymbol{\theta} : \Delta \boldsymbol{\theta}^\top \underbrace{(\mathbf{J}^\top \mathbf{V}_\epsilon^{-1} \mathbf{J})}_{\mathbf{V}_\theta^{-1}} \Delta \boldsymbol{\theta} \leq \chi_{\max}^2 - \chi^2(\boldsymbol{\theta}_{MLE}^*)\} \quad (6.6.7)$$

The matrix at the heart of this expression is the inverse of the **parameter covariance matrix**, $\mathbf{V}_\theta$. This matrix,

$$\mathbf{V}_\theta = (\mathbf{J}^\top \mathbf{V}_\epsilon^{-1} \mathbf{J})^{-1} \quad (6.6.8)$$

is one of the most important results of the analysis, as it quantifies the uncertainties in and covariances among our estimated parameters. The diagonal elements of $\mathbf{V}_\theta$ are the variances of each parameter estimate, so the **standard errors** are $\sigma_{\theta_i} = \sqrt{[\mathbf{V}_\theta]_{ii}}$. These are the familiar “plus-or-minus” uncertainties reported for individual parameters. To construct a marginal confidence interval for a single parameter θ_i , we can project the full confidence ellipsoid onto the θ_i axis. This gives

$$|\theta_i - \theta_i^*| \leq \sqrt{c_\alpha^2} \sigma_{\theta_i} \quad (6.6.9)$$

where the scaling factor c_α^2 depends on which approach we use: for the absolute fit approach, $c_\alpha^2 = \chi_{\max}^2 - \chi^2(\boldsymbol{\theta}_{MLE}^*) = \chi_{\text{inv}}^2(1 - \alpha, N - n) - \chi^2(\boldsymbol{\theta}_{MLE}^*)$ and for the likelihood ratio approach, for a single parameter, $c_\alpha^2 = \chi_{\text{inv}}^2(1 - \alpha, 1)$. For example, in the likelihood ratio approach with $\alpha = 0.05$ (95% confidence), a single parameter has $c_\alpha^2 = \chi_{\text{inv}}^2(0.95, 1) \approx 3.84$, so $\sqrt{c_\alpha^2} \approx 1.96 \approx 2$, giving the familiar “ $\pm 2\sigma$ ” rule for 95% confidence intervals. (This rule is most reliable when the sample size is large enough for asymptotic approximations to hold.) The off-diagonal elements of $\mathbf{V}_\theta$ quantify correlations between parameter uncertainties. The inequality Eq. 6.6.7 describes the interior of an ellipse in 2D or a hyperellipsoid in higher dimensions (Figure 6.6.1).

The geometry of this ellipse tells us everything about our parameter uncertainties. If the matrix $\mathbf{V}_\theta$ is diagonal, its inverse is also diagonal, and the ellipse's axes are aligned with the parameter axes (left panel of

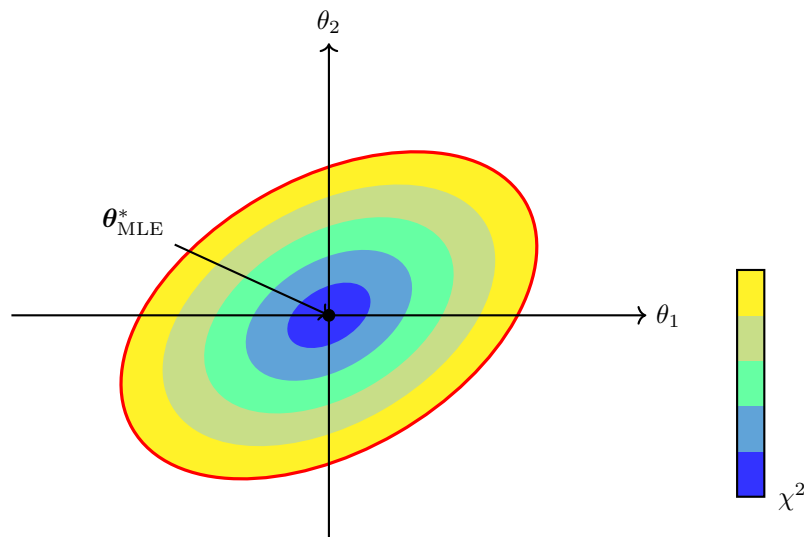


Figure 6.6.1: A hyperellipse confidence region in 2D, illustrating the region of indifference around the best-fit parameter estimate θ_{MLE}^* . The color scale indicates the χ^2 value, with the red boundary marking $\chi^2(\theta) = \chi_{\text{max}}^2$.

Figure 6.6.2). This means there is no correlation between the uncertainties in our parameters; an error in estimating θ_1 is independent of the error in estimating θ_2 .

In general, $\mathbf{V}_\theta$ is not diagonal. The off-diagonal terms represent the covariance between parameters, with the correlation coefficient between parameters i and j given by $\rho_{ij} = [\mathbf{V}_\theta]_{ij} / (\sigma_{\theta_i} \sigma_{\theta_j})$. This results in a tilted ellipse that means that the parameter uncertainties are correlated (right panel of Figure 6.6.2). The tilt shows that an overestimation of one parameter can be partially compensated for by an under- or overestimation of another. In other words, information about one parameter constrains the other. The overall volume of this uncertainty ellipse is proportional to $\sqrt{\det(\mathbf{V}_\theta)}$, which represents the total parameter uncertainty volume in n -dimensional space.

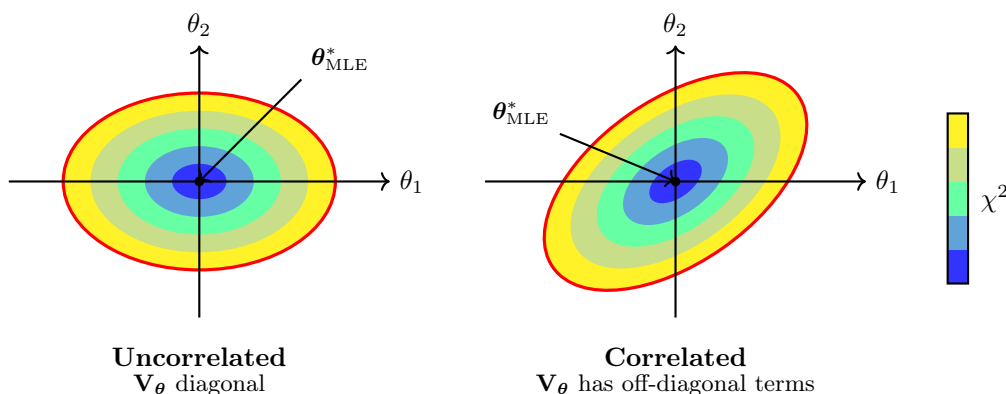


Figure 6.6.2: Left: An axis-aligned ellipse representing uncorrelated parameter uncertainties ($\mathbf{V}_\theta$ is diagonal). Right: A tilted ellipse representing correlated parameter uncertainties ($\mathbf{V}_\theta$ has off-diagonal elements). The red boundary in both cases marks $\chi^2(\theta) = \chi_{\text{max}}^2$.

To better analyze these correlations, we can diagonalize the inverse covariance matrix, $\mathbf{V}_\theta^{-1} = \mathbf{W}\mathbf{\Lambda}\mathbf{W}^\top$, where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues λ_i and $\mathbf{W}$ is the matrix of corresponding eigenvectors. The eigenvectors define the **principal axes** of the uncertainty ellipse (Figure 6.6.3), which are the directions in parameter space along which uncertainties are uncorrelated. By transforming our parameter deviations into

this natural coordinate system, $\mathbf{q} = \mathbf{W}^\top \Delta \boldsymbol{\theta}$, the inequality simplifies beautifully to

$$\sum_i \lambda_i q_i^2 \leq \chi_{\max}^2 - \chi^2(\boldsymbol{\theta}_{MLE}^*) \quad (6.6.10)$$

In this new basis, the parameter uncertainties are completely decoupled. The extent of the uncertainty region along each principal axis q_i is easily found: $q_{i,\max} = \sqrt{(\chi_{\max}^2 - \chi^2(\boldsymbol{\theta}_{MLE}^*)) / \lambda_i}$. Smaller eigenvalues of $\mathbf{V}_{\boldsymbol{\theta}}^{-1}$ (equivalently, larger eigenvalues of $\mathbf{V}_{\boldsymbol{\theta}}$) correspond to larger uncertainties along that direction; that is, the data provides weaker constraints in those parameter combinations. This eigenvalue decomposition gives us a way to understand and report the principal uncertainties of our model fit.

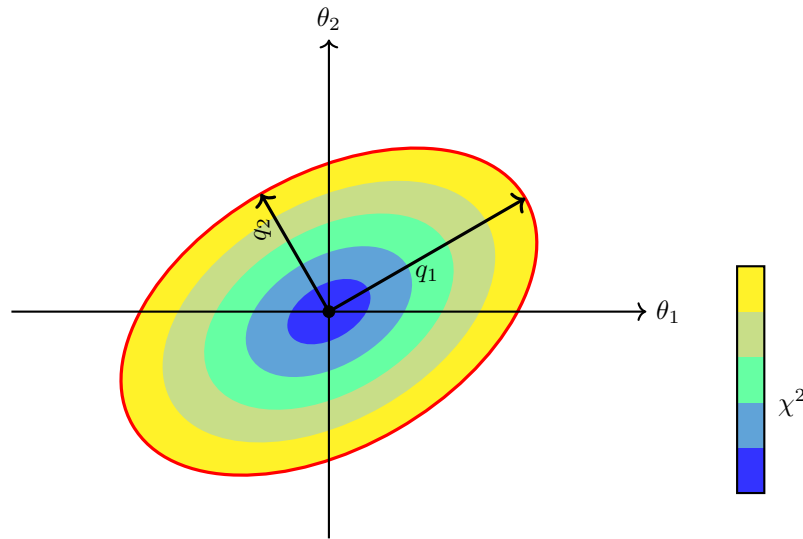


Figure 6.6.3: The confidence ellipse shown with its principal axes q_1 and q_2 , which are the eigenvectors of $\mathbf{V}_{\boldsymbol{\theta}}^{-1}$. In this rotated coordinate system, the parameter uncertainties are decoupled. The axis lengths are proportional to $1/\sqrt{\lambda_i}$, where λ_i are the eigenvalues of $\mathbf{V}_{\boldsymbol{\theta}}^{-1}$ (or equivalently, proportional to $\sqrt{\lambda'_i}$ where λ'_i are eigenvalues of $\mathbf{V}_{\boldsymbol{\theta}}$).

When the Ellipsoidal Approximation Breaks Down

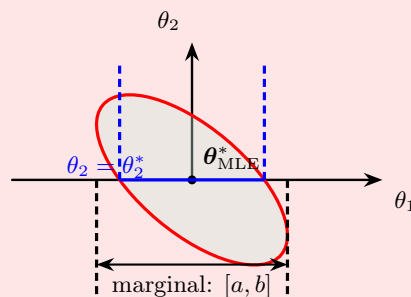
The quadratic approximation underlying the ellipsoidal confidence regions is excellent for linear models and near the MLE for well-behaved nonlinear models. However, it can fail in several situations: (1) highly nonlinear models, (2) small sample sizes, and (3) parameter bounds or constraints. There exists other methods like profile likelihood methods to capture these cases, but we will not cover them here. They incur additional computational expense.

Warning: Marginal Confidence Intervals and Parameter Correlations

When parameters are correlated, reporting uncertainties as $\theta_i = \theta_i^* \pm \Delta\theta_i$ can be misleading. Consider the tilted confidence ellipse shown to the right. If we project this ellipse onto the θ_1 axis, we obtain the interval $[a, b]$, which might tempt us to report

$$\theta_1 = \theta_1^* \pm \frac{b-a}{2}$$

However, this interval is only valid when we allow θ_2 to vary freely across all values consistent with the data. If we instead fix $\theta_2 = \theta_2^*$, the allowed range for θ_1 is much smaller, as shown by the horizontal slice through the ellipse.



The marginal uncertainty (obtained by projection) is always larger than or equal to the conditional uncertainty (obtained by slicing at fixed values of other parameters). When reporting $\theta_1 = \theta_1^* \pm \sqrt{c_\alpha^2 \sigma_{\theta_1}}$ as a confidence interval (or even just $\theta_1^* \pm \sigma_{\theta_1}$ as a standard error), we are implicitly reporting the marginal uncertainty. This marginal interval represents the range of θ_1 values consistent with the data when θ_2 is allowed to vary over all its plausible values. In contrast, if we *knew* the true value of θ_2 (which we don't), the uncertainty in θ_1 would be smaller, as shown by the conditional slice. The full covariance matrix $\mathbf{V}_\theta$ contains all the information about these correlations and should be reported alongside individual parameter uncertainties.

Example: Arrhenius Kinetics, Weighted Linear Regression, and Goodness-of-Fit

In chemical kinetics, the Arrhenius model says that

$$\ln k = \ln A - \frac{E_a}{R} \frac{1}{T} \quad (6.6.11)$$

so with $x_i = 1/T_i$ and $y_i = \ln k_i$, the model is linear: $y = \beta_0 + \beta_1 x$ with $\beta_0 = \ln A$ and $\beta_1 = -E_a/R$. Using the heteroscedastic Gaussian errors σ_i for $y_i = \ln k_i$, the WLS/MLE fit minimizes the same chi-square introduced above in Eq. 6.5.2,

$$\chi^2(\beta_0, \beta_1) = \sum_{i=1}^N \frac{(y_i - \beta_0 - \beta_1 x_i)^2}{\sigma_i^2} \quad (6.6.12)$$

and returns $\hat{\beta} = (\hat{\beta}_0, \hat{\beta}_1)$. For the ten temperatures shown in Fig. 6.6.4a, the optimum gives $\chi_{\text{obs}}^2 = 7.58$, and with $n = 2$ fitted parameters the degrees of freedom are

$$\nu = N - n = 8 \quad (6.6.13)$$

so the goodness-of-fit p-value is the upper tail

$$p = P(\chi_\nu^2 \geq \chi_{\text{obs}}^2) = P(\chi_8^2 \geq 7.58) = 0.476 \quad (6.6.14)$$

illustrated by the shaded survival area in Fig. 6.6.4b; the reduced statistic $\chi_\nu^2 = \chi_{\text{obs}}^2/\nu = 0.95$ is close to the “about one” rule-of-thumb, so the linear Arrhenius model is not rejected. To quantify parameter uncertainty, we use Eq. 6.6.8 to write the parameter covariance as

$$\mathbf{V}_\beta = (\mathbf{J}^\top \mathbf{V}_\epsilon^{-1} \mathbf{J})^{-1} \quad (6.6.15)$$

so the marginal 95% confidence intervals (likelihood-ratio view, $\Delta\chi^2 = 3.84$ for one parameter) are the familiar

$$\beta_0 = \hat{\beta}_0 \pm 1.96\sigma_{\beta_0}, \quad \beta_1 = \hat{\beta}_1 \pm 1.96\sigma_{\beta_1}, \quad (6.6.16)$$

which translate back to chemistry as $A = \exp(\beta_0)$ and $E_a = -R\beta_1$. Correlation between β_0 and β_1 is captured by the off-diagonal of $\mathbf{V}_\beta$, and the *joint* 95% region is the ellipse

$$(\beta - \hat{\beta})^\top \mathbf{V}_\beta^{-1} (\beta - \hat{\beta}) \leq 5.99 \quad (6.6.17)$$

shown in Fig. 6.6.4c. Hopefully it is clear how this example shows the flow of the theory above: (i) fit by minimizing χ^2 , (ii) judge absolute fit with the upper-tail p-value (survival function) at $\nu = N - n$, and (iii) read off marginal and joint parameter uncertainties from the Hessian geometry.

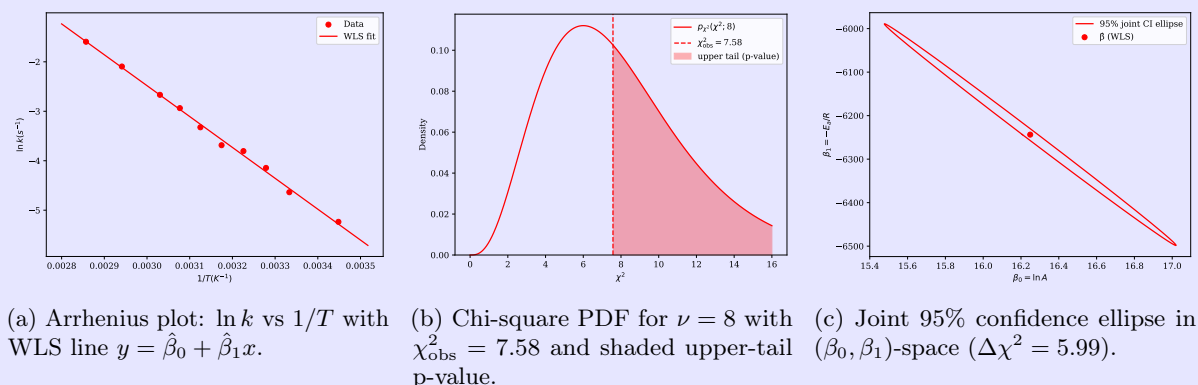


Figure 6.6.4: Arrhenius WLS analysis: data and fit [a](#), goodness-of-fit tail probability [b](#), and joint parameter uncertainty [c](#).

6.7 The Bayesian View of Uncertainty

The frequentist approach we have developed thus far treats the model parameters θ as unknown but fixed quantities. We estimate them from the data, quantify their uncertainties through the parameter covariance matrix $\mathbf{V}_\theta$, and construct confidence regions based on the chi-square distribution. This framework is powerful and widely used, but it represents only one philosophical perspective on the nature of uncertainty.

The Bayesian view offers a different interpretation. Rather than treating parameters as fixed unknowns, Bayesian inference regards the model parameters themselves as random variables that are described by a certain probability distribution. This perspective naturally incorporates our uncertainty about the parameters directly into the probabilistic framework. Before observing any data, we express our prior beliefs about the parameters through a prior distribution, $p(\theta)$. After observing data $\mathcal{D}$, we update these beliefs using Bayes' theorem to obtain the posterior distribution:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})} = \frac{p(\mathcal{D}|\theta)p(\theta)}{\int p(\mathcal{D}|\theta')p(\theta')d\theta'} \quad (6.7.1)$$

Here, $p(\mathcal{D}|\theta)$ is the likelihood function we encountered in the maximum likelihood framework, representing the probability of observing the data given a particular parameter value. The denominator, $p(\mathcal{D})$, is the evidence or marginal likelihood, which serves as a normalization constant ensuring that the posterior integrates to one. This normalization integral, which sums over all possible parameter values weighted by their prior probabilities and likelihoods, can be computationally challenging or even intractable, especially in high-dimensional parameter spaces.

The posterior distribution $p(\theta|\mathcal{D})$ encapsulates all our updated knowledge about the parameters after seeing the data. Unlike the frequentist approach, which provides a single point estimate (the MLE) plus uncertainty quantification through confidence intervals, the Bayesian framework maintains the full probability distribution over parameter space. Uncertainty in the model parameters is naturally reflected by the shape and spread of

this distribution. A narrow, concentrated posterior means that the data strongly constrains the parameters, while a broad posterior means significant remaining uncertainty even after observing the data.

To report parameter estimates in a format comparable to frequentist confidence intervals, Bayesian inference uses **credible intervals** (also called credible regions in higher dimensions). A credible interval Ω is defined such that the probability that the true parameter lies within this region, according to the posterior distribution, equals some specified value (typically 95%):

$$\Pr(\boldsymbol{\theta} \in \Omega) = \int_{\Omega} p(\boldsymbol{\theta}|\mathcal{D})d\boldsymbol{\theta} \quad (6.7.2)$$

The interpretation of a Bayesian credible interval is more intuitive than that of a frequentist confidence interval. A 95% credible interval means there is a 95% probability (given the data and our model assumptions) that the true parameter value lies within the interval. This is a direct probabilistic statement about the parameters themselves. In contrast, a frequentist 95% confidence interval means that if we repeated the experiment many times and constructed intervals each time using the same procedure, 95% of those intervals would contain the true parameter value—a more subtle distinction about the procedure rather than about this particular interval.

Any credible interval Ω can be used for this calculation, but conventionally we choose regions that contain the highest posterior density. Note that there can be multiple, disconnected highest posterior density regions when the posterior is multimodal. Finding or computing this region in more than one or two dimensions is itself a nontrivial numerical problem and/or ill-posed. In practice, this is usually done using samples from the posterior (e.g., via the Markov chain Monte Carlo methods covered later in the course, [chapter 7](#)) rather than any closed-form calculation. Just as with the frequentist framework, we can also marginalize the joint posterior distribution to isolate the probability distribution for individual parameters. If $\boldsymbol{\theta} = (\theta_1, \theta_2, \dots, \theta_n)$, the marginal posterior for parameter θ_i is obtained by integrating over all other parameters:

$$p(\theta_i|\mathcal{D}) = \int p(\boldsymbol{\theta}|\mathcal{D})d\boldsymbol{\theta}_{\neq i} \quad (6.7.3)$$

where $\boldsymbol{\theta}_{\neq i}$ denotes all parameters except θ_i . This marginalization naturally accounts for correlations between parameters and provides the full distribution of our beliefs about each parameter individually, from which we can construct marginal credible intervals.

6.8 Conjugate Priors*

While the Bayesian framework is conceptually elegant, a practical challenge immediately arises: how do we actually compute the posterior distribution? The normalization integral in the denominator of Bayes' theorem requires integrating the product of the likelihood and prior over the entire parameter space. For many combinations of likelihood functions and prior distributions, this integral has no closed-form analytical solution and must be evaluated numerically, often requiring computationally expensive Monte Carlo integration methods. Fortunately, there exists a special class of probability distributions called **exponential families** for which analytical solutions are possible when paired with appropriate priors. An exponential family is a probability distribution that can be written in the specific form:

$$p_{\mathbf{x}}(x|\boldsymbol{\theta}) = h(x) \exp[\boldsymbol{\eta} \cdot \mathbf{T}(x) - A(\boldsymbol{\eta})] \quad (6.8.1)$$

where $\boldsymbol{\eta}$ is the natural parameter vector (a function of $\boldsymbol{\theta}$), $\mathbf{T}(x)$ is the sufficient statistic, $h(x)$ is the base measure, and $A(\boldsymbol{\eta})$ is the log-partition function that ensures proper normalization. Many of the most commonly used probability distributions, including the normal (Gaussian), binomial, Poisson, exponential, gamma, and beta distributions, belong to exponential families.

For example, the normal distribution with mean μ and variance σ^2 can be written in exponential family form with natural parameters $\boldsymbol{\eta} = (\frac{\mu}{\sigma^2}, -\frac{1}{2\sigma^2})$, sufficient statistics $\mathbf{T}(x) = (x, x^2)$, and

$$h(x) = \frac{1}{\sqrt{2\pi}}, \quad A(\boldsymbol{\eta}) = -\frac{\eta_1^2}{4\eta_2} - \frac{1}{2} \log(-2\eta_2) \quad (6.8.2)$$

The key property of exponential families is that they possess **conjugate priors**, which are prior distributions that, when combined with the likelihood from the exponential family, give a posterior distribution in the same family as the prior. This closure property means that if the prior is in the conjugate family, the posterior can be computed in closed form simply by updating the parameters of that family. Conjugate priors make Bayesian inference analytically tractable and provide intuitive interpretations of how data updates our beliefs.

Example: The Beta-Binomial Conjugacy

Consider the problem of evaluating whether a coin is fair by observing N repeated tosses. The number of heads, k , follows a binomial distribution with unknown parameter p (the true probability of heads):

$$p_k(k; p) = \binom{N}{k} p^k (1-p)^{N-k} \quad (6.8.3)$$

This is our likelihood function. To perform Bayesian inference, we need to specify a prior distribution for the unknown parameter p . Since p must lie in the interval $[0, 1]$ and we want a flexible distribution that can represent various prior beliefs, the natural choice is the beta distribution, which is the conjugate prior for the binomial likelihood:

$$p_p(p; \alpha, \beta) = \frac{p^{\alpha-1} (1-p)^{\beta-1}}{B(\alpha, \beta)} \quad (6.8.4)$$

The parameters α and β control the shape of the prior. Setting $\alpha = \beta = 1$ yields a uniform prior (expressing complete ignorance), while larger values of α relative to β express a prior belief that the coin is biased toward heads.

After observing N_H heads and N_T tails (where $N = N_H + N_T$), we apply Bayes' theorem. The posterior is proportional to the product of the likelihood and prior:

$$p_{p|\mathcal{D}}(p|N_H, N_T) \propto p^{N_H} (1-p)^{N_T} \times p^{\alpha-1} (1-p)^{\beta-1} \propto p^{\alpha+N_H-1} (1-p)^{\beta+N_T-1} \quad (6.8.5)$$

Recognizing this functional form as another beta distribution, we can immediately write down the normalized posterior:

$$p_{p|\mathcal{D}}(p|N_H, N_T) = \frac{p^{\alpha+N_H-1} (1-p)^{\beta+N_T-1}}{B(\alpha+N_H, \beta+N_T)} \quad (6.8.6)$$

The posterior is a beta distribution with updated parameters $\alpha' = \alpha + N_H$ and $\beta' = \beta + N_T$. This elegant result shows that observing data simply adds the observed counts to the prior hyperparameters; the prior parameters α and β can be interpreted as “pseudo-counts” representing our prior belief expressed as if we had already observed $\alpha - 1$ heads and $\beta - 1$ tails before the experiment began.

Figure 6.8.1 illustrates several scenarios. When we start with a uniform prior ($\alpha = \beta = 1$) and observe balanced data ($N_H = N_T = 5$), the posterior is centered at $p = 0.5$ with moderate spread, reflecting the limited information from just ten tosses. With an informative prior ($\alpha = \beta = 5$) expressing a belief that the coin is fair, observing the same balanced data strengthens this belief, producing a narrower posterior. However, when we observe imbalanced data ($N_H = 8, N_T = 2$) with this same informative prior, we see the fundamental feature of Bayesian updating: the posterior represents a compromise between the prior belief and the evidence from the data. The likelihood alone would favor $p \approx 0.8$, but the prior pulls the posterior toward $p = 0.5$, resulting in a peak around $p \approx 0.65$.

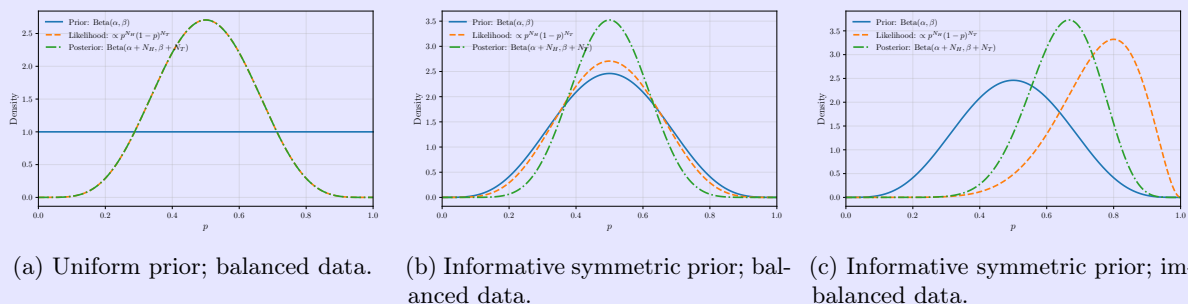


Figure 6.8.1: Bayesian inference for a coin flip experiment using beta–binomial conjugacy. Each panel shows the prior (blue), likelihood (orange), and posterior (yellow): (left) uniform prior with balanced data, (middle) informative symmetric prior with balanced data, (right) informative symmetric prior with imbalanced data showing how strong priors resist being updated by contradictory evidence.

An even more striking illustration appears when we use a very strong prior ($\alpha = \beta = 50$), shown in Figure 6.8.2. This prior encodes a strong belief that the coin is fair, equivalent to having already observed 98 prior tosses split evenly. Even when the actual data shows $N_H = 8$ and $N_T = 2$, suggesting bias, the posterior barely shifts from the prior. This demonstrates both the power and potential danger of strong priors: they allow us to incorporate genuine prior knowledge, but they also resist being updated by new evidence. The choice of prior is therefore not merely a technical detail but a substantive modeling decision that affects our conclusions.

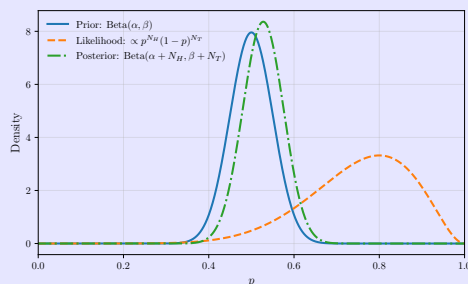


Figure 6.8.2: Effect of a strong prior ($\alpha = \beta = 50$) on Bayesian inference. Despite observing data ($N_H = 8$, $N_T = 2$) that suggests the coin is biased toward heads, the posterior (yellow) remains close to the prior (blue), demonstrating how strong prior beliefs resist updating even in the face of contradictory evidence.

Beyond the beta–binomial conjugacy, other important conjugate pairs exist throughout the exponential family. The normal distribution with known variance has a normal conjugate prior for its mean. The Poisson distribution has a gamma conjugate prior for its rate parameter. In general, exponential family distributions have conjugate priors that belong to a related exponential family, making them especially convenient for iterative Bayesian updating as more data arrives. When conjugate priors are not available or appropriate for the problem at hand, computational methods such as Markov chain Monte Carlo or variational inference must be employed to approximate the posterior distribution; we’ll learn more about the former later!

6.9 Prediction Uncertainty

Both the frequentist and Bayesian frameworks provide methods for quantifying uncertainty in model parameters, but we are often interested in a related question: how uncertain are we about predictions made by the model at new, unobserved points? When we use our fitted model to predict an outcome $\tilde{y}$ at a

new input location $\mathbf{x}$, the prediction itself is uncertain for two reasons. First, there is uncertainty in the parameters $\boldsymbol{\theta}$ used to make the prediction. Second, even if we knew the parameters perfectly, there is irreducible measurement noise ϵ that would cause the observed value to differ from the model's deterministic prediction.

In the frequentist framework, we use the parameter covariance matrix $\mathbf{V}_{\boldsymbol{\theta}}$ to propagate parameter uncertainty into prediction uncertainty. For a linear model $\tilde{y}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{x}^\top \boldsymbol{\theta}$, the prediction $\tilde{y}$ is itself a linear function of the random variables $\boldsymbol{\theta}$. If the parameter estimates have mean $\boldsymbol{\theta}_{\text{MLE}}^*$ and covariance $\mathbf{V}_{\boldsymbol{\theta}}$, then the prediction has mean $\mu_{\tilde{y}} = \mathbf{x}^\top \boldsymbol{\theta}_{\text{MLE}}^*$ and variance

$$\sigma_{\tilde{y}}^2 = \mathbf{x}^\top \mathbf{V}_{\boldsymbol{\theta}} \mathbf{x} \quad (6.9.1)$$

This variance quantifies only the contribution from parameter uncertainty. The total prediction uncertainty must also account for measurement noise. If the observations have error variance σ_ϵ^2 , the total variance of a new observation at $\mathbf{x}$ is

$$\sigma_{\text{total}}^2 = \sigma_{\tilde{y}}^2 + \sigma_\epsilon^2 = \mathbf{x}^\top \mathbf{V}_{\boldsymbol{\theta}} \mathbf{x} + \sigma_\epsilon^2 \quad (6.9.2)$$

For nonlinear models $\tilde{y}(\mathbf{x}; \boldsymbol{\theta})$, we cannot compute prediction uncertainty exactly, but we can use a first-order Taylor expansion around the MLE, just as we did for the confidence ellipse. The variance is approximately

$$\sigma_{\tilde{y}}^2 \approx \left. \frac{\partial \tilde{y}}{\partial \boldsymbol{\theta}} \right|_{\mathbf{x}}^\top \mathbf{V}_{\boldsymbol{\theta}} \left. \frac{\partial \tilde{y}}{\partial \boldsymbol{\theta}} \right|_{\mathbf{x}} \quad (6.9.3)$$

where the gradient is evaluated at the MLE and at the prediction point $\mathbf{x}$. This linearization approach provides a local approximation to the prediction uncertainty and is widely used in practice.

Example: Prediction Uncertainty in a Temperature Model

Consider a two-parameter model $\tilde{T}(r; \boldsymbol{\theta})$ that predicts temperature as a function of radius r , where $\boldsymbol{\theta} = (T_{\text{surf}}, \dot{q})$ represents the surface temperature and an internal heat generation parameter. Suppose the parameters have a diagonal covariance matrix (i.e., they are independent), so

$$\mathbf{V}_{\boldsymbol{\theta}} = \begin{bmatrix} \sigma_{T_{\text{surf}}}^2 & 0 \\ 0 & \sigma_{\dot{q}}^2 \end{bmatrix} \quad (6.9.4)$$

To predict the temperature at a new radius r with uncertainty, we first compute the sensitivity of the model to each parameter:

$$\sigma_{\tilde{T}}^2(r) = \left[\left. \frac{\partial \tilde{T}}{\partial T_{\text{surf}}} \right|_r, \left. \frac{\partial \tilde{T}}{\partial \dot{q}} \right|_r \right] \begin{bmatrix} \sigma_{T_{\text{surf}}}^2 & 0 \\ 0 & \sigma_{\dot{q}}^2 \end{bmatrix} \begin{bmatrix} \left. \frac{\partial \tilde{T}}{\partial T_{\text{surf}}} \right|_r \\ \left. \frac{\partial \tilde{T}}{\partial \dot{q}} \right|_r \end{bmatrix} \quad (6.9.5)$$

Expanding this expression:

$$\sigma_{\tilde{T}}(r) \sim \left[\left(\frac{\partial \tilde{T}(r)}{\partial T_{\text{surf}}} \sigma_{T_{\text{surf}}} \right)^2 + \left(\frac{\partial \tilde{T}(r)}{\partial \dot{q}} \sigma_{\dot{q}} \right)^2 \right]^{1/2} \quad (6.9.6)$$

This shows that the prediction uncertainty is a weighted combination of the individual parameter uncertainties, where the weights are the sensitivities of the prediction to each parameter. At locations where the model is highly sensitive to T_{surf} (such as near the surface), uncertainty in that parameter dominates the prediction uncertainty. Conversely, at locations where the model is more sensitive to $\dot{q}$, uncertainty in the heat generation parameter becomes more important.

The Bayesian approach to prediction uncertainty is conceptually more comprehensive. Rather than propagating uncertainty from a point estimate through a linearization, Bayesian prediction explicitly acknowledges that we do not know the true parameter values and instead have a probability distribution $p(\boldsymbol{\theta}|\mathcal{D})$ representing our beliefs. When making a prediction at a new point $\mathbf{x}$, we should account for all plausible parameter

values weighted by their posterior probabilities. The **posterior predictive distribution** is obtained by marginalizing over the parameter uncertainty:

$$p(y|\mathbf{x}, \mathcal{D}) = \int p(y|\mathbf{x}, \boldsymbol{\theta}') p(\boldsymbol{\theta}'|\mathcal{D}) d\boldsymbol{\theta}' \quad (6.9.7)$$

Here, $p(y|\mathbf{x}, \boldsymbol{\theta}')$ is the likelihood at the new point, or the probability of observing outcome y if the parameters were $\boldsymbol{\theta}'$, and $p(\boldsymbol{\theta}'|\mathcal{D})$ is the posterior distribution over parameters given the observed data. This integral effectively averages predictions across all possible parameter values, weighted by how probable each parameter value is given the data.

The posterior predictive distribution $p(y|\mathbf{x}, \mathcal{D})$ is the full probability distribution for the new observation, naturally incorporating both parameter uncertainty (through the integral over the posterior) and measurement noise (through the likelihood $p(y|\mathbf{x}, \boldsymbol{\theta}')$). From this distribution, we can extract point predictions (such as the mean or mode), prediction intervals, or any other quantity of interest. However, just as computing the posterior distribution required integrating the likelihood times the prior, computing the posterior predictive distribution requires another potentially difficult integral. When closed-form solutions are not available, Monte Carlo methods provide a practical computational approach: we draw samples $\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}, \dots, \boldsymbol{\theta}^{(M)}$ from the posterior $p(\boldsymbol{\theta}|\mathcal{D})$, make a prediction with each sampled parameter set, and the distribution of these predictions approximates the posterior predictive distribution.

6.10 Frequentist vs. Bayesian: A Summary

We have now developed two parallel frameworks for quantifying uncertainty in models and their predictions. It is worth pausing to compare and contrast these approaches, as each has its own strengths, limitations, and appropriate contexts for application.

In the frequentist view, which we developed through maximum likelihood estimation and chi-square analysis, model parameters are treated as unknown but fixed quantities. We summarize the likelihood function using the chi-square statistic, assuming Gaussian measurement errors, to assess the significance of discrepancies between model and data. The chi-square distribution provides a formal hypothesis testing framework: we can identify regions of indifference in parameter space in which the chi-square goodness-of-fit test does not reject the model. These regions form confidence ellipsoids around the maximum likelihood estimate. The size and shape of these ellipsoids are determined by the local curvature of the chi-square surface, encoded in the parameter covariance matrix $\mathbf{V}_{\boldsymbol{\theta}} = (\mathbf{J}^T \mathbf{V}_{\epsilon}^{-1} \mathbf{J})^{-1}$. For prediction uncertainty, we linearize the model at the MLE and propagate parameter uncertainties through the model's sensitivity derivatives, treating predictions as approximately normally distributed with variance $\mathbf{x}^T \mathbf{V}_{\boldsymbol{\theta}} \mathbf{x}$ plus measurement noise.

The frequentist approach has several notable advantages. It requires minimal assumptions beyond the error model since we do not need to specify prior beliefs about parameters. The central limit theorem provides a theoretical justification for the asymptotic normality of parameter estimates and the validity of the chi-square distribution for large sample sizes, even when the underlying errors are not exactly Gaussian. The computations are typically straightforward: find the MLE through optimization, compute the Hessian (or its Gauss-Newton approximation), and invert to obtain the covariance matrix. The framework provides well-calibrated hypothesis tests with clear significance levels and p-values. However, the frequentist approach has limitations. The interpretation of confidence intervals is subtle: they describe the long-run frequency properties of the procedure rather than making probability statements about this particular interval containing the true parameter. The local quadratic approximation underlying the confidence ellipse can break down for highly nonlinear models or when far from the MLE. The method provides no natural way to incorporate external information or constraints beyond what is in the current dataset. Perhaps most importantly, once we have the MLE and covariance matrix, we discard information about the shape of the full likelihood surface, which may contain important asymmetries or multiple modes.

The Bayesian framework gives us a different view. Parameters are treated as random variables described by probability distributions. We begin with a prior distribution $p(\boldsymbol{\theta})$ encoding our beliefs before seeing data, then update to a posterior distribution $p(\boldsymbol{\theta}|\mathcal{D})$ using Bayes' theorem. The posterior distribution represents

the complete state of our knowledge about the parameters after observing the data. We define credible intervals as regions where the probability (according to the posterior) that the true parameter lies within that region equals some specified value. The interpretation is direct and intuitive: a 95% credible interval means there is a 95% probability the parameter lies in that interval given the data and model. For predictions, we compute the posterior predictive distribution $p(y|\mathbf{x}, \mathcal{D})$ by marginalizing over parameter uncertainty, naturally accounting for our incomplete knowledge of the parameters.

The Bayesian framework's advantages include its intuitive probabilistic interpretation, its natural incorporation of prior information (from previous experiments, physical constraints, or expert knowledge), and its provision of the full posterior distribution rather than just a point estimate and covariance. It handles nonlinear models gracefully without relying on linearization approximations. Sequential updating is straightforward: today's posterior becomes tomorrow's prior when new data arrives. The framework provides a principled way to perform model comparison through Bayes factors or the evidence $p(\mathcal{D})$. Yet Bayesian inference also faces challenges. The choice of prior can be controversial or arbitrary, especially for problems where genuine prior information is lacking. The normalization integral in Bayes' theorem is often intractable, requiring sophisticated computational methods (MCMC, variational inference) that can be computationally expensive and slow to converge. Different priors can lead to different conclusions, particularly with limited data. The framework requires more assumptions (both the likelihood and the prior) and more computational machinery than frequentist approaches.

In practice, the two frameworks often produce similar results when data is abundant and informative. The frequentist confidence ellipse and the Bayesian credible region with a flat or weakly informative prior will often nearly coincide. The MLE and the MAP estimate converge as sample size increases and the prior becomes relatively less influential. However, the frameworks can diverge substantially in cases of limited data, strong prior information, or complex model structures. Many modern applications use hybrid approaches to get the computational efficiency of frequentist methods while still incorporating Bayesian ideas when prior information is valuable or when the full posterior distribution provides insights beyond point estimates.

6.11 Optimal Experimental Design

Before moving forward with more advanced modeling techniques, we should take a step back to think about why we run experiments in the first place. In some contexts, experiments serve a purely practical purpose. We might be physically manufacturing a molecule, synthesizing a new material, or producing a component where the experimental process itself is the end goal. However, more commonly in research settings, experiments are performed to obtain information. We run experiments to test hypotheses and assess whether data is consistent with theoretical predictions, to identify unknown parameters such as rate constants for reactive processes, to discriminate between competing candidate models that make different predictions, or to probe structure-function relationships in complex systems. Each of these objectives represents a different type of information-gathering task, and the experiments that are most valuable for one purpose may not be optimal for another.

This recognition leads naturally to the field of **optimal experimental design**, which provides formal frameworks for identifying the most informative or valuable experiments to perform given our objectives, constraints, and current state of knowledge. Rather than selecting experimental conditions arbitrarily or through trial and error, optimal design methods allow us to strategically choose where to sample, which variables to manipulate, and in what combinations in order to maximize the information gained per experiment. This is particularly important when experiments are expensive, time-consuming, dangerous, or limited by ethical considerations. The connection to our previous discussion of parameter uncertainty is very direct. As we saw before, identifying the maximum likelihood or maximum a posteriori estimate of parameters is a global optimization problem that may have many local minima, especially when we have a large number of parameters or when the model is highly nonlinear (such as those arising from ordinary differential equation initial value problem simulations). The choice of prior in Bayesian inference can lead to significant differences between MLE and MAP estimates. In both the frequentist and Bayesian views, we developed methods to quantify parametric uncertainty. Optimal experimental design takes these uncertainty quantification methods and inverts the question. Rather than asking how uncertain our parameters are given a particular dataset,

we ask which experiments we should perform to minimize this uncertainty.

6.11.1 Space-Filling Designs

When we do not have a trusted model structure, like when we lack confidence in assuming a simple linear relationship between inputs and outputs, a common approach is to use **space-filling designs**. These designs are agnostic to the specific model form and operate under the general premise that diversifying the inputs will lead to an informative distribution of outputs that can reveal the underlying system behavior. The philosophy is to spread experimental conditions throughout the input space so that whatever functional relationship exists between inputs and outputs, we will have sufficient coverage to detect it.

The simplest and perhaps most intuitive approach to experimentation is the **one-factor-at-a-time** (OFAT) strategy. In OFAT designs, we begin with a baseline set of conditions and then systematically vary each input factor individually while holding all others constant. For example, in a two-factor experiment with inputs x_1 and x_2 , we might start at some central point and then create a series of experiments that explore variations in x_1 alone, followed by another series exploring x_2 alone (Figure 6.11.1). This approach has intuitive appeal and is straightforward to implement and interpret. However, OFAT designs have a major limitation: they cannot reliably estimate interaction effects between variables. Consider a model of the form $y = \theta_1 x_1 + \theta_2 x_2 + \theta_3 (x_1 x_2) + \epsilon$, where the output depends not only on the individual effects of x_1 and x_2 but also on their product. If we vary x_1 while holding x_2 fixed, the apparent effect of x_1 depends on which value of x_2 we chose to fix. By not exploring combinations where both factors vary simultaneously, OFAT misses the synergistic or antagonistic interactions that are often crucial to understanding system behavior.

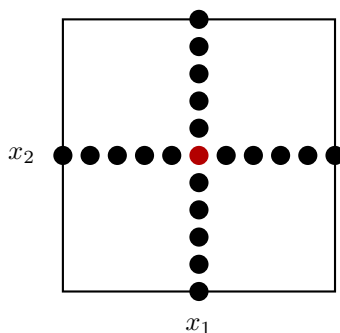


Figure 6.11.1: OFAT experimental design showing systematic variation of one factor at a time.

To address this limitation, **factorial designs** systematically explore combinations of input values. A **full factorial design** includes all possible combinations of input variables at a specified number of discrete levels. For instance, a 2^2 factorial design for two factors at two levels each (typically denoted as low and high, or -1 and $+1$) consists of four experimental conditions corresponding to the corners of a square in the input space (Figure 6.11.2a). Similarly, a 5^2 factorial design uses five levels for each of two factors, resulting in a grid of 25 experimental points (Figure 6.11.2b). Full factorial designs are comprehensive and allow for the estimation of all main effects and all interaction terms up to the highest order. However, the number of required experiments grows exponentially with the number of factors: a full factorial design for N factors at two levels each requires 2^N experiments, which quickly becomes impractical for systems with many inputs. We note that with only two levels per factor, quadratic (curvature) terms (e.g., x_i^2) are not estimable without adding center or axial points (e.g., central composite designs) or using three or more levels per factor.

When the dimensionality is high and full factorial designs become prohibitively expensive, **fractional factorial designs** offer a practical compromise. These designs use a strategically chosen subset of the full factorial combinations, sacrificing the ability to estimate certain higher-order interactions in exchange for a dramatic reduction in the number of experiments. For example, a 2^{3-1} fractional factorial design uses only four experiments instead of the eight required for a full 2^3 factorial (Figure 6.11.2c). A common 2^{3-1} design

might use the design matrix

$$\mathbf{X} = \begin{bmatrix} -1 & -1 & 1 \\ 1 & -1 & -1 \\ -1 & 1 & -1 \\ 1 & 1 & 1 \end{bmatrix} \quad (6.11.1)$$

representing the four vertices of a cube that are selected such that certain interaction effects are confounded (made indistinguishable) with main effects or with each other. The selection of which subset to use is guided by design theory that ensures main effects remain estimable while higher-order interactions (which are often less important) are aliased together. Fractional factorial designs are particularly valuable in screening experiments where the goal is to identify which of many potential factors significantly affect the output, with the understanding that more detailed exploration can follow.

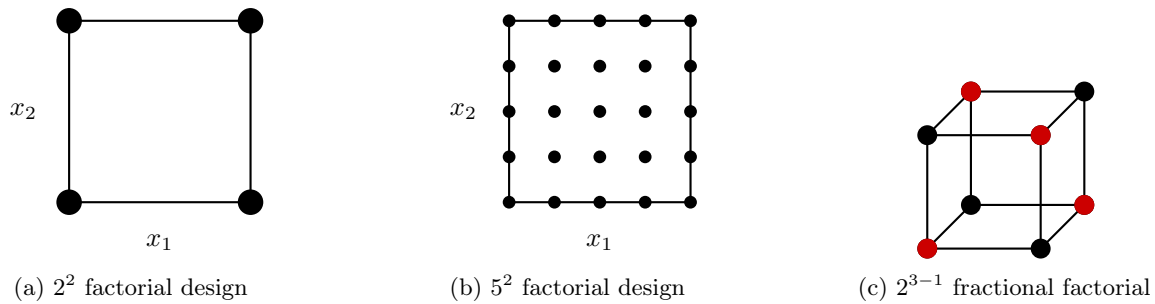


Figure 6.11.2: Factorial design configurations showing (a) full factorial with two factors at two levels, (b) full factorial with two factors at five levels, and (c) fractional factorial design highlighting a strategically chosen subset (red) of the full 2^3 factorial (black).

Beyond factorial designs, other space-filling approaches offer different advantages. **Box-Behnken designs** are subsets of 3^N factorial designs specifically constructed for fitting quadratic models (those including squared terms like x_i^2 in addition to linear and interaction terms). These designs place experimental points at the midpoints of edges and at the center of the design space, avoiding the corners (Figure 6.11.3). They can include repeated measurements at the center point. These repeated points allow for assessment of pure experimental error, which can then be distinguished from lack-of-fit when the model is inadequate.

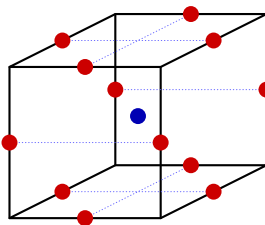


Figure 6.11.3: Box-Behnken design showing edge midpoints (red) and center point (blue) for a three-factor experiment.

Latin hypercube designs take a different approach and ensure that the marginal distribution of each input variable is uniform. Imagine dividing the range of each input into equally spaced intervals; a Latin hypercube design selects points such that each interval is sampled exactly once for each variable (Figure 6.11.4). This creates excellent space-filling properties while using relatively few points. The name derives from the mathematical concept of a Latin square extended to higher dimensions. Latin hypercube sampling is popular in computer experiments and uncertainty quantification where we can evaluate the model cheaply and want to explore a high-dimensional input space efficiently.

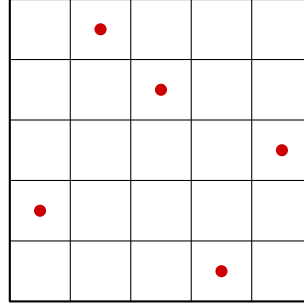


Figure 6.11.4: Latin hypercube design with five levels per factor, ensuring each row and column contains exactly one sample point.

6.11.2 Model-Based Experimental Design for Linear Models

While space-filling designs are valuable when we lack a trusted model, a more powerful approach becomes available when we do have a model structure in mind and want to design experiments specifically to estimate its parameters as accurately as possible. For linear models, we can develop this approach very cleanly. Recall from earlier that the parameter covariance matrix quantifying uncertainty in our estimated parameters is given by

$$\mathbf{V}_\theta = (\mathbf{J}^\top \mathbf{V}_\epsilon^{-1} \mathbf{J})^{-1} \quad (6.11.2)$$

where $\mathbf{J}$ is the Jacobian matrix (or sensitivity matrix) with elements $J_{ij} = \partial \tilde{y}_i / \partial \theta_j$ describing how each model prediction depends on each parameter. If we assume that measurements are independent with equal variances, so that $\mathbf{V}_\epsilon = \sigma^2 \mathbf{I}$, then for a linear model of the form $\tilde{y}(\mathbf{x}; \theta) = \mathbf{X}\theta$, the Jacobian is simply the design matrix itself: $\mathbf{J} = \mathbf{X}$. Therefore, the parameter covariance becomes

$$\mathbf{V}_\theta = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1} \quad (6.11.3)$$

So, in this case, the parametric uncertainty does not depend on the actual outcomes of the experiments, only on the input conditions we choose to explore! Before running any experiments, before collecting any data, we can predict exactly how uncertain our parameter estimates will be based solely on our choice of design matrix $\mathbf{X}$. This property enables us to optimize the experimental design in advance.

The matrix $\mathbf{X}^\top \mathbf{X}$ is known as the **Fisher information matrix**. It quantifies how much information about the parameters is contained in the chosen experimental design. A larger Fisher information matrix (in an appropriate matrix sense) corresponds to smaller parameter uncertainties. For nonlinear models, the sensitivity matrix $\mathbf{J}$ does depend on the parameter values, so we must evaluate it at some approximate parameter values $\theta \approx \theta_{\text{MLE}}^*$, perhaps from a pilot experiment or from prior knowledge. This introduces an iterative aspect to optimal design for nonlinear systems: we design experiments based on current parameter estimates, run those experiments to update the estimates, then redesign subsequent experiments based on the improved estimates.

To understand the importance of the Fisher information matrix, let's consider a simple example. Suppose we wish to fit a linear model with three parameters: $y = \theta_1 + \theta_2 x_1 + \theta_3 x_2 + \epsilon$. First of all, to see how this scalar equation translates to matrix form, consider a single experiment where we set the inputs to $x_1 = 2$ and $x_2 = 2$. For this experiment, the model becomes:

$$y = \theta_1 \cdot 1 + \theta_2 \cdot 2 + \theta_3 \cdot 2 + \epsilon = [1, 2, 2][\theta_1, \theta_2, \theta_3]^\top + \epsilon \quad (6.11.4)$$

If we run n experiments with different input values $(x_1^{(i)}, x_2^{(i)})$ for $i = 1, \dots, n$, stacking these rows gives the full matrix system:

$$\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots & \vdots \\ 1 & x_1^{(n)} & x_2^{(n)} \end{bmatrix} \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix} + \begin{bmatrix} \epsilon^{(1)} \\ \epsilon^{(2)} \\ \vdots \\ \epsilon^{(n)} \end{bmatrix} \quad (6.11.5)$$

The matrix in the middle is our design matrix $\mathbf{X}$. Now, imagine we unwisely choose to run four experiments where the two input variables always take the same value: experiment 1 with $(x_1, x_2) = (1, 1)$, experiment 2 with $(2, 2)$, experiment 3 with $(3, 3)$, and experiment 4 with $(4, 4)$. This gives the design matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 3 & 3 \\ 1 & 4 & 4 \end{bmatrix} \quad (6.11.6)$$

Computing the Fisher information matrix, we get

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 4 & 10 & 10 \\ 10 & 30 & 30 \\ 10 & 30 & 30 \end{bmatrix} \quad (6.11.7)$$

This matrix is singular. Conceptually, this occurs because we have no experiments where $x_1 \neq x_2$, so we cannot distinguish the effect of θ_2 from that of θ_3 . The model is fundamentally unidentifiable with this design. Mathematically, the least squares solution $\boldsymbol{\theta}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ requires inverting $\mathbf{X}^\top \mathbf{X}$, which is impossible when the matrix is singular. This is an extreme example of poor experimental design.

Even when we avoid complete singularity, poor designs can still lead to enormous parameter uncertainties. Consider a slight modification where we allow x_1 and x_2 to differ very slightly: experiments at $(x_1, x_2) = (1, 1)$, $(2, 2.01)$, $(3, 3.01)$, and $(4, 4.01)$:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2.01 \\ 1 & 3 & 3.01 \\ 1 & 4 & 4.01 \end{bmatrix} \quad (6.11.8)$$

Now the Fisher information matrix is technically invertible with

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 4 & 10 & 10.03 \\ 10 & 30 & 30.09 \\ 10.03 & 30.09 & 30.1803 \end{bmatrix} \quad (6.11.9)$$

but its inverse reveals severe problems:

$$\mathbf{V}_\theta \propto (\mathbf{X}^\top \mathbf{X})^{-1} \approx \begin{bmatrix} 1.5 & -0.5 & 6 \times 10^{-12} \\ -0.5 & 33534 & -33433 \\ 8 \times 10^{-12} & -33433 & 33333 \end{bmatrix} \quad (6.11.10)$$

The diagonal elements, which represent the variances of individual parameters, are enormous for θ_2 and θ_3 (uncertainties on the order of $\sqrt{33534}\sigma \approx 183\sigma$). Moreover, the off-diagonal terms tell us that there is extreme correlation between these parameters. Because we are still choosing experiments where x_1 and x_2 barely vary relative to each other, the data cannot reliably separate their individual effects. Near-singularity is nearly as problematic as exact singularity.

In contrast, consider a better design where the input variables vary independently. Using just three experiments at $(x_1, x_2) = (1, 1)$, $(0, 1)$, and $(1, 0)$, we get the design matrix

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} \quad (6.11.11)$$

with parameter covariance

$$\mathbf{V}_\theta = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1} = \begin{bmatrix} 3 & -2 & -2 \\ -2 & 2 & 1 \\ -2 & 1 & 2 \end{bmatrix} \sigma^2 \quad (6.11.12)$$

While there are still correlations (off-diagonal terms), the parameter variances (diagonal elements) are now on the order of σ^2 rather than tens of thousands of σ^2 . This is a great improvement.

Let's now consider replication. If we repeat each of the three experiments in the design above, stacking them to create a larger design matrix, we get

$$\mathbf{X}' = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_N \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ \vdots & \vdots & \vdots \\ 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} \quad (6.11.13)$$

the parameter covariance becomes

$$\mathbf{V}_\theta = \sigma^2 (\mathbf{X}'^\top \mathbf{X}')^{-1} = \frac{1}{N} \begin{bmatrix} 3 & -2 & -2 \\ -2 & 2 & 1 \\ -2 & 1 & 2 \end{bmatrix} \sigma^2 \quad (6.11.14)$$

where N is the number of replications. The parameter uncertainties scale as $1/\sqrt{N}$, following the familiar statistical principle that uncertainty decreases with the square root of sample size. Statistical replication (performing the same experiment multiple times) therefore helps identify sources of true systematic variation in the output without obfuscation by measurement noise. Replicates do not provide new information about different parts of the input space, but they do improve our confidence in the parameter estimates by averaging out random errors.

6.11.3 Optimality Criteria for Experimental Design

We have established that for linear models with uncorrelated, homoscedastic errors, the parameter covariance matrix $\mathbf{V}_\theta = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1}$ quantifies the uncertainty in our parameter estimates as a function of the experimental design $\mathbf{X}$. But how do we choose the best design? The challenge is that $\mathbf{V}_\theta$ is a matrix, not a scalar. We need to define what we mean by minimizing a matrix. Different objectives lead to different optimality criteria, each with its own interpretation and mathematical properties.

One natural approach is **A-optimality** (the “A” stands for “average”), which minimizes the trace of the covariance matrix:

$$\text{A-optimal: } \min_{\mathbf{X}} \text{trace}(\mathbf{V}_\theta) = \min_{\mathbf{X}} \sum_{i=1}^n [\mathbf{V}_\theta]_{ii} \quad (6.11.15)$$

The trace is the sum of the diagonal elements, which are the individual parameter variances. A-optimal designs therefore minimize the total variance of all parameters. They treat each parameter's uncertainty as equally important and ignore any correlations between parameters. This criterion is intuitive and computationally straightforward, so it is a popular choice in practice.

Perhaps the most widely used criterion is **D-optimality** (where “D” refers to “determinant”), which minimizes the determinant of the covariance matrix:

$$\text{D-optimal: } \min_{\mathbf{X}} \det(\mathbf{V}_\theta) = \min_{\mathbf{X}} \det \left[(\mathbf{X}^\top \mathbf{X})^{-1} \right] \equiv \max_{\mathbf{X}} \det (\mathbf{X}^\top \mathbf{X}) \quad (6.11.16)$$

The determinant of a matrix has a geometric interpretation as the volume of the parallelepiped defined by the matrix rows (or columns). In the context of parameter estimation, like we saw, $\sqrt{\det(\mathbf{V}_\theta)}$ is proportional to the hypervolume of the confidence ellipsoid in parameter space. Minimizing $\det(\mathbf{V}_\theta)$ therefore also minimizes this volume, making the region of indifference as small as possible and constraining the parameters most tightly. D-optimal designs have nice theoretical properties and are equivalent to maximizing the determinant of the Fisher information matrix $\det(\mathbf{X}^\top \mathbf{X})$, which explains why D-optimality is often described as maximizing the information content of the design.

To illustrate the impact of design choices on D-optimality, let's reconsider the 2^{3-1} fractional factorial design with design matrix

$$\mathbf{X} = \begin{bmatrix} -1 & -1 & 1 \\ 1 & -1 & -1 \\ -1 & 1 & -1 \\ 1 & 1 & 1 \end{bmatrix} \quad (6.11.17)$$

For this design, $\det(\mathbf{X}^\top \mathbf{X}) = 64$, so if $\sigma^2 = 1$, then $\det(\mathbf{V}_\theta) = 1/64$. Suppose we modify a single entry, changing the first element of the first column from -1 to $+1$:

$$\mathbf{X} = \begin{bmatrix} +1 & -1 & 1 \\ 1 & -1 & -1 \\ -1 & 1 & -1 \\ 1 & 1 & 1 \end{bmatrix} \quad (6.11.18)$$

This seemingly small change halves the determinant: $\det(\mathbf{X}^\top \mathbf{X}) = 32$, corresponding to $\det(\mathbf{V}_\theta) = 1/32$. Since the ellipsoid volume scales as $\sqrt{\det(\mathbf{V}_\theta)}$, this halving of the determinant increases the confidence ellipsoid volume by a factor of $\sqrt{2} \approx 1.41$; the design is now worse. Had we instead used the full 2^3 factorial design with eight experiments, we would achieve $\det(\mathbf{X}^\top \mathbf{X}) = 512$ (so $\det(\mathbf{V}_\theta) = 1/512$). The full design provides a $\sqrt{8} \approx 2.83$ times smaller ellipsoid volume (in the D-optimal sense) compared to the half-fraction, which is sensible given that it uses twice as many experiments. Figure 6.11.5 shows the three designs geometrically as subsets of the vertices of a cube.

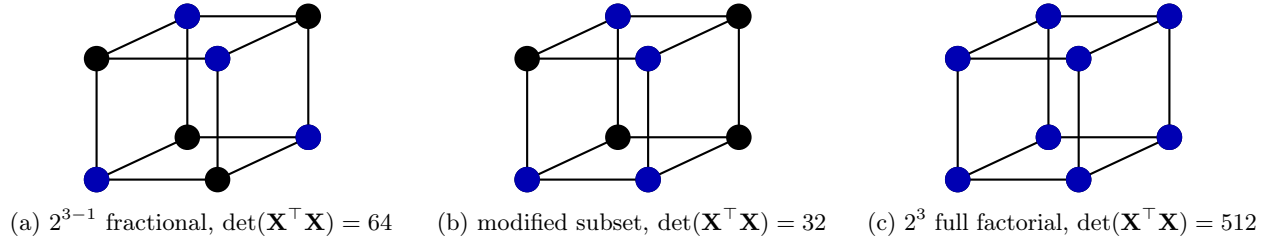


Figure 6.11.5: Comparison of D-optimality for three-factor designs. Blue points indicate the selected runs in each design.

Another criterion, **E-optimality** (for “eigenvalue”), takes a minimax approach:

$$\text{E-optimal: } \min_{\mathbf{X}} \max_i \lambda_i(\mathbf{V}_\theta) \quad (6.11.19)$$

where $\lambda_i(\mathbf{V}_\theta)$ denotes the i -th eigenvalue of the covariance matrix. We saw earlier that the eigenvalues of $\mathbf{V}_\theta$ determine the lengths of the principal axes of the confidence ellipsoid. The largest eigenvalue corresponds to the longest axis, the direction in parameter space along which uncertainty is greatest. E-optimal designs minimize this worst-case uncertainty so that even the most uncertain linear combination of parameters is as well-determined as possible. This criterion is appropriate when we care about all parameters equally and want to avoid situations where one particular parameter combination is poorly constrained even if others are well-determined.

While the criteria above focus on parameter uncertainty, we are often ultimately interested in prediction uncertainty. **V-optimality** (for “variance”) addresses this directly by minimizing the integrated prediction variance over a specified region of interest Ω in the input space:

$$\text{V-optimal: } \min_{\mathbf{X}} \int_{\Omega} \mathbf{x}^\top \mathbf{V}_\theta \mathbf{x} \, d\mathbf{x} \quad (6.11.20)$$

Here, $\mathbf{x}^\top \mathbf{V}_\theta \mathbf{x}$ is the prediction variance at input location $\mathbf{x}$. By integrating over the region Ω , we minimize the total prediction uncertainty where we want to make accurate predictions. This is useful when the goal is not parameter estimation per se but rather accurate interpolation or extrapolation in specific regions. The

region Ω might represent operating conditions of practical interest, a domain where we plan to apply the model, or simply the convex hull of the design space.

A related criterion is **G-optimality**, which takes a minimax approach to prediction:

$$\text{G-optimal: } \min_{\mathbf{X}} \max_{\mathbf{x}} \mathbf{x}^\top \mathbf{V}_{\boldsymbol{\theta}} \mathbf{x} \quad (6.11.21)$$

This minimizes the worst-case prediction variance, the largest prediction uncertainty at any point in the input space. G-optimal designs improve certainty at the least certain point so that predictions are as reliable as possible even in the most unfavorable location.

Modern computational tools have made optimal experimental design increasingly accessible. Software packages can numerically search for designs that optimize the chosen criterion subject to constraints. For example, the function `candexch` in MATLAB's Statistics Toolbox can select a D-optimal design from a user-provided candidate set of possible experiments and allows for constraints on which experimental conditions are feasible or allowable. The related function `cordexch` generates coordinate-exchange D-optimal designs without requiring a predefined candidate set and explores the continuous design space algorithmically. Similar tools exist in R, Python, and specialized design-of-experiments software. These computational methods handle the complexity of evaluating and optimizing the matrix-valued criteria discussed above.

6.12 Bayesian Optimal Experimental Design*

The optimal design criteria we developed in the previous section—A-, D-, E-, V-, and G-optimality—are all rooted in the frequentist framework. They operate by minimizing various functions of the parameter covariance matrix $\mathbf{V}_{\boldsymbol{\theta}}$, treating parameters as unknown but fixed quantities. However, we can also approach optimal experimental design from the Bayesian perspective, where parameters are themselves random variables characterized by probability distributions. This Bayesian approach to experimental design offers additional flexibility and a natural framework for incorporating prior information, though at the cost of increased computational complexity.

The main construct in Bayesian experimental design is a utility function $U(\mathbf{X})$ that quantifies how useful a particular experimental design $\mathbf{X}$ will be for our inferential goals. The challenge is that before we actually run the experiments, we do not know what outcomes $\mathbf{y}$ we will observe. The usefulness of a design depends on what we learn from the data, but the data are not yet available. To resolve this, we define the expected utility by averaging over all possible experimental outcomes, weighted by their probabilities:

$$U(\mathbf{X}) = \mathbb{E}_{\mathbf{y}}[U(\mathbf{X}, \mathbf{y})] = \int_{\mathbf{y}} U(\mathbf{X}, \mathbf{y}) p(\mathbf{y}|\mathbf{X}) d\mathbf{y} \quad (6.12.1)$$

Here, $p(\mathbf{y}|\mathbf{X})$ is the prior predictive distribution. This is the probability distribution of potential outcomes given the design choice $\mathbf{X}$ before actually observing the data. This distribution itself depends on our prior beliefs about the parameters. Through the law of total probability (marginalization over the parameter distribution), we have

$$p(\mathbf{y}|\mathbf{X}) = \int_{\boldsymbol{\theta}} p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta} \quad (6.12.2)$$

The prior predictive distribution accounts for our uncertainty about which parameter values are true by averaging the likelihood $p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta})$ over the prior distribution $p(\boldsymbol{\theta})$.

One elegant choice for the utility function is based on information theory. We can ask: which experiments will provide the most information about the parameters in the sense of producing the largest change from our prior beliefs to our posterior beliefs? A natural measure of this change is the KL divergence (we introduced this concept in the optional probability section on distances between distributions) between the posterior distribution $p(\boldsymbol{\theta}|\mathbf{X}, \mathbf{y})$ and the prior distribution $p(\boldsymbol{\theta})$. For a given outcome $\mathbf{y}$, the KL divergence is

$$U_{\text{KL}}(\mathbf{X}, \mathbf{y}) = \int_{\boldsymbol{\theta}'} p(\boldsymbol{\theta}'|\mathbf{X}, \mathbf{y}) \log \left[\frac{p(\boldsymbol{\theta}'|\mathbf{X}, \mathbf{y})}{p(\boldsymbol{\theta}')} \right] d\boldsymbol{\theta}' \quad (6.12.3)$$

This quantity is always non-negative and equals zero only when the posterior is identical to the prior (meaning the data provided no information). To find the expected utility, we average this KL divergence over possible outcomes:

$$\begin{aligned}
 U(\mathbf{X}) &= \int_{\boldsymbol{\theta}'} \int_{\mathbf{y}} p(\boldsymbol{\theta}' | \mathbf{X}, \mathbf{y}) p(\mathbf{y} | \mathbf{X}) \log \left[\frac{p(\boldsymbol{\theta}' | \mathbf{X}, \mathbf{y})}{p(\boldsymbol{\theta}')} \right] d\mathbf{y} d\boldsymbol{\theta}' \\
 &= \int_{\boldsymbol{\theta}'} \int_{\mathbf{y}} p(\boldsymbol{\theta}') p(\mathbf{y} | \mathbf{X}, \boldsymbol{\theta}') \log \left[\frac{p(\boldsymbol{\theta}' | \mathbf{X}, \mathbf{y})}{p(\boldsymbol{\theta}')} \right] d\mathbf{y} d\boldsymbol{\theta}' \\
 &= \int_{\boldsymbol{\theta}'} \int_{\mathbf{y}} p(\boldsymbol{\theta}') p(\mathbf{y} | \mathbf{X}, \boldsymbol{\theta}') \log \left[\frac{p(\mathbf{y} | \mathbf{X}, \boldsymbol{\theta}')}{p(\mathbf{y} | \mathbf{X})} \right] d\mathbf{y} d\boldsymbol{\theta}'
 \end{aligned} \tag{6.12.4}$$

where we have used Bayes' theorem to rewrite the logarithm in the final line. This quantity is known as the mutual information between the parameters $\boldsymbol{\theta}$ and the data $\mathbf{y}$ given the design $\mathbf{X}$. Maximizing this mutual information corresponds to selecting experiments where the anticipated data will provide the largest update to our prior beliefs and therefore the most information about the parameters.

The mathematical framework for Bayesian experimental design reveals why it is considerably more complex than frequentist design. The utility function may depend on the posterior distribution, which in turn depends on the experimental outcomes, which depend on the unknown parameters, which are themselves treated as random variables. This creates a nested hierarchy of probability distributions and expectations that must be computed, often requiring multiple layers of integration over high-dimensional spaces. The situation is further complicated by the many possible choices for utility functions. In addition to the information-theoretic approach based on KL divergence described above, practitioners have developed Bayesian analogs of the frequentist alphabet criteria (D-, E-, G-optimality) that operate on the posterior distribution or posterior covariance of parameters. Other information-theoretic utilities include minimizing the (expected) Shannon entropy of the posterior or minimizing posterior variance in specific parameters or predictions of interest.

6.12.1 Model Comparison and the Bayes Factor*

Beyond parameter estimation, experimental design can also be oriented toward the goal of discriminating between competing models or hypotheses. In the Bayesian framework, model comparison is performed using the Bayes factor, which quantifies how much the observed data shift our relative beliefs between two models.

Consider two competing models or hypotheses, denoted $\mathcal{M}_0$ and $\mathcal{M}_1$. These might represent entirely different functional forms (e.g., a linear versus a quadratic model), or they might represent different ranges of parameter values within the same model structure (e.g., testing whether a rate constant is below or above a critical threshold). The Bayes factor is defined as the ratio of marginal likelihoods (evidence) for the two models:

$$B_{01}(\mathcal{D}) = \frac{p(\mathcal{D} | \mathcal{M}_0)}{p(\mathcal{D} | \mathcal{M}_1)} = \frac{\int_{\boldsymbol{\theta}} p(\boldsymbol{\theta} | \mathcal{M}_0) p(\mathcal{D} | \boldsymbol{\theta}, \mathcal{M}_0) d\boldsymbol{\theta}}{\int_{\boldsymbol{\theta}} p(\boldsymbol{\theta} | \mathcal{M}_1) p(\mathcal{D} | \boldsymbol{\theta}, \mathcal{M}_1) d\boldsymbol{\theta}} \tag{6.12.5}$$

Each marginal likelihood integrates the likelihood function over the prior distribution of parameters within that model. The Bayes factor represents the factor by which the data increase or decrease our relative beliefs about these two hypotheses. Through Bayes' theorem, the relationship between prior and posterior odds is given by

$$\frac{p(\mathcal{M}_0 | \mathcal{D})}{p(\mathcal{M}_1 | \mathcal{D})} = B_{01}(\mathcal{D}) \times \frac{p(\mathcal{M}_0)}{p(\mathcal{M}_1)} \tag{6.12.6}$$

In words: the posterior odds equal the Bayes factor times the prior odds. A Bayes factor greater than one means that the data favor $\mathcal{M}_0$ over $\mathcal{M}_1$, while values less than one favor $\mathcal{M}_1$.

For experimental design aimed at model discrimination, we can define utility functions based on the expected Bayes factor. Since we do not yet know what data will be observed, we compute the expectation of the Bayes factor (or some function thereof) by integrating or sampling over possible outcomes according to the prior predictive distribution. Experiments that are expected to produce large magnitude Bayes factors, regardless of which direction they point, are those most likely to decisively discriminate between the competing models. This framework generalizes to multiple competing models and connects to a broader landscape of information criteria for model selection, but a detailed treatment of these methods is beyond our scope here.

6.13 Sequential Experimental Design*

All of the experimental design approaches we have discussed so far assume that we plan the entire set of experiments at the outset before collecting any data. However, a more adaptive strategy is possible. This is sequential experimental design, where we plan experiments iteratively using the information gained from earlier experiments to inform the design of subsequent ones.

For linear models with known error structure, sequential design offers no advantage from a purely information-theoretic standpoint. The parameter covariance matrix $\mathbf{V}_\theta = \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$ depends only on the design matrix $\mathbf{X}$, not on the outcomes. Whether we choose all experiments simultaneously or adaptively, as long as we end up with the same total design matrix, the parameter uncertainties will be identical. However, for nonlinear models and in the Bayesian framework, sequential design becomes genuinely beneficial.

In the frequentist case with nonlinear models, recall that the sensitivity matrix $\mathbf{J}$ depends on the parameter values at which it is evaluated. When designing the first set of experiments, we must evaluate $\mathbf{J}$ at some initial parameter guess, which may be quite uncertain. After running these experiments and obtaining the maximum likelihood estimate θ_{MLE}^* , we can re-evaluate the sensitivity matrix at this improved estimate and design the next batch of experiments accordingly. This iterative refinement continues: each round of experiments reduces parameter uncertainty, which improves our estimate of where the sensitivities are largest, which informs better designs for the next round. Over successive iterations, the design becomes increasingly well-matched to the true system behavior.

In the Bayesian framework, sequential design is very natural. After conducting an initial set of experiments and observing the data $\mathcal{D}_1$, we compute the posterior distribution $p(\theta|\mathcal{D}_1)$. When designing the next experiment, this posterior becomes our new prior: $p(\theta)_{\text{new}} = p(\theta|\mathcal{D}_1)$. We optimize the utility function using this updated prior distribution to select the next experimental condition. After observing the new data $\mathcal{D}_2$, we update again to obtain $p(\theta|\mathcal{D}_1, \mathcal{D}_2)$, which becomes the prior for the third experiment, and so on. Each experiment refines our beliefs, and we (hopefully) use our refined beliefs to ask better questions. Over time, as data accumulate, the posterior distribution concentrates around the true parameter values, and the model becomes increasingly accurate in its predictions.

One important practical consideration is batch size. Rather than designing one experiment at a time (which may be inefficient, especially when experiments can be run in parallel), we often want to select batches of multiple experiments to conduct simultaneously. The optimal batching strategy depends on the specific utility function and computational constraints. Some utility functions naturally extend to batch selection by identifying sets of experiments that collectively maximize information gain, while others are more naturally suited to greedy, one-at-a-time selection. How to efficiently select optimal batches of multiple experiments remains an active area of research; this connects to topics in machine learning such as batch active learning and optimal sensor placement. These advanced techniques are increasingly important in modern applications where high-throughput experimentation enables the simultaneous execution of dozens or hundreds of experiments and careful design can dramatically accelerate the rate of discovery.

6.14 Exercises

Exercise 6.1

Suppose I set a p -value threshold for significance at $p = 0.01$. If I regress my 3-parameter model to 20 data points, what is the $\nu(\text{DOF})$ in the χ^2 distribution I expect? What is the largest χ_{max}^2 that would still look consistent with the data? (You'll need to use MATLAB.)

Exercise 6.2

Suppose we have planned 2 experiments out of a total of 3 with the goal of estimating θ for $y = \theta_1 x_1 + \theta_2 x_2 + \epsilon$. What conditions should we choose for the third experiment if we want it to be A-optimal? Set $x_1 = 0$,

$$x_2 = \alpha \in [0, 1].$$

$$\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 0 & 0.5 \\ 0 & \alpha \end{bmatrix}.$$

Chapter 7

Monte Carlo Methods

In the previous section, we developed methods for parameter estimation and uncertainty quantification. The Bayesian approach gives us a posterior distribution $p(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})$ over the parameter space. We saw that we can find point estimates like the MAP, but many practical tasks require computing integrals over this entire distribution. These types of integrals appear everywhere in scientific computing. In Bayesian inference, normalizing the posterior requires computing

$$Z = p(\mathcal{D}) = \int p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}) d\boldsymbol{\theta} \quad (7.0.1)$$

and extracting information about specific parameters requires marginalizing over the others:

$$p(\theta_1, \theta_2|\mathcal{D}) = \int p(\boldsymbol{\theta}|\mathcal{D}) d\theta_3 d\theta_4 \cdots d\theta_n \quad (7.0.2)$$

In statistical mechanics, thermodynamic quantities like the partition function $Z = \int \exp(-U(\mathbf{q})/kT) d\mathbf{q}$ and ensemble averages $\langle U \rangle = \int U(\mathbf{q})p(\mathbf{q}) d\mathbf{q}$ have the same mathematical form. In quantum chemistry, computing molecular properties often involves integrating over electronic configurations. In chemical kinetics, calculating reaction rates may require integrating over phase space. In polymer physics, determining conformational properties involves sums over chain configurations. The common thread here is that we need to evaluate integrals (or sums) of the form

$$\mathbb{E}_{\mathbf{x} \sim p}[f(\mathbf{x})] = \int f(\mathbf{x}) p(\mathbf{x}) d\mathbf{x} \quad (7.0.3)$$

where $\mathbf{x}$ lives in a high-dimensional space and $p(\mathbf{x})$ is some probability distribution. The problem is that traditional numerical integration methods break down when the dimension gets large. This failure has a special name: the curse of dimensionality. To circumvent this problem, we will develop a new class of methods called Monte Carlo methods.

7.1 The Curse of Dimensionality

To understand why Monte Carlo methods are necessary, we need to see why traditional numerical integration fails in high dimensions. Let's start with a concrete example. Suppose we want to compute a typical high-dimensional integral:

$$I = \int_0^1 \int_0^1 \int_0^1 \cdots \int_0^1 f(x_1, x_2, x_3, \dots, x_N) dx_1 dx_2 dx_3 \cdots dx_N \quad (7.1.1)$$

This is an N -dimensional integral over the unit hypercube. How would we evaluate it numerically? The most basic approach uses the trapezoidal rule, in which we create a grid of points in each dimension and evaluate the function at these grid points. If we use just two points per dimension (the minimum for the trapezoidal

rule), how many function evaluations do we need? In one dimension, we need 2 points. In two dimensions, we need $2 \times 2 = 4$ points. In three dimensions, $2 \times 2 \times 2 = 8$ points. For N dimensions, we need 2^N function evaluations; that is, there is an exponential scaling of the number of function evaluations with dimension!

But it gets worse. Suppose we're not satisfied with the accuracy of our calculation and want to improve it by an order of magnitude. For the trapezoidal rule, the error scales as $(\Delta x)^2$, where Δx is the grid spacing. To reduce the error by a factor of 10, we need to reduce Δx by a factor of $\sqrt{10}$, which means using about 3.16 times more points in each direction. In N dimensions, this requires $(\sqrt{10})^N = 10^{N/2}$ times more function evaluations. Let's put this in perspective. Suppose we're working with a modest 10-dimensional problem and want to improve our accuracy by a single order of magnitude. We would need $10^{10/2} = 10^5 = 100,000$ times more function evaluations. If each evaluation takes a millisecond, we've gone from nearly instant to highly inconvenient on typical hardware. For 20 dimensions, the situation gets completely out of hand. The baseline calculation with just two points per dimension already requires $2^{20} \approx 10^6$ evaluations, taking about one second at a microsecond per evaluation. To improve accuracy by a single order of magnitude, we need $10^{20/2} = 10^{10}$ times more evaluations. This transforms our one-second calculation into one requiring 10^{10} seconds, or roughly 300 years.

This exponential explosion makes traditional numerical integration completely impractical for almost all of the problems we actually care about. The examples we discussed earlier all suffer from this curse. Consider the posterior distribution from parameter estimation:

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{1}{Z} (2\pi)^{-m/2} |\mathbf{V}_\varepsilon|^{-1/2} \exp\left(-\frac{\chi^2(\boldsymbol{\theta})}{2}\right) p(\boldsymbol{\theta}) \quad (7.1.2)$$

where m is the number of data points. To compute the normalization constant Z , we must integrate this function over all parameters $\boldsymbol{\theta}$. If we have 20 parameters and want even modest accuracy, we're already in the realm of impossibility with traditional methods. Or consider marginalization. To find the joint distribution of just two parameters, we must integrate over all the others:

$$p(\theta_1, \theta_2|\mathcal{D}) = \int p(\boldsymbol{\theta}|\mathcal{D}) d\theta_3 d\theta_4 \cdots d\theta_n \quad (7.1.3)$$

If we have 50 parameters and want to examine the relationship between two of them, we face a 48-dimensional integral. With traditional methods, this single marginalization is computationally infeasible. Yet in a complete Bayesian analysis, we typically want to examine all pairwise relationships, which requires hundreds or thousands of such integrals. The resulting corner plots (Figure 7.1.1), which display all marginal and joint distributions, would be impossible to compute.

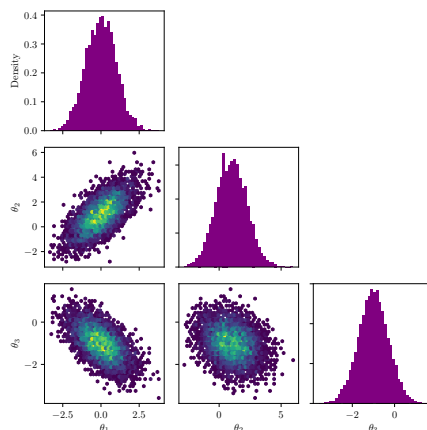


Figure 7.1.1: A corner plot showing marginal and joint distributions from a Bayesian analysis. Each marginal distribution requires integrating over all other parameters, which is a high-dimensional integration problem.

By now, the point should be clear that we need to compute high-dimensional integrals to do useful science, but traditional numerical methods fail miserably as dimension increases. This is where Monte Carlo integration

offers a way forward. As we will see, unlike grid-based methods, Monte Carlo approaches do not suffer from exponential scaling with dimension. Their convergence rate depends only on the number of samples drawn, not on the dimension of the space, which allows them to be used in high-dimensional problems.

7.2 Monte Carlo Integration

To see the path forward, we must realize that any integral can be written as the size of the integration domain multiplied by the average value of the function over that domain. In particular, consider an integral over some domain Ω in N -dimensional space:

$$I = \int_{\Omega} f(\mathbf{x}) d\mathbf{x}, \quad \mathbf{x} \in \mathbb{R}^N \quad (7.2.1)$$

We can always rewrite this as

$$I = \left(\int_{\Omega} 1 d\mathbf{x} \right) \langle f \rangle \quad (7.2.2)$$

where $\langle f \rangle$ is the average value of f over the domain. The first term is just the volume of Ω . The second term is what we need to estimate. To make this precise, we introduce a probability distribution $p(\mathbf{x})$ that is uniform over Ω . This means $p(\mathbf{x}) = 1/V$ for $\mathbf{x} \in \Omega$ and $p(\mathbf{x}) = 0$ otherwise, where $V = \int_{\Omega} 1 d\mathbf{x}$ is the volume of the domain. We can then write the average value as an expectation:

$$\langle f \rangle = \mathbb{E}[f(\mathbf{x})] = \int_{\Omega} f(\mathbf{x}) p(\mathbf{x}) d\mathbf{x} \quad (7.2.3)$$

Since $p(\mathbf{x}) = 1/V$ is constant over Ω , this gives

$$\langle f \rangle = \frac{1}{V} \int_{\Omega} f(\mathbf{x}) d\mathbf{x} \quad (7.2.4)$$

and therefore $I = V \langle f \rangle$. The Monte Carlo method estimates this expectation using random sampling. We generate a sequence of M points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_M$ that are uniformly distributed within Ω . By the law of large numbers, the sample average converges to the true mean as the number of samples increases:

$$\langle f \rangle = \lim_{M \rightarrow \infty} \frac{1}{M} \sum_{i=1}^M f(\mathbf{x}_i) \quad (7.2.5)$$

For a finite number of samples, we estimate the average as

$$\langle f \rangle_M = \frac{1}{M} \sum_{i=1}^M f(\mathbf{x}_i) \quad (7.2.6)$$

and our Monte Carlo estimate of the integral is

$$I_{\text{MC}} = V \langle f \rangle_M = \frac{V}{M} \sum_{i=1}^M f(\mathbf{x}_i) \quad (7.2.7)$$

The power of this approach lies in its error scaling. For independent and identically distributed samples, the variance of the sample mean is σ^2/M , where σ^2 is the variance of the individual samples. The central limit theorem additionally tells us that this estimator is asymptotically normal. Practically, this means that if you take M random samples, the typical size of the error in I_{MC} shrinks like $M^{-1/2}$:

$$I_{\text{MC}} - I \sim M^{-1/2} \quad (7.2.8)$$

Convergence Rate Comparison: Let's compare the computational effort required to improve a calculation's accuracy by a factor of 10 using different methods. Like we discussed before, for standard grid-based integration like the trapezoidal rule, the error typically scales with the grid spacing Δx as $\sim (\Delta x)^2$. To reduce the error by 10, one must decrease Δx by $\sqrt{10} \approx 3.16$. In N dimensions, this requires $10^{N/2}$ times more function evaluations. In contrast, the error in the Monte Carlo method scales with the number of samples M as $\sim M^{-1/2}$. To reduce its error by a factor of 10, one simply needs to increase the samples by $10^2 = 100$ times, *regardless of the problem's dimension*.

This dimensional independence is Monte Carlo's main advantage. For a 10-dimensional problem, a grid-based method would need $10^{10/2} = 10^5$ (or 100,000) times more evaluations to achieve the 10x accuracy boost. Monte Carlo, however, would still only need 100 times more samples. If the problem had 20 dimensions, the grid-based method's cost would explode to 10^{10} (ten billion) times more evaluations, while Monte Carlo's cost remains fixed at 100 times more samples.

This dimension-independent convergence rate tames the curse of dimensionality. In low dimensions, carefully designed grid methods can improve faster with each added function evaluation. But as dimension grows, grid methods require exponentially many evaluations to keep shrinking the grid spacing, while Monte Carlo's $M^{-1/2}$ improvement stays the same no matter the dimension. That is why high-dimensional integrals that are intractable with grids become feasible (though still sometimes costly) with Monte Carlo.

It is worth being precise about exactly what this dimension-independence means. Like big-O notation, which hides a constant prefactor while capturing only the dominant scaling exponent (recall the discussion of computational complexity earlier in the notes), Monte Carlo's $M^{-1/2}$ rate is dimension-independent only in the sense that the *exponent* on M does not change with N . The constant multiplying that rate typically grows substantially with dimension for realistic problems, which is why designing a good sampling or proposal distribution is still an essential consideration high-dimensional Monte Carlo, even if we have $M^{-1/2}$ scaling by construction.

Let's work through a classic example that demonstrates the method. We'll compute the area of a circle with diameter 2 (radius 1). This is a problem where we know the exact answer (π), so we can check our result. The domain Ω is the square $-1 \leq x_1, x_2 \leq 1$, which has area $V = 4$. We want to integrate the indicator function

$$f(\mathbf{x}) = H(1 - (x_1^2 + x_2^2)) \quad (7.2.9)$$

where H is the Heaviside step function: $H(z) = 1$ if $z > 0$ and $H(z) = 0$ otherwise. This function equals 1 inside the circle and 0 outside. The integral is

$$I = \int_{\Omega} f(\mathbf{x}) d\mathbf{x} = (\text{area of circle}) = \pi \quad (7.2.10)$$

To estimate this with Monte Carlo, we generate M random points uniformly distributed in the square and compute the fraction that fall inside the circle:

$$\langle f \rangle_M = \frac{1}{M} \sum_{i=1}^M f(\mathbf{x}_i) = \frac{\text{number of points inside circle}}{M} \quad (7.2.11)$$

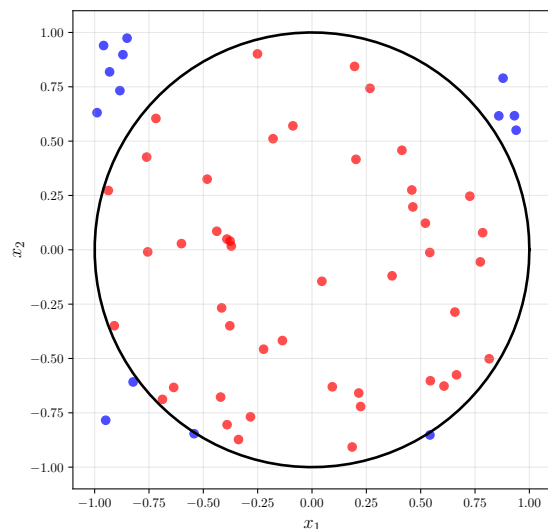
Our estimate is then

$$I_{\text{MC}} = V \langle f \rangle_M = 4 \times \frac{\text{number of points inside circle}}{M} \quad (7.2.12)$$

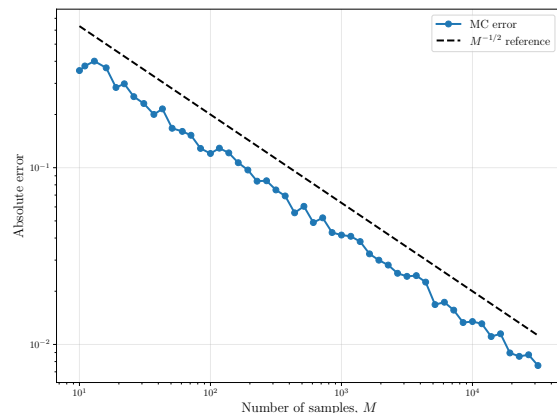
Here's a simple implementation in MATLAB:

```
M = 100;
x = 2 * rand(M, 2) - 1;           % uniform in [-1,1]^2
f = (x(:, 1).^2 + x(:, 2).^2) < 1; % inside circle? (logical)
fs = mean(f);                     % sample average
I = 4 * fs;                       % scale by domain size, and we have our estimate
```

Running this code (and visualizing the results), we get something like in Figure 7.2.1a. With $M = 100$ samples, the expected relative error is around 5%. With $M = 10,000$ samples, the error drops to around 0.5% (a factor of 10 improvement in accuracy from 100 times more samples, as predicted by the $M^{-1/2}$ scaling). The convergence of the estimate to the true value (by changing M in the code above) is shown in Figure 7.2.1b.



(a) Monte Carlo estimation of the area of a circle. Random points are uniformly sampled in the enclosing square. Points inside the circle (red) contribute 1 to the integral, points outside (blue) contribute 0.



(b) Convergence of Monte Carlo integration for estimating the area of a circle. The error decreases as $M^{-1/2}$, so improving accuracy by a factor of 10 requires 100 times more samples. The dimension-independence of this scaling is what makes Monte Carlo practical for high-dimensional problems.

Figure 7.2.1: Monte Carlo integration methods.

While Monte Carlo integration breaks the curse of dimensionality, the uniform-sampling approach we've described has some practical limitations. First, when using uniform sampling over a domain Ω , we need to know the volume V of that domain to compute $I = V\langle f \rangle$. For simple domains like hypercubes this is trivial, but for arbitrary domains it may be difficult, and if the domain is the entire space $\mathbb{R}^N$, uniform sampling makes no sense.¹ Second, even when the domain is finite, uniform sampling can be extremely inefficient. If the function $f(\mathbf{x})$ has large values only in a small region of the domain (imagine two sharp peaks separated by a vast expanse where $f \approx 0$, as in Figure 7.2.2) then most of our random samples will land in regions that contribute almost nothing to the integral. We're wasting computational effort evaluating the function where it doesn't matter. Both of these problems have the same solution. Instead of sampling uniformly, we sample from a different probability distribution that concentrates samples where they matter most. This is the idea behind importance sampling.

¹Note: If we already have a normalized probability density $p(\mathbf{x})$ and want to compute $\mathbb{E}_p[f] = \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x}$, we can sample directly from p and simply average $f(\mathbf{x}_i)$, and no volume factor is needed. This is common in Bayesian applications where we sample from the posterior. In the more common practical case, we only have an unnormalized weight $w(\mathbf{x}) \propto p(\mathbf{x})$ —exactly the situation exploited later in this chapter for MCMC. There, we draw samples $\mathbf{x}_i$ from a tractable proposal $q(\mathbf{x})$ and form ratios $r(\mathbf{x}_i) = w(\mathbf{x}_i)/q(\mathbf{x}_i)$, but since we don't know the true normalizing constant of p , we “self-normalize” by dividing by the sum of the weights instead:

$$\mathbb{E}_p[f] \approx \frac{\sum_{i=1}^M r(\mathbf{x}_i)f(\mathbf{x}_i)}{\sum_{i=1}^M r(\mathbf{x}_i)}.$$

Unlike the vanilla importance-sampling estimator derived above, which is unbiased, this ratio estimator is *biased* for any finite M (the bias is $O(1/M)$, vanishing as $M \rightarrow \infty$). This distinction is worth remembering if you later encounter particle filters or sequential Monte Carlo, both of which rely on self-normalized weights.

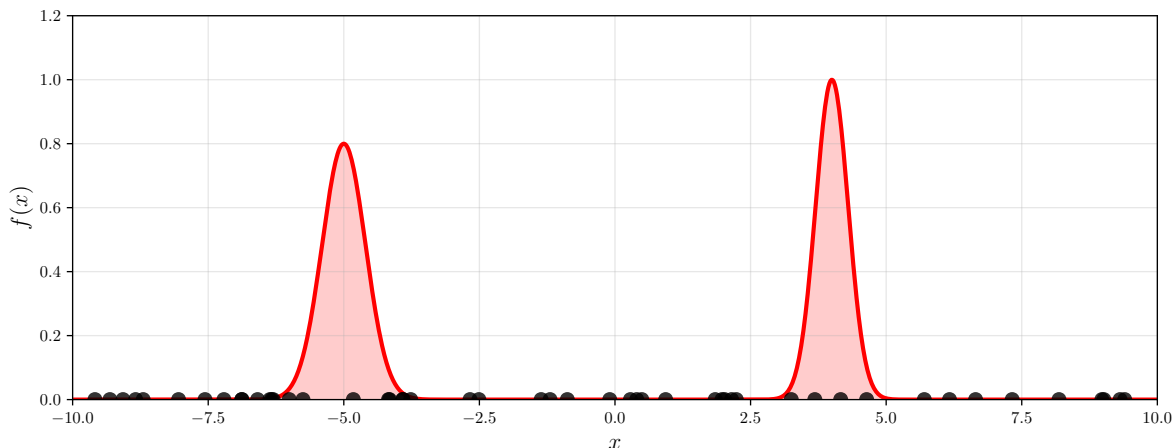


Figure 7.2.2: The inefficiency of uniform sampling when the integrand has certain characteristics like sharp peaks. Most samples (black dots on the x -axis) land in regions where $f(x) \approx 0$, contributing little to the integral estimate. We would get much better accuracy by sampling preferentially near the peaks.

7.3 Importance Sampling

Basic Monte Carlo with uniform sampling works, but it's inefficient when the integrand has structure that we can exploit. Like we mentioned, if $f(\mathbf{x})$ has large values only in small regions of the domain, most uniform samples land where the function is nearly zero and contribute little to the integral, so we should try to sample more near the peaks. Fortunately, we can write any integral as an expectation over more general probability distributions than just the uniform distribution. To see this, we start with the same integral from before, namely

$$I = \int_{\Omega} f(\mathbf{x}) d\mathbf{x} \quad (7.3.1)$$

and multiply and divide by any probability distribution $p(\mathbf{x})$ that is nonzero wherever $f(\mathbf{x})$ is nonzero:

$$I = \int_{\Omega} \frac{f(\mathbf{x})}{p(\mathbf{x})} p(\mathbf{x}) d\mathbf{x} = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} \left[\frac{f(\mathbf{x})}{p(\mathbf{x})} \right] \quad (7.3.2)$$

This is an expectation of the weighted function $f(\mathbf{x})/p(\mathbf{x})$ with respect to the distribution $p(\mathbf{x})$. We can estimate this expectation using Monte Carlo. Generate M samples $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_M$ from the distribution $p(\mathbf{x})$ (not uniformly!), and compute

$$I \approx \frac{1}{M} \sum_{i=1}^M \frac{f(\mathbf{x}_i)}{p(\mathbf{x}_i)} \quad (7.3.3)$$

This is importance sampling. Each sample is weighted by $1/p(\mathbf{x}_i)$ to correct for the fact that we're sampling from p rather than uniformly.

Warning: It is important to remember that expectation values of functions of random variables depend on the distribution of those random variables. When we write $\mathbb{E}[g(\mathbf{x})]$, we must always be clear about what distribution $\mathbf{x}$ is drawn from. The importance sampling formula works precisely because we're computing the expectation of $f(\mathbf{x})/p(\mathbf{x})$ with $\mathbf{x}$ drawn from $p(\mathbf{x})$.

The obvious question is which distribution $p(\mathbf{x})$ we should choose. We want to minimize the variance of our estimator so that our integral estimate is as reliable as possible. To minimize the estimator's variance, it is enough to minimize the per-sample variance $\text{Var}[f(\mathbf{x})/p(\mathbf{x})]$, because the variance of our Monte Carlo

estimator scales as

$$\text{Var} \left[\frac{1}{M} \sum_{i=1}^M \frac{f(\mathbf{x}_i)}{p(\mathbf{x}_i)} \right] = \frac{1}{M} \text{Var} \left[\frac{f(\mathbf{x})}{p(\mathbf{x})} \right] \quad (7.3.4)$$

The per-sample variance is

$$\text{Var} \left[\frac{f(\mathbf{x})}{p(\mathbf{x})} \right] = \mathbb{E} \left[\left(\frac{f(\mathbf{x})}{p(\mathbf{x})} \right)^2 \right] - \left(\mathbb{E} \left[\frac{f(\mathbf{x})}{p(\mathbf{x})} \right] \right)^2 = \mathbb{E} \left[\left(\frac{f(\mathbf{x})}{p(\mathbf{x})} \right)^2 \right] - I^2 \quad (7.3.5)$$

where both expectations are with respect to $p(\mathbf{x})$. Expanding the first term,

$$\mathbb{E} \left[\left(\frac{f(\mathbf{x})}{p(\mathbf{x})} \right)^2 \right] = \int_{\Omega} \left(\frac{f(\mathbf{x})}{p(\mathbf{x})} \right)^2 p(\mathbf{x}) d\mathbf{x} = \int_{\Omega} \frac{f(\mathbf{x})^2}{p(\mathbf{x})} d\mathbf{x} \quad (7.3.6)$$

To minimize the variance, we need to minimize this integral subject to the constraint that $p(\mathbf{x})$ is a normalized probability distribution. By the Cauchy-Schwarz inequality, we have

$$\left(\int_{\Omega} \frac{|f(\mathbf{x})|}{p(\mathbf{x})} p(\mathbf{x}) d\mathbf{x} \right)^2 \leq \int_{\Omega} \frac{f(\mathbf{x})^2}{p(\mathbf{x})} d\mathbf{x} \int_{\Omega} p(\mathbf{x}) d\mathbf{x} \quad (7.3.7)$$

The right-hand side simplifies to $\int f^2/p d\mathbf{x}$ since p is normalized. The left-hand side is $(\int |f(\mathbf{x})| d\mathbf{x})^2$. Equality holds when $|f(\mathbf{x})|$ is proportional to $p(\mathbf{x})$. Therefore, the variance is minimized when

$$p^*(\mathbf{x}) = \frac{|f(\mathbf{x})|}{\int_{\Omega} |f(\mathbf{x})| d\mathbf{x}} \quad (7.3.8)$$

This is the optimal importance sampling distribution. If we sample from $p^*(\mathbf{x})$, the ratio $f(\mathbf{x})/p^*(\mathbf{x})$ is constant up to sign. Specifically, $f(\mathbf{x})/p^*(\mathbf{x}) = (\int_{\Omega} |f(\mathbf{x})| d\mathbf{x}) \cdot \text{sign}(f(\mathbf{x}))$, which means all samples contribute equally in magnitude to the integral estimate. In the ideal case where $f(\mathbf{x}) \geq 0$ everywhere, we have $p^*(\mathbf{x}) = f(\mathbf{x})/I$ and the variance is exactly zero, which means that every sample gives the same value I when weighted appropriately.² In practice, the effectiveness of importance sampling is governed entirely by variance control. If $p(\mathbf{x})$ underweights regions where $|f(\mathbf{x})|$ is large, the weights f/p explode and the estimator's variance becomes impractically large.

The Catch-22 of Optimal Importance Sampling: The optimal distribution gives us perfect estimates, but we can't use it without already knowing the answer. When $f(\mathbf{x}) \geq 0$ and we sample from $p^*(\mathbf{x}) = f(\mathbf{x})/I$, every sample gives

$$\frac{f(\mathbf{x}_i)}{p^*(\mathbf{x}_i)} = \frac{f(\mathbf{x}_i)}{f(\mathbf{x}_i)/I} = I \quad (7.3.9)$$

The estimate is exactly I with *zero variance*. But to construct p^* , we need its normalization constant, which is precisely I , the integral we're trying to compute!

For a concrete example, suppose we want to compute $I = \int_0^1 e^x dx$. The optimal importance sampling distribution is

$$p^*(x) = \frac{e^x}{\int_0^1 e^x dx} = \frac{e^x}{e - 1} \quad (7.3.10)$$

Notice how we had to compute the normalization constant to construct p^* , and the normalization constant required knowing the very integral we're trying to compute. If we already knew $I = e - 1$, we wouldn't need to run Monte Carlo integration in the first place. This paradox means that in practice, we seek distributions that approximate $|f(\mathbf{x})|$ without requiring knowledge of I . For instance, we might use $p(\mathbf{x}) \propto |f(\mathbf{x})|$ with a tractable normalization or concentrate samples where we expect $|f(\mathbf{x})|$ to be large.

²Note that when $f(\mathbf{x})$ changes sign, the optimal distribution involves $|f(\mathbf{x})|$, but large cancellations between positive and negative contributions can still make the variance large. This is known as the sign problem and is particularly severe in quantum Monte Carlo and other applications (not anything you need to worry about in 10.34).

So we can't actually use the optimal importance sampling distribution in practice, but all is not lost, as we can uncover some useful information by looking at its form. Indeed, the optimal distribution shows us that our strategy should be to choose $p(\mathbf{x})$ to be roughly proportional to $|f(\mathbf{x})|$. And as we saw in the example above, we must choose a distribution that approximates this without requiring knowledge of the integral we're computing (or else there would be no need for MC sampling in the first place). For example, if $f(\mathbf{x})$ has sharp peaks at known locations, we might choose a Gaussian mixture centered on those peaks. If $f(\mathbf{x}) = \exp(-U(\mathbf{x}))$ for some energy function U , we might sample from a simpler distribution with similar structure.

Here's the importance sampling algorithm:

Importance sampling for computing $I = \int_{\Omega} f(\mathbf{x}) d\mathbf{x}$:

1. Choose an importance distribution $p(\mathbf{x})$ that is nonzero wherever $f(\mathbf{x}) \neq 0$ (try to match the structure of $f(\mathbf{x})$ as closely as possible)
2. Generate M samples $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_M$ from $p(\mathbf{x})$
3. Compute the weighted average:

$$I \approx \frac{1}{M} \sum_{i=1}^M \frac{f(\mathbf{x}_i)}{p(\mathbf{x}_i)} \quad (7.3.11)$$

This gives us a numerical recipe for approximating integrals more efficiently than uniform sampling. But there's a practical problem here: how do we actually generate samples from $p(\mathbf{x})$? For a uniform distribution, this is trivial; we can use any programming language's uniform random number generator. But for an arbitrary probability distribution, especially in multiple dimensions, sampling is nontrivial.

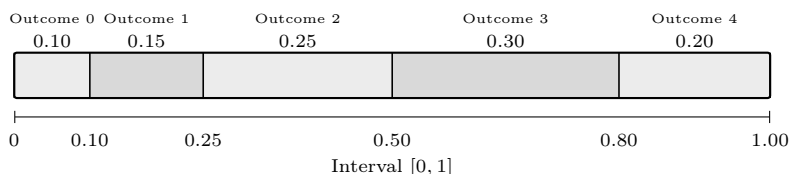
Sampling from Arbitrary Discrete Distributions In the discrete case, we can sample from any target distribution using only a uniform random number generator. Let's see how this works with a concrete example. Imagine we have 5 possible outcomes (labeled 0 through 4) with the following probabilities:

Outcome	0	1	2	3	4
Probability	0.10	0.15	0.25	0.30	0.20

The method, called the discrete inverse transform method, works in two main stages. First, we pre-compute the cumulative distribution by summing the probabilities. This creates a series of "bins" that divide the interval $[0, 1]$:

Outcome	0	1	2	3	4
Cumulative	0.10	0.25	0.50	0.80	1.00

Each outcome now corresponds to a sub-interval: outcome 0 gets $[0, 0.10)$, outcome 1 gets $[0.10, 0.25)$, outcome 2 gets $[0.25, 0.50)$, and so on. Notice that the size of each bin is exactly equal to its target probability.



Second, to generate a sample, we draw a single random number u uniformly from $[0, 1]$ (i.e., $u \sim \mathcal{U}(0, 1)$) and find which bin u falls into. For example, if we draw $u = 0.65$, we look at our cumulative table. Since 0.65 is greater than 0.50 (the top of bin 2) but less than or equal to 0.80 (the top of bin 3), our sample falls into bin 3. We return outcome 3. This works because the probability of u landing in any specific bin is equal to the bin's width, which we designed to be the outcome's target probability. For instance, the probability of u falling in the range $[0.50, 0.80)$ is $0.80 - 0.50 = 0.30$, which is exactly the probability we wanted for outcome 3.

Figure 7.3.1 illustrates this process. The left panel shows the target probabilities, and the right panel shows the cumulative distribution (the bins that we compute by summing the probabilities). Our random draw $u = 0.65$ lands in the interval corresponding to outcome 3. If we keep repeating this process, we will generate outcomes with the correct frequencies according to our target distribution. To be explicit here, calling our discrete target distribution's PMF $p(x)$ and its CDF $F(x)$, we are saying that this procedure maps a uniform random number u to a sample $x \sim p(x)$ via the relation $x = F^{-1}(u)$.

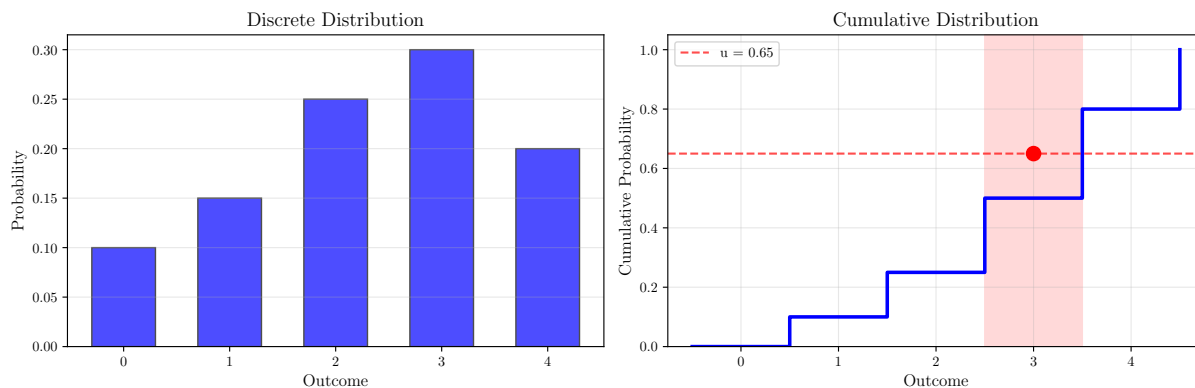


Figure 7.3.1: Sampling from a discrete distribution. We draw $u \sim \mathcal{U}(0,1)$ and find the bin where the cumulative distribution exceeds u . Here, $u = 0.65$ falls into the bin for outcome 3.

Sampling from Arbitrary Continuous Distributions The same idea extends to continuous distributions. Suppose we want to sample from a continuous PDF $p(x)$. The sampling process is analogous to the discrete case:

1. Draw a random number u uniformly from $[0, 1]$ (i.e., $u \sim \mathcal{U}(0, 1)$).
2. Compute the corresponding x by inverting the CDF: $x = F^{-1}(u)$.

This value x is our sample drawn from the distribution $p(x)$.

Why does this work? The CDF $F(x)$ maps x -values to probabilities in $[0, 1]$. Since it's monotonically increasing, its inverse $F^{-1}(u)$ exists and maps uniform probabilities from $[0, 1]$ back to x -values. We can show this formally by checking the cumulative probability of our generated sample x :

$$\Pr(x \leq a) = \Pr(F^{-1}(u) \leq a) \quad (7.3.12)$$

Applying F to both sides of the inequality (which preserves the relation since F is increasing), we get

$$\Pr(u \leq F(a)) = F(a) \quad (7.3.13)$$

The final equality holds because u is uniform on $[0, 1]$, so the probability of u being less than some value v is simply v . Since the resulting sample x has the cumulative distribution $F(a)$, it must be distributed according to the original density $p(x)$.³

Figure 7.3.2 shows this for a bimodal distribution. The right panel shows the CDF $F(x)$. We draw $u = 0.7$ on the y -axis, find its corresponding point on the curve, and map it down to the x -axis to get our sample x . This process correctly maps uniform draws into regions of high probability density.

Despite its simplicity, the inverse transform method has major practical limitations:

- The inverse CDF F^{-1} is often hard to compute. For many common distributions (like the Gaussian or the bimodal example in the figure), $F(x)$ and its inverse $F^{-1}(u)$ cannot be written in a simple

³This argument assumes F is strictly increasing so that F^{-1} is well-defined. For distributions with flat regions or discrete jumps, we define $F^{-1}(u)$ as the smallest x with $F(x) \geq u$, which still works.

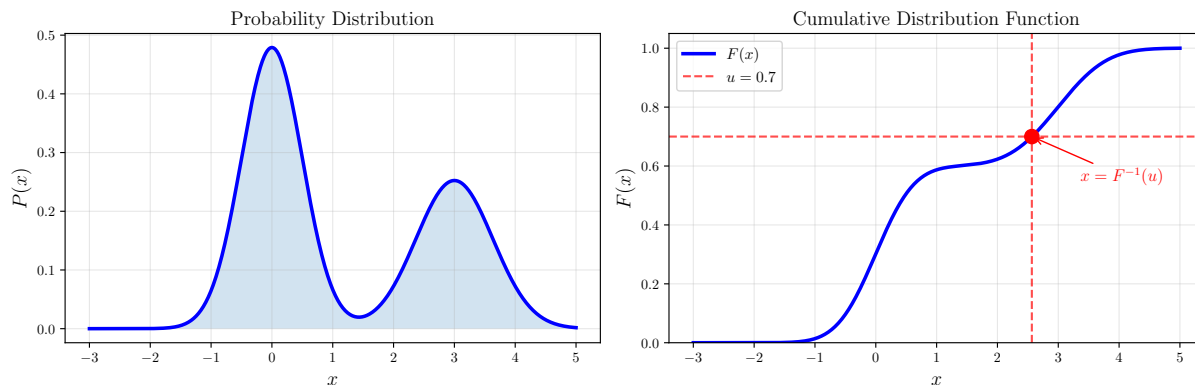


Figure 7.3.2: Inverse transform sampling in one dimension. The CDF $F(x)$ maps the domain of x to $[0, 1]$. Its inverse $F^{-1}(u)$ maps uniform random numbers u back to samples x from the target distribution $p(x)$.

closed form. In these cases, we must use expensive numerical root-finding to solve $F(x) = u$ or use a pre-computed lookup table, which requires storage and interpolation.

- The method does not extend well to high dimensions. For multivariate distributions in $\mathbb{R}^N$, there is no unique or simple way to define a CDF and its inverse. While we could use this method for N independent variables, most interesting problems involve complex dependencies.

Still, inverse transform sampling is extremely fast for discrete distributions (requiring just a binary search on the cumulative table) and is the method of choice for any 1D continuous distribution (like the exponential distribution) where F^{-1} has an efficient closed-form expression. For high-dimensional problems, more sophisticated techniques are available.

7.4 Rejection Sampling*

While inverse transform sampling is elegant and exact, we saw that it has the major limitation that we need to analytically compute and invert the cumulative distribution function. For many distributions, especially with complex functional forms, this is practically impossible. Rejection sampling gives us a complementary approach that works for any distribution we can evaluate, even if we cannot integrate it. Importantly, rejection sampling does not require the target distribution $p(\mathbf{x})$ to be normalized. We only need to be able to evaluate something proportional to $p(\mathbf{x})$, which is often much easier to do. Rejection sampling is best understood from a geometric perspective. Instead of transforming uniform samples through the CDF, we uniformly sample from a region that encloses the target distribution and then randomly “reject” points to sculpt out the desired shape (see Figure 7.4.1 for a 1D example).

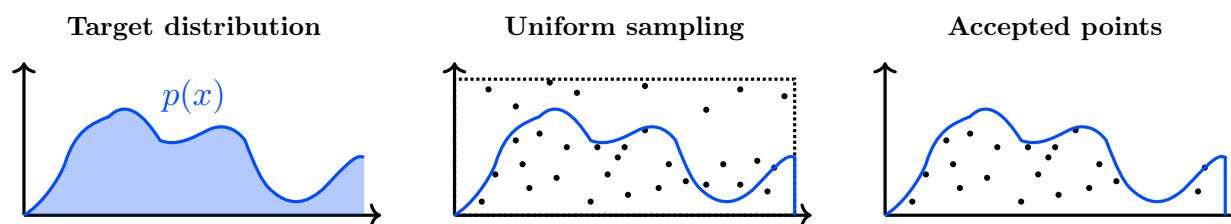


Figure 7.4.1: The rejection sampling process. We generate many samples uniformly, then accept only those that appropriately represent the target distribution $p(\mathbf{x})$.

The method works as follows. We need a proposal distribution $q(\mathbf{x})$ that we can easily sample from (typically

uniform) and a constant M such that the target distribution is always bounded by the scaled proposal:

$$p(\mathbf{x}) \leq Mq(\mathbf{x}) \quad \forall \mathbf{x} \in \Omega \quad (7.4.1)$$

This ensures that $Mq(\mathbf{x})$ forms an envelope over $p(\mathbf{x})$ (in the 1D example shown in Figure 7.4.1, the bound $Mq(\mathbf{x})$ is shown as the black dotted line (a constant) since we are using a uniform proposal distribution). For each proposed sample $\mathbf{x}_i$ drawn from $q(\mathbf{x})$, we generate a uniform random number $u_i \in [0, 1]$ and accept the sample if

$$u_i < \frac{p(\mathbf{x}_i)}{Mq(\mathbf{x}_i)} \quad (7.4.2)$$

Otherwise, we discard it and try again. The accepted samples are distributed according to $p(\mathbf{x})$.

Rejection sampling algorithm:

1. Choose a proposal distribution $q(\mathbf{x})$ that is easy to sample from (e.g., uniform)
2. Find a constant M such that $p(\mathbf{x}) \leq Mq(\mathbf{x})$ for all $\mathbf{x}$ in the domain
3. Generate a sample $\mathbf{x}_i$ from $q(\mathbf{x})$
4. Generate a uniform random number $u_i \in [0, 1]$
5. If $u_i < p(\mathbf{x}_i)/(Mq(\mathbf{x}_i))$, accept $\mathbf{x}_i$ as a sample from $p(\mathbf{x})$
6. Otherwise, reject $\mathbf{x}_i$ and return to step 3
7. Repeat until the desired number of accepted samples is obtained

To see how this works in practice, let's consider a simple example. We want to draw 100 samples from a distribution on $[-1, +1]$ where $p(x < 0) = 1/3$ and $p(x > 0) = 2/3$, with the distribution uniform within each region. We can write this as

$$p(x) = \frac{1}{3}H(-x) + \frac{2}{3}H(x) \quad (7.4.3)$$

where H is the Heaviside function. We'll use a uniform proposal distribution $q(x) = 1/2$ on $[-1, 1]$. To satisfy $p(x) < Mq(x)$ everywhere, we need $M = 4/3$, giving $Mq(x) = 2/3$. We implement this in Python and show the results in Figure 7.4.2. With only 100 accepted samples, the histogram shows the general structure but with significant noise. With 10,000 accepted samples, the empirical distribution closely matches the target, with approximately 1/3 of samples in the negative region and 2/3 in the positive region.

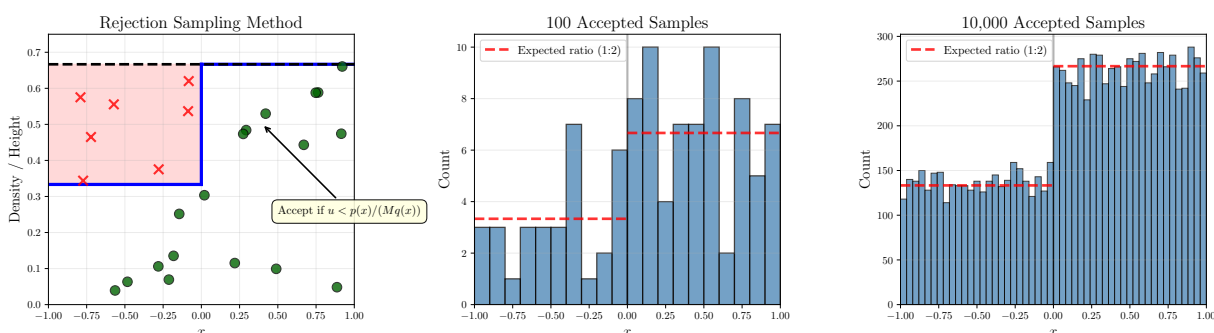


Figure 7.4.2: Rejection sampling from a piecewise-constant target $p(x)$ using a uniform proposal on $[-1, 1]$ and envelope $Mq(x)$. Left: 100 accepted samples give an empirical split of 32/68 between $x < 0$ and $x > 0$ (target 1/3 vs. 2/3). Right: with 10,000 accepted samples (3428/6572), the histogram aligns closely with the theoretical piecewise densities. The dashed lines show the expected per-bin counts; points above the envelope (rejected proposals) illustrate the accept/reject rule.

While rejection sampling is conceptually simple and works for arbitrary distributions, it has significant practical limitations. The first problem is finding an appropriate bound. We need to choose a proposal

distribution $q(\mathbf{x})$ and constant M such that $p(\mathbf{x}) \leq Mq(\mathbf{x})$ everywhere. For complex distributions, especially in high dimensions, this can be difficult. If $p(\mathbf{x})$ has sharp peaks or heavy tails, we may need a very large M to ensure the inequality holds everywhere. This leads to the second, more serious problem of computational efficiency. The acceptance probability for each proposed sample is $p(\mathbf{x}_i)/(Mq(\mathbf{x}_i))$. If M is large, most proposals are rejected. In the worst case, if $p(\mathbf{x})$ is concentrated in a tiny region while $Mq(\mathbf{x})$ must cover a vast domain, we might reject 99.99% of proposals or more. Each rejection wastes a random number generation and a function evaluation. To obtain N accepted samples, we typically need on the order of $N \times M$ proposals, which can be prohibitively expensive.

In high dimensions, both problems become severe. The volume of high-dimensional spaces grows exponentially, making it hard to find tight bounds, and the rejection rate can become astronomical. For the problems we actually care about (sampling from posterior distributions in Bayesian inference, computing partition functions in statistical mechanics, exploring configuration spaces of molecules, etc.), rejection sampling is generally impractical. The solution is Markov Chain Monte Carlo, which we'll develop next. This method avoids the need for a bounding envelope and can efficiently explore high-dimensional distributions by constructing a sequence of samples where each new sample depends only on the previous one.

7.5 Markov Chain Monte Carlo

Both the inverse transform method and rejection sampling are generally impractical in high dimensions. Markov chain Monte Carlo (MCMC) fixes this by taking a completely different approach than both of these methods. Instead of generating independent samples, we construct a sequence of correlated samples where each new sample depends only on the previous one. This seemingly simple idea leads to powerful algorithms that can efficiently explore high-dimensional probability distributions.

7.5.1 Markov Chains

A sequence of random variables $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \dots, \mathbf{x}_M$ forms a Markov chain if the probability of each state depends only on the immediately preceding state. In general, the joint probability of a sequence factors as

$$\Pr(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \dots, \mathbf{x}_M) = \Pr(\mathbf{x}_1) \Pr(\mathbf{x}_2|\mathbf{x}_1) \Pr(\mathbf{x}_3|\mathbf{x}_2, \mathbf{x}_1) \cdots \Pr(\mathbf{x}_M|\mathbf{x}_{M-1}, \dots, \mathbf{x}_2, \mathbf{x}_1) \quad (7.5.1)$$

The sequence is a Markov chain if it satisfies the Markov property:

$$\Pr(\mathbf{x}_i|\mathbf{x}_{i-1}, \dots, \mathbf{x}_2, \mathbf{x}_1) = \Pr(\mathbf{x}_i|\mathbf{x}_{i-1}), \quad i = 2, 3, \dots, M \quad (7.5.2)$$

The conditional probability $\Pr(\mathbf{x}_i|\mathbf{x}_{i-1})$ describes the likelihood of observing a value $\mathbf{x}_i$ at step i given the value $\mathbf{x}_{i-1}$ at step $i-1$. The future depends only on the present, not on the past.

Markov chains appear throughout science. The panels in [Figure 7.5.1](#) give two such cases. [Figure 7.5.1a](#) shows a discrete-state chain used for disease progression: the chain moves between Healthy and Sick with the indicated transition probabilities, and the state Dead is absorbing (once entered, it cannot be left). [Figure 7.5.1b](#) shows a continuous-state process (Brownian motion): the next position depends only on the current position. Formally,

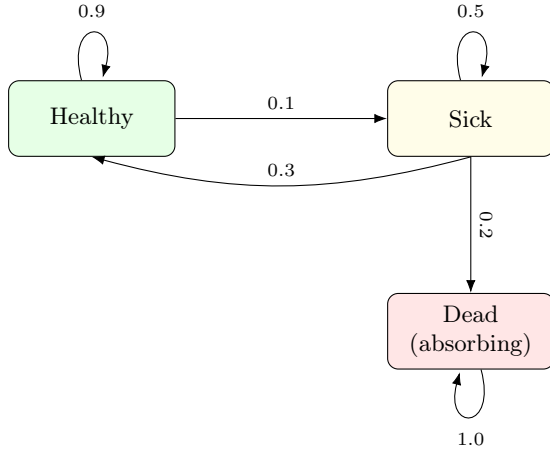
$$p(\mathbf{x}_{t+\Delta t} | \mathbf{x}_t) \sim \mathcal{N}(\mathbf{x}_t, \alpha \Delta t \mathbf{I}), \quad (7.5.3)$$

where α is the diffusion coefficient. Both panels highlight the Markov property, which is that the future is conditionally independent of the past given the present.

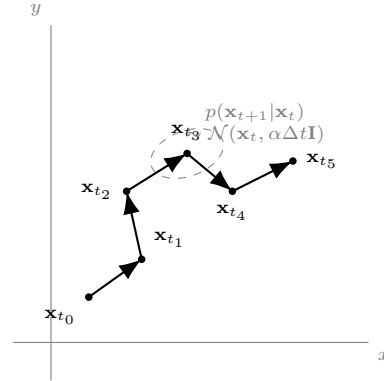
The conditional probability $\Pr(\mathbf{x}_i|\mathbf{x}_{i-1})$ is the transition probability density that governs movement between states $\mathbf{x}_{i-1}$ and $\mathbf{x}_i$. If this transition probability is independent of i , the Markov chain is homogeneous. For a homogeneous chain, we write the transition probability as

$$\Pr(\mathbf{x}_i|\mathbf{x}_{i-1}) = T(\mathbf{x}_i, \mathbf{x}_{i-1}) \quad (7.5.4)$$

where $T(\mathbf{x}, \mathbf{y})$ is the probability density for transitioning from state $\mathbf{y}$ to state $\mathbf{x}$. For a homogeneous Markov chain, we can define a stationary distribution (also called an invariant distribution) $\pi(\mathbf{x})$ that represents the



(a) Discrete Markov chain (disease progression). Each arrow shows the probability of jumping to the next state in one step. “Dead” is absorbing: once there, you never leave.



(b) Continuous-state Markov process (Brownian motion / diffusion). Each new position depends only on the immediately previous one.

Figure 7.5.1: Examples of Markov chains / processes.

steady state of the chain. This is the distribution that remains unchanged under the action of the transition probability:

$$\pi(\mathbf{x}) = \int_{\Omega} T(\mathbf{x}, \mathbf{y}) \pi(\mathbf{y}) d\mathbf{y} \quad (7.5.5)$$

If the chain starts from the stationary distribution, it stays in that distribution. Under mild ergodicity assumptions (irreducibility and aperiodicity, which we’ll discuss momentarily), the distribution of $\mathbf{x}_i$ converges to this stationary distribution regardless of the starting point, making it the limiting distribution of the chain.

To understand this convergence, consider how the distribution evolves. If we start with some initial distribution $\Pr(\mathbf{x}_1)$ and apply the transition probability, we get the distribution at step 2:

$$\Pr(\mathbf{x}_2) = \int_{\Omega} T(\mathbf{x}_2, \mathbf{x}_1) \Pr(\mathbf{x}_1) d\mathbf{x}_1 \quad (7.5.6)$$

Applying the transition again gives

$$\Pr(\mathbf{x}_3) = \int_{\Omega} T(\mathbf{x}_3, \mathbf{x}_2) \Pr(\mathbf{x}_2) d\mathbf{x}_2 \quad (7.5.7)$$

Repeated application of the transition probability causes the distribution to converge:

$$\pi(\mathbf{x}) = \lim_{M \rightarrow \infty} \Pr(\mathbf{x}_M) = \lim_{M \rightarrow \infty} \int_{\Omega} T(\mathbf{x}, \mathbf{x}_{M-1}) \Pr(\mathbf{x}_{M-1}) d\mathbf{x}_{M-1} = \int_{\Omega} T(\mathbf{x}, \mathbf{y}) \pi(\mathbf{y}) d\mathbf{y} \quad (7.5.8)$$

The remarkable fact is that the chain converges to $\pi(\mathbf{x})$ without us needing explicit knowledge of $\pi(\mathbf{x})$ beforehand. We only need to construct a transition probability $T(\mathbf{x}, \mathbf{y})$ that has $\pi(\mathbf{x})$ as its limiting distribution. In fact, this observation is the key to MCMC. If we can design a Markov chain whose limiting distribution is the target distribution $\Pr(\mathbf{x})$ we want to sample from, then running the chain long enough will produce samples from $\Pr(\mathbf{x})$. But how do we construct such a transition probability?

The answer lies in a condition called detailed balance. A sufficient condition for $\pi(\mathbf{x})$ to be a stationary distribution (and therefore the unique limiting distribution if the chain is also irreducible and aperiodic) is that the transition probability satisfies the detailed balance

$$\pi(\mathbf{x}) T(\mathbf{y}, \mathbf{x}) = \pi(\mathbf{y}) T(\mathbf{x}, \mathbf{y}) \quad (7.5.9)$$

for all $\mathbf{x}, \mathbf{y} \in \Omega$. We can interpret this condition physically as saying that in the limiting distribution, the probability flux from state $\mathbf{x}$ to state $\mathbf{y}$ equals the flux from $\mathbf{y}$ to $\mathbf{x}$. The probability of being at $\mathbf{x}$ and transitioning to $\mathbf{y}$ is balanced by the probability of being at $\mathbf{y}$ and transitioning to $\mathbf{x}$.

To verify that detailed balance implies $\pi(\mathbf{x})$ is the limiting distribution, integrate the detailed balance equation over $\mathbf{y}$:

$$\int_{\Omega} \pi(\mathbf{x}) T(\mathbf{y}, \mathbf{x}) d\mathbf{y} = \int_{\Omega} \pi(\mathbf{y}) T(\mathbf{x}, \mathbf{y}) d\mathbf{y} \quad (7.5.10)$$

The left side is $\pi(\mathbf{x}) \int_{\Omega} T(\mathbf{y}, \mathbf{x}) d\mathbf{y} = \pi(\mathbf{x})$ since T is normalized (integrating the transition probability from $\mathbf{x}$ over all possible destinations gives 1). The right side is exactly the definition of the stationary distribution. Therefore, $\pi(\mathbf{x})$ is indeed the stationary distribution. If the chain is also ergodic (irreducible and aperiodic), this stationary distribution is unique and is the limiting distribution of the chain.

The detailed balance is sufficient but not necessary for a limiting distribution to exist.

Modern MCMC methods, including non-reversible (“lifted”) samplers and, more practically relevant, Hamiltonian Monte Carlo (HMC), deliberately construct transitions that violate detailed balance. It is provable that well-designed non-reversible chains mix faster than any reversible chain using the same computational effort, which is part of why HMC and its relatives have become the default choice for sampling continuous, high-dimensional distributions in modern practice. We will not spend time discussing the details here.

However, detailed balance provides a constructive way to design transition probabilities. If we want samples from a target distribution $\pi(\mathbf{x})$, we need to construct a transition probability $T(\mathbf{x}, \mathbf{y})$ that satisfies

$$\pi(\mathbf{x}) T(\mathbf{y}, \mathbf{x}) = \pi(\mathbf{y}) T(\mathbf{x}, \mathbf{y}) \implies \frac{T(\mathbf{y}, \mathbf{x})}{T(\mathbf{x}, \mathbf{y})} = \frac{\pi(\mathbf{y})}{\pi(\mathbf{x})} \quad (7.5.11)$$

The ratio of transition probabilities must equal the ratio of target probabilities. This is the central design principle for MCMC algorithms.

7.5.2 The Metropolis-Hastings Algorithm

The problem is now reduced to finding a transition probability $T(\mathbf{x}, \mathbf{y})$ that satisfies the detailed balance condition for our desired target distribution, $\pi(\mathbf{x})$. The Metropolis-Hastings algorithm provides a general recipe for constructing exactly such a transition. The idea is to split the transition T into two sub-steps: a proposal and an acceptance.

1. **Propose:** From the current state $\mathbf{x}_i$, propose a new candidate state $\mathbf{x}^*$ according to a proposal distribution $M(\mathbf{x}^*, \mathbf{x}_i)$.
2. **Accept/Reject:** Accept this proposed move with some probability $A(\mathbf{x}^*, \mathbf{x}_i)$.

The full transition probability $T(\mathbf{x}^*, \mathbf{x}_i)$ (for $\mathbf{x}^* \neq \mathbf{x}_i$) is the probability of proposing $\mathbf{x}^*$ and accepting it:

$$T(\mathbf{x}^*, \mathbf{x}_i) = M(\mathbf{x}^*, \mathbf{x}_i) A(\mathbf{x}^*, \mathbf{x}_i) \quad (7.5.12)$$

(The probability of remaining at $\mathbf{x}_i$ is the sum of probabilities of all rejected moves.) Now, we can plug this composite transition into the detailed balance equation:

$$\pi(\mathbf{x}_i) T(\mathbf{x}^*, \mathbf{x}_i) = \pi(\mathbf{x}^*) T(\mathbf{x}_i, \mathbf{x}^*) \quad (7.5.13)$$

$$\pi(\mathbf{x}_i) M(\mathbf{x}^*, \mathbf{x}_i) A(\mathbf{x}^*, \mathbf{x}_i) = \pi(\mathbf{x}^*) M(\mathbf{x}_i, \mathbf{x}^*) A(\mathbf{x}_i, \mathbf{x}^*) \quad (7.5.14)$$

Rearranging this gives a condition on the ratio of the acceptance probabilities:

$$\frac{A(\mathbf{x}^*, \mathbf{x}_i)}{A(\mathbf{x}_i, \mathbf{x}^*)} = \frac{\pi(\mathbf{x}^*) M(\mathbf{x}_i, \mathbf{x}^*)}{\pi(\mathbf{x}_i) M(\mathbf{x}^*, \mathbf{x}_i)} \quad (7.5.15)$$

This gives us a way to design A for any given proposal M and target π . The specific choice proposed by Metropolis and Hastings is given below.

Metropolis-Hastings Acceptance Probability:

$$A(\mathbf{x}^*, \mathbf{x}_i) = \min \left\{ 1, \frac{\pi(\mathbf{x}^*)M(\mathbf{x}_i, \mathbf{x}^*)}{\pi(\mathbf{x}_i)M(\mathbf{x}^*, \mathbf{x}_i)} \right\} \quad (7.5.16)$$

This choice provably satisfies the detailed balance condition. To show this, let

$$R = \frac{\pi(\mathbf{x}^*)M(\mathbf{x}_i, \mathbf{x}^*)}{\pi(\mathbf{x}_i)M(\mathbf{x}^*, \mathbf{x}_i)}.$$

The acceptance probability for the reverse move is $A(\mathbf{x}_i, \mathbf{x}^*) = \min\{1, 1/R\}$. If $R \geq 1$, $A(\mathbf{x}^*, \mathbf{x}_i) = 1$ and $A(\mathbf{x}_i, \mathbf{x}^*) = 1/R$. The ratio is $A(\mathbf{x}^*, \mathbf{x}_i)/A(\mathbf{x}_i, \mathbf{x}^*) = 1/(1/R) = R$. If $R < 1$, $A(\mathbf{x}^*, \mathbf{x}_i) = R$ and $A(\mathbf{x}_i, \mathbf{x}^*) = 1$. The ratio is $A(\mathbf{x}^*, \mathbf{x}_i)/A(\mathbf{x}_i, \mathbf{x}^*) = R/1 = R$. In both cases, the ratio is R , satisfying the condition. This gives us a complete algorithm for generating a Markov chain.

The Metropolis-Hastings Algorithm

1. Initialize at some $\mathbf{x}_1$. Set $i = 1$.
2. Draw a candidate state $\mathbf{x}^*$ from the proposal distribution $M(\mathbf{x}^*, \mathbf{x}_i)$.
3. Compute the acceptance probability:

$$A(\mathbf{x}^*, \mathbf{x}_i) = \min \left\{ 1, \frac{\pi(\mathbf{x}^*)M(\mathbf{x}_i, \mathbf{x}^*)}{\pi(\mathbf{x}_i)M(\mathbf{x}^*, \mathbf{x}_i)} \right\} \quad (7.5.17)$$

4. Accept/Reject:
 - (a) Draw a uniform random number $u \sim \mathcal{U}(0, 1)$.
 - (b) If $u < A(\mathbf{x}^*, \mathbf{x}_i)$, accept the move: set $\mathbf{x}_{i+1} = \mathbf{x}^*$.
 - (c) If $u \geq A(\mathbf{x}^*, \mathbf{x}_i)$, reject the move: set $\mathbf{x}_{i+1} = \mathbf{x}_i$. (Note: the current state is added to the chain again).
5. Increment i and return to step 2.

The Power of Ratios Notice that the acceptance probability A depends only on the *ratio* of the target distribution at the two points, $\pi(\mathbf{x}^*)/\pi(\mathbf{x}_i)$. This is the most powerful feature of MCMC. Like we've said before, in many real-world problems, our target distribution is only known up to an intractable normalization constant Z .

$$\pi(\mathbf{x}) = \frac{f(\mathbf{x})}{\int_{\Omega} f(\mathbf{x}') d\mathbf{x}'} = \frac{f(\mathbf{x})}{Z} \quad (7.5.18)$$

When we compute the ratio, Z cancels out:

$$\frac{\pi(\mathbf{x}^*)}{\pi(\mathbf{x}_i)} = \frac{f(\mathbf{x}^*)/Z}{f(\mathbf{x}_i)/Z} = \frac{f(\mathbf{x}^*)}{f(\mathbf{x}_i)} \quad (7.5.19)$$

This means we can sample from *any* distribution $\pi(\mathbf{x})$ as long as we can compute the value of a function $f(\mathbf{x})$ that is proportional to it. We never need to compute the (often impossible) integral Z . The acceptance probability becomes:

$$A(\mathbf{x}^*, \mathbf{x}_i) = \min \left\{ 1, \frac{f(\mathbf{x}^*)M(\mathbf{x}_i, \mathbf{x}^*)}{f(\mathbf{x}_i)M(\mathbf{x}^*, \mathbf{x}_i)} \right\} \quad (7.5.20)$$

The Proposal Distribution The algorithm works for any proposal distribution M , but its efficiency depends heavily on this choice. A common and simple choice is a symmetric (or reversible) proposal, where

the probability of proposing $\mathbf{x}^*$ from $\mathbf{x}_i$ is the same as proposing $\mathbf{x}_i$ from $\mathbf{x}^*$.

$$M(\mathbf{x}^*, \mathbf{x}_i) = M(\mathbf{x}_i, \mathbf{x}^*) \quad (7.5.21)$$

An example is proposing a move from a uniform distribution centered on the current point: $\mathbf{x}^* = \mathbf{x}_i + \Delta\mathbf{x}$, where $\Delta\mathbf{x} \sim \mathcal{U}(-a, a)$. With a symmetric proposal, the M terms cancel in the acceptance ratio, simplifying it to the original Metropolis algorithm:

$$A(\mathbf{x}^*, \mathbf{x}_i) = \min \left\{ 1, \frac{\pi(\mathbf{x}^*)}{\pi(\mathbf{x}_i)} \right\} = \min \left\{ 1, \frac{f(\mathbf{x}^*)}{f(\mathbf{x}_i)} \right\} \quad (7.5.22)$$

In this case, the algorithm is even simpler. We always accept a move to a state of higher probability ($f(\mathbf{x}^*) > f(\mathbf{x}_i)$), and sometimes accept a move to a state of lower probability.

Heuristic: The 0.234 Rule. A widely used rule of thumb (due to Roberts, Gelman, and Gilks 1997) holds that for random-walk Metropolis-Hastings with a well-tuned Gaussian proposal in d dimensions, the proposal step size should shrink roughly like $d^{-1/2}$ as dimension grows, giving an optimal acceptance rate of approximately 0.234 that is largely insensitive to the target’s specific shape. This is an empirical/asymptotic guideline. Because the step size shrinks with d , the number of steps needed to traverse the target distribution grows with dimension even though each individual step stays computationally cheap. This is the reason for our earlier disclaimer that the *constant* multiplying Monte Carlo’s $M^{-1/2}$ rate grows with dimension. It’s also part of why gradient-informed samplers, such as Langevin-based methods and Hamiltonian Monte Carlo (HMC), are the modern default for continuous, high-dimensional sampling problems.

7.5.3 Convergence and Ergodicity

Running the MCMC algorithm generates a sequence of samples $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_M\}$. What we want is for the empirical distribution of these samples to approximate our target distribution, $\pi(\mathbf{x})$. As the number of iterations M increases, this approximation gets better. Figure 7.5.2 shows this process for a bimodal

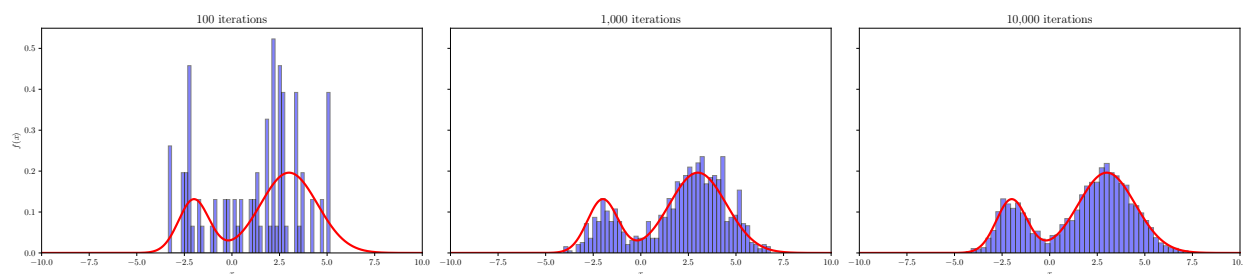


Figure 7.5.2: Convergence of an MCMC sampler. The empirical distribution of samples (histograms) progressively converges to the true target distribution (red line) as the number of iterations increases.

target distribution (red line). After only 100 iterations, the histogram is a very poor match for the bimodal target distribution. After 1,000, the shape begins to emerge. By 10,000 and 100,000 iterations, the histogram of samples provides an excellent approximation of the true distribution. However, this convergence is not automatic. It is only guaranteed if the Markov chain is **ergodic**. An ergodic chain has two main properties. First, it is irreducible. This means that every state must be reachable from every other state (perhaps not in one step, but in a finite number of steps). The chain must be able to explore the entire distribution. Second, it is aperiodic. This means that the chain must not get trapped in deterministic cycles (e.g., oscillating between state A and state B forever). Most standard MCMC samplers (like Metropolis-Hastings with a random proposal) are designed to be aperiodic. Irreducibility, however, is a major practical concern.

Practical Irreducibility and Proposal Choice

A chain can be theoretically irreducible but *practically* fail to explore the state space in a reasonable amount of time. This often happens when the target distribution has multiple, well-separated modes (regions of high probability).

Consider the target distribution $\pi(x) \propto \exp(-(x-10)^2) + 2\exp(-(x+10)^2)$, which has two sharp peaks at $x = -10$ and $x = 10$. If we use a small proposal step (e.g., $\Delta x \sim \mathcal{U}(-1, 1)$), the chain will explore one mode (e.g., around $x = 10$) very well. However, proposing a jump all the way to $x = -10$ is effectively impossible on practical timescales. The chain gets stuck in one mode and fails to sample the other, even though the chain is technically irreducible. If we use a large proposal step (e.g., $\Delta x \sim \mathcal{U}(-10, 10)$), the chain can propose moves that jump between modes. This allows it to explore the full distribution and sample both modes in their correct proportions (twice as many samples near $x = -10$ as near $x = 10$). This is illustrated in Figure 7.5.3. The choice of proposal distribution M is important for ensuring the chain mixes on a practical timescale.

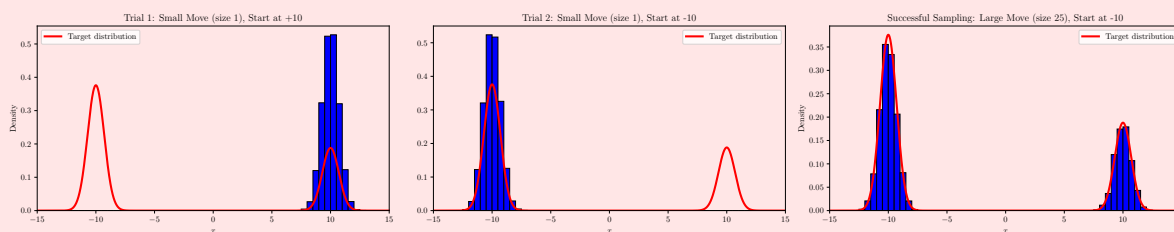


Figure 7.5.3: The role of the proposal distribution for irreducibility. Left and middle: With a small proposal step (max size 1), the chain gets stuck in whichever mode it starts in (Trial 1 vs. Trial 2). It fails to sample the full distribution. Right: With a large proposal step (large enough to traverse between modes), the chain can jump between the modes at $x = -10$ and $x = 10$, correctly sampling the entire target distribution.

7.5.4 Practical MCMC

Running the Metropolis-Hastings algorithm effectively requires handling a few important details carefully. We can use the following techniques to improve the performance of the algorithm.

Burn-In The convergence guarantee of MCMC states that the chain approaches the stationary distribution $\pi(\mathbf{x})$ in the limit of infinite steps. However, we must start the chain somewhere. If our initial state $\mathbf{x}_1$ is a bad guess—that is, it lies in a region of very low probability—the first several (or many) steps of the chain will not be representative samples from $\pi(\mathbf{x})$. They instead represent the chain’s transient walk towards the high-probability region. To correct for this, we use a process called burn-in. We simply discard some number of initial samples from the MCMC run. For a chain of M total steps, we might discard the first M_{burn} samples and compute our averages only using the remaining $M - M_{\text{burn}}$ samples, which are more likely to be drawn from the true limiting distribution. Note that when you discard burn-in, you should only use post-burn-in samples for computing expectations. You do not need to further “thin” the samples (e.g., keeping only every k -th sample) to get correctness; thinning only helps with storage. In practice, the right burn-in length is a heuristic rather than something you can be proven or optimized in advance. It is common to check it informally with simple diagnostics like plotting the values of the chain, or by running several chains from different starting points and confirming they settle into agreement.

Tempering/Simulated Annealing A related challenge, as seen in Figure 7.5.3, is that a chain can get stuck in a local probability mode, failing to explore the full distribution. Tempering is a technique that can help with this. The idea is to first sample from a “flatter” version of the target distribution, which is easier to explore, and then gradually “cool” the distribution to approach the true one. This is done by introducing an

artificial temperature $T_j > 1$ and sampling from $\pi_j(\mathbf{x}) \propto \pi(\mathbf{x})^{1/T_j}$. When T_j is large, $\pi_j(\mathbf{x})$ is much flatter than $\pi(\mathbf{x})$ (Figure 7.5.4), allowing the chain to easily jump over probability barriers. The procedure starts with a high T_1 , runs a chain, then uses its result to initialize a new chain with a smaller $T_2 < T_1$, and so on:

$$\pi(\mathbf{x})^{1/T_1}, \pi(\mathbf{x})^{1/T_2}, \dots, \pi(\mathbf{x})^{1/T_M} \quad \text{where} \quad T_1 > T_2 > \dots > T_M = 1 \quad (7.5.23)$$

The final chain at $T_M = 1$ samples from the true target distribution, but it has hopefully been guided to the globally important regions by the previous “hotter” chains.

It is worth clarifying terminology. The procedure just described is meant to improve sampling. We gradually cool from a high temperature T_1 to $T_M = 1$ and then keep sampling at $T = 1$ to estimate expectations with respect to the true target $\pi(\mathbf{x})$. This idea is closely related to (and historically motivated by) **simulated annealing**, but the goal of simulated annealing is different. In simulated annealing, we introduce a temperature T and define a family of tempered distributions $\pi_T(\mathbf{x}) \propto \pi(\mathbf{x})^{1/T}$. We then lower T toward zero, not toward 1. In the limit $T \rightarrow 0$, $\pi_T(\mathbf{x})$ concentrates all of its probability mass on the global maximum of $\pi(\mathbf{x})$ (or, equivalently, the global minimum of an associated energy or loss function). The Markov chain is run while T is slowly decreased so that it can escape bad local optima at high temperature but eventually settles into a single best mode at very low temperature. In other words, simulated annealing is an *optimization* algorithm: it is designed to find the most probable/highest-weight state. By contrast, tempering as used in MCMC for inference does not cool all the way to $T \rightarrow 0$. Instead, we stop at $T = 1$ and keep sampling so that the chain continues to explore all important regions of $\pi(\mathbf{x})$ in proportion to their probability. In that sense, annealing to $T = 0$ is about finding a single answer, while tempering down to $T = 1$ is about correctly representing uncertainty across many answers.

In practice, *parallel tempering* (also called replica exchange) is often preferred to this serial cooling schedule: rather than cooling a single chain over time, it runs multiple chains simultaneously across a range of temperatures and periodically proposes swaps between neighboring-temperature chains, letting a sample stuck in the cold (target) chain escape by hopping through a “hot” chain that mixes easily. This is the version of tempering actually implemented in most molecular simulation packages and Bayesian inference software. It is an area of active research.

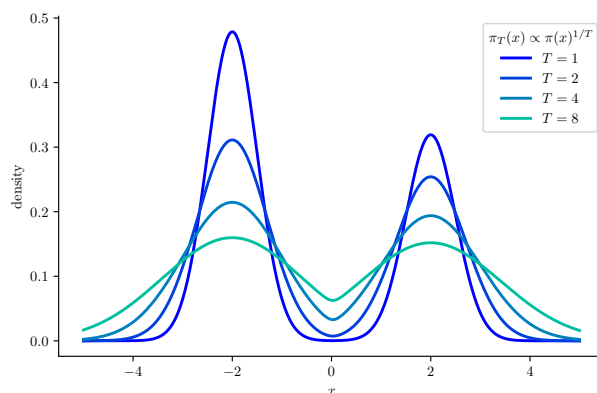


Figure 7.5.4: Effect of temperature in tempering. We plot a bimodal target density $\pi(x)$ ($T=1$) along with tempered versions $\pi_T(x) \propto \pi(x)^{1/T}$ for hotter temperatures $T > 1$. Higher temperatures flatten the distribution, lowering energy barriers between modes and making it easier for an MCMC chain to move between distant regions before cooling back down to $T = 1$.

Autocorrelation and Effective Sample Size

MCMC outputs correlated samples. The estimator $\frac{1}{M} \sum_{i=1}^M f(\mathbf{x}_i)$ is consistent, but its uncertainty depends on the chain’s autocorrelation time. Error bars should be based on an *effective* sample size rather than M itself (e.g., via batch means or integrated autocorrelation time). This is not something

you'll need to do in 10.34, but it's a good thing to be aware of.

7.5.5 Applications

This brings us back to our original motivation. Why did we start talking about sampling from distributions? To compute expected values. The goal is to compute high-dimensional integrals of the form:

$$I = \mathbb{E}[f] = \int f(\mathbf{x}) \pi(\mathbf{x}) d\mathbf{x} \quad (7.5.24)$$

If we use an MCMC algorithm (like Metropolis-Hastings) to generate a sequence of M samples $\{\mathbf{x}_i\}$ from a chain whose limiting distribution is $\pi(\mathbf{x})$, then by the law of large numbers for Markov chains, the sample mean converges to the true expectation:

$$I \approx \frac{1}{M} \sum_{i=1}^M f(\mathbf{x}_i) \quad (7.5.25)$$

We replace an intractable high-dimensional integral with a simple average over samples generated from a Markov chain! This lets us solve deterministic integration problems using stochastic sampling methods that greatly mitigate the curse of dimensionality. The error of Monte Carlo integration typically scales like $1/\sqrt{M}$ regardless of dimension, which avoids the exponential growth in grid points required by classical grid-based methods. Figure 7.5.5 illustrates this process in two dimensions. The Markov chain starts at an arbitrary point and performs a random walk that preferentially explores regions of high probability. The trajectory (right panel) may seem random, but over time, the density of the visited points (far right) perfectly reconstructs the target distribution.

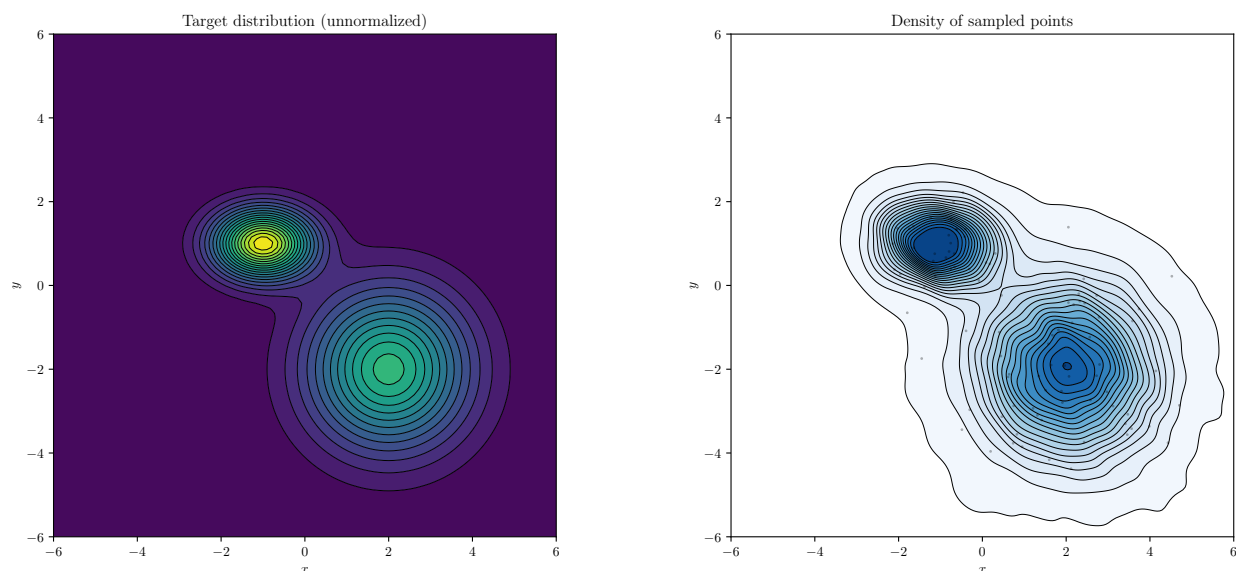


Figure 7.5.5: MCMC in two dimensions. The algorithm explores the state space, and the density of the samples converges to the target distribution.

The single most important feature of MCMC, which we have already used and stated a couple times before, is that the acceptance probability depends only on the ratio of probabilities. This means we never need to know the normalization constant Z . We can work directly with any unnormalized function $w(\mathbf{x})$ that is proportional to our target distribution, $\pi(\mathbf{x}) \propto w(\mathbf{x})$. Our goal is to compute expectations $\langle f \rangle = \int f(\mathbf{x}) \pi(\mathbf{x}) d\mathbf{x}$. Using the unnormalized weight $w(\mathbf{x})$, this is

$$\langle f \rangle = \frac{\int f(\mathbf{x}) w(\mathbf{x}) d\mathbf{x}}{\int w(\mathbf{x}) d\mathbf{x}} \quad (7.5.26)$$

But the MCMC samples $\{\mathbf{x}_i\}$ are drawn with a probability proportional to $w(\mathbf{x})$ precisely to handle this. The simple average of f over the MCMC samples correctly estimates this value:

$$\langle f \rangle \approx \frac{1}{M} \sum_{i=1}^M f(\mathbf{x}_i) \quad (7.5.27)$$

We'll now look at applying MCMC to statistical mechanics and Bayesian parameter estimation, which are two of MCMC's most powerful applications.

Statistical Mechanics In statistical mechanics, the probability of a system being in a state $\mathbf{q}$ (with energy $U(\mathbf{q})$) is given by the Boltzmann distribution:

$$\pi(\mathbf{q}) = \frac{1}{Z} \exp\left(-\frac{U(\mathbf{q})}{k_B T}\right) \quad (7.5.28)$$

where k_B is Boltzmann's constant and T is the temperature. Here, $w(\mathbf{q}) = \exp(-U(\mathbf{q})/k_B T)$ is the unnormalized weight. We can compute the expectation of any quantity (like energy, $\langle U \rangle = \int U(\mathbf{q}) \pi(\mathbf{q}) d\mathbf{q}$) by sampling from $\pi(\mathbf{q})$ using MCMC. For a symmetric proposal $\mathbf{q}_i \rightarrow \mathbf{q}^*$, the Metropolis acceptance probability simplifies to

$$A(\mathbf{q}^*, \mathbf{q}_i) = \min\left\{1, \frac{\pi(\mathbf{q}^*)}{\pi(\mathbf{q}_i)}\right\} = \min\left\{1, \frac{\exp(-U(\mathbf{q}^*)/k_B T)}{\exp(-U(\mathbf{q}_i)/k_B T)}\right\} \quad (7.5.29)$$

$$= \min\left\{1, \exp\left(-\frac{U(\mathbf{q}^*) - U(\mathbf{q}_i)}{k_B T}\right)\right\} = \min\left\{1, \exp\left(-\frac{\Delta U}{k_B T}\right)\right\} \quad (7.5.30)$$

The decision to accept a move depends *only on the change in energy* $\Delta U = U(\mathbf{q}^*) - U(\mathbf{q}_i)$. This makes a lot of physical sense! If the energy goes down (i.e., $\Delta U < 0 \implies \exp(-\Delta U/k_B T) > 1$), we always accept the move. If the energy goes up (i.e., $\Delta U > 0 \implies \exp(-\Delta U/k_B T) < 1$), we accept the move with some probability that depends on the difference in energy and the temperature.

Bayesian Parameter Estimation In parameter estimation, our target distribution is the posterior:

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{1}{Z} p(\mathcal{D}|\boldsymbol{\theta}) p(\boldsymbol{\theta}) \quad (7.5.31)$$

Assuming a Gaussian likelihood, $p(\mathcal{D}|\boldsymbol{\theta}) \propto \exp(-\chi^2(\boldsymbol{\theta})/2)$. The unnormalized posterior is $w(\boldsymbol{\theta}) = \exp(-\chi^2(\boldsymbol{\theta})/2)p(\boldsymbol{\theta})$. The Metropolis acceptance probability for a symmetric proposal $\boldsymbol{\theta}_i \rightarrow \boldsymbol{\theta}^*$ is:

$$A(\boldsymbol{\theta}^*, \boldsymbol{\theta}_i) = \min\left\{1, \frac{p(\boldsymbol{\theta}^*|\mathcal{D})}{p(\boldsymbol{\theta}_i|\mathcal{D})}\right\} = \min\left\{1, \frac{\exp(-\chi^2(\boldsymbol{\theta}^*)/2)p(\boldsymbol{\theta}^*)}{\exp(-\chi^2(\boldsymbol{\theta}_i)/2)p(\boldsymbol{\theta}_i)}\right\} \quad (7.5.32)$$

$$= \min\left\{1, \frac{p(\boldsymbol{\theta}^*)}{p(\boldsymbol{\theta}_i)} \exp\left(-\frac{\chi^2(\boldsymbol{\theta}^*) - \chi^2(\boldsymbol{\theta}_i)}{2}\right)\right\} \quad (7.5.33)$$

The acceptance probability depends only on the change in the weighted sum of squared errors and the ratio of the prior. If the prior is uniform, $p(\boldsymbol{\theta}^*) = p(\boldsymbol{\theta}_i)$, it depends only on the change in χ^2 . Once we have run the chain and collected samples $\{\boldsymbol{\theta}_i\}$, we can do much more than just find the mean. We can ask/answer any question about the posterior distribution:

- **Marginalization:** To find $p(\theta_1|\mathcal{D})$, we simply make a histogram of the θ_1 components of all our samples.
- **Credible intervals:** What is the probability that a parameter θ_j is in a certain region Ω ? We just count the samples: $\Pr(\theta_j \in \Omega) \approx \frac{1}{M} \sum_{i=1}^M \mathbb{I}(\theta_{i,j} \in \Omega)$, where $\mathbb{I}$ is the indicator function.
- **Prediction:** What value y might we measure for a new input x ? We can generate a predictive distribution by taking each sample $\boldsymbol{\theta}_i$ and drawing a corresponding y from the model: $y_i = g(x; \boldsymbol{\theta}_i) + \epsilon_i$, where ϵ_i is drawn from the measurement noise. The histogram of $\{y_i\}$ shows our full prediction.

7.6 Kinetic Monte Carlo

Everything we have done so far uses Monte Carlo to sample from a *static* distribution. “Static” refers to the fact that we draw samples from some target $\pi(\mathbf{x})$ to compute averages at equilibrium. But many problems in chemical engineering are inherently *dynamic*. Reactions fire, molecules adsorb and desorb, crystals grow. The question is not “what is the equilibrium state?” but “what happens next, and when?” Kinetic Monte Carlo (kMC) methods answer this question by simulating the time evolution of a system one random event at a time.⁴

The setting is a system that can be in one of many discrete states $\mathbf{s}$ with R possible events (reactions, transitions) that can occur. Each event μ has a *propensity* $a_\mu(\mathbf{s})$ that depends on the current state and plays the role of a rate, which means that the probability that event μ fires in a small time interval $[t, t + dt)$ is $a_\mu dt$. The total propensity is $a_0 = \sum_{\mu=1}^R a_\mu$, and the time until the next event of *any* kind is exponentially distributed:

$$\tau \sim \text{Exp}(a_0), \quad \text{i.e.,} \quad p(\tau) = a_0 e^{-a_0 \tau} \quad (7.6.1)$$

When that event fires, it is reaction μ with probability a_μ/a_0 . This is exact. It is mathematically equivalent to solving the chemical master equation, which tracks $p(\mathbf{s}, t)$ for every state $\mathbf{s}$, but without ever writing down or storing the (enormous) transition matrix!

The Gillespie (kMC) Algorithm

1. Initialize the state $\mathbf{s}$ and set $t = 0$.
2. Compute all propensities $a_\mu(\mathbf{s})$ for $\mu = 1, \dots, R$ and their sum $a_0 = \sum_{\mu} a_\mu$.
3. If $a_0 = 0$, stop (no more events can occur).
4. Draw the waiting time: $\tau = -\ln(r_1)/a_0$, where $r_1 \sim \mathcal{U}(0, 1)$.
5. Select which event fires: find the μ satisfying $\sum_{\nu=1}^{\mu-1} a_\nu < r_2 a_0 \leq \sum_{\nu=1}^{\mu} a_\nu$, where $r_2 \sim \mathcal{U}(0, 1)$.
6. Update the state $\mathbf{s}$ according to event μ and advance $t \rightarrow t + \tau$.
7. Return to step 2.

The algorithm presented here is usually called the Gillespie algorithm when applied to well-mixed chemical kinetics and kMC when applied to spatially resolved systems such as surface reactions on a lattice. The underlying mathematical framework (exact stochastic simulation of a continuous-time Markov chain) is the same in both cases.

The intuition behind this algorithm is a picture of competing clocks. Imagine each possible event μ has its own independent alarm clock that rings after a random, exponentially distributed time with rate a_μ . All R clocks are ticking simultaneously, and whatever rings first is the event that happens. The time to the first ring out of all R clocks is exponential with rate $a_0 = \sum a_\mu$ (the more clocks ticking, the sooner one goes off), and the probability that it was clock μ that rang first is a_μ/a_0 . This is exactly what steps 4 and 5 sample. The property that makes this work is the *memorylessness* property of the exponential distribution, the unique continuous distribution for which $P(\text{wait} > t + s \mid \text{wait} > t) = P(\text{wait} > s)$. After we fire an event and recompute the propensities, the remaining clocks don’t “remember” how long they’ve been ticking, so we can throw them all away and start fresh. This is why the algorithm loops back to step 2 with no leftover state.

One subtlety that is easy to overlook is that the propensity a_μ must be expressed in terms of molecule counts, not concentrations. For a unimolecular reaction $A \rightarrow \dots$ with deterministic rate $r = k[A]$, the stochastic propensity is simply $a = kN_A$ (units of k : s^{-1}). But for a bimolecular reaction $A + B \rightarrow \dots$ with deterministic rate $r = k[A][B]$, the macroscopic rate constant k (units of $\text{M}^{-1}\text{s}^{-1}$) must be converted to a stochastic rate constant $c = k/(N_A V)$, giving propensity $a = c N_A N_B$, where V is the system volume and

⁴As a fun fact, the earliest application of kMC to an atomistic system appears to have been [Beeler’s 1966 simulation of radiation damage annealing in iron](#).

N_{Av} is Avogadro's number. Using the macroscopic k directly would produce simulation timescales that are wrong by orders of magnitude.

As a concrete example, consider the consecutive reaction $A \xrightarrow{k_1} B \xrightarrow{k_2} C$. There are $R = 2$ possible events: the conversion of one A molecule to B (propensity $a_1 = k_1 N_A$) and the conversion of one B molecule to C (propensity $a_2 = k_2 N_B$). Because each event changes the molecule count by ± 1 , the trajectories are staircase functions. Figure 7.6.1 shows several stochastic realizations alongside the deterministic ODE solution $dN_A/dt = -k_1 N_A$, $dN_B/dt = k_1 N_A - k_2 N_B$. With only 50 molecules, individual trajectories fluctuate visibly around the deterministic curve. With 500 molecules, the relative fluctuations shrink (they scale as $1/\sqrt{N}$) and the stochastic trajectories collapse onto the ODE solution. This is the *thermodynamic limit*, where deterministic kinetics emerge as a large- N limit of the stochastic process.

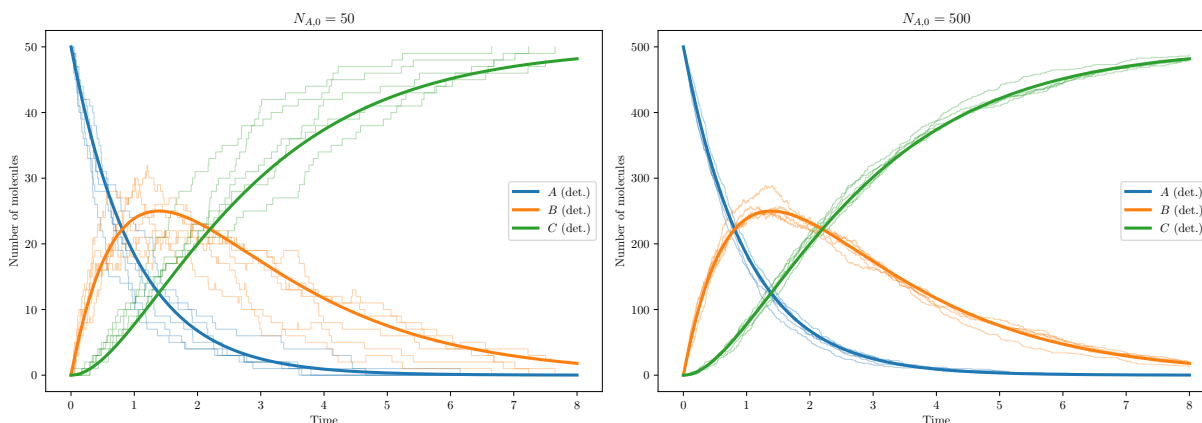


Figure 7.6.1: Kinetic Monte Carlo simulation of $A \rightarrow B \rightarrow C$ ($k_1 = 1$, $k_2 = 0.5$). Faint step functions are individual stochastic realizations; solid curves are the deterministic ODE solution. Left: with 50 molecules, fluctuations are significant. Right: with 500, the stochastic trajectories closely follow the deterministic limit.

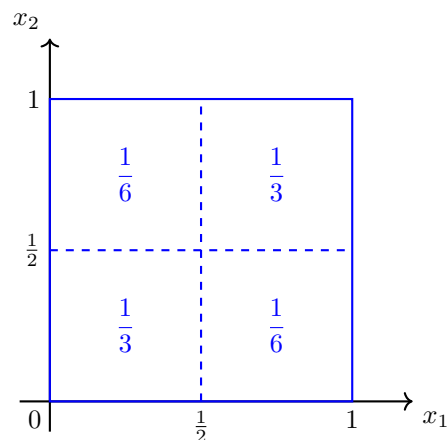
Because the algorithm fires one event at a time, its computational cost is proportional to the number of reaction events. This is fine when molecule counts are small, but becomes prohibitive for large populations or fast rates (e.g., 10^{23} molecules). Tau-leaping (Gillespie, 2001) addresses this by leaping forward a larger time step Δt and firing approximately $a_\mu \Delta t$ reactions per channel (drawn from Poisson distributions) rather than resolving each event individually. As Δt grows, tau-leaping smoothly interpolates between the exact kMC and the deterministic ODE limit.

7.7 Exercises

Exercise 7.1

Write a MATLAB script that uses rejection sampling to draw samples from the probability density function over $\mathbf{x} \in \mathbb{R}^2$ with PMF:

$$p(\mathbf{x}) = \begin{cases} 1/3, & x_1, x_2 \leq 1/2 \text{ or } x_1, x_2 > 1/2 \\ 1/6, & x_1 \leq 1/2, x_2 > 1/2 \text{ or } x_1 > 1/2, x_2 \leq 1/2 \\ 0, & x_1, x_2 \notin [0, 1] \end{cases}$$



Plot a histogram of 1000 samples.

Exercise 7.2

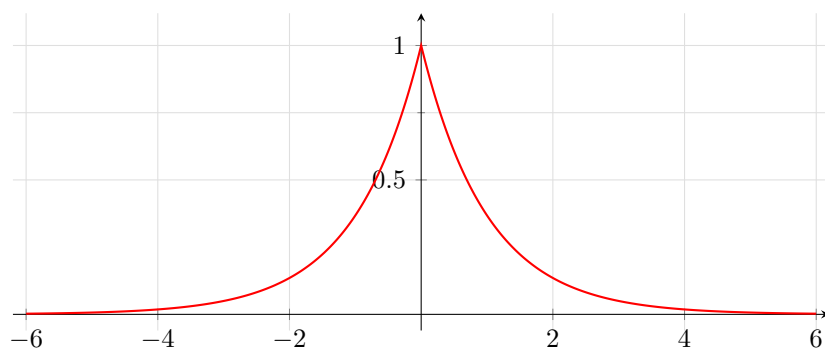
For a target distribution: $\pi(\mathbf{x}) \propto \frac{1}{1 + |\mathbf{x}|^2}$, with $\mathbf{x} \in \mathbb{R}^2$, write a MATLAB script that uses a Markov chain to draw samples. Plot a histogram of 1000 samples.

Exercise 7.3

A Slater-type orbital is a PDF that is sometimes used to model atomic orbitals in computational chemistry. One Slater-type orbital is described by

$$R(x) = N \exp(-|x|),$$

where N is the normalization constant. Analytically we know $N = 1/2$.



- (a) The following code aims to estimate $1/N$, but it consistently underestimates. (The estimates are usually around 1.7.) Why?

```

N_samples = 1e6;
f = @(x) exp(-abs(x));
samples = -2 + 4*rand(N_samples,1);
f_vals = 4 * f(samples);
N_estimate = mean(f_vals);
fprintf('%.6f\n', N_estimate);

```

- (b) The following code aims to estimate $1/N$, but it consistently underestimates. (The estimates are usually around 0.52.) Why?

```

N_samples = 1e6;
f = @(x) exp(-abs(x));
samples = randn(N_samples,1);
f_vals = f(samples);
N_estimate = mean(f_vals);
fprintf('%.6f\n', N_estimate);

```

- (c) Does the following code correctly sample points from $R(x)$?

```

N_samples = 1e6;
pi = @(x) exp(-abs(x));
A = @(x,y) min(1, pi(x) / pi(y));
samples = zeros(N_samples,1);
for i = 1:N_samples
    xstar = samples(i) + (2*rand(1) - 1);
    if (rand(1) < A(xstar, samples(i)))
        samples(i+1) = xstar;
    else
        samples(i+1) = samples(i);
    end
end

```

- (d) Assuming we have correctly sampled points from $R(x)$, can we use these samples to estimate N ? Explain.

Exercise 7.4

Consider the following code:

```

function samples = MCMC(g, x0)
M = 100000;
samples = zeros(1, M);
samples(1) = x0;
j = 2;
while j <= M
    proposal = samples(j-1) + 2*rand()-1;
    u = rand();
    if u < min(1, g(proposal)/g(samples(j-1)))
        samples(j) = proposal;
    else
        samples(j) = samples(j-1);
    end
    j = j + 1;
end

```

```

end

g = @(x) 0; % ???
f = @(x) 0; % ???
samples = MCMC(g, 0);
mean(f(samples))

```

Can we use this code to evaluate the integrals below? For each integral, answer whether it is possible: if so, specify how f and g should be defined; if not, explain why. As a bonus, also find the integral value by inspection of the integrand.

(a) $I = \int_{\mathbb{R}} x \cdot \frac{e^{-x^2}}{Z} dx, \quad Z = \int_{\mathbb{R}} e^{-x^2} dx$

(b) $I = \int_{\mathbb{R}} \frac{e^{-x^2}}{Z} dx, \quad Z = \int_{\mathbb{R}} e^{-x^2} dx$

(c) $I = \int_{\mathbb{R}} e^{-x^2} dx$

Exercise 7.5

In the Metropolis-Hastings algorithm, we first choose a proposal (transition) distribution $M(\mathbf{x}^*, \mathbf{x}_i)$, which gives the probability of proposing the candidate state $\mathbf{x}^*$ from the current state $\mathbf{x}_i$, and then construct the acceptance probability

$$A(\mathbf{x}^*, \mathbf{x}_i) = \min \left\{ 1, \frac{\pi(\mathbf{x}^*) M(\mathbf{x}_i, \mathbf{x}^*)}{\pi(\mathbf{x}_i) M(\mathbf{x}^*, \mathbf{x}_i)} \right\},$$

where π is the target distribution. For each of the following choices of M , describe what issues we would encounter (if any) and any potential solutions.

- (a) M is not symmetric, $M(\mathbf{x}^*, \mathbf{x}_i) \neq M(\mathbf{x}_i, \mathbf{x}^*)$
- (b) M is not "irreducible" (can't get to every other $\mathbf{x}^*$ from $\mathbf{x}_i$)

Chapter 8

Numerical Solution of Ordinary Differential Equations

8.1 Ordinary Differential Equations: Initial Value Problems (ODE-IVPs)

Many physical processes that we model in engineering and science are dynamic. This means that first-principles models describing these processes can be formulated as differential equations, which describe how system variables change with respect to time or space. We can express a general dynamic model as a system of first-order ordinary differential equations (ODEs):

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t), t; \boldsymbol{\theta}) \quad (8.1.1)$$

This system is coupled with an initial condition, which specifies the state of the system at a starting time t_0 :

$$\mathbf{x}(t_0) = \mathbf{x}_0 \quad (8.1.2)$$

Here, $\mathbf{x}(t)$ is the state vector, which contains the set of dynamic variables we want to solve for; t is time; $\mathbf{u}(t)$ is a vector of time-dependent inputs that we specify (e.g., an external force), which is sometimes considered part of $\mathbf{f}$; $\boldsymbol{\theta}$ is a vector of time-independent parameters (e.g., mass, reaction constants); and $\mathbf{x}_0$ is the vector of initial values for the state variables at time t_0 .

Usually, we are interested in finding the solution (the values of the state vector $\mathbf{x}(t)$) within some time domain, $t \in [t_0, t_f]$. This task is formally known as the **initial value problem (IVP)**. For simplicity, we will often group the inputs $\mathbf{u}(t)$ and parameters $\boldsymbol{\theta}$ into the function $\mathbf{f}$ and write the IVP more compactly:

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{f}(\mathbf{x}(t), t) \quad \forall t \in [t_0, t_f], \text{ given } \mathbf{x}(t_0) = \mathbf{x}_0 \quad (8.1.3)$$

By convention, the initial time t_0 is often set to zero.

Example: Consecutive Reaction System

Consider a series of first-order reactions $A \xrightarrow{k_1} B \xrightarrow{k_2} C$ in a batch reactor. Define the state vector as $\mathbf{x}(t) = [C_A(t), C_B(t), C_C(t)]^\top$, where C_i represents the concentration of species i . The system is

described by

$$\frac{d\mathbf{x}}{dt} = \begin{bmatrix} \dot{C}_A \\ \dot{C}_B \\ \dot{C}_C \end{bmatrix} = \begin{bmatrix} -k_1 C_A \\ k_1 C_A - k_2 C_B \\ k_2 C_B \end{bmatrix} \quad (8.1.4)$$

with rate constants $\boldsymbol{\theta} = [k_1, k_2]^\top$ and initial concentrations $\mathbf{x}_0 = [C_{A,0}, C_{B,0}, C_{C,0}]^\top$ at $t_0 = 0$.

The challenge comes from the function $\mathbf{f}(\mathbf{x}(t), t)$. Since $\mathbf{f}$ can be an arbitrary nonlinear function of the state vector, a closed-form, analytical solution for $\mathbf{x}(t)$ rarely exists. We must therefore turn to numerical methods. The goal of a numerical method is to solve this equation by finding the state vector $\mathbf{x}$ at a finite number of discrete points within the time domain $[t_0, t_f]$. When developing and choosing these methods, we will need to characterize two important properties: the **accuracy** (how close our numerical solution is to the true, unknown solution) and the **stability** (whether errors grow uncontrollably) of the solution methods.

8.1.1 Rewriting Higher-Order ODEs

The numerical methods we will study are designed for systems of *first-order* ODEs. Fortunately, any higher-order differential equation can be rewritten as a system of first-order equations by introducing new variables for the derivatives. The procedure is very straightforward: for an n th-order ODE, define n new state variables where each represents a successive derivative of the original variable. For a second-order equation in $x(t)$, we define:

$$x_1 = x \quad (8.1.5)$$

$$x_2 = \frac{dx}{dt} \quad (8.1.6)$$

Then the n th derivative becomes a first-order equation: $dx_n/dt = (\text{expression from the original ODE})$.

Example: Driven Mass-Spring-Damper

The second-order equation $m\ddot{x} + b\dot{x} + kx = f(t)$ (dots are time derivatives, a common notation in physics/engineering) can be rewritten by defining the state vector $\mathbf{x}(t) = [x(t), v(t)]^\top$ where $v = \dot{x}$. This gives the first-order system:

$$\frac{d\mathbf{x}}{dt} = \begin{bmatrix} \frac{dx}{dt} \\ \frac{dv}{dt} \end{bmatrix} = \begin{bmatrix} v \\ \frac{f(t) - bv - kx}{m} \end{bmatrix} \quad (8.1.7)$$

with initial condition $\mathbf{x}_0 = [x(t_0), v(t_0)]^\top$.

This pattern generalizes to ODEs of any order: simply add new state variables for each derivative up to order $n - 1$.

8.1.2 Local Truncation Error

To understand the accuracy of a method, we must first estimate the local truncation error (LTE). This is the error we introduce in a single step, assuming that the solution we start with at t_i , $\mathbf{x}(t_i)$, is perfectly accurate (i.e., $\mathbf{x}(t_i) = \mathbf{x}_{\text{exact}}(t_i)$). We can find the exact value of the solution at t_{i+1} by using a Taylor expansion of $\mathbf{x}_{\text{exact}}(t)$ around the point t_i :

$$\mathbf{x}_{\text{exact}}(t_{i+1}) = \mathbf{x}_{\text{exact}}(t_i) + \sum_{j=1}^{\infty} \frac{1}{j!} (\Delta t_i)^j \left. \frac{d^j \mathbf{x}}{dt^j} \right|_{t_i} \quad (8.1.8)$$

Our numerical method provides an approximation $\mathbf{x}(t_{i+1})$. The local truncation error is the difference between this exact value and our approximation.

The **local truncation error**, ϵ , is defined as the magnitude (norm) of the difference between the exact solution at t_{i+1} and the numerical approximation at t_{i+1} , assuming the numerical solution was exact at t_i :

$$\epsilon = \|\mathbf{x}_{\text{exact}}(t_{i+1}) - \mathbf{x}(t_{i+1})\| \quad (8.1.9)$$

We typically characterize this error using big-O notation to describe how it scales with the step size Δt_i . We say the local truncation error is of order p , or $O(\Delta t_i^p)$, if the following limit exists for some constant $C > 0$:

$$\lim_{\Delta t_i \rightarrow 0^+} \frac{\|\mathbf{x}_{\text{exact}}(t_{i+1}) - \mathbf{x}(t_{i+1})\|}{(\Delta t_i)^p} = C \quad (8.1.10)$$

A method with LTE of $O(\Delta t_i^p)$ is said to be $(p - 1)$ -th order accurate.

8.2 Explicit Methods for Solving ODE-IVPs

Numerical methods for IVPs work by stepping forward in time. We start at t_i with a known solution $\mathbf{x}(t_i)$ and try to approximate the solution at a future point in time, t_{i+1} . An **explicit method** calculates the future state $\mathbf{x}(t_{i+1})$ using only information from the current (or previous) time steps. The general form for a one-step explicit method is:

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t_i \mathbf{g}(\mathbf{x}(t_i), t_i, \Delta t_i) \quad (8.2.1)$$

Here, $\Delta t_i = t_{i+1} - t_i$ is the step size, which controls the local truncation error in the solution. The function $\mathbf{g}(\mathbf{x}(t_i), t_i, \Delta t_i)$ is the workhorse of the method. It is designed to be a good approximation of the change in $\mathbf{x}$ over the step. It has the goal of minimizing the error at a given Δt_i while also maintaining numerical stability.

8.2.1 The Explicit (Forward) Euler Method

The simplest explicit method is the explicit Euler (also known as forward Euler) method. It is defined by the formula

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t_i \mathbf{f}(\mathbf{x}(t_i), t_i) \quad (8.2.2)$$

This is an instance of our general explicit form where the function $\mathbf{g}$ is simply $\mathbf{g}(\mathbf{x}(t_i), t_i, \Delta t_i) = \mathbf{f}(\mathbf{x}(t_i), t_i)$. This method can be derived directly from a forward finite difference approximation for the derivative $\mathbf{x}'(t)$ at time t_i :

$$\left. \frac{d\mathbf{x}}{dt} \right|_{t_i} \approx \frac{\mathbf{x}(t_{i+1}) - \mathbf{x}(t_i)}{\Delta t_i} \quad (8.2.3)$$

Since we know that $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$ (this is the definition of the function $\mathbf{f}$ as per [Eq. 8.1.3](#)), we can set them equal:

$$\mathbf{f}(\mathbf{x}(t_i), t_i) \approx \frac{\mathbf{x}(t_{i+1}) - \mathbf{x}(t_i)}{\Delta t_i} \quad (8.2.4)$$

Rearranging for $\mathbf{x}(t_{i+1})$ gives the forward Euler formula. Visually, this method can be understood by looking at the integral form of the ODE. The exact solution is:

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \int_{t_i}^{t_{i+1}} \mathbf{f}(\mathbf{x}(t'), t') dt' \quad (8.2.5)$$

The forward Euler method approximates this integral by assuming the integrand, $\mathbf{f}(\mathbf{x}(t'), t')$, is constant over the entire interval $[t_i, t_{i+1}]$ and is equal to its value at the start of the interval, $\mathbf{f}(\mathbf{x}(t_i), t_i)$. This is the simplest rectangular approximation (a zero-order hold) of the integral. The method, therefore, takes a step from $\mathbf{x}(t_i)$ by moving in the direction of the tangent $\mathbf{f}(\mathbf{x}(t_i), t_i)$ for a duration of Δt_i .

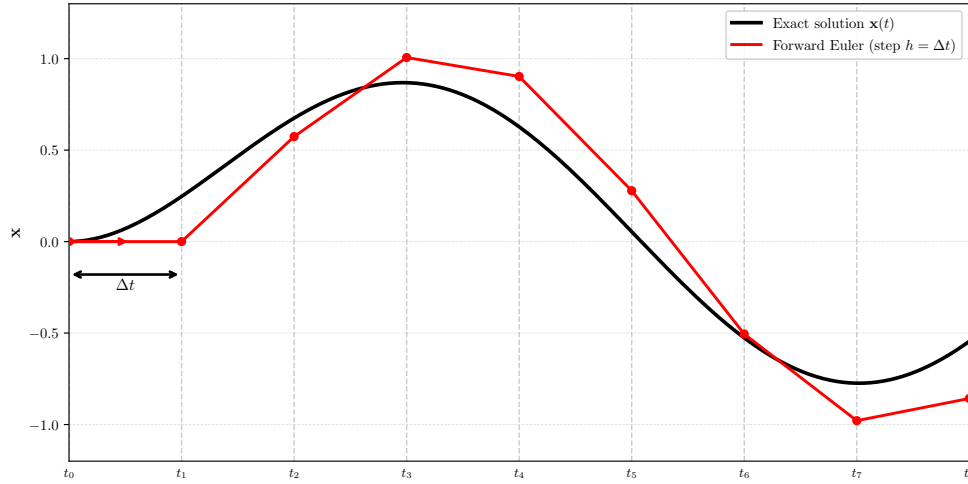


Figure 8.2.1: Forward (explicit) Euler visualization. The exact solution $\mathbf{x}(t)$ (black) is compared with the forward Euler approximation (red) using step $h = \Delta t$. At each grid point t_i , the method advances via $\mathbf{x}_{i+1} = \mathbf{x}_i + \Delta t \mathbf{f}(\mathbf{x}_i, t_i)$ using the slope at the left endpoint.

Local Error of Euler's Method Let's estimate the local truncation error for this method. We again start with the Taylor expansion of the exact solution:

$$\mathbf{x}_{\text{exact}}(t_{i+1}) = \mathbf{x}_{\text{exact}}(t_i) + \Delta t_i \left. \frac{d\mathbf{x}}{dt} \right|_{t_i} + \frac{1}{2}(\Delta t_i)^2 \left. \frac{d^2\mathbf{x}}{dt^2} \right|_{t_i} + O(\Delta t_i^3) \quad (8.2.6)$$

By our problem definition, $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$. Assuming $\mathbf{x}(t_i) = \mathbf{x}_{\text{exact}}(t_i)$, we have

$$\left. \frac{d\mathbf{x}}{dt} \right|_{t_i} = \mathbf{f}(\mathbf{x}_{\text{exact}}(t_i), t_i) = \mathbf{f}(\mathbf{x}(t_i), t_i) \quad (8.2.7)$$

Substituting this into the expansion gives

$$\mathbf{x}_{\text{exact}}(t_{i+1}) = \underbrace{\mathbf{x}(t_i) + \Delta t_i \mathbf{f}(\mathbf{x}(t_i), t_i)}_{\text{forward Euler approx. } \mathbf{x}(t_{i+1})} + \frac{1}{2}(\Delta t_i)^2 \left. \frac{d^2\mathbf{x}}{dt^2} \right|_{t_i} + O(\Delta t_i^3) \quad (8.2.8)$$

The first two terms are precisely the forward Euler approximation $\mathbf{x}(t_{i+1})$. The local truncation error ϵ is therefore the remainder:

$$\epsilon = \|\mathbf{x}_{\text{exact}}(t_{i+1}) - \mathbf{x}(t_{i+1})\| = \left\| \frac{1}{2}(\Delta t_i)^2 \left. \frac{d^2\mathbf{x}}{dt^2} \right|_{t_i} + O(\Delta t_i^3) \right\| \quad (8.2.9)$$

Thus, the local truncation error is $O(\Delta t_i^2)$. As $\Delta t_i \rightarrow 0$, the ratio $\epsilon/(\Delta t_i)^2$ approaches a constant $C > 0$. This is the error from just one step. What about the global error? Over many iterations of this method, how bad does our solution become?

Global Error of Euler's Method Let's assume a constant step size Δt . At each step, we introduce a local error of $O(\Delta t^2)$.

$$\mathbf{x}(t_1) = \mathbf{x}(t_0) + \Delta t \mathbf{f}(\mathbf{x}_0, t_0) = \mathbf{x}_{\text{exact}}(t_1) + O(\Delta t^2) \quad (8.2.10)$$

$$\begin{aligned} \mathbf{x}(t_2) &= \mathbf{x}(t_1) + \Delta t \mathbf{f}(\mathbf{x}(t_1), t_1) \\ &= (\mathbf{x}_{\text{exact}}(t_1) + O(\Delta t^2)) + \Delta t \mathbf{f}(\mathbf{x}_{\text{exact}}(t_1) + O(\Delta t^2), t_1) \end{aligned} \quad (8.2.11)$$

But $\mathbf{f}(\mathbf{x} + \boldsymbol{\delta}) \approx \mathbf{f}(\mathbf{x}) + O(\boldsymbol{\delta})$ (with appropriate continuity assumptions). So,

$$\mathbf{f}(\mathbf{x}_{\text{exact}}(t_1) + O(\Delta t^2), t_1) = \mathbf{f}(\mathbf{x}_{\text{exact}}(t_1), t_1) + O(\Delta t^2) \quad (8.2.12)$$

and therefore

$$\begin{aligned} \mathbf{x}(t_2) &= (\mathbf{x}_{\text{exact}}(t_1) + O(\Delta t^2)) + \Delta t(\mathbf{f}(\mathbf{x}_{\text{exact}}(t_1), t_1) + O(\Delta t^2)) \\ &= [\mathbf{x}_{\text{exact}}(t_1) + \Delta t\mathbf{f}(\mathbf{x}_{\text{exact}}(t_1), t_1)] + O(\Delta t^2) + O(\Delta t^3) \\ &= [\mathbf{x}_{\text{exact}}(t_2) + O(\Delta t^2)] + O(\Delta t^2) \\ &= \mathbf{x}_{\text{exact}}(t_2) + O(2\Delta t^2) \end{aligned} \quad (8.2.13)$$

After N steps to reach a final time $t_f = t_0 + N\Delta t$, the accumulated error will be

$$\mathbf{x}(t_N) = \mathbf{x}_{\text{exact}}(t_N) + O(N\Delta t^2) \quad (8.2.14)$$

This tells us how the error accumulates. To find the global error in terms of Δt , we note that for a fixed time interval $[t_0, t_f]$, the number of steps N is $N = (t_f - t_0)/\Delta t$. Therefore, N is proportional to $1/\Delta t$, or $N \sim 1/\Delta t$. Therefore, the global accumulated error is $O(N\Delta t^2) = O((1/\Delta t)\Delta t^2) = O(\Delta t)$. Because the global error scales linearly with Δt , the forward Euler method is a first-order method. This means that if you halve your step size Δt , you should expect to halve your global error.

Interpreting the Errors

It can be a little confusing to interpret the errors when you see them written as $O(\Delta t^2)$ and $O(\Delta t)$. Let's unpack this a bit. Since step sizes are small ($\Delta t < 1$), higher powers of Δt give us *smaller* errors. For example, if $\Delta t = 0.1$, then $\Delta t^2 = 0.01$ is much smaller than $\Delta t = 0.1$. This means the local error behavior $O(\Delta t^2)$ is better (smaller) than the global error behavior $O(\Delta t)$. If you halve your step size Δt , the local error decreases by a factor of 4, but the global error only decreases by a factor of 2. Also, you should keep in mind that the error analysis is asymptotic in the small step size limit; that is, it describes behavior as $\Delta t \rightarrow 0$. This is in contrast to asymptotics we saw earlier in the course that were in the large limit.

Stability of Euler's Method We have analyzed the error of the forward Euler method, but this is only half the story. The other property we need to look at is stability. A method is unstable if small errors introduced at one step (like round-off error or the local truncation error itself) grow unboundedly in subsequent steps, eventually destroying the solution. To analyze stability, we use a simple test problem.

Let's identify a 1D test problem. We'll use the simple, linear ODE

$$\frac{dx}{dt} = \lambda x, \quad x(0) = 1 \quad (8.2.15)$$

Here, λ is a (potentially complex) constant. The analytical solution is $x(t) = e^{\lambda t}$. This analytical solution decays to zero when $\text{Re}(\lambda) < 0$, and is neutrally stable (bounded but non-decaying) when $\text{Re}(\lambda) = 0$. Now, when is the numerical solution stable? We apply the explicit Euler step to this problem:

$$\begin{aligned} x(t_{i+1}) &= x(t_i) + \Delta t f(x(t_i), t_i) \\ &= x(t_i) + \Delta t(\lambda x(t_i)) \\ &= (1 + \lambda \Delta t)x(t_i) \end{aligned}$$

If we apply this recursively back to the initial condition $x(t_0)$, we find that

$$x(t_{i+1}) = (1 + \lambda \Delta t)x(t_i) = (1 + \lambda \Delta t)^2 x(t_{i-1}) = \cdots = (1 + \lambda \Delta t)^{i+1} x(t_0) \quad (8.2.16)$$

For this numerical solution to remain bounded (i.e., not explode to infinity) as i increases, the magnitude of the growth factor $(1 + \lambda \Delta t)$ must be less than or equal to one. This gives us the stability condition

$$|1 + \lambda \Delta t| \leq 1 \quad (8.2.17)$$

This concept extends to N-dimensional (ND) systems. Consider the ND linear test problem:

$$\frac{d\mathbf{x}}{dt} = \mathbf{A}\mathbf{x}(t) \quad (8.2.18)$$

where $\mathbf{A}$ is a constant matrix. Applying the forward Euler method gives

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t \mathbf{A} \mathbf{x}(t_i) = (\mathbf{I} + \Delta t \mathbf{A}) \mathbf{x}(t_i) \quad (8.2.19)$$

To analyze the stability, we can decouple this system by diagonalizing the matrix $\mathbf{A}$ (assuming it is diagonalizable) as $\mathbf{A} = \mathbf{W}\mathbf{\Lambda}\mathbf{W}^{-1}$, where $\mathbf{\Lambda}$ is the diagonal matrix of eigenvalues λ_j and the columns of $\mathbf{W}$ are the corresponding eigenvectors. By defining a new set of variables $\mathbf{y}(t_i) = \mathbf{W}^{-1}\mathbf{x}(t_i)$, the system becomes a set of independent, 1D problems:

$$y_j(t_{i+1}) = (1 + \lambda_j \Delta t) y_j(t_i) \quad (8.2.20)$$

If the true solution is to decay to zero, we require $\text{Re}(\lambda_j) < 0$ for all eigenvalues. For the numerical solution to also decay to zero, we must satisfy the 1D stability condition for *every* eigenvalue:

$$\frac{|y_j(t_{i+1})|}{|y_j(t_i)|} \leq |1 + \lambda_j \Delta t| \leq 1 \quad (8.2.21)$$

The stability of the method depends on the product $z_j = \lambda_j \Delta t$, which is a complex number. The condition $|1 + z_j| \leq 1$ defines a **region of absolute stability** in the complex plane. We can find its shape:

$$|1 + \lambda_j \Delta t|^2 = (1 + \text{Re}(\lambda_j \Delta t))^2 + (\text{Im}(\lambda_j \Delta t))^2 \leq 1 \quad (8.2.22)$$

This inequality describes a circle in the complex plane centered at $(-1, 0)$ with a radius of 1 (Figure 8.2.2).

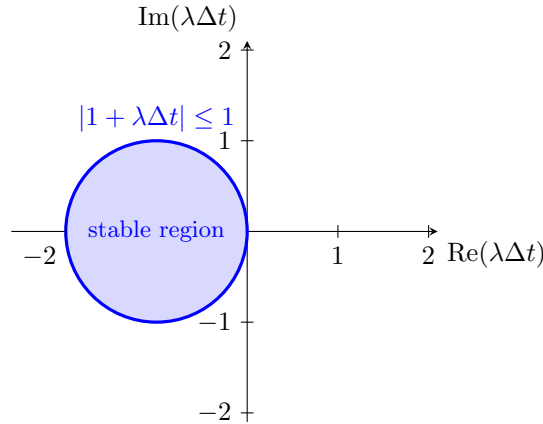


Figure 8.2.2: Region of absolute stability of forward (explicit) Euler: the unit disk centered at -1 in the $z = \lambda \Delta t$ plane.

So, the numerical method is only stable if the scaled step, $z_j = \lambda_j \Delta t$, falls *inside* this circular region for all eigenvalues λ_j of the system. This is a significant problem. The true system is stable for any λ_j in the entire left-half of the complex plane ($\text{Re}(\lambda_j) \leq 0$), but our numerical method is only stable for a small circular patch within that region. This means that even if the true solution is stable, our numerical solution can become unstable and explode if we choose a step size Δt that is too large. This is known as conditional stability.

If λ is real and negative (e.g., in a simple decay problem), we can define the characteristic time-scale of the problem as $\tau = -1/\lambda$. The stability condition becomes $|1 - \Delta t/\tau| \leq 1$, which simplifies to

$$0 \leq \Delta t \leq 2\tau \quad (8.2.23)$$

The step size must be less than twice the characteristic time-scale of the system or the solution will become unstable.

8.2.2 The Midpoint Method

The forward Euler method is simple but has a large global error ($O(\Delta t)$) and a very restrictive stability region. We can develop more accurate methods. Let's consider another explicit method: the midpoint method. The midpoint method is a two-stage process. First, it takes a trial half-step using forward Euler to estimate the state at the midpoint of the time interval. Then, it uses the derivative at that estimated midpoint to take the full step from t_i to t_{i+1} . This is visualized in Figure 8.2.3.

The Midpoint Method

Given $\mathbf{x}(t_i)$, find $\mathbf{x}(t_{i+1})$ by doing the following:

1. Estimate midpoint: $\mathbf{y} = \mathbf{x}(t_i) + \frac{\Delta t_i}{2} \mathbf{f}(\mathbf{x}(t_i), t_i)$
2. Take full step: $\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t_i \mathbf{f}(\mathbf{y}, t_i + \Delta t_i/2)$

This can be written in our general form $\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t_i \mathbf{g}(\mathbf{x}(t_i), t_i, \Delta t_i)$, where the function $\mathbf{g}$ is

$$\mathbf{g}(\mathbf{x}(t_i), t_i, \Delta t_i) = \mathbf{f}\left(\mathbf{x}(t_i) + \frac{\Delta t_i}{2} \mathbf{f}(\mathbf{x}(t_i), t_i), t_i + \Delta t_i/2\right) \quad (8.2.24)$$

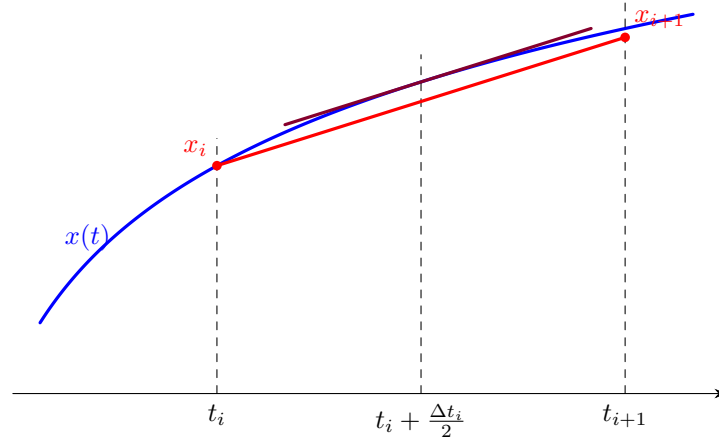


Figure 8.2.3: Midpoint method visualization. To improve over forward Euler, evaluate the derivative at the midpoint ($t_i + \Delta t_i/2$) (purple) and step from (x_i) to (x_{i+1}) (red) toward the exact solution ($x(t)$) (blue).

Local Truncation Error of the Midpoint Method Let's estimate the LTE of this method. We need to compare the Taylor expansion of $\mathbf{g}$ with the Taylor expansion of the exact solution. First, let's expand $\mathbf{g}$ with respect to Δt_i around $\Delta t_i = 0$:

$$\begin{aligned} \mathbf{g}(\mathbf{x}(t_i), t_i, \Delta t_i) &= \mathbf{f}(\mathbf{x}(t_i), t_i) + \Delta t_i \left. \frac{\partial \mathbf{g}}{\partial \Delta t_i} \right|_{\Delta t_i=0} + O(\Delta t_i^2) \\ &= \mathbf{f}(\mathbf{x}(t_i), t_i) + \frac{\Delta t_i}{2} \left. \frac{d\mathbf{f}}{dt} \right|_{\mathbf{x}(t_i), t_i} + O(\Delta t_i^2) \\ &= \mathbf{f}(\mathbf{x}(t_i), t_i) + \frac{\Delta t_i}{2} \left. \frac{d^2 \mathbf{x}}{dt^2} \right|_{t_i} + O(\Delta t_i^2) \end{aligned}$$

Now, our numerical approximation is $\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t_i \mathbf{g}(\dots)$. Substituting our expansion for $\mathbf{g}$, we get

$$\begin{aligned}\mathbf{x}(t_{i+1}) &= \mathbf{x}(t_i) + \Delta t_i \left(\mathbf{f}(\mathbf{x}(t_i), t_i) + \frac{\Delta t_i}{2} \frac{d^2 \mathbf{x}}{dt^2} \Big|_{t_i} + O(\Delta t_i^2) \right) \\ &= \mathbf{x}(t_i) + \Delta t_i \mathbf{f}(\mathbf{x}(t_i), t_i) + \frac{(\Delta t_i)^2}{2} \frac{d^2 \mathbf{x}}{dt^2} \Big|_{t_i} + O(\Delta t_i^3)\end{aligned}$$

Comparing this to the Taylor expansion of the *exact* solution, we have

$$\mathbf{x}_{\text{exact}}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t_i \frac{d\mathbf{x}}{dt} \Big|_{t_i} + \frac{(\Delta t_i)^2}{2} \frac{d^2 \mathbf{x}}{dt^2} \Big|_{t_i} + \frac{(\Delta t_i)^3}{6} \frac{d^3 \mathbf{x}}{dt^3} \Big|_{t_i} + \dots \quad (8.2.25)$$

Since $\mathbf{f}(\mathbf{x}(t_i), t_i) = \frac{d\mathbf{x}}{dt} \Big|_{t_i}$, we can see that the first three terms (the $O(1)$, $O(\Delta t_i)$, and $O(\Delta t_i^2)$ terms) of our numerical approximation and the exact solution match perfectly. The local truncation error $\epsilon = \|\mathbf{x}_{\text{exact}}(t_{i+1}) - \mathbf{x}(t_{i+1})\|$ is therefore the first term that *doesn't* match, which comes from the $O(\Delta t_i^3)$ term.

$$\epsilon = \left\| \sum_{j=3}^{\infty} \frac{1}{j!} \Delta t_i^j \frac{d^j \mathbf{x}}{dt^j} \Big|_{t_i} - \Delta t_i O(\Delta t_i^2) \right\| \sim O(\Delta t_i^3) \quad (8.2.26)$$

This local error is one order higher than forward Euler's ($O(\Delta t_i^2)$). This means the global error of the midpoint method is $O(\Delta t^2)$, making it a second-order method.

Stability of the Midpoint Method Let's analyze the stability of the midpoint method using the same ND test problem, $\mathbf{f}(\mathbf{x}(t), t) = \mathbf{A}\mathbf{x}(t)$. First, we find the intermediate step $\mathbf{y}$:

$$\mathbf{y} = \mathbf{x}(t_i) + \frac{\Delta t_i}{2} \mathbf{A}\mathbf{x}(t_i) = \left(\mathbf{I} + \frac{\Delta t_i}{2} \mathbf{A} \right) \mathbf{x}(t_i) \quad (8.2.27)$$

Next, we find the full step $\mathbf{x}(t_{i+1})$:

$$\begin{aligned}\mathbf{x}(t_{i+1}) &= \mathbf{x}(t_i) + \Delta t_i \mathbf{A}\mathbf{y} \\ &= \mathbf{x}(t_i) + \Delta t_i \mathbf{A} \left[\left(\mathbf{I} + \frac{\Delta t_i}{2} \mathbf{A} \right) \mathbf{x}(t_i) \right] \\ &= \left[\mathbf{I} + \Delta t_i \mathbf{A} + \frac{(\Delta t_i)^2}{2} \mathbf{A}^2 \right] \mathbf{x}(t_i)\end{aligned} \quad (8.2.28)$$

Again, we can analyze this by looking at the eigenvalues λ_j of $\mathbf{A}$. The stability condition for each decoupled component $y_j(t_i)$ is

$$y_j(t_{i+1}) = \left[1 + \lambda_j \Delta t_i + \frac{(\lambda_j \Delta t_i)^2}{2} \right] y_j(t_i) \quad (8.2.29)$$

For the numerical solution to be stable, the growth factor must have a magnitude less than or equal to 1. The condition is

$$\left| 1 + \lambda_j \Delta t_i + \frac{(\lambda_j \Delta t_i)^2}{2} \right| \leq 1 \quad (8.2.30)$$

Plotting this region in the complex plane shows a bean-like shape (see [Figure 8.2.4](#)). The midpoint method has a larger stability region in the complex plane (its region strictly contains the forward Euler disk). However, along the negative real axis, both methods share the same stability interval $[0, 2/|\lambda|]$ for the scalar test problem. Note that purely imaginary eigenvalues (except at the origin) lie outside the stability region, which makes the midpoint method unsuitable for undamped oscillatory problems.

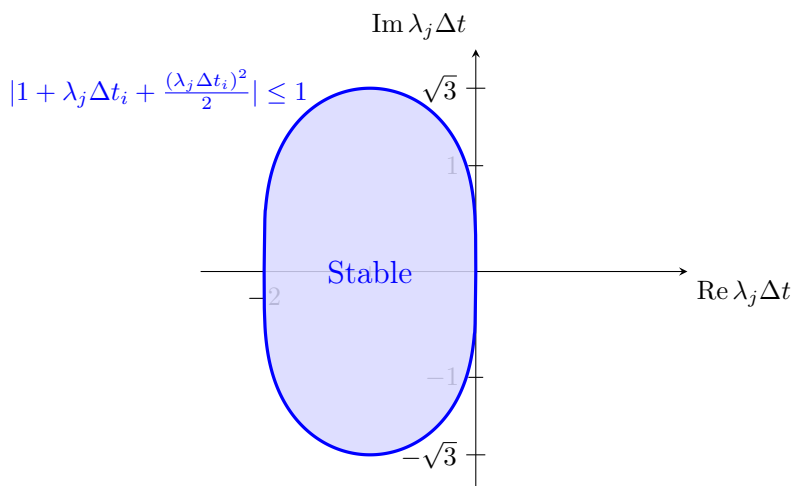


Figure 8.2.4: Region of absolute stability of the explicit midpoint method in the $z = \lambda_j \Delta t$ plane. The boundary passes through $z = -2, 0$ and has extrema at $z = -1 \pm i\sqrt{3}$.

8.2.3 Comparing Explicit Methods: Euler vs. Midpoint

We now have two methods: forward Euler (first-order, $O(\Delta t)$ global error) and midpoint (second-order, $O(\Delta t^2)$ global error). Let's compare them on the test problem

$$\frac{dy}{dt} = -y, \quad y(0) = 1, \quad (8.2.31)$$

whose exact solution is $y(t) = e^{-t}$.

Accuracy at $t = 1$ (global error vs. Δt). We measure the global error at a fixed time, $|y(1) - e^{-1}|$, while varying the step size Δt , and plot it on the log-log plot in Figure 8.2.5. Forward Euler (red) has slope ≈ 1 (error $\propto \Delta t$) and the midpoint method (blue) has slope ≈ 2 (error $\propto \Delta t^2$). This means halving Δt roughly halves the Euler error but quarters the midpoint error. Thus, for the same Δt , midpoint is significantly more accurate.

Stability over time (solutions $y(t)$ for various Δt). Figure 8.2.6 plots $y(t)$ produced by each method for several step sizes; the true solution is the dotted black curve. For small steps (e.g., $\Delta t = 0.01, 0.05, 0.65$) both methods track e^{-t} well. The stability question becomes interesting near the boundary:

$$\text{Forward Euler: } r_E(\Delta t) = 1 - \Delta t, \quad \text{Midpoint: } r_M(\Delta t) = 1 - \Delta t + \frac{1}{2}\Delta t^2$$

For this scalar test with $\lambda = -1$, absolute stability requires $|r(\Delta t)| \leq 1$, which holds for $0 \leq \Delta t \leq 2$ for both methods. At the edge,

- $\Delta t = 2$: Euler is marginally stable with $r_E = -1 \implies y_n = (-1)^n$ (alternating, no decay); midpoint is marginally stable with $r_M = 1 \implies y_n = 1$ (flat, no decay).
- $\Delta t > 2$ (e.g., 2.5): both methods are unstable and the numerical solutions diverge.

Thus, for this problem, the stable interval on the negative real axis is the same ($\Delta t \leq 2$), but the qualitative behavior at the boundary differs. Euler flips sign each step while midpoint holds its value. To obtain asymptotic decay toward zero (as desired), we need $\Delta t < 2$.

8.2.4 Runge-Kutta Methods

The explicit/forward Euler method and the midpoint method are both examples of a much broader and more powerful class of numerical integrators called Runge-Kutta (RK) methods. These methods are the go-to

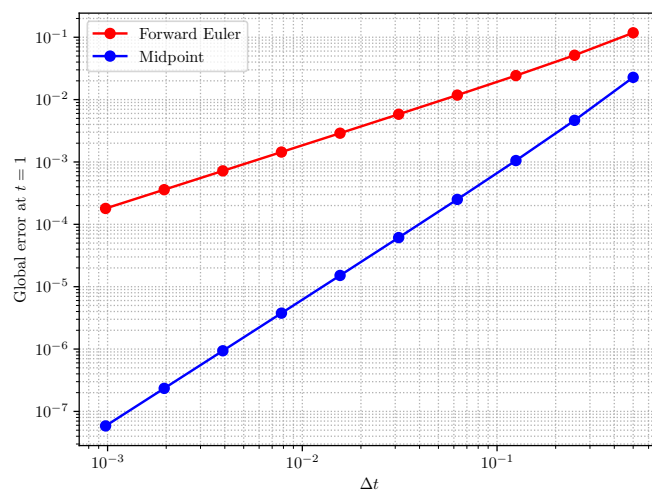


Figure 8.2.5: Global error at $t = 1$ vs. Δt (log-log). Forward Euler (red) shows first-order decay; midpoint (blue) shows second-order decay.

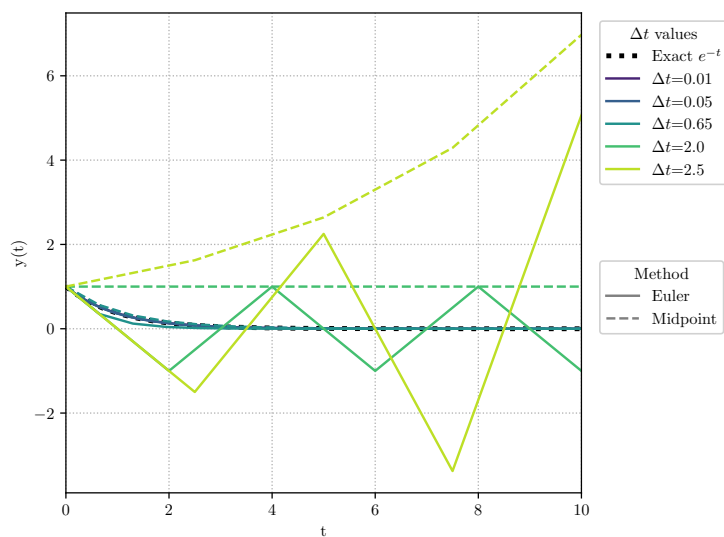


Figure 8.2.6: Time-series stability comparison. The exact solution (dotted black) and numerical solutions for several Δt values; colors encode Δt , with solid lines for Euler and dashed lines for midpoint. At $\Delta t = 2$ both methods are marginal (no decay); at $\Delta t = 2.5$ both diverge.

explicit methods for solving ODE-IVPs. They work by computing one or more intermediate stages (like the $\mathbf{y}$ vector in the midpoint method) and then using a weighted average of the derivatives at these stages to compute the final step.

An s -stage explicit Runge-Kutta method computes $\mathbf{x}(t_{i+1})$ from $\mathbf{x}(t_i)$ as follows:

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t_i \sum_{j=1}^s b_j \mathbf{k}_j \quad (8.2.32)$$

where each stage $\mathbf{k}_j$ is a derivative evaluation:

$$\mathbf{k}_j = \mathbf{f} \left(\mathbf{x}(t_i) + \Delta t_i \sum_{k=1}^{j-1} a_{jk} \mathbf{k}_k, t_i + c_j \Delta t_i \right) \quad (8.2.33)$$

The method is defined by the set of coefficients: the weights $\mathbf{b} = (b_j)$, the node times $\mathbf{c} = (c_j)$, and the internal stage matrix $\mathbf{A} = (a_{jk})$. For explicit methods, $\mathbf{A}$ is strictly lower triangular ($a_{jk} = 0$ for $k \geq j$). We also require $\sum_{k=1}^{j-1} a_{jk} = c_j$ for $j = 2, \dots, s$.

The coefficients $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ are chosen (usually represented in something called a Butcher tableau) to minimize the local truncation error by matching as many terms as possible in the Taylor series expansion of the true solution. An s -stage RK method can achieve order p by satisfying the RK order conditions; the resulting LTE is $O(\Delta t^{p+1})$ and global error is $O(\Delta t^p)$. For explicit RK methods, higher order generally requires more stages (e.g., order 5 typically needs ≥ 6 stages). For example, the forward Euler method is a 1-stage RK method ($s = 1$) with $b_1 = 1$ and $c_1 = 0$ that achieves $O(\Delta t)$ global error. The midpoint method is a 2-stage RK method ($s = 2$) with coefficients $b_1 = 0, b_2 = 1, c_1 = 0, c_2 = 1/2$, and $a_{21} = 1/2$, which achieves $O(\Delta t^2)$ global error. A well-known example is RK4 (Runge-Kutta 4th-order), which uses 4 stages ($s = 4$) to achieve a very accurate global error of $O(\Delta t^4)$ and is extremely common in engineering applications.

8.3 Implicit Methods for Solving ODE-IVPs

So far, we have looked at explicit methods where the future state $\mathbf{x}(t_{i+1})$ is calculated directly from known information at the current or previous time steps. While explicit methods are easy to implement and computationally cheap per step, we found that their stability regions are bounded. This imposes a strict limit on the step size Δt ; if the step is too large, the solution explodes, even if the physics of the problem are stable. To overcome this stability limitation, we turn to **implicit methods**. The idea behind these methods is different: we approximate the solution at t_{i+1} using knowledge of the solution at t_i and the unknown solution at t_{i+1} itself. The general form for such a method is

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t_i \mathbf{g}(\mathbf{x}(t_{i+1}), \mathbf{x}(t_i), t_{i+1}, t_i, \Delta t_i) \quad (8.3.1)$$

The main difference from explicit methods is that $\mathbf{x}(t_{i+1})$ appears on both sides of the equation (inside the function $\mathbf{g}$). This means we have to *solve* for $\mathbf{x}(t_{i+1})$ rather than just plug in numbers. As with explicit methods, the step size Δt_i controls the local truncation error, and the function $\mathbf{g}$ is designed to minimize this error.

8.3.1 The Implicit (Backward) Euler Method

The simplest implicit method is the implicit Euler (or backward Euler) method. Recall that explicit Euler used the derivative at the *start* of the interval. Implicit Euler uses the derivative at the *end* of the interval:

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t_i \mathbf{f}(\mathbf{x}(t_{i+1}), t_{i+1}) \quad (8.3.2)$$

This formula is derived from the finite difference approximation of the derivative $\mathbf{x}'(t)$ at time t_{i+1} , rather than t_i . To visualize the difference between the explicit and implicit Euler methods, it is helpful to view the ODE as an integration problem. Since $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}(t), t)$, we have

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \int_{t_i}^{t_{i+1}} \mathbf{f}(\mathbf{x}(t), t) dt \quad (8.3.3)$$

If we *conceptually* simplify $\mathbf{f}(\mathbf{x}(t), t)$ to a function of time $\mathbf{f}(t)$ (ignoring the $\mathbf{x}$ dependence for a moment to visualize), we can relate the ODE solvers to Riemann sums: explicit Euler approximates the integral using the rectangle rule with the left endpoint (t_i), which corresponds to a left Riemann sum. Meanwhile, implicit Euler approximates the integral using the rectangle rule with the right endpoint (t_{i+1}), which corresponds to a right Riemann sum. Note that this is just for intuition; formally $\mathbf{f}$ depends on $\mathbf{x}$, which is what makes the method implicit.

Explicit vs. Implicit Euler: Consider a simple scalar example: $x' = -x$. The explicit Euler method gives

$$x_{i+1} = x_i + \Delta t(-x_i) = (1 - \Delta t)x_i \quad (8.3.4)$$

The implicit Euler method gives

$$x_{i+1} = x_i + \Delta t(-x_{i+1}) \quad (8.3.5)$$

In the implicit case, we must rearrange to find x_{i+1} :

$$x_{i+1}(1 + \Delta t) = x_i \implies x_{i+1} = \frac{x_i}{1 + \Delta t} \quad (8.3.6)$$

Accuracy of Implicit Euler We estimate the LTE for implicit Euler just as we did for the explicit case. The Taylor expansion of the exact solution around t_{i+1} is

$$\mathbf{x}_{\text{exact}}(t_i) = \mathbf{x}_{\text{exact}}(t_{i+1}) - \Delta t_i \left. \frac{d\mathbf{x}}{dt} \right|_{t_{i+1}} + O(\Delta t_i^2) \quad (8.3.7)$$

Rearranging for $\mathbf{x}_{\text{exact}}(t_{i+1})$ and substituting $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$, we get

$$\mathbf{x}_{\text{exact}}(t_{i+1}) = \mathbf{x}_{\text{exact}}(t_i) + \Delta t_i \mathbf{f}(\mathbf{x}_{\text{exact}}(t_{i+1}), t_{i+1}) + O(\Delta t_i^2) \quad (8.3.8)$$

Comparing this to the numerical formula, the local truncation error is

$$\epsilon = \|\mathbf{x}_{\text{exact}}(t_{i+1}) - \mathbf{x}(t_{i+1})\| \sim O(\Delta t_i^2) \quad (8.3.9)$$

As $\Delta t_i \rightarrow 0$, the ratio $\epsilon/\Delta t_i^2 \rightarrow C > 0$. (That is, there exists a constant C such that $\epsilon \approx C\Delta t_i^2$ for sufficiently small step sizes). Just like explicit Euler, the local error is quadratic ($O(\Delta t^2)$) and the global accumulated error is linear ($O(\Delta t)$). It is a first-order method. Note that because we sum $N \sim T/\Delta t$ steps, the global error is one order lower than the local error.¹

Computational Cost: Solving the Nonlinear Equations If implicit Euler has the same accuracy as explicit Euler, why use it? The answer lies in stability (discussed below), but first, we must address the cost. In explicit Euler, calculating $\mathbf{x}(t_{i+1})$ is a simple plug-and-chug operation. In implicit Euler, $\mathbf{x}(t_{i+1})$ is defined implicitly:

$$\mathbf{x}(t_{i+1}) - \Delta t_i \mathbf{f}(\mathbf{x}(t_{i+1}), t_{i+1}) - \mathbf{x}(t_i) = 0 \quad (8.3.10)$$

For a general nonlinear ODE, this requires solving a system of nonlinear algebraic equations at *every single time step*. Let the unknown be $\mathbf{y} = \mathbf{x}(t_{i+1})$. We are looking for the root of the residual function:

$$\mathbf{r}(\mathbf{y}) = \mathbf{y} - \Delta t_i \mathbf{f}(\mathbf{y}, t_{i+1}) - \mathbf{x}(t_i) = 0 \quad (8.3.11)$$

This is typically solved using Newton's method (of course!). Let's recall the scalar Newton iteration for finding a root $r(y) = 0$:

$$y^{(k+1)} = y^{(k)} - \frac{r(y^{(k)})}{r'(y^{(k)})} \quad (8.3.12)$$

¹This "sum of N local errors gives a global error one order lower" argument is only rigorous because implicit Euler is also *stable* in the numerical-analysis sense. Errors introduced at one step are not amplified without bound over subsequent steps. The precise statement combining consistency and (zero-)stability to guarantee convergence is the Dahlquist/Lax equivalence theorem; the details are not relevant for this course.

Generalizing to the vector case, we update an initial guess $\mathbf{y}^{(0)}$ by solving a linear system for the update $\Delta\mathbf{y}$:

$$\mathbf{J}(\mathbf{y}^{(k)})\Delta\mathbf{y} = -\mathbf{r}(\mathbf{y}^{(k)}) \quad (8.3.13)$$

and then applying it:

$$\mathbf{y}^{(k+1)} = \mathbf{y}^{(k)} + \Delta\mathbf{y} \quad (8.3.14)$$

where $\mathbf{J}$ is the Jacobian of the residual system:

$$\mathbf{J} = \frac{\partial \mathbf{r}}{\partial \mathbf{y}} = \mathbf{I} - \Delta t_i \nabla_{\mathbf{y}} \mathbf{f}(\mathbf{y}, t_{i+1}) \quad (8.3.15)$$

Practical Details. First, a good guess speeds up convergence. We usually use the solution from the previous step ($\mathbf{y}^{(0)} = \mathbf{x}(t_i)$) or an explicit Euler prediction ($\mathbf{y}^{(0)} = \mathbf{x}(t_i) + \Delta t_i \mathbf{f}(\mathbf{x}(t_i), t_i)$). Second, the cost of solving the linear system is $O(N^3)$ (using Gaussian elimination/LU). Third, if the ODE is linear ($\mathbf{f} = \mathbf{A}\mathbf{x}$), the Jacobian $\mathbf{J} = \mathbf{I} - \Delta t_i \mathbf{A}$ is constant (for fixed Δt). You only need to factorize it once and can reuse the LU factors, making it much cheaper.

This makes implicit methods much more computationally expensive per step than explicit methods. We have to evaluate the Jacobian and solve a linear system multiple times per time step. However, as we will see, the stability properties allow us to take much larger steps.

The Hidden Cost of Implicit Methods: Implicit methods provide stability, but they trade computational effort for it. In explicit Euler, calculating $\mathbf{x}_{i+1}$ is a direct formula evaluation. In implicit Euler, $\mathbf{x}_{i+1}$ is trapped inside a nonlinear function.

Finding $\mathbf{x}_{i+1}$ requires solving a root-finding problem $\mathbf{r}(\mathbf{y}) = 0$ at **every single time step**. This usually requires an iterative solver like Newton's Method, which involves:

1. Evaluating the Jacobian matrix $\mathbf{J}$.
2. Solving a linear system $\mathbf{J}\Delta\mathbf{y} = -\mathbf{r}$.

Consequently, one implicit step is much more expensive (in CPU time) than one explicit step. We only use implicit methods when the stability restriction on explicit methods forces the step size Δt to be impractically small (i.e., stiff problems).

8.3.2 Stability of Implicit Euler

We analyze the stability using the prototype linear problem $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$. Applying the implicit formula,

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t_i \mathbf{A}\mathbf{x}(t_{i+1}) \quad (8.3.16)$$

Rearranging to solve for $\mathbf{x}(t_{i+1})$,

$$(\mathbf{I} - \Delta t_i \mathbf{A})\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) \quad (8.3.17)$$

$$\mathbf{x}(t_{i+1}) = (\mathbf{I} - \Delta t_i \mathbf{A})^{-1} \mathbf{x}(t_i) \quad (8.3.18)$$

Decoupling the system using the eigenvalues λ_j of $\mathbf{A}$, the relationship for each component y_j is

$$y_j(t_{i+1}) = (1 - \Delta t_i \lambda_j)^{-1} y_j(t_i) \quad (8.3.19)$$

For stability, we require the magnitude of the amplification factor to be less than or equal to 1:

$$\left| \frac{y_j(t_{i+1})}{y_j(t_i)} \right| = |(1 - \Delta t_i \lambda_j)^{-1}| \leq 1 \quad (8.3.20)$$

It is helpful to define the complex variable $z = \lambda \Delta t$ and the stability function $R(z) = (1 - z)^{-1}$. The condition becomes $|R(z)| \leq 1$, or

$$|1 - z| \geq 1 \quad (8.3.21)$$

Expanding this with $z = x + iy$, we get

$$|(1 - z)|^2 = (1 - x)^2 + y^2 \geq 1 \quad (8.3.22)$$

Geometrically, $|1 - z| = 1$ is a circle in the complex plane with radius 1 centered at $z = +1$. The stability condition $|1 - z| \geq 1$ is satisfied everywhere *outside* this circle. Since the entire left half-plane ($\text{Re}(z) \leq 0$) lies outside this circle, implicit Euler is stable for any stable linear system, regardless of the step size.

A-Stability

A numerical method is called **A-stable** if its region of absolute stability includes the entire left half-plane of the complex numbers ($\text{Re}(z) \leq 0$). This means that if the exact solution decays ($\text{Re}(\lambda) < 0$), the numerical solution will also decay for *any* step size $\Delta t > 0$. Implicit Euler is A-stable. Explicit Euler is not.

All of the stability analysis in this chapter is carried out for the linear scalar test equation $\dot{x} = \lambda x$, and this guarantee does not automatically transfer to nonlinear $\mathbf{f}$. Nonlinear stiff systems require stronger notions, such as B-stability or conditions on the one-sided Lipschitz constant (logarithmic norm) of $\mathbf{f}$, which are beyond the scope of this course. In practice, this means you should not over-generalize “implicit Euler is unconditionally stable” to nonlinear stiff systems like combustion or reaction kinetics which is exactly where stiff solvers are commonly used in chemical engineering.

The region of stability for implicit Euler is shown in Figure 8.3.1.

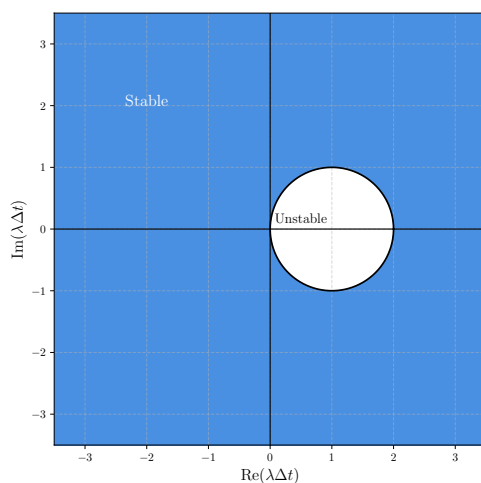


Figure 8.3.1: Implicit Euler stability region. Unlike explicit Euler, which was only stable inside a small circle, implicit Euler is stable everywhere outside the small white circle. It is stable for the entire left half-plane.

Importantly, if the real part of λ_j is negative (which corresponds to a stable physical system where the true solution decays), implicit Euler is always stable, regardless of the step size Δt_i . This property is extremely powerful.

8.3.3 The Crank-Nicolson Method

We can improve the accuracy of the implicit approach by combining the ideas of the trapezoidal rule for integration. The Crank-Nicolson method averages the slope at the beginning and the end of the interval:

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \frac{\Delta t_i}{2} [\mathbf{f}(\mathbf{x}(t_{i+1}), t_{i+1}) + \mathbf{f}(\mathbf{x}(t_i), t_i)] \quad (8.3.23)$$

Or, written in our general form,

$$\mathbf{g}(\dots) = \frac{1}{2} [\mathbf{f}(\mathbf{x}(t_{i+1}), t_{i+1}) + \mathbf{f}(\mathbf{x}(t_i), t_i)] \quad (8.3.24)$$

This corresponds to the trapezoidal rule for integration. Because of this, the method is called the implicit trapezoidal rule.

Accuracy of Crank-Nicolson Estimating the local truncation error via Taylor expansion shows that

$$\mathbf{x}(t_{i+1}) \approx \mathbf{x}(t_i) + \Delta t_i \mathbf{f}(t_i) + \frac{\Delta t_i^2}{2} \mathbf{f}'(t_i) + O(\Delta t_i^3) \quad (8.3.25)$$

That is, the local truncation error is $O(\Delta t_i^3)$. Consequently, the global truncation error is $O(\Delta t^2)$. Crank-Nicolson is thus a second-order method, meaning that it has better accuracy than implicit Euler while maintaining implicit properties.

Stability of Crank-Nicolson Applying the method to the prototype problem $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$,

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \frac{\Delta t_i}{2} \mathbf{A}(\mathbf{x}(t_{i+1}) + \mathbf{x}(t_i)) \quad (8.3.26)$$

Rearranging terms to group $\mathbf{x}(t_{i+1})$ on the left and $\mathbf{x}(t_i)$ on the right,

$$\left[\mathbf{I} - \frac{\Delta t_i}{2} \mathbf{A} \right] \mathbf{x}(t_{i+1}) = \left[\mathbf{I} + \frac{\Delta t_i}{2} \mathbf{A} \right] \mathbf{x}(t_i) \quad (8.3.27)$$

For a component y_j corresponding to eigenvalue λ_j , we have

$$y_j(t_{i+1}) = \left(1 - \frac{\Delta t_i \lambda_j}{2} \right)^{-1} \left(1 + \frac{\Delta t_i \lambda_j}{2} \right) y_j(t_i) \quad (8.3.28)$$

The stability condition is

$$\left| \frac{1 + \frac{\Delta t_i \lambda_j}{2}}{1 - \frac{\Delta t_i \lambda_j}{2}} \right| \leq 1 \quad (8.3.29)$$

To see why, let $z = \frac{\Delta t_i \lambda_j}{2} = a + ib$. Then the condition is

$$\left| \frac{1+z}{1-z} \right| \leq 1 \iff |1+z|^2 \leq |1-z|^2 \quad (8.3.30)$$

Substituting $z = a + ib$, we have

$$(1+a)^2 + b^2 \leq (1-a)^2 + b^2 \quad (8.3.31)$$

$$1 + 2a + a^2 + b^2 \leq 1 - 2a + a^2 + b^2 \quad (8.3.32)$$

$$4a \leq 0 \implies a \leq 0 \quad (8.3.33)$$

Since $a = \text{Re}(z) = \frac{\Delta t_i}{2} \text{Re}(\lambda_j)$, this inequality holds whenever $\text{Re}(\lambda_j) \leq 0$. Thus, Crank-Nicolson is A-stable, meaning its stability region includes the entire left half-plane.

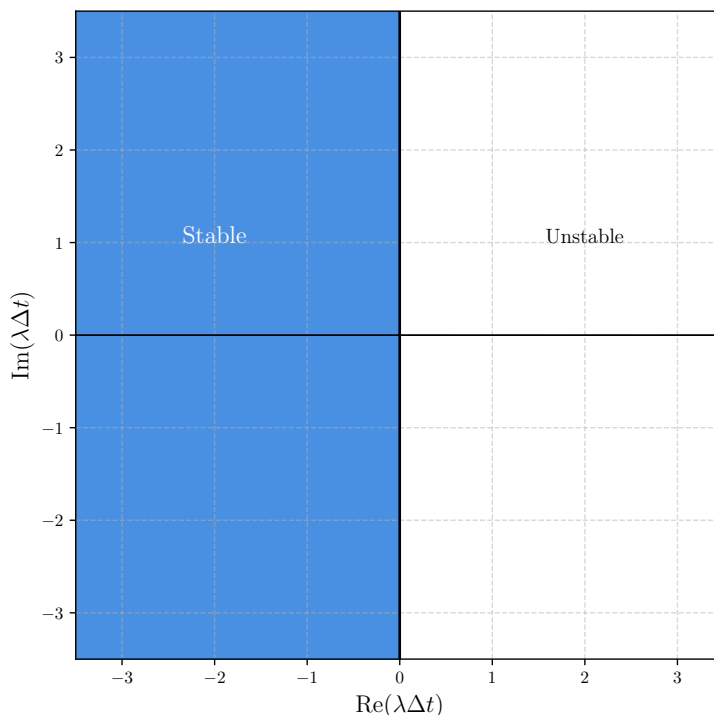


Figure 8.3.2: Stability region of the Crank-Nicolson method. The method is stable for all eigenvalues with a negative real part (the entire left half-plane).

8.3.4 Comparison of Implicit Behavior

While both implicit Euler and Crank-Nicolson are stable in the left half-plane, their behavior for very large step sizes differs. Consider the simple scalar decay equation $dy/dt = -y$ (where $\lambda = -1$). For implicit Euler, as $\Delta t \rightarrow \infty$, the amplification factor $(1 - \lambda\Delta t)^{-1} \rightarrow 0$. The numerical solution decays smoothly to zero, matching the physics. For Crank-Nicolson, as $\Delta t \rightarrow \infty$, the amplification factor approaches $\frac{\lambda\Delta t/2}{-\lambda\Delta t/2} = -1$. The solution does not explode, but it oscillates between $+1$ and -1 without decaying. This oscillatory behavior (ringing) in Crank-Nicolson can be problematic for very stiff systems (the concept of stiffness will be discussed momentarily) where high-frequency components should decay rapidly but instead persist as numerical noise.

Stability vs. Monotonicity (Ringing)

Consider the decay equation $y' = -y$ (exact solution e^{-t}). While both implicit Euler and Crank-Nicolson are unconditionally stable (the solution does not explode, see Figure 8.3.2), they behave very differently for large time steps (e.g., $\Delta t = 10$).

1. Implicit Euler (Smooth Decay) The update rule is $y_{i+1} = \frac{1}{1+\Delta t}y_i$. Note that the denominator is always positive. For $\Delta t = 10$,

$$y_{i+1} = \frac{1}{11}y_i \approx 0.09y_i$$

The solution remains positive and decays monotonically, matching the physics.

2. Crank-Nicolson (Oscillation) The update rule is $y_{i+1} = \frac{2-\Delta t}{2+\Delta t}y_i$. If $\Delta t > 2$, the numerator becomes negative. For $\Delta t = 10$,

$$y_{i+1} = \frac{2-10}{2+10}y_i = -\frac{2}{3}y_i \approx -0.67y_i$$

The magnitude decreases ($0.67 < 1$, so it is stable), but the sign flips at every step. This artificial oscillation is called ringing.

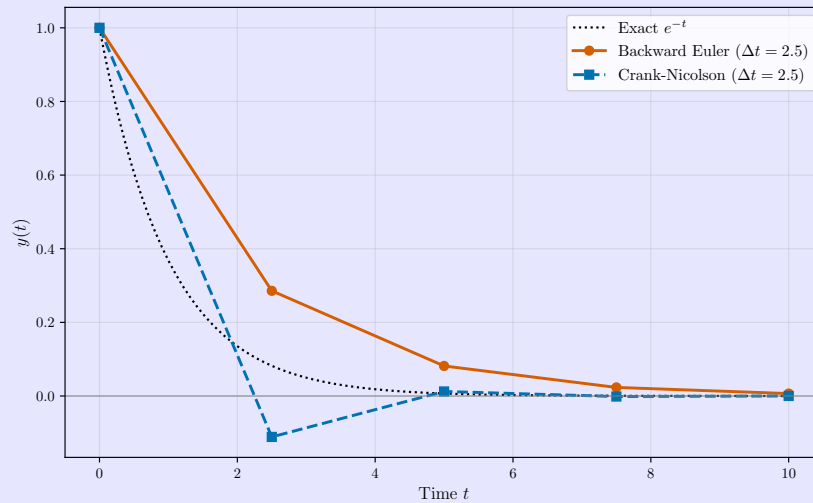


Figure 8.3.3: Implicit stability example solving $y' = -y$. While both methods are stable, Crank-Nicolson (dashed) can exhibit oscillations for large time steps Δt , whereas implicit Euler (solid) decays monotonically.

L-Stability

A method is **L-stable** if it is A-stable *and* the amplification factor approaches 0 as the step size goes to infinity:

$$\lim_{|z| \rightarrow \infty} |R(z)| = 0 \quad (\text{for } \text{Re}(z) < 0) \quad (8.3.34)$$

Methods that are L-stable (like implicit Euler) quickly damp out high-frequency noise/transients. Methods that are A-stable but not L-stable (like Crank-Nicolson, where $|R(z)| \rightarrow 1$) can allow high-frequency noise to persist as ringing.

8.3.5 Revisiting the Prototype Problem

To formalize these observations, let us look again at the prototype problem:

$$\frac{d\mathbf{x}}{dt} = \mathbf{A}\mathbf{x}(t), \quad \mathbf{x}(0) = \mathbf{x}_0 \quad (8.3.35)$$

The exact solution is $\mathbf{x}(t) = \mathbf{W}e^{\mathbf{A}t}\mathbf{W}^{-1}\mathbf{x}_0$. The system is analytically stable if $\text{Re}(\lambda_i) \leq 0$ for all $i = 1, \dots, N$. The system dynamics are characterized by time constants $\tau_i = |\lambda_i^{-1}|$. If we have a system where the time constants are roughly the same magnitude, explicit methods work well. However, consider what happens when the ratio of the largest to smallest time constant is very large:

$$\frac{\tau_{\max}}{\tau_{\min}} \gg 1 \quad \text{or} \quad \frac{|\lambda|_{\max}}{|\lambda|_{\min}} \gg 1 \quad (8.3.36)$$

This situation implies the solution spans a wide range of time scales. The fast modes (small τ) decay very quickly, but to capture them stably with an explicit method, Δt must be very small. However, we likely want to simulate the system long enough to see the slow modes (large τ) evolve. This forces us to take an exorbitant number of tiny steps. This specific difficulty is known as stiffness, which we discuss next.

8.4 Stiff Differential Equations

In the previous section on implicit methods, we alluded to a class of problems where the stability constraints of explicit methods become prohibitive. These are known as **stiff equations**. Stiffness usually arises in systems where physical phenomena occurring at widely varying time scales are coupled together. For example, a chemical reaction might involve one intermediate species that reacts in microseconds, while the overall product forms over minutes. To resolve the fast reaction explicitly, the time step Δt must be on the order of microseconds. However, to simulate the full process, we need to take millions of those tiny steps, even after the fast transient has died out and the system is evolving slowly.

We can characterize stiffness by the ratio of the largest to the smallest time constants in the system. For a linear system with eigenvalues λ_i , for decaying modes, $\tau_i \approx -1/\text{Re}(\lambda_i)$; for real negative λ_i , this is proportional to $1/|\lambda_i|$. From this, we see that the ratio of time constants is equivalent to the ratio of eigenvalue magnitudes:

$$\text{Stiffness} \sim \frac{\max |\lambda_i|}{\min |\lambda_i|} \approx \frac{\tau_{\max}}{\tau_{\min}} \quad (8.4.1)$$

Alternatively, considering the simulation duration t_f , the difficulty is proportional to $t_f/\tau_{\min}$.

The conflict in stiff problems is fundamental. Explicit solvers require Δt to be bounded by the smallest time constant ($\Delta t < C\tau_{\min}$) to prevent explosion. However, if $\tau_{\min} \ll t_f$, the number of steps $N_{\text{steps}} = t_f/\Delta t$ becomes enormous. Furthermore, each of these steps adds a small amount of finite-precision floating point error to the state vector. If we take too many steps, the accumulated round-off error destroys accuracy.

Some implicit methods, such as backward Euler and the trapezoidal rule, possess the property of A-stability. An A-stable method has a stability region that contains the entire left half of the complex plane ($\text{Re}(z) < 0$ for $z = \lambda\Delta t$), so it doesn't impose a stability step-size restriction on decaying modes. They allow the step size Δt to be chosen based on the accuracy required to track the solution's curvature rather than being dictated by the fastest decaying mode. While each implicit step is more expensive (requiring a Jacobian solve), the ability to take steps orders of magnitude larger makes them the only viable choice for stiff problems.

Remark: IMEX (Implicit-Explicit) Splitting Methods

The choice between fully explicit and fully implicit is not strictly binary. When $\mathbf{f}$ splits into a stiff part and a nonstiff part, $\mathbf{f}(\mathbf{x}, t) = \mathbf{f}_{\text{stiff}}(\mathbf{x}, t) + \mathbf{f}_{\text{nonstiff}}(\mathbf{x}, t)$ —for example, a linear diffusion term and a nonlinear reaction term in a reaction-diffusion PDE—an IMEX method treats $\mathbf{f}_{\text{stiff}}$ implicitly and $\mathbf{f}_{\text{nonstiff}}$ explicitly within a single step. This avoids both the tiny step sizes an explicit method would need to resolve the stiff term and the cost of solving a fully implicit *nonlinear* system at every step, since the implicit solve is only ever applied to the linear stiff part.

Example: Stiff Coupled System. Consider:

$$y_1' = -y_1, \quad y_2' = -1000y_2 \quad (8.4.2)$$

Here, $\lambda_1 = -1$ (slow) and $\lambda_2 = -1000$ (fast). Explicit Euler requires $|1 + \lambda\Delta t| \leq 1$. For $\lambda_2 = -1000$, we need $|1 - 1000\Delta t| \leq 1 \implies \Delta t \leq 0.002$. The update for the fast mode is $y_{n+1} = (1 - 1000\Delta t)y_n$. If we choose $\Delta t = 0.1$, the amplification factor is $(1 - 100) = -99$, so the error grows by $99\times$ each step, causing explosion.

On the other hand, implicit Euler is stable for any Δt . The update is $y_{n+1} = \frac{1}{1 - \lambda\Delta t}y_n$. For $\Delta t = 0.1$ and $\lambda = -1000$, the amplification factor is $\frac{1}{1+100} \approx 0.01$. The fast mode decays rapidly to zero, as it

should physically. We can safely choose $\Delta t = 0.1$ to capture the slow evolution of y_1 without worry.

Example: The Van der Pol Oscillator A classic example of a stiff system is the Van der Pol oscillator with a large nonlinearity parameter that we call λ here. The governing equations are

$$\frac{d\mathbf{x}}{dt} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ \lambda(1 - x_1^2)x_2 - x_1 \end{bmatrix} \quad (8.4.3)$$

When λ is large, the damping term $\lambda(1 - x_1^2)x_2$ dominates. To analyze the stiffness, we define a perturbation $\delta\mathbf{x} = \mathbf{x} - \mathbf{x}^*$. Linearizing the dynamics around $\mathbf{x}^*$ gives

$$\frac{d}{dt}\delta\mathbf{x} \approx \mathbf{J}(\mathbf{x}^*)\delta\mathbf{x} = \begin{bmatrix} 0 & 1 \\ -2\lambda x_1^*x_2^* - 1 & \lambda(1 - (x_1^*)^2) \end{bmatrix} \delta\mathbf{x} \quad (8.4.4)$$

Consider the state where the system passes through $x_1 \approx 0$. The Jacobian becomes:

$$\mathbf{J} \approx \begin{bmatrix} 0 & 1 \\ -1 & \lambda \end{bmatrix} \quad (8.4.5)$$

The eigenvalues μ satisfy $\mu^2 - \lambda\mu + 1 = 0$, which implies $\mu \approx \lambda$ and $\mu \approx 1/\lambda$ for large λ . The corresponding time constants are $T_{\max} \approx \lambda$ and $T_{\min} \approx 1/\lambda$. The stiffness ratio scales as:

$$\frac{T_{\max}}{T_{\min}} \sim \frac{\lambda}{1/\lambda} = O(\lambda^2) \quad (8.4.6)$$

Physically, the solution rapidly traverses the fast segments (associated with the large eigenvalue $\sim \lambda$) and then slowly drifts along the slow manifold (time scale $\sim \lambda$). A nonstiff solver must use tiny steps everywhere because of the brief fast phases. For $\lambda = 100$, the stiffness ratio is 10,000. An explicit solver would require steps 10,000 times smaller than the slow dynamics suggest.

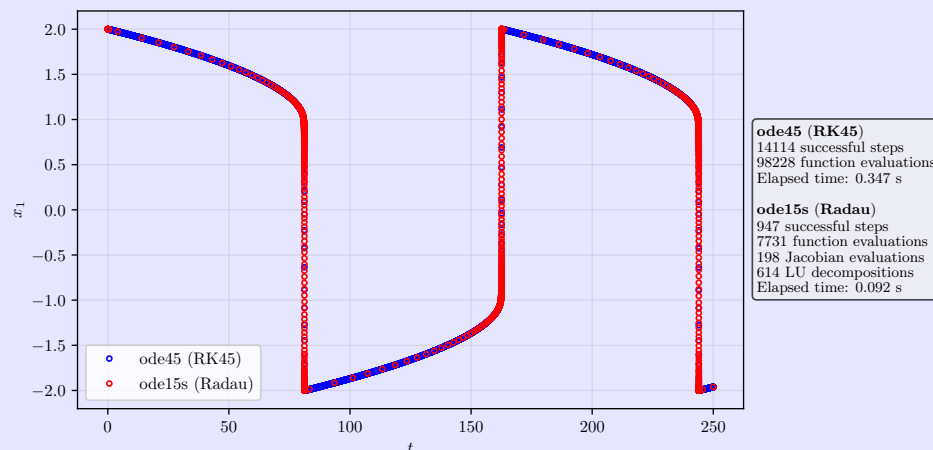


Figure 8.4.1: Numerical solution of the stiff Van der Pol oscillator with $\lambda = 100$ using a nonstiff explicit solver (ode45 / RK45, blue) and a stiff implicit solver (ode15s / Radau, red). Both methods trace the same limit cycle in $x_1(t)$, but the stiff solver uses far fewer, adaptively placed time steps and function evaluations (see statistics box), making it substantially more efficient for this stiff problem.

8.5 Adaptive Time Stepping*

In practical applications, we rarely use a fixed step size Δt . The solution may vary slowly for long periods (allowing large steps) and then change rapidly (requiring small steps). To handle this, we employ adaptive time stepping. The goal is to choose Δt_i such that the LTE is kept below a user-specified tolerance ϵ . Since we do not know the exact solution, we must estimate the error using available data. A common strategy involves using the properties of the numerical method itself. Consider the explicit midpoint method we discussed earlier:

$$\mathbf{y} = \mathbf{x}_i + \frac{\Delta t_i}{2} \mathbf{f}(\mathbf{x}_i, t_i) \quad (8.5.1)$$

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \Delta t_i \mathbf{f}(\mathbf{y}, t_i + \Delta t_i/2) \quad (8.5.2)$$

As part of the error estimate, we additionally evaluate the function at the end of the step, $\mathbf{f}(\mathbf{x}_{i+1}, t_{i+1})$, giving us three values of $\mathbf{f}$ over the step (start, midpoint, and end). It can be shown via Taylor expansion that a linear combination of these evaluations acts as a proxy for the LTE:

$$\text{Error Proxy} \approx \Delta t_i [\mathbf{f}(\mathbf{x}_i, t_i) + \mathbf{f}(\mathbf{x}_{i+1}, t_{i+1}) - 2\mathbf{f}(\mathbf{y}, t_i + \Delta t_i/2)] \sim O(\Delta t_i^3) \quad (8.5.3)$$

This quantity acts as an error indicator (conceptually proportional to Δt^3 times the curvature of $\mathbf{f}$ along the step). If the function were perfectly linear, this term would vanish.

Adaptive Step Algorithm* The adaptive procedure works by first computing the trial step to get $\mathbf{x}_{i+1}$ and then calculating the error estimate $\mathbf{E} = \|\Delta t_i [\dots]\|_2$. We then check if this error satisfies the tolerance condition $\mathbf{E} \leq \epsilon_a + \epsilon_r \max(\|\mathbf{x}_i\|_2, \|\mathbf{x}_{i+1}\|_2)$. If the error is too high, we reject the step, reduce Δt_i (e.g., by half), and try again. If the error is acceptable, we accept $\mathbf{x}_{i+1}$ and increase Δt_{i+1} (e.g., by $1.2\times$) for the next step to improve efficiency. More formally, standard integrators often update the step size using a formula like $\Delta t_{\text{new}} = \Delta t_{\text{old}} (\text{tol}/E)^{1/(p+1)}$ (where p is the global order of the method), but simple halving/doubling logic is enough to get the main idea. This logic applies generally to any ODE solver (e.g., Runge-Kutta 4/5 pairs like `ode45` in MATLAB/Python).

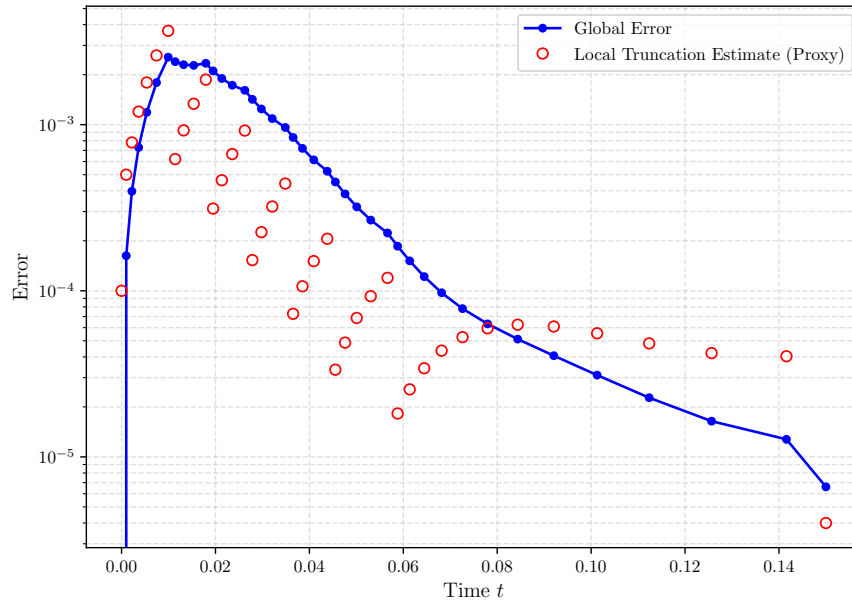


Figure 8.5.1: Adaptive time stepping applied to $dx/dt = -100x$ using the explicit midpoint method. The algorithm automatically adjusts the step size Δt to keep the estimated local truncation error (hollow red circles) below a set tolerance. Consequently, the global error (solid blue line) remains bounded throughout the simulation.

8.6 Existence and Uniqueness of Solutions*

Before relying on numerical solutions, we must ensure that a mathematical solution actually exists and is unique. If the analytical solution ceases to exist at finite time t , a numerical solver will eventually fail or output garbage. While linear systems with constant coefficients generally behave predictably, nonlinear equations can contain hidden traps. A classic example demonstrates how a simple nonlinearity can cause the solution to cease existing in a finite amount of time, leading to solver failure.

Finite Escape Time Consider the initial value problem:

$$\frac{dx}{dt} = x^2, \quad x(0) = x_0 \quad (8.6.1)$$

Solving via separation of variables ($\int x^{-2} dx = \int dt$), we get the analytical solution that

$$x(t) = \frac{1}{1/x_0 - t} \quad (8.6.2)$$

Notice that the solution does not exist for all time; as $t \rightarrow 1/x_0$, the denominator vanishes and $x(t) \rightarrow \infty$. This is known as finite escape time. Local existence and uniqueness are guaranteed near $t = 0$, but the solution cannot be extended past $t = 1/x_0$; global existence fails because the solution becomes unbounded.

Now, let's see the numerical trap. Consider the specific case where $x(0) = 1$. Mathematically, this solution explodes at $t = 1$. However, a numerical solver is not guaranteed to spot this. If you blindly run a fixed-step solver (e.g., $\Delta t = 0.1$), it might calculate a step from $t = 0.9$ to $t = 1.0$, effectively jumping over the singularity and producing a finite, nonsensical result on the other side. Numerical stability does not imply physical existence; always ensure your model is well-posed.

8.6.1 Uniqueness*

Now, consider the problem

$$t \frac{dx}{dt} = x - 1, \quad x(0) = 1 \quad (8.6.3)$$

Rearranging for $t \neq 0$, we get $\frac{dx}{x-1} = \frac{dt}{t}$, which integrates to $\ln|x-1| = \ln|t| + C$, or $x(t) = 1 + c_2 t$. At $t = 0$, $x(0) = 1$ is satisfied regardless of the constant c_2 . Thus, there are infinite solutions (one for every slope c_2). The solution is not unique. Note that $f(x, t) = (x - 1)/t$ is not even continuous at $t = 0$, so the standard existence-uniqueness theorem does not apply there.

Lipschitz Continuity* The fundamental theorem for ODEs states that a unique solution exists if the function $\mathbf{f}(\mathbf{x}, t)$ is Lipschitz continuous.

Lipschitz Continuity

A function $\mathbf{f}(\mathbf{x}, t)$ is Lipschitz continuous on a domain $\mathcal{D}$ if there exists a constant $L > 0$ (the Lipschitz constant) such that:

$$\|\mathbf{f}(\mathbf{x}, t) - \mathbf{f}(\mathbf{z}, t)\| \leq L \|\mathbf{x} - \mathbf{z}\| \quad (8.6.4)$$

for all $\mathbf{x}, \mathbf{z} \in \mathcal{D}$.

If $\mathbf{f}$ is Lipschitz continuous, the Picard-Lindelöf theorem guarantees that the ODE-IVP has a unique solution near the initial condition. Intuitively, this condition prevents the derivative from changing too rapidly (e.g., infinite slope).

Continuity vs. Lipschitz Continuity.

First, consider $f(x) = x$. This function is globally Lipschitz continuous ($L = 1$). Therefore, a unique solution exists globally for all time.

Second, consider $f(x) = x^2$. This function is locally Lipschitz (on any bounded interval), so a unique solution exists locally. However, it is not globally Lipschitz (the slope $2x$ grows without bound). This explains why the solution $x(t) = (1/x_0 - t)^{-1}$ escapes to infinity in finite time: we lose global existence, though local uniqueness holds up until the blow-up.

Third, consider $x' = \sqrt{|x|}$ with $x(0) = 0$. This function is continuous everywhere but not Lipschitz at $x = 0$ (the slope goes to infinity). Consequently, we lose local uniqueness: both $x(t) = 0$ and $x(t) = t^2/4$ are valid solutions starting from the same initial condition.

8.7 Exercises

Exercise 8.1

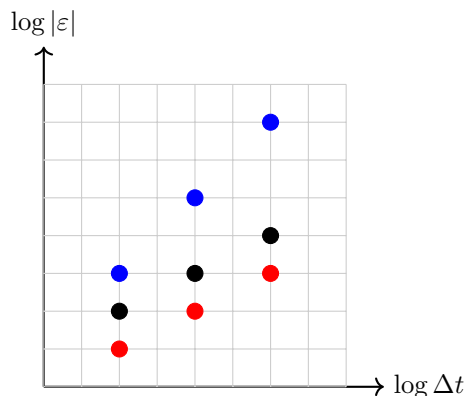
Consider a reaction system $A \xrightarrow{k_1} B \xrightarrow{k_2} 2C$ with $k_1 = 1 \text{ s}^{-1}$ and $k_2 = 10 \text{ s}^{-1}$.

$$\text{i.e., } \frac{d}{dt} \begin{pmatrix} C_A \\ C_B \end{pmatrix} = \begin{pmatrix} -1 & 0 \\ +1 & -10 \end{pmatrix} \begin{pmatrix} C_A \\ C_B \end{pmatrix}$$

What is the largest step size Δt that we should use if solving this problem with forward Euler?

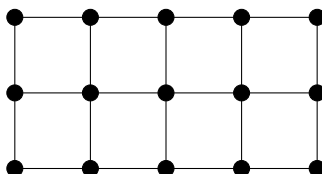
Exercise 8.2

Plotted (black) is the Global Truncation Error for the forward Euler method. Which trace is more likely to describe the GTE of Crank-Nicolson for the same problem? **Red** or **blue**?



Exercise 8.3

How many unknowns will we have if we are using Finite Volumes to approximate a 2D concentration profile over the following (very coarse) discretized domain?



Exercise 8.4

(*) What is the order of accuracy, with respect to $\max(\Delta x_{i-1}, \Delta x_i)$, of the following approximation of the first derivative at a point? Assume $\Delta x_{i-1} \neq \Delta x_i$, with $\Delta x_{i-1} = x_i - x_{i-1}$ and $\Delta x_i = x_{i+1} - x_i$. What if $\Delta x_{i-1} = \Delta x_i$?

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{\Delta x_i + \Delta x_{i-1}}$$

Chapter 9

Boundary Value Problems

So far, we have focused exclusively on IVPs, where the state of the system and its derivatives are fully known at a single starting point, t_0 . The solver's task was simply to march forward in time. However, many physical problems, particularly those involving space rather than time, provide information at two different boundaries. These are known as **boundary value problems (BVPs)**. Formally, a BVP consists of a system of differential equations defined on an interval $[t_0, t_f]$:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t) \quad (9.0.1)$$

coupled with a set of boundary conditions (BCs) that relate the state at the start and the end of the interval:

$$\mathbf{g}(\mathbf{x}(t_0), \mathbf{x}(t_f)) = 0 \quad (9.0.2)$$

Here, $\mathbf{x} : [t_0, t_f] \rightarrow \mathbb{R}^N$ is the state vector, and the independent variable t often represents space (e.g., position along a beam) rather than time. Unlike IVPs, where existence and uniqueness are usually guaranteed by Lipschitz continuity, BVPs are much harder to solve and may have no solution, a unique solution, or infinitely many solutions depending on the boundary conditions.

Example: Reaction-Diffusion in a Slab

Consider the steady-state concentration $C(x)$ of a chemical species diffusing through a slab while undergoing a reaction $r(C)$. The governing equation is a second-order ODE:

$$\frac{d^2C}{dx^2} = -r(C) \quad (9.0.3)$$

To solve this, we must convert it to a system of first-order ODEs. Let $\mathbf{y} = [C, C']^\top$. The system becomes:

$$\frac{d}{dx} \begin{bmatrix} C \\ C' \end{bmatrix} = \begin{bmatrix} C' \\ -r(C) \end{bmatrix} \quad (9.0.4)$$

Usually, we know the concentration at the surface ($x = 0$) and perhaps that the flux is zero at the center ($x = 1$). These form the boundary conditions:

$$C(0) = 1, \quad C'(1) = 0 \quad \implies \quad \mathbf{g}(\mathbf{y}(0), \mathbf{y}(1)) = \begin{bmatrix} C(0) - 1 \\ C'(1) \end{bmatrix} = \mathbf{0} \quad (9.0.5)$$

9.1 Introduction to Boundary Value Problems

9.1.1 Classification of Boundary Conditions

The function $\mathbf{g}$ can take many forms, but in engineering, three specific types of boundary conditions appear most frequently.

Dirichlet boundary conditions specify the value of the solution itself at the boundaries. For a second-order system, this might look like $C(0) = 1$ and $C(1) = 0$. In vector notation, this restricts specific components of the state vector $\mathbf{x}$; i.e., $\mathbf{B}_0\mathbf{x}(t_0) + \mathbf{B}_1\mathbf{x}(t_f) = \mathbf{b}$.

Neumann boundary conditions specify the value of the derivative (flux) at the boundaries. For example, $C'(0) = 1$ and $C'(1) = 0$. In the first-order system formulation, this corresponds to constraining the specific vector components that represent velocities or fluxes.

Robin boundary conditions are linear combinations of Dirichlet and Neumann conditions. These often arise in convection problems (Newton's law of cooling) or elastic supports. Mathematically, they take the form:

$$a_1C(0) + a_2C'(0) = 1 \quad \text{and} \quad b_1C(1) + b_2C'(1) = 0 \quad (9.1.1)$$

In the first-order system formulation, all of these conditions (Dirichlet, Neumann, and Robin) are simply algebraic constraints on the state vector at the boundaries, generalized as $\mathbf{B}_0\mathbf{x}(t_0) + \mathbf{B}_1\mathbf{x}(t_f) = \mathbf{b}$.

9.1.2 Linear BVPs

Before tackling the general nonlinear case, we consider linear BVPs, where both the differential equations and the boundary conditions are linear. The problem is formulated as

$$\frac{d\mathbf{x}}{dt} = \mathbf{A}\mathbf{x}, \quad \mathbf{B}_0\mathbf{x}(0) + \mathbf{B}_T\mathbf{x}(T) + \mathbf{b} = 0 \quad (9.1.2)$$

where $\mathbf{A}$ is a constant matrix. If we assume $\mathbf{A}$ is diagonalizable with eigenvectors $\mathbf{W}$ and eigenvalues $\mathbf{\Lambda}$, we already know the general solution to the ODE from our study of IVPs. If we impose an arbitrary initial condition $\mathbf{x}(0) = \mathbf{c}$, the solution at time t is:

$$\mathbf{x}(t) = \mathbf{W}e^{\mathbf{\Lambda}t}\mathbf{W}^{-1}\mathbf{c} \quad (9.1.3)$$

The unique challenge of the BVP is that we do not know $\mathbf{c}$; it is an unknown vector we must find. However, we can substitute the analytical solution form into the boundary condition equation:

$$\mathbf{B}_0\mathbf{c} + \mathbf{B}_T(\mathbf{W}e^{\mathbf{\Lambda}T}\mathbf{W}^{-1}\mathbf{c}) + \mathbf{b} = 0 \quad (9.1.4)$$

Factoring out the unknown vector $\mathbf{c}$, we arrive at a system of linear algebraic equations:

$$(\mathbf{B}_0 + \mathbf{B}_T\mathbf{W}e^{\mathbf{\Lambda}T}\mathbf{W}^{-1})\mathbf{c} = -\mathbf{b} \quad (9.1.5)$$

This result is powerful because it reduces a calculus problem to a linear algebra problem $\mathbf{M}\mathbf{c} = -\mathbf{b}$. A solution exists if the vector $\mathbf{b}$ lies in the column space (range) of the matrix $\mathbf{M} = \mathbf{B}_0 + \mathbf{B}_T\mathbf{W}e^{\mathbf{\Lambda}T}\mathbf{W}^{-1}$. Furthermore, a *unique* solution exists if and only if the determinant of this matrix is non-zero:

$$\det(\mathbf{B}_0 + \mathbf{B}_T\mathbf{W}e^{\mathbf{\Lambda}T}\mathbf{W}^{-1}) \neq 0 \quad (9.1.6)$$

If a unique solution exists, we can solve for $\mathbf{c}$ simply by inverting the matrix:

$$\mathbf{c} = -(\mathbf{B}_0 + \mathbf{B}_T\mathbf{W}e^{\mathbf{\Lambda}T}\mathbf{W}^{-1})^{-1}\mathbf{b} \quad (9.1.7)$$

Once $\mathbf{c}$ is found, the full solution $\mathbf{x}(t)$ is obtained by propagating this initial condition forward. In particular, combining this with the general solution $\mathbf{x}(t) = \mathbf{W}e^{\mathbf{\Lambda}t}\mathbf{W}^{-1}\mathbf{c}$, we can write the full solution explicitly as

$$\begin{aligned} \mathbf{x}(t) &= \mathbf{W}e^{\mathbf{\Lambda}t}\mathbf{W}^{-1}\mathbf{c} \\ &= -\mathbf{W}e^{\mathbf{\Lambda}t}\mathbf{W}^{-1}(\mathbf{B}_0 + \mathbf{B}_T\mathbf{W}e^{\mathbf{\Lambda}T}\mathbf{W}^{-1})^{-1}\mathbf{b} \end{aligned} \quad (9.1.8)$$

Example: Consider the boundary value problem

$$\frac{d\mathbf{x}}{dt} = \mathbf{A}\mathbf{x}, \quad \mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad \mathbf{x}(t) \in \mathbb{R}^2 \quad (9.1.9)$$

with final time $T = 1$ and boundary conditions

$$x_1(0) = 0, \quad x_2(1) = 1 \quad (9.1.10)$$

We write these boundary conditions in the form

$$\mathbf{B}_0\mathbf{x}(0) + \mathbf{B}_T\mathbf{x}(1) + \mathbf{b} = \mathbf{0} \quad (9.1.11)$$

with

$$\mathbf{B}_0 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad \mathbf{B}_T = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ -1 \end{bmatrix} \quad (9.1.12)$$

since this gives

$$\mathbf{B}_0\mathbf{x}(0) + \mathbf{B}_T\mathbf{x}(1) + \mathbf{b} = \begin{bmatrix} x_1(0) \\ x_2(1) - 1 \end{bmatrix} = \mathbf{0} \quad (9.1.13)$$

The matrix A is already diagonal:

$$\mathbf{A} = \mathbf{W}\mathbf{\Lambda}\mathbf{W}^{-1}, \quad \mathbf{W} = \mathbf{I}, \quad \mathbf{\Lambda} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad (9.1.14)$$

For an arbitrary initial condition $\mathbf{x}(0) = \mathbf{c}$, the solution is

$$\mathbf{x}(t) = \mathbf{W}e^{\mathbf{\Lambda}t}\mathbf{W}^{-1}\mathbf{c} = \begin{bmatrix} e^t & 0 \\ 0 & e^{-t} \end{bmatrix} \mathbf{c} \quad (9.1.15)$$

To enforce the boundary conditions, form

$$\mathbf{M}\mathbf{c} = -\mathbf{b}, \quad \mathbf{M} := \mathbf{B}_0 + \mathbf{B}_T\mathbf{W}e^{\mathbf{\Lambda}T}\mathbf{W}^{-1} \quad (9.1.16)$$

Here $T = 1$ and

$$e^{\mathbf{\Lambda}T} = e^{\mathbf{\Lambda}} = \begin{bmatrix} e & 0 \\ 0 & e^{-1} \end{bmatrix} \quad (9.1.17)$$

so

$$\mathbf{M} = \mathbf{B}_0 + \mathbf{B}_T\mathbf{W}e^{\mathbf{\Lambda}T}\mathbf{W}^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} e & 0 \\ 0 & e^{-1} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & e^{-1} \end{bmatrix} \quad (9.1.18)$$

Since $\det(\mathbf{M}) = e^{-1} \neq 0$, there is a unique solution:

$$\mathbf{M}\mathbf{c} = -\mathbf{b} \implies \begin{bmatrix} 1 & 0 \\ 0 & e^{-1} \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \implies c_1 = 0, \quad c_2 = e \quad (9.1.19)$$

Thus

$$\mathbf{c} = \mathbf{x}(0) = \begin{bmatrix} 0 \\ e \end{bmatrix} \quad (9.1.20)$$

and the full solution is

$$\mathbf{x}(t) = \begin{bmatrix} e^t & 0 \\ 0 & e^{-t} \end{bmatrix} \begin{bmatrix} 0 \\ e \end{bmatrix} = \begin{bmatrix} 0 \\ e^{1-t} \end{bmatrix} \quad (9.1.21)$$

We can easily check that $x_1(0) = 0$ and $x_2(1) = 1$, so the BVP is solved.

9.2 Shooting Methods

9.2.1 The Shooting Method

When the differential equation $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, t)$ or the boundary conditions are nonlinear, we cannot use linear superposition or matrix inversion. Instead, we employ an iterative numerical approach known as the shooting method. The intuition comes from artillery. Suppose we have a cannon at position x_0 and we want to hit a target at x_T at time T . We know the starting position, but we do not know the necessary initial velocity (angle and speed). We guess an initial velocity $\mathbf{v}(0)$, fire the cannon (integrate the ODE), and see where the shell lands. Based on the “miss distance” (the residual), we adjust the initial velocity (in an intelligent way, using Newton-Raphson) and fire again (Figure 9.2.1).

Mathematically, this corresponds to us transforming the BVP into a root-finding problem. We temporarily assume we know the full initial state $\mathbf{c} \in \mathbb{R}^N$ such that $\mathbf{x}(0) = \mathbf{c}$. This converts the BVP into an IVP, which we denote as $\mathbf{x}(t; \mathbf{c})$. We can solve this IVP numerically to find the state at the final time, $\mathbf{x}(T; \mathbf{c})$. We then define a residual function $\mathbf{h}$ (where $\mathbf{h} : \mathbb{R}^N \rightarrow \mathbb{R}^N$) representing the error in the boundary conditions:

$$\mathbf{h}(\mathbf{c}) := \mathbf{g}(\mathbf{c}, \mathbf{x}(T; \mathbf{c})) = 0 \quad (9.2.1)$$

Here, $\mathbf{g} : \mathbb{R}^N \times \mathbb{R}^N \rightarrow \mathbb{R}^N$ encodes the boundary conditions at both ends. Our goal is to find the root vector $\mathbf{c}^*$ such that $\mathbf{h}(\mathbf{c}^*) = 0$. Since $\mathbf{h}$ is a nonlinear function, we use the Newton-Raphson method.

Newton-Raphson for Shooting To apply Newton-Raphson, we start with an initial guess $\mathbf{c}_i$ and iteratively update it using the Jacobian matrix $\mathbf{J}_{\mathbf{h}}(\mathbf{c}) \in \mathbb{R}^{N \times N}$:

$$\mathbf{c}_{i+1} = \mathbf{c}_i - (\mathbf{J}_{\mathbf{h}}(\mathbf{c}_i))^{-1} \mathbf{h}(\mathbf{c}_i) \quad (9.2.2)$$

(Note: In practice, we do not invert the matrix. Instead, we solve the linear system $\mathbf{J}_{\mathbf{h}}(\mathbf{c}_i) \delta \mathbf{c} = -\mathbf{h}(\mathbf{c}_i)$ for the correction $\delta \mathbf{c}$). By the chain rule, this Jacobian depends on how the boundary function $\mathbf{g}$ changes with respect to the initial condition $\mathbf{c}$:

$$\mathbf{J}_{\mathbf{h}}(\mathbf{c}) = \frac{\partial \mathbf{g}}{\partial \mathbf{c}} + \frac{\partial \mathbf{g}}{\partial \mathbf{x}(T)} \frac{\partial \mathbf{x}(T; \mathbf{c})}{\partial \mathbf{c}} \quad (9.2.3)$$

The terms $\frac{\partial \mathbf{g}}{\partial \mathbf{c}}$ and $\frac{\partial \mathbf{g}}{\partial \mathbf{x}}$ are algebraic derivatives of the boundary condition function, which are usually easy to calculate. However, the term $\frac{\partial \mathbf{x}(T; \mathbf{c})}{\partial \mathbf{c}}$ corresponds to the sensitivity of the ODE solution at time T to changes in the initial condition at time 0. This term cannot be found algebraically; it requires solving differential equations.

9.2.2 Sensitivity Analysis

We define the **sensitivity matrix** $\mathbf{S}(t; \mathbf{c})$ as the derivative of the state with respect to the initial condition:

$$\mathbf{S}(t; \mathbf{c}) = \frac{\partial \mathbf{x}(t; \mathbf{c})}{\partial \mathbf{c}} \quad (9.2.4)$$

Here $\mathbf{S}(t; \mathbf{c})$ is an $N \times N$ matrix. In code, we typically flatten it into a vector of length N^2 and integrate it alongside $\mathbf{x}$. (Alternatively, we could approximate $\mathbf{J}_{\mathbf{h}}$ via finite differences in $\mathbf{c}$, but integrating the variational equation is usually more accurate and efficient locally). At time $t = 0$, the sensitivity is simply the identity matrix, $\mathbf{S}(0) = \mathbf{I}$, because $\mathbf{x}(0) = \mathbf{c}$. To find how $\mathbf{S}$ evolves over time, we differentiate the original ODE $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$ with respect to $\mathbf{c}$. Using the chain rule and switching the order of differentiation:

$$\frac{d}{dt} \left(\frac{\partial \mathbf{x}}{\partial \mathbf{c}} \right) = \frac{\partial}{\partial \mathbf{c}} \mathbf{f}(\mathbf{x}(t; \mathbf{c}), t) = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \mathbf{c}} \quad (9.2.5)$$

Substituting $\mathbf{S}$ and the Jacobian of the dynamics $\mathbf{J}(\mathbf{x}, t) = \frac{\partial \mathbf{f}}{\partial \mathbf{x}}$, we arrive at the variational equation, which is a matrix-valued IVP:

$$\frac{d\mathbf{S}}{dt} = \mathbf{J}(\mathbf{x}, t) \mathbf{S}, \quad \mathbf{S}(0) = \mathbf{I} \quad (9.2.6)$$

In practice, to perform one iteration of the Newton-Raphson shooting method, we must numerically integrate a coupled system of size $N + N^2$:

$$\begin{cases} \frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t), & \mathbf{x}(0) = \mathbf{c} \\ \frac{d\mathbf{S}}{dt} = \mathbf{J}(\mathbf{x}, t)\mathbf{S}, & \mathbf{S}(0) = \mathbf{I} \end{cases} \quad (9.2.7)$$

Solving this coupled system from $t = 0$ to $t = T$ yields both the residual $\mathbf{h}(\mathbf{c})$ (via $\mathbf{x}(T)$) and the Jacobian $\mathbf{J}_h(\mathbf{c})$ (via $\mathbf{S}(T)$), allowing us to compute the Newton step.

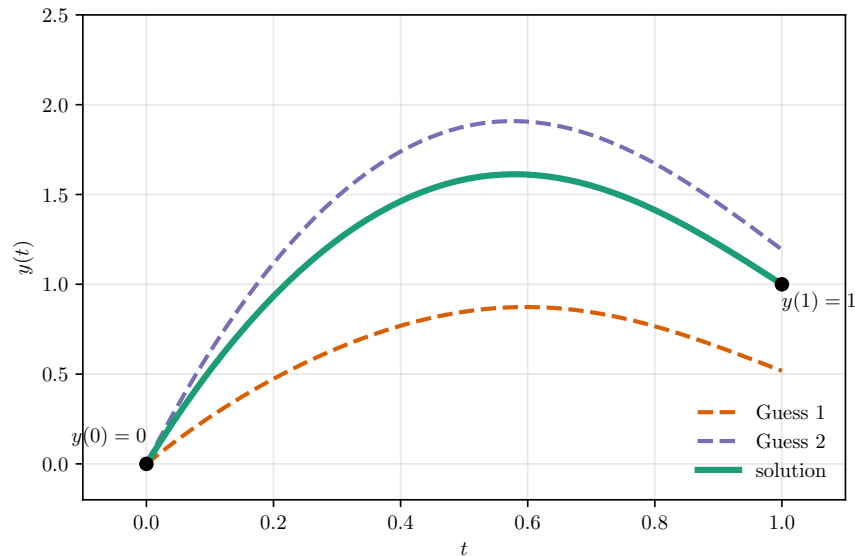


Figure 9.2.1: Shooting method for the boundary value problem $y'' + 2y' + 5y = 2\sin(2\pi t)$ with $y(0) = 0$ and $y(1) = 1$. Two initial guesses for the unknown initial slope produce trajectories that miss the boundary condition at $t = 1$, while the final trajectory corresponds to the solution obtained by adjusting the initial slope with a root-finding method.

9.2.3 The Multiple Shooting Method

While the shooting method is conceptually simple, it suffers from a major weakness: sensitivity to initial conditions. The Newton-Raphson update relies on the inverse of the Jacobian $\mathbf{J}_h(\mathbf{c})$. This matrix essentially contains the sensitivity term $\mathbf{S}(T; \mathbf{c})$. If the physical system is chaotic (e.g., the Lorenz system) or simply has rapidly growing modes, the sensitivity $\mathbf{S}(T)$ can become exponentially large. This leads to an ill-conditioned Jacobian. Numerically, this means that a microscopic change in the initial guess $\mathbf{c}$ leads to a massive change in the trajectory $\mathbf{x}(T)$, causing the numerical integrator to diverge or the Newton iterations to overshoot wildly.

Sensitivity in the Lorenz System Consider a chaotic Lorenz system:

$$\dot{x} = 10(y - x), \quad \dot{y} = x(30 - z) - y, \quad \dot{z} = xy - 2z \quad (9.2.8)$$

If we attempt to solve a BVP on this system over a long time interval, we find extreme sensitivity. In such cases, both the physical system and the numerical method exhibit extreme sensitivity: numerical roundoff and integration error are also amplified by the large sensitivity matrix $\mathbf{S}(T)$. A change in the initial condition as small as 10^{-4} (e.g., changing $z(0)$ from -0.0023 to -0.0024) can cause the trajectories to diverge completely, ending up in different lobes of the attractor. This makes the standard shooting method unstable for long integration times.

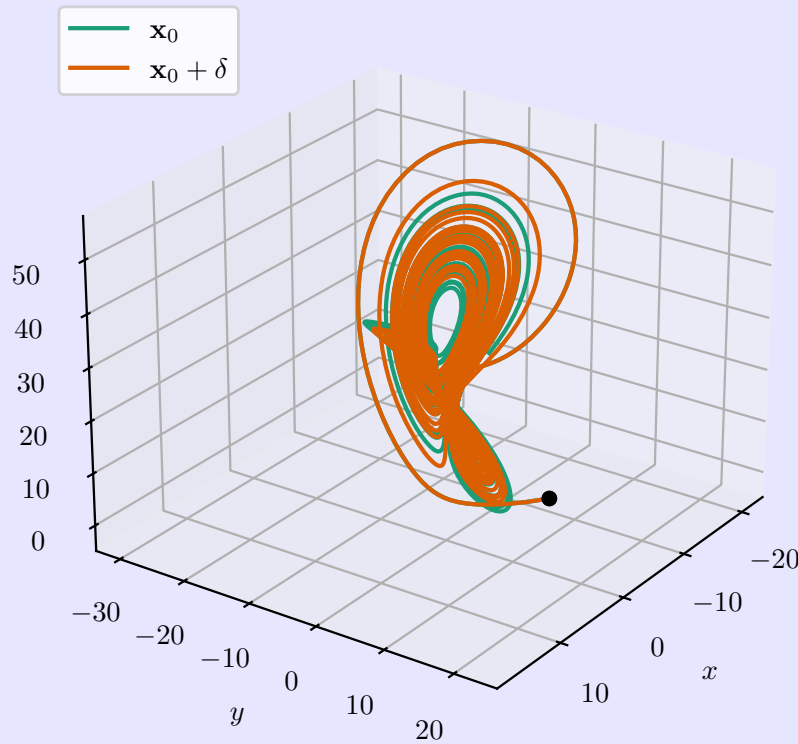


Figure 9.2.2: Sensitivity in the Lorenz system. Two trajectories are integrated from nearly identical initial conditions, differing only in the z component by 10^{-4} . Although they initially track each other closely, they eventually diverge. This shows the extreme sensitivity to initial conditions that makes standard shooting methods unstable over long time intervals for chaotic systems.

The Multiple Shooting Method To mitigate the sensitivity problem, we can use the multiple shooting method. The idea is to break the long integration interval $[0, T]$ into M smaller sub-intervals. That is, we partition the interval as $0 = t_0 < t_1 < \dots < t_M = T$, where $t_i = i\Delta t$ (Figure 9.2.3). For each sub-interval $[t_i, t_{i+1}]$, we introduce an unknown initial state $\mathbf{c}_i \approx \mathbf{x}(t_i)$ and define the local solution $\mathbf{x}_i(t)$ as the solution of:

$$\dot{\mathbf{x}}_i = \mathbf{f}(\mathbf{x}_i, t), \quad \mathbf{x}_i(t_i) = \mathbf{c}_i, \quad t \in [t_i, t_{i+1}] \quad (9.2.9)$$

To make sure these separate pieces form a valid continuous solution for the original BVP, we impose continuity constraints at the nodes such that the end of one interval matches the start of the next.

$$\mathbf{x}_i(t_{i+1}) - \mathbf{c}_{i+1} = \mathbf{0}, \quad i = 0, \dots, M-2 \quad (9.2.10)$$

Combined with the boundary conditions (which now map the very first $\mathbf{c}_0$ and the very last $\mathbf{x}_{M-1}(t_M)$), we now have a system of equations in $N \cdot M$ unknowns. In practice, this large nonlinear system is solved by Newton's method. The Jacobian has a sparse block structure due to the local coupling between neighboring intervals. By keeping the sub-intervals short, we prevent the sensitivity matrix $\mathbf{S}$ from growing exponentially large within any single step and keep the condition number manageable.

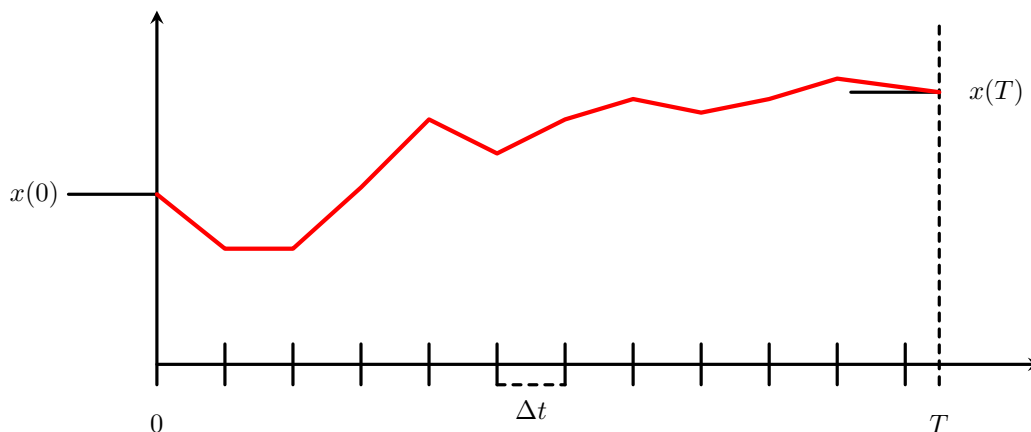


Figure 9.2.3: Schematic of the multiple shooting method. The long interval $[0, T]$ is partitioned into M sub-intervals $[t_i, t_{i+1}]$ with $t_i = i\Delta t$. On each sub-interval, a local trajectory $\mathbf{x}_i(t)$ is integrated from an independent initial value $\mathbf{c}_i \approx \mathbf{x}(t_i)$. Continuity constraints enforce that the end point of one segment matches the initial value of the next, $\mathbf{x}_i(t_{i+1}) = \mathbf{c}_{i+1}$, so that the segments form a single continuous solution of the BVP. Note that the solution is not necessarily piecewise linear (it just is for the case shown here).

9.3 Discretization Methods

9.3.1 The Finite Difference Method

We can arrive at a powerful alternative to shooting methods by taking the concept of multiple shooting to its logical limit. Recall that in multiple shooting, we divided the domain $[0, T]$ into M sub-intervals and solved an IVP over each small segment $[t_i, t_{i+1}]$. We then glued these solutions together by enforcing continuity $\mathbf{x}(t_{i+1}; \mathbf{c}_i) = \mathbf{c}_{i+1}$. Now, imagine we decrease the interval size Δt until it is so small that a single step of the simplest explicit Euler method is sufficiently accurate to represent the ODE dynamics. In this limit, the condition that the solution at the end of interval i matches the start of interval $i + 1$ becomes a simple algebraic relationship:

$$\mathbf{c}_{i+1} = \mathbf{c}_i + \Delta t \mathbf{f}(\mathbf{c}_i, t_i) \quad (9.3.1)$$

This equation relates the values of the solution at adjacent grid points directly without needing to run an ODE solver. If we rearrange this, we see that it effectively approximates the derivative using the slope between points. This viewpoint is what motivates finite differences. Instead of integrating the differential equation, we discretize the differential operator itself. We replace the continuous domain with a discrete grid of $M + 1$ points (nodes) and replace the derivatives in the differential equation with algebraic difference quotients involving the values at these nodes. This converts the BVP into a large system of coupled nonlinear algebraic equations, which we can solve using Newton's method.

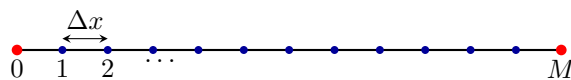


Figure 9.3.1: Discretization of the domain into M intervals of width Δx .

Finite Difference Approximations To formalize this, we define a grid $x_m = x_0 + m\Delta x$ for $m = 0, 1, \dots, M$, where Δx is the grid spacing. We seek approximate values $C_m \approx c(x_m)$. We approximate the derivatives $f^{(p)}(x)$ at a node using a weighted sum of function values at neighboring nodes:

$$f^{(p)}(x_0) \approx \frac{1}{(\Delta x)^p} \sum_m \alpha_m f(x_m) \quad (9.3.2)$$

We can derive these coefficients α_m and analyze their accuracy using Taylor series expansions. For the first derivative $f'(x)$, a symmetric **central difference** uses the points immediately to the left and right:

$$f'(x) \approx \frac{f(x + \Delta x) - f(x - \Delta x)}{2\Delta x} \quad (9.3.3)$$

By expanding $f(x + \Delta x)$ and $f(x - \Delta x)$ in Taylor series up to the third order and subtracting them, terms involving $f(x)$ and $f''(x)$ cancel out, leaving the first derivative term plus an error term proportional to $(\Delta x)^2 f'''(x)$. Thus, this approximation is *second-order accurate*, or $O(\Delta x^2)$.

For the second derivative $f''(x)$, which appears frequently in diffusion and wave problems, we combine the values at $x - \Delta x$, x , and $x + \Delta x$:

$$f''(x) \approx \frac{f(x - \Delta x) - 2f(x) + f(x + \Delta x)}{(\Delta x)^2} \quad (9.3.4)$$

Again, Taylor analysis confirms this approximation is second-order accurate. Specifically, the error term scales with $(\Delta x)^2 f^{(4)}(x)$. High-order accuracy is desirable because it implies that refining the grid (reducing Δx) leads to a rapid reduction in numerical error.

Application: Reaction-Diffusion System Let us apply this method to the concrete physical example of a steady-state reaction-diffusion system in a 1D slab. The governing equation is

$$\frac{d^2 c}{dx^2} = -r(c) \quad (9.3.5)$$

where $c(x)$ is the concentration and $r(c)$ is the reaction rate. The domain is $x \in [0, 1]$. We impose generalized Robin boundary conditions at both ends, which relate the concentration and its flux:

$$x = 0 : \quad A_0 \frac{dc}{dx} + B_0 c = C_0 \quad (9.3.6)$$

$$x = 1 : \quad A_1 \frac{dc}{dx} + B_1 c = C_1 \quad (9.3.7)$$

We discretize the domain into M intervals of size $\Delta x = 1/M$, creating $M + 1$ unknown variables $c_0, c_1, \dots, c_M$. For every interior node $m = 1, \dots, M - 1$, we replace the second derivative with the central difference approximation derived above. Substituting this into the ODE produces one algebraic equation per node:

$$\frac{1}{(\Delta x)^2} (c_{m-1} - 2c_m + c_{m+1}) + r(c_m) = 0 \quad (9.3.8)$$

This provides $M - 1$ equations for our $M + 1$ unknowns. To close the system, we must derive equations for the two boundary nodes $m = 0$ and $m = M$ using the boundary conditions. The boundary conditions involve the first derivative dc/dx . We must approximate this derivative using the available grid points. At the left boundary ($x = 0$, node $m = 0$), we cannot use a central difference because there is no node at x_{-1} . Instead, we can use a forward difference. A simple first-order forward difference is $(c_1 - c_0)/\Delta x$. However, since our interior scheme is second-order accurate ($O(\Delta x^2)$), using a first-order boundary approximation ($O(\Delta x)$) would degrade the global accuracy of the solution. To maintain second-order accuracy, we employ a three-point forward difference approximation for the derivative at the boundary. The boundary equation at $x = 0$ becomes

$$g_0(\mathbf{c}) = A_0 \left[\frac{-3c_0 + 4c_1 - c_2}{2\Delta x} \right] + B_0 c_0 - C_0 = 0 \quad (9.3.9)$$

Similarly, at the right boundary ($x = 1$, node $m = M$), we use a three-point backward difference:

$$g_M(\mathbf{c}) = A_1 \left[\frac{3c_M - 4c_{M-1} + c_{M-2}}{2\Delta x} \right] + B_1 c_M - C_1 = 0 \quad (9.3.10)$$

(Note: these three-point formulas require $M \geq 2$ so that the necessary interior points exist.) Collecting the equations for the boundaries and the interior points, we obtain a system of $M + 1$ nonlinear algebraic equations for the $M + 1$ unknowns. We define the residual vector function $\mathbf{G}(\mathbf{c}) = \mathbf{0}$, where

$$\mathbf{G}(\mathbf{c}) = \begin{bmatrix} g_0(\mathbf{c}) \\ \frac{1}{(\Delta x)^2}(c_0 - 2c_1 + c_2) + r(c_1) \\ \vdots \\ \frac{1}{(\Delta x)^2}(c_{M-2} - 2c_{M-1} + c_M) + r(c_{M-1}) \\ g_M(\mathbf{c}) \end{bmatrix} = \mathbf{0} \quad (9.3.11)$$

This system is usually sparse and structured (tridiagonal or pentadiagonal), which makes it computationally efficient to solve using Newton-Raphson methods, such as `fsolve` in MATLAB or `scipy.optimize.root` in Python.

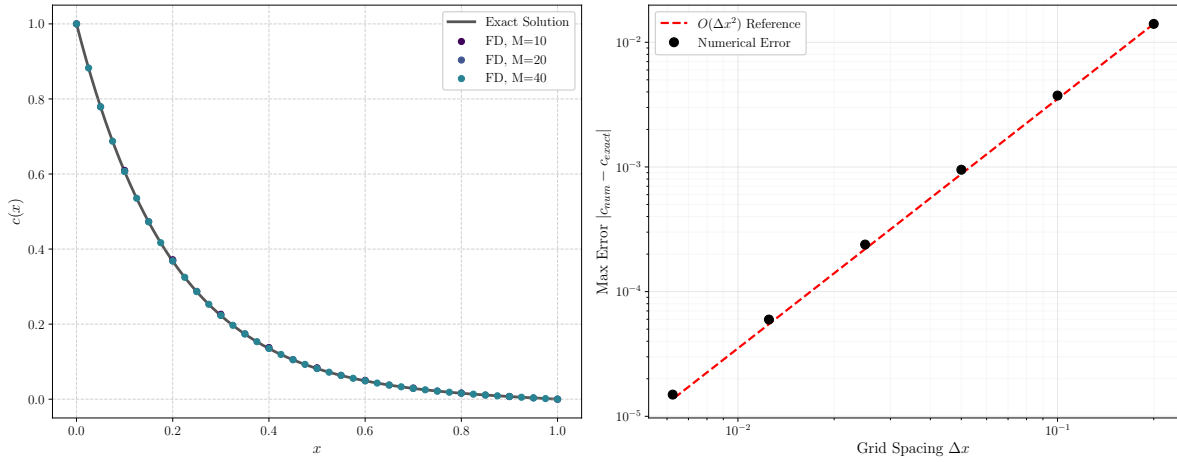


Figure 9.3.2: Finite difference solution and convergence analysis. Left: Numerical solutions for the reaction-diffusion problem ($c'' = -kc$ with boundary conditions $c(0) = 1, c(1) = 0$) using varying grid resolutions ($M = 10, 20, 40$) compared to the exact analytical solution (solid gray line). Right: Log-log convergence plot showing the maximum error between the numerical and exact solutions versus the grid spacing Δx . The error data points (black dots) align almost perfectly with the red dashed reference line representing quadratic scaling, which confirms the theoretical $O(\Delta x^2)$ accuracy of the central difference scheme.

The accuracy of the numerical solution depends heavily on the grid spacing. As shown in Figure 9.3.2, a plot of the error against the grid spacing Δx on a logarithmic scale has a slope of 2. This empirically verifies that our choice of central differences for the interior and three-point differences for the boundaries gives a globally second-order accurate solution. While the finite difference method requires solving a large simultaneous system (unlike the shooting method, which solves small IVPs sequentially), it is generally more robust and less sensitive to initial guesses, particularly for stiff or unstable differential equations.

Accuracy Analysis of Finite Differences To justify the accuracy of the finite difference approximations used in the previous section, we use Taylor's theorem. This allows us to quantify the local truncation error introduced by discretizing the continuous differential operator. Consider the central difference approximation for the first derivative. We expand the function values at the neighboring nodes $x + \Delta x$ and $x - \Delta x$ around the central point x :

$$f(x + \Delta x) = f(x) + \Delta x f'(x) + \frac{(\Delta x)^2}{2} f''(x) + \frac{(\Delta x)^3}{6} f'''(x) + O(\Delta x^4) \quad (9.3.12)$$

$$f(x - \Delta x) = f(x) - \Delta x f'(x) + \frac{(\Delta x)^2}{2} f''(x) - \frac{(\Delta x)^3}{6} f'''(x) + O(\Delta x^4) \quad (9.3.13)$$

Subtracting the second equation from the first causes the even-order derivative terms to cancel out. We are left with the difference $f(x + \Delta x) - f(x - \Delta x) = 2\Delta x f'(x) + \frac{(\Delta x)^3}{3} f'''(x) + \dots$. Dividing by $2\Delta x$ isolates the first derivative:

$$\frac{f(x + \Delta x) - f(x - \Delta x)}{2\Delta x} = f'(x) + \frac{(\Delta x)^2}{6} f'''(x) + \dots \quad (9.3.14)$$

The dominant error term is proportional to $(\Delta x)^2$. Consequently, we say this approximation is second-order accurate, or $O(\Delta x^2)$. A similar procedure for the second derivative approximation $f''(x) \approx (f(x + \Delta x) - 2f(x) + f(x - \Delta x))/(\Delta x)^2$ also yields an error term proportional to $(\Delta x)^2 f^{(4)}(x)$, confirming that the standard central difference scheme maintains second-order consistency throughout the domain.

Relation to Multiple Shooting We've mentioned this before, but it is instructive to recognize that the finite difference method is not entirely distinct from the shooting methods discussed earlier; rather, it is a limiting case. Recall that in the multiple shooting method, we partition the domain $[0, T]$ into M sub-intervals and solve an IVP on each segment, enforcing continuity $\mathbf{c}_i = \mathbf{x}(t_i; \mathbf{c}_{i-1})$ at the nodes. If we decrease the interval size Δt until it is small enough that a single step of an explicit Euler integrator is sufficient, the continuity condition simplifies. The integration step becomes $\mathbf{c}_i = \mathbf{c}_{i-1} + \Delta t \mathbf{f}(\mathbf{c}_{i-1}, t_{i-1})$. Rearranging this yields $(\mathbf{c}_i - \mathbf{c}_{i-1})/\Delta t = \mathbf{f}(\mathbf{c}_{i-1})$, which is exactly the finite difference formulation using a forward difference. Thus, we can view the finite difference method as a multiple shooting method where the number of intervals M approaches the number of grid points, and the “shooting” (IVP integration) is replaced by a simple algebraic step. The resulting system of nonlinear equations involves $(M + 1)N$ variables and equations, which can be solved simultaneously using Newton-Raphson.

9.3.2 Basis Function Expansions

Basis function expansion methods (also called spectral methods or weighted residual methods) take a different approach than the grid-discretization methods discussed above. We approximate the solution $c(x)$ as a continuous function constructed from a linear combination of pre-selected basis functions $\{\Phi_j(x)\}_{j=1}^M$. The approximate solution is written as

$$c(x) \approx \sum_{j=1}^M c_j \Phi_j(x) \quad (9.3.15)$$

The problem of solving the differential equation is thus transformed into the problem of finding the optimal set of scalar coefficients $c_1, \dots, c_M$. This approach is powerful because, with a wise choice of basis functions (such as orthogonal polynomials or Fourier modes), we can achieve high accuracy with relatively few coefficients.

Handling Boundary Conditions Before determining the coefficients, we must ensure the boundary conditions are satisfied. Consider a general BVP with Dirichlet boundary conditions:

$$\frac{d^2 c}{dx^2} = f(x), \quad c(0) = a, \quad c(1) = b \quad (9.3.16)$$

If our basis functions are arbitrary, satisfying $c(0) = a$ and $c(1) = b$ constrains the coefficients c_j in a complicated way. To simplify this, we use a splitting technique. We assume the solution can be decomposed into a particular function $c_p(x)$ and a homogeneous function $c_h(x)$:

$$c(x) = c_p(x) + c_h(x) \quad (9.3.17)$$

The particular function $c_p(x)$ is chosen solely to satisfy the boundary conditions. For instance, the linear function $c_p(x) = a + (b - a)x$ satisfies $c_p(0) = a$ and $c_p(1) = b$. The remaining part, $c_h(x)$, must then satisfy homogeneous boundary conditions ($c_h(0) = 0, c_h(1) = 0$) so that the sum matches the original BCs. By substituting this decomposition into the original ODE, we get a new differential equation for $c_h(x)$ where the forcing term is modified by derivatives of $c_p(x)$ but the boundary conditions are strictly zero. Without loss of generality, we can therefore restrict our study to BVPs with homogeneous boundary conditions. We choose basis functions $\{\Phi_j(x)\}$ that individually satisfy $\Phi_j(0) = 0$ and $\Phi_j(1) = 0$. Consequently, any linear combination of these functions automatically satisfies the boundary conditions, allowing us to focus entirely on satisfying the differential equation inside the domain.

Weighted Residuals We substitute our expansion $c(x) = \sum c_j \Phi_j(x)$ into the differential equation $c'' - f(x) = 0$ (Eq. 9.3.16). Since our approximation is not exact, this substitution produces a non-zero residual function $R(x; \mathbf{c})$:

$$R(x; \mathbf{c}) = \sum_{j=1}^M c_j \Phi_j''(x) - f(x) \neq 0 \quad (9.3.18)$$

We cannot force the residual to be zero everywhere (that would require the exact solution). Instead, we enforce that the residual is zero in a weighted average sense. We introduce a set of test functions (or weight functions) $\{\Psi_i(x)\}_{i=1}^M$ and require the integral of the residual against each test function to vanish:

$$\int_0^1 \Psi_i(x) R(x; \mathbf{c}) dx = 0, \quad \text{for } i = 1, \dots, M \quad (9.3.19)$$

Expanding this integral, we get a system of linear equations for the coefficients c_j :

$$\sum_{j=1}^M c_j \left(\int_0^1 \Psi_i(x) \Phi_j''(x) dx \right) = \int_0^1 \Psi_i(x) f(x) dx \quad (9.3.20)$$

This can be written as a matrix system $\mathbf{A}\mathbf{c} = \mathbf{b}$. The choice of the test functions $\{\Psi_i(x)\}$ determines the specific numerical method. Two such choices are the collocation method and the Galerkin method.

The Collocation Method In the collocation method, we choose the test functions to be Dirac delta functions centered at specific points x_i : $\Psi_i(x) = \delta(x - x_i)$. The integral property of the delta function sifts out the value of the residual at x_i . Physically, this implies we are forcing the differential equation to hold exactly at a set of M discrete collocation points $\{x_i\}_{i=1}^M$. The equations become

$$\sum_{j=1}^M c_j \Phi_j''(x_i) = f(x_i), \quad \text{for } i = 1, \dots, M \quad (9.3.21)$$

Usually, the collocation points are chosen to be the roots of orthogonal polynomials (such as Legendre polynomials) to minimize the error between points, a technique known as orthogonal collocation.

The Galerkin Method In the Galerkin method, we choose the test functions to be the same as the basis functions: $\Psi_i(x) = \Phi_i(x)$. We are basically projecting the residual onto the space spanned by the basis functions and demanding that the error is orthogonal to our approximation space. The condition is

$$\sum_{j=1}^M c_j \left(\int_0^1 \Phi_i(x) \Phi_j''(x) dx \right) = \int_0^1 \Phi_i(x) f(x) dx \quad (9.3.22)$$

If the basis functions $\{\Phi_j(x)\}$ are eigenfunctions of the differential operator (e.g., $\Phi_j'' = \lambda_j \Phi_j$) and are orthogonal (meaning $\int \Phi_i \Phi_j dx = 0$ for $i \neq j$), the matrix on the left-hand side becomes diagonal or highly structured, making the system trivial to solve.

Example: Solving a BVP with Basis Functions

Consider the following BVP:

$$\frac{d^2 c}{dx^2} - c = \exp(-x^2), \quad c(0) = c(1) = 0 \quad (9.3.23)$$

Since the boundary conditions are homogeneous and Dirichlet, a natural choice for basis functions are the sine modes, which are eigenfunctions of the second derivative operator:

$$\Phi_j(x) = \sin(j\pi x), \quad j = 1, \dots, M \quad (9.3.24)$$

These functions satisfy $\Phi_j(0) = \Phi_j(1) = 0$ automatically. We seek $c(x) = \sum c_j \sin(j\pi x)$.

1. Collocation Approach: We choose M points x_i evenly spaced in $(0, 1)$. We substitute the sum into the ODE ($c'' - c$) and evaluate at each x_i :

$$\sum_{j=1}^M c_j [-(j\pi)^2 \sin(j\pi x_i) - \sin(j\pi x_i)] = \exp(-x_i^2) \quad (9.3.25)$$

This is a linear system $\mathbf{A}\mathbf{c} = \mathbf{b}$ where $A_{ij} = -((j\pi)^2 + 1) \sin(j\pi x_i)$.

2. Galerkin Approach: We multiply the residual by $\Phi_i(x) = \sin(i\pi x)$ and integrate from 0 to 1. Due to the orthogonality of sines, the cross terms in the sum vanish ($\int \sin(i\pi x) \sin(j\pi x) dx = 0$ for $i \neq j$). The term involving the second derivative becomes:

$$\int_0^1 \sin(i\pi x) \frac{d^2}{dx^2} (c_i \sin(i\pi x)) dx = -c_i (i\pi)^2 \int_0^1 \sin^2(i\pi x) dx = -c_i \frac{(i\pi)^2}{2} \quad (9.3.26)$$

Similarly, the term $\int c \Phi_i dx$ yields $c_i/2$. The equation for coefficient c_i decouples completely:

$$-\frac{1}{2} [(i\pi)^2 + 1] c_i = \int_0^1 \exp(-x^2) \sin(i\pi x) dx \quad (9.3.27)$$

We can calculate the integral on the right-hand side (numerically or analytically) and directly divide by the scalar factor $-\frac{1}{2}[(i\pi)^2 + 1]$ to find each c_i .

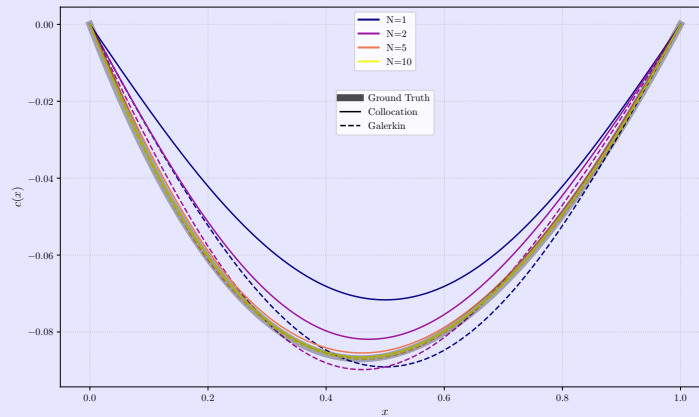


Figure 9.3.3: Comparison of collocation and Galerkin methods for the example BVP. As the number of basis functions N increases (colored lines), both methods converge to the true solution (gray band). The Galerkin method minimizes the global integrated error, while collocation drives the error to zero specifically at the chosen points.

9.3.3 The Finite Element Method

The basis function expansion methods discussed previously, particularly when using global polynomials or eigenfunctions (like sine waves), have a significant computational issue. This issue is that these functions are global; that is, they are non-zero over the entire domain. Consequently, when we compute the inner products for the Galerkin method, every basis function interacts with every other basis function. This results in a dense system matrix $\mathbf{A}$, where every entry is potentially non-zero. Solving a dense linear system requires $O(M^3)$ operations, which becomes prohibitively expensive as the number of modes M increases. Furthermore, finding suitable global basis functions for complex, irregular geometries is often impossible.

To overcome these limitations, we introduce the **finite element method (FEM)**. The philosophy of FEM is to retain the integral formulation of the Galerkin method but to replace the global basis functions with *local* basis functions.

Local Basis Functions We partition the domain $[0, 1]$ into a grid of nodes $x_0, x_1, \dots, x_M$, similar to the finite difference grid. However, instead of approximating the derivative at a point, we define basis functions $\Phi_m(x)$ that are non-zero only over a small sub-interval around node x_m . The simplest choice is the piecewise linear “hat” or “tent” function defined as:

$$\Phi_m(x) = \begin{cases} \frac{x-x_{m-1}}{\Delta x} & x_{m-1} < x \leq x_m \\ \frac{x_{m+1}-x}{\Delta x} & x_m < x < x_{m+1} \\ 0 & \text{otherwise} \end{cases} \quad (9.3.28)$$

where Δx is the spacing between nodes. Each function $\Phi_m(x)$ rises linearly from 0 to 1 and falls back to 0 over a span of $2\Delta x$. Taking linear combinations of these local tent functions allows us to construct a global piecewise-linear approximation of any given function. Importantly, $\Phi_i(x)$ and $\Phi_j(x)$ have no overlap if $|i - j| > 1$. This locality property guarantees that the vast majority of integrals in the Galerkin formulation will be zero, leading to a sparse, banded matrix that is computationally efficient to solve.

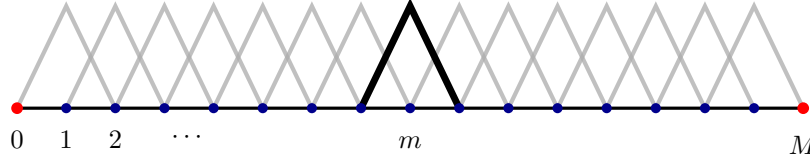


Figure 9.3.4: Local tent basis functions for the FEM.

The Weak Formulation Let us apply this to our model problem:

$$\frac{d^2 c}{dx^2} - c = \exp(-x^2), \quad c(0) = 0, \quad c(1) = 0 \quad (9.3.29)$$

We approximate the solution as a linear combination of these tent functions: $c(x) \approx \sum_{j=1}^{M-1} c_j \Phi_j(x)$. Note that we exclude $j = 0$ and $j = M$ from the unknowns because the boundary conditions $c(0) = 0$ and $c(1) = 0$ fix those coefficients to zero. We now plug this approximation into the Galerkin condition. If we tried to do this in the “strong” form, i.e.,

$$\int_0^1 (c_h''(x) - c_h(x) - f(x)) \Phi_i(x) dx = 0 \quad (9.3.30)$$

we immediately run into the issue that since Φ_j is only piecewise linear, Φ_j' is piecewise constant and Φ_j'' is not an ordinary function at all (it lives at the nodes as delta spikes). Considering this, the standard FEM trick is to integrate by parts once so that we only need first derivatives. Concretely,

$$\int_0^1 c_h''(x) \Phi_i(x) dx = \left[c_h'(x) \Phi_i(x) \right]_0^1 - \int_0^1 c_h'(x) \Phi_i'(x) dx \quad (9.3.31)$$

For interior test functions $i = 1, \dots, M-1$, the hats satisfy $\Phi_i(0) = \Phi_i(1) = 0$, so the boundary term drops:

$$\left[c_h'(x) \Phi_i(x) \right]_0^1 = 0 \quad (9.3.32)$$

So the weak (integrated-by-parts) Galerkin equations become

$$-\int_0^1 c_h'(x) \Phi_i'(x) dx - \int_0^1 c_h(x) \Phi_i(x) dx = \int_0^1 f(x) \Phi_i(x) dx \quad i = 1, \dots, M-1 \quad (9.3.33)$$

where here $f(x) = e^{-x^2}$. Now, we substitute the expansion $c_h = \sum_{j=1}^{M-1} c_j \Phi_j$ and pull the coefficients c_j outside the integrals:

$$\sum_{j=1}^{M-1} c_j \left(- \int_0^1 \Phi_j'(x) \Phi_i'(x) dx - \int_0^1 \Phi_j(x) \Phi_i(x) dx \right) = \int_0^1 f(x) \Phi_i(x) dx \quad (9.3.34)$$

At this point, it is helpful to name the two standard FEM matrices. The derivative term is the *stiffness* matrix,

$$K_{ij} = \int_0^1 \Phi_j'(x) \Phi_i'(x) dx \quad (9.3.35)$$

and the term with the functions themselves is the *mass* matrix,

$$M_{ij} = \int_0^1 \Phi_j(x) \Phi_i(x) dx \quad (9.3.36)$$

The right-hand side is just the load vector entry

$$b_i = \int_0^1 f(x) \Phi_i(x) dx \quad (9.3.37)$$

With this notation, the linear system is simply

$$\sum_{j=1}^{M-1} (-K_{ij} - M_{ij}) c_j = b_i \quad i = 1, \dots, M-1, \quad (9.3.38)$$

or, in matrix form, $\mathbf{A}\mathbf{c} = \mathbf{b}$ with $A_{ij} = -K_{ij} - M_{ij}$. (Note that since our operator is formally $d^2/dx^2 - I$, which is negative definite, the matrix $\mathbf{A}$ is negative definite. In many texts, the problem is multiplied by -1 to yield a positive-definite system $(K + M)\mathbf{c} = -\mathbf{b}$. Just be aware of this.) The big practical win here is sparsity: each hat Φ_i only overlaps with Φ_{i-1} and Φ_{i+1} , so

$$K_{ij} = M_{ij} = 0 \quad \text{whenever} \quad |i - j| > 1 \quad (9.3.39)$$

meaning $\mathbf{A}$ is tridiagonal. On a uniform grid with spacing Δx , the stiffness entries come straight from the fact that Φ_i' is $\frac{1}{\Delta x}$ on the left element and $-\frac{1}{\Delta x}$ on the right element. For the diagonal,

$$K_{ii} = \int_{x_{i-1}}^{x_i} \left(\frac{1}{\Delta x} \right)^2 dx + \int_{x_i}^{x_{i+1}} \left(-\frac{1}{\Delta x} \right)^2 dx = \frac{2}{\Delta x} \quad (9.3.40)$$

and for the neighbors (say i and $i+1$), the derivatives have opposite signs on the shared element $[x_i, x_{i+1}]$, giving

$$K_{i,i+1} = \int_{x_i}^{x_{i+1}} \left(-\frac{1}{\Delta x} \right) \left(\frac{1}{\Delta x} \right) dx = -\frac{1}{\Delta x} \quad (9.3.41)$$

and similarly $K_{i,i-1} = -\frac{1}{\Delta x}$. For the mass matrix, you can either do the little triangles-by-hand integrals or just remember the standard results for linear elements. The diagonal is

$$M_{ii} = \int_0^1 \Phi_i(x)^2 dx = \frac{2}{3} \Delta x, \quad (9.3.42)$$

and the off-diagonal neighbor coupling is

$$M_{i,i+1} = \int_0^1 \Phi_i(x) \Phi_{i+1}(x) dx = \frac{1}{6} \Delta x \quad (9.3.43)$$

with the same value for $M_{i,i-1}$. Putting these together, the i th row of the FEM system (only involving c_{i-1}, c_i, c_{i+1}) reads

$$-\left(K_{i,i-1} c_{i-1} + K_{ii} c_i + K_{i,i+1} c_{i+1} \right) - \left(M_{i,i-1} c_{i-1} + M_{ii} c_i + M_{i,i+1} c_{i+1} \right) = b_i, \quad (9.3.44)$$

i.e.,

$$-\left(-\frac{1}{\Delta x}c_{i-1} + \frac{2}{\Delta x}c_i - \frac{1}{\Delta x}c_{i+1}\right) - \left(\frac{\Delta x}{6}c_{i-1} + \frac{2\Delta x}{3}c_i + \frac{\Delta x}{6}c_{i+1}\right) = \int_0^1 e^{-x^2} \Phi_i(x) dx \quad (9.3.45)$$

If we group the terms in the nicer “stencil” form, we get

$$\frac{1}{\Delta x}(c_{i-1} - 2c_i + c_{i+1}) - \frac{\Delta x}{6}(c_{i-1} + 4c_i + c_{i+1}) = \int_0^1 e^{-x^2} \Phi_i(x) dx \quad (9.3.46)$$

This system is tridiagonal and sparse, so we have solved the computational efficiency problem of global basis functions. Interestingly, if we inspect the first term, we see it matches the standard central difference approximation multiplied by Δx . However, the FEM formulation also has a mass matrix term (the second group) which averages the reaction term over neighboring nodes (this causes higher accuracy for certain problem classes).

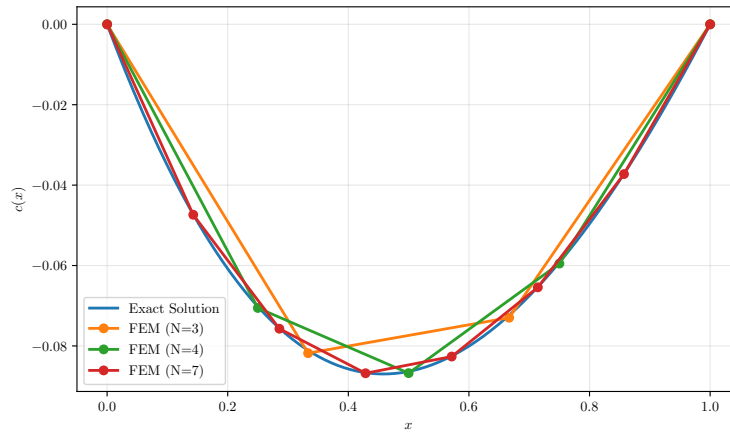


Figure 9.3.5: Finite element method solution to $c'' - c = \exp(-x^2)$. Using piecewise linear tent basis functions, the solution converges rapidly. Even with a small number of elements ($N = 3, 4, 7$), the method captures the curvature of the solution effectively.

The finite element method thus combines the best of both worlds: (1) the geometric flexibility and sparse matrix structure of finite differences, and (2) the error minimization and integral formulation of weighted residual methods.

9.3.4 The Finite Volume Method

FEM is very clean mathematically (same functions for both the expansion and the testing), but in a lot of transport problems, we really want to build conservation in by construction. That is the motivating idea behind the finite volume method: instead of asking for the residual to be orthogonal to the same tent functions, we ask for the residual to have zero *average* over a little control volume around each node. You can still think of this as a Galerkin/weighted-residual method, but with a different set of test functions, the *conjugate basis functions*. To be explicit, we keep the same piecewise-linear approximation for the solution using the tent functions $\{\Phi_i\}_{i=1}^{M-1}$,

$$c(x) \approx \sum_{j=1}^{M-1} c_j \Phi_j(x) \quad (9.3.47)$$

so $c(x)$ is continuous and varies linearly between nodes. The change is the test function. For each interior node $x_i = i\Delta x$, we associate a control volume that extends halfway to the neighboring nodes; it is convenient to denote its faces by

$$x_{i-1/2} = x_i - \frac{\Delta x}{2}, \quad x_{i+1/2} = x_i + \frac{\Delta x}{2} \quad (9.3.48)$$

The conjugate basis function Ψ_i is just a normalized top-hat on that control volume (so integrals become averages):

$$\Psi_i(x) = \begin{cases} \frac{1}{\Delta x}, & (i - \frac{1}{2})\Delta x < x < (i + \frac{1}{2})\Delta x, \\ 0, & \text{otherwise} \end{cases} \quad (9.3.49)$$

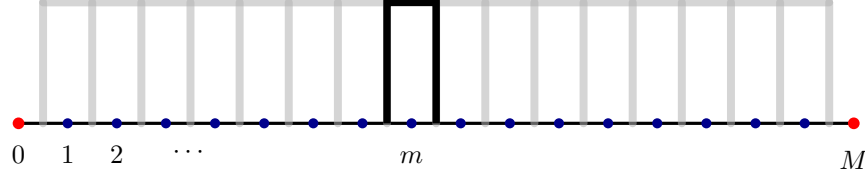


Figure 9.3.6: Finite volume method conjugate basis functions (control volumes).

Now, we apply this to our model problem

$$\frac{d^2 c}{dx^2} - c = \exp(-x^2), \quad c(0) = c(1) = 0 \quad (9.3.50)$$

The weighted residual statement with the conjugate test function is

$$\int_0^1 \left[\frac{d^2 c}{dx^2} - c \right] \Psi_i(x) dx = \int_0^1 \exp(-x^2) \Psi_i(x) dx, \quad i = 1, 2, \dots, M-1 \quad (9.3.51)$$

and because Ψ_i is only nonzero on the control volume, this is just the local averaged balance

$$\frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} \left[\frac{d^2 c}{dx^2} - c \right] dx = \frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} \exp(-x^2) dx \quad (9.3.52)$$

The second derivative term turns into a clean flux difference automatically (this is the conservation-law flavor showing up). Using the fundamental theorem of calculus,

$$\frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{d^2 c}{dx^2} dx = \frac{1}{\Delta x} \left[\left. \frac{dc}{dx} \right|_{x_{i+1/2}} - \left. \frac{dc}{dx} \right|_{x_{i-1/2}} \right] \quad (9.3.53)$$

Since our approximation is linear on each interval, $\frac{dc}{dx}$ is constant on each cell, so the face slopes are the usual two-point slopes:

$$\left. \frac{dc}{dx} \right|_{x_{i+1/2}} \approx \frac{c_{i+1} - c_i}{\Delta x}, \quad \left. \frac{dc}{dx} \right|_{x_{i-1/2}} \approx \frac{c_i - c_{i-1}}{\Delta x} \quad (9.3.54)$$

Plugging these into the flux difference gives the familiar central-difference stencil (but now it came from a flux balance, not Taylor series):

$$\frac{1}{\Delta x} \left[\left. \frac{dc}{dx} \right|_{x_{i+1/2}} - \left. \frac{dc}{dx} \right|_{x_{i-1/2}} \right] \approx \frac{1}{(\Delta x)^2} (c_{i+1} - 2c_i + c_{i-1}) \quad (9.3.55)$$

The other term is the “volume-averaged reaction” piece. In the integrated equation, it appears as

$$-\frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} c(x) dx \quad (9.3.56)$$

and we evaluate it by substituting the same tent-function expansion

$$c(x) \approx \sum_{j=1}^{M-1} c_j \Phi_j(x) \quad (9.3.57)$$

Over the control volume $[x_{i-1/2}, x_{i+1/2}]$, only Φ_{i-1} , Φ_i , and Φ_{i+1} overlap, so the integral reduces to those three coefficients. Carrying out the little “rectangle meets triangle” area calculation gives

$$-\frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} c(x) dx \approx -\frac{1}{8} (c_{i+1} + 6c_i + c_{i-1}) \quad (9.3.58)$$

Intuitively, this is a weighted average over the volume: the center node gets weight 6/8 and the neighbors get 1/8 each, because the control volume cuts through the tent shapes at half-height. On the right-hand side, we just get the control-volume average of the source term:

$$\frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} \exp(-x^2) dx \quad (9.3.59)$$

which you can evaluate numerically (Gaussian quadrature, etc.). Putting the flux term and the volume-averaged terms together, the FVM discretization for each interior node is

$$\frac{1}{(\Delta x)^2} (c_{i+1} - 2c_i + c_{i-1}) - \frac{1}{8} (c_{i+1} + 6c_i + c_{i-1}) = \frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} \exp(-x^2) dx, \quad i = 1, 2, \dots, M-1 \quad (9.3.60)$$

The boundary conditions still just fix the endpoint coefficients,

$$c_0 = 0, \quad c_M = 0 \quad (9.3.61)$$

so, just like FEM, we end up with a sparse tridiagonal linear system $\mathbf{A}\mathbf{c} = \mathbf{b}$ (only c_{i-1} , c_i , c_{i+1} show up in the i th equation). The main difference is *where the stencil comes from*: FVM is literally enforcing a control-volume balance (flux in/out + volume-averaged reaction = volume-averaged source), which is why it’s so popular in transport and fluids.

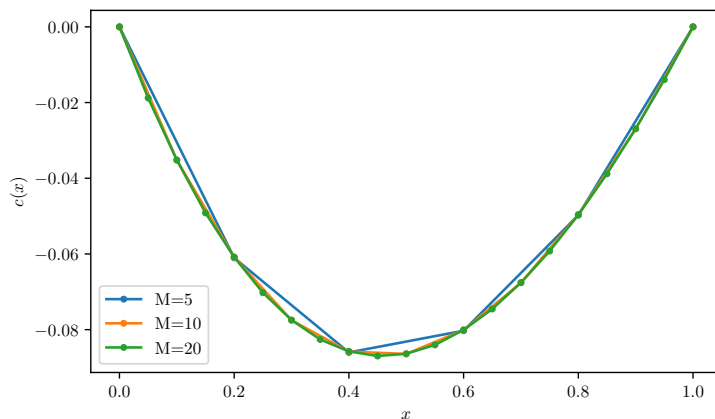


Figure 9.3.7: Finite volume method solution ($M = 5, 10, 20$) for the same problem as FEM. While the curve looks similar to FEM, the coefficients in the underlying matrix differ slightly due to the specific averaging properties of the rectangular test functions.

9.4 Summary of BVP Methods

We’ve looked at several families of BVP solvers, and a useful way to compare them is by the mental model each method imposes on the problem: do you treat the BVP like an IVP, do you approximate the operator directly, or do you enforce the equation in an averaged/projection sense?

The shooting method reframes a boundary value problem as a sequence of initial value problems. You guess the missing initial condition(s), integrate forward with an ODE solver, and then adjust the guess so that the

terminal boundary condition is satisfied (often using a Newton or secant update built from the mismatch at the far boundary). When it works, it's conceptually straightforward and takes great advantage of the robustness of IVP integrators, but it can become delicate as the domain grows or the dynamics become stiff. In these cases, small errors in the “aim” can amplify exponentially, and multiple solutions (or unstable manifolds) can make convergence very sensitive to the initial guess.

The finite difference method stays closest to the differential equation itself. You put down a mesh, replace derivatives by difference quotients derived from local Taylor expansions, and solve the resulting algebraic system. This is typically the quickest path from PDE/ODE to code on a simple 1D domain, and it produces sparse banded matrices with predictable structure. The main limitations show up when the geometry, coefficients, or boundary conditions stop being simple: complex boundaries, strong coefficient variation, and nonuniform meshes require more bookkeeping, and it's easy to lose consistency or accuracy if stencils aren't adapted carefully.

In basis-function expansions (spectral methods), you take the opposite stance: instead of local stencils, you approximate the solution globally as a combination of smooth basis functions (Fourier modes, Chebyshev polynomials, eigenfunctions, etc.). For smooth solutions on simple domains, this can be extraordinarily accurate; errors often drop faster than any fixed algebraic order as you increase the number of modes. The tradeoff is that the discretization tends to couple many degrees of freedom at once, which leads to dense matrices (or global transforms) and makes complicated geometries, sharp layers, discontinuities, or highly localized features more challenging without additional machinery.

The finite element method builds the approximation from local pieces (low-order polynomials on elements) and enforces the equation in a weak (integral) sense by testing against the same basis functions (Galerkin). This perspective naturally handles variable coefficients, complicated geometries (in higher dimensions), and mixed boundary conditions. Yet, it still preserves a clear notion of stability and error control through functional analysis. In practice, FEM's strength is that it gives you a principled way to construct sparse systems with good approximation properties and a rigorous framework for refinement and convergence.

Finally, the finite volume method is closest to a conservation-law viewpoint. You still represent the solution with local degrees of freedom, but you enforce the governing equation by integrating it over control volumes. Equivalently, you test against piecewise-constant “top-hat” functions. This makes the discrete equations read like a physical balance: flux in minus flux out plus any volumetric production/consumption equals the volumetric source. That structure is exactly why FVM dominates in transport and fluid problems. In the FVM, conservation is built in by construction, and it remains meaningful even when coefficients are discontinuous or the solution develops sharp gradients.

9.5 Exercises

Exercise 9.1

Suppose we are solving a BVP using the Shooting Method. If an initial guess c gives a result $x(\tau; c) > x_f$, where $x(\tau) = x_f$ is the boundary condition for

$$\frac{dx}{dt} = g(f(t), t) \quad t \in [0, \tau]$$

and the sensitivity, $S(\tau; c)$, is negative, would the Shooting Method suggest to increase or decrease c ?

Exercise 9.2

If we are using collocation to approximate the solution of

$$\frac{dx}{dt} = f(x(t), t) \quad g(x(t_0), x(t_f)) = 0$$

using

$$x(t) = \sum_{i=1}^N c_i \cos\left(\frac{i\pi t}{t_f - t_0}\right),$$

how many collocation points, $\{t_i\}$, do we need to choose?

Exercise 9.3

(*) Consider the following initial value problem:

$$\frac{dx}{dt} = -x, \quad x(0) = x_0.$$

- Derive a linear system whose solution gives x at each grid point. Use a step size of Δt , a centered 2nd-order finite difference for the interior points, and a backward 1st-order finite difference for the final point.
- What is the sparsity pattern of the resulting linear system? Is there a relevant algorithm for solving systems with this structure? How does the computational complexity scale with Δt when the simulation horizon $[0, T]$ is fixed?

Exercise 9.4

Classify each of the following as Dirichlet, Neumann, Robin, or none of these.

$$(a) \quad x(0) \cdot \frac{dx}{dt} \Big|_{t=0} = 1.$$

$$(b) \quad \text{For the ODE } \frac{d^2 C}{dx^2} = -r(C), \text{ the condition } C'(0) = 1.$$

Exercise 9.5

Let $\mathbf{S} = \frac{\partial \mathbf{x}(T)}{\partial \mathbf{c}}$ be the sensitivity matrix of the ODE solution at the final time with respect to the initial condition $\mathbf{c} = \mathbf{x}(t_0)$, for the BVP $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$, $\mathbf{g}(\mathbf{x}(t_0), \mathbf{x}(T)) = \mathbf{0}$.

- Why is $\mathbf{S}$ useful when applying the shooting method?
- Describe two methods for computing $\mathbf{S}$.

Chapter 10

Numerical Solution of Partial Differential Equations

So far, our work on differential equations has focused on *ordinary* differential equations, where the unknown function depends on a single independent variable (usually time, t , or a single spatial coordinate, x). However, many physical phenomena involve quantities that vary in both space and time, or across multiple spatial dimensions simultaneously. To model these systems, we must turn to *partial* differential equations (PDEs). Mathematically, the distinction comes from the dimensionality of the domain. An ODE involves an unknown function c and its derivatives with respect to a single variable. For a time-dependent problem over the interval $[t_0, t_f]$, the general form is

$$f\left(t; c, \frac{dc}{dt}, \frac{d^2c}{dt^2}, \dots\right) = 0 \quad \forall t \in [t_0, t_f] \quad (10.0.1)$$

A PDE, by contrast, involves an unknown function $c(x_1, \dots, x_n)$ depending on multiple independent variables and relates its partial derivatives. For a general domain Ω , the equation takes the form

$$f\left(x_1, \dots, x_n; c, \frac{\partial c}{\partial x_1}, \dots, \frac{\partial^2 c}{\partial x_1^2}, \dots\right) = 0 \quad \forall (x_1, \dots, x_n) \in \Omega \quad (10.0.2)$$

PDEs are typically formulated as BVPs (much like the ODE-BVP case). We require the PDE to hold within the interior of the domain Ω , and we must specify conditions on the boundary $\partial\Omega$ to close the system. For example, a Dirichlet boundary condition prescribes the value of the function on the boundary:

$$c(x_1, \dots, x_n) = 0 \quad \forall (x_1, \dots, x_n) \in \partial\Omega \quad (10.0.3)$$

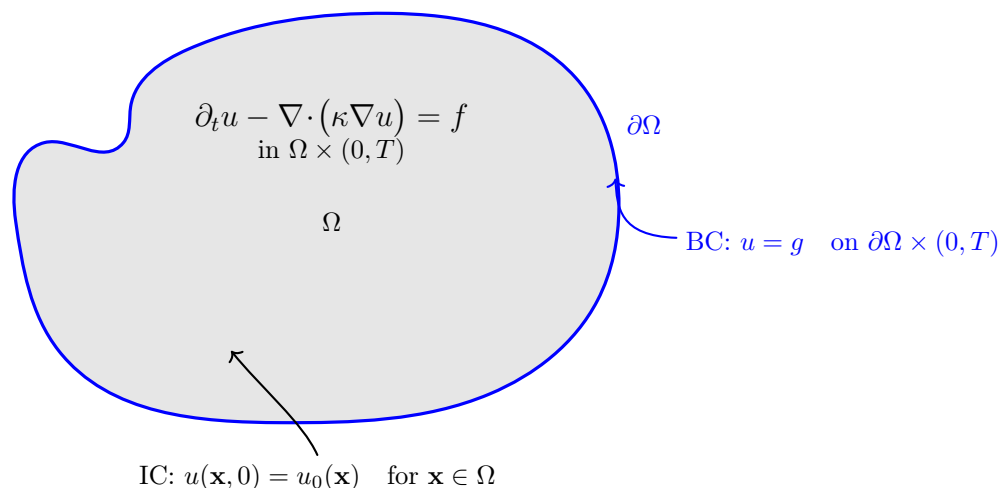


Figure 10.0.1: A generic PDE domain Ω with boundary $\partial\Omega$, a sample PDE, and sample BC/IC.

Why do we need this higher complexity? We live in a three-dimensional world, and engineering systems like plug flow reactors or heat exchangers have spatial structure that cannot always be lumped into a single point. Furthermore, we are often interested in transient behaviors, or how a system evolves from a non-steady state. While ODEs can handle time evolution (IVPs) or spatial equilibrium (BVPs), PDEs allow us to handle both simultaneously.

Solving these equations is a significant challenge. Unlike ODEs, where existence and uniqueness theorems are often straightforward (e.g., Lipschitz continuity for IVPs), characterizing the solution of a PDE is generally far from trivial. Analytical solutions exist only for a small number of simple instances. In most practical engineering scenarios involving nonlinearity or irregular domains, analytical solutions are impossible to find. Therefore, we focus on numerically solving classes of PDEs using methods like the finite difference method, finite element method, finite volume method, and some others.

10.1 Classification of PDEs

Before choosing a numerical method, we must understand the mathematical character of the PDE. Just as conic sections in geometry are classified into ellipses, parabolas, and hyperbolas based on their algebraic form, second-order linear PDEs are classified into three categories: elliptic, parabolic, and hyperbolic. This classification dictates the physical behavior of the solution and the stability and suitability of numerical schemes.

We focus on linear, second-order PDEs in two independent variables (say, x and y). The general form is

$$A \frac{\partial^2 c}{\partial x^2} + B \frac{\partial^2 c}{\partial x \partial y} + C \frac{\partial^2 c}{\partial y^2} + D \frac{\partial c}{\partial x} + E \frac{\partial c}{\partial y} + F(x, y) = 0 \quad (10.1.1)$$

The classification is determined solely by the coefficients of the highest-order derivatives (A , B , and C) via the discriminant $\Delta = B^2 - 4AC$. (Please be aware that many references write the mixed term as $2B \frac{\partial^2 c}{\partial x \partial y}$; in this case, the discriminant becomes $B^2 - AC$.) Note that this classification is in general a *local*, not global, property: for variable-coefficient or nonlinear PDEs, A , B , and C (and hence the sign of Δ) can depend on position or on the solution itself, so a single equation can be elliptic in one part of the domain and hyperbolic in another.

10.1.1 Elliptic PDEs

If the discriminant is negative ($B^2 - 4AC < 0$), the PDE is elliptic. The prototypical example is Laplace's equation, which describes steady-state diffusion or potential fields. In 2D, this is written as

$$\frac{\partial^2 c}{\partial x^2} + \frac{\partial^2 c}{\partial y^2} = 0 \quad (10.1.2)$$

Here, $A = 1$, $C = 1$, and $B = 0$, so $B^2 - 4AC = -4 < 0$.

Physically, elliptic PDEs represent equilibrium or steady-state problems. Information in these systems propagates instantly and globally; a disturbance at any point in the domain Ω or on the boundary $\partial\Omega$ affects the solution everywhere immediately. Consequently, all points in the domain are strongly coupled. These problems are strictly boundary value problems, usually defined on a closed spatial domain like $[0, 1] \times [0, 1]$.

Consider the steady-state concentration profile in a square domain with fixed concentrations on the edges. The formulation might look like this:

$$\frac{\partial^2 c}{\partial x^2} + \frac{\partial^2 c}{\partial y^2} = 0 \quad \forall (x, y) \in [0, 1] \times [0, 1] \quad (10.1.3)$$

Subject to boundary conditions:

$$\begin{aligned} c(x, 0) &= 1, & c(x, 1) &= 0 & \forall x \in [0, 1] \\ c(0, y) &= 1, & c(1, y) &= 0 & \forall y \in [0, 1] \end{aligned} \quad (10.1.4)$$

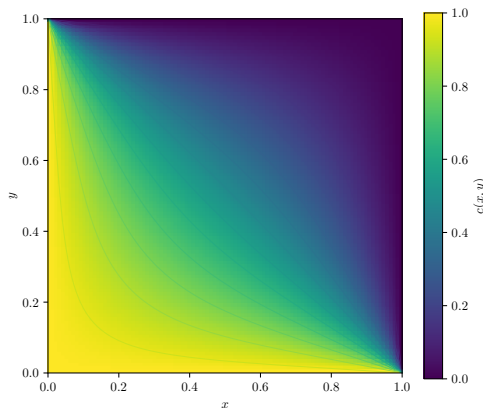


Figure 10.1.1: Solution to an elliptic PDE (Laplace's equation). The solution is smooth and determined by the global equilibrium of boundary values.

10.1.2 Parabolic PDEs

If the discriminant is zero ($B^2 - 4AC = 0$), the PDE is parabolic. The classic example is the 1D diffusion (or heat) equation, which describes time-dependent transient diffusion:

$$\frac{\partial c}{\partial t} = D \frac{\partial^2 c}{\partial x^2} \quad (10.1.5)$$

Rearranging this to match the general form (treating t as the coordinate y), we have $D \frac{\partial^2 c}{\partial x^2} - \frac{\partial c}{\partial t} = 0$. Here $A = D$, while $B = 0$ and $C = 0$ (since there is no second derivative in time). Thus, $B^2 - 4AC = 0$.

Parabolic PDEs describe evolutionary processes that smooth out distributions over time. They are a mix of an IVP (in time) and a BVP (in space). An important physical feature is that disturbances have immediate

nonzero influence everywhere for any $t > 0$ (infinite propagation speed) but with decaying amplitude over distance. Solutions spread over a characteristic length scale $\ell(t) \sim \sqrt{Dt}$.

A standard problem setup for a domain $(t, x) \in [0, \infty) \times (-\infty, \infty)$ is

$$\begin{aligned} \frac{\partial c}{\partial t} &= D \frac{\partial^2 c}{\partial x^2} \\ c(t, \pm\infty) &= 0 \quad \forall t \in [0, \infty) \\ c(0, x) &= \delta(x) \quad \forall x \in (-\infty, \infty) \end{aligned} \tag{10.1.6}$$

where $\delta(x)$ is the Dirac delta function, representing an initial point source that spreads out as a Gaussian distribution over time.

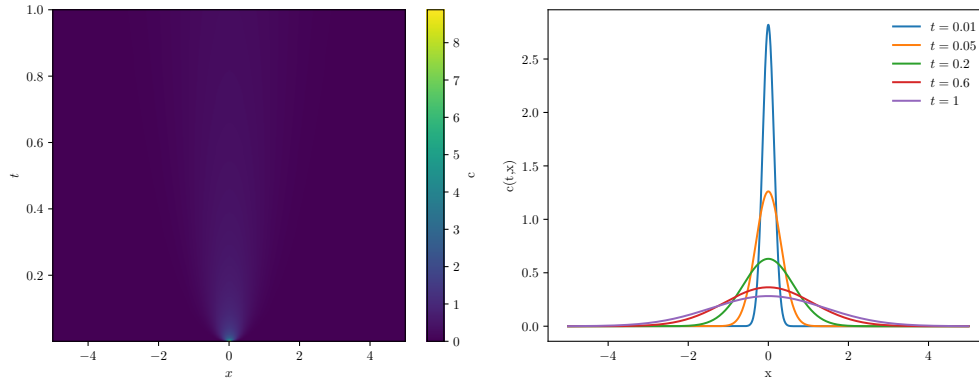


Figure 10.1.2: Solution to a parabolic PDE (diffusion equation). An initial spike spreads and decays over time, scaling with $\sqrt{Dt}$.

10.1.3 Hyperbolic PDEs

If the discriminant is positive ($B^2 - 4AC > 0$), the PDE is hyperbolic. While the second-order wave equation $\frac{\partial^2 c}{\partial t^2} = u^2 \frac{\partial^2 c}{\partial x^2}$ is the standard mathematical example, in transport phenomena we often encounter first-order hyperbolic equations, such as the 1D advection equation:

$$\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = 0 \quad \forall (t, x) \in [0, \infty) \times (-\infty, \infty) \tag{10.1.7}$$

with initial condition $c(0, x) = \bar{c}(x)$. Although the advection equation is first-order (hyperbolic in the characteristics sense), differentiating it appropriately implies the second-order wave equation, which is hyperbolic under the discriminant criterion. From Eq. 10.1.7, we differentiate in t to get

$$\frac{\partial^2 c}{\partial t^2} + u \frac{\partial^2 c}{\partial t \partial x} = 0 \tag{10.1.8}$$

and differentiate in x to get

$$\frac{\partial^2 c}{\partial x \partial t} + u \frac{\partial^2 c}{\partial x^2} = 0 \implies \frac{\partial^2 c}{\partial x \partial t} = -u \frac{\partial^2 c}{\partial x^2} \tag{10.1.9}$$

Substituting the second into the first, we get

$$\frac{\partial^2 c}{\partial t^2} - u^2 \frac{\partial^2 c}{\partial x^2} = 0 \tag{10.1.10}$$

For this equation, $A = -u^2$, $B = 0$, $C = 1$, so $B^2 - 4AC = 4u^2 > 0$.

Hyperbolic PDEs model wave-like propagation or pure advection. Unlike parabolic equations, they do not smooth out discontinuities; sharp fronts in the initial condition can persist indefinitely. Boundary data is communicated at a finite wave speed u . The general solution for the advection equation is a traveling wave:

$$c(x, t) = c(x - ut, 0) \quad (10.1.11)$$

This simply translates the initial profile to the right with velocity u without changing its shape. This assumes $u > 0$ so $x = 0$ is an inflow boundary; if $u < 0$, the inflow boundary is at the right end instead.

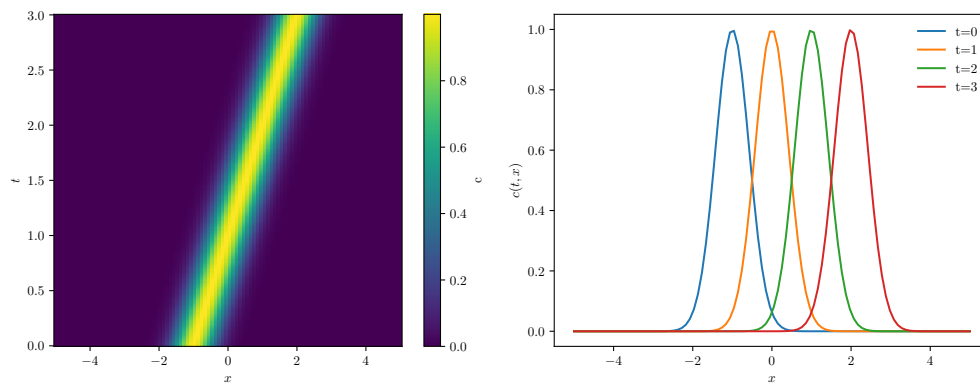


Figure 10.1.3: Solution to a hyperbolic PDE (advection equation). The initial profile translates at speed u without changing shape.

Physical Intuition: The Transport Limits We can unify these classifications by considering the concentration $c(x, t)$ in a 1D channel with fluid velocity u and diffusivity D . The governing time-dependent advection-diffusion equation is

$$\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = D \frac{\partial^2 c}{\partial x^2} \quad (10.1.12)$$

By taking limits of the physical parameters, we recover the three fundamental PDE types. In the limit of a stagnant fluid ($u \approx 0$) where the system evolves over time, the advection term vanishes, leaving the **parabolic** heat equation ($\partial_t c = D \partial_{xx} c$) which smooths distributions diffusively. Conversely, if diffusion is negligible compared to flow ($D \approx 0$), the second-order term drops out and we get the **hyperbolic** advection equation ($\partial_t c + u \partial_x c = 0$) where profiles translate without distortion. Finally, if the system reaches a steady state ($\partial_t c = 0$) dominated by diffusion, we obtain the **elliptic** 1D Laplace equation ($\partial_{xx} c = 0$). This context is vital because numerical schemes are often specialized; a method designed for the smooth solutions of parabolic diffusion may fail for the sharp fronts of hyperbolic advection. The limits are shown schematically in [Figure 10.1.4](#) and also summarized in [Table 10.1.1](#).

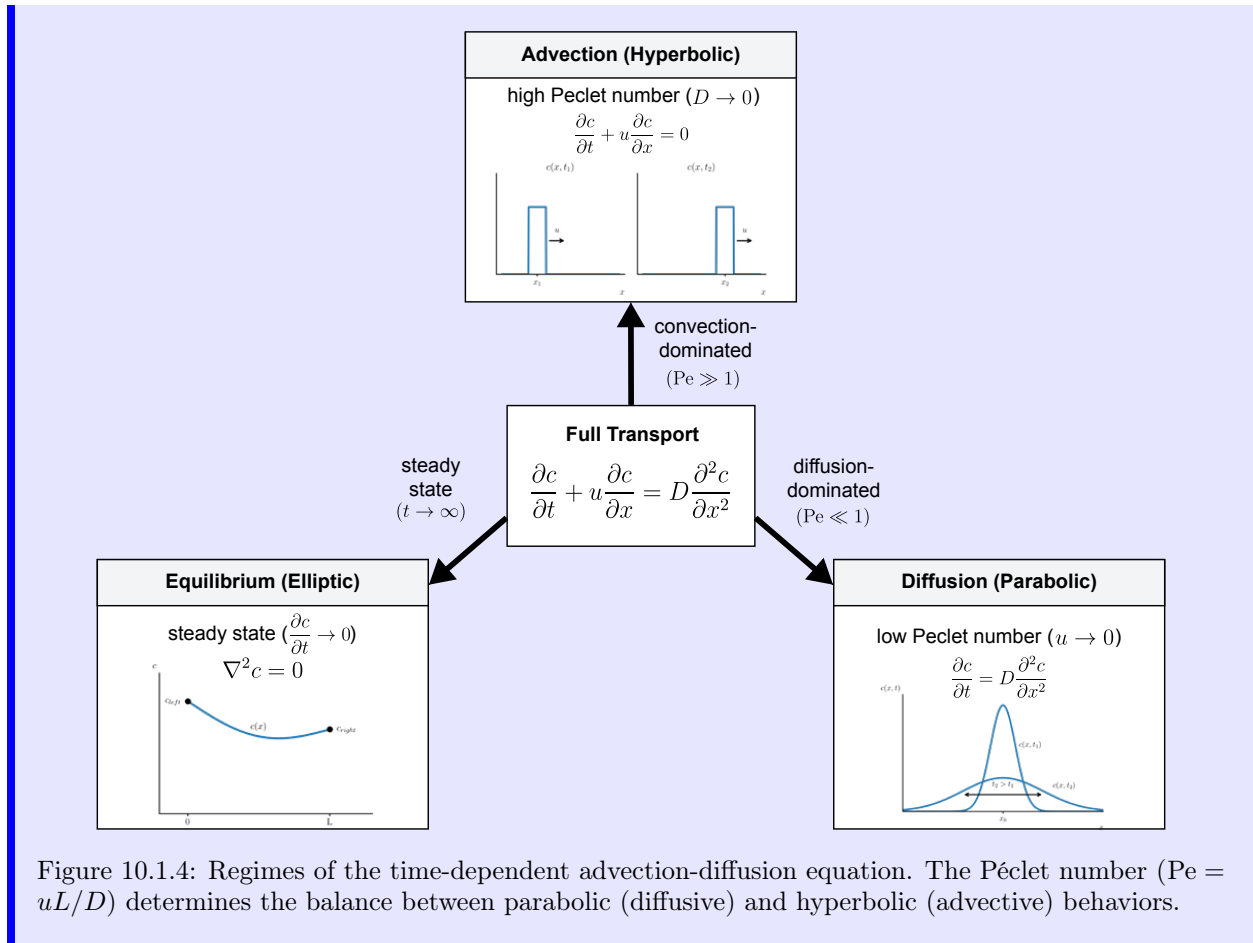


Figure 10.1.4: Regimes of the time-dependent advection-diffusion equation. The Péclet number ($Pe = uL/D$) determines the balance between parabolic (diffusive) and hyperbolic (advective) behaviors.

Table 10.1.1: Classification of second-order PDEs by discriminant, with representative model equations and typical physical interpretations.

Type	Elliptic	Parabolic	Hyperbolic
Discriminant	$B^2 - 4AC < 0$	$B^2 - 4AC = 0$	$B^2 - 4AC > 0$
Example	$\frac{\partial^2 c}{\partial x^2} + \frac{\partial^2 c}{\partial y^2} = 0$	$\frac{\partial c}{\partial t} = D \frac{\partial^2 c}{\partial x^2}$	$\frac{\partial^2 c}{\partial t^2} = u^2 \frac{\partial^2 c}{\partial x^2}$
Physics	Steady-state diffusion	Transient diffusion	Transient advection

10.2 Spatial Discretization (Elliptic Examples)

10.2.1 The Finite Difference Method

To illustrate the numerical solution of elliptic partial differential equations, we consider the two-dimensional Laplace equation with Dirichlet boundary conditions. This problem seeks the steady-state distribution of a quantity $c(x, y)$ (such as concentration or temperature) within a unit square domain. The mathematical formulation of this problem is

$$\frac{\partial^2 c}{\partial x^2} + \frac{\partial^2 c}{\partial y^2} = 0 \quad \forall (x, y) \in [0, 1] \times [0, 1] \quad (10.2.1)$$

subject to the boundary conditions

$$\begin{aligned} c(x, 0) = 1, \quad c(x, 1) = 0 & \quad \forall x \in [0, 1] \\ c(0, y) = 0, \quad c(1, y) = 0 & \quad \forall y \in [0, 1] \end{aligned} \quad (10.2.2)$$

Our goal is to transform this continuous PDE into a system of linear equations that can be solved numerically with the methods we have previously developed. To do this, we will use the finite difference method, now applied to PDEs instead of ODEs (the idea is the same—we just have to discretize in more than one dimension simultaneously).

10.2.1.1 Discretization and the 5-Point Stencil

We begin by constructing a mesh over the domain. We define discrete nodes (i, j) corresponding to spatial coordinates $(x_i, y_j) = (i\Delta, j\Delta)$, where $\Delta = 1/(N+1)$ is the grid spacing (here N denotes the number of *interior* points per dimension). For every interior node, we approximate the second partial derivatives using the second-order accurate central finite difference formula. The discrete Laplacian at node (i, j) becomes:

$$\frac{c_{i-1,j} - 2c_{i,j} + c_{i+1,j}}{\Delta^2} + \frac{c_{i,j-1} - 2c_{i,j} + c_{i,j+1}}{\Delta^2} = 0 \quad (10.2.3)$$

Multiplying through by Δ^2 allows us to write the equation in terms of the coefficients of the node and its immediate neighbors. This results in the standard 5-point stencil equation:

$$c_{i-1,j} + c_{i+1,j} + c_{i,j-1} + c_{i,j+1} - 4c_{i,j} = 0 \quad (10.2.4)$$

This algebraic equation must hold for every interior node in the mesh.

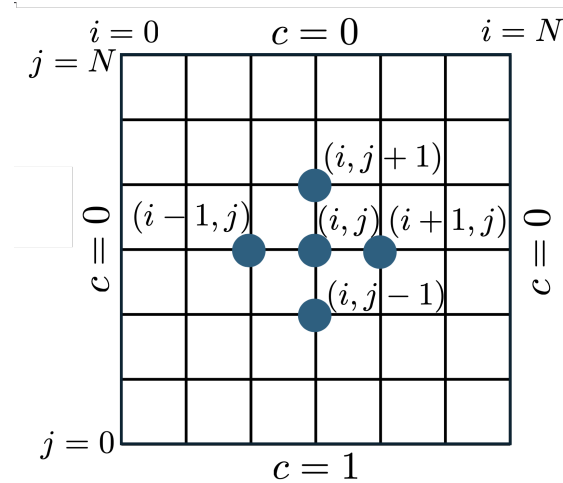


Figure 10.2.1: The 5-point stencil for the 2D Laplace equation. The value at the central node (i, j) is coupled to its four nearest neighbors.

The boundary nodes are treated separately. For Dirichlet boundary conditions, the values are fixed directly by the problem statement. For example, nodes at the bottom boundary satisfy $c_{i,0} = 1$, while the other boundaries are set to 0. There is a technical ambiguity at the corners (e.g., node $(0,0)$ belongs to two boundaries), but for the interior 5-point stencil they never enter the interior equations, so it doesn't matter.¹

¹For consistency in plots, we set each corner to match one adjacent edge (or an average).

10.2.1.2 Handling Neumann Boundary Conditions

If the problem instead involves a Neumann boundary condition, such as a specified flux at a wall, we must approximate the derivative at the boundary. Consider the condition:

$$\left. \frac{\partial c}{\partial x} \right|_{x=0} = 0 \quad \forall y \in [0, 1] \quad (10.2.5)$$

There are two primary methods to implement this. The first is to use a first-order forward difference approximation at the boundary node $(0, j)$:

$$\frac{c_{1,j} - c_{0,j}}{\Delta} = 0 \implies c_{0,j} = c_{1,j} \quad (10.2.6)$$

While simple, this is only first-order accurate, which may degrade the global accuracy of the solution. A more robust alternative is to introduce “ghost points” at $i = -1$ (the name comes from the fact that they are not part of the physical domain, see [Figure 10.2.2](#)). We apply the central difference approximation at the boundary $i = 0$:

$$\frac{c_{1,j} - c_{-1,j}}{2\Delta} = 0 \implies c_{-1,j} = c_{1,j} \quad (10.2.7)$$

We then write the standard PDE stencil for the boundary node $(0, j)$ involving this ghost point:

$$c_{-1,j} + c_{1,j} + c_{0,j-1} + c_{0,j+1} - 4c_{0,j} = 0 \quad (10.2.8)$$

Substituting the ghost point relation $c_{-1,j} = c_{1,j}$ into the stencil eliminates the unknown ghost variable, and we get a specialized equation for the boundary nodes:

$$2c_{1,j} + c_{0,j-1} + c_{0,j+1} - 4c_{0,j} = 0 \quad (10.2.9)$$

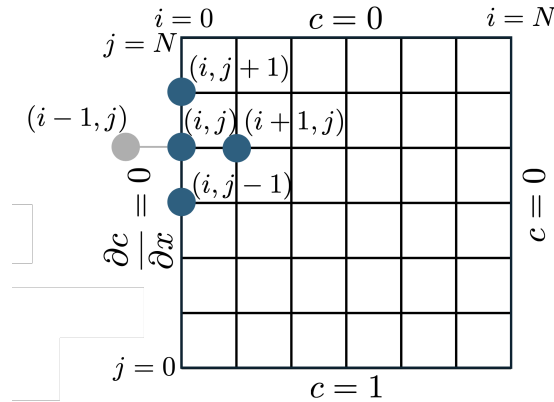


Figure 10.2.2: An illustration of using ghost points to enforce Neumann boundary conditions.

10.2.1.3 The System of Linear Equations

To solve the collection of discrete equations simultaneously, we must organize them into the matrix form $\mathbf{A}\mathbf{c} = \mathbf{b}$. This requires flattening the 2D grid indexing (i, j) into a single 1D index q . A common strategy is lexicographic ordering, where we number nodes row by row. The resulting vector $\mathbf{c}$ contains all the unknown nodal values:

$$\mathbf{c} = (\dots, c_{q-1}, c_q, c_{q+1}, \dots)^\top \quad (10.2.10)$$

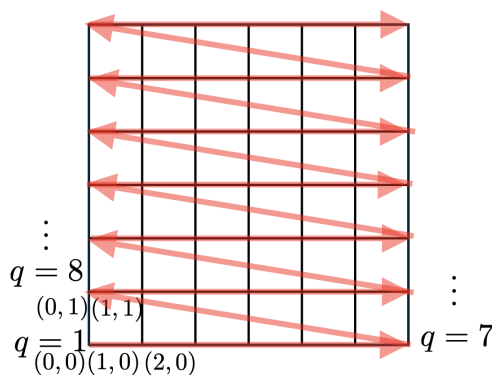


Figure 10.2.3: An illustration of lexicographic ordering of the nodes to flatten the matrix into a vector.

The matrix $\mathbf{A}$ contains the linear relationships defined by the finite difference stencils. For the standard second-order Laplacian, most rows of $\mathbf{A}$ will contain exactly five non-zero entries (corresponding to the node and its four neighbors). A typical row looks like:

$$[\cdots \quad 1 \quad \cdots \quad 1 \quad -4 \quad 1 \quad \cdots \quad 1 \quad \cdots] \quad (10.2.11)$$

Consequently, $\mathbf{A}$ is a highly sparse matrix: it has M^2 total entries (where $M = N^2$ is the number of unknowns) but only $O(M)$ nonzeros ($\approx 5M$ for the 5-point stencil). The right-hand side vector $\mathbf{b}$ contains terms arising from the boundary conditions and any source terms in the PDE. Note that even if the PDE is homogeneous (Laplace equation), non-zero boundary conditions will result in a non-zero $\mathbf{b}$.

$$\underbrace{\begin{pmatrix} \ddots & & & & & & & \\ & \ddots & & & & & & \\ & & \ddots & & & & & \\ \cdots & 1 & \cdots & 1 & -4 & 1 & \cdots & 1 & \cdots \\ & & & & \ddots & & & & \\ & & & & & \ddots & & & \\ & & & & & & \ddots & & \\ & & & & & & & \ddots & \end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix} \vdots \\ c_{q-N} \\ \vdots \\ c_{q-1} \\ c_q \\ c_{q+1} \\ \vdots \\ c_{q+N} \\ \vdots \end{pmatrix}}_{\mathbf{c}} = \underbrace{\begin{pmatrix} \vdots \\ 0 \\ \vdots \end{pmatrix}}_{\mathbf{b}} \quad (10.2.12)$$

If we were to employ a higher-order finite difference approximation to improve accuracy (e.g., fourth-order), the stencil would extend further, involving more neighbors. For instance, a fourth-order approximation involves coefficients like $-1/12$ and $4/3$ extending two nodes away in each direction. This increases accuracy but also increases the number of non-zeros per row (making $\mathbf{A}$ denser) and complicates the boundary treatment.

10.2.1.4 Solving the Sparse System

For a grid of size $N \times N$, the matrix $\mathbf{A}$ has dimension $N^2 \times N^2$. As N increases to reduce discretization error, the system size grows rapidly. As we know, direct solution approaches, such as Gaussian elimination (or LU/Cholesky factorization), generally have a complexity of $O(M^3)$ for a dense matrix of size M . Here $M = N^2$, making the cost prohibitive. Although sparse direct solvers scale better, iterative methods are often preferred for very large systems. Iterative methods (such as the Jacobi and Gauss-Seidel methods that we covered earlier in the course, and also a class of methods known as Krylov subspace methods) provide scalable solutions using sparse matrix-vector multiplication ($\mathbf{y} = \mathbf{A}\mathbf{c}$). This operation has complexity $O(M)$ when exploiting sparsity. These methods correct an initial guess over repeated iterations and are highly

parallelizable. Furthermore, we do not even need to form the matrix $\mathbf{A}$ explicitly; only a function that computes the action of $\mathbf{A}$ on a vector is required.

The Conjugate Gradient Method* The conjugate gradient (CG) method is a Krylov subspace method designed specifically for systems where the matrix is symmetric and positive definite (SPD). To be clear, for our standard 5-point stencil that we used above, the matrix $\mathbf{A}$ has negative diagonal entries (-4) and is formally negative definite. However, the system is equivalent to $(-\mathbf{A})\mathbf{c} = (-\mathbf{b})$, where $(-\mathbf{A})$ is SPD. Thus, we apply CG to the negated system (or simply define the Laplacian with positive signs).² CG finds the solution in at most M iterations (in exact arithmetic) by searching within the Krylov subspace:

$$\mathcal{K}_M(\mathbf{A}, \mathbf{r}_0) = \text{span}(\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \dots, \mathbf{A}^{M-1}\mathbf{r}_0\}) \quad (10.2.13)$$

The method generates a sequence of search directions $\mathbf{p}_k$ that are “ $\mathbf{A}$ -conjugate” (orthogonal with respect to the inner product defined by $\mathbf{A}$). The algorithm proceeds as follows: First, initialize the residual and search direction:

$$\begin{aligned} \mathbf{r}_0 &:= \mathbf{b} - \mathbf{A}\mathbf{x}_0 \\ \mathbf{p}_0 &:= \mathbf{r}_0 \end{aligned} \quad (10.2.14)$$

Then, repeat the following steps until the residual $\mathbf{r}_{k+1}$ is sufficiently small:

$$\begin{aligned} \alpha_k &:= \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k} \\ \mathbf{x}_{k+1} &:= \mathbf{x}_k + \alpha_k \mathbf{p}_k \\ \mathbf{r}_{k+1} &:= \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k \\ \beta_k &:= \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k} \\ \mathbf{p}_{k+1} &:= \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k \end{aligned} \quad (10.2.15)$$

The most expensive operation in each iteration is the matrix-vector product $\mathbf{A}\mathbf{p}_k$. CG can be implemented with fixed memory, as it does not require storing the full history of vectors. To improve convergence speed, especially for ill-conditioned systems, CG is often used with a preconditioner $\mathbf{P}^{-1}$, solving the equivalent system $\mathbf{P}^{-1}\mathbf{A}\mathbf{c} = \mathbf{P}^{-1}\mathbf{b}$.

10.2.2 The Finite Volume Method

Just as in the ODE case, we saw that FDM for PDEs approximates derivatives directly at grid points. However, we can also adapt FVM, which bakes in the physics of conservation laws directly in its construction, to solve PDEs. Instead of tracking values at nodal points, we focus on cells. We define the discrete unknown $c_{i,j}$ as the spatial average of the field variable (e.g., concentration) over a control volume (cell) spanning $[(i-1)\Delta, i\Delta] \times [(j-1)\Delta, j\Delta]$ (centered at $((i-1/2)\Delta, (j-1/2)\Delta)$):

$$c_{i,j} = \frac{1}{\Delta^2} \int_{(i-1)\Delta}^{i\Delta} \int_{(j-1)\Delta}^{j\Delta} c(x, y) dy dx \quad (10.2.16)$$

This shift in definition from “value at a point” to “average over a volume” is what makes FVM particularly suitable for problems where conservation of mass, momentum, or energy is very important. As you might imagine, then, as chemical engineers, we use FVM a lot. Here’s the concrete reason: because FVM balances what flows in and out of each cell exactly, it stays exactly conservative even when a material property like diffusivity or permeability jumps abruptly across an interface—think of a packed bed, a boundary between two phases, or the layers of a composite material—or when the geometry is irregular, like the actual shape of a reactor. A plain finite-difference approximation, which just approximates a derivative at a point, can end up violating the overall mass balance in these situations; FVM cannot, by construction.

²With pure Neumann BCs, the discrete Laplacian is singular; CG then needs modification or a constraint to remove the nullspace.

10.2.2.1 Deriving the Conservation Law

To obtain the discrete equations, we integrate the governing PDE over the control volume. For the Laplace equation $\nabla^2 c = 0$, the integral form is

$$\int_{(j-1)\Delta}^{j\Delta} \int_{(i-1)\Delta}^{i\Delta} \left(\frac{\partial^2 c}{\partial x^2} + \frac{\partial^2 c}{\partial y^2} \right) dy dx = 0 \quad (10.2.17)$$

Using the fundamental theorem of calculus (or the divergence theorem), we can convert the volume integral of the second derivatives into the sum of fluxes through the four faces of the cell. The integral separates into contributions from the x -fluxes (integrated along vertical faces) and y -fluxes (integrated along horizontal faces):

$$\int_{(j-1)\Delta}^{j\Delta} \left[\frac{\partial c}{\partial x} \Big|_{x=i\Delta} - \frac{\partial c}{\partial x} \Big|_{x=(i-1)\Delta} \right] dy + \int_{(i-1)\Delta}^{i\Delta} \left[\frac{\partial c}{\partial y} \Big|_{y=j\Delta} - \frac{\partial c}{\partial y} \Big|_{y=(j-1)\Delta} \right] dx = 0 \quad (10.2.18)$$

We approximate the face-normal derivatives using differences between adjacent cell centers. For example, on the right face of cell (i, j) (located at $x = i\Delta$), the normal derivative is

$$\frac{\partial c}{\partial x} \Big|_{x=i\Delta} \approx \frac{c_{i+1,j} - c_{i,j}}{\Delta} \quad (10.2.19)$$

Because each face integral has length Δ , the right-face contribution to the *integrated* balance is $\Delta(c_{i+1,j} - c_{i,j})/\Delta = c_{i+1,j} - c_{i,j}$, and similarly for the other faces. Dividing by the cell area Δ^2 (equivalently, working with fluxes per unit area) yields the discrete flux-balance equation

$$c_{i-1,j} + c_{i+1,j} + c_{i,j-1} + c_{i,j+1} - 4c_{i,j} = 0 \quad (10.2.20)$$

For interior cells, this result is algebraically identical to the 5-point stencil derived via finite differences, but the physical interpretation of balancing fluxes through cell faces is different (recall that we had this exact same sort of correspondence between FDM and FVM in the ODE case).

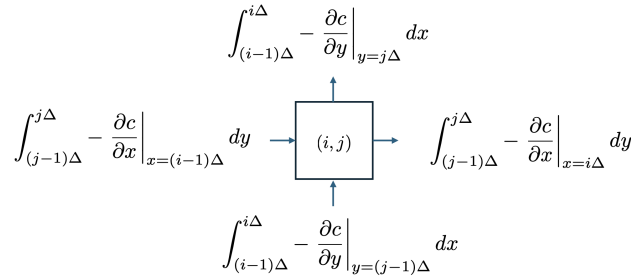


Figure 10.2.4: FVM balances fluxes through the faces of a control volume. For the Laplace equation, the net flux through the four faces must sum to zero.

10.2.2.2 Boundary Conditions in FVM

With FVM, we can explicitly model the flux at the boundary faces. Note that in this derivation we are effectively balancing derivatives; if defining physical flux $\mathbf{J} = -D\nabla c$, the signs in the balance equation would flip but the result is identical. Consider a Neumann boundary condition $\partial c/\partial x = 0$ on the left boundary. In the FVM framework, this implies that the flux through the left face of the boundary cell $(1, j)$ is exactly zero. The balance equation for this cell then involves only the fluxes through the top, bottom, and right faces:

$$(c_{2,j} - c_{1,j}) + (c_{1,j+1} - c_{1,j}) - (c_{1,j} - c_{1,j-1}) = 0 \quad (10.2.21)$$

Simplifying this expression, we get

$$c_{2,j} + c_{1,j-1} + c_{1,j+1} - 3c_{1,j} = 0 \quad (10.2.22)$$

For Dirichlet boundary conditions, such as $c = 1$ at the left wall, the treatment is slightly different. The value is specified at the face itself, not at a node. The distance from the center of the boundary cell $(1, j)$ to the left face is only $\Delta/2$, not Δ . Therefore, the gradient driving the flux through the left face is approximated as $(c_{1,j} - 1)/(\Delta/2)$. Including this steeper gradient in the flux balance gives

$$c_{2,j} + c_{1,j-1} + c_{1,j+1} - 5c_{1,j} + 2 = 0 \quad (10.2.23)$$

Note the appearance of the factor 5 and the constant term 2 that result from the half-cell spacing approximation.

Example: FVM for Steady Diffusion on a Square Consider steady diffusion in the unit square with no sources,

$$\nabla^2 c = 0 \quad \text{for } (x, y) \in (0, 1) \times (0, 1), \quad (10.2.24)$$

with Dirichlet conditions on the left and right walls and zero-flux (Neumann) on the top and bottom,

$$c(0, y) = 1, \quad c(1, y) = 0, \quad \frac{\partial c}{\partial y}(x, 0) = 0, \quad \frac{\partial c}{\partial y}(x, 1) = 0 \quad (10.2.25)$$

The exact solution is simply $c(x, y) = 1 - x$, so this is a clean test problem for the finite-volume construction above.

We discretize the domain into an $N \times N$ array of square control volumes of width $\Delta = 1/N$ (here N denotes the number of *cells* per direction), and take the discrete unknown $c_{i,j}$ to be the cell-average (or, equivalently, the value at the cell center to second order). For interior cells, integrating $\nabla^2 c = 0$ over the control volume and approximating face-normal gradients by adjacent cell-center differences gives us the standard flux-balance stencil

$$c_{i-1,j} + c_{i+1,j} + c_{i,j-1} + c_{i,j+1} - 4c_{i,j} = 0 \quad (10.2.26)$$

At boundaries, the only change is the face flux model. On the left wall $x = 0$, the Dirichlet value is imposed at the boundary face a half-cell away, so the left-face gradient uses $\Delta/2$. For the cells adjacent to $x = 0$ this produces

$$c_{2,j} + c_{1,j-1} + c_{1,j+1} - 5c_{1,j} = -2 \quad (10.2.27)$$

where the right-hand side comes from the prescribed face value $c = 1$. On the right wall $x = 1$, the same half-cell geometry applies but the face value is 0, giving

$$c_{N-1,j} + c_{N,j-1} + c_{N,j+1} - 5c_{N,j} = 0 \quad (10.2.28)$$

On the bottom and top walls, the Neumann condition $\partial c / \partial y = 0$ means the corresponding face flux is exactly zero, so boundary-adjacent cells satisfy

$$c_{i-1,1} + c_{i+1,1} + c_{i,2} - 3c_{i,1} = 0, \quad c_{i-1,N} + c_{i+1,N} + c_{i,N-1} - 3c_{i,N} = 0 \quad (10.2.29)$$

and at the corners you apply both modifications simultaneously. For example, at the bottom-left corner cell $(1, 1)$ (Dirichlet at $x = 0$ and Neumann at $y = 0$), the stencil becomes

$$c_{2,1} + c_{1,2} - 4c_{1,1} = -2 \quad (10.2.30)$$

i.e., the diagonal is reduced by one due to the missing Neumann neighbor in y and increased by one due to the half-cell Dirichlet face in x . Assembling these equations for all N^2 cells gives a sparse linear system $\mathbf{A}\mathbf{c} = \mathbf{b}$. Indeed, by stacking the unknowns row-by-row,

$$\mathbf{c} = (c_{1,1}, c_{2,1}, \dots, c_{N,1}, c_{1,2}, \dots, c_{N,2}, \dots, c_{1,N}, \dots, c_{N,N})^T \in \mathbb{R}^{N^2} \quad (10.2.31)$$

With this ordering, $\mathbf{A} \in \mathbb{R}^{N^2 \times N^2}$ is block tridiagonal, though the diagonal blocks depend on j :

$$\mathbf{A} = \begin{bmatrix} \mathbf{B}_1 & \mathbf{I} & & & 0 \\ \mathbf{I} & \mathbf{B}_{\text{int}} & \mathbf{I} & & \\ & \ddots & \ddots & \ddots & \\ & & \mathbf{I} & \mathbf{B}_{\text{int}} & \mathbf{I} \\ 0 & & & \mathbf{I} & \mathbf{B}_N \end{bmatrix}, \quad \mathbf{B}_{\text{int}} = \begin{bmatrix} -5 & 1 & & & 0 \\ 1 & -4 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & -4 & 1 \\ 0 & & & 1 & -5 \end{bmatrix} \quad (10.2.32)$$

Here $\mathbf{B}_{\text{int}}$ corresponds to the interior rows ($j = 2 \dots N - 1$) where the vertical stencil is standard. For the top and bottom rows ($j = 1, N$), the diagonal blocks $\mathbf{B}_1, \mathbf{B}_N$ are modified to reflect the Neumann boundary conditions (e.g., -3 on the diagonals instead of -4 , with corner adjustments). The $+1$ entries in $\mathbf{A}$'s off-diagonal blocks capture the coupling to $j \pm 1$. The right-hand side $\mathbf{b}$ is zero everywhere except in the equations adjacent to the left boundary $x = 0$, where the Dirichlet value $c = 1$ contributes -2 (one per leftmost cell in each row), consistent with

$$c_{2,j} + c_{1,j-1} + c_{1,j+1} - 5c_{1,j} = -2 \quad (10.2.33)$$

(For the top and bottom Neumann walls, the corresponding rows use -3 on the diagonal and omit the missing neighbor, e.g. $c_{i-1,1} + c_{i+1,1} + c_{i,2} - 3c_{i,1} = 0$ for bottom-edge cells away from corners; at the two corners on that edge, the diagonal is -4 because the Dirichlet half-cell modification in x is also present.) After solving this system with a linear solver, we get the solution shown in Figure 10.2.5. In this case, the solution is exact to almost machine precision (RMS error vs. the analytical solution is $1.58 \cdot 10^{-16}$).

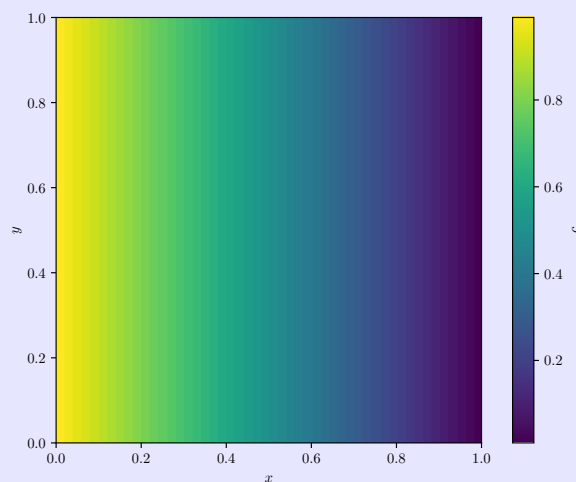


Figure 10.2.5: Numerical solution to the square domain diffusion problem Eq. 10.2.25.

10.3 Method of Lines for Time-Dependent PDEs (Parabolic/Hyperbolic)

We now turn our attention to transient PDEs, such as parabolic (diffusion) or hyperbolic (advection) equations, which describe systems evolving in time. A canonical example is the 1-D diffusion-advection-reaction system:

$$\frac{\partial c}{\partial t} = D \frac{\partial^2 c}{\partial x^2} - u \frac{\partial c}{\partial x} - kc \quad (10.3.1)$$

subject to initial conditions $c(x, 0) = \bar{c}(x)$ and boundary conditions, for instance $c(0, t) = 1$ and $c(1, t) = 0$. Solving this requires discretizing both the spatial domain x and the time domain t . Simultaneous discretization can be complex due to numerical stability constraints, specifically the relationship required between the spatial step Δx and time step Δt , which we will explicitly address in the CFL section.

10.3.1 The Method of Lines

A good (and very simple!) strategy to handle these difficulties is the method of lines (MOL). The idea behind MOL is to decouple the spatial and temporal discretizations. We explicitly discretize the spatial derivatives

using FDM (or FVM) while leaving the time derivative continuous. This transforms the single PDE into a large system of coupled ODEs.

To be concrete, let's consider the pure advection equation $\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = 0$. By applying a centered finite difference approximation to the spatial derivative on a grid $x_j = j\Delta x$, we obtain an ODE for the concentration at each node j :³

$$\frac{dc_j(t)}{dt} + u \frac{c_{j+1}(t) - c_{j-1}(t)}{2\Delta x} = 0 \quad (10.3.2)$$

We can now use sophisticated ODE-IVP solvers to integrate this system in time. While adaptive solvers (like `ode45`) handle error control, they do not eliminate stability restrictions. For diffusive problems, the system becomes stiff, often requiring implicit solvers (like `ode15s`) to avoid prohibitively small time steps.

10.3.2 Implementation and Boundary Handling

For the full diffusion-advection-reaction equation, the semi-discrete ODE for an interior node j becomes

$$\frac{dc_j(t)}{dt} = D \frac{c_{j-1}(t) - 2c_j(t) + c_{j+1}(t)}{\Delta x^2} - u \frac{c_{j+1}(t) - c_{j-1}(t)}{2\Delta x} - kc_j(t) \quad (10.3.3)$$

Handling boundary conditions with the MOL requires care. We have two main choices. The first is to treat the boundary conditions as algebraic equations coupled with the differential equations; this results in a system of differential-algebraic equations (DAEs), which we will discuss later on in the course. The second, and usually simpler approach, is to eliminate the boundary variables directly from the ODE system. For example, if we have a Dirichlet boundary condition $c_0(t) = 1$ at the left boundary, we do not write an ODE for node 0. Instead, we substitute the value 1 wherever $c_0(t)$ appears in the equation for the first interior node ($j = 1$). The ODE for node 1 becomes

$$\frac{dc_1(t)}{dt} = D \frac{1 - 2c_1(t) + c_2(t)}{\Delta x^2} - u \frac{c_2(t) - 1}{2\Delta x} - kc_1(t) \quad (10.3.4)$$

For the right boundary $c_N(t) = 0$, we similarly substitute 0 into the equation for node $N - 1$. If we index grid nodes as $j = 0, 1, \dots, N$ (so there are $N + 1$ nodes and $\Delta x = 1/N$), then imposing Dirichlet values at $j = 0$ and $j = N$ eliminates those boundary unknowns and leaves a system of $N - 1$ ODEs for $j = 1, \dots, N - 1$. When high accuracy is required near boundaries, we might also employ higher-order one-sided difference formulas for the spatial derivatives to maintain consistency with the interior scheme.

10.4 Finite Differences for Linear Advection: Upwinding

We now discuss the full FDM approach where we discretize over both space and time simultaneously rather than separating them as in MOL. Let us define our discrete approximation c_i^j as the value of the concentration at time step i and spatial node j :

$$c_i^j = c(i\Delta t, j\Delta x) \quad (10.4.1)$$

Consider the 1D advection equation:

$$\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = 0 \quad (10.4.2)$$

A naive first attempt might be to use a forward FD scheme for the time derivative and a central FD scheme for the spatial derivative (**f**orward **t**ime, **c**entered **s**patial $\Rightarrow$ called “FTCS”). Substituting these approximations into the PDE, we obtain

$$\frac{c_{i+1}^j - c_i^j}{\Delta t} + u \frac{c_i^{j+1} - c_i^{j-1}}{2\Delta x} = 0 \quad (10.4.3)$$

If we implement this scheme and march forward in time, the solution has severe numerical issues. As shown in the simulation results ([Figure 10.4.1](#)), we do not see a wave propagating forward cleanly; instead, the solution

³With centered spatial advection, the semi-discrete operator has (typically) imaginary eigenvalues (nondissipative), so explicit time integrators still require a stability-limited step size, and solutions near sharp features can have dispersive oscillations unless upwinding/limiting is used.

explodes with non-physical oscillations. In the FTCS combination, the centered spatial derivative together with forward Euler time stepping produces an amplification factor with $|G| > 1$ (more on this later), hence instability. Intuitively, the stencil does not enforce upwind information flow, so without added dissipation the scheme cannot remain stable.

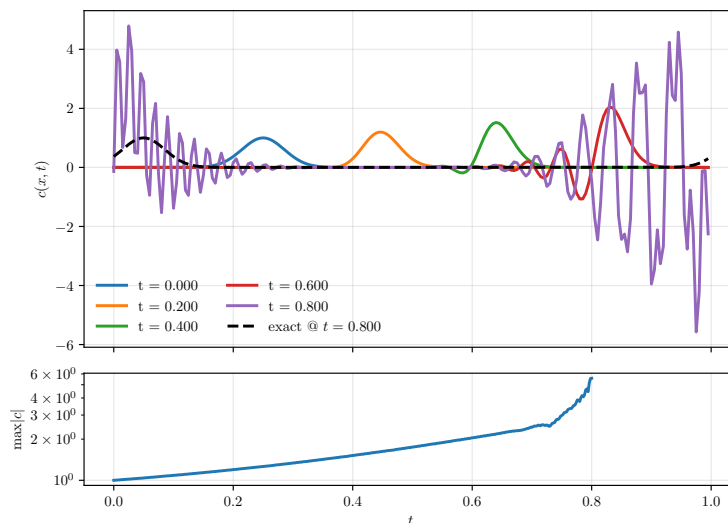


Figure 10.4.1: The naive forward-time central-space approximation for advection leads to numerical instability.

To remedy this, we must choose a discretization that respects the physics of the flow. We replace the spatial derivative with a backward FD (also known as an “upwind difference”), which uses information only from the current node and the upstream node. The scheme becomes:

$$\frac{c_{i+1}^j - c_i^j}{\Delta t} + u \frac{c_i^j - c_i^{j-1}}{\Delta x} = 0 \quad (10.4.4)$$

With this upwind scheme, stability is restored, and the wave propagates in the correct direction. However, while stable, the accuracy is low. The scheme is numerically diffusive (dissipative). This means that it introduces artificial diffusion that smears sharp gradients and reduces peak amplitude over time (Figure 10.4.2).

This qualitative statement can be made more precise: substituting Taylor expansions for c_{i+1}^j, c_i^j into the upwind scheme and using the leading-order PDE to eliminate the time derivatives shows that the scheme actually solves, to leading order, an advection-diffusion equation $\partial_t c + u \partial_x c = D_{\text{num}} \partial_{xx} c$ with an added numerical diffusivity

$$D_{\text{num}} = \frac{u \Delta x}{2} (1 - \alpha), \quad \alpha := \frac{u \Delta t}{\Delta x} \quad (10.4.5)$$

(the Courant number α defined formally in the CFL discussion below). The “smearing” is a known function of the grid spacing, time step, and wave speed, and it vanishes only in the limit $\alpha \rightarrow 1$.

10.5 Stability of Time-Stepping Schemes: CFL and von Neumann Analysis

10.5.1 The CFL Condition

The stability of explicit time-stepping schemes is governed by the Courant-Friedrichs-Lewy (CFL) condition. This condition is a statement about causality: the physical domain of dependence must be contained within the numerical domain of dependence.

If we trace the characteristics of the advection equation back in time by Δt , the physical information comes from a location $u \Delta t$ upwind. Numerically, our grid only connects us to the neighbor at Δx upwind. For the

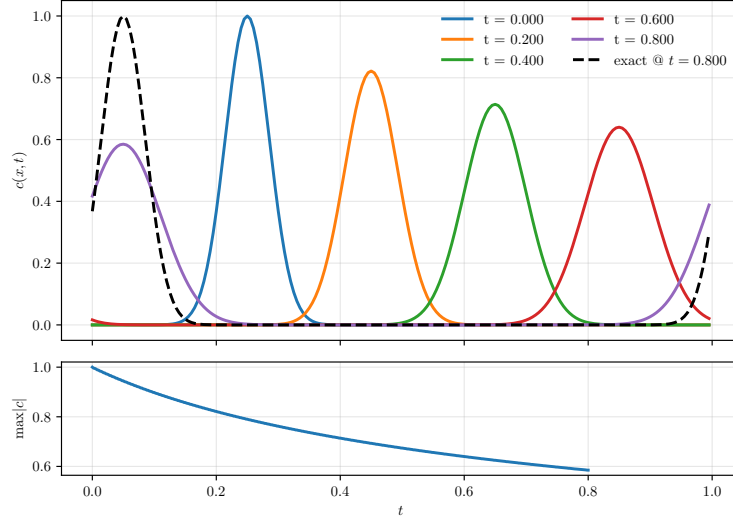


Figure 10.4.2: Forward-time upwind-space discretization for 1D advection. The upwind stencil restores stability and propagates the wave in the correct direction, but introduces numerical diffusion. The profile broadens and the peak amplitude decays over time compared to the exact (purely shifted) solution.

numerical scheme to capture the physics, the physical wave speed must not exceed the “grid speed” (make sure you take a minute to fully understand this before proceeding!). This leads to the requirement that

$$\frac{u\Delta t}{\Delta x} \leq 1 \quad (10.5.1)$$

For the correct upwind discretization, the condition is $0 \leq \alpha \leq 1$ where $\alpha = u\Delta t/\Delta x$ after choosing the upwind stencil consistent with $\text{sign}(u)$. If we keep a fixed stencil regardless of flow direction, stability is lost. The quantity $\frac{u\Delta t}{\Delta x}$ is often called the CFL number. This result implies that the temporal discretization cannot be chosen independently of the spatial discretization; if we refine the grid ($\Delta x \rightarrow 0$), we must proportionally reduce the time step.

10.5.2 Von Neumann Stability Analysis

A more rigorous mathematical tool for determining the stability of linear finite difference schemes is von Neumann stability analysis.⁴ We assume that the solution (or equivalently, the error) can be decomposed into a Fourier series. We then examine how a single Fourier mode evolves over time. Let the solution be written as

$$c(t, x) = \sum_k \epsilon_k(t) e^{\hat{i} k x} \quad (10.5.2)$$

where $\hat{i} = \sqrt{-1}$ and k is the wave number. Because the difference equations are linear, we can analyze the behavior of a single eigenmode $\epsilon_i e^{\hat{i} k j \Delta x}$ (where ϵ_i is the time-dependent amplitude at step i). We substitute this mode into our discretized equation to see if the amplitude ϵ_i grows or decays.

10.5.2.1 Hyperbolic Case: Advection

Let us apply this analysis to the upwind scheme derived earlier:

$$\frac{c_{i+1}^j - c_i^j}{\Delta t} + u \frac{c_i^j - c_i^{j-1}}{\Delta x} = 0 \quad (10.5.3)$$

⁴Von Neumann analysis is exact for periodic/infinite domains; with boundaries, it still gives the correct interior stability constraint for these linear constant-coefficient schemes.

Substituting $c_i^j = \epsilon_i e^{ikj\Delta x}$, we obtain

$$\frac{\epsilon_{i+1} - \epsilon_i}{\Delta t} e^{ikj\Delta x} + \frac{u}{\Delta x} \epsilon_i (e^{ikj\Delta x} - e^{ik(j-1)\Delta x}) = 0 \quad (10.5.4)$$

Dividing through by $e^{ikj\Delta x}$ simplifies the expression to

$$\frac{\epsilon_{i+1} - \epsilon_i}{\Delta t} + \frac{u}{\Delta x} \epsilon_i (1 - e^{-ik\Delta x}) = 0 \quad (10.5.5)$$

Rearranging to solve for the amplification of the error from step i to $i + 1$,

$$\epsilon_{i+1} = \epsilon_i - \frac{u\Delta t}{\Delta x} (1 - e^{-ik\Delta x}) \epsilon_i \quad (10.5.6)$$

$$\epsilon_{i+1} = \left[1 - \frac{u\Delta t}{\Delta x} (1 - e^{-ik\Delta x}) \right] \epsilon_i \quad (10.5.7)$$

For the method to be numerically stable, the magnitude of the amplification factor must be less than or equal to 1 for all wave numbers k :

$$\left| 1 - \frac{u\Delta t}{\Delta x} (1 - e^{-ik\Delta x}) \right| \leq 1 \quad (10.5.8)$$

Defining $\alpha := \frac{u\Delta t}{\Delta x}$, we can visualize this condition in the complex plane. The term $1 - \alpha(1 - e^{-ik\Delta x})$ traces a circle centered at $1 - \alpha$ with radius α . For this circle to remain within the unit circle (stability region), we must satisfy $\alpha \leq 1$. This confirms that the von Neumann analysis recovers the same stability criterion as the CFL condition.

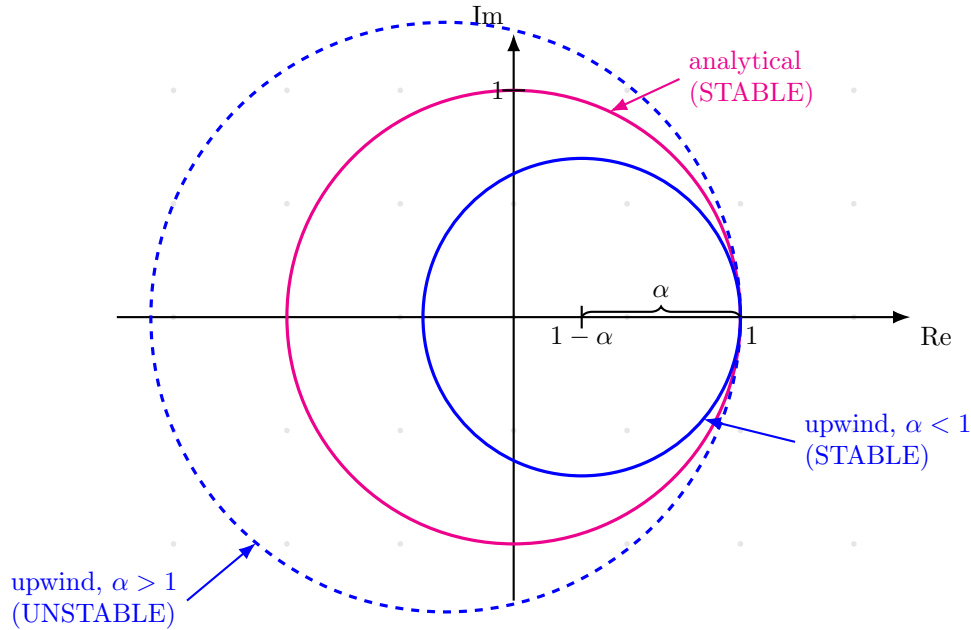


Figure 10.5.1: Complex-plane visualization for von Neumann stability of the upwind advection scheme. The analytical stability requirement is $|G| \leq 1$ (magenta unit circle). For $G(\theta) = 1 - \alpha(1 - e^{-i\theta})$ with $\theta = k\Delta x$, the upwind amplification factor traces a circle centered at $1 - \alpha$ with radius α (blue). This locus lies entirely within the unit circle when $0 \leq \alpha \leq 1$ (stable) and extends outside it when $\alpha > 1$ (unstable).

Non-Upwind Discretization In contrast, if we apply this analysis to the non-upwind discretization (forward in time, central in space, a.k.a. FTCS), which we suspected was unstable, we have

$$\frac{c_{i+1}^j - c_i^j}{\Delta t} + u \frac{c_i^{j+1} - c_i^{j-1}}{2\Delta x} = 0 \quad (10.5.9)$$

Proceeding with the von Neumann analysis as usual, we substitute a single Fourier mode $c_i^j = \epsilon_i e^{ikj\Delta x}$ so that

$$c_{i+1}^j = \epsilon_{i+1} e^{ikj\Delta x}, \quad c_i^{j\pm 1} = \epsilon_i e^{ik(j\pm 1)\Delta x}. \quad (10.5.10)$$

Substituting into Eq. 10.5.9 gives

$$\frac{\epsilon_{i+1} e^{ikj\Delta x} - \epsilon_i e^{ikj\Delta x}}{\Delta t} + u \frac{\epsilon_i e^{ik(j+1)\Delta x} - \epsilon_i e^{ik(j-1)\Delta x}}{2\Delta x} = 0 \quad (10.5.11)$$

Factoring out the common term $e^{ikj\Delta x}$,

$$\frac{\epsilon_{i+1} - \epsilon_i}{\Delta t} + u \epsilon_i \frac{e^{ik\Delta x} - e^{-ik\Delta x}}{2\Delta x} = 0 \quad (10.5.12)$$

With the Courant number $\alpha := \frac{u\Delta t}{\Delta x}$, we can rewrite this as

$$\epsilon_{i+1} - \epsilon_i = -\alpha \epsilon_i \frac{e^{ik\Delta x} - e^{-ik\Delta x}}{2} \quad (10.5.13)$$

Using Euler's identity,

$$\frac{e^{i\theta} - e^{-i\theta}}{2} = i \sin \theta, \quad \theta := k\Delta x \quad (10.5.14)$$

Eq. 10.5.13 becomes

$$\epsilon_{i+1} = [1 - \alpha i \sin(k\Delta x)] \epsilon_i \quad (10.5.15)$$

Hence, the amplification factor is

$$G(\theta) = 1 - \alpha i \sin \theta \quad (10.5.16)$$

Its magnitude is

$$|G(\theta)|^2 = (1)^2 + (\alpha \sin \theta)^2 = 1 + \alpha^2 \sin^2 \theta \quad (10.5.17)$$

so

$$|G(\theta)| = \sqrt{1 + \alpha^2 \sin^2 \theta} \quad (10.5.18)$$

For any nonzero α , there exist modes with $\sin \theta \neq 0$ (e.g. $\theta = \pi/2$), and then

$$|G(\theta)| > 1 \quad (10.5.19)$$

Therefore the FTCS discretization for linear advection is *unconditionally unstable*: no choice of $\Delta t > 0$ can make Eq. 10.5.9 von Neumann stable for all Fourier modes.

10.5.2.2 Parabolic Case: Diffusion

Now, we consider the parabolic diffusion equation $\frac{\partial c}{\partial t} = D \frac{\partial^2 c}{\partial x^2}$. We apply a forward difference in time and a central difference in space:

$$\frac{c_{i+1}^j - c_i^j}{\Delta t} = D \frac{c_i^{j-1} - 2c_i^j + c_i^{j+1}}{\Delta x^2} \quad (10.5.20)$$

Applying the von Neumann ansatz $c_i^j = \epsilon_i e^{ikj\Delta x}$,

$$(\epsilon_{i+1} - \epsilon_i) e^{ikj\Delta x} = \frac{\Delta t D}{\Delta x^2} \epsilon_i (e^{ik(j-1)\Delta x} - 2e^{ikj\Delta x} + e^{ik(j+1)\Delta x}) \quad (10.5.21)$$

Dividing out the common exponential term and using Euler's formula ($e^{-ikx} + e^{ikx} = 2 \cos(kx)$),

$$\epsilon_{i+1} - \epsilon_i = \frac{\Delta t D}{\Delta x^2} \epsilon_i (2 \cos(k\Delta x) - 2) \quad (10.5.22)$$

Letting $\beta = \frac{\Delta t D}{\Delta x^2}$ and using the identity $\cos(\theta) - 1 = -2 \sin^2(\theta/2)$, we can write the amplification relation as:

$$\epsilon_{i+1} = [1 - 2\beta + 2\beta \cos(k\Delta x)] \epsilon_i \quad (10.5.23)$$

For numerical stability, we require the bracketed term to lie between -1 and 1. The most restrictive condition comes from the lower bound, leading to

$$\beta \leq \frac{1}{2} \implies \frac{D\Delta t}{\Delta x^2} \leq \frac{1}{2} \quad (10.5.24)$$

This condition is a significant limitation of explicit methods for parabolic PDEs. If we wish to increase the spatial resolution by a factor of 10, the term Δx^2 in the denominator decreases by a factor of 100. To maintain stability ($\beta \leq 0.5$), we must decrease the time step Δt by a factor of 100. This makes forward-in-time, centered-in-space schemes numerically inefficient for high-resolution diffusion problems.

10.5.2.3 Implicit Methods for Parabolic Equations

To overcome the strict time-step restrictions of explicit methods, we can use implicit methods. Let's consider the backward-in-time, centered-in-space (backward Euler) scheme for diffusion:

$$\frac{c_i^j - c_{i-1}^j}{\Delta t} = D \frac{c_i^{j-1} - 2c_i^j + c_i^{j+1}}{\Delta x^2} \quad (10.5.25)$$

To perform von Neumann analysis, we assume a single Fourier mode

$$c_i^j = \epsilon_i e^{ikj\Delta x} \quad (10.5.26)$$

which implies the shifted values

$$c_{i-1}^j = \epsilon_{i-1} e^{ikj\Delta x} \quad (10.5.27)$$

$$c_i^{j\pm 1} = \epsilon_i e^{ik(j\pm 1)\Delta x} \quad (10.5.28)$$

Substituting into the scheme yields

$$\frac{\epsilon_i e^{ikj\Delta x} - \epsilon_{i-1} e^{ikj\Delta x}}{\Delta t} = D \frac{\epsilon_i e^{ik(j-1)\Delta x} - 2\epsilon_i e^{ikj\Delta x} + \epsilon_i e^{ik(j+1)\Delta x}}{\Delta x^2} \quad (10.5.29)$$

Canceling the common factor $e^{ikj\Delta x}$, we obtain

$$\frac{\epsilon_i - \epsilon_{i-1}}{\Delta t} = D \frac{\epsilon_i (e^{-ik\Delta x} - 2 + e^{ik\Delta x})}{\Delta x^2} \quad (10.5.30)$$

Now, let $\theta := k\Delta x$ and use $e^{i\theta} + e^{-i\theta} = 2\cos\theta$ to get

$$\frac{\epsilon_i - \epsilon_{i-1}}{\Delta t} = D \frac{\epsilon_i (2\cos\theta - 2)}{\Delta x^2} \quad (10.5.31)$$

Equivalently,

$$\frac{\epsilon_i - \epsilon_{i-1}}{\Delta t} = -2D \frac{\epsilon_i (1 - \cos\theta)}{\Delta x^2} \quad (10.5.32)$$

Defining the diffusion number $r := \frac{D\Delta t}{\Delta x^2}$, we multiply by Δt and rearrange to isolate ϵ_i in terms of ϵ_{i-1}

$$\epsilon_i - \epsilon_{i-1} = -2r (1 - \cos\theta) \epsilon_i \quad (10.5.33)$$

$$[1 + 2r (1 - \cos\theta)] \epsilon_i = \epsilon_{i-1} \quad (10.5.34)$$

Therefore, the amplification factor $G(\theta) := \epsilon_i / \epsilon_{i-1}$ is

$$G(\theta) = \frac{1}{1 + 2r (1 - \cos\theta)} \quad (10.5.35)$$

Using the identity $1 - \cos\theta = 2\sin^2(\theta/2)$ gives an equivalent form

$$G(\theta) = \frac{1}{1 + 4r \sin^2(\theta/2)} \quad (10.5.36)$$

Since $D \geq 0$ implies $r \geq 0$ and $1 - \cos \theta \geq 0$ for all θ , we have

$$1 + 2r(1 - \cos \theta) \geq 1 \quad (10.5.37)$$

Hence,

$$0 < G(\theta) \leq 1 \quad (10.5.38)$$

and in particular

$$|G(\theta)| \leq 1 \quad (10.5.39)$$

for all wave numbers k and any choice of Δt . The $k = 0$ mode satisfies $G = 1$ (the constant solution is preserved), while all nonzero modes satisfy $G < 1$ and are damped, so the scheme is unconditionally stable.

10.6 Method of Characteristics for First-Order Hyperbolic PDEs

We consider the 1-D advection equation again to introduce a very useful analytical and conceptual tool known as the method of characteristics. The problem is defined by

$$\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = 0 \quad t \in [0, \infty), x \in (-\infty, \infty) \quad (10.6.1)$$

subject to the given initial condition $c(x, 0)$ for $x \in (-\infty, \infty)$.

The main idea behind this method is that instead of attempting to solve the PDE over the entire space-time domain simultaneously, we can identify specific curves in the $x - t$ plane, called characteristic curves, along which the PDE simplifies into an ODE. Along these curves, the independent variables t and x are not independent but are instead parameterized by a single variable s such that we track the solution $c(x, t) = c(s)$.

We can find these parameterizations by differentiating the solution with respect to s . Applying the chain rule gives

$$\frac{dc}{ds} = \frac{dt}{ds} \frac{\partial c}{\partial t} + \frac{dx}{ds} \frac{\partial c}{\partial x} \quad (10.6.2)$$

Comparing this total derivative expression to our original PDE, $\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = 0$, we can establish a correspondence between the terms. By matching the coefficients, we obtain a system of ODEs:

$$\frac{dt}{ds} = 1, \quad \frac{dx}{ds} = u, \quad \frac{dc}{ds} = 0 \quad (10.6.3)$$

Solving the first two equations tells us the geometry of the characteristics. Since $\frac{dt}{ds} = 1$, we can effectively identify s with t . Then, $\frac{dx}{dt} = u$ implies that the curves are lines given by $x = ut + x_0$. The third equation, $\frac{dc}{ds} = 0$, tells us that for all (x, t) on such a curve, the solution value $c(x, t)$ remains constant. Therefore, any solution $c(x, t)$ can be obtained simply by tracing back along the characteristic line to the point where $t = 0$. This produces the general solution for the infinite domain:

$$c(x, t) = c(x - ut, 0) \quad (10.6.4)$$

10.6.1 Initial and Boundary Value Problems

The situation becomes slightly more involved when we consider a semi-infinite domain, which introduces a boundary condition. Let's consider the following initial and boundary value problem:

$$\begin{cases} \frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = 0 & t \in [0, \infty), x \in [0, \infty) \\ c(x, 0) \text{ is given} & x \in [0, \infty) \\ c(0, t) \text{ is given} & t \in [0, \infty) \end{cases} \quad (10.6.5)$$

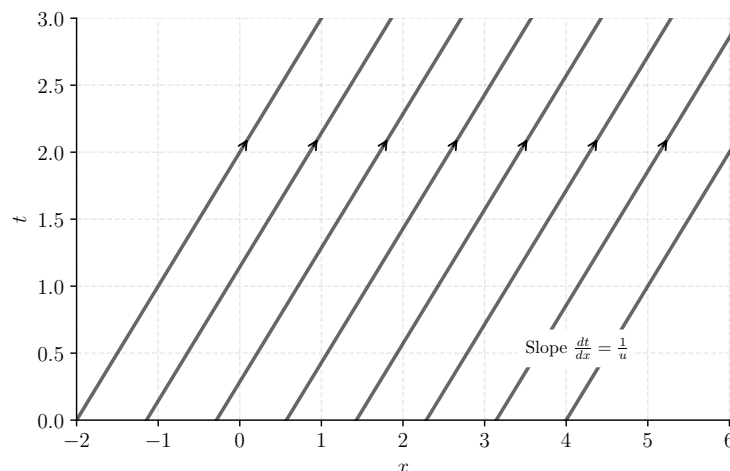


Figure 10.6.1: Characteristic curves in the $x-t$ plane for the 1-D advection equation. On an infinite domain, the solution $c(x, t)$ is constant along the lines $x - ut = \text{const}$. The value at any point (x, t) is determined by tracing the characteristic back to the initial condition at $t = 0$.

The solution is again obtained by tracing back over the characteristic lines, but now the origin of the information depends on where we are in the $x-t$ plane.

For points where $x \geq ut$, the characteristic line originates from the x -axis ($t = 0$). Here, the solution depends on the initial condition $c(x - ut, 0)$. However, for points where $x < ut$, the characteristic line originates from the t -axis ($x = 0$). In this region, the solution is determined by the boundary condition at the earlier time $t - x/u$. We can express this piecewise solution as

$$c(x, t) = \begin{cases} c(x - ut, 0), & x \geq ut \\ c(0, t - \frac{x}{u}), & x < ut \end{cases} \quad (10.6.6)$$

This explicitly demonstrates how information propagates from the boundaries into the domain.

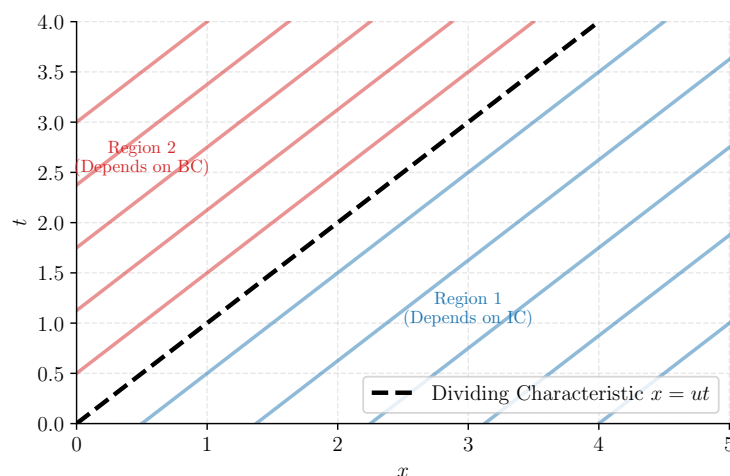


Figure 10.6.2: The domain of dependence for the semi-infinite problem. The characteristic line $x = ut$ divides the $x-t$ plane into two regions. In Region 1 ($x \geq ut$), the solution depends on the initial condition $c(x, 0)$. In Region 2 ($x < ut$), the solution depends on the boundary condition $c(0, t)$.

10.6.2 Second-Order Hyperbolic Equations

The method of characteristics is not limited to first-order equations; it can also be used for solving second-order hyperbolic equations, such as the wave equation:

$$\frac{\partial^2 c}{\partial t^2} = u^2 \frac{\partial^2 c}{\partial x^2} \quad (10.6.7)$$

To apply the method, we first transform the second-order PDE into a system of first-order PDEs. We introduce an auxiliary variable d (here d is an auxiliary field introduced so the second-order equation can be written as a symmetric first-order hyperbolic system) and write the system as

$$\frac{\partial c}{\partial t} + u \frac{\partial d}{\partial x} = 0 \quad \text{and} \quad \frac{\partial d}{\partial t} + u \frac{\partial c}{\partial x} = 0 \quad (10.6.8)$$

Equivalently, we can write these coupled equations in matrix-vector form:

$$\frac{\partial}{\partial t} \begin{bmatrix} c \\ d \end{bmatrix} + \begin{bmatrix} 0 & u \\ u & 0 \end{bmatrix} \frac{\partial}{\partial x} \begin{bmatrix} c \\ d \end{bmatrix} = 0 \quad (10.6.9)$$

To simplify the notation, let us define the vector of unknowns $\mathbf{c} := \begin{bmatrix} c \\ d \end{bmatrix}$ and the coefficient matrix $\mathbf{A} := \begin{bmatrix} 0 & u \\ u & 0 \end{bmatrix}$.

The system then becomes

$$\frac{\partial \mathbf{c}}{\partial t} + \mathbf{A} \frac{\partial \mathbf{c}}{\partial x} = \mathbf{0} \quad (10.6.10)$$

We can decouple this system by performing an eigendecomposition of the matrix $\mathbf{A}$. We write $\mathbf{A} = \mathbf{W} \mathbf{\Lambda} \mathbf{W}^{-1}$, where $\mathbf{W}$ is the matrix of eigenvectors and $\mathbf{\Lambda}$ is the diagonal matrix of eigenvalues. Substituting this into our equation,

$$\frac{\partial \mathbf{c}}{\partial t} + \mathbf{W} \mathbf{\Lambda} \mathbf{W}^{-1} \frac{\partial \mathbf{c}}{\partial x} = \mathbf{0} \quad (10.6.11)$$

Multiplying the entire equation from the left by $\mathbf{W}^{-1}$ allows us to move the eigenvector matrix inside the derivatives (assuming constant coefficients):

$$\frac{\partial \mathbf{W}^{-1} \mathbf{c}}{\partial t} + \mathbf{\Lambda} \frac{\partial \mathbf{W}^{-1} \mathbf{c}}{\partial x} = \mathbf{0} \quad (10.6.12)$$

If we define a new set of transformed variables $\tilde{\mathbf{c}} = \mathbf{W}^{-1} \mathbf{c}$, the system simplifies to

$$\frac{\partial \tilde{\mathbf{c}}}{\partial t} + \mathbf{\Lambda} \frac{\partial \tilde{\mathbf{c}}}{\partial x} = \mathbf{0} \quad (10.6.13)$$

Because $\mathbf{\Lambda}$ is diagonal, this vector equation is a set of uncoupled scalar equations. Each entry of the transformed system is simply

$$\frac{\partial \tilde{c}_k}{\partial t} + \lambda_k \frac{\partial \tilde{c}_k}{\partial x} = 0 \quad (10.6.14)$$

which can be solved independently as a 1-D advection equation using the method of characteristics derived previously.

10.7 Summary of PDE Methods

It is easy for us as chemical engineers to picture how PDEs govern the flow of information through systems since we have the nice physical intuition of fluid/heat/mass transport that obeys these same classes of PDEs. We have to think about two questions when analyzing PDEs and the solution methods we will apply to them. First, how do we represent space? Second, how do we evolve or solve in time?

Elliptic problems are steady and globally coupled, so we discretize space with finite differences or finite volumes and solve the resulting sparse linear system with iterative solvers (e.g., preconditioned CG for SPD

systems; GMRES for nonsymmetric cases) and sometimes multigrid methods for optimal scaling. Parabolic problems evolve in time and diffuse, so a method of lines view is natural. We discretize space first, then choose time stepping with stiffness in mind since explicit diffusion often forces very small Δt . Hyperbolic problems transport at finite speed, so stability and accuracy depend on respecting flow direction, which is why upwinding and CFL limits matter and why characteristics give the right intuition for which boundary and initial data actually affect a point.

10.8 Exercises

Exercise 10.1

Categorize the following boundary conditions as Dirichlet, Neumann, Robin, or Cauchy:

(a) $c(x, y), \quad \nabla^2 c = 0$

$$c(0, y) = c(x, 0) = 0$$

$$c(L, y) = c(0, L) = 1$$

(b) $c(x), \quad \frac{d^2 c}{dx^2} = 0$

$$\left. \frac{dc}{dx} \right|_{x=0} = \left. \frac{dc}{dx} \right|_{x=L} = 0$$

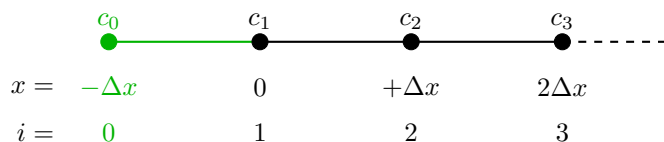
(c) $c(x, t), \quad \frac{dc}{dt} = \mathcal{D} \frac{\partial^2 c}{\partial x^2}$

$$c(x, 0) = c_0(x)$$

$$c(0, t) = c(L, t) = 1$$

Exercise 10.2

What equation would enforce the correct boundary condition using a finite difference scheme with a ghost point c_0 ?



$$c(x=0) - \left. \frac{dc}{dx} \right|_{x=0} = \alpha$$

Write your answer using c_i and α .

Exercise 10.3

(*) What is the stability criterion you expect when solving the following PDE using a forward time / central difference discretization?

$$\frac{\partial c}{\partial t} + U \frac{\partial c}{\partial x} - \mathcal{D} \frac{\partial^2 c}{\partial x^2} = 0$$

Chapter 11

Differential-Algebraic Equations

Engineering problems often contain combinations of differential and algebraic equations. Differential equations describe the evolution of states (how things change over time), and algebraic equations can appear to enforce constraints, such as the conservation of energy, mass, momentum, atomic species, or charge, on the state variables. Moreover, algebraic equations may represent pseudo-steady-state approximations for fast dynamics or imposed control objectives. When we mix these two types of equations, we obtain a system of differential-algebraic equations (DAEs). To formalize this, we define a state vector $\mathbf{z}(t)$ that contains all the variables of interest. The most general governing equation for a DAE system is written as a fully implicit function of time, the state, and the time-derivative of the state:

$$\mathbf{f}(t, \mathbf{z}, \dot{\mathbf{z}}) = \mathbf{0}, \quad \text{where } \dot{\mathbf{z}} := \frac{d\mathbf{z}}{dt} \quad (11.0.1)$$

This looks similar to the implicit ODE formulations we have seen previously, but there are key differences that we will explore in this chapter. We will explore analytical properties of DAEs first and discuss numerical solutions later. Solving them numerically is very similar to solving ODE-IVPs.

Motivation: Reaction Kinetics and Mass Conservation To see why DAEs naturally arise, let's consider the chemical reaction network $A \longrightarrow B \rightleftharpoons 2D$. If we model this using standard kinetics, we obtain a system of three ODEs describing the rate of change for each species concentration:

$$\frac{dC_A}{dt} = -k_1 C_A \quad (11.0.2)$$

$$\frac{dC_B}{dt} = k_1 C_A - k_2 C_B + k_3 C_D^2 \quad (11.0.3)$$

$$\frac{dC_D}{dt} = 2k_2 C_B - 2k_3 C_D^2 \quad (11.0.4)$$

In this ODE system, the total mass is implicitly conserved by the kinetics, but numerical integration errors might cause the total mass to drift over time during a simulation. Using any of the ODE-IVP solvers we have discussed, we might expect the truncation error at each timestep to be small, but know that these errors can accumulate over many timesteps as the simulation proceeds. Similarly, if we were to sum over our estimates of $C_A(t)$, $C_B(t)$, and $C_D(t)$ at different timepoints, we will observe this drift.

As an alternative, we can formulate this as a DAE system. We keep the kinetic rate laws for species A and B , but we replace the differential equation for D with an algebraic constraint enforcing the exact conservation of atomic species (mass balance). If Q_2 is a constant total mass invariant (i.e., $Q_2 = \text{const.}$), the system

becomes

$$\frac{dC_A}{dt} = -k_1 C_A \quad (11.0.5)$$

$$\frac{dC_B}{dt} = k_1 C_A - k_2 C_B + k_3 C_D^2 \quad (11.0.6)$$

$$Q_2 = C_A + C_B + \frac{1}{2} C_D \quad (11.0.7)$$

In this DAE formulation, the exact conservation of mass is enforced directly by the algebraic equation. This prevents the drift that might occur in the pure ODE system (Figure 11.0.1). Analytically, both formulations describe the same physical system, but numerically, the DAE approach can achieve better conservation properties by enforcing the algebraic constraint Eq. 11.0.7 to hold at every time point.

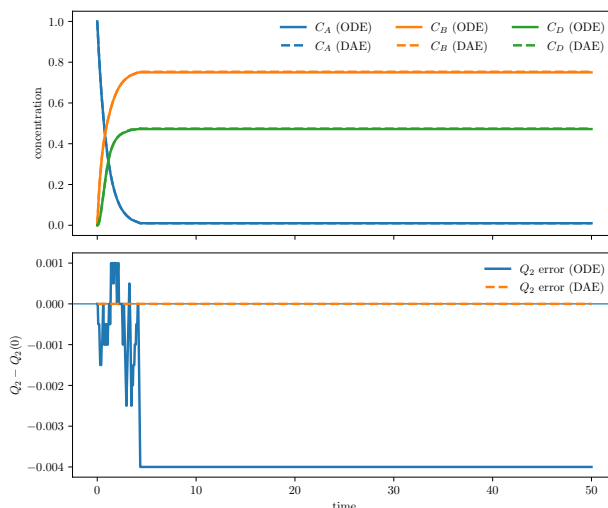


Figure 11.0.1: ODE vs. DAE simulation of the reaction network $A \rightarrow B \rightleftharpoons 2D$. Top: species concentrations from a forward-Euler time discretization ($\Delta t = 0.05$) with $k_1 = 1.0$, $k_2 = 0.6$, $k_3 = 2.0$ and $(C_A, C_B, C_D)(0) = (1, 0, 0)$. In the ODE formulation, C_A , C_B , and C_D are advanced by their kinetic rate laws; in the DAE formulation, only C_A and C_B are integrated and C_D is recovered each step by directly enforcing the mass balance constraint $Q_2 = C_A + C_B + \frac{1}{2} C_D$ (with $Q_2 = Q_2(0)$). Bottom: mass-balance violation $Q_2(t) - Q_2(0)$; rounding each Euler update to 10^{-3} induces visible drift for the ODE, whereas the DAE remains exactly on the constraint manifold.

11.1 Standard Forms of DAEs

We represent DAEs in three common forms. The first, as introduced above, is the **fully implicit form**:

$$\mathbf{f}(t, \mathbf{z}, \dot{\mathbf{z}}) = \mathbf{0} \quad (11.1.1)$$

In many physical systems, the variables naturally separate into those that appear with time derivatives (differential states, $\mathbf{x}(t)$) and those that do not (algebraic states, $\mathbf{y}(t)$). This leads to the **semi-explicit form**:

$$\begin{aligned} \dot{\mathbf{x}} &= \mathbf{f}(t, \mathbf{x}, \mathbf{y}) \\ \mathbf{0} &= \mathbf{g}(t, \mathbf{x}, \mathbf{y}) \end{aligned} \quad \text{with } \mathbf{z}(t) = \begin{bmatrix} \mathbf{x}(t) \\ \mathbf{y}(t) \end{bmatrix} \quad (11.1.2)$$

It is often convenient to write equations in this manner, as many physical balances naturally give us this structure. Identifying which states are differential and which are algebraic is as simple as checking whether the time derivative of a variable appears in the governing equations.

Interestingly, the semi-explicit form can be rewritten trivially in the third form, the **mass matrix form**, by introducing a singular mass matrix:

$$\underbrace{\begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}}_{\text{mass matrix, } \mathbf{M}} \begin{bmatrix} \dot{\mathbf{x}} \\ \dot{\mathbf{y}} \end{bmatrix} = \begin{bmatrix} \mathbf{f}(t, \mathbf{x}, \mathbf{y}) \\ \mathbf{g}(t, \mathbf{x}, \mathbf{y}) \end{bmatrix} \quad (11.1.3)$$

More generally, the mass matrix form is defined as

$$\mathbf{M}(t, \mathbf{z}) \dot{\mathbf{z}} = \mathbf{f}(t, \mathbf{z}) \quad (11.1.4)$$

11.1.0.1 Connection to ODEs

Recall the standard ODE formulation $\dot{\mathbf{x}} = \mathbf{f}(t, \mathbf{x})$. If the mass matrix $\mathbf{M}$ is invertible, we can simply multiply [Eq. 11.1.4](#) by $\mathbf{M}^{-1}$ to arrive at an ODE system:

$$\dot{\mathbf{z}} = \mathbf{M}(t, \mathbf{z})^{-1} \mathbf{f}(t, \mathbf{z}) \quad (11.1.5)$$

However, the noninvertibility of the mass matrix is what makes a DAE a DAE, and not an ODE. In the semi-explicit block matrix example above, it is clear that the mass matrix is singular. Consequently, we cannot convert the system to an ODE just by inverting $\mathbf{M}$.

Regarding numerical solutions, MATLAB's `ode15s` and `ode23t` solvers can handle DAEs in mass matrix form. MATLAB's `ode15i` solver is designed to solve fully implicit forms provided the DAEs are of index 1 (a concept we will define shortly).

11.1.0.2 Example DAE Systems

Example: Fluctuating Tank Feed Let us solidify these concepts with a concrete example of a fluctuating tank feed. Consider a stirred tank with an inlet concentration $c_1(t)$ and an outlet concentration $c_2(t)$. The volume V and flow rate Q are constant. The inlet concentration is specified by a forcing function $\gamma(t)$, which may represent a time-varying feed concentration. This system is illustrated in [Figure 11.1.1](#).

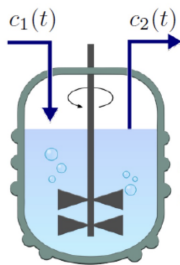


Figure 11.1.1: A continuous stirred tank reactor with a fluctuating inlet concentration $c_1(t)$.

The governing equations for this system are the component balance for the tank and the forcing function for the inlet:

$$\frac{dc_2}{dt} = \frac{Q}{V}(c_1(t) - c_2(t)) \quad (11.1.6)$$

$$c_1(t) = \gamma(t) \quad (11.1.7)$$

subject to initial conditions $c_1(0) = \gamma(0)$ and $c_2(0) = c_0$. We can derive an analytical solution for this system. The inlet concentration is simply the forcing function itself, $c_1(t) = \gamma(t)$. For the tank concentration, we solve the linear ODE using an integrating factor to obtain

$$c_2(t) = c_0 e^{-(Q/V)t} + \frac{Q}{V} \int_0^t \gamma(t') e^{-(Q/V)(t-t')} dt' \quad (11.1.8)$$

To analyze this system as a DAE, we must identify our states. Which are differential and which are algebraic? Here, c_2 is a differential state because its time derivative appears in the governing equations. c_1 is an algebraic state because no time derivative of c_1 appears; its value is determined instantaneously by the algebraic constraint $c_1(t) = \gamma(t)$.

We can express this system in our standard DAE forms. In the fully implicit form, we move everything to one side to satisfy $\mathbf{f}(t, \mathbf{z}, \dot{\mathbf{z}}) = \mathbf{0}$:

$$\begin{bmatrix} \frac{dc_2}{dt} - \frac{Q}{V}(c_1(t) - c_2(t)) \\ c_1(t) - \gamma(t) \end{bmatrix} = \mathbf{0} \quad (11.1.9)$$

Alternatively, we can write this in the mass matrix form, $\mathbf{M}(t, \mathbf{z})\dot{\mathbf{z}} = \mathbf{f}(t, \mathbf{z})$. To do so, we first define our state vector $\mathbf{c}$ (equivalent to $\mathbf{z}$ in our typical notation) containing both algebraic and differential variables:

$$\mathbf{c}(t) = \begin{bmatrix} c_1(t) \\ c_2(t) \end{bmatrix} \quad (11.1.10)$$

Next, we rearrange our governing equations to explicitly display the coefficients for the time derivatives ($\dot{c}_1, \dot{c}_2$) on the left and the state variables (c_1, c_2) on the right. Note that since c_1 is algebraic, its derivative $\dot{c}_1$ has a coefficient of 0 in both equations:

$$\begin{aligned} 0 \cdot \dot{c}_1 + 1 \cdot \dot{c}_2 &= \frac{Q}{V}c_1 - \frac{Q}{V}c_2 \\ 0 \cdot \dot{c}_1 + 0 \cdot \dot{c}_2 &= -1 \cdot c_1 + 0 \cdot c_2 + \gamma(t) \end{aligned} \quad (11.1.11)$$

By collecting these coefficients into matrices, we arrive at the linear mass matrix form:

$$\underbrace{\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}}_{\mathbf{M}} \underbrace{\begin{bmatrix} \dot{c}_1 \\ \dot{c}_2 \end{bmatrix}}_{\dot{\mathbf{c}}} = \underbrace{\begin{bmatrix} \frac{Q}{V} & -\frac{Q}{V} \\ -1 & 0 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} c_1 \\ c_2 \end{bmatrix}}_{\mathbf{c}} + \underbrace{\begin{bmatrix} 0 \\ \gamma(t) \end{bmatrix}}_{\mathbf{b}(t)} \quad (11.1.12)$$

Note the singularity of the mass matrix $\mathbf{M}$ on the left-hand side (specifically, the row of zeros). This explicitly confirms that the system is a DAE since $\mathbf{M}$ cannot be inverted to recover a standard ODE.

Example 2: Specified Tank Concentration Let us reconsider the stirred tank reactor from the previous section (Figure 11.1.1). In the first example, we specified the *input* concentration and solved for the output. Now, consider the inverse problem: we wish to specify the *output* concentration profile $c_2(t) = \gamma(t)$ and determine the necessary input concentration $c_1(t)$ to achieve it. The governing equations are still the component balance and the constraint, but the constraint now applies to the outlet variable:

$$\frac{dc_2}{dt} = \frac{Q}{V}(c_1(t) - c_2(t)) \quad (11.1.13)$$

$$c_2(t) = \gamma(t) \quad (11.1.14)$$

The initial conditions must be consistent with the constraints that $c_2(0) = \gamma(0)$ and $c_1(0) = \gamma(0) + \frac{V}{Q}\dot{\gamma}(0)$.

Analytically, we can solve this by substituting the constraint directly into the differential equation. Since $c_2(t) = \gamma(t)$, it follows that $\frac{dc_2}{dt} = \dot{\gamma}(t)$. Substituting these into the component balance and rearranging for $c_1(t)$ gives the analytical solution:

$$c_1(t) = \gamma(t) + \frac{V}{Q}\dot{\gamma}(t) \quad (11.1.15)$$

$$c_2(t) = \gamma(t) \quad (11.1.16)$$

There is a big difference here compared to the previous example. The solution for $c_1(t)$ now depends on the *derivative* of the forcing function $\dot{\gamma}(t)$. This dependence on derivatives is characteristic of “higher-index” DAEs, which we will define formally in a moment.

We can also write this system in our standard forms. The fully implicit form is

$$\begin{bmatrix} \frac{dc_2}{dt} - \frac{Q}{V}(c_1(t) - c_2(t)) \\ c_2(t) - \gamma(t) \end{bmatrix} = \mathbf{0} \quad (11.1.17)$$

In mass matrix form, we again define the state vector as $\mathbf{c} = [c_1, c_2]^\top$. We rearrange the governing equations to isolate the time derivative terms on the left and the state terms on the right. For the component balance, $\dot{c}_2$ appears with a coefficient of 1. For the constraint equation, no time derivatives appear, so both coefficients are 0:

$$\begin{aligned} 0 \cdot \dot{c}_1 + 1 \cdot \dot{c}_2 &= \frac{Q}{V}c_1 - \frac{Q}{V}c_2 \\ 0 \cdot \dot{c}_1 + 0 \cdot \dot{c}_2 &= 0 \cdot c_1 - 1 \cdot c_2 + \gamma(t) \end{aligned} \quad (11.1.18)$$

Collecting these terms into matrix notation,

$$\underbrace{\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}}_{\mathbf{M}} \underbrace{\begin{bmatrix} \dot{c}_1 \\ \dot{c}_2 \end{bmatrix}}_{\dot{\mathbf{c}}} = \underbrace{\begin{bmatrix} \frac{Q}{V} & -\frac{Q}{V} \\ 0 & -1 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} c_1 \\ c_2 \end{bmatrix}}_{\mathbf{c}} + \underbrace{\begin{bmatrix} 0 \\ \gamma(t) \end{bmatrix}}_{\mathbf{b}(t)} \quad (11.1.19)$$

As before, the second row of the mass matrix is zero because the second equation is purely algebraic ($0 = -c_2 + \gamma(t)$).

11.2 The Differential Index (or Differentiation Index)

The behavior we observed above, where the solution depends on the derivative of the input, leads us to the concept of the **differential index**. The index of a DAE is defined as the number of times the system must be differentiated to obtain a complete set of first-order ODEs for all states (both algebraic and differential). It can be thought of as a measure of how far the system is from being an ODE. As we have already hinted at, it will turn out that “index 1” DAEs are substantially more well-behaved than “high-index” DAEs. To understand this definition, we look at the procedure of differentiating the algebraic constraints until we can express the time derivative of every state variable, $\dot{\mathbf{z}}$, as a function of the states and inputs.

11.2.1 Index 1 vs. Index 2: The Tank Filling Problem

Let us examine a mechanical tank filling problem (Figure 11.2.1); this is distinct from the chemical concentration problem above and lets us drop some extra constant terms. We define two variables, the fluid volume in the tank $x_1(t)$ and the outlet flow rate $x_2(t)$. The inlet flow rate is $g_1(t)$.

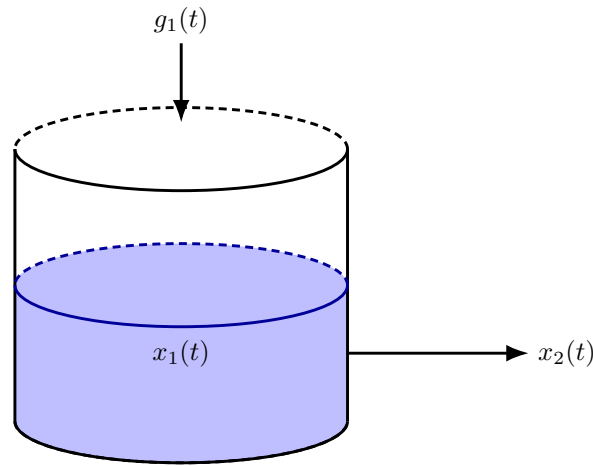


Figure 11.2.1: Tank filling problem setup.

11.2.1.1 Case A: Specified Outlet Flow (Index 1)

Suppose we specify the outlet flow rate as $x_2 = g_2(t)$. The governing equations are the mass balance and the flow constraint:

$$(1) \quad \dot{x}_1 + x_2 = g_1(t) \quad (11.2.1)$$

$$(2) \quad x_2 = g_2(t) \quad (11.2.2)$$

To find the index, we check how many derivatives are needed to define $\dot{x}_1$ and $\dot{x}_2$. We already have an expression for $\dot{x}_1$ from equation (1): $\dot{x}_1 = g_1(t) - x_2$. To get $\dot{x}_2$, we differentiate equation (2) once:

$$\dot{x}_2 = \dot{g}_2(t) \quad (11.2.3)$$

We have now found explicit ODEs for all states ($\dot{x}_1$ and $\dot{x}_2$) using only one differentiation step. Therefore, this is an index 1 DAE. The analytical solution involves an integral of the inputs:

$$x_1(t) = x_1(t_0) + \int_{t_0}^t (g_1(t') - g_2(t')) dt', \quad x_2(t) = g_2(t) \quad (11.2.4)$$

Because integration is a smoothing operation, this system is well-behaved. Noise in the forcing functions g_1 or g_2 does not cause catastrophic errors in the solution.

11.2.1.2 Case B: Specified Internal Volume (Index 2)

Now consider a different control objective where we specify the internal volume trajectory $x_1 = g_2(t)$ directly. The equations are

$$\dot{x}_1 + x_2 = g_1(t) \quad (1)$$

$$x_1 = g_2(t) \quad (2)$$

First, we address the algebraic constraint. Equation (2) defines x_1 directly, but Equation (1) depends on $\dot{x}_1$. To link these, we differentiate the constraint (2) with respect to time:

$$\frac{d}{dt}(x_1) = \frac{d}{dt}(g_2(t)) \implies \dot{x}_1 = \dot{g}_2(t) \quad (2')$$

Next, we substitute this expression for $\dot{x}_1$ into Equation (1). This allows us to solve explicitly for the algebraic variable, x_2 :

$$\underbrace{\dot{g}_2(t)}_{\dot{x}_1} + x_2 = g_1(t) \implies x_2 = g_1(t) - \dot{g}_2(t) \quad (11.2.5)$$

At this stage, we have expressions for x_1 and x_2 , but we do not yet have a differential equation describing how x_2 changes over time. To find $\dot{x}_2$, we must differentiate the derived expression for x_2 :

$$\dot{x}_2 = \frac{d}{dt}(g_1(t) - \dot{g}_2(t)) = \dot{g}_1(t) - \ddot{g}_2(t) \quad (11.2.6)$$

We have now successfully derived the complete set of first-order ODEs. We must count the number of differentiation steps required. We differentiated the original constraint ($x_1 = g_2(t)$) once to find $\dot{x}_1$ and a second time to find $\dot{x}_2$. Because two differentiations were required to transform the DAE into an explicit ODE system, this is classified as an index 2 system.

The analytical solution is

$$x_1(t) = g_2(t), \quad x_2(t) = g_1(t) - \dot{g}_2(t) \quad (11.2.7)$$

Note the dependence on $\dot{g}_2(t)$. If the specified volume signal $g_2(t)$ is noisy, differentiation will amplify that noise. For example, if $g_2(t)$ has a small step change, its derivative contains an impulse (Dirac delta), causing violent transients in the calculated flow rate x_2 . Index 2 (and higher) systems are essentially “numerical differentiators,” and lead to noise exaggeration.

This high index is often a result of poor modeling choices.¹ Physically, how could we possibly force the volume to be *exactly* $g_2(t)$ at all times? A more realistic model would implement a controller that adjusts the outlet flow x_2 based on the error between the actual volume x_1 and the setpoint g_2 . Such a control implementation should result in a well-behaved index 1 system.

11.2.2 Another Example: Calculation of Higher Indices

We can apply this differentiation procedure to arbitrary systems to determine their index. Consider the following coupled system:

$$(1) \quad \dot{x}_2 = x_1 \quad (11.2.8)$$

$$(2) \quad \dot{x}_3 = x_2 \quad (11.2.9)$$

$$(3) \quad x_3 = \gamma(t) \quad (11.2.10)$$

To find the index, we differentiate the algebraic constraint (3) repeatedly until we have expressions for $\dot{x}_1$, $\dot{x}_2$, and $\dot{x}_3$. Note that here x_1 is an algebraic variable (it lacks a derivative in the original equations). We follow the convention of differentiating until the first derivative expressions are explicit for all state variables.

Differentiation 1: Differentiate (3) to get $\dot{x}_3$:

$$(3') \quad \dot{x}_3 = \dot{\gamma}(t) \quad (11.2.11)$$

Substituting (3') into (2), we find an algebraic relationship for x_2 :

$$x_2 = \dot{\gamma}(t) \quad (11.2.12)$$

Differentiation 2: Differentiate the expression for x_2 :

$$(2') \quad \dot{x}_2 = \ddot{\gamma}(t) \quad (11.2.13)$$

Substituting (2') into (1), we find an algebraic relationship for x_1 :

$$x_1 = \ddot{\gamma}(t) \quad (11.2.14)$$

Differentiation 3: Differentiate the expression for x_1 to finally get $\dot{x}_1$:

$$(1') \quad \dot{x}_1 = \dddot{\gamma}(t) \quad (11.2.15)$$

We required 3 differentiations to obtain the first order differential equation for the final state x_1 ; we conclude that this is an index 3 DAE. The complete set of underlying ODEs is:

$$\dot{x}_1 = \dddot{\gamma}(t) \quad (11.2.16)$$

$$\dot{x}_2 = \ddot{\gamma}(t) \quad (11.2.17)$$

$$\dot{x}_3 = \dot{\gamma}(t) \quad (11.2.18)$$

As with the index 2 tank example, we should expect that solving this system numerically will be difficult. Accurately tracking the true solution requires matching the third derivative of the forcing function $\gamma(t)$, which is extremely sensitive to numerical error. In general, index 1 DAEs are much more amenable to numerical solution than systems with index > 1 .

¹In process modeling specifically, the usual culprits are idealized/perfect controllers, ideal (instantaneous) separation specifications. Each of these effectively imposes an exact algebraic condition on a variable that, physically, is only ever driven *toward* a target with finite dynamics. Commercial process simulators (e.g., gPROMS, Aspen) therefore run automatic structural index-reduction algorithms on the flowsheet model rather than relying on the user to manually reformulate it.

Warning: In this course, the differential index is the number of times we must differentiate the DAE system until we can write *explicit first-order ODEs for every component of the full state* $\mathbf{z} = [\mathbf{x}; \mathbf{y}]$. This usually means differentiating the algebraic constraints until we can write explicit ODEs for every component of $\mathbf{z}$. Here are some notes to keep in mind:

- Do not equate the number of algebraic equations with index. A system can have many algebraic constraints and still be index 1, or have a single constraint and be index 2+.
- Elimination can hide the index. If you substitute constraints to eliminate variables, you may get an ODE for $\mathbf{x}$ quickly, but under our definition you must still ask: how many differentiations are needed to determine $\dot{\mathbf{y}}$ (and hence $\dot{\mathbf{z}}$)?
- Watch for hidden consistency conditions. Differentiating constraints can introduce additional conditions that must hold at t_0 (e.g. involving $\dot{\gamma}(t_0)$, $\ddot{\gamma}(t_0)$). If initial conditions violate these, the model is not (classically) solvable without projection/re-initialization. We will see this in more detail later.
- Index is a structural property, not “how hard it feels.” Stiffness and fast/slow time scales affect numerics, but they are not the index. The index is about how many differentiations are required to make $\dot{\mathbf{z}}$ explicit.

11.3 Distinguishing Index 1 from Higher Index Systems

We mentioned earlier that index 1 DAEs are generally well-behaved and solvable with standard methods, while higher-index systems pose significant numerical challenges. But how do we determine the index of a given system without performing the repeated differentiation manually every time? It turns out that there is a convenient shortcut that can distinguish index 1 from higher-index DAEs, although it does not provide the actual index in the latter case.

Let us revisit the standard semi-explicit form:

$$\dot{\mathbf{x}} = \mathbf{f}(t, \mathbf{x}, \mathbf{y}) \quad (11.3.1)$$

$$\mathbf{0} = \mathbf{g}(t, \mathbf{x}, \mathbf{y}) \quad (11.3.2)$$

Recall that $\mathbf{x}$ represents the differential states (those appearing with time derivatives) and $\mathbf{y}$ represents the algebraic states. We can differentiate Eq. 11.3.2 with respect to time using the chain rule:

$$\mathbf{0} = \frac{\partial \mathbf{g}}{\partial t} + \frac{\partial \mathbf{g}}{\partial \mathbf{x}} \dot{\mathbf{x}} + \frac{\partial \mathbf{g}}{\partial \mathbf{y}} \dot{\mathbf{y}}. \quad (11.3.3)$$

We know $\dot{\mathbf{x}}$ from Eq. 11.3.1, and substitute it into this new relation:

$$\mathbf{0} = \frac{\partial \mathbf{g}}{\partial t} + \frac{\partial \mathbf{g}}{\partial \mathbf{x}} \mathbf{f}(t, \mathbf{x}, \mathbf{y}) + \frac{\partial \mathbf{g}}{\partial \mathbf{y}} \dot{\mathbf{y}} \quad (11.3.4)$$

Now, observe the term attached to $\dot{\mathbf{y}}$. If the Jacobian matrix $\frac{\partial \mathbf{g}}{\partial \mathbf{y}}$ is non-singular (invertible), we can rearrange Eq. 11.3.4 to isolate $\dot{\mathbf{y}}$ on the left-hand side:

$$\dot{\mathbf{y}} = - \left(\frac{\partial \mathbf{g}}{\partial \mathbf{y}} \right)^{-1} \left(\frac{\partial \mathbf{g}}{\partial t} + \frac{\partial \mathbf{g}}{\partial \mathbf{x}} \mathbf{f}(t, \mathbf{x}, \mathbf{y}) \right) \quad (11.3.5)$$

If this inversion is possible, we have successfully derived a complete set of ODEs for both $\mathbf{x}$ and $\mathbf{y}$ with just one step of differentiation, i.e., this is an index 1 DAE.

Therefore, the condition for a DAE to be index 1 is that the Jacobian of the algebraic equations with respect to the algebraic variables, $\frac{\partial \mathbf{g}}{\partial \mathbf{y}}$, is non-singular.

Example: Let's apply this Jacobian test to the coupled system we analyzed previously to determine if it is an index 1 or higher-index DAE:

$$(1) \quad \dot{x}_2 = x_1 \quad (11.3.6)$$

$$(2) \quad \dot{x}_3 = x_2 \quad (11.3.7)$$

$$(3) \quad x_3 = \gamma(t) \quad (11.3.8)$$

First, we must classify the variables. The variables x_2 and x_3 appear with derivatives, so they are differential states. The variable x_1 appears only on the right-hand side of (1) and has no explicit derivative in the system; in a semi-explicit formulation $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{y})$, the variable x_1 plays the role of the algebraic state $\mathbf{y}$ because it defines the rate of change of a differential state but is not one itself. The algebraic equation is $0 = x_3 - \gamma(t)$.

Let us check the Jacobian of the constraint function $g = x_3 - \gamma(t)$ with respect to the algebraic state x_1 :

$$\frac{\partial g}{\partial x_1} = \frac{\partial}{\partial x_1}(x_3 - \gamma(t)) = 0 \quad (11.3.9)$$

The Jacobian is zero (a 1×1 singular matrix). Because the Jacobian is singular, we cannot solve for $\dot{x}_1$ directly from the differentiated constraint. This confirms that the system is index > 1 (as we saw before, it is actually index 3).

11.4 Reducing the Index of a DAE

If we encounter a high-index problem, we are not necessarily stuck. In many/most regular DAEs encountered, we can apply an **index reduction** procedure (usually involving differentiating some constraints and/or introducing auxiliary variables) to obtain a formulation that is low index and therefore compatible with standard DAE integrators. This should not be interpreted as a guarantee that every DAE can be transformed in a globally equivalent way. If the model is structurally singular, inconsistent, or poorly posed, index reduction may fail or may introduce additional consistency conditions that must be enforced. In general, index reduction can produce an analytically equivalent system, but the numerical solution may differ.

Why do we care? Most general-purpose DAE solvers are designed for index-1 systems. When given an index > 1 formulation, they likely either (i) fail during initialization/Newton solves, (ii) exhibit severe order reduction, or (iii) attempt internal symbolic manipulations (e.g., differentiating constraints) to reduce the index. A practical way to recognize index > 1 behavior is that the solution (or consistent initialization) depends on time derivatives of prescribed signals and/or on derivatives of algebraic constraints.

11.4.1 Method of Auxiliary Variables

A common index-reduction technique is the **method of auxiliary variables**. The idea is simple:

- Identify a quantity that appears as a *derivative* (e.g., $\dot{x}_1$) but is *not* directly represented as an independent variable in the model.
- Introduce a new variable to represent that derivative (e.g., $x_2 := \dot{x}_1$).
- Replace every occurrence of the derivative by the auxiliary variable.
- Add additional equations obtained by differentiating the appropriate algebraic constraint(s) until the enlarged system becomes index 1.

This procedure trades hidden derivative constraints (which cause high index) for explicit algebraic equations involving the new auxiliary variables. The resulting system will have more variables and more equations, but if done correctly, will be lower index and thus much easier to initialize and solve numerically.

As an example, let's consider the system

$$(1) \quad \dot{x}_1 = y_1 - y_2 \quad (11.4.1)$$

$$(2) \quad x_1 = e^{-t} \quad (11.4.2)$$

$$(3) \quad y_1 + y_2 = 1 \quad (11.4.3)$$

Here, x_1 is constrained algebraically by (2), while (1) requires $\dot{x}_1$; this is a common signature of a high-index formulation. We differentiate the algebraic constraints once:

$$(2') \quad \dot{x}_1 = -e^{-t} \quad (11.4.4)$$

$$(3') \quad \dot{y}_1 + \dot{y}_2 = 0 \quad (11.4.5)$$

Equation (2') is an implicit (hidden) constraint on the dynamics, which we will discuss in more detail in [section 11.6](#). Since $x_1 = e^{-t}$ must hold for all t , its derivative must also hold, so $\dot{x}_1$ is not free. Combining (1) and (2') already forces

$$y_1 - y_2 = -e^{-t} \quad (11.4.6)$$

and further differentiations are required to obtain explicit ODEs for all variables. Indeed, this is an index-2 system.

Now, we define an auxiliary variable

$$x_2 := \dot{x}_1 \quad (11.4.7)$$

and replace $\dot{x}_1$ in (1) by x_2 . Then, we make the hidden constraint explicit by including (2') as an algebraic equation:

$$(1) \quad x_2 = y_1 - y_2 \quad (11.4.8)$$

$$(2) \quad x_1 = e^{-t} \quad (11.4.9)$$

$$(3) \quad y_1 + y_2 = 1 \quad (11.4.10)$$

$$(4) \quad x_2 = -e^{-t} \quad (11.4.11)$$

This enlarged system ends up being an all-algebraic index 1 system. (In general, we do not expect this to always happen.) The constraints now determine (x_1, x_2, y_1, y_2) directly at each time without requiring further differentiation.

Foreshadowing [section 11.6](#), we might also notice that at $t = t_0$, equation (2) fixes $x_1(t_0) = e^{-t_0}$, equation (4) fixes $x_2(t_0) = -e^{-t_0}$, and equations (1) and (3) then uniquely determine $(y_1(t_0), y_2(t_0))$:

$$y_1(t_0) = \frac{1 + x_2(t_0)}{2}, \quad y_2(t_0) = \frac{1 - x_2(t_0)}{2} \quad (11.4.12)$$

So the four equations in this DAE system leave us with 0 degrees of freedom when initializing our state variables at $t = 0$ —all variables are fixed by the constraints.

Example: Let's apply the method of auxiliary variables to the index-2 tank filling problem where the volume x_1 is specified:

$$(1) \quad \dot{x}_1 + x_2 = g_1(t) \quad (11.4.13)$$

$$(2) \quad x_1 = g_2(t) \quad (11.4.14)$$

Introducing an auxiliary variable for the derivative,

$$x_3 := \dot{x}_1 \quad (11.4.15)$$

and making the differentiated constraint explicit gives

$$(1) \quad x_3 + x_2 = g_1(t) \quad (11.4.16)$$

$$(2) \quad x_1 = g_2(t) \quad (11.4.17)$$

$$(3) \quad x_3 = \dot{g}_2(t) \quad (11.4.18)$$

We have reduced the index from 2 to 1 by introducing an auxiliary variable.

Warning: Index reduction can introduce differentiated constraints such as $x_3 = \dot{g}_2(t)$ in the example above (or, more generally, relations obtained by differentiating $\mathbf{g}(t, \mathbf{x}, \mathbf{y}) = \mathbf{0}$). Two practical warnings:

1. Differentiated constraints impose extra requirements at t_0 (e.g., involving $\dot{g}_2(t_0)$, $\ddot{g}_2(t_0)$, etc.). If these are violated, the original model is not classically solvable without re-initialization/projection.
2. If we replace an original constraint by only its differentiated form(s) and do not re-enforce the original constraint (via projection or stabilization), numerical solutions may drift off the true constraint manifold. In the reformulations above, we retained the original constraints (e.g., $x_1 = g_2(t)$), so those constraints cannot drift at the discrete solution points.

11.5 Numerical Simulation of DAEs

We approach solving DAEs numerically in the same way as ODE-IVPs: we discretize time and estimate the solution at a finite set of times, stepping Δt by Δt . However, the presence of algebraic constraints resembles a highly stiff ODE and makes the use of implicit methods necessary.

11.5.1 Connection to Stiffness in ODEs

We can think of a DAE as the limit of a very stiff ODE. Consider the system:

$$\dot{\mathbf{x}} = \mathbf{f}(t, \mathbf{x}, \mathbf{y}) \quad (11.5.1)$$

$$\epsilon \dot{\mathbf{y}} = \mathbf{g}(t, \mathbf{x}, \mathbf{y}) \quad (11.5.2)$$

If we take the limit as $\epsilon \rightarrow 0$, we recover $\mathbf{0} = \mathbf{g}(t, \mathbf{x}, \mathbf{y})$ which is a DAE in semi-explicit form. For a very small but non-zero ϵ , the term $\dot{\mathbf{y}}$ would be very large, corresponding to extremely fast dynamics that must be resolved on arbitrarily short time scales. This is the definition of a stiff problem. Qualitatively, we can interpret a DAE as an “infinitely stiff” ODE. Consequently, explicit methods (like forward Euler) generally fail, and we must use implicit methods (stiff solvers).

11.5.2 Failure of Forward Euler

Consider trying to use the forward Euler method on a semi-explicit DAE $\dot{\mathbf{x}} = \mathbf{f}(t, \mathbf{x}, \mathbf{y})$. We calculate the next step for the differential states as

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t \mathbf{f}(t_i, \mathbf{x}(t_i), \mathbf{y}(t_i)) \quad (11.5.3)$$

However, simply updating $\mathbf{x}$ does not guarantee that the new state $\mathbf{x}(t_{i+1})$ satisfies the algebraic constraint $\mathbf{0} = \mathbf{g}(t_{i+1}, \mathbf{x}(t_{i+1}), \mathbf{y}(t_{i+1}))$. The solution will drift off the constraint manifold.

The key issue is that an explicit update of $\mathbf{x}$ alone does not enforce the algebraic constraints. A potential remedy for index-1 systems is an explicit–projection step: first compute $\mathbf{x}(t_{i+1})$ explicitly using forward Euler, and then determine $\mathbf{y}(t_{i+1})$ by solving the algebraic constraint at the new time,

$$\mathbf{x}(t_{i+1}) = \mathbf{x}(t_i) + \Delta t \mathbf{f}(t_i, \mathbf{x}(t_i), \mathbf{y}(t_i)), \quad (11.5.4)$$

$$\mathbf{0} = \mathbf{g}(t_{i+1}, \mathbf{x}(t_{i+1}), \mathbf{y}(t_{i+1})). \quad (11.5.5)$$

This generally requires a (possibly nonlinear) solve for $\mathbf{y}(t_{i+1})$ only (assuming $\partial \mathbf{g} / \partial \mathbf{y}$ is nonsingular). This method also might lead to drift over many steps unless the projection is done very accurately.

Remember, the distinction between differential and algebraic states is *not* that we use $\mathbf{f}$ to solve for the differential states and $\mathbf{g}$ for the algebraic ones; the algebraic states might not even appear in the algebraic equations for high-index DAEs! Solving for $\mathbf{y}(t_{i+1})$ will be ill-posed in that case. We already saw exactly this in the index-3 example above: the algebraic constraint there was $g = x_3 - \gamma(t)$, which does not involve the algebraic state x_1 at all (recall $\partial g / \partial x_1 = 0$), which is precisely why the naïve Jacobian test failed and three differentiations were needed to recover $\dot{x}_1$.

11.5.3 Backward Euler and Simulation Errors

To handle this coupling robustly, DAE solvers typically use fully implicit methods to solve for $\mathbf{x}(t_{i+1})$ and $\mathbf{y}(t_{i+1})$ simultaneously. Consider the simplest implicit method, backward Euler. For a general DAE $\mathbf{f}(t, \mathbf{z}, \dot{\mathbf{z}}) = \mathbf{0}$, we approximate the derivative with a backward difference:

$$\mathbf{0} = \mathbf{f}(t_{i+1}, \mathbf{z}(t_{i+1}), \dot{\mathbf{z}}(t_{i+1})) \approx \mathbf{f}\left(t_{i+1}, \mathbf{z}(t_{i+1}), \frac{\mathbf{z}(t_{i+1}) - \mathbf{z}(t_i)}{\Delta t}\right) \quad (11.5.6)$$

At each time step, this requires solving a system of nonlinear equations for $\mathbf{z}(t_{i+1})$. Generalized multistep methods, such as those used in MATLAB's `ode15s`, `ode23t`, and `ode15i`, are built on this principle.

For high-index DAEs, low-order approximations like backward Euler can result in surprisingly poor accuracy. Let's return to our index 2 stirred tank system (specified output) to analyze the error scaling by hand. The equations are

$$\frac{dc_2}{dt} = \frac{Q}{V}(c_1(t) - c_2(t)) \quad (11.5.7)$$

$$c_2(t) = \gamma(t) \quad (11.5.8)$$

Using backward Euler, the discrete step for c_2 is simply the constraint $c_2(t_{i+1}) = \gamma(t_{i+1})$. We substitute the backward difference approximation for the derivative, $\left.\frac{dc_2}{dt}\right|_{t+1} = \frac{c_2(t_{i+1}) - c_2(t_i)}{\Delta t} + O(\Delta t)$, into Eq. 11.5.7 to solve for the input c_1 :

$$c_1(t_{i+1}) = c_2(t_{i+1}) + \frac{V}{Q} \left(\frac{c_2(t_{i+1}) - c_2(t_i)}{\Delta t} \right) + O(\Delta t) \quad (11.5.9)$$

If c_2 were known *exactly* at the grid points (e.g., if we can evaluate $\gamma(t_i)$ exactly and enforce $c_2(t_i) = \gamma(t_i)$ to solver tolerance), then this derivative approximation incurs an $O(\Delta t)$ truncation error, and the resulting error in the estimate of $c_1(t_{i+1})$ is correspondingly $O(\Delta t)$. If we consider how this local truncation error (LTE) accumulates over multiple steps to produce a global truncation error (GTE), we find that the GTE scales as $O(1)$ —it does not decrease as Δt decreases. That is a severe problem! Even if we take very small time steps, the error in $c_1(t)$ is not guaranteed to ever improve.

To improve the error scaling, we can use a higher-order implicit method, such as Crank-Nicolson or the trapezoidal rule. Under that discretization scheme, the update for c_2 becomes

$$c_2(t_{i+1}) = c_2(t_i) + \frac{\Delta t}{2} \left[\left.\frac{dc_2}{dt}\right|_{t_i} + \left.\frac{dc_2}{dt}\right|_{t_{i+1}} \right] + O(\Delta t^3) \quad (11.5.10)$$

Solving for $c_1(t_{i+1})$, we now get an expression where the error terms scale more favorably:

$$c_1(t_{i+1}) = c_2(t_{i+1}) - c_1(t_i) + c_2(t_i) + \frac{2V}{Q\Delta t}(c_2(t_{i+1}) - c_2(t_i)) + O(\Delta t^2) \quad (11.5.11)$$

With an LTE of $O(\Delta t^2)$, the GTE scales as $O(\Delta t)$, which is much more reasonable and helps ensure convergence as the step size decreases.

11.6 Initialization of Differential-Algebraic Equations

A challenge that distinguishes DAEs from ODEs is the problem of initialization. For a standard ODE system, finding a valid starting point is usually a matter of choice; for a DAE, it is a matter of consistency. Consider a generic system written in fully implicit form:

$$\mathbf{f}(\dot{\mathbf{x}}(t), \mathbf{x}(t), t) = \mathbf{0} \quad (11.6.1)$$

To start a numerical integrator at time t_0 , we theoretically need to determine the values of both the state vector $\mathbf{x}(t_0)$ and its time derivative $\dot{\mathbf{x}}(t_0)$. If the system consists of n equations with n variables, there are a total of $2n$ unknowns involved in the initialization: the set $\{\mathbf{x}(t_0), \dot{\mathbf{x}}(t_0)\}$. Since we have n equations, we nominally have n degrees of freedom remaining.

For an explicit ODE system where $\dot{\mathbf{x}} = \mathbf{h}(\mathbf{x}, t)$, this is straightforward. We specify the initial state $\mathbf{x}(t_0)$, and the equations themselves immediately determine the derivative $\dot{\mathbf{x}}(t_0)$. Our initial assumption might be that for a DAE in semi-explicit form, we similarly would specify the initial state $\mathbf{x}(t_0)$. However, we cannot simply choose an arbitrary $\mathbf{x}(t_0)$ because it might violate the algebraic equations. Algebraic constraints restrict the manifold on which the solution can exist. Furthermore, DAEs often contain hidden or **implicit constraints** that constrain values beyond the original set of governing equations. We must be careful to identify which of the initial values can actually be freely specified.

Definition (Implicit Constraint): An *implicit constraint* in a DAE is a relationship among the variables that is not explicitly stated in the original set of equations but emerges when differentiating algebraic constraints to reveal hidden dependencies. These constraints must be satisfied by the initial conditions to ensure consistency of the DAE solution. An index k DAE contains *at least* $k - 1$ implicit constraints. As a heuristic, an index k DAE will usually contain exactly $k - 1$ implicit constraints, although exceptions exist.

11.6.1 Consistent Initialization and Implicit Constraints

To illustrate the complexity of consistent initialization, let us revisit the stirred tank reactor system with a variable volume, which we previously identified as an index 2 DAE. The governing equations are

$$\begin{aligned} \frac{dc_2}{dt} &= \frac{Q}{V}(c_1(t) - c_2(t)) \\ c_2(t) &= \gamma(t) \end{aligned} \quad (11.6.2)$$

We assume $\gamma(t)$ is sufficiently smooth near t_0 so that the required derivatives (e.g., $\dot{\gamma}(t_0)$) exist. If we examine the discretized update equation (which a numerical solver would use to march forward), we see a dependency on the previous time step:

$$c_1(t_{i+1}) = c_2(t_{i+1}) - c_1(t_i) + c_2(t_i) + \frac{2V}{Q\Delta t}(c_2(t_{i+1}) - c_2(t_i)) + O(\Delta t^2) \quad (11.6.3)$$

To implement this numerical solution, we clearly need specific values for both $c_1(t_0)$ and $c_2(t_0)$ to initialize the loop. But can we set these arbitrarily?

Clearly not. The algebraic equation explicitly constrains the second concentration:

$$c_2(t_0) = \gamma(t_0) \quad (11.6.4)$$

Moreover, because this constraint holds for all time ($c_2(t) = \gamma(t)$), its time derivative must also hold ($\dot{c}_2(t) = \dot{\gamma}(t)$). Substituting this implicit constraint into the differential equation reveals a constraint on c_1 :

$$\frac{dc_2}{dt} = \frac{d\gamma}{dt} = \frac{Q}{V}(c_1 - c_2) \implies c_1 = c_2 + \frac{V}{Q}\dot{\gamma} = \gamma + \frac{V}{Q}\dot{\gamma} \quad (11.6.5)$$

Thus, $c_1(t_0)$ cannot be freely specified either! It is completely determined by the forcing function $\gamma(t)$ and its derivative. In this DAE, we have zero degrees of freedom.

In general, we can summarize the process of determining consistent initialization as follows:

1. Identify the total number of differential states (n) and algebraic states (m) in the DAE system. At t_0 , a total of $2n + m$ values need to be specified: $\{\mathbf{x}(t_0), \dot{\mathbf{x}}(t_0), \mathbf{y}(t_0)\}$. A well-posed DAE will typically have $n + m$ equations, leaving us with n degrees of freedom before considering implicit constraints.
2. Determine the index of the DAE. For an index k system, recognize that there are at least $k - 1$ implicit constraints that must be uncovered by differentiating algebraic equations.
3. Differentiate the algebraic constraints as needed to reveal implicit constraints. As a tip, we can often find these constraints by examining the derivatives that arise during the identification of the DAE's index.
4. Now, with knowledge of implicit constraints, recalculate the degrees of freedom available for initialization. Analyze the full set of equations to determine which specific variables can be freely specified while satisfying all explicit and implicit constraints; even if we have > 0 degrees of freedom to “spend”, we must make sure we choose variables that are not already fixed by the constraints.

Example Index 1 DAE Initialization: Consider the system:

$$\begin{aligned} \dot{x}_1 + x_2 &= g_1(t) \\ x_2 &= g_2(t) \end{aligned} \tag{11.6.6}$$

We need to specify the set $\{\dot{x}_1(t_0), x_1(t_0), x_2(t_0)\}$, totaling 3 values. We have 2 obvious constraints from the original equations at t_0 :

$$\dot{x}_1(t_0) + x_2(t_0) = g_1(t_0) \quad \text{and} \quad x_2(t_0) = g_2(t_0) \tag{11.6.7}$$

This leaves 1 degree of freedom (DoF). Note that we can check for implicit constraints by differentiating the algebraic equation: $\dot{x}_2(t_0) = \dot{g}_2(t_0)$. However, neither of the original equations relies on $\dot{x}_2$, so this does not constrain our choices for x_1 or $\dot{x}_1$.

Looking at the two original equations, we see that $x_2(t_0)$ and $\dot{x}_1(t_0)$ are already fully constrained. Thus, $x_1(t_0)$ is the only value we can freely choose with our one DoF.

Example Index 3 DAE Initialization: Consider the now-familiar index 3 system:

$$\begin{aligned} (1) \quad \dot{x}_2 &= x_1 \\ (2) \quad \dot{x}_3 &= x_2 \\ (3) \quad x_3 &= \gamma(t) \end{aligned} \tag{11.6.8}$$

Here, we must identify values for $\{\dot{x}_2, \dot{x}_3, x_2, x_3, x_1\}$ at t_0 . Because this is an index 3 problem, we know there are at least 2 implicit constraints. To find them, we look at the derivatives generated while obtaining the index, which are also the equations used when trying to reduce the index through the method of auxiliary variables. Differentiating the algebraic constraint (3) once gives an implicit constraint on x_2 when combined with (2):

$$\dot{x}_3 = \dot{\gamma}(t) \implies x_2(t_0) = \dot{\gamma}(t_0) \tag{11.6.9}$$

Differentiating again and combining with (1) gives an implicit constraint on x_1 :

$$\dot{x}_2 = \ddot{\gamma}(t) \implies x_1(t_0) = \ddot{\gamma}(t_0) \tag{11.6.10}$$

With five initial values to specify and three explicit equations, we start with $5 - 3 = 2$ degrees of freedom, which are then fully consumed by the two implicit constraints we uncovered. Consequently, the values

of $x_3(t_0)$, $x_2(t_0)$, and $x_1(t_0)$ are all fixed by the external forcing function $\gamma(t)$ and its derivatives. There are no degrees of freedom left for us to specify, despite the appearance of multiple (differential) variables.

Aside: Unusual Constraint Structures While the heuristic “index k has $k - 1$ implicit constraints” is fairly reliable, we should always check the specific system. It is possible (though uncommon) for an index 1 system to possess implicit constraints if the equations are structurally singular. For example, let’s look at the system

$$\begin{aligned}\dot{x}_1 + 2\dot{x}_2 &= x_1 - x_2 \\ \dot{x}_1 + 2\dot{x}_2 &= 3x_1 - 2x_2 - 1\end{aligned}\tag{11.6.11}$$

Subtracting the first equation from the second eliminates the derivatives entirely to uncover an implied algebraic constraint:

$$0 = 2x_1 - x_2 - 1 \implies 0 = 2\dot{x}_1 - \dot{x}_2\tag{11.6.12}$$

This system behaves as a DAE despite appearing to be a system of implicit ODEs.

Practical Comments on Solvers When using numerical software, consistent initialization is non-negotiable. For example, MATLAB’s `ode15i` (for fully implicit problems of the form $F(t, y, y') = 0$) requires the user to provide both $y(t_0)$ and $y'(t_0)$, and these must satisfy $F(t_0, y(t_0), y'(t_0)) = 0$. If the supplied values are inconsistent, the nonlinear solve at the initial step can fail (e.g., due to lack of convergence or step-size reduction). MATLAB therefore provides a helper function `decic` to compute consistent initial conditions close to user-provided guesses; `decic` can itself fail to converge, in which case consistent values must be determined by additional analysis of the model.

(Please see slides for the index 2 example showing some of these methods in practice; uploading this book without that example for now.)

11.7 Simulation of Mixed Continuous-Discrete Systems

Many real-world systems exhibit both continuous dynamics (described by DAEs) and discrete events (instantaneous changes in state or structure), such as when a valve becomes fully open or fully closed, a tank becomes full or runs dry, a safety interlock system kicks in, or the number of fluid phases changes in a tank. Discrete changes in the dynamic equations for the system can occur implicitly as well, and ignoring these changes can lead to incorrect results that arbitrarily deviate from reality.

If we know in advance when a discrete change is going to occur, such as a valve being opened, we can divide the full time interval of interest into sub-intervals where the system dynamics are continuous and solve each sub-interval separately, passing the final state of one interval as the initial condition for the next. Each of the sub-intervals can be treated as a standard DAE simulation problem.

In many cases, discrete events occur when certain conditions are met, such as a tank reaching a certain level or pressure, or a temperature exceeding a threshold. In these cases, we need to monitor the system during simulation to detect when these events occur. A simulator for mixed discrete-continuous simulations will continually track all possible events that can occur during the numerical simulation and, once an event has been detected, will iterate the time to determine the precise t^* at which the event has occurred. The simulator will then re-initialize the DAE system at t^* , applying any discrete changes to the system equations or states, and continue the simulation from that point onward.

11.8 Exercises

Exercise 11.1

What are the differential and algebraic states of:

$$(1) \dot{x}_1 + \dot{x}_2 = g_1(t)$$

$$(2) x_2 + x_3 = g_2(t)$$

$$(3) x_3 = g_3(t)$$

Exercise 11.2

Does the following index-2 DAE have implicit constraints?

$$(1) \dot{x}_1 - x_2 = 0$$

$$(2) x_1 = g_1(t)$$

Exercise 11.3

Identify the differential states, algebraic states, index, implicit constraints (if any), and a consistent initialization for the following DAE:

$$\begin{cases} \dot{x}_1 = y_1 - y_2 \\ y_1 = 1 \\ y_2 = \arctan(x_1) \end{cases}$$

Exercise 11.4

Identify the differential states, algebraic states, index, implicit constraints (if any), and a consistent initialization for the following DAE:

$$\begin{cases} \dot{x}_1 = y_1 - y_2 \\ y_1 = 2y_2 \\ x_1 = 1 + \sin(t) \end{cases}$$

Exercise 11.5

Identify the differential states, algebraic states, index, implicit constraints (if any), and a consistent initialization for the following DAE:

$$\begin{cases} \dot{x}_1 = y_1 - y_2 \\ \dot{x}_2 = y_2 \\ x_1 = 1 + \sin(t) \\ y_2 = e^{x_1} \end{cases}$$

Exercise 11.6

Identify the differential states, algebraic states, index, implicit constraints (if any), and a consistent initialization for the following DAE:

$$\begin{cases} \dot{x} = v \\ \dot{v} = a \\ \dot{a} = \sin(t) \end{cases}$$

Exercise 11.7

Identify the differential states, algebraic states, index, implicit constraints (if any), and a consistent initialization for the following DAE:

$$\begin{cases} a = -2x - 3v \\ \dot{x} = v \\ \dot{v} = a \end{cases}$$

Exercise 11.8

Identify the differential states, algebraic states, index, implicit constraints (if any), and a consistent initialization for the following DAE:

$$\begin{cases} v^2 + 2x = E \\ E = e^{-t} \\ \dot{x} = v \\ \dot{v} = a \end{cases}$$

Exercise 11.9

Identify the differential states, algebraic states, index, implicit constraints (if any), and a consistent initialization for the following DAE:

$$\begin{cases} x = 5t^2 \\ \dot{x} = v \\ \dot{v} = a \end{cases}$$

Exercise 11.10

Identify the differential states, algebraic states, index, implicit constraints (if any), and a consistent initialization for the following DAE:

$$\begin{cases} u_1^2 + u_2^2 = 1 \\ \dot{u}_1^2 + \dot{u}_2^2 = u_3 \\ 5u_2 + u_3 = 1 \end{cases}$$

Appendix A

Solutions to Exercises

Disclaimer: This part of the notes has been reviewed less rigorously than the main text, so it may contain errors. Students are welcome to report any errors to the course instructors or TAs.

Solution 1.1

- (a) False. Machine epsilon is the difference between 1 and the next largest representable number, and it bounds the relative error due to rounding. The smallest positive normalized double-precision number is $2^{-1022} \approx 2.23 \times 10^{-308}$, which is far smaller than $\epsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$.
- (b) False. There are only finitely many representable numbers, so most real numbers must be rounded to the nearest representable one.
- (c) False. This is the highest accuracy achievable with the double-precision representation of numbers, not a bug.
- (d) We cannot tell. Big-O notation describes only the asymptotic growth rate and hides the constant prefactor, which we do not know. For small N , or when the prefactor of the $O(N^2)$ algorithm is large, the $O(N^2 \log N)$ algorithm can be faster.
- (e) False. Changing the precision changes only the cost of each elementary operation, not the number of operations performed: the speedup is a constant factor (hardware-dependent, often around $2\times$ from the halved memory traffic), the same for every N . The time complexity class is unchanged.
- (f) False. `1e-20` itself is stored without difficulty. The rounding happens in the addition: the exact sum $1 + 10^{-20}$ falls between two representable numbers near 1, which are separated by about ϵ_{mach} , so the sum is rounded back to 1.

Solution 1.2

- (a) Each parameter takes 8, 4, and 2 bytes, respectively:

Format	Memory
float64	$70 \times 10^9 \times 8 \text{ B} = 560 \text{ GB}$
float32	280 GB
float16	140 GB

- (b) None of the three formats fits on a single 80 GB GPU; even in float16 the parameters occupy 140 GB, requiring at least two GPUs. This is why large models are distributed across multiple GPUs.

Solution 1.3

- (a) Replace the exact comparison with a test on the absolute error, e.g., $\text{abs}(\mathbf{s} - \mathbf{s_ref}) < 1\text{e-}10$. Here the computed sum is off by about 1.1×10^{-16} , so the modified test accepts the implementation. For quantities of magnitude around 1, absolute and relative error are of the same order, so an absolute tolerance works well.
- (b) No. Rounding error is relative to the magnitude of the numbers involved: for this array the computed sum deviates from the exact value by about 4×10^{-4} , which corresponds to a relative error of about 8×10^{-16} ; the absolute-tolerance test from (a) would reject a correct implementation. The more reasonable criterion is the relative error, e.g., $\text{abs}(\mathbf{s} - \mathbf{s_ref}) / \text{abs}(\mathbf{s_ref}) < 1\text{e-}10$, which is scale-independent.

Solution 1.4

- (a) $N^{0.01}$
- (b) N
- (c) 1.01^N
- (d) Neither
- (e) Neither

Solution 1.5

- (a) $O(N^2)$
- (b) $O(N^3)$
- (c) $O(N^3)$
- (d) $O(1)$
- (e) $O(N \log N)$
- (f) $O(2^N)$
- (g) $O(N^2)$
- (h) $O(N)$

Solution 1.6

- (a) $O(N^2 + N^3) = O(N^3)$. The two steps run one after the other, so their costs add, and the dominant term determines the overall complexity.
- (b) $O(N \cdot N^2) = O(N^3)$. Every one of the N iterations performs the full $O(N^2)$ work, so the costs multiply.
- (c) Costs *add* when the parts run sequentially, one after the other, each executed once. Costs *multiply* when the parts are nested, i.e., one part is executed once per unit of work of the other (e.g., a loop body inside a loop).

Solution 1.7

- (a) There are n entries on the main diagonal and $n - 1$ on each of the two adjacent diagonals, for a total of $3n - 2$ possibly nonzero entries. Storing only these requires $O(n)$ memory, compared to $O(n^2)$ for a dense matrix.
- (b) Each row has at most 3 nonzero entries, so each entry of the result requires at most 3 multiplications and 2 additions. The total cost is $O(n)$, compared to $O(n^2)$ for a dense matrix-vector multiplication.

- (c) We have three ‘for loops’, each of roughly length n , so the scaling is $O(n)$.
- (d) The matrix-vector multiplication is parallelizable: each entry of the result depends only on its own row, so all entries can be computed independently. The Thomas algorithm is not: the forward sweep is a recurrence (computing c'_i requires c'_{i-1}), which forces sequential execution. Parallel algorithms for tridiagonal systems do exist, however; a classical example is *cyclic reduction*, which solves the system in $O(\log n)$ parallel steps.

Solution 2.1

- (a) False. For example, $(1 + i)(1 - i) = 2$. In fact, $z\bar{z} = a^2 + b^2 = |z|^2$ is real for *every* complex number z ; this is how the magnitude of a complex number is defined.
- (b) False. It suffices that the dimensions are compatible both ways: for $\mathbf{A} \in \mathbb{R}^{2 \times 3}$ and $\mathbf{B} \in \mathbb{R}^{3 \times 2}$, the product $\mathbf{AB}$ is 2×2 and $\mathbf{BA}$ is 3×3 , yet neither matrix is square.
- (c) True. Although matrix multiplication is generally not commutative, diagonal matrices are an exception: both products are diagonal with entries $a_{ii}b_{ii}$.
- (d) False. The outer product is defined for any pair of vectors and gives an $m \times n$ matrix; only the inner product requires equal dimensions.
- (e) True. In general the transpose reverses the order, $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$, so the identity fails for most pairs; but it does hold in special cases, e.g., $\mathbf{A} = \mathbf{B} = \mathbf{I}$, or whenever $\mathbf{A}^\top$ and $\mathbf{B}^\top$ commute.
- (f) False. Take $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$ and $\mathbf{B} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$: then $\mathbf{AB} = \mathbf{0}$, so $\text{tr}(\mathbf{AB}) = 0$, yet $\text{tr}(\mathbf{A}) = \text{tr}(\mathbf{B}) = 1$.
- (g) True. The first claim follows from the general ordering $\|\mathbf{v}\|_2 \leq \|\mathbf{v}\|_1$: indeed, $\|\mathbf{v}\|_1^2 = (\sum_{i=1}^n |v_i|)^2 = \sum_{i=1}^n v_i^2 + \sum_{i \neq j} |v_i| |v_j| \geq \|\mathbf{v}\|_2^2$. The converse fails: for $\mathbf{v} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, we have $\|\mathbf{v}\|_2 = 1$ but $\|\mathbf{v}\|_1 = \sqrt{2} > 1$. (More generally, $\|\mathbf{v}\|_\infty \leq \|\mathbf{v}\|_2 \leq \|\mathbf{v}\|_1$ holds for every vector, while the reverse inequalities hold only up to dimension-dependent factors.)
- (h) False. The induced norm bounds the amplification for *every* vector: $\|\mathbf{Ax}\|_\infty \leq \|\mathbf{A}\|_\infty \|\mathbf{x}\|_\infty = 0.5 \|\mathbf{x}\|_\infty$. Choosing a larger $\mathbf{x}$ does not help: both sides scale linearly ($\|\mathbf{A}(c\mathbf{x})\|_\infty = |c| \|\mathbf{Ax}\|_\infty$), so the amplification ratio depends only on the direction of $\mathbf{x}$, not its magnitude.
- (i) True. If $\mathbf{b}$ is not in the column space, there is no solution. If a solution exists, then with more unknowns than equations ($n > m$), $\text{rank}(\mathbf{A}) \leq m < n$, so the nullity is positive and infinitely many solutions exist. A unique solution would require a trivial null space, which is impossible here.
- (j) False. Scaling the matrix scales *every* row, and each row contributes a factor: $\det(2\mathbf{A}) = 2^n \det(\mathbf{A})$. The statement holds only for $n = 1$.
- (k) False. If $\mathbf{b}$ lies in the column space of $\mathbf{A}$, the system has (infinitely many) solutions; a zero determinant rules out a *unique* solution for *every* $\mathbf{b}$, not the existence of a solution for a particular $\mathbf{b}$.
- (l) False. The set is not closed under either operation: the sum of two unit vectors is generally not a unit vector, a scalar multiple $c\mathbf{v}$ with $|c| \neq 1$ is not a unit vector, and the set does not contain $\mathbf{0}$.
- (m) False. The set does not contain $\mathbf{0}$ (since $\mathbf{A}\mathbf{0} = \mathbf{0} \neq \mathbf{b}$), and it is not closed under addition: if $\mathbf{Ax} = \mathbf{Ay} = \mathbf{b}$, then $\mathbf{A}(\mathbf{x} + \mathbf{y}) = 2\mathbf{b} \neq \mathbf{b}$. (For $\mathbf{b} = \mathbf{0}$, the solution set *is* a subspace: the null space.)
- (n) True. Closedness under the two operations is inherited: the smaller set is closed, and its vectors live in $\mathbb{R}^n$.
- (o) False. All three vectors can lie on a single line (e.g., three multiples of the same vector), in which case they span only that line.
- (p) False. At most two vectors in $\mathbb{R}^2$ can be linearly independent: a third is always a linear combination of an independent pair (the dimension of the space caps the count).

- (q) False as stated: the zero vector is orthogonal to every vector, so a set containing $\mathbf{0}$ is mutually orthogonal but linearly dependent. For *nonzero* mutually orthogonal vectors, the statement is true: taking the dot product of $\sum_i c_i \mathbf{v}_i = \mathbf{0}$ with $\mathbf{v}_j$ leaves $c_j \|\mathbf{v}_j\|_2^2 = 0$, so every $c_j = 0$.
- (r) True. Orthonormal columns of a square matrix mean $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$, so $\mathbf{Q}^{-1} = \mathbf{Q}^\top$; then $\mathbf{Q}\mathbf{Q}^\top = \mathbf{I}$ as well, which says exactly that the rows are orthonormal.
- (s) False. A trivial null space guarantees *uniqueness*, not *existence*. For a square matrix it implies both (invertibility), but for a tall matrix with more equations than unknowns, the columns cannot span the whole space, so there are right-hand sides $\mathbf{b}$ with no solution; see the example in the existence and uniqueness section.
- (t) False. Solvability depends on which subspace $\mathbf{b}$ lies in: $\mathbf{A}\mathbf{x} = \mathbf{b}$ requires $\mathbf{b}$ to be in the column space of $\mathbf{A}$, while $\mathbf{A}^\top \mathbf{x} = \mathbf{b}$ requires $\mathbf{b}$ to be in the column space of $\mathbf{A}^\top$ (the row space of $\mathbf{A}$), and these are different subspaces in general. For $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, the system $\mathbf{A}\mathbf{x} = \mathbf{b}$ is solved by $\mathbf{x} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, while $\mathbf{A}^\top \mathbf{x} = \begin{bmatrix} 0 \\ x_1 \end{bmatrix}$ can never equal $\mathbf{b}$.
- (u) True. Solvability for every $\mathbf{b}$ means $\text{rank}(\mathbf{A}) = n$, i.e., $\mathbf{A}$ is invertible; then $\mathbf{A}^\top$ is invertible as well (its rank equals that of $\mathbf{A}$), so the transposed system is solvable (in fact, uniquely) for every $\mathbf{c}$.
- (v) True. The two matrices have the same null space: if $\mathbf{A}\mathbf{x} = \mathbf{0}$, then $\mathbf{A}^\top \mathbf{A}\mathbf{x} = \mathbf{0}$; conversely, if $\mathbf{A}^\top \mathbf{A}\mathbf{x} = \mathbf{0}$, then $\mathbf{x}^\top \mathbf{A}^\top \mathbf{A}\mathbf{x} = \|\mathbf{A}\mathbf{x}\|_2^2 = 0$, so $\mathbf{A}\mathbf{x} = \mathbf{0}$. Both matrices have n columns, so equal nullity implies equal rank by the rank-nullity theorem.
- (w) False. The elimination itself needs to be performed only once, at cost $O(n^3)$; each additional right-hand side then requires only the substitution steps, at cost $O(n^2)$. The total is $O(n^3 + kn^2)$, much less than $O(n^3k)$.
- (x) False. The two are related through the condition number: from $\hat{\mathbf{x}} - \mathbf{x}^* = \mathbf{A}^{-1}(\mathbf{A}\hat{\mathbf{x}} - \mathbf{b})$ and $\|\mathbf{b}\| \leq \|\mathbf{A}\| \|\mathbf{x}^*\|$, we obtain $\|\hat{\mathbf{x}} - \mathbf{x}^*\|/\|\mathbf{x}^*\| \leq \kappa(\mathbf{A}) \|\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}\|/\|\mathbf{b}\|$, and this bound is attainable. So for an ill-conditioned matrix with $\kappa(\mathbf{A}) \geq 10^{10}$, the relative error can be of order one despite the tiny relative residual.
- (y) True on both counts. If eigenvectors with distinct eigenvalues were dependent, applying $\mathbf{A}$ to a shortest dependency relation and subtracting λ times it would produce a shorter one, a contradiction. With n distinct eigenvalues, the n independent eigenvectors form a complete set, so the matrix is diagonalizable.
- (z) False. This holds for normal matrices (e.g., symmetric ones), but not in general. For $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$, both eigenvalues are 0, while $\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$ gives singular values 1 and 0.
- (aa) True. Strict diagonal dominance guarantees that the spectral radius of the iteration matrix is less than 1, so the error contracts from *any* starting point. The initial guess affects only how many iterations are needed, not whether the method converges.

Solution 2.2

- (a) Comparing entries: $((\mathbf{AB})^\top)_{ij} = (\mathbf{AB})_{ji} = \sum_k a_{jk} b_{ki} = \sum_k (\mathbf{B}^\top)_{ik} (\mathbf{A}^\top)_{kj} = (\mathbf{B}^\top \mathbf{A}^\top)_{ij}$.
- (b) Writing out both traces as double sums: $\text{tr}(\mathbf{AB}) = \sum_{i=1}^m \sum_{k=1}^n a_{ik} b_{ki} = \sum_{k=1}^n \sum_{i=1}^m b_{ki} a_{ik} = \text{tr}(\mathbf{BA})$; the two expressions contain exactly the same terms, summed in a different order.
- (c) Apply the scalar triangle inequality entry by entry: $\|\mathbf{u} + \mathbf{v}\|_1 = \sum_i |u_i + v_i| \leq \sum_i (|u_i| + |v_i|) = \|\mathbf{u}\|_1 + \|\mathbf{v}\|_1$.

Solution 2.3

- (a) In the row view, each entry $b_i = \mathbf{a}_i^R \mathbf{x}$ requires at most k multiplications and $k - 1$ additions, so the total cost over the m rows is $O(mk)$, compared to $O(mn)$ for a dense matrix.
- (b) In the column view, $\mathbf{Ax} = \sum_{j=1}^n x_j \mathbf{a}_j^C$, and each column contributes at most k nonzero terms, so the total cost is $O(nk)$. The two viewpoints give the same kind of saving: in both cases the cost is proportional to the total number of nonzero entries of $\mathbf{A}$, regardless of how they are distributed.

Solution 2.4

- (a) The outer product $\mathbf{xy}^\top$ has mn entries $x_i y_j$; scaling by α and adding to $\mathbf{A}$ costs a constant number of operations per entry, so the total is $O(mn)$.
- (b) The product $\mathbf{AA}^\top$ dominates: as a generic matrix-matrix product ($p = m$), the cost is $O(m^2 n)$; the accumulation adds only $O(m^2)$. Since the update preserves symmetry ($(\mathbf{C} + \mathbf{AA}^\top)^\top = \mathbf{C} + \mathbf{AA}^\top$ when $\mathbf{C}$ is symmetric), only the $m(m+1)/2$ entries on and above the diagonal need to be computed, which halves the operation count—exactly what `syrk` exploits—although the complexity class remains $O(m^2 n)$.
- (c) Each column of $\mathbf{X}$ solves one triangular system $\mathbf{Lx}_j^C = \mathbf{b}_j^C$ by forward substitution, which (like back substitution) costs $O(n^2)$. With p right-hand sides, the total is $O(n^2 p)$.

Solution 2.5

- (a) Symmetrize a random matrix:

```
A = randn(n);
M = A + A';
```

The result is symmetric since $(\mathbf{A} + \mathbf{A}^\top)^\top = \mathbf{A}^\top + \mathbf{A}$.

- (b) One option is

```
A = randn(n);
M = A * A' + eye(n);
```

The matrix $\mathbf{M}$ is symmetric by the previous exercise, and it is positive definite since $\mathbf{x}^\top \mathbf{Mx} = \|\mathbf{A}^\top \mathbf{x}\|_2^2 + \|\mathbf{x}\|_2^2 > 0$ for every $\mathbf{x} \neq \mathbf{0}$. ($\mathbf{AA}^\top$ alone is positive definite whenever $\mathbf{A}$ is nonsingular, which for a random matrix happens with probability 1; adding the identity removes the need for this caveat.)

- (c) Make the factor rank-deficient by using fewer columns than rows, e.g.,

```
A = randn(n, n-1);
M = A * A';
```

Then $\mathbf{x}^\top \mathbf{Mx} = \|\mathbf{A}^\top \mathbf{x}\|_2^2 \geq 0$, so $\mathbf{M}$ is positive semi-definite, but $\text{rank}(\mathbf{M}) \leq n - 1 < n$, so there is a nonzero $\mathbf{x}$ with $\mathbf{Mx} = \mathbf{0}$ and hence $\mathbf{x}^\top \mathbf{Mx} = 0$: the matrix is not positive definite.

- (d) Multiply two random factors with inner dimension k :

```
M = randn(n, k) * randn(k, n);
```

The rank of a product is at most the rank of each factor, and each factor has rank at most k , so $\text{rank}(\mathbf{M}) \leq k$ (with probability 1, the rank is exactly k).

- (e) Multiply a random factor by its own transpose:

```
B = randn(n, k);
M = B * B';
```

The result is symmetric, $(\mathbf{B}\mathbf{B}^\top)^\top = \mathbf{B}\mathbf{B}^\top$, and its rank is at most $\text{rank}(\mathbf{B}) \leq k$.

Solution 2.6

The 1-norm is the sum of the absolute values of the components of the vector:

$$\|\mathbf{v}\|_1 = 8.$$

The 2-norm is the euclidean distance:

$$\|\mathbf{v}\|_2 = \left(\sum_{i=1}^5 |v_i|^2 \right)^{1/2} = (1 + 1 + 1 + 4 + 9)^{1/2} = \sqrt{16} = 4.$$

The ∞ -norm is the absolute value of the largest value in the vector:

$$\|\mathbf{v}\|_\infty = 3.$$

Solution 2.7

Start by picking the row or column with the most zeros to make things easier. I'll pick column 4.

$$\det \mathbf{A} = -(0)|\cdots| + (0)|\cdots| - (9) \begin{vmatrix} 1 & 0 & 2 \\ 2 & 7 & 0 \\ 0 & 1 & 2 \end{vmatrix} + (1) \begin{vmatrix} 1 & 0 & 2 \\ 2 & 7 & 0 \\ 4 & 0 & 0 \end{vmatrix}$$

I've left the minors that are multiplied by 0 blank since they don't contribute to the calculation. Now there are two 3×3 determinants left to find. Again we choose the row/column with the most zeros.

For the first one, I'll use column 3:

$$\begin{aligned} \begin{vmatrix} 1 & 0 & 2 \\ 2 & 7 & 0 \\ 0 & 1 & 2 \end{vmatrix} &= (2) \begin{vmatrix} 2 & 7 \\ 0 & 1 \end{vmatrix} - (0)|\cdots| + (2) \begin{vmatrix} 1 & 0 \\ 2 & 7 \end{vmatrix} \\ &= 2(2 \times 1 - 7 \times 0) + 0 + 2(1 \times 7 - 2 \times 0) \\ &= 4 + 14 = 18 \end{aligned}$$

For the second one, I'll also use column 3:

$$\begin{aligned} \begin{vmatrix} 1 & 0 & 2 \\ 2 & 7 & 0 \\ 4 & 0 & 0 \end{vmatrix} &= (2) \begin{vmatrix} 2 & 7 \\ 4 & 0 \end{vmatrix} - (0)|\cdots| + (0)|\cdots| \\ &= 2(2 \times 0 - 4 \times 7) = -56 \end{aligned}$$

Plugging these results back in,

$$\det \mathbf{A} = -9 \times 18 + 1 \times (-56) = -218.$$

Solution 2.8

The dimension of the column space of a matrix is equal to the rank, which is the number of nonzero rows or columns.

Using the following row operations, we get the echelon form of $\mathbf{A}$:

- $R_2 \leftarrow R_2 - 2R_1$
- $R_3 \leftarrow R_3 - R_1$
- $R_3 \leftarrow R_3 - R_2$

Echelon form of $\mathbf{A}$:

$$\tilde{\mathbf{A}} = \begin{bmatrix} 1 & 3 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The number of nonzero rows is 2 so the rank is 2.

$$\dim \mathcal{C}(\mathbf{A}) = r = 2$$

Note that there are other ways to find the dimension of the column space. You could also look at the columns of $\mathbf{A}$ and determine how many are linearly independent. This is feasible for this simple matrix. We can easily see that column 2 is $3 \times C_1$ and column 3 is $C_1 + C_4$. C_1 and C_4 are not linear combinations of each other. This means that only C_1 and C_4 are independent so the column space has dimension 2.

Solution 2.9

- (a) By the rank-nullity theorem, $\text{nullity}(\mathbf{A}) = 6 - \text{rank}(\mathbf{A}) = 2$.
- (b) Yes: $\text{rank}(\mathbf{A}) = 4$ equals the number of rows (full row rank), so the column space is all of $\mathbb{R}^4$ and every $\mathbf{b}$ is reachable. No solution is ever unique: the nullity is $2 > 0$, so any solution can be shifted by a nonzero null-space vector.
- (c) No: the columns of $\mathbf{B}$ span only a 3-dimensional subspace of $\mathbb{R}^5$, so most right-hand sides $\mathbf{c}$ are not reachable. When a solution does exist, it is unique: $\text{rank}(\mathbf{B}) = 3$ equals the number of columns (full column rank), so the nullity is 0 and the null space is trivial.

Solution 2.10

- (a) Uniqueness is guaranteed whenever a solution exists: by rank-nullity, the nullity is $n - n = 0$. Existence is not guaranteed when $m > n$: the columns span only an n -dimensional subspace of $\mathbb{R}^m$, so most right-hand sides are unreachable. (When $m = n$, the matrix is invertible and a unique solution exists for every $\mathbf{b}$.)
- (b) Only the trivial solution $\mathbf{x} = \mathbf{0}$, since the null space is trivial.
- (c) This $\mathbf{b}$ lies in the column space by construction, so a solution exists: $\mathbf{x} = (\alpha, 0, \dots, 0)^\top$. By (a), it is the unique one.

Solution 2.11

- (a) Differentiation maps $c_i x^i$ to $i c_i x^{i-1}$, so the coefficient of x^{i-1} in p' is $i c_i$: the matrix has the entries $1, 2, \dots, k$ on its superdiagonal and zeros elsewhere. For $k = 3$,

$$\mathbf{D} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

- (b) $\mathbf{D}^j$ represents the j -th derivative. $\mathbf{D}^{k+1} = \mathbf{0}$: differentiating a polynomial of degree at most k more than k times yields zero, so $\mathbf{D}$ is *nilpotent*.
- (c) No. $\mathbf{D}$ is singular: its first column is zero (the derivative of a constant is zero), so $\det(\mathbf{D}) = 0$ and the null space contains all constant polynomials. Differentiation destroys the constant term, and no matrix can recover it; this is the matrix version of the arbitrary constant of integration.
- (d) Yes, both. Each set contains the zero polynomial and is closed under addition and scalar multiplication (sums and multiples of even polynomials are even, and likewise for odd). In coefficient form, they are the subspaces with zero entries in the odd (respectively even) positions.
- (e) Yes. If $p(3) = 0$ and $q(3) = 0$, then $(p + q)(3) = 0$ and $(cp)(3) = 0$, so the set is closed under both operations (and it contains the zero polynomial).

Solution 2.12

- (a) The blank subtracts the component of the current column along an already-built direction:

```
function Q = gram_schmidt(A)
% Orthonormalize the columns of A.
[n, k] = size(A);
Q = zeros(n, k);
for j = 1:k
    v = A(:, j);
    for i = 1:j-1
        v = v - (Q(:, i))' * A(:, j)) * Q(:, i);
    end
    Q(:, j) = v / norm(v);
end
end
```

- (b) Processing column j requires $j - 1$ inner products and $j - 1$ vector updates, each costing $O(n)$, plus one $O(n)$ normalization. Summing over $j = 1, \dots, k$ gives $\sum_j O(nj) = O(nk^2)$.

Solution 2.13

The Gaussian elimination steps:

- $R_2 \leftarrow R_2 - \frac{1}{3}R_1$
- $R_3 \leftarrow R_3 - \frac{2}{3}R_1$
- $R_3 \leftarrow R_3 + R_2$

This results in:

$$\begin{bmatrix} 3 & 1 & 0 \\ 0 & 2/3 & -1 \\ 0 & 0 & 0 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 1 \\ 2/3 \\ 1 \end{bmatrix}.$$

The last row of the reduced coefficient matrix is all zeros. Writing this last row out in equation form we have:

$$0x_1 + 0x_2 + 0x_3 = 1.$$

There are no values of $\mathbf{x}$ that can make this true. **There is no solution to this system of equations.**

Solution 2.14

Reorder the unknowns and equations as $(x_2, x_3, x_4, x_5, x_1)$; the system becomes

$$\begin{bmatrix} 2 & 0 & 0 & 0 & 1 \\ 0 & 2 & 0 & 0 & 1 \\ 0 & 0 & 2 & 0 & 1 \\ 0 & 0 & 0 & 2 & 1 \\ 1 & 1 & 1 & 1 & 4 \end{bmatrix} \begin{bmatrix} x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \\ 3 \\ 3 \\ 8 \end{bmatrix},$$

with the arrow now pointing down-right. Eliminating with this ordering causes *no fill-in*: subtracting $\frac{1}{2}$ of each of the first four rows from the last row only touches the last row, giving

$$(4 - 4 \cdot \frac{1}{2})x_1 = 8 - 4 \cdot \frac{3}{2} \implies 2x_1 = 2 \implies x_1 = 1,$$

and back substitution gives $2x_i + x_1 = 3$, i.e., $x_i = 1$ for $i = 2, \dots, 5$. The solution is $\mathbf{x} = (1, 1, 1, 1, 1)$. The lesson: for sparse matrices, the *ordering* of the eliminations determines how much fill-in occurs, and a good reordering can reduce the work dramatically (here, to $O(n)$).

Solution 2.15

(a) The Thomas algorithm eliminates a single subdiagonal entry per row at constant cost, so its complexity is $O(n)$. (b) Standard Gaussian elimination does not exploit the sparsity and costs $O(n^3)$ —for large n , a steep price for ignoring that almost all entries are already zero.

Solution 2.16

- (a) The system is $(n + m) \times (n + m)$, so a dense direct solve costs $O((n + m)^3)$.
- (b) The first block row reads $\mathbf{D}\Delta\mathbf{x} + \mathbf{A}^\top\Delta\mathbf{y} = \mathbf{r}_1$; since $\mathbf{D}$ is diagonal with positive entries, it is invertible, and $\Delta\mathbf{x} = \mathbf{D}^{-1}(\mathbf{r}_1 - \mathbf{A}^\top\Delta\mathbf{y})$. Substituting into the second block row $\mathbf{A}\Delta\mathbf{x} = \mathbf{r}_2$ gives $\mathbf{A}\mathbf{D}^{-1}\mathbf{r}_1 - \mathbf{A}\mathbf{D}^{-1}\mathbf{A}^\top\Delta\mathbf{y} = \mathbf{r}_2$, which rearranges to the normal equations.
- (c) Applying $\mathbf{D}^{-1}$ costs $O(n)$ (it is diagonal); forming the $m \times m$ matrix $\mathbf{A}\mathbf{D}^{-1}\mathbf{A}^\top$ costs $O(m^2n)$; solving costs $O(m^3)$; and recovering $\Delta\mathbf{x}$ costs $O(mn)$. The total is $O(m^2n + m^3)$. When $m \ll n$, this is far cheaper than the direct solve of (a): the cost grows only linearly in the large dimension n , instead of cubically.

Solution 2.17

- (a) The square root of a non-positive number: the algorithm breaks down when $a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2 \leq 0$ (the subsequent divisions by l_{jj} are then impossible as well). It can be shown that the algorithm succeeds if and only if $\mathbf{A}$ is positive definite. For example, for $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$, the first column gives $l_{11} = 1$ and $l_{21} = 2$, but then $l_{22} = \sqrt{1 - 2^2} = \sqrt{-3}$, and the algorithm fails. (Indeed, the eigenvalues of this matrix are 3 and -1 , so it is not positive definite.)
- (b) $\mathbf{A}$ is invertible: using the multiplicative property, $\det(\mathbf{L}^\top) = \det(\mathbf{L})$, and the triangular-determinant property, $\det(\mathbf{A}) = \det(\mathbf{L})\det(\mathbf{L}^\top) = \left(\prod_{j=1}^n l_{jj}\right)^2 > 0$.
- (c) Since $\mathbf{L}$ is invertible, we have $\mathbf{x}^\top \mathbf{A} \mathbf{x} = \mathbf{x}^\top \mathbf{L} \mathbf{L}^\top \mathbf{x} = \|\mathbf{L}^\top \mathbf{x}\|_2^2 > 0$ for every $\mathbf{x} \neq \mathbf{0}$, which is exactly the definition of a positive definite matrix: $\mathbf{A}$ is positive definite. Thus, the eigenvalues are all positive: choosing $\mathbf{x}$ to be an eigenvector with $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$ gives $\mathbf{x}^\top \mathbf{A} \mathbf{x} = \lambda\|\mathbf{x}\|_2^2 > 0$, so $\lambda > 0$.
- (d) Simply attempt the factorization: since the algorithm succeeds if and only if the matrix is positive definite, success *is* the certificate, and failure (a non-positive square-root argument) shows the matrix is not positive definite. This is the standard practical test, and it is much cheaper than computing all eigenvalues.

- (e) The eigenvalues of $\mathbf{A} + \delta\mathbf{I}$ are $\lambda_i + \delta$, so the shifted matrix is positive definite as soon as $\delta > -\lambda_{\min}$. Using the test from (d): start with some $\delta_0 > 0$; attempt the factorization of $\mathbf{A} + \delta\mathbf{I}$; if it fails, double δ and try again, returning the first δ for which the factorization succeeds. The doubling exceeds $-\lambda_{\min}$ after finitely many attempts. (This trial-and-error strategy is used in practice, for example, to modify Hessians in Newton-type optimization methods.)

Solution 2.18

- (a) Find roots of $\det(\mathbf{A} - \lambda\mathbf{I})$:

$$\det \begin{bmatrix} 3-\lambda & 0 & 2 \\ 0 & 1-\lambda & 0 \\ -1 & 0 & 3-\lambda \end{bmatrix} = 0$$

Applying Laplace's expansion formula:

$$(3-\lambda) \det \begin{bmatrix} 1-\lambda & 0 \\ 0 & 3-\lambda \end{bmatrix} + (-1) \det \begin{bmatrix} 0 & 2 \\ 1-\lambda & 0 \end{bmatrix} = 0$$

$$(3-\lambda)^2(1-\lambda) + 2(1-\lambda) = 0$$

$$\Rightarrow \boxed{\begin{array}{l} \lambda_1 = 1 \\ \lambda_{2,3} = 3 \pm \sqrt{2}i \end{array}}$$

- (b) For $\lambda_1 = 1$: solving $(\mathbf{A} - \mathbf{I})\mathbf{v} = \mathbf{0}$ forces $v_1 = v_3 = 0$, so $\mathbf{v}_1 = (0, 1, 0)^\top$. For $\lambda_{2,3} = 3 \pm \sqrt{2}i$: the second row of $\mathbf{A} - \lambda\mathbf{I}$ forces $v_2 = 0$, and the first row, $\mp\sqrt{2}i v_1 + 2v_3 = 0$, gives $\mathbf{v}_{2,3} = (\sqrt{2}, 0, \pm i)^\top$. (One can check $\mathbf{A}\mathbf{v}_{2,3} = (3 \pm \sqrt{2}i)\mathbf{v}_{2,3}$.)
- (c) No: any nonzero scalar multiple of an eigenvector (including a complex multiple) is again an eigenvector for the same eigenvalue. Since all three eigenvalues here are distinct, each eigenvector is unique only *up to scaling*—the *eigendirections* are determined, the vectors are not.

Solution 2.19

- (a) Since $\mathbf{A}$ is triangular, $\det(\mathbf{A} - \lambda\mathbf{I}) = (1-\lambda)(2-\lambda)^2$, so the eigenvalues are 1 and 2 (the latter with algebraic multiplicity 2). There are three cases.

- $\lambda \notin \{1, 2\}$: $\det(\mathbf{A} - \lambda\mathbf{I}) \neq 0$, so $\mathbf{A} - \lambda\mathbf{I}$ is invertible and the rank is 3.

- $\lambda = 1$: $\mathbf{A} - \mathbf{I} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$ has two linearly independent rows, so the rank is 2.

- $\lambda = 2$: $\mathbf{A} - 2\mathbf{I} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$ also has rank 2.

Note that at $\lambda = 2$, the rank drops only by 1 even though the eigenvalue has algebraic multiplicity 2: the nullity of $\mathbf{A} - 2\mathbf{I}$ is the *geometric* multiplicity, which is 1 here, so the matrix is defective (not diagonalizable).

- (b) By definition, the geometric multiplicity is the dimension of the eigenspace, which is the nullity of $\mathbf{B} - \lambda\mathbf{I}$. By the rank-nullity theorem, $\text{rank}(\mathbf{B} - \lambda\mathbf{I}) = n - g$. The algebraic multiplicity does not enter; as (a) shows, it can exceed the rank drop.

Solution 2.20

Place the eigenvectors in the columns of $\mathbf{V}$ and the eigenvalues on the diagonal of $\mathbf{D}$. By the 2×2 inverse formula:

$$\mathbf{V} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix}, \quad \mathbf{V}^{-1} = \frac{1}{\det \mathbf{V}} \begin{bmatrix} -1 & -1 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix}.$$

Then

$$\mathbf{A} = (\mathbf{V}\mathbf{D})\mathbf{V}^{-1} = \begin{bmatrix} 0 & 2 \\ 0 & -2 \end{bmatrix} \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

One can check that $\mathbf{A}\mathbf{v}_1 = \mathbf{0}$ and $\mathbf{A}\mathbf{v}_2 = 2\mathbf{v}_2$.

Yes, the matrix is unique: the two eigenvectors are linearly independent, so they form a basis of $\mathbb{R}^2$, and the two eigenpair conditions specify $\mathbf{A}\mathbf{v}$ for every basis vector—which determines $\mathbf{A}\mathbf{v}$ for every $\mathbf{v}$ by linearity, and hence determines $\mathbf{A}$.

Solution 2.21

(a) $\mathbf{J}^2 = \begin{bmatrix} \lambda^2 & 2\lambda \\ 0 & \lambda^2 \end{bmatrix}$, $\mathbf{J}^3 = \begin{bmatrix} \lambda^3 & 3\lambda^2 \\ 0 & \lambda^3 \end{bmatrix}$, and in general

$$\mathbf{J}^k = \begin{bmatrix} \lambda^k & k\lambda^{k-1} \\ 0 & \lambda^k \end{bmatrix},$$

which follows by induction: the top-right entry of $\mathbf{J}^{k+1} = \mathbf{J}^k \mathbf{J}$ is $\lambda^k \cdot 1 + k\lambda^{k-1} \cdot \lambda = (k+1)\lambda^k$.

- (b) Yes. Both $\lambda^k \rightarrow 0$ and $k\lambda^{k-1} \rightarrow 0$: the exponential decay of λ^{k-1} dominates the linear growth of the factor k .
- (c) No. The off-diagonal entry has magnitude k , which grows without bound, even though no eigenvalue exceeds 1 in magnitude. A diagonalizable matrix with $|\lambda_i| \leq 1$ has bounded powers ($\mathbf{A}^k = \mathbf{W}\mathbf{\Lambda}^k\mathbf{W}^{-1}$ with bounded $\mathbf{\Lambda}^k$). The conclusion: for defective matrices, marginal eigenvalues ($|\lambda| = 1$) are not safe, and the convergence condition for iterative methods must be the *strict* inequality $\rho < 1$.

Solution 2.22

- (a) Singular values are the square roots of the eigenvalues of $\mathbf{A}^T \mathbf{A}$.

$\Rightarrow$ find solutions of $\det(\mathbf{A}^T \mathbf{A} - \lambda \mathbf{I}) = 0$

$$\det \begin{bmatrix} 10 - \lambda & 3 \\ 3 & 10 - \lambda \end{bmatrix} = 0 \iff (10 - \lambda)^2 - 9 = 0$$

$$\Rightarrow \begin{array}{l} \lambda_1 = 13 \\ \lambda_2 = 7 \end{array} \Rightarrow \boxed{\begin{array}{l} \sigma_1 = \sqrt{13} \\ \sigma_2 = \sqrt{7} \end{array}}$$

- (b) The matrix 2-norm is the largest singular value: $\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A}) = \sqrt{13}$.

- (c) The trace of $\mathbf{A}\mathbf{A}^T$ is the sum of the squared singular values: $\text{tr}(\mathbf{A}\mathbf{A}^T) = \sigma_1^2 + \sigma_2^2 = 13 + 7 = 20$. (Both $\mathbf{A}\mathbf{A}^T$ and $\mathbf{A}^T \mathbf{A}$ have the σ_i^2 as their nonzero eigenvalues, and the trace is the sum of the eigenvalues.)

Solution 2.23

Since $\mathbf{U}$ and $\mathbf{V}$ are orthogonal and $\mathbf{\Sigma} + \mathbf{I}$ is diagonal with decreasing non-negative entries $\sigma_i + 1$, the expression $\mathbf{U}(\mathbf{\Sigma} + \mathbf{I})\mathbf{V}^T$ is itself a valid singular value decomposition. The singular values of $\mathbf{B}$ are therefore $\sigma_i(\mathbf{A}) + 1$, and $\|\mathbf{B}\|_2 = \sigma_{\max}(\mathbf{B}) = \sigma_{\max}(\mathbf{A}) + 1$.

Solution 2.24

One example is the (not diagonally dominant) matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}.$$

For Jacobi, the iteration matrix is

$$-\mathbf{D}^{-1}\mathbf{R} = \begin{bmatrix} 0 & -2 \\ -2 & 0 \end{bmatrix},$$

with eigenvalues ± 2 , so the spectral radius is $2 > 1$. For Gauss-Seidel, the iteration matrix is

$$-(\mathbf{D} + \mathbf{L})^{-1}\mathbf{U} = \begin{bmatrix} 0 & -2 \\ 0 & 4 \end{bmatrix},$$

with eigenvalues 0 and 4, so the spectral radius is $4 > 1$. In both cases, the error is multiplied by the iteration matrix at every step, so any error component along an eigenvector with $|\lambda| > 1$ grows without bound: both methods diverge for a generic initial guess. (This is the same indefinite matrix for which the Cholesky factorization fails.)

Solution 2.25

- (a) Jacobi iteration: each equation is solved for its diagonal unknown using only values from the previous iterate.
- (b) $O(n)$: each of the n updates involves a constant number of operations (two multiplications, two subtractions, one division).
- (c) By the sufficient condition for Jacobi convergence, strict diagonal dominance suffices: $|a_{ii}| > |a_{i,i-1}| + |a_{i,i+1}|$ for every i .
- (d) The direct method wins on serial operation count, but the Jacobi sweep is fully parallelizable (all n updates are independent), while the Thomas recurrences are inherently sequential. With many processors, or with a good initial guess (for example, warm-starting from the solution of a nearby system) so that only a few sweeps are needed, the iteration can win in wall-clock time.

Solution 2.26

- (a) Powers pass through the eigendecomposition: $\mathbf{A}^2 = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}\mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1} = \mathbf{V}\mathbf{\Lambda}^2\mathbf{V}^{-1}$, so $p(\mathbf{A}) = \mathbf{V}(3\mathbf{I} - 3\mathbf{A} + \mathbf{A}^2)\mathbf{V}^{-1} = \mathbf{V}p(\mathbf{\Lambda})\mathbf{V}^{-1}$, and $\mathbf{A}^{-1} = \mathbf{V}\mathbf{\Lambda}^{-1}\mathbf{V}^{-1}$. Therefore $p(\mathbf{A}) - \mathbf{A}^{-1} = \mathbf{V}(p(\mathbf{\Lambda}) - \mathbf{\Lambda}^{-1})\mathbf{V}^{-1}$, whose eigenvalues are the diagonal entries $p(\lambda_i) - 1/\lambda_i = -(1 - \lambda_i)^3/\lambda_i$ by the scalar identity above. With $|1 - \lambda_i| \leq \rho < 1$, every eigenvalue of the error matrix is at most ρ^3/λ_i in magnitude: $p(\mathbf{A})$ is a good approximation of $\mathbf{A}^{-1}$ whenever all eigenvalues of $\mathbf{A}$ are close to 1.
- (b) MATLAB evaluates $\mathbf{A} * \mathbf{A} * \mathbf{v}$ from left to right, as $(\mathbf{A} * \mathbf{A}) * \mathbf{v}$: the matrix $\mathbf{A}^2$ is formed explicitly—a matrix-matrix product costing $O(n^3)$ —before it is applied to $\mathbf{v}$. The fix is to apply $\mathbf{A}$ to the vector twice, $\mathbf{A}^2\mathbf{v} = \mathbf{A}(\mathbf{A}\mathbf{v})$, so that only matrix-vector products, at $O(n^2)$ each, are performed:

```
function z = poly_precond(A, v)
% Apply z = p(A) * v for p(x) = 3 - 3x + x^2, using only
% matrix-vector products; the matrix p(A) is never formed.
Av = A * v;
AAv = A * Av;
z = 3 * v - 3 * Av + AAv;
end
```

- (c) Each step applies $p(\mathbf{A})$ to the current residual using the function from (b):

```

function x = precondition_solve(A, b, tol)
% Solve A x = b by the preconditioned iteration
% x <- x + p(A) (b - A x), applying p(A) with poly_precond.
x = zeros(size(b));
r = b;
while norm(r) > tol * norm(b)
    x = x + poly_precond(A, r);
    r = b - A * x;
end
end

```

The error satisfies $\mathbf{e}^{(k+1)} = (\mathbf{I} - p(\mathbf{A})\mathbf{A})\mathbf{e}^{(k)}$, and by part (a) the eigenvalues of $\mathbf{I} - p(\mathbf{A})\mathbf{A}$ are $(1 - \lambda_i)^3$. So if $|1 - \lambda_i| \leq \rho < 1$ for all i , the error contracts by a factor of at least ρ^3 per iteration—each step is worth three plain Richardson steps ($\omega = 1$), at the cost of three matrix-vector products per iteration: two inside $p(\mathbf{A})$ and one for the residual.

Solution 3.1

$$\mathbf{J}(\mathbf{x}) = \begin{bmatrix} \cos x_1 & e^{x_3} & x_2 e^{x_3} \\ x_2 x_3 & x_1 x_3 & x_1 x_2 \\ e^{-x_2} & -x_1 e^{-x_2} & -\sin x_3 \end{bmatrix}, \text{ arrived at by computing } J_{ij}(\mathbf{x}) = \frac{\partial f_i}{\partial x_j}(\mathbf{x}), \forall i, j \in \{1, 2, 3\}.$$

Solution 3.2

Linearization is given by $\hat{\mathbf{f}}(\mathbf{x}) = \mathbf{f}(\mathbf{x}_0) + \mathbf{J}(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0)$,

where $J_{ij}(\mathbf{x}) = \frac{\partial f_i}{\partial x_j}(\mathbf{x})$ with $f_i(\mathbf{x}) = (\mathbf{x}^T \mathbf{A} \mathbf{x}) \cdot x_i = \left(\sum_{m=1}^N \sum_{n=1}^N A_{mn} x_n x_m \right) x_i$.

$$\begin{aligned} \Rightarrow \frac{\partial f_i}{\partial x_j}(\mathbf{x}) &= (\mathbf{x}^T \mathbf{A} \mathbf{x}) \frac{\partial x_i}{\partial x_j} + x_i \cdot \frac{\partial}{\partial x_j} (\mathbf{x}^T \mathbf{A} \mathbf{x}) \\ &= \begin{cases} \mathbf{x}^T \mathbf{A} \mathbf{x} + x_i \cdot \mathbf{x}^T (\mathbf{a}_j^R + \mathbf{a}_j^C), & i = j \\ x_i \cdot \mathbf{x}^T (\mathbf{a}_j^R + \mathbf{a}_j^C), & i \neq j \end{cases} \end{aligned}$$

where $\mathbf{a}_j^R$ and $\mathbf{a}_j^C$ are the j -th row and column of $\mathbf{A}$ as column vectors, respectively

$$\begin{aligned} \Rightarrow \mathbf{J}(\mathbf{x}) &= (\mathbf{x}^T \mathbf{A} \mathbf{x}) \cdot \mathbf{I} + \begin{bmatrix} x_1 \cdot \mathbf{x}^T (\mathbf{A} + \mathbf{A}^T) \\ x_2 \cdot \mathbf{x}^T (\mathbf{A} + \mathbf{A}^T) \\ \vdots \\ x_N \cdot \mathbf{x}^T (\mathbf{A} + \mathbf{A}^T) \end{bmatrix} \\ &= (\mathbf{x}^T \mathbf{A} \mathbf{x}) \cdot \mathbf{I} + \mathbf{x} \mathbf{x}^T (\mathbf{A} + \mathbf{A}^T) \end{aligned}$$

(“ $\cdot$ ” is scalar multiplication here, not a dot product!)

$$\text{Overall: } \hat{\mathbf{f}}(\mathbf{x}) = (\mathbf{x}_0^T \mathbf{A} \mathbf{x}_0) \mathbf{x}_0 + \left[(\mathbf{x}_0^T \mathbf{A} \mathbf{x}_0) \mathbf{I} + \mathbf{x}_0 \mathbf{x}_0^T (\mathbf{A} + \mathbf{A}^T) \right] (\mathbf{x} - \mathbf{x}_0)$$

Solution 4.1

- (a) False. The Hessian reflects the local curvature of the function and can be indefinite or negative definite. For example, $f(x) = -x^2$ has Hessian $f''(x) = -2 < 0$ everywhere, and $f(\mathbf{x}) = x_1^2 - x_2^2$ has Hessian

$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix}, \text{ which is indefinite.}$$

Side note. The Hessian is positive semi-definite for all $\mathbf{x}$ if and only if f is convex; for a general function, positive semi-definiteness is only guaranteed at a local minimum (the second-order necessary condition).

- (b) False. Negating the objective is correct: maximizing f is equivalent to minimizing $-f$, and the maximizer coincides with the minimizer. However, the constraint must be left unchanged, since the feasible set must stay the same. Negating the constraint to $-c(\mathbf{x}) \leq 0$ (i.e., $c(\mathbf{x}) \geq 0$) generally defines a different feasible set. The correct equivalent problem is

$$\min_{\mathbf{x}} -f(\mathbf{x}) \quad \text{subject to} \quad c(\mathbf{x}) \leq 0.$$

- (c) True. Since $\|\mathbf{F}(\mathbf{x})\|_2 \geq 0$ for all $\mathbf{x}$ and the map $t \mapsto t^2$ is strictly increasing on $[0, \infty)$, we have $\|\mathbf{F}(\mathbf{x}_1)\|_2 \leq \|\mathbf{F}(\mathbf{x}_2)\|_2$ if and only if $\|\mathbf{F}(\mathbf{x}_1)\|_2^2 \leq \|\mathbf{F}(\mathbf{x}_2)\|_2^2$, so the two objectives order the points of $\mathbb{R}^N$ identically and have the same minimizers (local and global).

Side note. The two formulations are not equally convenient numerically, however: $\|\mathbf{F}(\mathbf{x})\|_2 = \sqrt{\sum_{i=1}^M F_i(\mathbf{x})^2}$ is not differentiable at points where $\mathbf{F}(\mathbf{x}) = \mathbf{0}$, which is exactly where the solution of a well-fit problem lies, whereas the squared objective is smooth with gradient $2\nabla\mathbf{F}(\mathbf{x})^\top \mathbf{F}(\mathbf{x})$. This is why least-squares problems are always posed with the square.

- (d) False. A global minimum is itself a local minimum, so the two can coincide; for example, for a convex function, any local minimum is also a global minimum. In general, the objective value at a global minimum is less than or equal to the value at any local minimum, and ties are possible even between distinct points (e.g., $f(x) = \sin x$ attains the same value at every one of its local minima, each of which is a global minimum).
- (e) False. An optimization problem may have no local optimum at all. For example, $\min_{x \in \mathbb{R}} x$ is unbounded below and has no minimum, and $\min_x e^x$ is bounded below (by 0) but the infimum is never attained, so no local (or global) minimum exists. A constrained problem can also be infeasible, in which case there is no optimum either.
- (f) False. A point can be a local maximum and a local minimum at the same time: the definitions require $f(\bar{\mathbf{x}}) \geq f(\mathbf{x})$ (respectively $f(\bar{\mathbf{x}}) \leq f(\mathbf{x})$) for all $\mathbf{x}$ in a neighborhood of $\bar{\mathbf{x}}$, and both can hold simultaneously if f is constant near $\bar{\mathbf{x}}$. For example, for a constant function $f(\mathbf{x}) = 1$, every point is both a local maximum and a local minimum (indeed, a global maximum and a global minimum).
- (g) False. Take the two unit disks $A = \{\mathbf{x} \in \mathbb{R}^2 : \|\mathbf{x} - (-2, 0)^\top\|_2 \leq 1\}$ and $B = \{\mathbf{x} \in \mathbb{R}^2 : \|\mathbf{x} - (2, 0)^\top\|_2 \leq 1\}$, each of which is convex. The points $(-2, 0)^\top \in A$ and $(2, 0)^\top \in B$ both lie in $A \cup B$, but their midpoint $(0, 0)^\top$ lies in neither, so the union is not convex.

Side note. A union of convex sets is convex only in special cases, such as when one of the sets contains the other.

- (h) True. Let $\mathbf{x}_1, \mathbf{x}_2 \in A \cap B$ and $\lambda \in [0, 1]$. Since $\mathbf{x}_1, \mathbf{x}_2 \in A$ and A is convex, $\lambda\mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \in A$; the same argument applied to B gives $\lambda\mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \in B$. Hence the point lies in $A \cap B$, which is therefore convex.

Side note. The argument never uses that there are only two sets, so an intersection of *any* number of convex sets is convex. This is the reason the feasible set of a convex optimization problem is convex: it is the intersection of the sets defined by the individual constraints, one per constraint.

- (i) True. For convex g and h and any $\mathbf{x}_1, \mathbf{x}_2$ and $\lambda \in [0, 1]$, applying the definition of convexity to g and h separately and adding the two inequalities shows that $g + h$ satisfies the definition of convexity.
- (j) False. For example, $g(x) = x^2$ and $h(x) = (x - 1)^2$ are both convex, but their product is

$$p(x) = x^2(x - 1)^2 = x^4 - 2x^3 + x^2, \quad p''(x) = 12x^2 - 12x + 2.$$

At $x = 1/2$ we get $p''(1/2) = 3 - 6 + 2 = -1 < 0$, so the second-order characterization of convexity fails and p is not convex.

- (k) False. A convex optimization problem requires the feasible set to be convex, and the feasible set $\{\mathbf{x} : c(\mathbf{x}) = 0\}$ of an equality constraint need not be convex even when c is a convex function. For example, $c(\mathbf{x}) = x_1^2 + x_2^2 - 2$ is convex, but the feasible set $\{\mathbf{x} : x_1^2 + x_2^2 = 2\}$ is a circle, which is not convex (the segment between the feasible points $(1, 1)^\top$ and $(-1, -1)^\top$ leaves the set).

Side note. This is why convex optimization problems require equality constraints to be *linear (affine)*: then the feasible set is an affine set (e.g., a hyperplane), which is convex.

- (l) False. If the step size is too large relative to the curvature of the function, a gradient descent step can overshoot and *increase* the objective. For example, for $f(x) = x^2$ with step size $\alpha = 1.1$ and $x_0 = 1$, the update $x_1 = x_0 - \alpha f'(x_0) = 1 - 2.2 = -1.2$ gives $f(x_1) = 1.44 > f(x_0) = 1$, and the iterates diverge.

Side note. Descent is guaranteed only when the step size is sufficiently small relative to the curvature; this is why practical implementations use a line search (e.g., backtracking with a sufficient-decrease condition) rather than a fixed step size.

- (m) False. The optimal step size α^* minimizes only the *one-dimensional restriction* $\phi(\alpha) = f(\mathbf{x}_k + \alpha \mathbf{d}_k)$ of f along the search direction, not f itself. The optimality condition for the line search is

$$\phi'(\alpha^*) = \nabla f(\mathbf{x}_k + \alpha^* \mathbf{d}_k)^\top \mathbf{d}_k = 0,$$

which says that the gradient at the new point is *orthogonal* to the search direction, not that it is zero. Unless the gradient happens to vanish there, the new point is not a stationary point of f , and hence not a local minimum.

Side note. For example, for $f(\mathbf{x}) = x_1^2 + 10x_2^2$, an exact line search from a generic starting point does not land on the minimizer; the gradient at the new point is nonzero (and orthogonal to the previous direction), which is what produces the characteristic zig-zag path of steepest descent.

- (n) False, Newton's method for optimization requires the derivative of the gradient, but for the max function, there are points where the second derivative isn't defined.
- (o) False. For small α_k , the change in the objective is governed by the directional derivative

$$\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k = -\nabla f(\mathbf{x}_k)^\top (\nabla^2 f(\mathbf{x}_k))^{-1} \nabla f(\mathbf{x}_k),$$

which is guaranteed to be negative only when the Hessian is positive definite. If the Hessian is not positive definite, the step $\mathbf{d}_k$ may be an *ascent* direction, and even an arbitrarily small step increases f . For example, for $f(x) = -x^2$ at $x_k = 1$: $f'(x_k) = -2$, $f''(x_k) = -2$, so $d_k = -(-2)/(-2) = -1$ and $f'(x_k) d_k = 2 > 0$; every step size $\alpha_k \in (0, 1)$ *increases* f , as the iterate moves toward the maximizer at $x = 0$.

Side note. This reflects that this step seeks stationary points, which may be maxima or saddle points.

- (p) True. Since $\mathbf{H}_k \succ \mathbf{0}$, we have

$$\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k = -\nabla f(\mathbf{x}_k)^\top \mathbf{H}_k^{-1} \nabla f(\mathbf{x}_k) < 0,$$

so $\mathbf{d}_k$ is always a descent direction. The backtracking line search then terminates with a step size $\alpha_k > 0$ satisfying the sufficient-decrease condition, which guarantees $f(\mathbf{x}_{k+1}) < f(\mathbf{x}_k)$.

Side note. This is a key advantage of BFGS: it maintains a positive definite Hessian approximation by construction.

- (q) True, BFGS maintains a positive definite approximation of the Hessian by construction.
- (r) False. The sampled gradient ∇f_{j_k} is a *noisy* estimate of the full gradient ∇f : even at a minimizer $\mathbf{x}^*$ of f (where $\nabla f(\mathbf{x}^*) = \mathbf{0}$), the individual terms generally have $\nabla f_{j_k}(\mathbf{x}^*) \neq \mathbf{0}$. With a fixed step size, the iterates therefore do not converge but fluctuate in a neighborhood of $\mathbf{x}^*$ whose size scales with the step size; convergence requires diminishing step sizes, and the resulting rate is *sublinear* — slower than the linear rate of full-gradient steepest descent.

Side note. The appeal of stochastic gradient descent is not its convergence rate but its per-iteration cost: each step touches only one term instead of all M .

- (s) False. Newton's quadratic convergence is a property of the deterministic Newton method applied to the objective being minimized, near a nondegenerate minimizer. The step shown is a Newton step for the single sampled term f_{j_k} : it drives the iterate toward a stationary point of f_{j_k} , not of the sum f . Since the minimizers of the individual terms generally differ from the minimizer of f , the iteration hops between the different terms' targets and, in general, does not converge to a minimizer of f at all, let alone quadratically.

Side note. Using the curvature of a single term does not remove the sampling noise; the noise, not the lack of curvature information, is what limits the convergence of stochastic methods.

Solution 4.2

The gradient is the vector of partial derivatives with respect to each variable:

$$\mathbf{g}(\mathbf{x}) = \begin{bmatrix} -2x_1 e^{-x_1^2} \sin x_2 \\ e^{-x_1^2} \cos x_2 \\ \frac{1}{x_3} \end{bmatrix}.$$

The Hessian is the matrix of second-order partial derivatives with respect to each pair of variables:

$$\mathbf{H}(\mathbf{x}) = \begin{bmatrix} (4x_1^2 - 2)e^{-x_1^2} \sin x_2 & -2x_1 e^{-x_1^2} \cos x_2 & 0 \\ -2x_1 e^{-x_1^2} \cos x_2 & -e^{-x_1^2} \sin x_2 & 0 \\ 0 & 0 & -\frac{1}{x_3^2} \end{bmatrix}.$$

Solution 4.3

- (a) The Hessian is *symmetric*: for a twice continuously differentiable function the mixed partials are equal, $\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i}$, so $\nabla^2 f(\mathbf{x}) = \nabla^2 f(\mathbf{x})^\top$. Every off-diagonal entry therefore appears twice, and it is enough to store the diagonal together with one triangle:

$$N + \frac{N(N-1)}{2} = \frac{N(N+1)}{2} \quad \text{entries, instead of } N^2.$$

- (b) No. Expanding, $\frac{N(N+1)}{2} = \frac{1}{2}N^2 + \frac{1}{2}N$, which is still $O(N^2)$: big-O describes the growth rate and discards both the constant factor $\frac{1}{2}$ and the lower-order term. The saving is real, roughly halving the memory, but it does not change how the requirement scales with N .

Side note. Changing the complexity itself requires structure, not symmetry. In Exercise 4.5 the Hessian has only $O(N)$ nonzero entries, and storing just those brings the requirement down to $O(N)$.

Solution 4.4

The second-order Taylor expansion about $\mathbf{x}^*$ is

$$f(\mathbf{x}) = f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*)^\top (\mathbf{x} - \mathbf{x}^*) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^*)^\top \nabla^2 f(\mathbf{x}^*) (\mathbf{x} - \mathbf{x}^*) + O(\|\mathbf{x} - \mathbf{x}^*\|^3).$$

The gradient and Hessian are

$$\nabla f(\mathbf{x}) = \begin{bmatrix} 2x_1 x_2 \\ x_1^2 + e^{x_2} \end{bmatrix}, \quad \nabla^2 f(\mathbf{x}) = \begin{bmatrix} 2x_2 & 2x_1 \\ 2x_1 & e^{x_2} \end{bmatrix}.$$

Evaluating at $\mathbf{x}^* = (1, 0)^\top$ gives $f(\mathbf{x}^*) = 1$,

$$\nabla f(\mathbf{x}^*) = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \quad \nabla^2 f(\mathbf{x}^*) = \begin{bmatrix} 0 & 2 \\ 2 & 1 \end{bmatrix}.$$

Therefore,

$$f(\mathbf{x}) = 1 + 2x_2 + 2(x_1 - 1)x_2 + \frac{1}{2}x_2^2 + O(\|\mathbf{x} - \mathbf{x}^*\|^3).$$

Solution 4.5

- (a) Each term $(x_{i+1} - x_i)^2$ involves only the pair (x_i, x_{i+1}) : it contributes 2 to the diagonal entries H_{ii} and $H_{i+1, i+1}$ and -2 to the off-diagonal entries $H_{i, i+1} = H_{i+1, i}$. Each term e^{x_i} contributes e^{x_i} to H_{ii} . The Hessian is therefore tridiagonal:

$$H_{ij} = \begin{cases} 2 + e^{x_i} & i = j \in \{1, N\}, \\ 4 + e^{x_i} & i = j \in \{2, \dots, N-1\}, \\ -2 & |i - j| = 1, \\ 0 & \text{otherwise.} \end{cases}$$

- (b) The nonzero entries are the N diagonal entries and the $2(N-1)$ entries on the first sub- and super-diagonals, for a total of $3N-2$. Storing the Hessian in a sparse format therefore requires $O(N)$ storage, compared to $O(N^2)$ for a dense matrix.

Side note. Because f is a sum of terms that each involve only a few variables, the Hessian is sparse, and exploiting this sparsity is what makes large-scale problems (large N) tractable in practice.

Solution 4.6

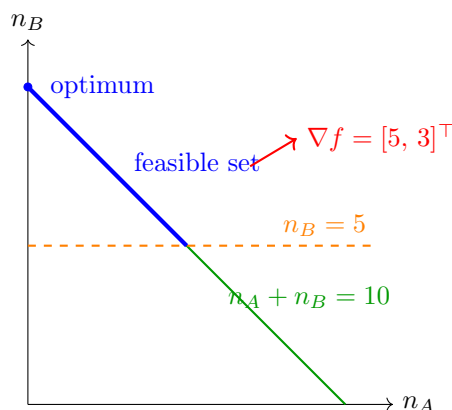
Let n_A and n_B denote the moles of species A and B respectively.

- (a) The unconstrained problem is:

$$\min_{n_A, n_B} f(n) = 5n_A + 3n_B$$

No solution exists. Suppose (n_A^*, n_B^*) were a solution; then $(n_A^* - 1, n_B^*)$ is also feasible (there are no constraints—not even nonnegativity) and has a strictly lower objective, contradicting optimality. The objective can thus be decreased without bound.

- (b) B costs the least, so we want to maximize it. The maximum value of n_B given the equality constraint is $n_B = 10$, so the optimum is $n_A = 0$, $n_B = 10$. The equality constraint is active, but the inequality constraint $n_B \geq 5$ is inactive. In the sketch below, the feasible set is the segment of the line $n_A + n_B = 10$ with $n_B \geq 5$; since f decreases in the direction $-\nabla f$, the solution is the endpoint of the segment farthest in that direction, $(n_A, n_B) = (0, 10)$.



- (c) With the constraint $n_A \geq 5$ and the equality $n_A + n_B = 10$, we have $n_B \leq 5$. Now the objective is minimized at $n_B = 5$, giving $n_A = 5$. Both the equality constraint and the inequality constraint $n_A \geq 5$ are active at the optimum.

Solution 4.7

Let v_1 and v_2 denote the outlet flow rates. The constraints are $v_1 + v_2 = 3$ and $v_1 = 2v_2$.

- (a) We can formulate this as minimizing the squared error:

$$\min_v \left\| \begin{bmatrix} v_1 + v_2 - 3 \\ v_1 - 2v_2 \end{bmatrix} \right\|_2^2$$

This is equivalent to minimizing the sum of squared constraint violations.

- (b) Yes. The constraints form a square linear system with a unique solution:

$$\begin{bmatrix} 1 & 1 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix} \Rightarrow v_1 = 2 \text{ L/s}, v_2 = 1 \text{ L/s}.$$

The error function is zero exactly at this solution and positive everywhere else, so the minimizer of the error norm coincides with the solution of the linear system.

Solution 4.8

- (a) LP: the objective and all constraints are linear, and all variables are continuous.
 (b) NLP: the variables are continuous, but the objective and the constraint are nonlinear (quadratic).
 (c) MILP: the objective and constraints are linear, and the problem mixes continuous variables (x_1, x_2) with a binary variable (y).
 (d) MINLP: the objective is nonlinear (quadratic in x), and the problem mixes a continuous variable (x) with a binary variable (y).

Solution 4.9

- (a) Not convex. This is the unit circle (only the boundary, not the disk): the segment between $(1, 0)$ and $(-1, 0)$ passes through the origin, which is not in the set.
 (b) Convex. This is the region above the parabola $x_2 = x_1^2$. Since x_1^2 is a convex function, for any two points in the set the line segment between them stays above the parabola. (In general, the region above the graph of a convex function, its *epigraph*, is a convex set.)
 (c) Not convex. The set is $(-\infty, -1] \cup [1, \infty)$: the segment between -1 and 1 leaves the set (e.g., 0 is not in the set).
 (d) Not convex. $f''(x) = 6x < 0$ for $x < 0$, so the second-order condition fails on part of the domain (f is convex only when restricted to $x \geq 0$).
 (e) Not convex. The Hessian is $\nabla^2 f(\mathbf{x}) = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, with eigenvalues ± 1 , so it is indefinite. For example, along the line $x_2 = -x_1$, $f(\mathbf{x}) = -x_1^2$ is concave.
 (f) Convex. For any $\mathbf{x}_1, \mathbf{x}_2$ and $\lambda \in [0, 1]$,

$$f(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) = \max\{g(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2), h(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2)\} \leq \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2),$$

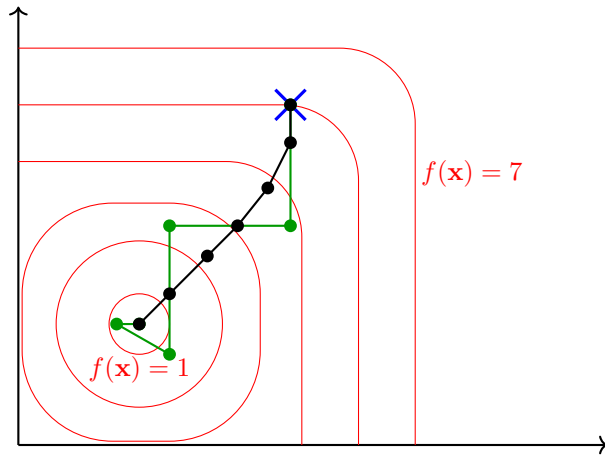
where the inequality uses convexity of g and h together with $g \leq f$ and $h \leq f$ at the endpoints. The pointwise maximum of convex functions is convex.

Solution 4.10

- (a) Local minimum. $\nabla f(\mathbf{x}) = (2x_1 - 2, 8x_2)^\top = \mathbf{0}$ at $\bar{\mathbf{x}} = (1, 0)^\top$, so $\bar{\mathbf{x}}$ is a stationary point. The Hessian $\nabla^2 f = \begin{bmatrix} 2 & 0 \\ 0 & 8 \end{bmatrix}$ is positive definite, so $\bar{\mathbf{x}}$ is a (strict) local minimum.
- (b) Local maximum. $\nabla f(\mathbf{x}) = (-2x_1 + 2x_2, 2x_1 - 4x_2)^\top = \mathbf{0}$ at $\bar{\mathbf{x}} = (0, 0)^\top$. The Hessian $\nabla^2 f = \begin{bmatrix} -2 & 2 \\ 2 & -4 \end{bmatrix}$ has negative trace (-6) and positive determinant $(8 - 4 = 4)$, so both eigenvalues are negative: the Hessian is negative definite and $\bar{\mathbf{x}}$ is a (strict) local maximum.
- (c) Saddle point. $\nabla f(\mathbf{x}) = (2x_1 + 4x_2, 4x_1 + 2x_2)^\top = \mathbf{0}$ at $\bar{\mathbf{x}} = (0, 0)^\top$. The Hessian $\nabla^2 f = \begin{bmatrix} 2 & 4 \\ 4 & 2 \end{bmatrix}$ has eigenvalues 6 and -2 (determinant $4 - 16 = -12 < 0$), so it is indefinite: $\bar{\mathbf{x}}$ is a saddle point (f increases along $(1, 1)^\top$ and decreases along $(1, -1)^\top$).
- (d) None of these. $\nabla f(\mathbf{x}) = (2x_1, 2x_2 - 2)^\top$, so $\nabla f(\bar{\mathbf{x}}) = (0, -2)^\top \neq \mathbf{0}$: $\bar{\mathbf{x}}$ is not a stationary point, and therefore cannot be a local minimum, local maximum, or saddle point. (The only stationary point of this function is $(0, 1)^\top$.)

Solution 4.11

Answers will vary. In general, you would expect a small step size (black) to propagate more slowly but be more stable, whereas you would expect a large step size (green) to propagate more quickly but more erratically. Steps should follow the gradient, which is perpendicular to the contours.

**Solution 4.12**

- (a) With $d = -\nabla f(\mathbf{x}_i)$, the model becomes:

$$m(\alpha) = f(\mathbf{x}_i) - \alpha \|\nabla f(\mathbf{x}_i)\|_2^2 + \frac{1}{2} \alpha^2 \nabla f(\mathbf{x}_i)^\top \nabla^2 f(\mathbf{x}_i) \nabla f(\mathbf{x}_i)$$

Minimizing with respect to α :

$$\frac{dm}{d\alpha} = -\|\nabla f(\mathbf{x}_i)\|_2^2 + \alpha \nabla f(\mathbf{x}_i)^\top \nabla^2 f(\mathbf{x}_i) \nabla f(\mathbf{x}_i) = 0$$

Thus:

$$\alpha_{\text{opt}} = \frac{\nabla f(\mathbf{x}_i)^\top \nabla f(\mathbf{x}_i)}{\nabla f(\mathbf{x}_i)^\top \nabla^2 f(\mathbf{x}_i) \nabla f(\mathbf{x}_i)} = \frac{\|\nabla f(\mathbf{x}_i)\|_2^2}{\nabla f(\mathbf{x}_i)^\top \nabla^2 f(\mathbf{x}_i) \nabla f(\mathbf{x}_i)}$$

- (b) With $d = -\mathbf{H}^{-1}\mathbf{g}$ where $\mathbf{g} = \nabla f(\mathbf{x}_i)$ and $\mathbf{H} = \nabla^2 f(\mathbf{x}_i)$:

$$m(\alpha) = f(\mathbf{x}_i) - \alpha \mathbf{g}^\top \mathbf{H}^{-1} \mathbf{g} + \frac{1}{2} \alpha^2 \mathbf{g}^\top (\mathbf{H}^{-1})^\top \mathbf{H} \mathbf{H}^{-1} \mathbf{g}$$

Since $\mathbf{H}$ is symmetric, $(\mathbf{H}^{-1})^\top = \mathbf{H}^{-1}$, so:

$$m(\alpha) = f(\mathbf{x}_i) - \alpha \mathbf{g}^\top \mathbf{H}^{-1} \mathbf{g} + \frac{1}{2} \alpha^2 \mathbf{g}^\top \mathbf{H}^{-1} \mathbf{g}$$

Minimizing:

$$\frac{dm}{d\alpha} = -\mathbf{g}^\top \mathbf{H}^{-1} \mathbf{g} + \alpha \mathbf{g}^\top \mathbf{H}^{-1} \mathbf{g} = 0$$

Thus $\alpha_{\text{opt}} = 1$. The Newton step is already optimal to the quadratic approximation when $\mathbf{H} \succ 0$.

Solution 4.13

(a) The gradient is $\nabla f(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}$, so gradient descent with fixed step size ω reads

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \omega \nabla f(\mathbf{x}_k) = \mathbf{x}_k - \omega(\mathbf{Ax}_k - \mathbf{b}) = \mathbf{x}_k + \omega(\mathbf{b} - \mathbf{Ax}_k),$$

which is exactly the Richardson iteration with relaxation parameter ω . Because $\mathbf{A} \succ \mathbf{0}$, the function f is strictly convex and its unique minimizer is the stationary point $\nabla f(\mathbf{x}^*) = \mathbf{0}$, i.e., the solution of

$$\mathbf{Ax}^* = \mathbf{b}.$$

Minimizing a convex quadratic and solving a symmetric positive definite linear system are therefore the same problem, and gradient descent on the former is the Richardson method on the latter.

(b) Since $\mathbf{Ax}^* = \mathbf{b}$, the minimizer is a fixed point: $\mathbf{x}^* = \mathbf{x}^* + \omega(\mathbf{b} - \mathbf{Ax}^*)$. Subtracting this from the iteration in (a),

$$\mathbf{e}_{k+1} = (\mathbf{x}_k - \mathbf{x}^*) - \omega(\mathbf{Ax}_k - \mathbf{Ax}^*) = (\mathbf{I} - \omega\mathbf{A}) \mathbf{e}_k, \quad \text{so} \quad \mathbf{e}_k = (\mathbf{I} - \omega\mathbf{A})^k \mathbf{e}_0.$$

The error tends to zero for every $\mathbf{e}_0$ if and only if $\rho(\mathbf{I} - \omega\mathbf{A}) < 1$. Since $\mathbf{A}$ is symmetric, $\mathbf{I} - \omega\mathbf{A}$ has the same eigenvectors as $\mathbf{A}$ and eigenvalues $1 - \omega\lambda_i$, so the requirement is $|1 - \omega\lambda_i| < 1$, i.e., $0 < \omega\lambda_i < 2$, for every i . The binding case is the largest eigenvalue, so gradient descent converges precisely for

$$0 < \omega < \frac{2}{\lambda_{\max}}.$$

(Compare the part of Exercise 4.1 on gradient descent with a fixed step size, where too large a step makes the objective increase.)

(c) The spectral radius is

$$\rho(\omega) = \max_i |1 - \omega\lambda_i| = \max(|1 - \omega\lambda_{\min}|, |1 - \omega\lambda_{\max}|),$$

because $\lambda \mapsto 1 - \omega\lambda$ is decreasing, so the extreme values over the eigenvalues are attained at $\lambda_{\min}$ and $\lambda_{\max}$. Each term is a V-shaped (piecewise linear) function of ω , with vertices at $\omega = 1/\lambda_{\min}$ and $\omega = 1/\lambda_{\max}$ respectively, so their maximum is minimized where the decreasing branch of the first meets the increasing branch of the second:

$$1 - \omega\lambda_{\min} = \omega\lambda_{\max} - 1 \implies \omega_{\text{opt}} = \frac{2}{\lambda_{\min} + \lambda_{\max}}.$$

(d) Substituting ω_{opt} into either branch,

$$\rho(\omega_{\text{opt}}) = 1 - \frac{2\lambda_{\min}}{\lambda_{\min} + \lambda_{\max}} = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa(\mathbf{A}) - 1}{\kappa(\mathbf{A}) + 1},$$

where the last step divides numerator and denominator by $\lambda_{\min}$ and uses $\kappa(\mathbf{A}) = \lambda_{\max}/\lambda_{\min}$ for a symmetric positive definite matrix. As $\kappa(\mathbf{A}) \rightarrow \infty$ we get $\rho \rightarrow 1$, and convergence becomes arbitrarily

slow. Quantitatively, the error contracts by a factor ρ per iteration, so reducing it by a factor ϵ requires about $\log(1/\epsilon)/\log(1/\rho)$ iterations; since $\rho = (\kappa - 1)/(\kappa + 1) \approx 1 - 2/\kappa$ for large κ , this is roughly

$$\frac{\kappa(\mathbf{A})}{2} \log \frac{1}{\epsilon}$$

iterations, growing linearly with the condition number.

Side note. This is the precise version of the statement that gradient descent is slow on ill-conditioned problems (long, narrow valleys), and it is the same condition number that governs the convergence of iterative methods for linear systems.

Solution 4.14

- (a) So that the search direction $\mathbf{d}$ is guaranteed to be a descent direction. Writing $\mathbf{B} = \mathbf{H} + \delta\mathbf{I}$, the line $\mathbf{d} = \mathbf{L}' \setminus (\mathbf{L} \setminus (-\mathbf{g}))$ solves $\mathbf{B}\mathbf{d} = -\mathbf{g}$, so

$$\mathbf{g}^\top \mathbf{d} = -\mathbf{g}^\top \mathbf{B}^{-1} \mathbf{g},$$

which is negative for every $\mathbf{g} \neq \mathbf{0}$ if and only if $\mathbf{B}^{-1} \succ \mathbf{0}$, i.e., if and only if $\mathbf{B} \succ \mathbf{0}$. Without this, the backtracking line search may fail to find any step that decreases f . The Hessian $\nabla^2 f(\mathbf{x})$ itself need not be positive definite away from a minimizer, which is exactly why the modification is needed.

- (b) A fixed shift is both insufficient and harmful.

It does not achieve what the modification is for. The eigenvalues of $\mathbf{H} + \delta\mathbf{I}$ are $\lambda_i(\mathbf{H}) + \delta$, so $\mathbf{B} \succ \mathbf{0}$ requires $\delta > -\lambda_{\min}(\mathbf{H})$. Whenever the Hessian has an eigenvalue less than or equal to -10^{-3} , the shifted matrix is still not positive definite and $\mathbf{d}$ need not be a descent direction, which is precisely the situation the modification exists to handle. No fixed choice of δ works at every iteration, because $\lambda_{\min}(\mathbf{H})$ changes from iteration to iteration and is not bounded below in general.

It also does harm when it is not needed. Near a minimizer the Hessian is positive definite and no modification is required; using $\mathbf{H} + \delta\mathbf{I}$ there means the step is no longer the true Newton step, which degrades the local convergence from quadratic to linear.

Finally, it destroys the certificate of part (d): with $\delta > 0$ at every iteration, the algorithm can never report that it has found a local minimum.

The loop in the code avoids all three problems: the shift is zero whenever $\mathbf{H}$ is already positive definite, and otherwise it is doubled until it is provably large enough.

- (c) The first blank is `p == 0`, which holds exactly when the Cholesky factorization succeeded, i.e., when $\mathbf{H} + \text{delta} \cdot \text{eye}(n)$ is positive definite. Since the eigenvalues of $\mathbf{H} + \delta\mathbf{I}$ are $\lambda_i(\mathbf{H}) + \delta$, the shifted matrix is positive definite as soon as $\delta > -\lambda_{\min}(\mathbf{H})$, so the doubling terminates after finitely many attempts. Note that the factorization is not wasted work: the factor $\mathbf{L}$ of the successful attempt is reused in the line $\mathbf{d} = \mathbf{L}' \setminus (\mathbf{L} \setminus (-\mathbf{g}))$, which costs $O(n^2)$ for two triangular solves instead of a fresh $O(n^3)$ solve.
- (d) The second blank is `delta == 0`.

Reaching that branch at all means `norm(g) <= tol`, i.e., $\nabla f(\mathbf{x}) \approx \mathbf{0}$: the first-order necessary condition holds, so $\mathbf{x}$ is (to within the tolerance) a stationary point. That by itself does not distinguish a minimum from a maximum or a saddle point, which is all the second status claims.

The stronger conclusion needs curvature, and the modification loop has already computed it. If `delta` is still 0, the Cholesky factorization of $\mathbf{H}$ itself succeeded, which happens if and only if $\nabla^2 f(\mathbf{x}) \succ \mathbf{0}$. Together with $\nabla f(\mathbf{x}) = \mathbf{0}$ this is exactly the second-order *sufficient* condition, so $\mathbf{x}$ is a strict local minimum. If instead `delta > 0`, the factorization of $\mathbf{H}$ failed, the Hessian is not positive definite, and $\mathbf{x}$ may well be a saddle point; only the first-order statement can be made.

Two remarks. First, the shift was introduced for a purely algorithmic reason, to guarantee a descent direction, but it doubles as a certificate of second-order optimality at no extra cost: the algorithm never needs to compute an eigenvalue to classify its own answer. Second, this is why the modification loop is

placed *before* the convergence test rather than after it, as would be more natural: the test needs `delta`. (As always with a finite tolerance, the conclusions are approximate: the gradient is only small, not exactly zero.)

The completed code:

```
function [x, status] = newton(f, grad_f, hess_f, x0, tol)
% Modified Newton method for min f(x).
    x      = x0;
    delta0 = 1e-3;
    n      = numel(x);

    for k = 1:100
        g = grad_f(x);
        H = hess_f(x);

        delta = 0;
        while true
            [L, p] = chol(H + delta*eye(n), 'lower');
            if p == 0
                break
            end
            delta = max(2*delta, delta0);
        end

        if norm(g) <= tol
            if delta == 0
                status = 'local minimum';
            else
                status = 'first-order point';
            end
            return
        end

        d      = L' \ (L \ (-g));
        alpha = backtracking(f, x, g, d);
        x      = x + alpha*d;
    end

    status = 'iteration limit reached';
end

function alpha = backtracking(f, x, g, d)
    alpha = 1;
    c      = 1e-4;
    fx     = f(x);
    while f(x + alpha*d) > fx + c*alpha*(g'*d)
        alpha = alpha/2;
        if alpha < 1e-12
            break
        end
    end
end
```


Solution 4.15

The Hessian is evaluated once, at $\mathbf{x}_0$, and then frozen, whereas Newton's method evaluates it at the *current* iterate. Newton's quadratic convergence comes from that re-evaluation: as the iterates approach $\mathbf{x}^*$, the Hessian used in the step approaches $\nabla^2 f(\mathbf{x}^*)$, the first-order term in the error cancels, and only a term of order $\|\mathbf{e}_k\|^2$ is left. With a frozen Hessian there is no such cancellation, so the error is multiplied by a fixed matrix at every step, which is linear convergence.

Side note. Writing $\mathbf{H}_0 = \nabla^2 f(\mathbf{x}_0)$ and expanding the gradient about $\mathbf{x}^*$, where $\nabla f(\mathbf{x}^*) = \mathbf{0}$, gives $\mathbf{g}_k = \nabla^2 f(\mathbf{x}^*) \mathbf{e}_k + O(\|\mathbf{e}_k\|^2)$, so the update $\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{H}_0^{-1} \mathbf{g}_k$ satisfies

$$\mathbf{e}_{k+1} = (\mathbf{I} - \mathbf{H}_0^{-1} \nabla^2 f(\mathbf{x}^*)) \mathbf{e}_k + O(\|\mathbf{e}_k\|^2),$$

with asymptotic rate $\rho(\mathbf{I} - \mathbf{H}_0^{-1} \nabla^2 f(\mathbf{x}^*))$. The linear term vanishes only when $\mathbf{H}_0 = \nabla^2 f(\mathbf{x}^*)$; for a quadratic objective the Hessian is constant, so there the two algorithms coincide and both converge in a single step.

Solution 4.16

(a) $O(MN)$. The gradient is

$$\nabla f(\mathbf{x}) = \sum_{j=1}^M 2(\mathbf{a}_j^\top \mathbf{x} - b_j) \mathbf{a}_j.$$

For each of the M terms, computing the residual $\mathbf{a}_j^\top \mathbf{x} - b_j$ costs $O(N)$ (a dot product) and accumulating the scaled vector $2(\mathbf{a}_j^\top \mathbf{x} - b_j) \mathbf{a}_j$ costs another $O(N)$, for $O(MN)$ in total. The update $\mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)$ adds only $O(N)$, so one gradient descent step costs $O(MN)$.

(b) $O(MN^2 + N^3)$. The Hessian is

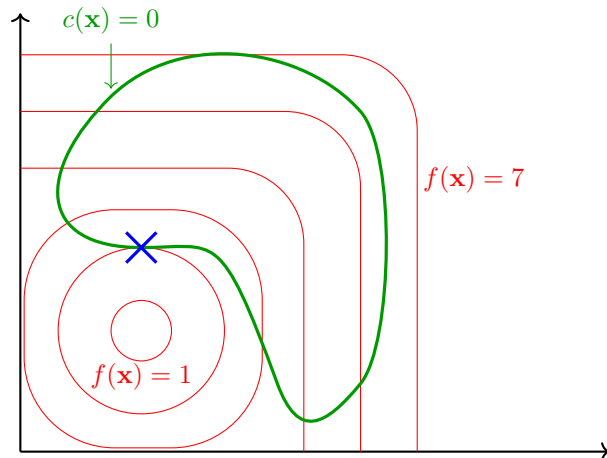
$$\nabla^2 f(\mathbf{x}) = \sum_{j=1}^M 2 \mathbf{a}_j \mathbf{a}_j^\top,$$

and forming it costs $O(N^2)$ per outer product, i.e., $O(MN^2)$ in total. Solving the $N \times N$ linear system $\nabla^2 f(\mathbf{x}_k) \mathbf{d}_k = -\nabla f(\mathbf{x}_k)$ by Gaussian elimination costs $O(N^3)$. One Newton step therefore costs $O(MN^2 + N^3)$, compared to $O(MN)$ for a gradient descent step.

Side note. This is the classic trade-off: a Newton step is far more expensive, but Newton's method typically converges in far fewer iterations.

Solution 4.17

A defining characteristic of constrained optimization is that the solution is at a tangent point between a contour and the constraint. The smallest contour in contact with the constraint is $f(\mathbf{x}) = 2$, and its tangent point to the constraint is shown in blue.



Solution 4.18

The constraint gradients are constant:

$$\nabla c_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \nabla c_2 = \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}.$$

(a) $\mathbf{x} = [1, 1, 1]^\top$. Compute the gradient of f :

$$\nabla f = \begin{bmatrix} 2x_1 \\ 4x_2 \\ 6x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}.$$

We check whether λ_1 and λ_2 exist such that $\nabla f = \lambda_1 \nabla c_1 + \lambda_2 \nabla c_2$:

$$\begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix} = \lambda_1 \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + \lambda_2 \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}, \quad \begin{cases} 2 = \lambda_1 + \lambda_2, \\ 4 = \lambda_1 - \lambda_2, \\ 6 = \lambda_1. \end{cases}$$

The third equation gives $\lambda_1 = 6$; substituting into the first two gives $\lambda_2 = -4$ and $\lambda_2 = 2$, which is inconsistent. No choice of λ_1, λ_2 can satisfy the gradient condition, so $\mathbf{x} = [1, 1, 1]^\top$ is **not** a potential minimum (even though the constraints are satisfied).

(b) $\mathbf{x} = [1, 0, 1/6]^\top$. Compute:

$$\nabla f = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}, \quad \nabla c_1 + \nabla c_2 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}.$$

The gradient condition is satisfied with $\lambda_1 = \lambda_2 = 1$. However, checking the constraints:

$$c_1(1, 0, \tfrac{1}{6}) = 1 + 0 + \tfrac{1}{6} - 3 = -\tfrac{11}{6} \neq 0, \quad c_2(1, 0, \tfrac{1}{6}) = 1 - 0 = 1 \neq 0.$$

The constraints are not satisfied, so $\mathbf{x} = [1, 0, 1/6]^\top$ is **not** a potential minimum.

(c) $\mathbf{x} = [1.2, 1.2, 0.6]^\top$. Compute:

$$\nabla f = \begin{bmatrix} 2.4 \\ 4.8 \\ 3.6 \end{bmatrix}, \quad 3.6 \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + (-1.2) \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2.4 \\ 4.8 \\ 3.6 \end{bmatrix}.$$

The gradient condition is satisfied with $\lambda_1 = 3.6$ and $\lambda_2 = -1.2$. Checking the constraints:

$$c_1(1.2, 1.2, 0.6) = 1.2 + 1.2 + 0.6 - 3 = 0, \quad c_2(1.2, 1.2, 0.6) = 1.2 - 1.2 = 0.$$

Both conditions are satisfied, so $\mathbf{x} = [1.2, 1.2, 0.6]^\top$ satisfies the first-order necessary conditions and is a potential constrained minimum. Note that these are necessary but not sufficient conditions: we cannot determine from this check alone whether the point is a minimum or a maximum, and checking whether the Hessian is positive definite at this point is not sufficient to make that determination for a constrained problem.

Solution 4.19

(a) With $c(\mathbf{x}) = x_1^2 + x_2^2 - 2$, the Lagrangian is

$$L(\mathbf{x}, \lambda) = x_1 + x_2 - \lambda(x_1^2 + x_2^2 - 2),$$

and the first-order necessary conditions are

$$\begin{aligned}\nabla_{\mathbf{x}}L &= \begin{bmatrix} 1 - 2\lambda x_1 \\ 1 - 2\lambda x_2 \end{bmatrix} = \mathbf{0}, \\ \nabla_{\lambda}L &= -(x_1^2 + x_2^2 - 2) = 0.\end{aligned}$$

The first two equations require $\lambda \neq 0$ and give $x_1 = x_2 = \frac{1}{2\lambda}$. Substituting into the constraint gives $2x_1^2 = 2$, so $x_1 = \pm 1$. There are two solutions:

$$\mathbf{x} = (1, 1)^\top, \quad \lambda = \frac{1}{2} \quad \text{and} \quad \mathbf{x} = (-1, -1)^\top, \quad \lambda = -\frac{1}{2}.$$

- (b) Only $\mathbf{x} = (-1, -1)^\top$ is a local solution. The feasible set (the circle of radius $\sqrt{2}$) is closed and bounded and the objective is continuous, so the problem attains its minimum and maximum, and both must occur at points satisfying the first-order necessary conditions. Comparing the objective values, $f(-1, -1) = -2 < 2 = f(1, 1)$: the point $(-1, -1)^\top$ is the (global, hence local) minimum, while $(1, 1)^\top$ is the constrained *maximum*, not a local solution.

Side note. This illustrates that the first-order necessary conditions alone do not certify local optimality: they are satisfied by minima, maxima, and saddle points of the constrained problem alike.

Solution 4.20

The pair does not satisfy them. With $c(\mathbf{x}) = x_1 + x_2 - 2$, the Lagrangian is $L(\mathbf{x}, \lambda) = x_1^2 + x_2^2 - \lambda(x_1 + x_2 - 2)$, and at the given pair

$$\nabla_{\mathbf{x}}L(\bar{\mathbf{x}}, \bar{\lambda}) = \begin{bmatrix} 2\bar{x}_1 - \bar{\lambda} \\ 2\bar{x}_2 - \bar{\lambda} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \neq \mathbf{0}.$$

It does not follow that $\bar{\mathbf{x}}$ is not a local solution, and in fact it is one. The first-order necessary conditions require that there *exists some* multiplier making the Lagrangian stationary, not that a particular given multiplier does. Solving $2\bar{x}_i - \lambda = 0$ gives $\lambda = 2$, and the pair $(\bar{\mathbf{x}}, 2)$ satisfies both stationarity and feasibility ($1 + 1 = 2$). The objective is convex and the constraint is linear, so $\bar{\mathbf{x}}$ is in fact the global, hence local, minimum.

Side note. The supplied multiplier is simply incorrect; local optimality is a property of the point $\bar{\mathbf{x}}$, not of the multiplier handed to you.

Solution 4.21

The constraints in standard form are $\mathbf{c}(\mathbf{x}) = \mathbf{Ax} - \mathbf{b} = \mathbf{0}$, so the Lagrangian is

$$L(\mathbf{x}, \boldsymbol{\lambda}) = \frac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x} + \mathbf{q}^\top \mathbf{x} - \boldsymbol{\lambda}^\top (\mathbf{Ax} - \mathbf{b}).$$

The first-order necessary conditions require the Lagrangian to be stationary with respect to both $\mathbf{x}$ and $\boldsymbol{\lambda}$:

$$\begin{aligned}\nabla_{\mathbf{x}}L &= \mathbf{Q}\mathbf{x} + \mathbf{q} - \mathbf{A}^\top \boldsymbol{\lambda} = \mathbf{0}, \\ \nabla_{\boldsymbol{\lambda}}L &= -(\mathbf{Ax} - \mathbf{b}) = \mathbf{0}.\end{aligned}$$

Both conditions are linear in $(\mathbf{x}, \boldsymbol{\lambda})$, so they can be stacked into the linear system

$$\begin{bmatrix} \mathbf{Q} & -\mathbf{A}^\top \\ \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} -\mathbf{q} \\ \mathbf{b} \end{bmatrix}.$$

Solution 4.22

The Hessian is wrong: the code uses the Hessian of the objective alone and ignores the barrier term. Differentiating ϕ twice,

$$\nabla_{\mathbf{x}}^2 \phi(\mathbf{x}, \mu) = \nabla^2 f(\mathbf{x}) - \frac{\mu}{h(\mathbf{x})} \nabla^2 h(\mathbf{x}) + \frac{\mu}{h(\mathbf{x})^2} \nabla h(\mathbf{x}) \nabla h(\mathbf{x})^\top.$$

Here $\nabla h = (-1, -1)^\top$ and $\nabla^2 h = \mathbf{0}$, so the middle term drops out but the last one does not. The corrected line is

$$\mathbf{H} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} + (\mu/h^2) * \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix};$$

(The gradient in the code is already correct.) With this change the function converges from $\mathbf{x}_0 = [1; 0]$ to $\mathbf{x} = (1.190983, 0.190983)^\top$, which is the exact minimizer of the barrier subproblem for $\mu = 1$.

Side note. Although the constraint is linear, the barrier $-\mu \log h(\mathbf{x})$ is not, and the term it contributes grows like $1/h^2$ as the iterate approaches the boundary. That growth is what makes the Newton step shorten near the boundary; omitting it leaves the steps too long, which is why the buggy version walks straight out of the feasible region.

Solution 4.23

The initial point is infeasible, and the interior-point method requires a *strictly feasible* starting point. At $\mathbf{x}_0 = (2, 0)^\top$ we have $h(\mathbf{x}_0) = 1 - 2 = -1 < 0$, so the barrier term $-\mu \log(h(\mathbf{x}_0)) = -\mu \log(-1)$ is undefined: the barrier objective ϕ and its derivatives cannot even be evaluated at $\mathbf{x}_0$, and Newton's method cannot start. (A starting point exactly on the boundary, with $h(\mathbf{x}_0) = 0$, fails as well, since $\log(0^+) = -\infty$.)

Side note. The barrier only acts as a wall when approached *from the interior*; it says nothing about, and cannot repair, infeasibility. The fix is to start from a strictly feasible point with $h(\mathbf{x}_0) > 0$ (here, any point with $x_1 < 1$, e.g., $\mathbf{x}_0 = (0, 0)^\top$); for problems where a strictly feasible point is not obvious, a preliminary ("phase I") problem is solved to find one, or an infeasible-start variant of the method must be used.

Solution 5.1

Let k denote the number of birds observed in one hour, with

$$k \sim \text{Poisson}(\lambda = 3).$$

The probability of seeing at least one bird is

$$\Pr(k \geq 1) = 1 - \Pr(k = 0).$$

Using the Poisson PMF,

$$\Pr(k = 0) = \frac{3^0 e^{-3}}{0!} = e^{-3}.$$

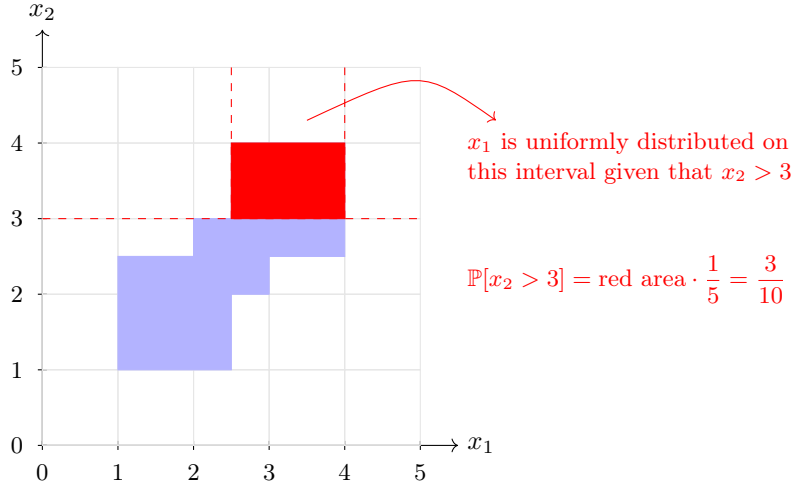
Therefore,

$$\Pr(k \geq 1) = 1 - e^{-3} = 0.95.$$

Solution 5.2

$$p(x_1 | x_2 > 3) = \begin{cases} \frac{2}{3}, & x_1 \in [2.5, 4] \\ 0, & x_1 \notin [2.5, 4] \end{cases}$$

which follows essentially by inspection:



A more formal argument based on the definition of conditional probability:

$$p(x_1|x_2 > 3) = \frac{p(x_1, x_2 > 3)}{p(x_2 > 3)} = \begin{cases} 0, & x_1 \notin [2.5, 4] \\ \frac{1/5}{p(x_2 > 3)} = \frac{1}{5} \cdot \frac{10}{3} = \frac{2}{3}, & x_1 \in [2.5, 4] \end{cases}$$

Solution 5.3

Posterior distribution:

$$P_{\text{post}}[p|\mathcal{D}] = \frac{P[\mathcal{D}|p] \cdot P_{\text{prior}}[p]}{P[\mathcal{D}]}$$

Here: $\mathcal{D}$ is N heads in a row:

$$\begin{aligned} \Rightarrow P[\mathcal{D}|p] &= p^N \\ \Rightarrow P[\mathcal{D}] &= \int_0^1 P[\mathcal{D}|p] P_{\text{prior}}[p] dp \\ &= \int_0^{0.5} 2p^N dp = \frac{2^{-N}}{N+1} \end{aligned}$$

So overall:

$$\begin{aligned} P_{\text{post}}[p|\mathcal{D}] &= \begin{cases} 0, & 0.5 \leq p \leq 1 \\ \frac{N+1}{2^{-N}} \cdot 2 \cdot p^N, & \text{otherwise} \end{cases} \\ &= \begin{cases} 0, & 0.5 \leq p \leq 1 \\ (N+1) \cdot 2^{N+1} \cdot p^N, & \text{otherwise} \end{cases} \\ &\rightarrow \delta(p - \frac{1}{2}) \text{ as } N \rightarrow \infty \end{aligned}$$

Solution 5.4

(a) Requiring the PDF to integrate to 1 over $\mathbb{R}$:

$$\int_{-\infty}^{\infty} p_x(x) dx = 1 = \int_{-\infty}^2 0 dx + \int_2^5 c dx + \int_5^{\infty} 0 dx.$$

$$cx|_2^5 = 5c - 2c = 3c = 1 \quad \Rightarrow \quad c = \frac{1}{3}.$$

Thus

$$p_{\mathbf{x}}(x) = \begin{cases} \frac{1}{3}, & x \in [2, 5], \\ 0, & \text{otherwise.} \end{cases}$$

(b) For the CDF:

$$F_{\mathbf{x}}(x) = \begin{cases} \int_{-\infty}^2 0 \, dx, & x < 2, \\ \int_{-\infty}^2 0 \, dx + \int_2^x \frac{1}{3} \, dx', & x \in [2, 5], \\ \int_{-\infty}^2 0 \, dx + \int_2^5 \frac{1}{3} \, dx' + \int_5^x 0 \, dx, & x > 5, \end{cases}$$

which evaluates to

$$F_{\mathbf{x}}(x) = \begin{cases} 0, & x < 2, \\ \frac{x-2}{3}, & x \in [2, 5], \\ 1, & x > 5. \end{cases}$$

(c)

$$\mu_{\mathbf{x}} = \mathbb{E}[\mathbf{x}] = \int_{-\infty}^{\infty} x p_{\mathbf{x}}(x) \, dx = \int_2^5 x \cdot \frac{1}{3} \, dx = 3.5.$$

(d) Note that smaller values of y are more probable: the events $\mathbf{x} \in [2, 3]$ and $\mathbf{x} \in [4, 5]$ are equally likely, but they map to $y \in [4, 9]$ and $y \in [16, 25]$, which are intervals of different sizes.

Method (i) — using $p_y(y)$:

$$\mathbb{E}[y] = \int_4^{25} y \cdot \frac{1}{6\sqrt{y}} \, dy = \int_4^{25} \frac{\sqrt{y}}{6} \, dy = 13.$$

Method (ii) — using $p_{\mathbf{x}}(x)$:

$$\mathbb{E}[y] = \int_2^5 x^2 \cdot \frac{1}{3} \, dx = 13.$$

Solution 5.5

(a) Using $\text{Cov}(\mathbf{a}, \mathbf{a}) = \text{Var}(\mathbf{a})$, $\text{Cov}(\mathbf{a}, \mathbf{a} + \mathbf{b}) = \text{Cov}(\mathbf{a}, \mathbf{a}) + \text{Cov}(\mathbf{a}, \mathbf{b})$, and $\text{Cov}(\xi_1, \xi_2) = 0$ (independence):

$$\begin{aligned} \mathbf{C} &= \begin{bmatrix} \text{Var}(\xi_1) & \text{Cov}(\xi_1, \xi_1 + \xi_2) \\ \text{Cov}(\xi_1 + \xi_2, \xi_1) & \text{Var}(\xi_1 + \xi_2) \end{bmatrix} \\ &= \begin{bmatrix} 1 & \text{Var}(\xi_1) + \text{Cov}(\xi_1, \xi_2) \\ \text{Var}(\xi_1) + \text{Cov}(\xi_1, \xi_2) & \text{Var}(\xi_1) + \text{Var}(\xi_2) \end{bmatrix} = \begin{bmatrix} 1 & 1+0 \\ 1+0 & 1+1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}. \end{aligned}$$

(b) Therefore $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with $\boldsymbol{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ and $\boldsymbol{\Sigma} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$. Applying the multivariate Gaussian PDF formula:

$$\begin{aligned} p_{\mathbf{x}}(\mathbf{x}) &= (2\pi)^{-k/2} \det(\boldsymbol{\Sigma})^{-1/2} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right) \\ &= \exp\left(-\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \end{bmatrix}\right)^\top \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}^{-1} \left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \end{bmatrix}\right) / 2\right) / \left(2\pi \sqrt{\det \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}}\right) \\ &= \frac{1}{2\pi} \exp\left(x_1 x_2 - x_1^2 - \frac{x_2^2}{2}\right). \end{aligned}$$

(c)

$$\begin{aligned}
 p_{x_2}(x_2) &= \int_{-\infty}^{\infty} \frac{1}{2\pi} \exp\left(x_1 x_2 - x_1^2 - \frac{x_2^2}{2}\right) dx_1 \\
 &= \frac{1}{2\pi} \int_{-\infty}^{\infty} \exp\left(-(x_1 - x_2/2)^2 - x_2^2/4\right) dx_1 \\
 &= \frac{e^{-x_2^2/4}}{2\pi} \cdot \sqrt{\pi} = \frac{1}{\sqrt{4\pi}} \exp\left(-\frac{x_2^2}{4}\right) = \mathcal{N}(0, 2).
 \end{aligned}$$

(d) If x_1 is fixed, then $x_2 = x_1 + \xi_2$ has its mean shifted to x_1 , so $x_2 | x_1 \sim \mathcal{N}(x_1, 1)$; that is,

$$p_{x_2|x_1}(x_2 | x_1) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(x_2 - x_1)^2}{2}\right).$$

(e) Therefore

$$\begin{aligned}
 p_{x_2}(x_2) &= \int_{-\infty}^{\infty} p_{x_2|x_1}(x_2 | x_1) p_{x_1}(x_1) dx_1 \\
 &= \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(x_2 - x_1)^2}{2}\right) \cdot \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x_1^2}{2}\right) dx_1 \\
 &= \frac{1}{2\pi} \int_{-\infty}^{\infty} \exp\left(-(x_1 - x_2/2)^2 - x_2^2/4\right) dx_1 = \mathcal{N}(0, 2),
 \end{aligned}$$

which matches the result of (c).

Solution 5.6The integrand is the Erlang PDF with $k = 5$ and $\lambda = 2$:

$$\frac{2^5 x^{5-1} e^{-2x}}{(5-1)!} = \frac{32 x^4 e^{-2x}}{4!}.$$

Since every PDF integrates to 1 over its support, the value is 1.

Solution 5.7

$$F_x(5; 10, 5) = 1 - e^{-5 \cdot 5} \sum_{n=0}^9 \frac{(5 \cdot 5)^n}{n!}.$$

```

k = 10;
lambda = 5;
x = 5;

sum_terms = 0;
for n = 0:(k-1)
    sum_terms = sum_terms + (lambda*x)^n / factorial(n);
end

F_X = 1 - exp(-lambda*x) * sum_terms

```

Solution 5.8

- Day $t = 0$: $\frac{1}{2}$ cake (from the cake baked that day only).
- Day $t = 1$: $\frac{1}{2}$ (from day 1) + $\frac{1}{3}$ (from day 0) = $\frac{5}{6}$ cake.
- Day $t \geq 2$: $\frac{1}{2} + \frac{1}{3} + \frac{1}{6} = 1$ cake.

The daily baking schedule is a unit step function, and each portion of cake is consumed according to the PMF p_t . The convolution of a unit step with a PMF yields the CMF, so the total amount eaten per day stabilises at 1 once the full support of p_t has been covered.

Solution 5.9

$$(u * p_t)(t) = \int_{-\infty}^{\infty} u(\tau) p_t(t - \tau) d\tau = \int_0^{\infty} p_t(t - \tau) d\tau.$$

Substituting $x = t - \tau$ (so $dx = -d\tau$, and the limits $\tau = 0, \infty$ map to $x = t, -\infty$):

$$(u * p_t)(t) = \int_{x=t}^{x=-\infty} p_t(x) (-dx) = \int_{-\infty}^t p_t(x) dx = F_t(t).$$

Solution 5.10

$$C_{ij} = \langle x_i x_j \rangle - \langle x_i \rangle \langle x_j \rangle$$

First we need $\langle x_1 \rangle = 2$ and $\langle x_2 \rangle = 10$. We also need $\langle x_1 x_2 \rangle = 61/3$, $\langle x_1^2 \rangle = 14/3$, $\langle x_2^2 \rangle = 302/3$.

Then

$$C = \begin{bmatrix} 14/3 - 2 & 61/3 - 20 \\ 61/3 - 20 & 302/3 - 100 \end{bmatrix} = \begin{bmatrix} 2/3 & 1/3 \\ 1/3 & 2/3 \end{bmatrix}.$$

But this formula we used is for population variance. We used the data to estimate the mean, so our variance estimates are biased. So we multiply by $N/(N-1) = 3/2$.

$$C = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$

Solution 6.1

$\nu = \# \text{ data points} - \# \text{ parameters estimated} = 20 - 3 = 17$

$$\chi_{\max}^2 = \text{CDF}_{\chi^2, \nu}^{-1}(1 - p) = 33.4087$$

↑ MATLAB: `chi2inv(0.99, 17)`

Solution 6.2

Given: $\mathbf{X}(\alpha) = \begin{bmatrix} 1 & 0 \\ 0 & 1/2 \\ 0 & \alpha \end{bmatrix}$

Assume i.i.d. measurement errors with variance $\mathbf{V}_\epsilon = \sigma^2 \mathbf{I}$.

$$\Rightarrow \mathbf{V}_\theta = (\mathbf{X}^\top \mathbf{V}_\epsilon^{-1} \mathbf{X})^{-1} = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1}$$

A-optimal design: $\min_{\alpha \in [0,1]} \text{tr}(\mathbf{V}_\theta(\alpha)) = \min_{\alpha \in [0,1]} \sigma^2 \text{tr}((\mathbf{X}(\alpha)^\top \mathbf{X}(\alpha))^{-1})$

$$\Rightarrow \mathbf{X}(\alpha)^\top \mathbf{X}(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1/2 & \alpha \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1/2 \\ 0 & \alpha \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1/4 + \alpha^2 \end{bmatrix}$$

$$\Rightarrow \text{tr}((\mathbf{X}(\alpha)^\top \mathbf{X}(\alpha))^{-1}) = \frac{1}{1/4 + \alpha^2} + 1$$

Thus, A-optimal choice of α is given by $\alpha^* = 1$ as that clearly minimizes $\frac{\sigma^2}{1/4 + \alpha^2}$ on $[0, 1]$.

Solution 7.1

```

N = 1000;
X = zeros(N,2);
n = 0;

p = @(x1,x2) (1/3)*((x1<=0.5 & x2<=0.5) | (x1>0.5 & x2>0.5)) ...
    + (1/6)*((x1<=0.5 & x2> 0.5) | (x1>0.5 & x2<=0.5));

M = 1/3;

while n < N
    xp = rand(1,2);
    ntry = ntry + 1;
    if rand < p(xp(1),xp(2))/M
        n = n + 1;
        X(n,:) = xp;
    end
end

figure
h = histogram2(X(:,1),X(:,2),[2 2], 'Normalization','probability', '
    FaceColor','flat');
xlabel('x_1'); ylabel('x_2'); zlabel('p(x)'); colorbar
title('1000 rejection samples')

disp('BinCounts matrix (rows = x1 bins, cols = x2 bins):'); disp(h.
    BinCounts)

```

Solution 7.2

```

N = 1000;

p = @(x) 1 ./ (1 + x*x');

x = [0 0];
X = zeros(N,2);

for k = 1:N
    xp = x + randn(1,2);
    if rand < p(xp)/p(x)
        x = xp;
    end
    X(k,:) = x;
end

figure

```

```

histogram2(X(:,1),X(:,2),[20 20], 'Normalization','pdf','DisplayStyle','
    title')
xlabel('x_1'); ylabel('x_2'); colorbar; axis equal
title('1000 samples: 2-D histogram')

```

Note: In two dimensions

$$\int_{\mathbb{R}^2} \frac{d\mathbf{x}}{1+|\mathbf{x}|^2} = 2\pi \int_0^\infty \frac{r dr}{1+r^2} = \infty,$$

so π as written is not normalizable. The chain drifts outward forever.

Solution 7.3

- (a) This code attempts to estimate $1/N$ using the following scheme:

$$\int_{\mathbb{R}} R(x) dx = \int_{\mathbb{R}} N \exp(-|x|) dx = 1$$

$$1/N = \int_{\mathbb{R}} \exp(-|x|) dx = \int_{\mathbb{R}} \frac{\exp(-|x|)}{p(x)} \cdot p(x) dx$$

We will draw samples from $p(x)$ and evaluate $\frac{\exp(-|x|)}{p(x)}$ at those points. Here samples are taken from $p(x) = \mathcal{U}(-2, 2)$, but this doesn't cover all density of $R(x)$. So the contributions of the tails to the integral are lost.

- (b) The estimate of N is just the average value of $\exp(-|x|)$ over the samples, but we need to account for the probability of our samples. We do this by taking the mean of $\frac{\exp(-|x|)}{p(x)}$ instead. Corrected code:

```

N_samples = 1e6;
f = @(x) exp(-abs(x));
P = @(x) normpdf(x);
samples = randn(N_samples,1);
fP_vals = f(samples) ./ P(samples);
N_estimate = mean(fP_vals);
fprintf('%0.6f\n', N_estimate);

```

- (c) Yes.

- (d) No. The integral we are trying to estimate is $\int_{\mathbb{R}} \exp(-|x|) dx$. This is not in the form $\int f(x) P(x) dx$ where $P(x)$ is a properly normalised PDF. If we try to recast our integral in this form,

$$\int_{\mathbb{R}} \exp(-|x|) dx = \int_{\mathbb{R}} \frac{1}{N} N \exp(-|x|) dx = \int_{\mathbb{R}} \frac{1}{N} R(x) dx,$$

then $f(x) = 1/N$. We cannot evaluate this because we do not know N .

Solution 7.4

- (a) Yes: $g(x) = e^{-x^2}$, $f(x) = x$
 $g(x)$ doesn't need to be normalized for MCMC. The integral value is 0, due to symmetry of the integrand with respect to the bounds of integration.
- (b) Yes: $g(x) = e^{-x^2}$, $f(x) = 1$
The integral value is 1, as the integrand is a PDF.
- (c) No—not as is. This integral is not in the form of an expectation value. But if we multiply and divide by a PDF, we can cast this integral as an expectation value.

$$I = \int_{\mathbb{R}} \frac{e^{-x^2}}{p(x)} p(x) dx$$

The value of the integral is $\sqrt{\pi}$, as that is the inverse normalization of the standard gaussian distribution.

As a side note, when choosing $p(x)$, the closer the shape of $p(x)$ matches $|e^{-x^2}| / \int |e^{-x^2}| dx$, the faster the MCMC approximation converges to the integral (i.e., it is better to sample points where our function value is high because those points contribute the most to the integral).

Solution 7.5

- (a) This isn't a problem, as by construction our A will account for this asymmetry. If we use a symmetric M , then the M-H A simplifies to

$$A(\mathbf{x}^*, \mathbf{x}_i) = \min \left\{ 1, \frac{\pi(\mathbf{x}^*)}{\pi(\mathbf{x}_i)} \right\}.$$

- (b) We usually use an M that only makes transitions in a local region (is not irreducible) like $\mathcal{U}(\mathbf{x}_i - c, \mathbf{x}_i + c)$ or $\mathcal{N}(\mathbf{x}_i, c)$. This leads to more accepted transitions and a smoother exploration of the surface.

In this case however, we are not guaranteed to converge to our target distribution because we only take local steps and there could be two regions with nonzero density separated by a region of zero density. One solution is to create multiple chains with different initializations.

Solution 8.1

The matrix equation is given as follows:

$$\frac{d}{dt} \begin{pmatrix} C_A \\ C_B \end{pmatrix} = \begin{pmatrix} -1 & 0 \\ +1 & -10 \end{pmatrix} \begin{pmatrix} C_A \\ C_B \end{pmatrix}.$$

The eigenvalues of a triangular matrix (upper- or lower-triangle) are just the diagonal elements; in this case, -1 sec^{-1} and -10 sec^{-1} . The largest (magnitude) eigenvalue, corresponding to the shortest timescale, is -10 sec^{-1} .

For forward Euler, the region of stability is known to be $|1 + \lambda \Delta t| \leq 1$. Solve this inequality for Δt :

$$\begin{aligned} |1 - 10\Delta t| &\leq 1 \\ -1 &\leq 1 - 10\Delta t \leq 1 \\ -2 &\leq -10\Delta t \leq 0 \\ \frac{1}{5} &\geq \Delta t \geq 0. \end{aligned}$$

$\Delta t \geq 0$ is trivially satisfied, so the only limit is the upper limit of 0.2 sec.

Solution 8.2

The forward Euler scheme has second-order accurate local truncation error, so we approximate its global truncation error as first-order accurate (one order lower). Likewise, the Crank-Nicolson scheme has third-order accurate local truncation error, so we approximate its global truncation error as second-order accurate.

On a log-log plot, these power laws appear as lines, whose slopes are the exponents of the power laws. Therefore, the slope of the Crank-Nicolson line should be twice that of the forward Euler line, indicating that the blue dots represent Crank-Nicolson global truncation error.

Solution 8.3

There are 8 cells in the grid and we want to find the average concentration for all. $\Rightarrow$ 8 unknowns

Solution 8.4

Let

$$h_+ := \Delta x_i = x_{i+1} - x_i, \quad h_- := \Delta x_{i-1} = x_i - x_{i-1},$$

and assume f is sufficiently smooth. Expanding f about x_i :

$$\begin{aligned} f(x_{i+1}) &= f(x_i) + h_+ f'(x_i) + \frac{h_+^2}{2} f''(x_i) + \frac{h_+^3}{6} f^{(3)}(x_i) + \mathcal{O}(h_+^4), \\ f(x_{i-1}) &= f(x_i) - h_- f'(x_i) + \frac{h_-^2}{2} f''(x_i) - \frac{h_-^3}{6} f^{(3)}(x_i) + \mathcal{O}(h_-^4). \end{aligned}$$

Forming the numerator:

$$f(x_{i+1}) - f(x_{i-1}) = (h_+ + h_-) f'(x_i) + \frac{h_+^2 - h_-^2}{2} f''(x_i) + \frac{h_+^3 + h_-^3}{6} f^{(3)}(x_i) + \mathcal{O}(\max\{h_+^4, h_-^4\}).$$

Dividing by $h_+ + h_-$:

$$\frac{f(x_{i+1}) - f(x_{i-1})}{h_+ + h_-} = f'(x_i) + \underbrace{\frac{h_+ - h_-}{2} f''(x_i)}_{=: E_1} + \underbrace{\frac{h_+^3 + h_-^3}{6(h_+ + h_-)} f^{(3)}(x_i)}_{=: E_2} + \mathcal{O}(\max\{h_+^3, h_-^3\}).$$

Thus the truncation error is

$$\text{TE} = \frac{h_+ - h_-}{2} f''(x_i) + \frac{h_+^3 + h_-^3}{6(h_+ + h_-)} f^{(3)}(x_i) + \mathcal{O}(\max\{h_+^3, h_-^3\}),$$

and from the leading term:

$$\boxed{\frac{f(x_{i+1}) - f(x_{i-1})}{\Delta x_i + \Delta x_{i-1}} = f'(x_i) + \frac{\Delta x_i - \Delta x_{i-1}}{2} f''(x_i) + \mathcal{O}(\max\{\Delta x_i^2, \Delta x_{i-1}^2\}) .}$$

Therefore:

- *General (unequal spacing)*: the scheme is **first-order accurate** (error $\sim \mathcal{O}(\max\{h_+, h_-\})$) because the leading error contains the factor $(h_+ - h_-)/2$.
- *Special case* ($h_+ = h_- = h$): the f'' term vanishes and

$$\frac{f(x_{i+1}) - f(x_{i-1}))}{2h} = f'(x_i) + \frac{h^2}{6} f^{(3)}(x_i) + \mathcal{O}(h^4),$$

so the scheme is **second-order accurate**.

Solution 9.1

In first order approximation (Newton's method):

$$\tilde{x}(\tau; c + \Delta c) = x(\tau; c) + \delta(c; \tau) \Delta c$$

$$\Rightarrow \text{Choose } \Delta c \text{ st. } \tilde{x}(\tau; c + \Delta c) = x_f \Rightarrow \Delta c = \frac{x_f - x(\tau; c)}{\delta(c; \tau)}$$

Thus, the shooting method suggests increasing c !

Solution 9.2

Let the state x be M -dimensional. (M could be 1, in the case of a scalar state.) Then, there are $M \cdot N$ unknowns $c_i \in \mathbb{R}^M$, $i = 1, \dots, N$ in the truncated series expansion. Without any further information, we shouldn't assert that any of the BCs is implied by the choice of basis functions.

Thus, we need to pick $N - 1$ collocation points to obtain a square equation system:

$$\begin{array}{ll} \frac{dx}{dt}(t_i) = f(x(t_i), t_i), & i = 1, \dots, N - 1 \quad \longrightarrow \quad M(N - 1) \text{ equations} \\ g(x(t_0), x(t_f)) = 0 & \longrightarrow \quad M \text{ equations} \\ & \hline & M \cdot N \text{ equations} \end{array}$$

Solution 9.3

(a) Using the centered second-order finite difference for the time derivative at interior nodes:

$$\frac{x_{i+1} - x_{i-1}}{2\Delta t} \approx \left. \frac{dx}{dt} \right|_{x_i} = -x_i.$$

Rearranging gives the discrete algebraic equation

$$(-1)x_{i-1} + (2\Delta t)x_i + (1)x_{i+1} = 0.$$

Our vector of unknowns is

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix}.$$

From this we get, in the notation of Exercise 1.7, $a_i = -1$, $b_i = 2\Delta t$, and $c_i = 1$, with $a_0 = 0$ because of the boundary conditions.

For the case $i = 1$ our equation is

$$\frac{x_{i+1} - x_0}{2\Delta t} \approx -x_i,$$

which rearranges to

$$(2\Delta t)x_i + (1)x_{i+1} = x_0.$$

This gives $d_1 = x_0$.

For the case $i = N$, we use

$$\frac{x_N - x_{N-1}}{\Delta t} \approx -x_N,$$

which rearranges to

$$(-1)x_{N-1} + (1 + \Delta t)x_N = 0.$$

Then the linear system $A\mathbf{x} = \mathbf{b}$ has

$$A = \begin{pmatrix} 2\Delta t & 1 & & & \\ -1 & 2\Delta t & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2\Delta t & 1 \\ & & & -1 & (1 + \Delta t) \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} x_0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{pmatrix}.$$

- (b) The system is tridiagonal: each equation couples x_i only to its neighbors x_{i-1} and x_{i+1} . The relevant algorithm is the Thomas algorithm from Exercise 1.7, which solves a tridiagonal system in $\mathcal{O}(N)$ operations. With the horizon $[0, T]$ fixed, the number of grid points is $N = T/\Delta t$, so the cost scales as $\mathcal{O}(1/\Delta t)$: halving the step size doubles the work.

Solution 9.4

- (a) None of the three types. The condition is nonlinear since it involves the product of the solution and its derivative.
- (b) Neumann: the value of the first derivative C' is specified at the boundary $x = 0$.

Solution 9.5

- (a) $\mathbf{S}$ is useful for two reasons. First, it enables computation of the Jacobian of the shooting residual $\mathbf{g}$ with respect to $\mathbf{c}$:

$$\mathbf{J} = \nabla_{\mathbf{c}}(\mathbf{g}(\mathbf{x}(t_0) = \mathbf{c}, \mathbf{x}(T)(\mathbf{c}))) = \frac{\partial \mathbf{g}}{\partial \mathbf{c}} + \frac{\partial \mathbf{g}}{\partial \mathbf{x}} \mathbf{S}.$$

This tells us (to first order) how to update $\mathbf{c}$ to reduce the residual and whether the solution $\mathbf{c}^*$ is locally unique. Second, examining $\mathbf{J}$ at a candidate $\mathbf{c}^*$ can explain why `fsolve` failed to converge.

- (b) Two methods for computing $\mathbf{S}$:
- Numerical differentiation*: perturb each component of $\mathbf{c}$, solve the ODE for each perturbation, and estimate each column of $\mathbf{S}$ by finite differences.
 - Sensitivity ODE*: differentiate the ODE with respect to $\mathbf{c}$ to obtain a matrix-valued ODE-IVP for $\mathbf{S}$, solved alongside the original ODE.

Solution 10.1

- (a) Dirichlet
- (b) Neumann
- (c) Dirichlet

Solution 10.2

$$\text{B.C.: } c(x=0) - \frac{dc}{dx} \Big|_{x=0} = \alpha$$

$$\frac{dc}{dx} \Big|_{x=0} \approx \frac{-\frac{1}{2}c_0 + \frac{1}{2}c_2}{\Delta x}$$

$$c_1 - \frac{1}{2\Delta x}(c_2 - c_0) = \alpha$$

$$\boxed{c_0 = 2\Delta x\alpha - 2\Delta xc_1 + c_2}$$

You would then use the interior point equation (derived for a specific governing equation) for c at $i = 1$ and plug in this equation for c_0 to enforce the boundary condition.

Solution 10.3

Forward time, central difference discretization

$$\frac{c_{i+1}^j - c_i^j}{\Delta t} + U \frac{c_i^{j+1} - c_i^{j-1}}{2\Delta x} - \mathcal{D} \frac{c_i^{j+1} - 2c_i^j + c_i^{j-1}}{(\Delta x)^2} = 0.$$

For von Neumann stability analysis we use

$$c_i^j = \epsilon_i e^{\hat{i}kj\Delta x}$$

and substitute this into our discretized governing equation. (The $\hat{i}$ in the exponential is $\hat{i} = \sqrt{-1}$, not an index!)

$$\frac{\epsilon_{i+1}e^{\hat{i}kj\Delta x} - \epsilon_i e^{\hat{i}kj\Delta x}}{\Delta t} + U \frac{\epsilon_i (e^{\hat{i}k(j+1)\Delta x} - e^{\hat{i}k(j-1)\Delta x})}{2\Delta x} - \mathcal{D} \frac{\epsilon_i (e^{\hat{i}k(j+1)\Delta x} - 2e^{\hat{i}kj\Delta x} + e^{\hat{i}k(j-1)\Delta x})}{(\Delta x)^2} = 0.$$

$\downarrow$ divide by $e^{\hat{i}kj\Delta x}$ and simplify

$$\frac{\epsilon_{i+1} - \epsilon_i}{\Delta t} + U \frac{\epsilon_i (e^{\hat{i}k\Delta x} - e^{-\hat{i}k\Delta x})}{2\Delta x} - \mathcal{D} \frac{\epsilon_i (e^{\hat{i}k\Delta x} - 2 + e^{-\hat{i}k\Delta x})}{(\Delta x)^2} = 0.$$

$\downarrow$ rearrange for ϵ_{i+1}

$$\epsilon_{i+1} = \epsilon_i \left[1 - \frac{U\Delta t}{2\Delta x} (e^{\hat{i}k\Delta x} - e^{-\hat{i}k\Delta x}) + \frac{\mathcal{D}\Delta t}{(\Delta x)^2} (e^{\hat{i}k\Delta x} - 2 + e^{-\hat{i}k\Delta x}) \right].$$

$\downarrow$ use the identities $\begin{aligned} e^{\hat{i}k\Delta x} + e^{-\hat{i}k\Delta x} &= 2\cos(k\Delta x) \\ e^{\hat{i}k\Delta x} - e^{-\hat{i}k\Delta x} &= 2\hat{i}\sin(k\Delta x) \end{aligned}$

$$\epsilon_{i+1} = \epsilon_i \left[1 - \frac{U\Delta t \hat{i}\sin(k\Delta x)}{\Delta x} + \frac{2\mathcal{D}\Delta t}{(\Delta x)^2} (\cos(k\Delta x) - 1) \right].$$

We want $\left| \frac{\epsilon_{i+1}}{\epsilon_i} \right| \leq 1$ for stability.

stability criterion: $\left| 1 - \frac{U\Delta t \hat{i}\sin(k\Delta x)}{\Delta x} + \frac{2\mathcal{D}\Delta t}{(\Delta x)^2} (\cos(k\Delta x) - 1) \right| \leq 1$

Let $c = \frac{U\Delta t}{\Delta x}$ and $\alpha = \frac{\mathcal{D}\Delta t}{(\Delta x)^2}$.

$$\left| 1 - \hat{i}c\sin(k\Delta x) + 2\alpha[\cos(k\Delta x) - 1] \right| \leq 1$$

$\downarrow$ take magnitude and square

$$\left(1 + 2\alpha[\cos(k\Delta x) - 1] \right)^2 + c^2 \sin^2(k\Delta x) \leq 1$$

$\downarrow$ use $\sin^2 x = 1 - \cos^2 x = (1 - \cos x)(1 + \cos x)$

$$\left(1 - 2\alpha[1 - \cos(k\Delta x)] \right)^2 + c^2(1 - \cos(k\Delta x))(1 + \cos(k\Delta x)) \leq 1$$

$\downarrow$ expand

$$1 - 4\alpha[1 - \cos(k\Delta x)] + 4\alpha^2[1 - \cos(k\Delta x)]^2 + c^2[(1 - \cos(k\Delta x))(1 + \cos(k\Delta x))] \leq 1$$

$$-4\alpha[1 - \cos(k\Delta x)] + 4\alpha^2[1 - \cos(k\Delta x)]^2 + c^2[(1 - \cos(k\Delta x))(1 + \cos(k\Delta x))] \leq 0$$

If $k\Delta x = 0$, this is satisfied. If $k\Delta x \neq 0$, we can divide through by $4\alpha(1 - \cos(k\Delta x))$,

$$-1 + \alpha(1 - \cos(k\Delta x)) + \frac{c^2}{4\alpha}(1 + \cos(k\Delta x)) \leq 0$$

$$\boxed{\left(\alpha + \frac{c^2}{4\alpha}\right) + \left(\frac{c^2}{4\alpha} - \alpha\right)\cos(k\Delta x) \leq 1}$$

Case 1:

$$\frac{c^2}{4\alpha} - \alpha \leq 0 \implies c^2 \leq 4\alpha^2$$

The max in this case is when $\cos(k\Delta x) = -1$,

$$\alpha + \frac{c^2}{4\alpha} + \alpha - \frac{c^2}{4\alpha} \leq 1$$

$$2\alpha \leq 1$$

$$\boxed{\text{Case 1: } \frac{c}{\alpha} = \frac{U\Delta x}{\mathcal{D}} \leq 2, \quad \alpha = \frac{\mathcal{D}\Delta t}{(\Delta x)^2} \leq \frac{1}{2}}$$

Case 2:

$$\frac{c^2}{4\alpha} - \alpha > 0 \implies c^2 > 4\alpha^2$$

The max in this case is when $\cos(k\Delta x) = 1$,

$$\alpha + \frac{c^2}{4\alpha} + \frac{c^2}{4\alpha} - \alpha \leq 1$$

$$\frac{c^2}{2\alpha} \leq 1$$

$$\boxed{\text{Case 2: } c^2 > 4\alpha^2 \quad \text{and} \quad c^2 \leq 2\alpha}$$

In both cases (if you rearrange Case 2) you will find

$$\alpha = \frac{\mathcal{D}\Delta t}{(\Delta x)^2} \leq \frac{1}{2} \implies \Delta t \leq \frac{(\Delta x)^2}{2\mathcal{D}}.$$

Solution 11.1

DAE:

$$(1) \dot{x}_1 + \dot{x}_2 = g_1(t)$$

$$(2) x_2 + x_3 = g_2(t)$$

$$(3) x_3 = g_3(t)$$

Differential States: x_1, x_2

Algebraic States: x_3

Solution 11.2

We can answer this question immediately upon being told that the index is 2, as index-2 systems necessarily have at least one implicit constraint. We can actually calculate the implicit constraint by converting the DAE to an ODE and looking for intermediate equations that are used. Take two derivatives of the system:

$$\begin{array}{lll} (1) \dot{x}_1 - x_2 = 0 & (1') \ddot{x}_1 - \dot{x}_2 = 0 & (1'') \dddot{x}_1 - \ddot{x}_2 = 0 \\ (2) x_1 = g_1(t) & (2') \dot{x}_1 = g_1'(t) & (2'') \ddot{x}_1 = g_1''(t) \end{array}$$

Equation (1) gives an equation for $\dot{x}_1$. Equation (1') gives an equation for $\dot{x}_2$ in terms of $\ddot{x}_1$, which can be eliminated using equation (2''). The intermediate equation, (2'), is the implicit constraint.

Solution 11.3

- Differential states: x_1
- Algebraic states: y_1, y_2
- Index: 1
- Implicit constraints: none
- Initialization DoF: $(2)(1) + 2 - 3 - 0 = 1$
- Consistent initialization example: $\{x_1(0)\}$

Solution 11.4

- Differential states: x_1
- Algebraic states: y_1, y_2
- Index: 2
- Implicit constraints: $\dot{x}_1 = \cos(t)$
- Initialization DoF: $(2)(1) + 2 - 3 - 1 = 0$
- Consistent initialization example: all predetermined

Solution 11.5

- Differential states: x_1, x_2
- Algebraic states: y_1, y_2
- Index: 2
- Implicit constraints: $\dot{x}_1 = \cos(t)$
- Initialization DoF: $(2)(2) + 2 - 4 - 1 = 1$
- Consistent initialization example: $\{x_2(0)\}$

Solution 11.6

- Differential states: x, v, a
- Algebraic states: none
- Index: 0
- Implicit constraints: none
- Initialization DoF: $(2)(3) + 0 - 3 - 0 = 3$

- Consistent initialization example: $\{x(0), v(0), a(0)\}$

Solution 11.7

- Differential states: x, v
- Algebraic states: a
- Index: 1
- Implicit constraints: none
- Initialization DoF: $(2)(2) + 1 - 3 - 0 = 2$
- Consistent initialization example: $\{x(0), v(0)\}$

Solution 11.8

- Differential states: x, v
- Algebraic states: a, E
- Index: 2
- Implicit constraints: $2v\dot{v} + 2\dot{x} = -e^{-t}$
- Initialization DoF: $(2)(2) + 2 - 4 - 1 = 1$
- Consistent initialization example: $\{v(0)\}$

Solution 11.9

- Differential states: x, v
- Algebraic states: a
- Index: 3
- Implicit constraints: $\begin{cases} \dot{x} = 10t \\ \dot{v} = 10 \end{cases}$
- Initialization DoF: $(2)(2) + 1 - 3 - 2 = 0$
- Consistent initialization example: all predetermined

Solution 11.10

- Differential states: u_1, u_2
- Algebraic states: u_3
- Index: 1
- Implicit constraints: $2u_1\dot{u}_1 + 2u_2\dot{u}_2 = 0$
- Initialization DoF: $2(2) + 1 - 3 - 1 = 1$
- Consistent initialization example: $\{u_2(0)\}$ (note: will need to specify the signs of $u_1(0)$ and $\dot{u}_1(0)$)

Bibliography

- (1) Dupont, E.; Eisenberger, M.; Kozlovskii, B.; Mehrabian, A.; Ruiz, F. J. R.; See, A.; Zhou, R.; Alman, J.; Vassilevska Williams, V.; Balog, M. Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve, 2026.
- (2) Alman, J.; Duan, R.; Vassilevska Williams, V.; Xu, Y.; Xu, Z.; Zhou, R. More Asymmetry Yields Faster Matrix Multiplication, 2024.
- (3) Shewchuk, J. R. **1994**.